dego_vila@yahoo.com 30 Oct 2026

Let's Go!

LET'S GO *FURTHER*

ADVANCED PATTERNS FOR BUILDING, MANAGING AND DEPLOYING RESTFUL **JSON APIS AND WEB APPLICATIONS** IN GO



ALEX EDWARDS

1ST EDITION

dego_vila@yahoo.com 30 Oct 2026

Contents

- **1. Introduction**
  - 1.1. Prerequisites
- **2. Getting started**
  - 2.1. Project setup and skeleton structure
  - 2.2. A basic HTTP server
  - 2.3. API endpoints and RESTful routing
- **3. Sending JSON responses**
  - 3.1. Fixed-format JSON
  - 3.2. JSON encoding
  - 3.3. Encoding structs
  - 3.4. Formatting and enveloping responses
  - 3.5. Custom JSON encoding
  - 3.6. Sending error messages
- **4. Parsing JSON requests**
  - 4.1. JSON decoding
  - 4.2. Managing bad requests
  - 4.3. Restricting Inputs
  - 4.4. Custom JSON decoding
  - 4.5. Validating JSON input
- **5. Database setup and configuration**
  - 5.1. Setting up PostgreSQL
  - 5.2. Connecting to PostgreSQL
  - 5.3. Configuring the database connection pool
- **6. SQL migrations**
  - 6.1. An overview of SQL migrations
  - 6.2. Working with SQL migrations
- **7. CRUD operations**
  - 7.1. Setting up the movie model
  - 7.2. Creating a new movie
  - 7.3. Fetching a movie
  - 7.4. Updating a movie

dego_vila@yahoo.com 30 Oct 2026

- 7.5. Deleting a movie
- **8. Advanced CRUD operations**
  - 8.1. Handling partial updates
  - 8.2. Optimistic concurrency control
  - 8.3. Managing SQL query timeouts
- **9. Filtering, sorting, and pagination**
  - 9.1. Parsing query string parameters
  - 9.2. Validating query string parameters
  - 9.3. Listing data
  - 9.4. Filtering lists
  - 9.5. Full-text search
  - 9.6. Sorting Lists
  - 9.7. Paginating lists
  - 9.8. Returning pagination metadata
- **10. Rate limiting**
  - 10.1. Global rate limiting
  - 10.2. IP-based rate limiting
  - 10.3. Configuring the rate limiters
- **11. Graceful shutdown**
  - 11.1. Sending shutdown signals
  - 11.2. Intercepting shutdown signals
  - 11.3. Executing the shutdown
- **12. User model setup and registration**
  - 12.1. Setting up the users database table
  - 12.2. Setting up the users model
  - 12.3. Registering a user
- **13. Sending emails**
  - 13.1. SMTP server setup
  - 13.2. Creating email templates
  - 13.3. Sending a welcome email
  - 13.4. Sending background emails
  - 13.5. Graceful shutdown of background tasks
- **14. User activation**
  - 14.1. Setting up the tokens database table

dego_vila@yahoo.com 30 Oct 2026

- 14.2. Creating secure activation tokens
- 14.3. Sending activation tokens
- 14.4. Activating a user

- **15. Authentication**
  - 15.1. Authentication options
  - 15.2. Generating authentication tokens
  - 15.3. Authenticating requests
- **16. Permission-based authorization**
  - 16.1. Requiring user activation
  - 16.2. Setting up the permissions database table
  - 16.3. Setting up the permissions model
  - 16.4. Checking permissions
  - 16.5. Granting permissions
- **17. Cross-origin requests**
  - 17.1. An overview of CORS
  - 17.2. Demonstrating the same-origin policy
  - 17.3. Simple CORS requests
  - 17.4. Preflight CORS requests
- **18. Metrics**
  - 18.1. Exposing metrics with expvar
  - 18.2. Creating custom metrics
  - 18.3. Request-level metrics
  - 18.4. Recording HTTP status codes
- **19. Building, versioning and quality control**
  - 19.1. Creating and using makefiles
  - 19.2. Managing environment variables
  - 19.3. Quality-controlling code
  - 19.4. Module mirrors and vendoring
  - 19.5. Building binaries
  - 19.6. Managing and automating version numbers
- **20. Deployment and hosting**
  - 20.1. Creating a DigitalOcean droplet
  - 20.2. Server configuration and installing software
  - 20.3. Deployment and executing migrations

dego_vila@yahoo.com 30 Oct 2026

- 20.4. Running the API as a background service
- 20.5. Using Caddy as a reverse proxy

- **21. Appendices**
  - 21.1. Managing password resets
  - 21.2. Creating additional activation tokens
  - 21.3. Authentication with JSON Web Tokens
  - 21.4. JSON encoding nuances
  - 21.5. JSON decoding nuances
  - 21.6. Request context timeouts
  - 21.7. Aligning integer types
- **22. Feedback**

dego_vila@yahoo.com 30 Oct 2026

Chapter 1.

Introduction

In this book we're going to work through the start-to-finish build of an application called *Greenlight* — a JSON API for retrieving and managing information about movies. You can think of the core functionality as being a bit like the Open Movie Database API.

Ultimately, our *Greenlight* API will support the following endpoints and actions:

Method	URL Pattern	Action
GET	/v1/healthcheck	Show application health and version information
GET	/v1/movies	Show the details of all movies
POST	/v1/movies	Create a new movie
GET	/v1/movies/:id	Show the details of a specific movie
PATCH	/v1/movies/:id	Update the details of a specific movie
DELETE	/v1/movies/:id	Delete a specific movie
POST	/v1/users	Register a new user
PUT	/v1/users/activated	Activate a specific user
PUT	/v1/users/password	Update the password for a specific user
POST	/v1/tokens/authentication	Generate a new authentication token
POST	/v1/tokens/password-reset	Generate a new password-reset token
GET	/debug/vars	Display application metrics

To give you an idea of what the API will look like from a client's point of view, by the end of this book the `GET /v1/movies/:id` endpoint will return a HTTP response with a JSON body similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -H "Authorization: Bearer RIDBIAE3AMMK57T6IAEBUGA7ZQ" localhost:4000/v1/movies/1
{
    "movie": {
        "id": 1,
        "title": "Moana",
        "year": 2016,
        "runtime": "107m",
        "genres": [
            "animation",
            "adventure"
        ],
        "version": 1
    }
}
```

Behind the scenes, we'll use PostgreSQL as the database to persistently store all the data. And at the end of the book, we'll deploy the finished API to a Linux server running on DigitalOcean.

Conventions

Code blocks in this book are shown with a silver background, like the example below. If a code block is particularly long, any parts that aren't relevant may be replaced with an ellipsis. To make it easy to follow along, most code blocks also have a title bar at the top indicating the name of the file containing the code. Like this:

File: hello.go

```
package main

... // Indicates that some existing code has been omitted.

func sayHello() {
    fmt.Println("Hello world!")
}
```

Terminal (command line) instructions are shown with a darker background and generally start with a dollar symbol. These commands should work on any Unix-based operating system, including macOS and Linux. Sample output is shown in silver beneath the command, like so:

```
$ echo "Hello world!"
Hello world!
```

If you're using Windows, you should replace these with the command prompt equivalent or carry out the action via the normal Windows GUI.

dego_vila@yahoo.com 30 Oct 2026

Please note that the dates and timestamps shown in screenshots and the example output from commands are illustrative only. They do not necessarily align with each other, or progress chronologically throughout the book.

Some chapters in this book end with an *additional information* section. These sections contain information that isn't relevant to our application build, but is still important (or sometimes, just interesting) to know about.

> **Hint:** If you're following along with the application build I recommend using the HTML version of this book, instead of the PDF or EPUB. The HTML version works in all browsers, and the proper formatting of code blocks is retained if you want to copy-and-paste code directly from the book.
>
> When using the HTML version, you can also use the left and right arrow keys to navigate between chapters. And you can also click the ◐ symbol in the top right corner to toggle a dark mode theme for the duration of your browser session.

About the author

Hey, I'm Alex Edwards, a full-stack web developer and author. I live near Innsbruck, Austria.

I've been working with Go for over 10 years, building production applications for myself and commercial clients, and helping people all around the world improve their Go skills.

You can see more of my writing on my blog (where I publish detailed tutorials), some of my open-source work on GitHub, and you can also follow me on Instagram and Twitter.

Copyright and disclaimer

Let's Go Further. Copyright © 2026 Alex Edwards.

Last updated 2026-10-03 10:00:35 UTC. Version 1.27.1.

The Go gopher was designed by Renee French and is used under the Creative Commons 3.0 Attribution license. Cover gopher adapted from vectors by Egon Elbre.

The information provided within this book is for general informational purposes only. While the author and publisher have made every effort to ensure that the accuracy of the information within this book was correct at time of publication there are no representations or

dego_vila@yahoo.com 30 Oct 2026

warranties, express or implied, about the completeness, accuracy, reliability, suitability or availability with respect to the information, products, services, or related graphics contained in this book for any purpose. Any use of this information is at your own risk.

dego_vila@yahoo.com 30 Oct 2026

Chapter 1.1.

Prerequisites

Background knowledge

This book is written as a follow up to *Let's Go*, and we'll leverage a lot of the information and code patterns from that book again here.

If you've already read and enjoyed *Let's Go*, then this book should be a good fit for you and the ideal next step in your learning. If you haven't, then I highly recommend starting with *Let's Go* first — especially if you're a newcomer to Go.

You can read this as a standalone book, but please be aware that it is somewhat advanced — it doesn't explain the fundamentals in detail, and some topics (like testing) don't feature at all because they were covered heavily in the previous book. But if you're comfortable using Go and already have a decent amount of experience behind you, then this book may also be a good fit for you. Feel free to jump straight in.

Go 1.27

The information in this book is correct for Go version 1.27 and newer, and you should install this if you'd like to code along with the application build.

If you've already got Go installed, you can check the version number from your terminal by using the `go version` command. The output should look similar to this:

```
$ go version
go version go1.27.0 linux/amd64
```

If you need to upgrade your version of Go, then please go ahead and do that now. The instructions for your operating system can be found here.

Other software

There are a few other bits of software that you should make sure are available on your computer if you want to follow along fully. They are:

dego_vila@yahoo.com 30 Oct 2026

- The curl command-line tool for working with HTTP requests and responses from your terminal. On macOS and Linux machines it should be pre-installed or available in your software repositories. Otherwise, you can download the latest version from here.
- The hey tool for running some basic load tests. You can install hey with the `go install` command:

```
$ go install github.com/rakyll/hey@latest
```

- The git version control system. Installation instructions for all operating systems can be found here.
- A web browser with good developer tools. I'll be using Firefox in this book, but Chromium, Chrome or Microsoft Edge will work too.
- Your favorite text editor 😊

dego_vila@yahoo.com 30 Oct 2026

Chapter 2.

Getting started

In this first section of the book, we're going to set up a project directory and lay the groundwork for building our *Greenlight* API. We will:

- Create a skeleton directory structure for the project and explain at a high level how our Go code and other assets will be organized.
- Establish an HTTP server to listen for incoming HTTP requests.
- Introduce a sensible pattern for managing configuration settings (via command-line flags) and using *dependency injection* to make dependencies available to our handlers.
- Use the `httprouter` package to help implement a standard RESTful structure for the API endpoints.

dego_vila@yahoo.com 30 Oct 2026

Chapter 2.1.

Project setup and skeleton structure

Let's kick things off by creating a `greenlight` directory to act as the top-level 'home' for this project. I'm going to create my project directory at `$HOME/Projects/greenlight`, but feel free to choose a different location if you wish.

```
$ mkdir -p $HOME/Projects/greenlight
```

Then change into this directory and use the `go mod init` command to *enable modules* for the project.

When running this command, you'll need to specify a *module path*. This is essentially a unique identifier for your project. In this book I'll use `greenlight.alexedwards.net` as my module path, but if you're following along you should ideally swap this for something that is unique to you instead.

```
$ cd $HOME/Projects/greenlight
$ go mod init greenlight.alexedwards.net
go: creating new go.mod: module greenlight.alexedwards.net
```

At this point you'll see that a `go.mod` file has been created in the root of your project directory. If you open it up, it should look similar to this:

File: go.mod

```
module greenlight.alexedwards.net

go 1.27.0
```

We discussed modules in detail in the first *Let's Go* book, but as a quick refresher let's recap the main points here.

- When there is a valid `go.mod` file in the root of your project directory, your project *is a module*.
- When you're working inside your project directory and download a dependency with `go get`, then the exact version of the dependency will be recorded in the `go.mod` file. Because the exact version is known, this makes it much easier to ensure reproducible builds across different machines and environments.

dego_vila@yahoo.com 30 Oct 2026

- When you run or build the code in your project, Go will use the exact dependencies listed in the `go.mod` file. If the necessary dependencies aren't already on your local machine, then Go will automatically download them for you — along with any recursive dependencies too.
- The `go.mod` file also defines the *module path* (which is `greenlight.alexedwards.net` in my case). This is essentially the identifier that will be used as the *root import path* for the packages in your project.
- It's good practice to make the module path unique to you and your project. A common convention in the Go community is to base it on a URL that you own.

> **Hint**: If you feel unsure about any aspect of modules or how they work, the official Go Modules Wiki is an excellent resource and contains answers to a wide range of FAQs.

Generating the skeleton directory structure

Alright, now that our project directory has been created and we have a `go.mod` file, you can go ahead and run the following commands to generate a high-level skeleton structure for the project:

```
$ mkdir -p bin cmd/api internal migrations remote
$ touch Makefile
$ touch cmd/api/main.go
```

At this point your project directory should look exactly like this:

```
.
├── bin
├── cmd
│   └── api
│       └── main.go
├── internal
├── migrations
├── remote
├── go.mod
└── Makefile
```

Let's take a moment to talk through these files and folders and explain the purpose they'll serve in our finished project.

- The `bin` directory will contain our compiled application binaries, ready for deployment to a production server.

dego_vila@yahoo.com 30 Oct 2026

- The `cmd/api` directory will contain the application-specific code for our *Greenlight* API application. This will include the code for running the server, reading and writing HTTP requests, and managing authentication.
- The `internal` directory will contain various ancillary packages used by our API. It will contain the code for interacting with our database, doing data validation, sending emails and so on. Basically, any code that *isn't* application-specific and can potentially be reused will live in here. Our Go code under `cmd/api` will *import* the packages in the `internal` directory (but never the other way around).
- The `migrations` directory will contain the SQL migration files for our database.
- The `remote` directory will contain the configuration files and setup scripts for our production server.
- The `go.mod` file will declare our project dependencies, versions and module path.
- The `Makefile` will contain *recipes* for automating common administrative tasks — like auditing our Go code, building binaries, and executing database migrations.

It's important to point out that the directory name `internal` carries a special meaning and behavior in Go: any packages which live under this directory can only be imported by code *inside the parent of the* `internal` *directory*. In our case, this means that any packages which live in `internal` can only be imported by code inside our `greenlight` project directory.

Or, looking at it the other way, this means that any packages under `internal` *cannot be imported by code outside of our project*.

This is useful because it prevents other codebases from importing and relying on the (potentially unversioned and unsupported) packages in our `internal` directory — even if the project code is publicly available somewhere like GitHub.

Hello world!

Before we continue, let's quickly check that everything is set up correctly. Open the `cmd/api/main.go` file in your text editor and add the following code:

File: cmd/api/main.go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello world!")
}
```

dego_vila@yahoo.com 30 Oct 2026

Save this file, then use the `go run` command in your terminal to compile and execute the code in the `cmd/api` package. All being well, you will see the following output:

```
$ go run ./cmd/api
Hello world!
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 2.2.

A basic HTTP server

Now that the skeleton structure for our project is in place, let's focus our attention on getting an HTTP server up and running.

To start with, we'll configure our server to have just one endpoint: `/v1/healthcheck`. This endpoint will return some basic information about our API, including its current version number and operating environment (development, staging, production, etc.).

URL Pattern	Handler	Action
/v1/healthcheck	healthcheckHandler	Show application information

If you're following along, open up the `cmd/api/main.go` file and replace the 'hello world' application with the following code:

File: cmd/api/main.go

```
package main

import (
    "flag"
    "fmt"
    "log/slog"
    "net/http"
    "os"
    "time"
)

// Declare a string containing the application version number. Later in the book we'll
// generate this automatically at build time, but for now we'll just store the version
// number as a hard-coded global constant.
const version = "1.0.0"

// Define a config struct to hold all the configuration settings for our application.
// For now, the only configuration settings will be the network port that we want the
// server to listen on, and the name of the current operating environment for the
// application (development, staging, production, etc.). We will read in these
// configuration settings from command-line flags when the application starts.
type config struct {
    port int
    env  string
}

// Define an application struct to hold the dependencies for our HTTP handlers, helpers,
// and middleware. At the moment this only contains a copy of the config struct and a
// logger, but it will grow to include a lot more as our build progresses.
type application struct {
    config config
    logger *slog.Logger
}
```

dego_vila@yahoo.com 30 Oct 2026

```
func main() {
    // Declare an instance of the config struct.
    var cfg config

    // Read the value of the port and env command-line flags into the config struct. We
    // default to using the port number 4000 and the environment "development" if no
    // corresponding flags are provided.
    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")
    flag.Parse()

    // Initialize a new structured logger which writes log entries to the standard out
    // stream.
    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

    // Declare an instance of the application struct, containing the config struct and
    // the logger.
    app := &application{
        config: cfg,
        logger: logger,
    }

    // Declare a new servemux and add a /v1/healthcheck route which dispatches requests
    // to the healthcheckHandler method (which we will create in a moment).
    mux := http.NewServeMux()
    mux.HandleFunc("/v1/healthcheck", app.healthcheckHandler)

    // Declare an HTTP server which listens on the port provided in the config struct,
    // uses the servemux we created above as the handler, has some sensible timeout
    // settings, and writes any log messages to the structured logger at Error level.
    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", cfg.port),
        Handler:      mux,
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(logger.Handler(), slog.LevelError),
    }

    // Start the HTTP server.
    logger.Info("starting server", "addr", srv.Addr, "env", cfg.env)

    err := srv.ListenAndServe()
    logger.Error(err.Error())
    os.Exit(1)
}
```

> **Important:** If any of the terminology or patterns in the code above are unfamiliar to you, then I strongly recommend reading the first *Let's Go* book which explains them in detail.

Creating the healthcheck handler

The next thing we need to do is create the `healthcheckHandler` method for responding to HTTP requests. For now, we'll keep the logic in this handler really simple and have it return

dego_vila@yahoo.com 30 Oct 2026

a plain-text response containing three pieces of information:

- A fixed `"status: available"` string.
- The API version from the hard-coded `version` constant.
- The operating environment name from the `env` command-line flag.

Go ahead and create a new `cmd/api/healthcheck.go` file:

```
$ touch cmd/api/healthcheck.go
```

And then add the following code:

File: cmd/api/healthcheck.go

```
package main

import (
    "fmt"
    "net/http"
)

// Declare a handler which writes a plain-text response with information about the
// application status, operating environment and version.
func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "status: available")
    fmt.Fprintf(w, "environment: %s\n", app.config.env)
    fmt.Fprintf(w, "version: %s\n", version)
}
```

The important thing to point out here is that `healthcheckHandler` is implemented as a *method* on our `application` struct.

This is an effective and idiomatic way to make dependencies available to our handlers without resorting to global variables or closures — any dependency that the `healthcheckHandler` needs can simply be included as a field in the `application` struct when we initialize it in `main()`.

We can see this pattern already being used in the code above, where the operating environment name is retrieved from the `application` struct by calling `app.config.env`.

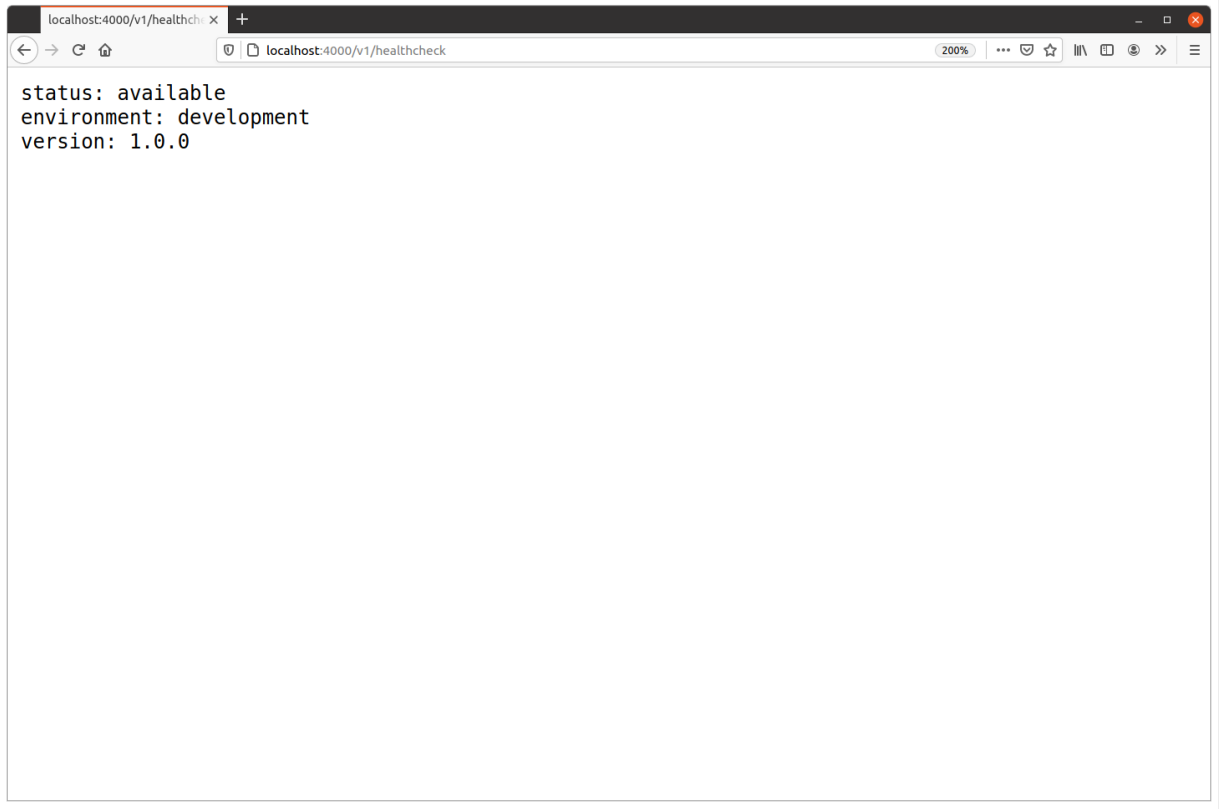
Demonstration

OK, let's try this out. Make sure that all your changes are saved, then use the `go run` command again to execute the `cmd/api` package code. You should see a log message confirming that the HTTP server is running, similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/api
time=2026-09-10T08:59:52.704+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

While the server is running, go ahead and try visiting `localhost:4000/v1/healthcheck` in your web browser. You should get a response from the `healthcheckHandler` which looks like this:



Alternatively, you can use `curl` to make the request from your terminal:

```
$ curl -i localhost:4000/v1/healthcheck
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 07:00:42 GMT
Content-Length: 58
Content-Type: text/plain; charset=utf-8

status: available
environment: development
version: 1.0.0
```

> **Note**: The `-i` flag in the command above instructs `curl` to display the HTTP response headers as well as the response body.

If you want, you can also verify that the command-line flags are working correctly by

dego_vila@yahoo.com 30 Oct 2026

specifying alternative `port` and `env` values when starting the application. When you do this, you should see the contents of the log message change accordingly. For example:

```
$ go run ./cmd/api -port=3030 -env=production
time=2026-09-10T09:01:48.542+02:00 level=INFO msg="starting server" addr=:3030 env=production
```

Additional information

API versioning

APIs which support real-world businesses and users often need to change their functionality and endpoints over time — sometimes in a backward-incompatible way. So, to avoid problems and confusion for clients, it's a good idea to always implement some form of API *versioning*.

There are two common approaches to doing this:

1. By prefixing all URLs with your API version, like `/v1/healthcheck` or `/v2/healthcheck`.
2. By using custom `Accept` and `Content-Type` headers on requests and responses to convey the API version, like `Accept: application/vnd.greenlight-v1`.

From an HTTP semantics point of view, using headers to convey the API version is the 'purer' approach. But from a user-experience point of view, using a URL prefix is arguably better. It makes it possible for developers to see which version of the API is being used at a glance, and it also means that the API can still be explored using a regular web browser (which is harder if custom headers are required).

Throughout this book we'll version our API by prefixing all the URL paths with `/v1/` — just like we did with the `/v1/healthcheck` endpoint in this chapter.

dego_vila@yahoo.com 30 Oct 2026

API endpoints and RESTful routing

Over the next few sections of this book, we're going to gradually build up our API so that the endpoints start to look like this:

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
GET	/v1/movies	listMoviesHandler	Show the details of all movies
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PUT	/v1/movies/:id	editMovieHandler	Update the details of a specific movie
DELETE	/v1/movies/:id	deleteMovieHandler	Delete a specific movie

If you've built APIs with REST style endpoints before, then the table above probably looks very familiar to you and doesn't require much explanation. But if you're new to this, then there are a couple of important things to point out.

The first thing is that requests with the same URL pattern will be routed to *different handlers based on the HTTP request method*. For both security and semantic correctness, it's important to use the appropriate HTTP method for the action that the handler is performing.

In summary:

dego_vila@yahoo.com 30 Oct 2026

Method	Usage
GET	Use for actions that retrieve information only and don't change the state of your application or any data.
POST	Use for non-idempotent actions that modify state. In the context of a REST API, `POST` is generally used for actions that *create* a new resource.
PUT	Use for idempotent actions that modify the state of a resource at a specific URL. In the context of a REST API, `PUT` is generally used for actions that *replace* or *update* an existing resource.
PATCH	Use for actions that *partially update* a resource at a specific URL. It's OK for the action to be either idempotent or non-idempotent.
DELETE	Use for actions that *delete* a resource at a specific URL.

The other important thing to point out is that our API endpoints will use clean URLs, with wildcard parameters interpolated in the URL path. So — for example — to retrieve the details of a specific movie a client will make a request like `GET /v1/movies/1`, instead of appending the movie ID in a query string parameter like `GET /v1/movies?id=1`.

Choosing a router

In this book we're going to use the popular third-party package `httprouter` as the router for our application, instead of using `http.ServeMux` from the standard library.

There are two reasons for this:

- We want our API to consistently send JSON responses wherever possible. Unfortunately, `http.ServeMux` sends plaintext (non-JSON) `404` and `405` responses when a matching route cannot be found, and it's not possible to easily customize these without causing a knock-on effect that inhibits the automatic sending of `405` responses. There is an open proposal to improve this in future versions of Go, but for now it's a fairly major drawback.
- Additionally, `http.ServeMux` does not automatically handle `OPTIONS` requests.

Both of these things are supported by `httprouter`, along with providing all the other functionality that we need. The `httprouter` package itself is stable and well-tested, and as a bonus it's also extremely fast thanks to its use of a radix tree for URL matching. If you're building a JSON REST API for public consumption, then `httprouter` is a solid choice and — on balance — it remains a better option than `http.ServeMux` right now.

dego_vila@yahoo.com 30 Oct 2026

If you're coding along with this book, please use `go get` to download the latest `v1.N.N` release of `httprouter` like so:

```
$ go get github.com/julienschmidt/httprouter@v1
go: downloading github.com/julienschmidt/httprouter v1.3.0
go get: added github.com/julienschmidt/httprouter v1.3.0
```

> **Note:** If you already have a copy of `httprouter` on your machine from another project, then your existing copy will be used and you won't see a `go: downloading...` message.

To demonstrate how `httprouter` works, we'll start by adding the two endpoints for *creating a new movie* and *showing the details of a specific movie* to our codebase. By the end of this chapter, our API endpoints will look like this:

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie

Encapsulating the API routes

To prevent our `main()` function from becoming cluttered as the API grows, let's encapsulate all the routing rules in a new `cmd/api/routes.go` file.

If you're following along, create this new file and add the following code:

```
$ touch cmd/api/routes.go
```

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

import (
    "net/http"

    "github.com/julienschmidt/httprouter"
)

func (app *application) routes() http.Handler {
    // Initialize a new httprouter router instance.
    router := httprouter.New()

    // Register the relevant methods, URL patterns and handler functions for our
    // endpoints using the HandlerFunc() method. Note that http.MethodGet and
    // http.MethodPost are constants which equate to the strings "GET" and "POST"
    // respectively.
    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)

    // Return the httprouter instance.
    return router
}
```

> **Hint**: The `httprouter` package also provides a `router.Handler()` method which you can use when you want to register a regular `http.Handler` (rather than handler functions, like we are in the code above).

There are a couple of benefits to encapsulating our routing rules in this way. The first benefit is that it keeps our `main()` function clean and ensures all our routes are defined in a single place. The other big benefit, which we demonstrated in the first *Let's Go* book, is that we can now easily access the router in any test code by initializing an `application` instance and calling the `routes()` method on it.

The next thing that we need to do is update the `main()` function to remove the `http.ServeMux` declaration, and use the `httprouter` instance returned by `app.routes()` as our server handler instead. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/main.go

```
package main

...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")
    flag.Parse()

    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

    app := &application{
        config: cfg,
        logger: logger,
    }

    // Use the httprouter instance returned by app.routes() as the server handler.
    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", cfg.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(logger.Handler(), slog.LevelError),
    }

    logger.Info("starting server", "addr", srv.Addr, "env", cfg.env)
    err := srv.ListenAndServe()
    logger.Error(err.Error())
    os.Exit(1)
}
```

Adding the new handler functions

Now that the routing rules are set up, we need to make the `createMovieHandler` and `showMovieHandler` methods for the new endpoints. The `showMovieHandler` is particularly interesting here, because as part of this we want to extract the movie ID parameter from the URL and use it in our HTTP response.

Go ahead and create a new `cmd/api/movies.go` file to hold these two new handlers:

```
$ touch cmd/api/movies.go
```

And then add the following code:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

import (
    "fmt"
    "net/http"
    "strconv"

    "github.com/julienschmidt/httprouter"
)

// Add a createMovieHandler for the "POST /v1/movies" endpoint. For now we simply
// return a plain-text placeholder response.
func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "create a new movie")
}

// Add a showMovieHandler for the "GET /v1/movies/:id" endpoint. For now, we retrieve
// the interpolated "id" parameter from the current URL and include it in a placeholder
// response.
func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    // When httprouter is parsing a request, any interpolated URL parameters will be
    // stored in the request context. We can use the ParamsFromContext() function to
    // retrieve a slice containing these parameter names and values.
    params := httprouter.ParamsFromContext(r.Context())

    // We can then use the ByName() method to get the value of the "id" parameter from
    // the slice. In our project all movies will have a unique positive integer ID, but
    // the value returned by ByName() is always a string. So we try to convert it to an
    // integer. If the parameter couldn't be converted, or is less than 1, we know the
    // ID is invalid so we use the http.NotFound() function to return a 404 Not Found
    // response.
    id, err := strconv.Atoi(params.ByName("id"))
    if err != nil || id < 1 {
        http.NotFound(w, r)
        return
    }

    // Otherwise, interpolate the movie ID in a placeholder response.
    fmt.Fprintf(w, "show the details of movie %d\n", id)
}
```

And with that, we're now ready to try this out!

Go ahead and restart the API application…

```
$ go run ./cmd/api
time=2026-09-10T09:04:36.755+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Then while the server is running, open a second terminal window and use `curl` to make some requests to the different endpoints. If everything is set up correctly, you will see some responses which look similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl localhost:4000/v1/healthcheck
status: available
environment: development
version: 1.0.0

$ curl -X POST localhost:4000/v1/movies
create a new movie

$ curl localhost:4000/v1/movies/123
show the details of movie 123
```

Notice how, in the final example, the value of the movie `id` parameter `123` has been successfully retrieved from the URL and included in the response?

You might also want to try making some requests for a particular URL *using an unsupported HTTP method*. For example, let's try making a `POST` request to `/v1/healthcheck`:

```
$ curl -i -X POST localhost:4000/v1/healthcheck
HTTP/1.1 405 Method Not Allowed
Allow: GET, OPTIONS
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Thu, 10 Sep 2026 07:05:29 GMT
Content-Length: 19

Method Not Allowed
```

That's looking really good. The `httprouter` package has automatically sent a `405 Method Not Allowed` response for us, including an `Allow` header which lists the HTTP methods that *are* supported for the endpoint.

Likewise, you can make an `OPTIONS` request to a specific URL and `httprouter` will send back a response with an `Allow` header detailing the supported HTTP methods. Like so:

```
$ curl -i -X OPTIONS localhost:4000/v1/healthcheck
HTTP/1.1 200 OK
Allow: GET, OPTIONS
Date: Thu, 10 Sep 2026 07:05:54 GMT
Content-Length: 0
```

Lastly, you might want to try making a request to the `GET /v1/movies/:id` endpoint with a negative number or a non-numerical `id` value in the URL. This should result in a `404 Not Found` response, similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i localhost:4000/v1/movies/abc
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Thu, 10 Sep 2026 07:06:15 GMT
Content-Length: 19

404 page not found
```

Creating a helper to read ID parameters

The code to extract an `id` parameter from a URL like `/v1/movies/:id` is something that we'll need repeatedly in our application, so let's abstract the logic for this into a small reusable helper method.

Go ahead and create a new `cmd/api/helpers.go` file:

```
$ touch cmd/api/helpers.go
```

And add a new `readIDParam()` method to the `application` struct, like so:

File: cmd/api/helpers.go

```
package main

import (
    "errors"
    "net/http"
    "strconv"

    "github.com/julienschmidt/httprouter"
)

// Retrieve the "id" URL parameter from the current request context, then convert it to
// an integer and return it. If the operation isn't successful, return 0 and an error.
func (app *application) readIDParam(r *http.Request) (int, error) {
    params := httprouter.ParamsFromContext(r.Context())

    id, err := strconv.Atoi(params.ByName("id"))
    if err != nil || id < 1 {
        return 0, errors.New("invalid id parameter")
    }

    return id, nil
}
```

dego_vila@yahoo.com 30 Oct 2026

> **Note**: The `readIDParam()` method doesn't use any dependencies from our `application` struct so it *could* just be a regular function, rather than a method on `application`. But in general, I suggest setting up *all* your application-specific handlers and helpers so that they are methods on `application`. It helps maintain consistency in your code structure, and also future-proofs your code for when those handlers and helpers change later and they *do* need access to a dependency.

With this helper method in place, the code in our `showMovieHandler` can now be made a lot simpler:

File: cmd/api/movies.go

```
package main

import (
    "fmt"
    "net/http"
)

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        http.NotFound(w, r)
        return
    }

    fmt.Fprintf(w, "show the details of movie %d\n", id)
}
```

Additional information

Conflicting routes

It's important to be aware that `httprouter` doesn't allow *conflicting routes* which potentially match the same request. So, for example, you cannot register a route like `GET /foo/new` *and* another route with a parameter segment that conflicts with it — like `GET /foo/:id`.

If you're using a standard REST structure for your API endpoints — like we will be in this book — then this restriction is unlikely to cause you many problems.

In fact, it's arguably a positive thing. Because conflicting routes aren't allowed, there are no

dego_vila@yahoo.com 30 Oct 2026

routing-priority rules that you need to worry about, and it reduces the risk of bugs and unintended behavior in your application.

But if you do need to support conflicting routes (for example, you might need to replicate the endpoints of an existing API exactly for backwards-compatibility), then I would recommend taking a look at `chi`, Gorilla `mux` or `flow` instead. All of these are good routers which *do* permit conflicting routes.

Customizing httprouter behavior

The `httprouter` package provides a few configuration options that you can use to customize the behavior of your application further, including enabling *trailing slash redirects* and enabling *automatic URL path cleaning*.

More information about the available settings can be found here.

dego_vila@yahoo.com 30 Oct 2026

Chapter 3.

Sending JSON responses

In this section of the book, we're going to update our API handlers so that they return JSON responses instead of just plain text.

JSON (which is an acronym for *JavaScript Object Notation*) is a human-readable text format that can be used to represent structured data. As an example, in our project a movie could be represented with the following JSON:

```
{
    "id": 123,
    "title": "Casablanca",
    "year": 1942,
    "genres": [
            "romance",
            "drama",
            "war"
    ],
    "version": 1
}
```

To give you a *very* quick overview of the JSON syntax…

- The parentheses `{}` define a *JSON object*, which is made up of comma-separated key-value pairs.
- The keys must be strings, and the values can be either strings, numbers, booleans, other JSON objects, or an *array* of any of those things (enclosed by `[]`).
- Strings must be surrounded by double quotes `"` (rather than single quotes `'`) and boolean values are either `true` or `false`.
- Outside of a string, whitespace characters in JSON *do not* carry any significance — it would be perfectly valid for us to represent the same movie using the following JSON instead:

```
{"id":123,"title":"Casablanca","year":1942,"genres":["romance","drama","war"],"version":1}
```

If you've not used JSON at all before, then this beginner's guide provides a comprehensive introduction and I recommend reading it before continuing.

In this section of the book you'll learn:

- How to send JSON responses from your REST API (including error responses).

dego_vila@yahoo.com 30 Oct 2026

- How to encode native Go types into JSON using the `encoding/json/v2` package.
- Different techniques for customizing how Go types are encoded to JSON — first by using struct tags, and then by using a custom marshaler.
- How to create a reusable helper for sending JSON responses, which will ensure that all your API responses have a sensible and consistent structure.

dego_vila@yahoo.com 30 Oct 2026

Chapter 3.1.

Fixed-format JSON

Let's begin by updating our `healthcheckHandler` to send a well-formed JSON response that looks like this:

```
{"status": "available", "environment": "development", "version": "1.0.0"}
```

At this stage, the thing I'd like to emphasize is that JSON is *just text*. Sure, it has certain control characters that give the text structure and meaning, but fundamentally, it is just text.

So that means you can write a JSON response from your Go handlers in the same way that you would write any other text response: using `w.Write()`, `io.WriteString()` or one of the `fmt.Fprint` functions.

In fact, the only special thing we need to do is set a `Content-Type: application/json` header on the response, so that the client knows it's receiving JSON and can interpret it accordingly.

Let's do exactly that.

Open up the `cmd/api/healthcheck.go` file and update the `healthcheckHandler` as follows:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/healthcheck.go

```
package main

import (
    "fmt"
    "net/http"
)

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    // Create a fixed-format JSON response from a string. Notice how we're using a raw
    // string literal (enclosed with backticks) so that we can include double quote
    // characters in the JSON without needing to escape them? We also use the %q verb to
    // wrap the interpolated values in double quotes.
    js := `{"status": "available", "environment": %q, "version": %q}`
    js = fmt.Sprintf(js, app.config.env, version)

    // Set the "Content-Type: application/json" header on the response. If you forget to do
    // this, Go will default to sending a "Content-Type: text/plain; charset=utf-8"
    // header instead.
    w.Header().Set("Content-Type", "application/json")

    // Write the JSON as the HTTP response body.
    w.Write([]byte(js))
}
```

Once you've made those changes, restart the API, open a second terminal, and use `curl` to make a request to the `GET /v1/healthcheck` endpoint. You should now get back a response that looks like this:

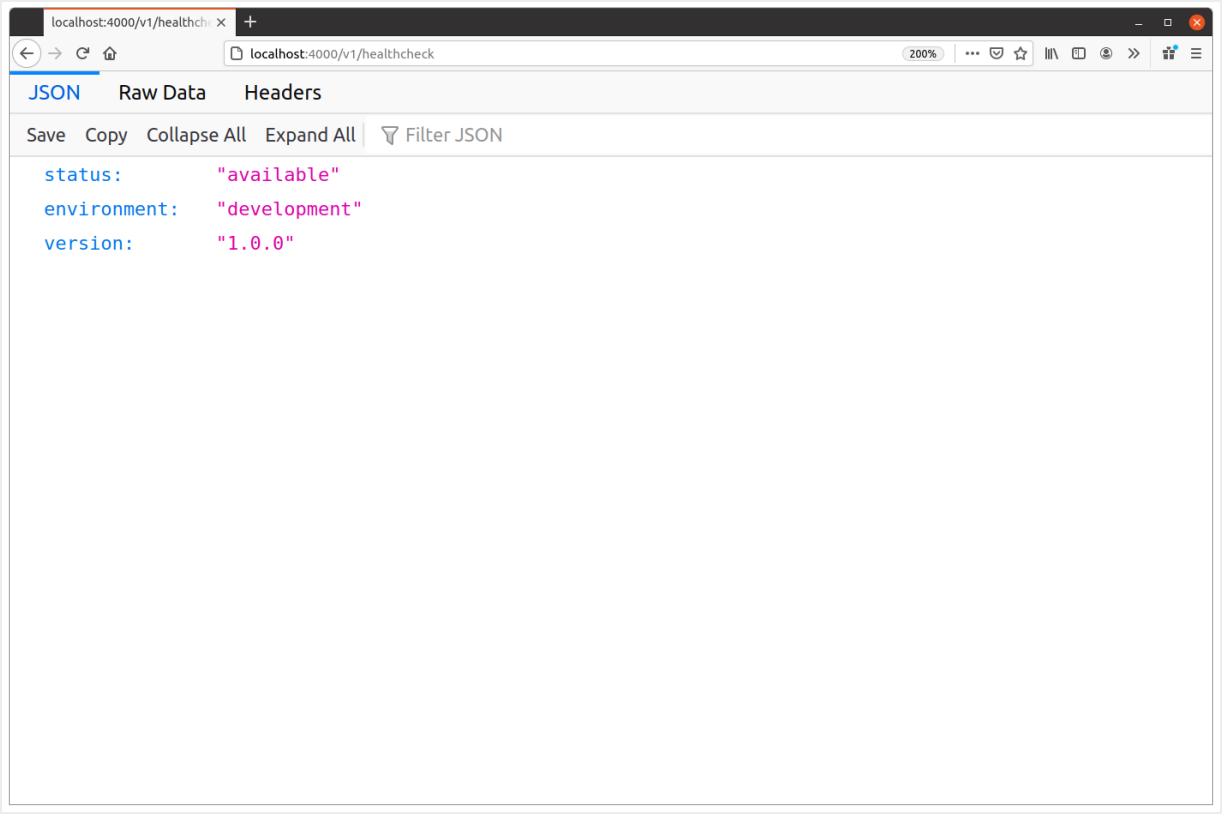
```
$ curl -i localhost:4000/v1/healthcheck
HTTP/1.1 200 OK
Content-Type: application/json
Date: Thu, 10 Sep 2026 07:12:02 GMT
Content-Length: 73

{"status": "available", "environment": "development", "version": "1.0.0"}
```

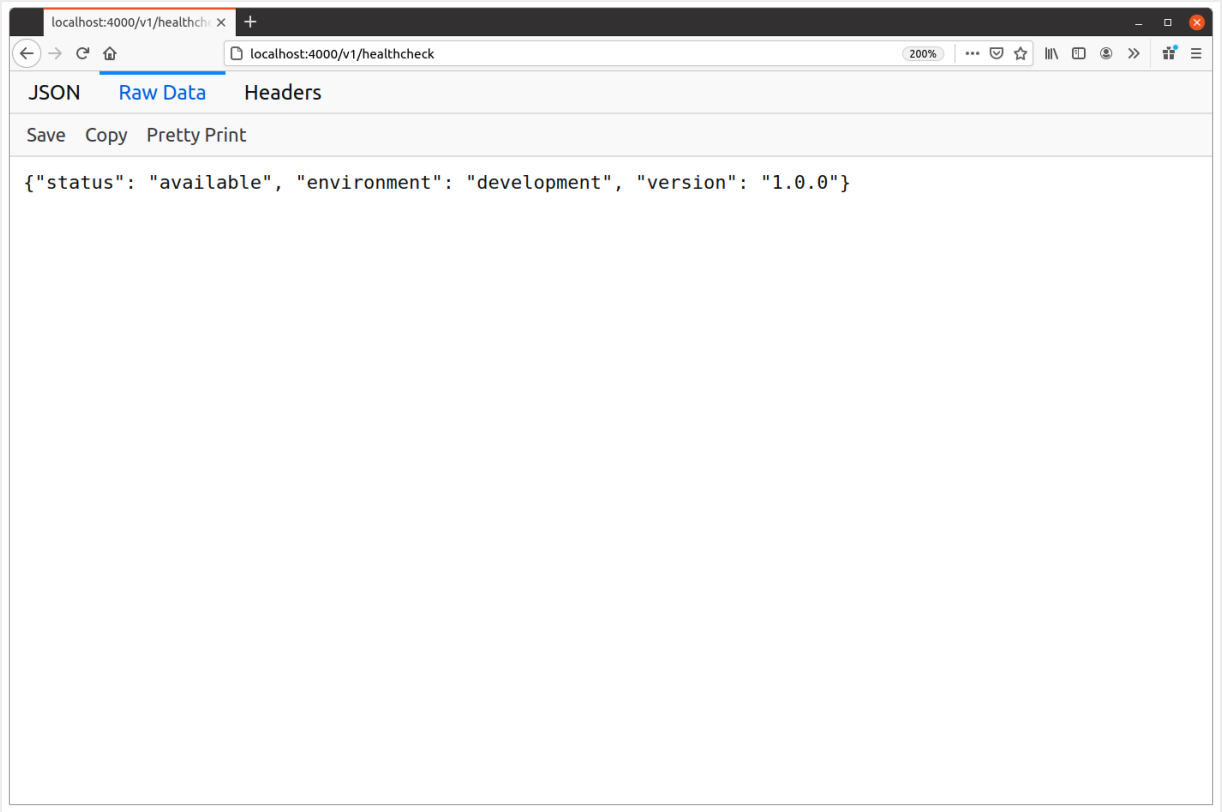
You might also like to try visiting `localhost:4000/v1/healthcheck` in your browser.

If you're running one of the newer versions of Firefox, you'll find that it knows the response contains JSON (due to the `Content-Type` header) and it will present the response using the built-in JSON viewer. Like so:

dego_vila@yahoo.com 30 Oct 2026

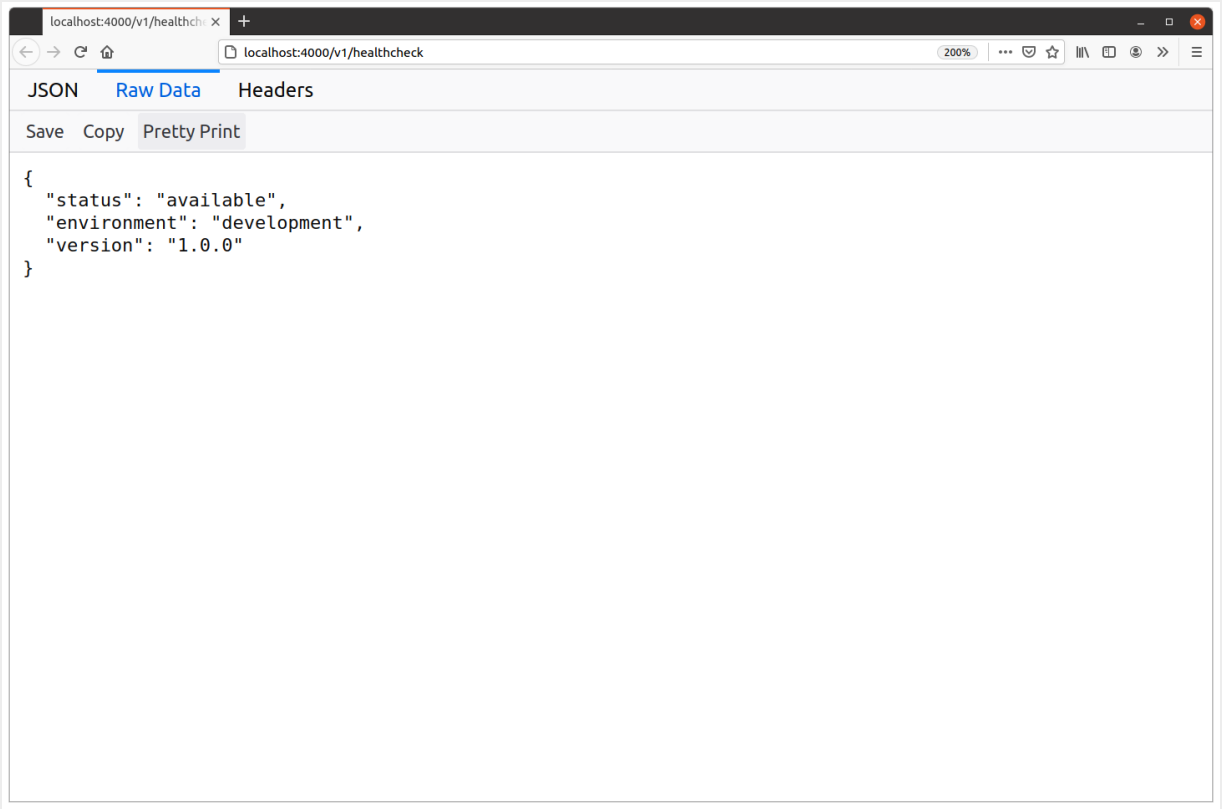


If you click on the **Raw Data** tab you should see the original unformatted JSON response:



Or you can even select the **Pretty Print** option to add whitespace, like so:

dego_vila@yahoo.com 30 Oct 2026



Of course, using a fixed-format string like we are in this chapter is a pretty simple and lo-fi approach to generating a JSON response. But it's worth remembering that it *is* a valid option. It can be useful for API endpoints that always return the same static JSON, or as a quick and easy way to generate small dynamic responses like we have here.

Additional information

JSON charset

In your programming career you might have come across other JSON APIs which send responses with the header `Content-Type: application/json; charset=utf-8`.

Including a `charset` parameter like this isn't normally necessary. The JSON RFC states:

> *JSON text exchanged between systems that are not part of a closed ecosystem MUST be encoded using UTF-8.*

The important word here is "must". Because our API will be a public-facing application, it means that our JSON responses must always be UTF-8 encoded. And it also means that *it's safe for the client to assume that the responses it gets are always UTF-8 encoded*. Because of

dego_vila@yahoo.com 30 Oct 2026

this, including a `charset=utf-8` parameter is redundant.

The RFC also explicitly notes that the `application/json` media type does not have a `charset` parameter defined, which means that it is technically incorrect to include one too.

Basically, in our application a `charset` parameter serves no purpose, and it's safe (and also correct) to not include one in our `Content-Type: application/json` header.

dego_vila@yahoo.com 30 Oct 2026

Chapter 3.2.

JSON encoding

Let's move on to something a bit more exciting and look at how to encode native Go types (like maps, structs and slices) to JSON.

But before we get into it, we need to quickly discuss the fact that Go has two packages for working with JSON. There is the legacy `encoding/json` package, and the new and shiny `encoding/json/v2` package — which became part of the standard library in Go 1.27.

The `encoding/json/v2` package is a big step forward for Go when it comes to working with JSON, and it improves on the original legacy package in a number of ways. At a very high level:

- It has safer defaults, which close off some potential security vulnerabilities.
- It's much faster for JSON decoding operations — up to 10x faster in some benchmarks.
- The error values that it returns are generally more consistent, predictable and easier to handle gracefully.
- It provides more configuration options to control and customize encoding/decoding behavior, and better support for custom encoding/decoding of specific types.

Right now, most of the Go code you look at online, the blog posts you read, or the code that AI tools generate, will be still be using the legacy `encoding/json` package. But if you're starting work on a new project, then it's a good idea to build it using `encoding/json/v2`.

As the documentation for the legacy package advises:

> *All new usages of "json" in Go should use the v2 package, but the v1 package will forever remain supported.*

It's really important to be aware that there are a lot of differences between the legacy `encoding/json` package and the new `encoding/json/v2` package — not just in syntax but also *behavior*.

If you've previously worked with Go and are already familiar with the legacy `encoding/json` package, it will probably be helpful to read the migration guidance here and here to get an overview of the behavioral differences between the two packages.

But for the rest of this book, we'll be using the new `encoding/json/v2` package, and the information and discussion is correct for that.

dego_vila@yahoo.com 30 Oct 2026

Encoding values

The `encoding/json/v2` package provides three different functions for encoding things to JSON: `json.Marshal()`, `json.MarshalEncode()`, and `json.MarshalWrite`.

We'll explain how all three functions work and the differences between them over the course of this chapter, but — for the purpose of sending JSON in an HTTP response — using `json.Marshal()` is generally the best choice. So let's start with that.

The way that `json.Marshal()` works is conceptually quite simple — you pass a native Go value to it as an argument, and it returns a JSON representation of that value in a `[]byte` slice. The function signature looks like this:

```
func Marshal(in any, opts ...Options) (out []byte, err error)
```

> **Note:** The `in` parameter in the above method has the type `any`. This effectively means that you're able to pass any Go type to `Marshal()` as the `in` argument.
>
> There is also a variadic `opts` parameter, which you can use to pass in additional `json.Options` values to control specific encoding behaviors. We'll demonstrate most of these different options as we progress through the book, but it's totally OK to call `json.Marshal()` without specifying any options if you're happy with the default behaviors.

Let's jump in and update our `healthcheckHandler` so that it uses `json.Marshal()` to generate a JSON response directly from a Go map — instead of using a fixed-format string like we were before. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/healthcheck.go

```
package main

import (
    "encoding/json/v2" // New import
    "net/http"
)

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    // Create a map which holds the information that we want to send in the response.
    data := map[string]string{
        "status":      "available",
        "environment": app.config.env,
        "version":     version,
    }

    // Pass the map to the json.Marshal() function, without specifying any non-default
    // options. This returns a []byte slice containing the encoded JSON. If there was
    // an error, we log it and send the client a generic error message.
    js, err := json.Marshal(data)
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
        return
    }

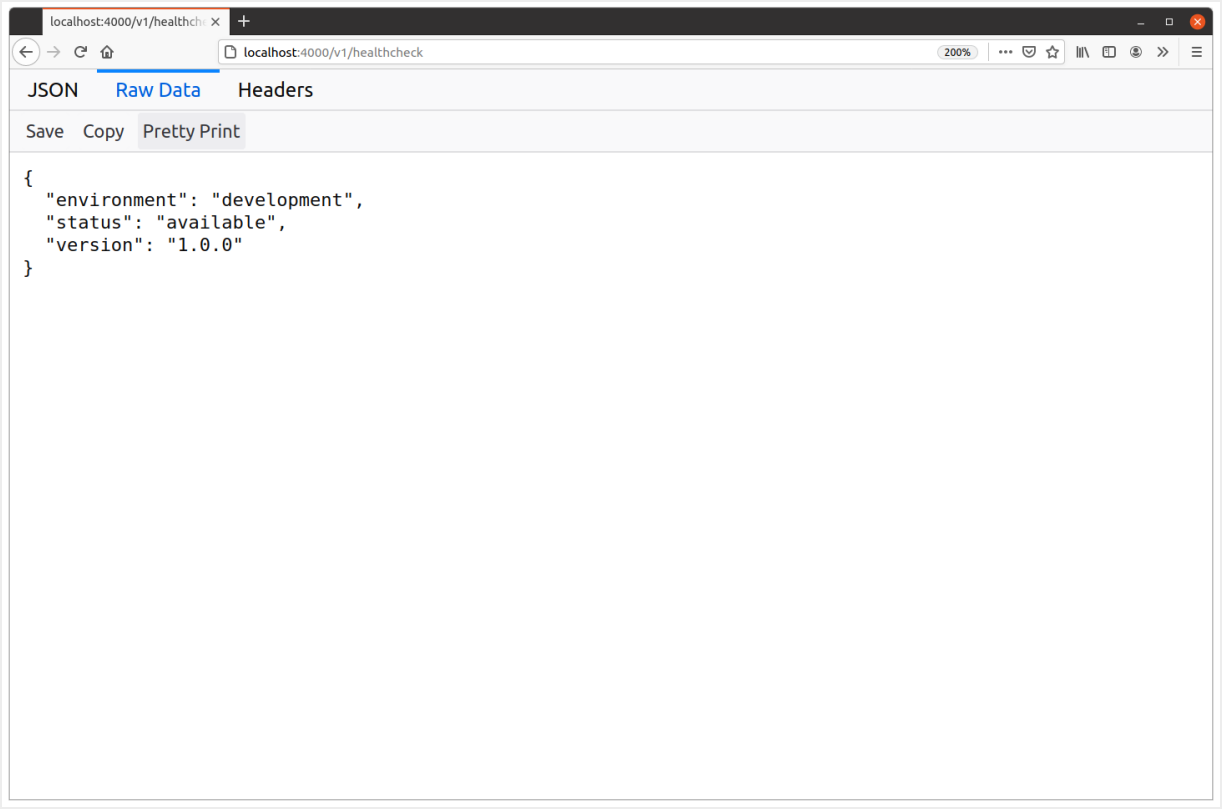
    // Append a newline to the JSON. This is just a small nicety to make it easier to
    // view in terminal applications.
    js = append(js, '\n')

    // At this point we know that encoding the data worked without any problems, so we
    // can safely set any necessary HTTP headers for a successful response.
    w.Header().Set("Content-Type", "application/json")

    // Use w.Write() to send the []byte slice containing the JSON as the response body.
    w.Write(js)
}
```

If you restart the API and visit `localhost:4000/v1/healthcheck` in your browser, you should now get a response similar to this:

dego_vila@yahoo.com 30 Oct 2026



That's looking good — we can see that the map has automatically been encoded to a JSON object for us, with the key-value pairs in the map appearing as key-value pairs in the JSON object.

Deterministic ordering

If you're following along with the application build, the key-value pairs displayed in your browser might appear in a different order to the screenshot above. And if you refresh the page enough times, you'll probably see the order of the key-value pairs changing between requests.

So why's this happening?

As you probably already know, maps in Go are unordered by design and their iteration order is non-deterministic. As the spec says: *"The iteration order over maps is not specified and is not guaranteed to be the same from one iteration to the next"*.

This also applies when encoding maps to JSON with the `encoding/json/v2` package — the map values are encoded to JSON in a non-deterministic, unpredictable, order.

As far as JSON goes, this is totally OK. The JSON RFC says that JSON objects are *"unordered collections of zero or more name/value pairs"*, so in theory anything that consumes the JSON

dego_vila@yahoo.com 30 Oct 2026

response shouldn't be relying on the values being in a specific order.

But practically, there can be some drawbacks of the order changing unpredictably. For example, if you are a user looking for a specific value in a large response when trying to debug a problem, it can be annoying if the value keeps moving to a different place each time. Or it could be a bit confusing for users if the responses they see don't appear to match the example responses that you provide in the documentation for your API.

Luckily, there is a `json.Deterministic()` configuration option that we can use, so that the same input value to `json.Marshal()` always results in exactly the same JSON output. Let's update `healthcheckHandler` to use this, like so:

File: cmd/api/healthcheck.go

```
package main

...

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]string{
        "status":      "available",
        "environment": app.config.env,
        "version":     version,
    }

    // Use the json.Deterministic(true) option to force the data map to be
    // encoded in a predictable, deterministic way.
    js, err := json.Marshal(data, json.Deterministic(true))
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
        return
    }

    js = append(js, '\n')

    w.Header().Set("Content-Type", "application/json")
    w.Write(js)
}
```

If you run this again now, you'll see that the responses always look the same, with the key-value pairs in the JSON sorted alphabetically by key name.

> **Note:** At the time of writing, the documentation for `json.Deterministic()` says *"the same input value will be serialized as the exact same output bytes"*. It doesn't explicitly call out that maps will be sorted by key, although you can verify this yourself by checking the underlying code here.

dego_vila@yahoo.com 30 Oct 2026

Creating a writeJSON helper method

As our API grows we're going to be sending a lot of JSON responses, so it makes sense to move some of this logic into a reusable `writeJSON()` helper method.

As well as creating and sending the JSON, we want to design this helper so that we can include *arbitrary headers* in successful responses later, such as a `Location` header after creating a new movie in our system.

If you're coding along, open the `cmd/api/helpers.go` file again and create the following `writeJSON()` method:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

import (
    "encoding/json/v2" // New import
    "errors"
    "net/http"
    "strconv"

    "github.com/julienschmidt/httprouter"
)

...

// Define a writeJSON() helper for sending responses. This takes the destination
// http.ResponseWriter, the HTTP status code to send, the data to encode to JSON, and a
// headers map containing any additional HTTP headers we want to include in the response.
func (app *application) writeJSON(w http.ResponseWriter, status int, data any, headers http.Header) error {
    // Encode the data to JSON, using the Deterministic option and immediately return
    // the error if there was one.
    js, err := json.Marshal(data, json.Deterministic(true))
    if err != nil {
        return err
    }

    // Append a newline to make it easier to view in terminal applications.
    js = append(js, '\n')

    // At this point, we know that we won't encounter any more errors before writing the
    // response, so it's safe to add any headers that we want to include. We loop
    // through the headers map (which behind the scenes has the type map[string][]string)
    // and add all the header keys and values to the http.ResponseWriter's header map.
    // Note that it's OK if the provided headers map is nil. Go doesn't throw an error
    // if you try to range over (or generally, read from) a nil map.
    for key, values := range headers {
        for _, value := range values {
            w.Header().Add(key, value)
        }
    }

    // Add the "Content-Type: application/json" header, then write the status code and
    // JSON response.
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    w.Write(js)

    return nil
}
```

Now that the `writeJSON()` helper is in place, we can significantly simplify the code in `healthcheckHandler`:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/healthcheck.go

```
package main

import (
    "net/http"
)

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]string{
        "status":      "available",
        "environment": app.config.env,
        "version":     version,
    }

    err := app.writeJSON(w, http.StatusOK, data, nil)
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
    }
}
```

If you run the application again now, everything will compile correctly, and a request to the `GET /v1/healthcheck` endpoint should result in the same HTTP response as before.

Additional information

How different Go types are encoded

In this chapter we've been encoding a `map[string]string` type to JSON, which resulted in a JSON object with JSON strings as the values in the key-value pairs. But Go supports encoding many other native types too.

The following table summarizes how different Go types are mapped to JSON data types during encoding:

dego_vila@yahoo.com 30 Oct 2026

Go type		JSON type
`bool`	⇒	JSON boolean
`string`	⇒	JSON string
integers, floating point numbers, `rune`	⇒	JSON number
arrays, slices	⇒	JSON array
structs, maps	⇒	JSON object
`nil` pointers, `nil` interface values	⇒	JSON null
`time.Time`	⇒	RFC3339-format JSON string
`netip.Addr`, `netip.AddrPort`, `netip.Prefix`	⇒	Network address JSON string
`byte` arrays, `byte` slices	⇒	Base64-encoded JSON string
`time.Duration`, `chan`, `func`, complex numbers	⇒	Not supported unless a custom marshaler is used

The last four of these are special cases which deserve a bit more explanation:

- `time.Time` values (which are actually a `struct` behind the scenes) will be encoded as a JSON string in RFC 3339 format like `"2020-11-08T06:27:59+01:00"`, rather than as a JSON object.
- `netip.Addr`, `netip.AddrPort` and `netip.Prefix` values (all of which are also a `struct` behind the scenes) will be encoded as a JSON strings containing the `netip` textual representation, like `"127.0.0.1"`, `"127.0.0.1:8080"` and `"127.0.0.1/24"`, respectively, rather than as a JSON object.
- `byte` slices and arrays (like `[]byte` and `[3]byte`) will be encoded as a base64-encoded JSON string, rather than as a JSON array. So, for example, a byte slice of `[]byte{'h','e','l','l','o'}` would appear as `"aGVsbG8="` in the JSON output. The base64 encoding uses padding and the standard character set.
- `time.Duration` (which is an `int64` behind the scenes), `chan`, `func`, `complex64` and `complex128` types do not have a default JSON representation, and trying to encode them will result in an error. To encode them, you need to create a custom marshaler for the type, which we'll explain how to do later in the book.

There are also a few other important things to mention:

dego_vila@yahoo.com 30 Oct 2026

- Encoding of nested types is supported. So, for example, if you have a *slice of structs* in Go that will encode to an *array of objects* in JSON.
- Any pointer values will encode as *the value pointed to*.
- By default, `nil` slices and maps are encoded to an empty JSON array or empty JSON object respectively. If you want `nil` slices and maps to encode to a JSON `null` value instead, you can pass the `json.FormatNilSliceAsNull(true)` and `json.FormatNilMapAsNull(true)` options to `json.Marshal()`. Please see this Go playground example for a demonstration.

Using json.MarshalWrite

At the start of this chapter I mentioned that it's also possible to use Go's `json.MarshalWrite()` function to perform the encoding. This allows you to encode a Go value to JSON *and write that JSON to anything that satisfies the* `io.Writer` *interface* in a single step.

Because `http.ResponseWriter` satisfies the `io.Writer` interface, this means that you could use `json.MarshalWrite()` to write a JSON HTTP response from a handler like so:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]string{
        "hello": "world",
    }

    // Set the "Content-Type: application/json" header on the response.
    w.Header().Set("Content-Type", "application/json")

    // Use the json.MarshalWrite() function to encode the data map to JSON and
    // write it to the http.ResponseWriter, passing in any options as necessary.
    err := json.MarshalWrite(w, data, json.Deterministic(true))
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
    }
}
```

This pattern works, and it's very neat and elegant, but if you consider it carefully you might notice a slight problem…

When we call `json.MarshalWrite(w, data)` the JSON is created and written to the `http.ResponseWriter` in a single step, which means there's no opportunity to set HTTP response headers conditionally based on whether `json.MarshalWrite()` returns an error or not.

Imagine, for example, that you want to set a `Cache-Control` header on a successful

dego_vila@yahoo.com 30 Oct 2026

response, *but not set a* `Cache-Control` *header if the JSON encoding fails and you have to return an error response*.

Implementing that cleanly while using `json.MarshalWrite()` is quite difficult.

You *could* set the `Cache-Control` header and then *delete it from the header map again* in the event of an error — but that's pretty hacky.

Another option is to write the JSON to an interim `bytes.Buffer` instead of directly to the `http.ResponseWriter`. You can then check for any errors, before setting the `Cache-Control` header and copying the JSON from the `bytes.Buffer` to `http.ResponseWriter`. But once you start doing that, it's simpler and cleaner (as well as slightly faster) to use the alternative `json.Marshal()` approach instead.

Using json.MarshalEncode

I also said at the start of this chapter that there is a third option you can use to encode data: the `json.MarshalEncode()` function.

This function is really intended for encoding and writing *streams* of JSON values, but in theory we could use it in a handler to send a single JSON object to a client too.

To use it, you first need to create a `jsontext.Encoder` type that writes to `http.ResponseWriter`, and then you pass this `jsontext.Encoder` to the `json.MarshalEncode()` function. Like so:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]string{
        "hello": "world",
    }

    // Set the "Content-Type: application/json" header on the response.
    w.Header().Set("Content-Type", "application/json")

    // Use the jsontext.NewEncoder() to initialize a jsontext.Encoder instance that
    // writes to the http.ResponseWriter.
    enc := jsontext.NewEncoder(w)

    // Pass the encoder to json.MarshalEncode(), along with the data to encode and
    // any options.
    err := json.MarshalEncode(enc, data, json.Deterministic(true))
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
    }
}
```

Again, this pattern works. But it feels more complicated than the other two options, and it suffers from the same issue as `json.MarshalWrite()` — it's awkward if you want to set HTTP

dego_vila@yahoo.com 30 Oct 2026

response headers conditionally based on whether the `json.MarshalEncode()` function returns an error or not.

However, if you're not worried about setting HTTP response headers conditionally, then `json.MarshalEncode()` may be an appropriate and efficient choice when you want to send a *stream* of JSON objects over HTTP to a client. I've included an example of streaming JSON objects using this and `http.ResponseController` in this gist.

Performance of different encoding options

At this point, you might be wondering if there's any performance difference between the code patterns we've illustrated for using `json.Marshal()`, `json.MarshalWrite()` and `json.MarshalEncode()` in this chapter.

The following benchmarks demonstrate the performance of the three patterns using the code in this gist (note that each benchmark test is repeated six times):

```
$ go test -run=^$ -bench=. -benchmem -count=6 -benchtime=2s
goos: linux
goarch: amd64
cpu: 12th Gen Intel(R) Core(TM) i7-12700H
BenchmarkMarshal-20          5279487    451.5 ns/op    120 B/op     4 allocs/op
BenchmarkMarshal-20          5207587    454.9 ns/op    120 B/op     4 allocs/op
BenchmarkMarshal-20          5185378    448.8 ns/op    120 B/op     4 allocs/op
BenchmarkMarshal-20          5278608    451.3 ns/op    120 B/op     4 allocs/op
BenchmarkMarshal-20          5450536    451.3 ns/op    120 B/op     4 allocs/op
BenchmarkMarshal-20          5358972    457.1 ns/op    120 B/op     4 allocs/op
BenchmarkMarshalWrite-20     5525970    432.1 ns/op     40 B/op     3 allocs/op
BenchmarkMarshalWrite-20     5398869    436.2 ns/op     40 B/op     3 allocs/op
BenchmarkMarshalWrite-20     5502087    440.0 ns/op     40 B/op     3 allocs/op
BenchmarkMarshalWrite-20     5527177    434.4 ns/op     40 B/op     3 allocs/op
BenchmarkMarshalWrite-20     5524167    430.2 ns/op     40 B/op     3 allocs/op
BenchmarkMarshalWrite-20     5578692    432.5 ns/op     40 B/op     3 allocs/op
BenchmarkMarshalEncode-20    2760849    800.8 ns/op    590 B/op    12 allocs/op
BenchmarkMarshalEncode-20    3102901    841.7 ns/op    590 B/op    12 allocs/op
BenchmarkMarshalEncode-20    3068803    817.3 ns/op    589 B/op    12 allocs/op
BenchmarkMarshalEncode-20    3153391    855.5 ns/op    590 B/op    12 allocs/op
BenchmarkMarshalEncode-20    2950111    863.2 ns/op    590 B/op    12 allocs/op
BenchmarkMarshalEncode-20    2931187    863.1 ns/op    589 B/op    12 allocs/op
```

For these specific benchmarks — which test encoding and writing a single JSON object in an HTTP response — we can see that the `json.MarshalWrite()` pattern is the clear winner.

In terms of speed (`ns/op`) the `json.Marshal()` and `json.MarshalWrite()` patterns are close to indistinguishable, but there is a big difference in memory use: the `json.Marshal()` pattern allocates 3 times as much memory (`B/op`) and makes one more heap allocation (`allocs/op`) than `json.MarshalWrite()` does.

For most applications, this difference in memory use is small enough in real terms that it

dego_vila@yahoo.com 30 Oct 2026

simply won't matter, and you shouldn't worry about it.

But if your application is handling a very high request volume, you're seeing noticeable memory pressure, and profiling your application leads you to believe that JSON encoding is the bottleneck… then it may be worth experimenting with switching to using the `json.MarshalWrite()` pattern, and accepting the problem of not being able to set HTTP response headers conditionally.

Additional JSON encoding nuances

Encoding things to JSON in Go is mostly quite intuitive. But there are a handful of behaviors which might catch you out or surprise you the first time you encounter them.

We've already mentioned a couple of these in this chapter (in particular — byte slices being base64 encoded, and `time.Duration` not being supported), but I've included a full list in this appendix.

dego_vila@yahoo.com 30 Oct 2026

Encoding structs

In this chapter we're going to revisit the `showMovieHandler` method that we made earlier and update it to return a JSON response which represents a single movie in our system. Similar to this:

```
{
    "id": 123,
    "title": "Casablanca",
    "year": 1942,
    "genres": [
        "romance",
        "drama",
        "war"
    ],
    "version": 1
}
```

Instead of encoding a map to create this JSON object (like we did in the previous chapter), this time we're going to encode a custom `Movie` struct.

So, first things first, we need to begin by defining a `Movie` struct. We'll do this inside a new `internal/models` package, which later will grow to encapsulate *all* the custom data types for our project, along with validation rules for those types and with the logic for interacting with our database.

> **Note:** If you don't like the package name *models*, then you could call it something like `internal/entities`, `internal/domain` or `internal/data` instead. We'll stick with the package name `internal/models` throughout this book, but whatever you prefer to call it, the principle is the same — this package will encapsulate our application's core data types, along with the code for validating and persisting them.

If you're following along, go ahead and create a new `internal/models` directory containing a `movies.go` file:

```
$ mkdir internal/models
$ touch internal/models/movies.go
```

And in this new file, let's define the `Movie` struct like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

import (
    "time"
)

type Movie struct {
    ID        int       // Unique integer ID for the movie
    CreatedAt time.Time // Timestamp for when the movie is added to our database
    Title     string    // Movie title
    Year      int       // Movie release year
    Genres    []string  // Slice of genres for the movie (romance, comedy, etc.)
    Version   int       // The version number starts at 1 and will be incremented each
                        // time the movie information is updated
}
```

> **Important**: It's crucial to point out here that all the fields in our `Movie` struct are exported (i.e. start with a capital letter), which is necessary for them to be visible to Go's `encoding/json/v2` package. Any fields which *aren't* exported won't be included when encoding a struct to JSON.

Now that's done, let's update our `showMovieHandler` to initialize an instance of the `Movie` struct containing some dummy data, and then send it as a JSON response using our `writeJSON()` helper.

It's quite simple in practice:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

import (
    "fmt"
    "net/http"
    "time" // New import

    "greenlight.alexedwards.net/internal/models" // New import
)

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        http.NotFound(w, r)
        return
    }

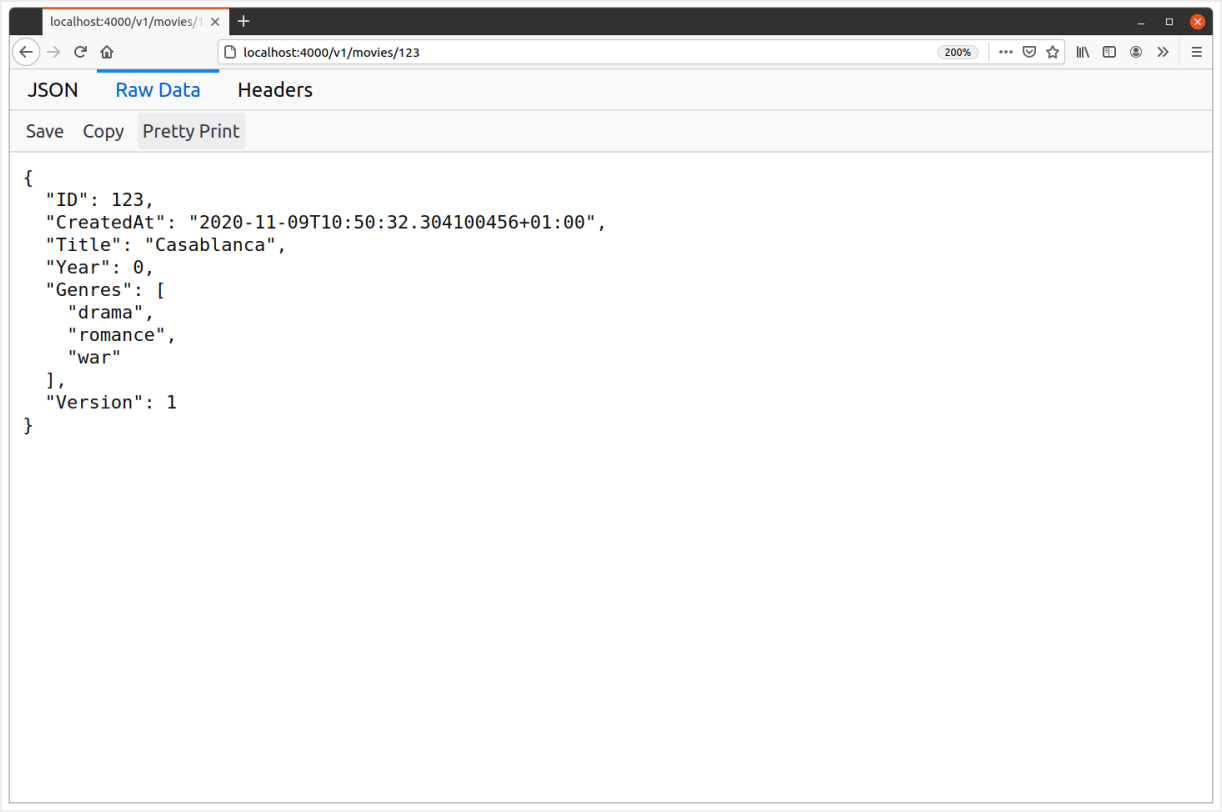
    // Create a new instance of the Movie struct, containing the ID we extracted from
    // the URL and some dummy data. Also notice that we deliberately haven't set a
    // value for the Year field.
    movie := models.Movie{
        ID:        id,
        CreatedAt: time.Now(),
        Title:     "Casablanca",
        Genres:    []string{"romance", "drama", "war"},
        Version:   1,
    }

    // Encode the struct to JSON and send it as the HTTP response.
    err = app.writeJSON(w, http.StatusOK, movie, nil)
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
    }
}
```

OK, let's give this a try!

Restart the API and then visit `localhost:4000/v1/movies/123` in your browser. You should see a JSON response which looks similar to this:

dego_vila@yahoo.com 30 Oct 2026



There are a few interesting things in this response to point out:

- Our `Movie` struct has been encoded into a single JSON object, with the field names and values as the key-value pairs.
- By default, the keys in the JSON object are equal to the field names in the struct (`ID`, `CreatedAt`, `Title` and so on). We'll talk about how to customize these shortly.
- If a struct field doesn't have an explicit value set, then the JSON-encoding of the *zero value* for the field will appear in the output. We can see an example of this in the response above — we didn't set a value for the `Year` field in our Go code, but it still appears in the JSON output with the value `0`.

Changing keys in the JSON object

One of the nice things about encoding structs in Go is that you can customize the JSON by annotating the fields with struct tags.

Probably the most common use of struct tags is to change the key names that appear in the JSON object. This can be useful when your struct field names aren't appropriate for public-facing responses, or you want to use an alternative casing style in your JSON output.

To illustrate how to do this, let's annotate our `Movies` struct with struct tags so that it uses

dego_vila@yahoo.com 30 Oct 2026

`snake_case` for the keys instead. Like so:

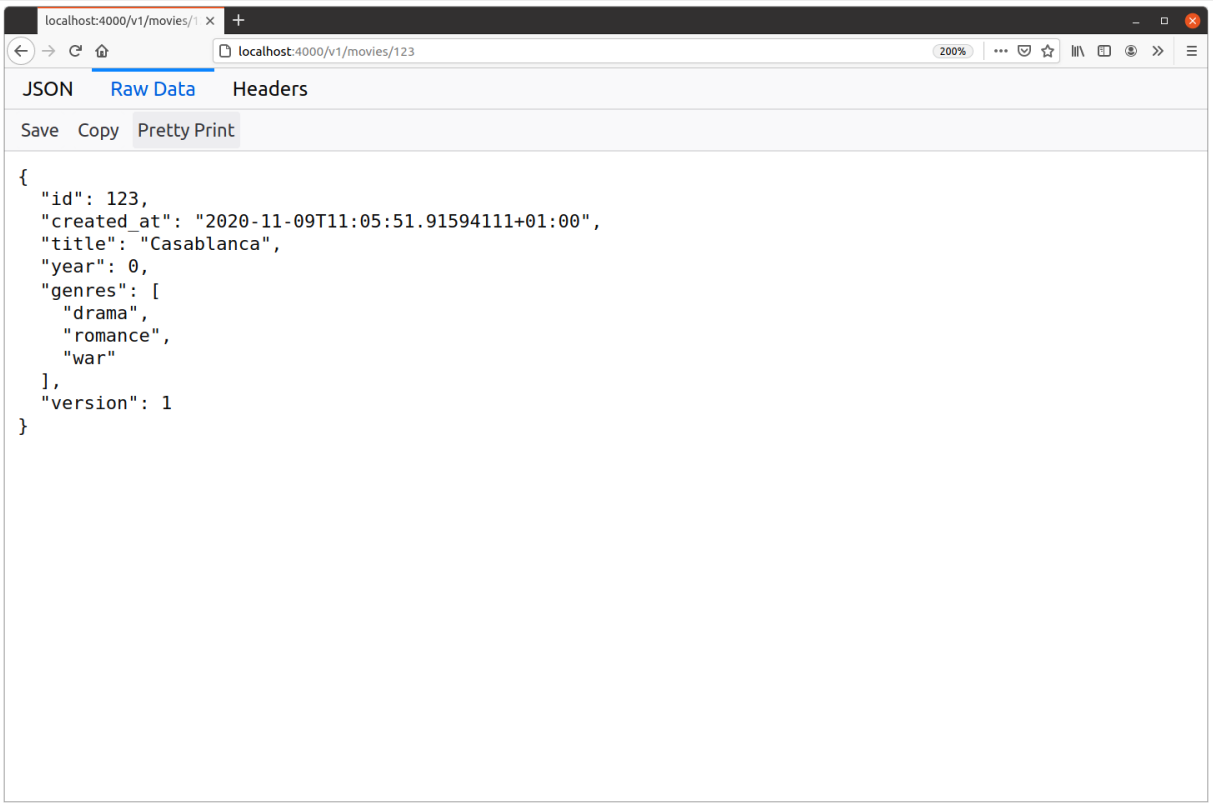
File: internal/models/movies.go

```
package models

...

// Annotate the Movie struct with struct tags to control how the keys appear in the
// JSON-encoded output.
type Movie struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"created_at"`
    Title     string    `json:"title"`
    Year      int       `json:"year"`
    Genres    []string  `json:"genres"`
    Version   int       `json:"version"`
}
```

And if you restart the server and visit `localhost:4000/v1/movies/123` again, you should now see a response with `snake_case` keys similar to this:



Hiding struct fields in the JSON object

It's also possible to control the visibility of individual struct fields in the JSON by using the `omitempty`, `omitzero` and `-` struct tag directives.

dego_vila@yahoo.com 30 Oct 2026

The `-` (hyphen) directive can be used when you *never* want a particular struct field to appear in the JSON output. This is useful for fields that contain internal system information that isn't relevant to your users, or sensitive information that you don't want to expose (like the hash of a password).

In out case, the `CreatedAt` field isn't relevant to our end users, so let's update the `Movie` struct and use the `-` directive to always hide this field.

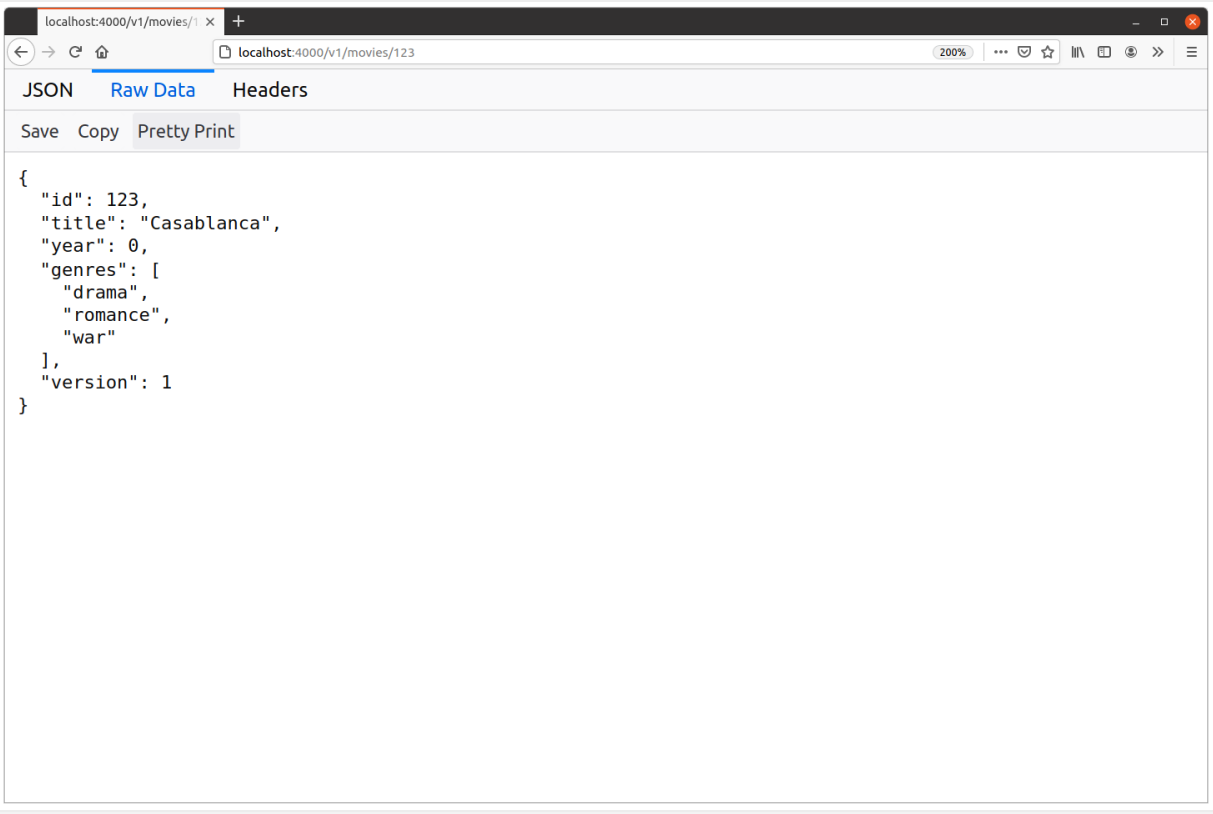
File: internal/models/movies.go

```
package models

...

type Movie struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"-"` // Use the - directive
    Title     string    `json:"title"`
    Year      int       `json:"year"`
    Genres    []string  `json:"genres"`
    Version   int       `json:"version"`
}
```

And if you restart the application and check `localhost:4000/v1/movies/123` now, you should see this field omitted from the JSON like so:



dego_vila@yahoo.com 30 Oct 2026

> **Note:** You can also prevent a struct field from appearing in the JSON output by simply making it unexported. But using the `json:"-"` struct tag is generally a better choice: it's an explicit indication to both Go and any future readers of your code that you don't want the field included in the JSON, and it helps prevent problems if someone changes the field to be exported in the future without realizing the consequences.

The `omitzero` and `omitempty` directives are a little more complicated, but really useful when you want to *hide fields that don't contain any meaningful data*.

When using `encoding/json/v2`:

- The `omitzero` directive hides a struct field in the JSON output *when its value is the zero value for its Go type*. So, for example, if you use `omitzero` on a struct field with the type `int`, it will omit the field from the JSON when the value is `0`.
- In contrast, the `omitempty` directive can be used to hide specific struct fields in the JSON output *when the struct field encodes to an empty JSON value*. In this case "empty" is defined as JSON `null`, or an empty JSON string (`""`), object (`{}`), or array (`[]`).

The following table summarizes how these two directives behave in practice for different Go types:

dego_vila@yahoo.com 30 Oct 2026

Go type	omitzero	omitempty
`bool`	Omits when `false`	Has no effect
`string`	Omits when `""`	Omits when `""`
integers, floating point numbers, `rune`	Omits when `0`	Has no effect
slices	Omits when slice is `nil`	Omits when slice is `nil` or is empty (i.e. has length of `0`)
maps	Omits when map is `nil`	Omits when map is `nil` or is empty (i.e. has length of `0`)
arrays	Omits when every element is the zero value	Omits when length is `0`
pointers	Omits when `nil`	Omits when `nil`
`time.Time`	Omits the `zero time`	Has no effect
structs	Omits when *`all`* fields in the struct (including unexported fields and those hidden with the `-` directive) have their zero value	Omits when the struct contains no JSON-marshalable fields

So, depending on the *type* of the struct field they are used on, sometimes `omitempty` and `omitzero` do the same thing, sometimes they do different things, and sometimes they have no effect and do nothing at all.

This means that when using `encoding/json/v2`, you need to think carefully about what you want to achieve and make sure to select the appropriate one for the job.

> **Note:** It's worth being aware that the behavior of `omitempty` in `encoding/json/v2` is quite different to the legacy `encoding/json` package, and the information above is correct for `encoding/json/v2` only. If you're upgrading code from the legacy package, I recommend reading the migrating to v2 documentation for more details of exactly what changed with `omitempty`.

To illustrate how to use both these directives, let's update our `Movie` struct so that:

- We use the `omitzero` directive on the `Year` field (which has the type `int`). This will hide

dego_vila@yahoo.com 30 Oct 2026

the field in the JSON whenever the value is `0`.

- We use the `omitempty` directive on the `Genres` field (which is a `[]string` slice). This will hide the field in the JSON whenever the slice is `nil` or it's empty and contains no elements.

Go ahead and update the struct tags like so:

File: internal/models/movies.go

```
package models

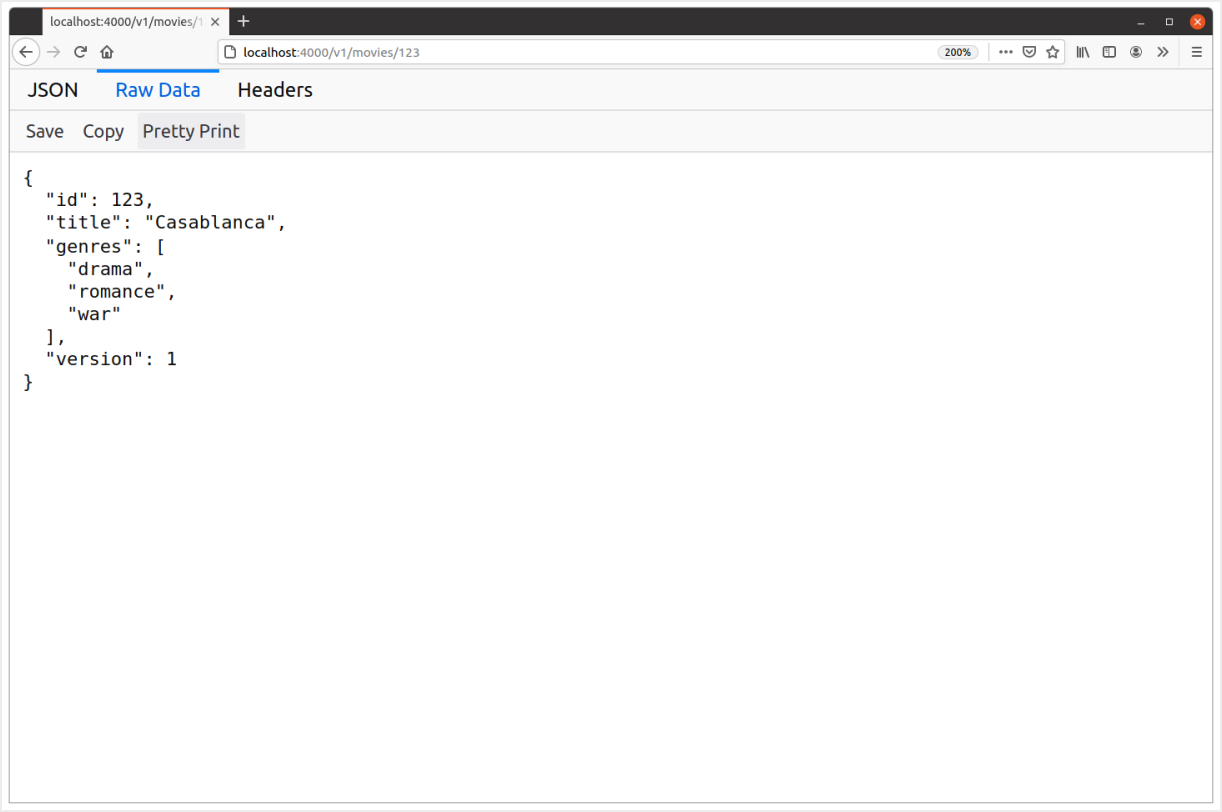
...

type Movie struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"-"`
    Title     string    `json:"title"`
    Year      int       `json:"year,omitzero"`    // Add the omitzero directive
    Genres    []string  `json:"genres,omitempty"` // Add the omitempty directive
    Version   int       `json:"version"`
}
```

> **Hint:** If you want to use `omitzero` or `omitempty` and *not* change the key name then you can leave it blank in the struct tag — like this: `json:",omitzero"`. Notice that the leading comma is still required.

Now when you restart the application and refresh your web browser, you should see a response which looks exactly like this:

dego_vila@yahoo.com 30 Oct 2026



We can see here that the `Year` field (which had the value `0`) doesn't appear either thanks to the `omitzero` directive. The `Genres` field that we used `omitempty` on has been unaffected because it isn't empty.

Feel free to play around with this if you like, setting different values in `showMovieHandler` until you're satisfied that the struct tags work as expected.

Additional information

The string directive

Another, less-frequently-used, struct tag directive is `string`. You can use this on individual struct fields to force Go number types to be encoded as a JSON string, instead of a JSON number.

For example, if we wanted the value of our `ID` field to be represented as a JSON string (instead of a number) we could use the `string` directive like this:

dego_vila@yahoo.com 30 Oct 2026

```
type Movie struct {
    ID        int       `json:"id,string"` // Add the string directive
    CreatedAt time.Time `json:"-"`
    Title     string    `json:"title"`
    Year      int       `json:"year,omitzero"`
    Genres    []string  `json:"genres,omitempty"`
    Version   int       `json:"version"`
}
```

And the resulting JSON output would look like this:

```
{
  "id": "123",            ← This is now a string
  "title": "Casablanca",
  "genres": [
    "romance",
    "drama",
    "war"
  ],
  "version": 1
}
```

Note that the `string` directive will only work on struct fields that encode to a JSON number. For any other type of struct field it will have no effect.

If you want to encode *all* numeric values to strings, rather than just specific struct fields, you can pass the `json.StringifyNumbers()` option when calling `json.Marshal()`. Like so:

```
js, err := json.Marshal(data, json.StringifyNumbers(true))
if err != nil {
    return err
}
```

dego_vila@yahoo.com 30 Oct 2026

Formatting and enveloping responses

So far in this book we've generally been making requests to our API using Firefox, which makes the JSON responses easy to read thanks to the 'pretty printing' provided by the built-in JSON viewer.

But if you try making some requests using curl, you'll see that the actual JSON response data is all on one line with no whitespace.

```
$ curl localhost:4000/v1/healthcheck
{"environment":"development","status":"available","version":"1.0.0"}

$ curl localhost:4000/v1/movies/123
{"id":123,"title":"Casablanca","genres":["romance","drama","war"],"version":1}
```

Adding whitespace with the jsontext package

We can make these easier to read in terminals by using the `jsontext.Multiline` option when encoding our response data. This will add whitespace to the JSON output, putting each element on a separate line and indenting it appropriately using the horizontal tab (`\t`) character, and adding a single whitespace after colon (`:`) characters.

It's important to note that the `jsontext.Multiline` option is defined in the `encoding/json/jsontext` package — *not* in the `encoding/json/v2` package like the previous options we've used. But it's still valid to use it when calling `json.Marshal()`.

At first, you might find it confusing that the possible options for `json.Marshal()` are defined in different places in the standard library, but I'll try to explain why.

At a very high level, the `encoding/json/jsontext` package contains things related to JSON *syntax processing*. In contrast, the `encoding/json/v2` package contains things related to *semantic processing*. Because of this delineation, options that relate to the *format* of JSON — like adding whitespace — live in `encoding/json/jsontext`, whereas options that relate to *how values are encoded and decoded* live in `encoding/json/v2`.

Let's update our `writeJSON()` helper to use `jsontext.Multiline` like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

import (
    "encoding/json/jsontext" // New import
    "encoding/json/v2"
    "errors"
    "net/http"
    "strconv"

    "github.com/julienschmidt/httprouter"
)

...

func (app *application) writeJSON(w http.ResponseWriter, status int, data any, headers http.Header) error {
    // For clarity and easy reading, let's create a json.Options slice containing
    // the options that we want to use when encoding. This will grow as we progress
    // through the application build, but for now we use the json.Deterministic and
    // jsontext.Multiline options.
    opts := []json.Options{
        json.Deterministic(true),
        jsontext.Multiline(true),
    }

    // Pass the options to json.Marshal().
    js, err := json.Marshal(data, opts...)
    if err != nil {
        return err
    }

    js = append(js, '\n')

    for key, values := range headers {
        for _, value := range values {
            w.Header().Add(key, value)
        }
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    w.Write(js)

    return nil
}
```

If you restart the API and try making the same requests from your terminal again, you will now receive some nicely whitespaced JSON responses similar to these:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl localhost:4000/v1/healthcheck
{
        "environment": "development",
        "status": "available",
        "version": "1.0.0"
}

$ curl localhost:4000/v1/movies/123
{
        "id": 123,
        "title": "Casablanca",
        "genres": [
                "romance",
                "drama",
                "war"
        ],
        "version": 1
}
```

> **Note:** As an aside, if you want to keep the JSON all on one line, but add whitespace after colon (`:`) or comma (`,`) characters, you can do so by using the `jsontext.SpaceAfterColon` and `jsontext.SpaceAfterComma` options instead of `jsontext.Multiline`.
>
> Or if you want multiline output, but wish to use a different indent character instead of a horizontal tab, you can use the `jsontext.WithIndent` option.

Enveloping responses

Next let's work on updating our responses so that the JSON data is always *enveloped* in an outer JSON object. Similar to this:

```
{
    "movie": {
        "id": 123,
        "title": "Casablanca",
        "genres": [
            "romance",
            "drama",
            "war"
        ],
        "version":1
    }
}
```

Notice how the movie data is nested under the key `"movie"` here, rather than being the top-level JSON object itself?

dego_vila@yahoo.com 30 Oct 2026

Enveloping response data like this isn't strictly necessary, and whether you choose to do so is partly a matter of style and taste. But there are a few tangible benefits:

1. Including a key name (like `"movie"`) at the top-level of the JSON helps make the response more self-documenting. For any humans who see the response out of context, it is a bit easier to understand what the data relates to.

2. It reduces the risk of errors on the client side, because it's harder to accidentally process one response thinking that it is something different. To get at the data, a client must explicitly reference it via the `"movie"` key.

3. If we always envelope the data returned by our API, then we mitigate a security vulnerability in older browsers which can arise if you return a JSON array as a response.

There are a couple of techniques that we could use to envelope our API responses, but we're going to keep things simple and do it by creating a custom `envelope` type with the underlying type `map[string]any`.

I'll demonstrate.

Let's start by updating the `cmd/api/helpers.go` file as follows:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

// Define an envelope type.
type envelope map[string]any

// Change the data parameter to have the type envelope instead of any.
func (app *application) writeJSON(w http.ResponseWriter, status int, data envelope, headers http.Header) error {
    opts := []json.Options{
        json.Deterministic(true),
        jsontext.Multiline(true),
    }

    js, err := json.Marshal(data, opts...)
    if err != nil {
        return err
    }

    js = append(js, '\n')

    for key, values := range headers {
        for _, value := range values {
            w.Header().Add(key, value)
        }
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    w.Write(js)

    return nil
}
```

Then we need to update our `showMovieHandler` to create an instance of the `envelope` map containing the movie data, and pass this onwards to our `writeJSON()` helper instead of passing the movie data directly.

Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        http.NotFound(w, r)
        return
    }

    movie := models.Movie{
        ID:        id,
        CreatedAt: time.Now(),
        Title:     "Casablanca",
        Genres:    []string{"romance", "drama", "war"},
        Version:   1,
    }

    // Create an envelope{"movie": movie} instance and pass it to writeJSON(), instead
    // of passing the plain movie struct.
    err = app.writeJSON(w, http.StatusOK, envelope{"movie": movie}, nil)
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
    }
}
```

We also need to update the code in our `healthcheckHandler` so that it passes an `envelope` type to the `writeJSON()` helper too:

File: cmd/api/healthcheck.go

```
package main

...

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    // Declare an envelope map containing the data for the response. Notice that the way
    // we've constructed this means the environment and version data will now be nested
    // under a system_info key in the JSON response.
    data := envelope{
        "status": "available",
        "system_info": map[string]string{
            "environment": app.config.env,
            "version":     version,
        },
    }

    err := app.writeJSON(w, http.StatusOK, data, nil)
    if err != nil {
        app.logger.Error(err.Error())
        http.Error(w, "The server encountered a problem and could not process your request", http.StatusInternalServerError)
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

Alright, let's try these changes out. Go ahead and restart the server, then use `curl` to make some requests to the API endpoints again. You should now get responses formatted like the ones below.

```
$ curl localhost:4000/v1/movies/123
{
        "movie": {
                "id": 123,
                "title": "Casablanca",
                "genres": [
                        "romance",
                        "drama",
                        "war"
                ],
                "version":1
        }
}

$ curl localhost:4000/v1/healthcheck
{
        "status": "available",
        "system_info": {
                "environment": "development",
                "version": "1.0.0"
        }
}
```

Additional information

Response structure

It's important to emphasize that there's no single right or wrong way to structure your JSON responses. There are some popular formats like JSON:API and jsend that you might like to follow or use for inspiration, but it's certainly not necessary and most APIs *don't* follow these formats.

But whatever you do, it *is* valuable to think about formatting upfront and to maintain a clear and consistent response structure across your different API endpoints — especially if they are intended for public use.

JSON Canonicalization

If you want, the Go standard library also supports formatting JSON so that it complies with the JSON Canonicalization Scheme (JCS).

The JCS outlines specific formatting (or *canonicalization*) rules for JSON, so that *logically*

dego_vila@yahoo.com 30 Oct 2026

equivalent JSON values always produce the exact same byte sequence. At a very high level, the canonicalization involves ordering key-value pairs in JSON objects based on the key name, serializing strings and number values in a consistent way, and removing all insignificant whitespace.

Canonicalizing JSON can be useful in a couple of situations. The most common situation is probably when hashing JSON — in some cases you might want to canonicalize the JSON before calculating the hash, so that *logically equivalent* JSON always produces the same hash output. Another use-case might be when you have multiple systems and want to ensure that they both produce data in exactly the same JSON format — then the JCS provides a clear common standard that all systems can follow.

Now, we *won't* be using the JCS in this book, so don't make any changes if you're following along. But if you do want to use it in the future, the functionality is available in the `encoding/json/jsontext` package.

To format your JSON to be JCS-compliant, you first need to take your *already-encoded* JSON and convert it to a `jsontext.Value` type, and then call the `Canonicalize()` method on it.

For example, if you wanted to update the `writeJSON()` helper to always produce JCS-compliant JSON, you could do so like this:

```
func (app *application) writeJSON(w http.ResponseWriter, status int, data any, headers http.Header) error {
    js, err := json.Marshal(data)
    if err != nil {
        return err
    }

    // Convert the JSON into a jsontext.Value type.
    jsv := jsontext.Value(js)

    // Call the Canonicalize() method on it.
    err = jsv.Canonicalize()
    if err != nil {
        return err
    }

    for key, values := range headers {
        for _, value := range values {
            w.Header().Add(key, value)
        }
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)

    // Write the jsontext.Value instead of the original JSON.
    w.Write(jsv)

    return nil
}
```

dego_vila@yahoo.com 30 Oct 2026

If you did this, the JCS-compliant JSON for a movie record would look similar to this:

```
{"movie":{"genres":["romance","drama","war"],"id":123,"title":"Casablanca","version":1}}
```

It's worth noting that there's no point in using the `json.Deterministic`, `jsontext.Multiline`, `jsontext.SpaceAfterColon`, `jsontext.SpaceAfterComma` or `jsontext.WithIndent` options if you are canonicalizing the JSON afterward, because the things they do will just be overridden during canonicalization.

dego_vila@yahoo.com 30 Oct 2026

Chapter 3.5.

Custom JSON encoding

By using struct tags, adding whitespace and enveloping the data, we've been able to add quite a lot of customization to our JSON responses already. But what happens when these things aren't enough, and you need the freedom to customize your JSON even more?

To help demonstrate the answer to this, let's add a new `Runtime` field to our `Movie` struct like so:

File: internal/models/movies.go

```
package models

import (
    "time"
)

// Add a new Runtime field with the type time.Duration.
type Movie struct {
    ID        int           `json:"id"`
    CreatedAt time.Time     `json:"-"`
    Title     string        `json:"title"`
    Year      int           `json:"year,omitzero"`
    Runtime   time.Duration `json:"runtime"`
    Genres    []string      `json:"genres,omitempty"`
    Version   int           `json:"version"`
}
```

And for demo purposes, let's add a runtime value to the dummy data in `showMovieHandler` too:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
   ...

    // Add a Runtime value of 1 hour 42 minutes to the dummy data.
    movie := models.Movie{
        ID:        id,
        CreatedAt: time.Now(),
        Title:     "Casablanca",
        Runtime:   time.Hour + 42*time.Minute,
        Genres:    []string{"romance", "drama", "war"},
        Version:   1,
    }

   ...
}
```

The important thing to notice here is that we've made this new field a `time.Duration` type.

As we mentioned earlier in the book, `time.Duration` has no default JSON representation, and isn't supported by `json.Marshal()` out of the box. So if you restart the application and make a request to the `GET /v1/movies/:id` endpoint again now, you should see an error response like this…

```
$ curl -i localhost:4000/v1/movies/123
HTTP/1.1 500 Internal Server Error
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Thu, 10 Sep 2026 07:58:20 GMT
Content-Length: 68

The server encountered a problem and could not process your request
```

And you should also see an `ERROR` level log message which looks similar to this:

```
time=2026-09-10T09:58:20.868+02:00 level=ERROR msg="json: cannot marshal from Go
    time.Duration within \"/movie/runtime\": no default representation"
```

To prevent this error, we need to create our own *custom marshaler* which tells Go how to encode a `time.Duration` type to a JSON value.

Creating and using a custom marshaler

There are a couple of ways to create a custom marshaler, but the way that I'd recommend

dego_vila@yahoo.com 30 Oct 2026

in most cases is by using the `json.MarshalToFunc()` function.

At first glance, the signature for this function looks a bit hairy:

```
func MarshalToFunc[T any](fn func(*jsontext.Encoder, T) error) *Marshalers
```

But when we break it down, it's not so complicated. Effectively, you use `json.MarshalToFunc()` to define a custom function `fn`, which writes the JSON representation of type `T` to an encoder. So, for example, if we wanted to represent a `time.Duration` value as a JSON number in whole minutes, we could use `json.MarshalToFunc()` to create a marshaler like so:

```
durationMarshaler := json.MarshalToFunc(
    // Notice that the val parameter below has the type time.Duration.
    func(enc *jsontext.Encoder, val time.Duration) error {
        // Calculate the number of minutes in the time.Duration, rounded to the
        // nearest full minute.
        mins := math.Round(val.Minutes())

        // Use the jsontext.Int() function to create an integer JSON token
        // containing the number of minutes.
        token := jsontext.Int(int64(mins))

        // Write the token to the encoder.
        return enc.WriteToken(token)
    },
)
```

Or, to give another example, we could create a custom marshaler that represents a `time.Duration` value as a JSON string in the format `"{minutes}m"` — such as `"102m"` or `"76m"`. The code to do that would look like this:

```
durationMarshaler := json.MarshalToFunc(
    func(enc *jsontext.Encoder, val time.Duration) error {
        mins := math.Round(val.Minutes())

        // This time use the jsontext.String() function to create a JSON string
        // token containing the number of minutes followed by an "m".
        token := jsontext.String(fmt.Sprintf("%dm", int(mins)))

        return enc.WriteToken(token)
    },
)
```

dego_vila@yahoo.com 30 Oct 2026

> **Note:** In the examples above, we've used the `jsontext.Int()` and `jsontext.String()` functions to create JSON tokens representing an integer JSON number and a JSON string respectively. But there are also `jsontext.Bool()`, `jsontext.Float()` and `jsontext.Uint()` functions that you can use to create other types of JSON token.
>
> The `encoding/json/jsontext` package also includes some "premade" tokens that you can pass directly to `enc.WriteToken()`. These premade tokens are documented here, but they include `jsontext.Null` (which represents a JSON `null` value) and `jsontext.True` and `jsontext.False` (representing JSON `true` and `false` values). There are also the premade tokens `jsontext.BeginObject`, `jsontext.EndObject`, `jsontext.BeginArray`, `jsontext.EndArray` that you can use to build up JSON objects and arrays. We'll look at an example of using these at the end of this chapter.

To actually use a custom marshaler, you need to include the `json.WithMarshalers` option when calling `json.Marshal()`.

Let's go ahead and implement this in our project now, creating and using a custom marshaler so that `time.Duration` values are encoded as a JSON string in the format `"{minutes}m"`. Open up the `writeJSON()` helper function and update it as follows:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "errors"
    "fmt"  // New import
    "math" // New import
    "net/http"
    "strconv"
    "time" // New import

    "github.com/julienschmidt/httprouter"
)

...

func (app *application) writeJSON(w http.ResponseWriter, status int, data envelope, headers http.Header) error {
    // Define a custom marshaler for the time.Duration type.
    durationMarshaler := json.MarshalToFunc(
        func(enc *jsontext.Encoder, val time.Duration) error {
            mins := math.Round(val.Minutes())
            token := jsontext.String(fmt.Sprintf("%dm", int(mins)))
            return enc.WriteToken(token)
        },
    )

    // Include a new json.WithMarshalers() option, passing in our custom duration
    // marshaler.
    opts := []json.Options{
        json.Deterministic(true),
        jsontext.Multiline(true),
        json.WithMarshalers(durationMarshaler),
    }

    js, err := json.Marshal(data, opts...)
    if err != nil {
        return err
    }

    js = append(js, '\n')

    for key, values := range headers {
        for _, value := range values {
            w.Header().Add(key, value)
        }
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    w.Write(js)

    return nil
}
```

Let's try this out by restarting the API and making a request to the `GET /v1/movies/:id` endpoint. You should now see a response containing the custom runtime value in the format `"{minutes}m"`, like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl localhost:4000/v1/movies/123
{
    "movie": {
        "id": 123,
        "title": "Casablanca",
        "runtime": "102m",
        "genres": [
            "romance",
            "drama",
            "war"
        ],
        "version": 1
    }
}
```

Using a custom type

So that works, but you might be starting to see a potential problem — our `durationMarshaler` logic will be applied to *all* `time.Duration` values that go through our `writeJSON()` helper.

What if we have another `time.Duration` field and representing it as whole minutes doesn't make sense? For example, suppose we want to include average shot length in our `Movie` struct too. Average shot length is normally a few seconds, so it would make much more sense to represent this value in the JSON output as a fractional number of seconds.

To deal with this, a nice approach would be to make our movie runtime value a custom `Runtime` type with the underlying type `time.Duration`. We could then create a custom marshaler for the `Runtime` type, and it would only be used when encoding this type specifically.

Let's go ahead and make this change.

First, create a new file in our `internal/models` package to hold the logic related to this:

```
$ touch internal/models/runtime.go
```

And then go ahead and create the `Runtime` type along with a custom marshaler for it, like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/runtime.go

```
package models

import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "fmt"
    "math"
    "time"
)

// Define a custom Runtime type with the underlying type time.Duration.
type Runtime time.Duration

// Define a custom marshaler for the Runtime type. This code here is basically the
// same as before, except it is acting on a Runtime type instead of a time.Duration.
var RuntimeMarshaler = json.MarshalToFunc(
    func(enc *jsontext.Encoder, val Runtime) error {
        // Note that we need to convert the val parameter back to its underlying
        // type so that we can call the Minutes() method on it.
        mins := math.Round(time.Duration(val).Minutes())
        token := jsontext.String(fmt.Sprintf("%dm", int(mins)))
        return enc.WriteToken(token)
    },
)
```

Then let's update our `Movie` struct definition to use the new `Runtime` type instead of a `time.Duration`:

File: internal/models/movies.go

```
package models

import (
    "time"
)

 // Update the Runtime field to have the type Runtime, instead of time.Duration.
type Movie struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"-"`
    Title     string    `json:"title"`
    Year      int       `json:"year,omitzero"`
    Runtime   Runtime   `json:"runtime"`
    Genres    []string  `json:"genres,omitempty"`
    Version   int       `json:"version"`
}
```

Next we also need to update `showMovieHandler` so that our dummy data initializes and sets a `Runtime` value in the `Movie` struct, instead of a `time.Duration`. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    ...

    // Convert the 1 hour 42 minute time.Duration to a Runtime type before setting
    // it.
    movie := models.Movie{
        ID:        id,
        CreatedAt: time.Now(),
        Title:     "Casablanca",
        Runtime:   models.Runtime(time.Hour + 42*time.Minute),
        Genres:    []string{"romance", "drama", "war"},
        Version:   1,
    }

   ...
}
```

And to finish this off, we should also update the `writeJSON()` function — removing the `durationMarshaler` code, and passing a `json.WithMarshalers(models.RuntimeMarshaler)` option to `json.Marshal()` instead of `json.WithMarshalers(durationMarshaler)`.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "errors"
    "net/http"
    "strconv"

    "greenlight.alexedwards.net/internal/models" // New import

    "github.com/julienschmidt/httprouter"
)

...

func (app *application) writeJSON(w http.ResponseWriter, status int, data envelope, headers http.Header) error {
    // Update the options slice to use the models.RuntimeMarshaler that we just
    // created.
    opts := []json.Options{
        json.Deterministic(true),
        jsontext.Multiline(true),
        json.WithMarshalers(models.RuntimeMarshaler),
    }

    js, err := json.Marshal(data, opts...)
    if err != nil {
        return err
    }

    js = append(js, '\n')

    for key, values := range headers {
        for _, value := range values {
            w.Header().Add(key, value)
        }
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    w.Write(js)

    return nil
}
```

So this is pretty neat. We're now telling `writeJSON()` to use the custom marshaler that we made specifically for `Runtime` values — and we're free to generate custom types and corresponding custom marshalers for any other `time.Duration` values that we want to encode differently.

If you try it out, you should still get a response with the movie runtime value in the format `"{minutes}m"`, like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl localhost:4000/v1/movies/123
{
    "movie": {
        "id": 123,
        "title": "Casablanca",
        "runtime": "102m",
        "genres": [
            "romance",
            "drama",
            "war"
        ],
        "version": 1
    }
}
```

But like with most good things, there's also a downside that comes with this pattern.

Using custom types in your codebase can often make things a bit more awkward when integrating your code with other packages — you may need to perform type conversions to change your custom type to and from a value that the other packages understand and accept. So you should lean towards being conservative here, and only create custom types *when you have a genuine need to do so*, rather than as something you do by default.

Omitzero and Omitempty

It's worth pointing out that the `omitzero` and `omitempty` directives still work like you would expect when using a custom marshaler.

If you use the `omitzero` directive and the value being encoded is the zero value for the type, then Go will omit it from the JSON and *your custom marshaler won't be called at all*.

Similarly, if you use the `omitempty` directive and your custom marshaler emits an empty JSON value, then it will be omitted. Remember, "empty" in this case is defined as JSON `null`, or an empty JSON string (`""`), object (`{}`), or array (`[]`).

To demonstrate this, let's quickly remove the `Runtime` value from our dummy data in `showMovieHandler`, so that the handler looks like this:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    ...

    // Don't set a value for the Runtime field.
    movie := models.Movie{
        ID:        id,
        CreatedAt: time.Now(),
        Title:     "Casablanca",
        Genres:    []string{"romance", "drama", "war"},
        Version:   1,
    }

    ...
}
```

When you make a request to this endpoint now, you should see the JSON value `"0m"` in the response, like this:

```
$ curl localhost:4000/v1/movies/123
{
    "movie": {
        "id": 123,
        "title": "Casablanca",
        "runtime": "0m",
        "genres": [
            "romance",
            "drama",
            "war"
        ],
        "version": 1
    }
}
```

This isn't very neat — it would be more consistent if we omitted this in the same way that we omit `"year"` and `"genres"` from the JSON when they don't contain any meaningful data.

So let's go ahead and update our `Movie` struct definition to use the `omitzero` directive on the `Runtime` field.

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

import (
    "time"
)

type Movie struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"-"`
    Title     string    `json:"title"`
    Year      int       `json:"year,omitzero"`
    Runtime   Runtime   `json:"runtime,omitzero"` // Update this to use omitzero
    Genres    []string  `json:"genres,omitempty"`
    Version   int       `json:"version"`
}
```

And if you restart the application and try it again, you'll see that the `"runtime"` key-value pair now no longer appears in the JSON response.

```
$ curl localhost:4000/v1/movies/123
{
    "movie": {
        "id": 123,
        "title": "Casablanca",
        "genres": [
            "romance",
            "drama",
            "war"
        ],
        "version": 1
    }
}
```

Additional information

Some more examples

We started this chapter by demonstrating how to create a custom marshaler for `time.Duration` types, because `time.Duration` doesn't have a default JSON representation and isn't supported by Go out-of-the-box.

But the patterns we've used in this chapter are completely generalizable and can be used to create custom marshalers for *any* Go type — including types that *do* have a default representation.

For example, in our Go code, the `ID` field in our `Movie` struct has the type `int`, which by

dego_vila@yahoo.com 30 Oct 2026

default encodes to a JSON number. But if you wanted to represent the movie ID as a hash-prefixed string like `"#123"` instead, you could change this field to be a custom `ID` type and create a marshaler for it like so:

```
type ID int

var IDMarshaler = json.MarshalToFunc(
    func(enc *jsontext.Encoder, val ID) error {
        token := jsontext.String(fmt.Sprintf("#%d", val))
        return enc.WriteToken(token)
    },
)
```

Or, thinking bigger, you could create a marshaler that controls how the whole `Movie` struct is encoded. By default, our `Movie` struct is encoded as a JSON object, with the struct fields represented as key-value pairs inside it.

But if — for example — you want to do something a bit wild and encode it as a JSON array instead, containing the movie ID, title and year like this…

```
[123, "Casablanca", 1942]
```

Then you could write a custom marshaler for the `Movie` type like so:

```
var MovieMarshaler = json.MarshalToFunc(
    func(enc *jsontext.Encoder, val Movie) error {
        tokens := []jsontext.Token{
            jsontext.BeginArray,
            jsontext.Int(int64(val.ID)),
            jsontext.String(val.Title),
            jsontext.Int(int64(val.Year)),
            jsontext.EndArray,
        }
        for _, t := range tokens {
            err := enc.WriteToken(t)
            if err != nil {
                return err
            }
        }
        return nil
    },
)
```

This is a nice example, because it demonstrates that your custom marshaler can write *multiple tokens*. In this case, we write the `jsontext.BeginArray` token to start a JSON array, then write the tokens representing the array values, then write the `jsontext.EndArray` token to close the array.

dego_vila@yahoo.com 30 Oct 2026

If you'd like to try this out, I've made a stripped-down example available on the Go playground.

The way that custom marshaling is supported in `encoding/json/v2` means that there is a lot of flexibility. With the techniques we've described in this chapter and a bit of thought, you should be able to do pretty much anything you need in terms of customizing JSON output.

The MarshalFunc alternative

In this chapter we've been using the `json.MarshalToFunc()` function to create custom marshalers. But there's an alternative function — `json.MarshalFunc()` — that you could also use.

This works in a similar way, but instead of writing a JSON token to an encoder, you return a `[]byte` slice containing the JSON value.

In theory you could rewrite the custom marshaler for our `Runtime` type so that it uses `json.MarshalFunc()` like so:

```
var RuntimeMarshaler = json.MarshalFunc(
    func(val Runtime) ([]byte, error) {
        // Calculate the number of minutes in the time.Duration, rounded to the
        // nearest full minute.
        mins := math.Round(time.Duration(val).Minutes())

        // Generate a string containing the movie runtime in the required format.
        jsonValue := fmt.Sprintf("%dm", int(mins))

        // Use the strconv.Quote() function on the string to wrap it in double quotes. It
        // needs to be surrounded by double quotes in order to be a valid *JSON string*.
        quotedJSONValue := strconv.Quote(jsonValue)

        // Convert the quoted string value to a byte slice and return it.
        return []byte(quotedJSONValue), nil
    },
)
```

Although this works, I think it's generally cleaner, clearer and safer to use `json.MarshalToFunc()` and the nice explicit helper functions like `jsontext.String()` and `jsontext.Int()` to create and write JSON tokens with a specific type.

Using multiple custom marshalers

If you have more than one custom marshaler that you want to use, you need to use the `json.JoinMarshalers()` function to join the marshalers before passing them as an option to `json.Marshal()`.

dego_vila@yahoo.com 30 Oct 2026

For example, if you have `fooMarshaler`, `barMarshaler` and `bazMarshaler`, and want to use all three, you would need to do something like this:

```
marshalers := json.JoinMarshalers(fooMarshaler, barMarshaler, bazMarshaler)

opts := []json.Options{
    json.Deterministic(true),
    jsontext.Multiline(true),
    json.WithMarshalers(marshalers),
}

js, err := json.Marshal(data, opts...)
if err != nil {
    return err
}
```

Perhaps surprisingly, you shouldn't pass more than one `json.WithMarshalers` option when calling `json.Marshal()`. For example, don't do something like this:

```
opts := []json.Options{
    json.Deterministic(true),
    jsontext.Multiline(true),
    json.WithMarshalers(fooMarshaler),
    json.WithMarshalers(barMarshaler),
    json.WithMarshalers(bazMarshaler),
}

js, err := json.Marshal(data, opts...)
if err != nil {
    return err
}
```

When you pass multiple `json.WithMarshalers` options in this way, *only the final one* will be used. In this example, `bazMarshaler` would be used, but `fooMarshaler` and `barMarshaler` will be ignored.

If you need to use multiple marshalers, join them with `json.JoinMarshalers()` first, and pass only one `json.WithMarshalers` option to `json.Marshal()`.

dego_vila@yahoo.com 30 Oct 2026

Chapter 3.6.

Sending error messages

At this point our API is sending well-formed JSON responses for successful requests, but if a client makes a bad request — or something goes wrong in our application — we're still sending them a plain-text error message using the `http.Error()` and `http.NotFound()` functions.

In this chapter we'll fix that by creating some additional helpers to manage errors and send the appropriate JSON responses to our clients.

If you're following along, go ahead and create a new `cmd/api/errors.go` file:

```
$ touch cmd/api/errors.go
```

And then add some helper methods like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/errors.go

```
package main

import (
    "fmt"
    "net/http"
)

// The logError() method is a helper for logging an error message, along
// with the current request method and URL as attributes in the log entry.
func (app *application) logError(r *http.Request, err error) {
    var (
        method = r.Method
        uri    = r.URL.RequestURI()
    )

    app.logger.Error(err.Error(), "method", method, "uri", uri)
}

// The errorResponse() method is a helper for sending JSON-formatted error
// messages to the client with a given status code. Note that we're using the any
// type for the message parameter, rather than just a string type, as this gives us
// more flexibility over the values that we can include in the response.
func (app *application) errorResponse(w http.ResponseWriter, r *http.Request, status int, message any) {
    env := envelope{"error": message}

    // Write the response using the writeJSON() helper. If this happens to return an
    // error then log it, and fall back to sending the client an empty response with a
    // 500 Internal Server Error status code.
    err := app.writeJSON(w, status, env, nil)
    if err != nil {
        app.logError(r, err)
        w.WriteHeader(http.StatusInternalServerError)
    }
}

// The serverErrorResponse() method will be used when our application encounters an
// unexpected problem at runtime. It logs the detailed error message, then uses the
// errorResponse() helper to send a 500 Internal Server Error status code and JSON
// response (containing a generic error message) to the client.
func (app *application) serverErrorResponse(w http.ResponseWriter, r *http.Request, err error) {
    app.logError(r, err)

    message := "the server encountered a problem and could not process your request"
    app.errorResponse(w, r, http.StatusInternalServerError, message)
}

// The notFoundResponse() method will be used to send a 404 Not Found status code and
// JSON response to the client.
func (app *application) notFoundResponse(w http.ResponseWriter, r *http.Request) {
    message := "the requested resource could not be found"
    app.errorResponse(w, r, http.StatusNotFound, message)
}

// The methodNotAllowedResponse() method will be used to send a 405 Method Not Allowed
// status code and JSON response to the client.
func (app *application) methodNotAllowedResponse(w http.ResponseWriter, r *http.Request) {
    message := fmt.Sprintf("the %s method is not supported for this resource", r.Method)
    app.errorResponse(w, r, http.StatusMethodNotAllowed, message)
}
```

Now that those are in place, let’s update our API handlers to use these new helpers instead

dego_vila@yahoo.com 30 Oct 2026

of the `http.Error()` and `http.NotFound()` functions. Like so:

File: cmd/api/healthcheck.go

```
package main

...

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    env := envelope{
        "status": "available",
        "system_info": map[string]string{
            "environment": app.config.env,
            "version":     version,
        },
    }

    err := app.writeJSON(w, http.StatusOK, env, nil)
    if err != nil {
        // Use the new serverErrorResponse() helper.
        app.serverErrorResponse(w, r, err)
    }
}
```

File: cmd/api/movies.go

```
package main

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        // Use the new notFoundResponse() helper.
        app.notFoundResponse(w, r)
        return
    }

    movie := models.Movie{
        ID:        id,
        CreatedAt: time.Now(),
        Title:     "Casablanca",
        Genres:    []string{"romance", "drama", "war"},
        Version:   1,
    }

    err = app.writeJSON(w, http.StatusOK, envelope{"movie": movie}, nil)
    if err != nil {
        // Use the new serverErrorResponse() helper.
        app.serverErrorResponse(w, r, err)
    }
}
```

Routing errors

Any error messages that our own API handlers send will now be well-formed JSON

dego_vila@yahoo.com 30 Oct 2026

responses. Which is great!

But what about the error messages that `httprouter` automatically sends when it can't find a matching route? By default, these will still be the same plain-text (non-JSON) responses that we saw earlier in the book.

Fortunately, `httprouter` allows us to set our own custom error handlers. These custom handlers must satisfy the `http.Handler` interface, which is good news for us because it means we can easily reuse the `notFoundResponse()` and `methodNotAllowedResponse()` helpers that we just made.

Open up the `cmd/api/routes.go` file and configure the `httprouter` instance like so:

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    // Convert the notFoundResponse() helper to a http.Handler using the
    // http.HandlerFunc() adapter, and then set it as the custom error handler for 404
    // Not Found responses.
    router.NotFound = http.HandlerFunc(app.notFoundResponse)

    // Likewise, convert the methodNotAllowedResponse() helper to a http.Handler and set
    // it as the custom error handler for 405 Method Not Allowed responses.
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)

    return router
}
```

Let's test out these changes. Restart the application, then try making some requests for endpoints that don't exist, or that use an unsupported HTTP method. You should now get some nice JSON error responses which look similar to these:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i localhost:4000/foo
HTTP/1.1 404 Not Found
Content-Type: application/json
Date: Thu, 10 Sep 2026 08:16:55 GMT
Content-Length: 58

{
    "error": "the requested resource could not be found"
}

$ curl -i localhost:4000/v1/movies/abc
HTTP/1.1 404 Not Found
Content-Type: application/json
Date: Thu, 10 Sep 2026 08:17:27 GMT
Content-Length: 58

{
    "error": "the requested resource could not be found"
}

$ curl -i -X PUT localhost:4000/v1/healthcheck
HTTP/1.1 405 Method Not Allowed
Allow: GET, OPTIONS
Content-Type: application/json
Date: Thu, 10 Sep 2026 08:17:41 GMT
Content-Length: 66

{
    "error": "the PUT method is not supported for this resource"
}
```

In this final example, notice that `httprouter` still automatically sets the correct `Allow` header for us, even though it is now using our custom error handler for the response.

Panic recovery

Lastly, let's consider what will happen if there is a runtime panic in our handler code.

1. Normal execution of the code in the handler will immediately stop.
2. Any deferred functions for the current goroutine will be run in reverse (last-in, first-out) order.
3. The panic will then be recovered by Go's `http.Server`, which will close the underlying HTTP connection.
4. An error message and stack trace will be output to `http.Server.ErrorLog`.
5. No other HTTP requests will be affected by the panic.

This behavior is *okay*, but it would be better for the client if we could also send a `500 Internal Server Error` response to explain that something has gone wrong — rather than just closing the HTTP connection without providing any explanation.

dego_vila@yahoo.com 30 Oct 2026

In *Let's Go* we talked through the details of how to do this by creating some middleware to recover the panic ourselves, and it makes sense to do the same thing again here.

If you're following along, go ahead and create a `cmd/api/middleware.go` file:

```
$ touch cmd/api/middleware.go
```

And inside that file add a new `recoverPanic()` middleware:

File: cmd/api/middleware.go

```
package main

import (
    "fmt"
    "net/http"
)

func (app *application) recoverPanic(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Create a deferred function (which will always be run in the event
        // of a panic).
        defer func() {
            // Use the built-in recover() function to check if a panic occurred.
            // If a panic did happen, recover() will return the panic value. If
            // a panic didn't happen, it will return nil.
            pv := recover()
            if pv != nil {
                // If there was a panic, set a "Connection: close" header on the
                // response. This acts as a trigger to make Go's HTTP server
                // automatically close the current connection after the response has been
                // sent.
                w.Header().Set("Connection", "close")
                // The value returned by recover() has the type any, so we use
                // fmt.Errorf() with the %v verb to coerce it into an error and
                // call our serverErrorResponse() helper. In turn, this will log the
                // error at the ERROR level and send the client a 500 Internal
                // Server Error response.
                app.serverErrorResponse(w, r, fmt.Errorf("%v", pv))
            }
        }()

        next.ServeHTTP(w, r)
    })
}
```

Once that's done, we need to update our `cmd/api/routes.go` file so that the `recoverPanic()` middleware wraps our router. This will ensure that the middleware runs for all our API endpoints.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)

    // Wrap the router with the panic recovery middleware.
    return app.recoverPanic(router)
}
```

Now that's in place, if there is a panic in one of our API handlers, the `recoverPanic()` middleware will recover it and call our regular `app.serverErrorResponse()` helper. In turn, that will log the error using our structured logger and send the client a nice `500 Internal Server Error` response with a JSON body.

Additional information

System-generated error responses

While we're on the topic of errors, I'd like to mention that in certain scenarios Go's `http.Server` may still automatically generate and send plain-text HTTP responses. These scenarios include when:

- The HTTP request specifies an unsupported HTTP protocol version.
- The HTTP request contains a missing or invalid `Host` header, or multiple `Host` headers.
- The HTTP request contains an empty `Content-Length` header.
- The HTTP request contains an unsupported `Transfer-Encoding` header.
- The size of the HTTP request headers exceeds the server's `MaxHeaderBytes` setting.
- The client makes an HTTP request to an HTTPS server.

For example, if we try sending a request with an invalid `Host` header value, we will get a response like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i -H "Host: こんにちは"  http://localhost:4000/v1/healthcheck
HTTP/1.1 400 Bad Request: malformed Host header
Content-Type: text/plain; charset=utf-8
Connection: close

400 Bad Request: malformed Host header
```

Unfortunately, these responses are hard-coded into the Go standard library, and there's nothing we can do to customize them to use JSON instead.

But while this is something to be aware of, it's not necessarily something to worry about. In a production environment it's relatively unlikely that well-behaved, non-malicious, clients would trigger these responses anyway, and we shouldn't be overly concerned if bad clients are sometimes sent a plain-text response instead of JSON.

Panic recovery in other goroutines

It's really important to realize that our middleware will only recover panics that happen in the *same goroutine that executed the* `recoverPanic()` *middleware*.

If, for example, you have a handler which spins up another goroutine (e.g. to do some background processing), then any panics that happen in the background goroutine will not be recovered — not by the `recoverPanic()` middleware… and not by the panic recovery built into `http.Server`. These panics will cause your application to exit and bring down the server.

So, if you are spinning up additional goroutines from within your handlers and there is any chance of a panic, you *must* make sure that you recover any panics from within those goroutines too.

We'll look at this topic in more detail later in the book, and demonstrate how to deal with it when we use a background goroutine to send welcome emails to our API users.

dego_vila@yahoo.com 30 Oct 2026

Chapter 4.

Parsing JSON requests

So far we've been looking at how to create and send JSON responses from our API, but in this next section of the book we're going to explore things from the other side and discuss how to *read and parse JSON requests* from clients.

To help demonstrate this, we're going to start working on the `POST /v1/movies` endpoint and `createMovieHandler` that we set up earlier.

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
POST	**/v1/movies**	**createMovieHandler**	**Create a new movie**
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie

When a client calls this endpoint, we expect them to provide a JSON request body containing data for the movie they want to create in our system. So, for example, if a client wants to add a record for the movie *Moana* to our API, they would send a request body similar to this:

```
{
    "title": "Moana",
    "year": 2016,
    "genres": ["animation", "adventure"]
}
```

For now, we'll just focus on the reading, parsing and validation aspects of dealing with this JSON request body. Specifically, you'll learn:

- How to read a request body and decode it to a native Go value using the `encoding/json/v2` package.
- How to deal with bad requests from clients and invalid JSON, and return clear, actionable, error messages.
- How to create a reusable helper package for validating data to ensure it meets your business rules.
- Different techniques for controlling and customizing how JSON is decoded.

dego_vila@yahoo.com 30 Oct 2026

Chapter 4.1.

JSON decoding

Just like JSON encoding, there are three different functions that you can use to *decode* JSON into a native Go type: `json.Unmarshal()`, `json.UnmarshalDecode()`, and `json.UnmarshalRead()`.

Again, we'll explain how all three functions work and the differences between them in this chapter, but for the purpose of decoding JSON from an HTTP request body, using `json.UnmarshalRead()` is generally the right choice.

It's easiest to demonstrate how `json.UnmarshalRead()` works with code, rather than words, so let's jump straight in and update our `createMovieHandler` like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

import (
    "encoding/json/v2" // New import
    "fmt"
    "net/http"
    "time"

    "greenlight.alexedwards.net/internal/models"
)

...

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    // Declare an anonymous struct to hold the information that we expect to be in the
    // HTTP request body (note that the field names and types in the struct are a subset
    // of the Movie struct that we created earlier). This struct will be our *target
    // decode destination*.
    var input struct {
        Title  string   `json:"title"`
        Year   int      `json:"year"`
        Genres []string `json:"genres"`
    }

    // Use json.UnmarshalRead() to read from the request body and decode the body
    // contents into the input struct. Importantly, notice that when we call
    // UnmarshalRead() we pass a *pointer* to the input struct. If there was an error
    // during decoding, we use our errorResponse() helper to send a 400 Bad Request
    // response with the error message to the client.
    err := json.UnmarshalRead(r.Body, &input)
    if err != nil {
        app.errorResponse(w, r, http.StatusBadRequest, err.Error())
        return
    }

    // Dump the contents of the input struct in an HTTP response.
    fmt.Fprintf(w, "%+v\n", input)
}

...
```

There are a few important and interesting things about this code to point out:

- When calling `json.UnmarshalRead()` you must pass a *non-nil pointer* as the target decode destination. If you don't use a pointer, it will return a `"value must be passed as a non-nil pointer reference"` error at runtime.
- If the target decode destination is a struct — like in our case — the struct fields must be exported (start with a capital letter). Just like with encoding, they need to be exported so that they're visible to the `encoding/json/v2` package.
- When decoding a JSON object into a struct, Go maps the JSON key-value pairs to struct fields based on the names specified in the struct tags. If there is no matching name in the struct tags, Go will look for a struct field name that matches the JSON key. In both cases,

dego_vila@yahoo.com 30 Oct 2026

a case-sensitive match is required. *Any JSON key-value pairs which cannot be successfully mapped to the struct fields will be silently ignored*.

- There is no need to close `r.Body` after it has been read. This will be done automatically by Go's `http.Server`, so you don't have to.

OK, let's take this for a spin.

Fire up the application, then open a second terminal window and make a request to the `POST /v1/movies` endpoint with a valid JSON request body containing some movie data. You should see a response similar to this:

```
# Create a BODY variable containing the JSON data that we want to send.
$ BODY='{"title":"Moana", "year":2016, "genres":["animation","adventure"]}'

# Use the -d flag to send the contents of the BODY variable as the HTTP request body.
# Note that curl will default to sending a POST request when the -d flag is used.
$ curl -i -d "$BODY" localhost:4000/v1/movies
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 08:48:17 GMT
Content-Length: 53
Content-Type: text/plain; charset=utf-8

{Title:Moana Year:2016 Genres:[animation adventure]}
```

Great! That seems to have worked well. We can see from the data dumped in the response that the values we provided in the request body have been decoded into the appropriate fields of our `input` struct.

Zero values

Let's take a quick look at what happens if we omit a particular key-value pair in our JSON request body. For example, let's make a request with no `year` in the JSON, like so:

```
$ BODY='{"title":"Moana", "genres":["animation","adventure"]}'
$ curl -d "$BODY" localhost:4000/v1/movies
{Title:Moana Year:0 Genres:[animation adventure]}
```

As you might have guessed, when we do this the `Year` field in our `input` struct is left with its zero value (which happens to be `0` because the `Year` field is an `int` type).

This leads to an interesting question: how can you tell the difference between a client *not providing a key-value pair*, and *providing a key-value pair but deliberately setting it to its zero value*? Like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"title":"Moana", "year":0, "genres":["animation","adventure"]}'
$ curl -d "$BODY" localhost:4000/v1/movies
{Title:Moana Year:0 Genres:[animation adventure]}
```

The end result is the same, despite the different HTTP requests, and it's not immediately obvious how to tell the difference between the two scenarios. We'll circle back to this topic later in the book, but for now, it's worth just being aware of this behavior.

Additional information

How different Go types are decoded

It's important to mention that certain JSON types can only be successfully decoded to certain Go types. For example, if you have the JSON string `"foo"` it can be decoded into a Go `string`, but trying to decode it into a Go `int` or `bool` will result in an error at runtime (as we'll demonstrate in the next chapter).

The following table shows the Go types that can be used as decode destinations for each JSON type. These mappings are generally the reverse of the encoding mappings that we talked about earlier in the book.

JSON type	⇒	Supported Go types
JSON boolean	⇒	`bool`
JSON string	⇒	`string`
JSON number	⇒	integers, floating point numbers, `rune`
JSON array	⇒	arrays, slices
JSON object	⇒	structs, maps
RFC3339-format JSON string	⇒	`time.Time`, `string`
`netip`-format JSON string	⇒	`netip.Addr`, `netip.AddrPort`, `netip.Prefix`, `string`
Base64-encoded JSON string	⇒	`byte` arrays, `byte` slices, `string`

Similar to encoding, the last three of these are special cases:

- If the JSON contains a RFC 3339 format string like `"2020-11-08T06:27:59+01:00"`, you

dego_vila@yahoo.com 30 Oct 2026

can decode it into either a Go `string` or a `time.Time` type.

- If the JSON contains a `netip` format string like `"127.0.0.1"`, `"127.0.0.1:8080"` or `"127.0.0.1/24"`, it can be decoded into either a Go `string` or a `netip.Addr`, `netip.AddrPort` and `netip.Prefix` respectively.
- If the JSON contains a base64-encoded JSON string (using standard encoding and padding), then it can be decoded into a Go `string`, `[]byte`, or `byte` array (like `[3]byte`). To see this in action, you can check out this Go playground example of decoding a base64 JSON string into a `[]byte` type.

Using json.Unmarshal and json.UnmarshalDecode

At the start of this chapter, I mentioned that it's also possible to use the `json.Unmarshal()` function to decode the JSON in an HTTP request body.

For example, you could use it in a handler like this:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Foo string `json:"foo"`
    }

    // Use io.ReadAll() to read the entire request body into a []byte slice.
    body, err := io.ReadAll(r.Body)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Use the json.Unmarshal() function to decode the JSON in the []byte slice to the
    // input struct. Again, notice that we are using a *pointer* to the input
    // struct as the decode destination.
    err = json.Unmarshal(body, &input)
    if err != nil {
        app.errorResponse(w, r, http.StatusBadRequest, err.Error())
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}
```

Or alternatively you could use the `json.UnmarshalDecode()` function. Really, this function is intended for reading and decoding *streams* of JSON values, rather than just a single JSON value. But in theory you could use it in a handler to decode a single JSON object in a request body like so:

dego_vila@yahoo.com 30 Oct 2026

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Foo string `json:"foo"`
    }

    // Use the jsontext.NewDecoder() to initialize a jsontext.Decoder instance that
    // reads from the request body.
    dec := jsontext.NewDecoder(r.Body)

    // Pass the decoder json.UnmarshalDecode(), along with a pointer to the input
    // struct as the decode destination.
    err := json.UnmarshalDecode(dec, &input)
    if err != nil {
        app.errorResponse(w, r, http.StatusBadRequest, err.Error())
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}
```

Both these patterns work, but they don't offer any benefits over and above the `json.UnmarshalRead()` pattern that we're already using. Not only is the code marginally more verbose, but — as we'll see in the benchmarks next — they are also less efficient for this specific use-case.

The following benchmarks demonstrate the differences between the three decoding patterns, using the code in this gist:

```
$ go test -run=^$ -bench=. -benchmem -count=6 -benchtime=2s
goos: linux
goarch: amd64
cpu: 12th Gen Intel(R) Core(TM) i7-12700H
BenchmarkUnmarshal-20          3652729    658.1 ns/op    560 B/op     3 allocs/op
BenchmarkUnmarshal-20          3590724    690.4 ns/op    560 B/op     3 allocs/op
BenchmarkUnmarshal-20          3766821    616.5 ns/op    560 B/op     3 allocs/op
BenchmarkUnmarshal-20          4036034    595.9 ns/op    560 B/op     3 allocs/op
BenchmarkUnmarshal-20          3991036    628.0 ns/op    560 B/op     3 allocs/op
BenchmarkUnmarshal-20          4538096    610.7 ns/op    560 B/op     3 allocs/op
BenchmarkUnmarshalRead-20      5829306    414.6 ns/op     48 B/op     2 allocs/op
BenchmarkUnmarshalRead-20      5785305    412.3 ns/op     48 B/op     2 allocs/op
BenchmarkUnmarshalRead-20      5810266    418.8 ns/op     48 B/op     2 allocs/op
BenchmarkUnmarshalRead-20      5654031    422.7 ns/op     48 B/op     2 allocs/op
BenchmarkUnmarshalRead-20      5661086    422.5 ns/op     48 B/op     2 allocs/op
BenchmarkUnmarshalRead-20      5615889    424.0 ns/op     48 B/op     2 allocs/op
BenchmarkUnmarshalDecode-20    1000000    2019 ns/op     5424 B/op    12 allocs/op
BenchmarkUnmarshalDecode-20    1201005    2062 ns/op     5424 B/op    12 allocs/op
BenchmarkUnmarshalDecode-20    1113412    2170 ns/op     5424 B/op    12 allocs/op
BenchmarkUnmarshalDecode-20     843244    2480 ns/op     5424 B/op    12 allocs/op
BenchmarkUnmarshalDecode-20     924385    2524 ns/op     5424 B/op    12 allocs/op
BenchmarkUnmarshalDecode-20    1097529    2145 ns/op     5424 B/op    12 allocs/op
```

From these benchmarks we can see that the `json.UnmarshalRead()` pattern is the clear winner — it's visibly more efficient in terms of both memory use (`B/op` and `allocs/op`) and speed (`ns/op`).

dego_vila@yahoo.com 30 Oct 2026

So, for the purpose of decoding a single JSON value in a HTTP request in your handlers, using the `json.UnmarshalRead()` pattern is generally the best choice for both code simplicity and performance.

Case-insensitive matching

Earlier in this chapter I said:

> *When decoding a JSON object into a struct, Go maps the JSON key-value pairs to struct fields based on the names specified in the struct tags. If there is no matching name in the struct tags, Go will look for a struct field name that matches the JSON key. In both cases, a case-sensitive match is required.*

If you'd like to change this logic to allow case-insensitive matches instead, you can pass the `json.MatchCaseInsensitiveNames` option when calling `json.UnmarshalRead()`. Like so:

```
err := json.UnmarshalRead(r.Body, &input, json.MatchCaseInsensitiveNames(true))
if err != nil {
   ...
}
```

Or alternatively, if you only want to allow a case-insensitive match for specific struct fields, rather than all of them, you can use the `case:ignore` directive in the struct tag. For example:

```
// With names in the struct tags.
var inputOne struct {
    Foo int `json:"foo"`             // Matches only the JSON key "foo".
    Bar int `json:"bar"`             // Matches only the JSON key "bar".
    Baz int `json:"baz,case:ignore"` // Matches the JSON keys "baz", "Baz", "BAZ", etc.
}

// Without names in the struct tags.
var inputTwo struct {
    Foo int                       // Matches only the JSON key "Foo".
    Bar int                       // Matches only the JSON key "Bar".
    Baz int `json:",case:ignore"` // Matches the JSON keys "Baz", "baz", "BAZ", etc.
}
```

Additional JSON decoding nuances

There are a few JSON decoding nuances that are important or interesting to know about, but which don't fit nicely into the main content of this book. I've included this appendix at the end which explains and demonstrates them in detail.

dego_vila@yahoo.com 30 Oct 2026

Chapter 4.2.

Managing bad requests

Our `createMovieHandler` now works well when it receives a valid JSON request body with the appropriate data. But at this point you might be wondering:

- What if the client sends something that isn't JSON, like XML or some random bytes?
- What happens if the JSON is malformed or contains an error?
- What if the JSON types don't match the types we are trying to decode into?
- What if the request body contains more than one JSON value?
- What if the request doesn't even contain a body?

Well… let's look and see!

dego_vila@yahoo.com 30 Oct 2026

```
# Send some XML as the request body
$ curl -d '<?xml version="1.0" encoding="UTF-8"?><note><to>Alice</to></note>' localhost:4000/v1/movies
{
    "error": "jsontext: invalid character '<' at start of value"
}

# Send some invalid JSON (missing closing } character)
$ curl -d '{"title": "Moana"' localhost:4000/v1/movies
{
    "error": "jsontext: unexpected EOF after offset 17"
}

# Send some invalid JSON (trailing comma in genres array)
$ curl -d '{"title": "Moana", "genres": ["animation",]}' localhost:4000/v1/movies
{
    "error": "jsontext: invalid character ',' at start of value within \"/genres\" after offset 41"
}

# Send an integer title value instead of string
$ curl -d '{"title": 123}' localhost:4000/v1/movies
{
    "error": "json: cannot unmarshal JSON number into Go string within \"/title\""
}

# Send a JSON array instead of an object
$ curl -d '["foo", "bar"]' localhost:4000/v1/movies
{
    "error": "json: cannot unmarshal JSON array into Go struct"
}

# Send a body with multiple JSON values
$ curl -d '{"title": "Moana"}{"title": "Top Gun"}' localhost:4000/v1/movies
{
    "error": "jsontext: invalid character '{' after top-level value after offset 18"
}

# Send an empty request body
$ curl -d '' localhost:4000/v1/movies
{
    "error": "jsontext: unexpected EOF"
}
```

> **Important:** Some of these error messages use JSON pointer notation to help indicate where the problem is in the JSON. For example, the message `"json: cannot unmarshal JSON number into Go string within \"/title\""` includes the (escaped) JSON pointer `/title` to indicate that the problem is with the `title` field.

In all these cases, we can see that our `createMovieHandler` is doing the right thing. When it receives an invalid request that can't be decoded into our `input` struct, no further processing takes place and the client is sent a JSON response containing the error message returned by the `json.UnmarshalRead()` method.

For a private API which won't be used by members of the public, this behavior is probably fine and you needn't do anything else.

dego_vila@yahoo.com 30 Oct 2026

But for a public-facing API, the error messages themselves aren't ideal. Some are too detailed and expose unnecessary implementation details to the client, such as `"json: cannot unmarshal JSON array into Go struct"`. Others aren't really descriptive enough, such as `"jsontext: unexpected EOF"`. The inconsistent `json`/`jsontext` prefixes are also confusing and don't really provide any value to public clients.

To improve this, we're going to triage the errors returned by `json.UnmarshalRead()` and replace them with more appropriate ones that help the client debug exactly what is wrong with their JSON, without including unnecessary information in them.

Triaging the errors

It's important to understand that `json.UnmarshalRead()` returns different *types* of error depending on what the exact problem is.

- If the JSON being read is syntactically invalid (that is, there is a problem with the JSON itself, regardless of the data it contains), then `json.UnmarshalRead()` will return an error with the type `jsontext.SyntacticError`.
- If the JSON is syntactically valid, but cannot be successfully decoded to the target destination, then `json.UnmarshalRead()` will return an error with the type `json.SemanticError`.
- The `json.UnmarshalRead()` function can also return other types of errors, such as an error if it fails to read from `r.Body`.

For our purpose, it's the first two types of error — `jsontext.SyntacticError` and `json.SemanticError` — that we're going to triage and handle as special cases.

Specifically, when `json.UnmarshalRead()` returns a `jsontext.SyntacticError` or `json.SemanticError`, we'll create a custom `badRequestError` containing an appropriate client-facing message. We'll then return this error to the client with a `400 Bad Request` status code.

So, first things first, let's head to `cmd/api/errors.go` and create a `badRequestError` type and an `app.badRequestResponse()` helper function that we can use to send the response, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/errors.go

```
package main

...

type badRequestError struct {
    Message string
}

func (e badRequestError) Error() string {
    return e.Message
}

func (app *application) badRequestResponse(w http.ResponseWriter, r *http.Request, err error) {
    app.errorResponse(w, r, http.StatusBadRequest, err.Error())
}
```

The logic for decoding JSON requests and handling errors is something that we'll need to duplicate *a lot* throughout this project, so to assist with this, let's create a new `readJSON()` helper in the `cmd/api/helpers.go` file.

In this helper we'll decode the JSON from the request body as normal, then triage the errors and return a `badRequestError` type as necessary. Let's start with the simplest possible implementation, and slowly improve it.

If you're coding along, go ahead and add the following code to the `cmd/api/helpers.go` file:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

func (app *application) readJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    // Decode the request body into the target destination.
    err := json.UnmarshalRead(r.Body, dst)
    if err != nil {
        // If there is an error, start the triage...
        var synErr *jsontext.SyntacticError
        var semErr *json.SemanticError

        switch {
        // Use the errors.As() function to check whether the error has the type
        // *jsontext.SyntacticError. If it does, then return a badRequestError
        // with a placeholder message.
        case errors.As(err, &synErr):
            return &badRequestError{
                Message: "there was a syntactic error",
            }

        // Likewise, use the errors.As() function to check whether the error has
        // the type *json.SemanticError. If it does, then return a badRequestError
        // with a placeholder message.
        case errors.As(err, &semErr):
            return &badRequestError{
                Message: "there was a semantic error",
            }

        // For any other type of error, return it as normal.
        default:
            return err
        }
    }

    return nil
}
```

With this new helper in place, let's head back to the `cmd/api/movies.go` file and update our `createMovieHandler` to use it. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

import (
    "errors" // New import
    "fmt"
    "net/http"
    "time"

    "greenlight.alexedwards.net/internal/models"
)

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title  string   `json:"title"`
        Year   int      `json:"year"`
        Genres []string `json:"genres"`
    }

    // Use the new readJSON() helper to decode the request body into the input struct.
    // If this returns an error we check to see if it has our badRequestError type. If
    // it does, we use the badRequestResponse() helper to send it to the client along
    // with a 400 Bad Request status code. For any other error type, we use the
    // serverErrorResponse() helper to log the error and send a 500 Internal Server Error
    // status code to the client.
    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}

...
```

And if you run this and try making some bad requests again, you should now see our custom error messages with a `400 Bad Request` status. Like so:

dego_vila@yahoo.com 30 Oct 2026

```
# Send some invalid JSON (trailing comma in genres array)
$ curl -i -d '{"title": "Moana", "genres": ["animation",]}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 09:01:55 GMT
Content-Length: 44

{
    "error": "there was a syntactic error"
}

# Send an integer title value instead of string
$ curl -i -d '{"title": 123}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 09:02:15 GMT
Content-Length: 43

{
    "error": "there was a semantic error"
}
```

That gives us a good foundation to work from. Now let's dig in and look at the details of triaging each specific type of error.

The jsontext.SyntacticError type

Behind the scenes, the `jsontext.SyntacticError` type is a struct containing structured information about the error. From the documentation:

```
type SyntacticError struct {
    // ByteOffset indicates that an error occurred after this byte offset.
    ByteOffset int64

    // JSONPointer indicates that an error occurred within this JSON value
    // as indicated using the JSON pointer notation (see RFC 6901).
    JSONPointer Pointer

    // Err is the underlying error.
    Err error
}
```

Rather than just sending `"there was a syntactic error"`, we can use the fields in the `jsontext.SyntacticError` type to build up a clear and actionable error message for the client. Specifically:

- `ByteOffset` is the character position in the JSON where the syntax error was identified.

  It's worth being aware that `ByteOffset` is the position where the parser *first identified there was a problem* — not necessarily *where the problem actually is*. This difference often becomes apparent when you forget to close a quoted value. For example, if you

dego_vila@yahoo.com 30 Oct 2026

have the JSON `{"title": "Moana, "year": 2016}` , the actual problem is in the JSON at character position 16, where `"Moana` is missing a closing double-quote. But the first time the parser knows for sure there is a problem is when it gets to the character `y` — and so the `ByteOffset` reported will be 19.

- `JSONPointer` is a JSON pointer to help indicate where the syntax error is. This field is only populated *sometimes*, when the syntax error is within part of the JSON that can be identified with JSON pointer notation.

- `Err` contains a human-readable description of the problem and — in some cases — a hint about how to fix it. The `Err` value in `jsontext.SyntacticError` is OK to send to the client as-is — it doesn't expose any information about the underlying Go implementation.

  However, it's worth pointing out that the descriptions can be a bit 'parsery' — as in they are descriptions of the problem from the viewpoint of a JSON parser. While they might be technically correct, they aren't always couched in the terms that a human would use to describe the problem. On balance though, they're still useful and worth sending to clients as a debugging aid. Although there might be some `Err` values — like `"unexpected EOF"` — that you might want to replace with something more descriptive like `"unexpected end of JSON"`.

In our case, we'll use these three fields to construct consistent `badRequestError` messages for the client using the following format:

```
<JSONPointer> contains malformed JSON at position <ByteOffset>: <Err>
```

If no `JSONPointer` is available, we'll use the words `"request body"` in the message instead.

There's one special case we need to handle though. If the client sends an empty request body, we want to break from this pattern and send a `"request body must not be empty"` message instead. Luckily this scenario is fairly easy to identify — when you try to read an empty request body, `json.UnmarshalRead()` will return a `jsontext.SyntacticError` which *wraps* an `io.ErrUnexpectedEOF` error *and* has a `ByteOffset` value of `0`.

OK, let's go back to our `cmd/api/helpers.go` file and create a new `convertSyntacticErrorToBadRequest()` helper function that converts the contents of a `jsontext.SyntacticError` into our custom `badRequestError` error, and use it in `readJSON()` like so:

File: cmd/api/helpers.go

```
package main
```

dego_vila@yahoo.com 30 Oct 2026

```
import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "errors"
    "fmt" // New import
    "io"  // New import
    "net/http"
    "strconv"
    "strings" // New import

    "greenlight.alexedwards.net/internal/models"

    "github.com/julienschmidt/httprouter"
)

...

func (app *application) readJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    err := json.UnmarshalRead(r.Body, dst)
    if err != nil {
        var synErr *jsontext.SyntacticError
        var semErr *json.SemanticError

        switch {
        case errors.As(err, &synErr):
            // If there is a jsontext.SyntacticError, convert it to a badRequestError
            // and return it.
            return convertSyntacticErrorToBadRequest(synErr)

        case errors.As(err, &semErr):
            return &badRequestError{
                Message: "there was a semantic error",
            }

        default:
            return err
        }
    }

    return nil
}

func convertSyntacticErrorToBadRequest(synErr *jsontext.SyntacticError) *badRequestError {
    // Handle the special case for an empty request body.
    if errors.Is(synErr, io.ErrUnexpectedEOF) && synErr.ByteOffset == 0 {
        return &badRequestError{Message: "request body must not be empty"}
    }

    // Otherwise, build up the error message containing the JSONPointer (if
    // available), ByteOffset and Err values.
    var buf strings.Builder

    pointer := string(synErr.JSONPointer)
    if pointer == "" {
        pointer = "request body"
    }
    fmt.Fprintf(&buf, "%s contains malformed JSON", pointer)

    offset := synErr.ByteOffset
    if offset > 0 {
        fmt.Fprintf(&buf, " at position %d", offset)
    }

    detail := synErr.Err.Error()
    // Replace any "unexpected EOF" value with a slightly more descriptive
    // message
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Message.
    if detail == "unexpected EOF" {
        detail = "unexpected end of JSON"
    }
    fmt.Fprintf(&buf, ": %s", detail)

    // Return a badRequestError containing the message.
    return &badRequestError{Message: buf.String()}
}
```

And if you restart the application and try making some of the same requests again, you'll now see nice, consistently formatted error messages like the ones below.

```
$ curl -d '' localhost:4000/v1/movies
{
    "error": "request body must not be empty"
}

$ curl -d '<?xml version="1.0" encoding="UTF-8"?><note><to>Alice</to></note>' localhost:4000/v1/movies
{
    "error": "request body contains malformed JSON: invalid character '<' at start of value"
}

$ curl -d '{"title": "Moana"' localhost:4000/v1/movies
{
    "error": "request body contains malformed JSON at position 17: unexpected end of JSON"
}

$ curl -d '{"title": "Moana", "genres": ["animation",]}' localhost:4000/v1/movies
{
    "error": "/genres contains malformed JSON at position 41: invalid character ',' at start of value"
}
```

The json.SemanticError type

While working with syntactic errors is relatively straightforward, dealing with `json.SemanticError` errors is more of a challenge.

As we've already mentioned, a `json.SemanticError` is returned if the JSON is syntactically valid, but cannot be successfully decoded to the target destination. At a high level, this can happen for two reasons:

1. The JSON contains values that are an unexpected type. As we explained in the previous chapter, only certain JSON types can be successfully decoded to certain Go types. So — for example — if you are expecting a JSON number in the request body, but it actually contains a string value like `"abc"` instead, then trying to decode this value into an `int` field will fail. Or as another example, if you are trying to decode a JSON number into an `int8` field, but the JSON number is unexpectedly greater than 127, then the decoding will fail.

dego_vila@yahoo.com 30 Oct 2026

Like syntax errors, these unexpected, invalid, values are a thing that the client needs to fix. *They are still problems with the JSON in the request body*. For the rest of this chapter, we'll refer to this class of problems as 'client-side' problems, and when they occur we want to send the client a `badRequestError` to let them know about it.

2. Alternatively, there could be a problem or logical inconsistency in our Go code. For example, using a `nil` pointer as the target decode destination, or having a malformed struct tag, can cause the decoding to fail. For the rest of this chapter, we'll refer to these as 'server-side' problems. For this class of problem, we want to log the error (so it can be reviewed and fixed), and send a `500 Internal Server` error response to the client.

Some of the server-side problems that can cause JSON decoding to fail are:

- The target decode destination is not a pointer, or it's a `nil` pointer.
- The target decode destination is a pointer to a `nil` interface value.
- When the target destination is a struct:
    - A `json` struct tag is malformed.
    - A `json` struct tag contains unknown or unsupported directives.
    - The `json` struct tags contain conflicting names.
    - An unexported field has a `json` struct tag other than `json:"-"`.
    - The struct contains only unexported fields, none of which are explicitly ignored with the `json:"-"` tag.
    - The decode field is ambiguous. This can happen when the `json.MatchCaseInsensitiveNames` option is enabled, and the struct contains field names that are duplicates under case folding.

As we can see, there are quite a few potential server-side problems that can cause JSON decoding to fail. When we receive a `json.SemanticError`, it's important that we try to distinguish these from the client-side problems so that we can handle them appropriately.

OK, let's take a look at the `json.SemanticError` struct and see what details it contains. From the documentation:

dego_vila@yahoo.com 30 Oct 2026

```
type SemanticError struct {
    // ByteOffset indicates that an error occurred after this byte offset.
    ByteOffset int64

    // JSONPointer indicates that an error occurred within this JSON value
    // as indicated using the JSON pointer notation (see RFC 6901).
    JSONPointer jsontext.Pointer

    // JSONKind is the JSON kind that could not be handled.
    JSONKind jsontext.Kind // may be zero if unknown

    // JSONValue is the JSON number or string that could not be unmarshaled.
    // It is not populated during marshaling.
    JSONValue jsontext.Value // may be nil if irrelevant or unknown

    // GoType is the Go type that could not be handled.
    GoType reflect.Type // may be nil if unknown

    // Err is the underlying error.
    Err error // may be nil
}
```

In a `json.SemanticError`, the `ByteOffset` and `JSONPointer` fields do the same thing as they do for syntactic errors — they help to identify the place in the JSON where the problem occurred.

The `JSONKind` field contains a `jsontext.Kind` value representing the *kind* of JSON token (e.g. string, number) that was being processed when the problem occurred.

Similarly, the `JSONValue` field contains the raw JSON value that was being processed when the problem occurred, and the `GoType` field contains the Go type that the value was trying to be decoded into.

The `Err` field contains an `error` describing the problem that triggered the semantic error. If you're running a public-facing API, you probably shouldn't return this `Err` value to the client since it often exposes implementation details of our Go code that are not relevant — and potentially confusing — to users. Often, but not always, the `Err` field is `nil` for the class of error we are calling 'client-side' problems.

So while these fields provide useful information for debugging, unfortunately there's nothing that explicitly tells us whether the semantic error is due to a client-side problem or a server-side problem. In other words, there's nothing in `json.SemanticError` that definitively tells us whether the problem arose because the JSON contained an unprocessable value, or because there is something wrong with our Go code.

This limitation has been confirmed by Joe Tsai (one of the main authors of `encoding/json/v2`), and hopefully it's something that will be improved in a future version of Go.

dego_vila@yahoo.com 30 Oct 2026

But in the meantime, how can we tell the difference?

In practice, the most useful pieces of information we have are the `Err`, `JSONKind` and `ByteOffset` fields, and we have to build a heuristic around these.

A heuristic for triaging semantic errors

The first thing we can do is check the value of the `Err` field. Most of the `Err` values are dynamically generated at runtime, but there are a handful of server-side problems that have fixed `Err` values associated with them and we can check for those fixed values.

Beyond that, we can look at the value of the `ByteOffset` field. If `ByteOffset` is `0`, it means that the problem occurred *right at the start of processing* and therefore is likely to be a server-side problem. Otherwise, when `ByteOffset` is greater than `0` it's likely that the problem is due to an unprocessable JSON value and is a client-side problem.

But there is a special case that breaks this `ByteOffset` assumption. In our application, we always expect the request body to contain a top-level *JSON object*, and if the client sends a different top-level JSON type, like a JSON array, this is a client-side problem. In this scenario Go will realize there is a mismatch when processing the first token in the JSON and it will — rightly — report a `ByteOffset` of `0`. Luckily, we can check the `JSONKind` field in this scenario, and if it's not a `jsontext.KindBeginObject` token — which indicates the start of a JSON object — then that indicates the client sent the wrong top-level type and we are dealing with a client-side problem.

OK… so that's all quite a lot to think about. But let's put this logic into practice and implement the heuristic in a new `isClientSemanticError()` function, which is called by our `readJSON()` helper.

While we're at it, we'll also create a `convertSemanticErrorToBadRequest()` function to convert a `json.SemanticError` into our custom `badRequestError` type.

Like so:

File: cmd/api/helpers.go

```
package main

import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "errors"
    "fmt"
    "io"
    "net/http"
```

dego_vila@yahoo.com 30 Oct 2026

```
    "strconv"
    "strings"
    "slices" // New import

    "greenlight.alexedwards.net/internal/models"

    "github.com/julienschmidt/httprouter"
)

...

func (app *application) readJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    err := json.UnmarshalRead(r.Body, dst)
    if err != nil {
        var synErr *jsontext.SyntacticError
        var semErr *json.SemanticError

        switch {
        case errors.As(err, &synErr):
            return convertSyntacticErrorToBadRequest(synErr)

        // If it is a semantic error, and our isClientSemanticError() heuristic
        // suggests it is a client-side problem with the JSON values, convert the
        // jsontext.SemanticError to a badRequestError and return it.
        case errors.As(err, &semErr) && isClientSemanticError(semErr):
            return convertSemanticErrorToBadRequest(semErr)

        default:
            return err
        }
    }

    return nil
}

// Create an isClientSemanticError function which returns true if the
// json.SemanticError likely represents a client-side problem with the JSON input
// values, and false if it likely represents a server-side problem with our Go
// code.
func isClientSemanticError(semErr *json.SemanticError) bool {
    // Some server-side problems have fixed Err values that we can check for. If
    // there's a match, we know the problem is with our Go code so we return false.
    if semErr.Err != nil {
        msgs := []string{
            "ambiguous object member name",
            "cannot derive concrete type for nil interface with finite type set",
            "Go struct has no exported fields",
            "value must be passed as a non-nil pointer reference",
        }
        if slices.Contains(msgs, semErr.Err.Error()) {
            return false
        }
    }

    // An offset of 0 means processing failed at the very start. If the first
    // token is not the { character we'd expect for a top-level JSON object, then
    // this suggests the client sent an invalid JSON value and we return true.
    if semErr.ByteOffset == 0 && semErr.JSONKind != jsontext.KindBeginObject {
        return true
    }

    // A positive offset means processing failed partway through the JSON, which
    // we assume is a client-side problem with the JSON values and we return true.
    if semErr.ByteOffset > 0 {
        return true
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Otherwise, we fall back to assuming a server-side problem and return false.
    return false
}

// Add a function to convert a json.SemanticError to a badRequestError, using a
// pattern similar to the one that we used to convert jsontext.SyntacticError errors.
func convertSemanticErrorToBadRequest(semErr *json.SemanticError) *badRequestError {
    var buf strings.Builder

    pointer := string(semErr.JSONPointer)
    if pointer == "" {
        pointer = "request body"
    }
    fmt.Fprintf(&buf, "%s contains an unsupported JSON type or invalid value", pointer)

    offset := semErr.ByteOffset
    if offset > 0 {
        fmt.Fprintf(&buf, " at position %d", offset)
    }

    return &badRequestError{Message: buf.String()}
}
```

If you wanted to, you could compact the logic in the `isClientSemanticError()` function into fewer lines of code, but when the logic is not intuitive like this, I think it's better to have the cases laid out as clearly as possible and optimize for clarity, rather than conciseness.

Let's restart the application and try this out, using `curl` to send request bodies that contain syntactically valid JSON, but with unexpected values that can't be decoded to our `input` struct. You should now see our custom error messages with a `400 Bad Request` status, like so:

dego_vila@yahoo.com 30 Oct 2026

```
# Send an integer title value instead of string
$ curl -i -d '{"title": 123}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 09:39:25 GMT
Content-Length: 89

{
    "error": "/title contains an unsupported JSON type or invalid value at position 10"
}

# Send an integer in the second place of the genres value
$ curl -i -d '{"title": "Moana", "genres": ["animation", 123]}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 09:39:09 GMT
Content-Length: 92

{
    "error": "/genres/1 contains an unsupported JSON type or invalid value at position 43"
}

# Send a JSON array as the top-level value, instead of an object
$ curl -i -d '["Moana", 2016]' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 09:38:37 GMT
Content-Length: 80

{
    "error": "request body contains an unsupported JSON type or invalid value"
}
```

Illustrating a server-side problem

To complete this chapter, let's quickly illustrate what happens now when there is a semantic error due to a problem with our Go code.

Let's go back to our `createMovieHandler` and introduce a deliberate error by giving two of the `input` struct fields the same `json:"title"` struct tag, like so:

dego_vila@yahoo.com 30 Oct 2026

File:

```
package main

...

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title  string   `json:"title"`
        Year   int      `json:"title"` // Give this a conflicting name
        Genres []string `json:"genres"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}

...
```

When you run the application and try to make a syntactically valid request now, you should receive a `500 Internal Server Error` response similar to this:

```
$ curl -i -d '{"title": "Moana", "year": 2016}' localhost:4000/v1/movies
HTTP/1.1 500 Internal Server Error
Content-Type: application/json
Date: Thu, 10 Sep 2026 09:43:06 GMT
Content-Length: 84

{
    "error": "the server encountered a problem and could not process your request"
}
```

And you should also see an error message describing the problem in the application logs. Like so:

```
time=2026-09-10T11:43:06.866+02:00 level=ERROR msg="json: unable to unmarshal JSON object
    into Go struct: Go struct fields Title and Year conflict over JSON object name
    \"title\"" method=POST uri=/v1/movies
```

If you want, you could also try giving one of the fields a malformed struct tag. For example:

dego_vila@yahoo.com 30 Oct 2026

```
var input struct {
    Title  string   `json:"title,"` // Give this a trailing comma so it's malformed
    Year   int      `json:"year"`
    Genres []string `json:"genres"`
}
```

And you should now get a log message like this when you make a request:

```
time=2026-09-10T11:44:50.880+02:00 level=ERROR msg="json: cannot unmarshal JSON object
    into Go struct: Go struct field Title has malformed `json` tag: invalid trailing ','
    character" method=POST uri=/v1/movies
```

Before you continue to the next chapter, please make sure to revert any problems with the `input` struct, so that it looks exactly like this again:

```
var input struct {
    Title  string   `json:"title"`
    Year   int      `json:"year"`
    Genres []string `json:"genres"`
}
```

Additional information

Heuristic flakiness

Using a heuristic like we are in our `isClientSemanticError()` to classify semantic errors into ‘client-side’ and ‘server-side’ problems is flaky — it relies heavily on an undocumented assumption about the content of `ByteOffset` to determine whether we are dealing with an unprocessable JSON value or a problem with our Go code.

But — for Go 1.27 at least — the combination of checking fixed errors and then `ByteOffset` that we’ve used in this chapter appears to be the best approach, and `isClientSemanticError()` does correctly classify every possible error I’ve been able to find through experimentation and fuzz testing.

As I said earlier, hopefully this is an area that will be improved and made a lot simpler in a future version of Go.

Duplicate keys in JSON objects

If you try to send a JSON object with duplicate key names — like `{"title": "Moana",`

dego_vila@yahoo.com 30 Oct 2026

`"title": "Top Gun"}` — then by default `json.UnmarshalRead()` will consider this a syntactic error, and our application will return a response that looks similar to this:

```
$  curl -i -d '{"title": "Moana", "title": "Top Gun"}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Fri, 21 Aug 2026 11:19:54 GMT
Content-Length: 92

{
    "error": "/title contains malformed JSON at position 19: duplicate object member name"
}
```

Technically though, this is not invalid JSON. In fact, the JSON specification doesn't define whether duplicate key names are permitted or not, so how duplicate key names are handled is a bit of a grey area.

If you do want to allow duplicate key names, rather than treating them as a syntactic error, you can do so by using the `jsontext.AllowDuplicateNames` option like so:

```
err := json.UnmarshalRead(r.Body, dst, jsontext.AllowDuplicateNames(true))
if err != nil {
    ...
}
```

When you use this, the last duplicate key in the JSON wins, and its value will override any previous ones. The exception to this is when the target Go type for the key is a `struct` or `map`, in which case the values associated with the later duplicate key will be merged with the earlier values.

Decoding alternative top-level JSON types

Throughout the course of this book, and the build of our Greenlight application, we will expect all JSON request bodies to always be formatted as a *JSON object*.

This is why we were able to include the `semErr.JSONKind != jsontext.KindBeginObject` check in the `isClientSemanticError()` function:

```
// An offset of 0 means processing failed at the very start. If the first
// token is not the { character we'd expect for a top-level JSON object, then
// this suggests the client sent an invalid JSON value and we return true.
if semErr.ByteOffset == 0 && semErr.JSONKind != jsontext.KindBeginObject {
    return true
}
```

But what if this isn't the case for another application you're working on? What if you want to

dego_vila@yahoo.com 30 Oct 2026

accept JSON request bodies with alternative top-level types, like a JSON array of values?

To support this, you could pass an additional argument to `readJSON()` and `isClientSemanticError()` that contains the `jsontext.Kind` value that you expect to see in the first byte of the JSON.

And you could even create some simple wrapper functions around this like `readJSONObject()` and `readJSONArray()` to make using this even easier. I've included an example of doing that in this gist.

dego_vila@yahoo.com 30 Oct 2026

Chapter 4.3.

Restricting Inputs

The changes that we made in the previous chapter to deal with invalid JSON and other bad requests were a big step in the right direction. But there are still a couple more things we can do to make our JSON processing even more robust.

One such thing is dealing with *unknown keys* in the JSON. For example, you can try sending a request containing the unknown key `rating` to our `createMovieHandler`, like so:

```
$ curl -i -d '{"title": "Moana", "rating":"PG"}' localhost:4000/v1/movies
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 10:00:39 GMT
Content-Length: 31
Content-Type: text/plain; charset=utf-8

{Title:Moana Year:0 Genres:[]}
```

Notice how this request works without any problems — there's no error to inform the client that the `rating` field is not recognized by our application. In certain scenarios, silently ignoring unknown fields may be exactly the behavior you want, but in our case it would be better if we could alert the client to the issue.

Fortunately, Go provides a `json.RejectUnknownMembers` option that we can use to generate an error when this happens. Specifically, it will generate a `json.SemanticError` which *wraps* a `json.ErrUnknownName` error.

Let's give this a try, updating our `readJSON()` helper to use the `json.RejectUnknownMembers` option, and updating the `convertSemanticErrorToBadRequest()` function to check for a `json.ErrUnknownName` error and handle it as a special case. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

func (app *application) readJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    // Use the json.RejectUnknownMembers option.
    err := json.UnmarshalRead(r.Body, dst, json.RejectUnknownMembers(true))
    if err != nil {
        var synErr *jsontext.SyntacticError
        var semErr *json.SemanticError

        switch {
        case errors.As(err, &synErr):
            return convertSyntacticErrorToBadRequest(synErr)

        case errors.As(err, &semErr) && isClientSemanticError(semErr):
            return convertSemanticErrorToBadRequest(semErr)

        default:
            return err
        }
    }

    return nil
}

...

func convertSemanticErrorToBadRequest(semErr *json.SemanticError) *badRequestError {
    // Use errors.Is to check if the json.SemanticError wraps json.ErrUnknownName.
    // If it does, then treat it as a special case and include an appropriate
    // message in the badRequestError.
    if errors.Is(semErr, json.ErrUnknownName) {
        return &badRequestError{
            Message: fmt.Sprintf("request body contains unknown key %s", semErr.JSONPointer),
        }
    }

    var buf strings.Builder

    pointer := string(semErr.JSONPointer)
    if pointer == "" {
        pointer = "request body"
    }
    fmt.Fprintf(&buf, "%s contains an unsupported JSON type or invalid value", pointer)

    offset := semErr.ByteOffset
    if offset > 0 {
        fmt.Fprintf(&buf, " at position %d", offset)
    }

    return &badRequestError{Message: buf.String()}
}
```

And if you restart the application and make the same request with an unknown `rating` field again, you should now get a `400 Bad Request` response containing a clear error message similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i -d '{"title": "Moana", "rating":"PG"}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 10:02:19 GMT
Content-Length: 58

{
    "error": "request body contains unknown key /rating"
}
```

Limiting request body size

Also, there's currently no upper limit on the maximum size of the request body. This means that our `createMovieHandler` would be a good target for any malicious clients that wish to perform a denial-of-service attack on our API.

We can address this by using the `http.MaxBytesReader()` function to limit the maximum size of the request body. If the maximum size is exceeded, then `json.UnmarshalRead()` will return an error with the type `http.MaxBytesError` that we can check for and handle gracefully.

Let's go ahead and use this in the `readJSON()` function like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

func (app *application) readJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    // Use http.MaxBytesReader() to limit the size of the request body to 1,048,576
    // bytes (1MB).
    r.Body = http.MaxBytesReader(w, r.Body, 1_048_576)

    err := json.UnmarshalRead(r.Body, dst, json.RejectUnknownMembers(true))
    if err != nil {
        var synErr *jsontext.SyntacticError
        var semErr *json.SemanticError
        // Add a new maxBytesErr variable.
        var maxBytesErr *http.MaxBytesError

        switch {
        // Use the errors.As() function to check whether the error has the type
        // *http.MaxBytesError. If it does, then it means the request body exceeded our
        // size limit of 1MB and we convert the http.MaxBytesError to a
        // badRequestError and return it.
        case errors.As(err, &maxBytesErr):
            return convertMaxBytesErrorToBadRequest(maxBytesErr)

        case errors.As(err, &synErr):
            return convertSyntacticErrorToBadRequest(synErr)

        case errors.As(err, &semErr) && isClientSemanticError(semErr):
            return convertSemanticErrorToBadRequest(semErr)

        default:
            return err
        }
    }

    return nil
}

...

func convertMaxBytesErrorToBadRequest(maxBytesErr *http.MaxBytesError) *badRequestError {
    return &badRequestError{
        Message: fmt.Sprintf("request body must not be larger than %d bytes", maxBytesErr.Limit),
    }
}
```

If you'd like to try this out, I've created a 1.5MB JSON file that you can download into your `/tmp` directory by running the following command:

```
$ wget -O /tmp/largefile.json https://www.alexedwards.net/static/largefile.json
```

Then if you try making a request to your `POST /v1/movies` endpoint with this file as the request body, the `http.MaxBytesReader()` check will kick in and you should get a response similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -d @/tmp/largefile.json localhost:4000/v1/movies
{
    "error": "request body must not be larger than 1048576 bytes"
}
```

And with that, you'll be glad to know that we're finally finished with error handling in the `readJSON()` helper 😊

Over the past two chapters, we've had to write a lot of error handling logic for what is ultimately a one-line call to `json.UnmarshalRead()`. But now that the code it's written, it's done. You don't need to touch it again, and it's something that you can copy-and-paste into other projects easily.

dego_vila@yahoo.com 30 Oct 2026

Custom JSON decoding

Earlier in this book, we created a custom marshaler so that movie runtime information would be displayed in the format `"{minutes}m"` in our JSON responses.

In this chapter, we're going to look at this from the other side and update our application so that the `createMovieHandler` *accepts* movie runtime information in this format.

To support this, the first thing we need to do is update the `createMovieHandler` to include a new `Runtime` field in the `input` struct. We'll give this field the custom `Runtime` type that we defined previously in the `internal/models/runtime.go` file, like so:

File: cmd/api/movies.go

```
package main

...

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    // Add a new Runtime field with the type models.Runtime.
    var input struct {
        Title   string         `json:"title"`
        Year    int            `json:"year"`
        Runtime models.Runtime `json:"runtime"`
        Genres  []string       `json:"genres"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}

...
```

At this point, it's worth sending a request with a JSON value like `{"runtime": "102m"}` just to see what happens. You should get a `400 Bad Request` response similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i -d '{"title": "Moana", "runtime": "102m"}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 10:32:58 GMT
Content-Length: 91

{
    "error": "/runtime contains an unsupported JSON type or invalid value at position 30"
}
```

Let's quickly unpack what is happening here.

In our `createMovieHandler`, the call to `readJSON()` attempts to decode the request body into the `input` struct. In our `input` struct, the `Runtime` field has the type `models.Runtime`, which in turn has the underlying type `time.Duration`. As you can hopefully remember from earlier in the book, `time.Duration` values don't have a default JSON representation, which means that — by default — Go doesn't support decoding values into them. It's the same problem that we had previously when trying to *encode* a `time.Duration` to JSON — just in reverse.

Because the `Runtime` field isn't supported as a decode destination, it triggers a `json.SemanticError` error, which is then triaged by the code in our `readJSON()` helper and the client is sent a `400 Bad Request` response containing the error message we see above.

So how do we fix this?

The solution — as you might be able to guess — is to create a *custom unmarshaler* for our `Runtime` type.

There are a couple of ways to do this, but the way I'd recommend is by using the `json.UnmarshalFromFunc()` function.

We can use `json.UnmarshalFromFunc()` to define a function which is called each time Go needs to decode some JSON into a `Runtime` type. Specifically, we can use it to define a function which reads a `json.Token` from a decoder, converts the token to a `Runtime` type, and then stores this converted value in the target destination.

Again, this is something that is easier to demonstrate with code rather than words, so let's head back to the `internal/models/runtime.go` file and create this custom unmarshaler as follows:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/runtime.go

```
package models

import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "errors" // New import
    "fmt"
    "math"
    "strings" // New import
    "time"
)

type Runtime time.Duration

...

var RuntimeUnmarshaler = json.UnmarshalFromFunc(
    // Define a function with pointer parameters to a jsontext.Decoder and the
    // type you want to decode, which in this case is a Runtime type. It's
    // important to note that you should always use pointer parameters in the
    // function signature here.
    func(dec *jsontext.Decoder, val *Runtime) error {
        // Use ReadToken() to read the JSON token from the decoder. In our case, we
        // are expecting the token to be a JSON string in the format "{minutes}m".
        token, err := dec.ReadToken()
        if err != nil {
            return err
        }

        // Sanity check that the token is indeed a JSON string, and that it has
        // the suffix 'm'. If it doesn't, then return an error.
        if token.Kind() != jsontext.KindString || !strings.HasSuffix(token.String(), "m") {
            return errors.New("invalid runtime value")
        }

        // Otherwise, use the token.String() method to get the unescaped string
        // value, and attempt to convert it into a time.Duration using the
        // time.ParseDuration() function. If this fails, return an error.
        d, err := time.ParseDuration(token.String())
        if err != nil {
            return fmt.Errorf("invalid runtime value: %w", err)
        }

        // Convert the time.Duration to a Runtime type, and then use the *
        // dereference operator to 'write through' and assign this as the underlying
        // value of the val pointer.
        *val = Runtime(d)
        return nil
    },
)
```

> **Note:** If dereferencing pointers is something that you are not familiar with, you might like to check out this introduction to working with pointers for more information and examples.

But before we move on and put this to use, we should take a moment to explain what

dego_vila@yahoo.com 30 Oct 2026

happens when this custom marshaler returns an error.

When the unmarshaler returns an error (like `errors.New("invalid runtime value")` in the code above), Go will automatically *wrap* this error in a `json.SemanticError` — with the `ByteOffset` and `GoType` fields set to the appropriate values, and the error value (like `"invalid runtime value"`) stored in the `Err` field. It's worth noting that the `JSONKind` field will *not* be set and will equal `jsontext.KindInvalid`.

In our application, the triage code in our `readJSON()` function will then kick in and recognize the `json.SemanticError`, and the client will be sent a `400 Bad Request` response.

OK, now that we have a custom unmarshaler for our `Runtime` type set up, we need to tell the `json.UnmarshalRead()` function that we want to use it. We can do this by using the `json.WithUnmarshalers` option, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

func (app *application) readJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    r.Body = http.MaxBytesReader(w, r.Body, 1_048_576)

    // For clarity and easy reading, let's create a json.Options slice
    // containing the options that we want to use when decoding. This includes
    // the json.RejectUnknownMembers option that we were already using, and a new
    // json.WithUnmarshalers option that tells Go we want to use our custom
    // models.RuntimeUnmarshaler unmarshaler.
    opts := []json.Options{
        json.RejectUnknownMembers(true),
        json.WithUnmarshalers(models.RuntimeUnmarshaler),
    }

    // Pass the options to json.UnmarshalRead().
    err := json.UnmarshalRead(r.Body, dst, opts...)
    if err != nil {
        var synErr *jsontext.SyntacticError
        var semErr *json.SemanticError
        var maxBytesErr *http.MaxBytesError

        switch {
        case errors.As(err, &maxBytesErr):
            return convertMaxBytesErrorToBadRequest(maxBytesErr)

        case errors.As(err, &synErr):
            return convertSyntacticErrorToBadRequest(synErr)

        case errors.As(err, &semErr) && isClientSemanticError(semErr):
            return convertSemanticErrorToBadRequest(semErr)

        default:
            return err
        }
    }

    return nil
}

...
```

If you're following along, feel free to try sending a request with a valid runtime value in the format `"{minutes}m"` now. For example:

```
$ curl -i -d '{"title": "Moana", "runtime": "102m"}' localhost:4000/v1/movies
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 10:41:04 GMT
Content-Length: 53
Content-Type: text/plain; charset=utf-8

{Title:Moana Year:0 Runtime:6120000000000 Genres:[]}
```

So that's looking really good. We can see that the request completed successfully, and that

dego_vila@yahoo.com 30 Oct 2026

the runtime value of `"102m"` was correctly decoded to a `time.Duration` value of `6120000000000` nanoseconds, and stored in the `Runtime` field of our `input` struct.

Let's finish up this chapter by making a couple of bad requests with an invalid `"runtime"` value in the JSON. As we talked about earlier, this should result in the triage code in our `readJSON()` function kicking in, and the client should receive a `400 Bad Request` response. Like so:

```
$ curl -i -d '{"title": "Moana", "runtime": 102}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 10:41:52 GMT
Content-Length: 91

{
    "error": "/runtime contains an unsupported JSON type or invalid value at position 30"
}

$ curl -i -d '{"title": "Moana", "runtime": "107 minutes"}' localhost:4000/v1/movies
HTTP/1.1 400 Bad Request
Content-Type: application/json
Date: Thu, 10 Sep 2026 10:42:13 GMT
Content-Length: 91

{
    "error": "/runtime contains an unsupported JSON type or invalid value at position 30"
}
```

Additional information

The UnmarshalFunc alternative

In this chapter we've used the `json.UnmarshalFromFunc()` function to create a custom unmarshaler. But there is also a `json.UnmarshalFunc()` function you could use instead.

Instead of creating a custom unmarshaler that reads a token from a `jsontext.Decoder`, it works on a `[]byte` slice containing the JSON value.

You could, if you want, redefine our `RuntimeUnmarshaler` using this alternative approach like so:

dego_vila@yahoo.com 30 Oct 2026

```
var RuntimeUnmarshaler = json.UnmarshalFunc(
    func(data []byte, val *Runtime) error {
        // Use strconv.Unquote() to remove the double quotes surrounding the JSON
        // string value.
        s, err := strconv.Unquote(string(data))
        if err != nil {
            return errors.New("invalid runtime value")
        }

        // Return an error if it doesn't have the suffix 'm'.
        if !strings.HasSuffix(s, "m") {
            return errors.New("invalid runtime value")
        }

        // Otherwise try to convert it to a time.Duration.
        d, err := time.ParseDuration(s)
        if err != nil {
            return fmt.Errorf("invalid runtime value: %w", err)
        }

        // And then convert the time.Duration to a Runtime type, and 'write
        // through' the value to the val pointer, just like before.
        *val = Runtime(d)
        return nil
    },
)
```

But, on balance, I think using `json.UnmarshalFromFunc()` and working with a `jsontext.Token` value is safer and clearer than working with a `[]byte` value. In particular, it is nice to be able to call `token.Kind()` to check the kind of the JSON value before processing it further, and it's helpful to be able to use methods like `token.String()` and `token.Int()` to get the JSON value as a specific Go type.

Using multiple custom unmarshalers

Similar to encoding, if you want to use more than one custom unmarshaler, you need to use the `json.JoinUnmarshalers` function to join the unmarshalers before passing them as an option to `json.UnmarshalRead()`. You *shouldn't* pass more than one `json.WithUnmarshalers` option when calling `json.UnmarshalRead()`.

For example, if you want to use the custom unmarshalers `fooUnmarshaler`, `barUnmarshaler` and `bazUnmarshaler`, then the correct pattern is to do this:

dego_vila@yahoo.com 30 Oct 2026

```
unmarshalers := json.JoinUnmarshalers(fooUnmarshaler, barUnmarshaler, bazUnmarshaler)

opts := []json.Options{
    json.RejectUnknownMembers(true),
    json.WithUnmarshalers(unmarshalers),
}

err := json.UnmarshalRead(r.Body, dst, opts...)
if err != nil {
    ...
}
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 4.5.

Validating JSON input

In many cases, you'll want to perform additional validation checks on the data from a client to make sure it meets your specific business rules before processing it. In this chapter, we'll illustrate how to do that in the context of a JSON API by updating our `createMovieHandler` to check that:

- The movie title provided by the client is not empty and is no more than 500 bytes long.
- The movie year is not empty and is between 1888 and the current year.
- The movie runtime is not empty and is a positive number.
- The movie has between one and five (unique) genres.

If any of those checks fail, we want to send the client a `422 Unprocessable Entity` response along with error messages which clearly describe the validation failures.

Creating a validator package

To help us with validation throughout this project, we're going to create a small `internal/validator` package with some simple reusable helper types and functions. If you're coding along, go ahead and create the following directory and file on your machine:

```
$ mkdir internal/validator
$ touch internal/validator/validator.go
```

Then in this new `internal/validator/validator.go` file, add the following code:

File: internal/validator/validator.go

```
package validator

import (
    "regexp"
    "slices"
)

// Declare a regular expression for sanity checking the format of email addresses (we'll
// use this later in the book). If you're interested, this regular expression pattern is
// taken from https://html.spec.whatwg.org/#valid-e-mail-address. Note: if you're
// reading this in PDF or EPUB format and cannot see the full pattern, please see the
// note further down the page.
var (
    EmailRX = regexp.MustCompile("^[a-zA-Z0-9.!#$%&'*+/=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\\.[a-zA-Z
)
```

dego_vila@yahoo.com 30 Oct 2026

```
// Define a new Validator type which contains a map of validation errors.
type Validator struct {
    Errors map[string]string
}

// New is a helper which creates a new Validator instance with an empty errors map.
func New() *Validator {
    return &Validator{Errors: make(map[string]string)}
}

// Valid returns true if the errors map doesn't contain any entries.
func (v *Validator) Valid() bool {
    return len(v.Errors) == 0
}

// AddError adds an error message to the map (so long as no entry already exists for
// the given key).
func (v *Validator) AddError(key, message string) {
    if _, exists := v.Errors[key]; !exists {
        v.Errors[key] = message
    }
}

// Check adds an error message to the map only if a validation check is not 'ok'.
func (v *Validator) Check(ok bool, key, message string) {
    if !ok {
        v.AddError(key, message)
    }
}

// Generic function which returns true if a specific value is in a list of permitted
// values.
func PermittedValue[T comparable](value T, permittedValues ...T) bool {
    return slices.Contains(permittedValues, value)
}

// Matches returns true if a string value matches a specific regexp pattern.
func Matches(value string, rx *regexp.Regexp) bool {
    return rx.MatchString(value)
}

// Generic function which returns true if all values in a slice are unique.
func Unique[T comparable](values []T) bool {
    uniqueValues := make(map[T]bool)

    for _, value := range values {
        uniqueValues[value] = true
    }

    return len(values) == len(uniqueValues)
}
```

To summarize this:

In the code above we've defined a custom `Validator` type which contains a map of errors. The `Validator` type provides a `Check()` method for conditionally adding errors to the map, and a `Valid()` method which returns whether the errors map is empty or not. We've also added `PermittedValue()`, `Matches()` and `Unique()` functions to help us perform some specific validation checks.

dego_vila@yahoo.com 30 Oct 2026

Conceptually this `Validator` type is quite basic, but that's not a bad thing. As we'll see over the course of this book, it's surprisingly powerful in practice and gives us a lot of flexibility and control over validation checks and how we perform them.

If you're reading this book in PDF or EPUB format and you can't see the full `EmailRX` regexp pattern in the code snippet above, here it is broken up into multiple lines:

```
"^[a-zA-Z0-9.!#$%&'*+/=?^_`{|}~-]+@[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?
(?:\\.[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)*$"
```

In your code, this regexp pattern should all be on a single line with no whitespace.

Performing validation checks

Alright, let's start putting the `Validator` type to use!

The first thing we need to do is update our `cmd/api/errors.go` file to include a new `failedValidationResponse()` helper, which writes a `422 Unprocessable Entity` and the contents of the errors map from our new `Validator` type as a JSON response body.

File: cmd/api/errors.go

```
package main

...

// Note that the errors parameter here has the type map[string]string, which is exactly
// the same as the errors map contained in our Validator type.
func (app *application) failedValidationResponse(w http.ResponseWriter, r *http.Request, errors map[string]string) {
    app.errorResponse(w, r, http.StatusUnprocessableEntity, errors)
}
```

Then once that's done, head back to your `createMovieHandler` and update it to perform the necessary validation checks on the `input` struct. Like so:

File: cmd/api/movies.go

```
package main

import (
    "errors"
    "fmt"
    "net/http"
    "time"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator" // New import
)
```

dego_vila@yahoo.com 30 Oct 2026

```
}

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title   string         `json:"title"`
        Year    int            `json:"year"`
        Runtime models.Runtime `json:"runtime"`
        Genres  []string       `json:"genres"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Initialize a new Validator instance.
    v := validator.New()

    // Use the Check() method to execute our validation checks. This will add the
    // provided key and error message to the errors map if the check does not evaluate
    // to true. For example, in the first line here we "check that the title is not
    // equal to the empty string". In the second, we "check that the length of the title
    // is less than or equal to 500 bytes" and so on.
    v.Check(input.Title != "", "title", "must be provided")
    v.Check(len(input.Title) <= 500, "title", "must not be more than 500 bytes long")

    v.Check(input.Year != 0, "year", "must be provided")
    v.Check(input.Year >= 1888, "year", "must be greater than 1888")
    v.Check(input.Year <= time.Now().Year(), "year", "must not be in the future")

    v.Check(input.Runtime != 0, "runtime", "must be provided")
    v.Check(input.Runtime > 0, "runtime", "must be a positive number")

    v.Check(input.Genres != nil, "genres", "must be provided")
    v.Check(len(input.Genres) >= 1, "genres", "must contain at least 1 genre")
    v.Check(len(input.Genres) <= 5, "genres", "must not contain more than 5 genres")
    // Note that we're using the Unique helper in the line below to check that all
    // values in the input.Genres slice are unique.
    v.Check(validator.Unique(input.Genres), "genres", "must not contain duplicate values")

    // Use the Valid() method to see if any of the checks failed. If they did, then use
    // the failedValidationResponse() helper to send a response to the client, passing
    // in the v.Errors map.
    if !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}

...
```

With that done, we should be good to try this out. Restart the API, then go ahead and issue a request to the `POST /v1/movies` endpoint containing some invalid data. Similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"title":"","year":1000,"runtime":"-123m","genres":["sci-fi","sci-fi"]}'
$ curl -i -d "$BODY" localhost:4000/v1/movies
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json
Date: Thu, 10 Sep 2026 10:48:31 GMT
Content-Length: 179

{
    "error": {
        "genres": "must not contain duplicate values",
        "runtime": "must be a positive number",
        "title": "must be provided",
        "year": "must be greater than 1888"
    }
}
```

That's looking great. Our validation checks are working to prevent the request from being executed successfully — and even better — the client is getting a well-formed JSON response with clear, informative, error messages for each problem.

You can also try sending a valid request, if you like. You should find that the checks pass successfully and the `input` struct is dumped in the HTTP response, just like before:

```
$ BODY='{"title":"Moana","year":2016,"runtime":"107m","genres":["animation","adventure"]}'
$ curl -i -d "$BODY" localhost:4000/v1/movies
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 10:49:36 GMT
Content-Length: 75
Content-Type: text/plain; charset=utf-8

{Title:Moana Year:2016 Runtime:6420000000000 Genres:[animation adventure]}
```

Making validation rules reusable

In large projects, it's likely that you'll want to reuse some of the same validation checks in multiple places. In our case — for example — we'll want to use many of these same checks later when a client *edits* the movie data.

To prevent duplication, we can collect the validation checks for a movie into a standalone `ValidateMovie()` function. In theory this function could live almost anywhere in our codebase — next to the handlers in the `cmd/api/movies.go` file, or possibly in the `internal/validators` package. But personally, I like to keep the validation checks close to the relevant domain type in the `internal/models` package.

If you're following along, reopen the `internal/models/movies.go` file and add a `ValidateMovie()` function containing the checks like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

import (
    "time"

    "greenlight.alexedwards.net/internal/validator" // New import
)

type Movie struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"-"`
    Title     string    `json:"title"`
    Year      int       `json:"year,omitzero"`
    Runtime   Runtime   `json:"runtime,omitzero"`
    Genres    []string  `json:"genres,omitzero"`
    Version   int       `json:"version"`
}

func ValidateMovie(v *validator.Validator, movie Movie) {
    v.Check(movie.Title != "", "title", "must be provided")
    v.Check(len(movie.Title) <= 500, "title", "must not be more than 500 bytes long")

    v.Check(movie.Year != 0, "year", "must be provided")
    v.Check(movie.Year >= 1888, "year", "must be greater than 1888")
    v.Check(movie.Year <= time.Now().Year(), "year", "must not be in the future")

    v.Check(movie.Runtime != 0, "runtime", "must be provided")
    v.Check(movie.Runtime > 0, "runtime", "must be a positive number")

    v.Check(movie.Genres != nil, "genres", "must be provided")
    v.Check(len(movie.Genres) >= 1, "genres", "must contain at least 1 genre")
    v.Check(len(movie.Genres) <= 5, "genres", "must not contain more than 5 genres")
    v.Check(validator.Unique(movie.Genres), "genres", "must not contain duplicate values")
}
```

> **Important:** Notice that the validation checks are now being performed *on a* `Movie` *struct* — not on the `input` struct in our handlers.

Once that's done, we need to head back to our `createMovieHandler` and update it to initialize a new `Movie` struct, copy across the data from our `input` struct, and then call this new validation function. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title   string         `json:"title"`
        Year    int            `json:"year"`
        Runtime models.Runtime `json:"runtime"`
        Genres  []string       `json:"genres"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Copy the values from the input struct to a new Movie struct.
    movie := models.Movie{
        Title:   input.Title,
        Year:    input.Year,
        Runtime: input.Runtime,
        Genres:  input.Genres,
    }

    // Initialize a new Validator.
    v := validator.New()

    // Call the ValidateMovie() function, and if any checks fail, return a response
    // containing the errors.
    if models.ValidateMovie(v, movie); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}

...
```

When you're looking at this code, there might be a couple of questions in your head.

Firstly, you might be wondering why we're initializing the `Validator` instance in our handler and passing it to the `ValidateMovie()` function — rather than initializing it in `ValidateMovie()` and passing it back as a return value.

This is because as our application gets more complex we will need to call *multiple* validation helpers from our handlers, rather than just one like we are above. So using a pattern of initializing the `Validator` in the handler, and then passing it around, gives us more

dego_vila@yahoo.com 30 Oct 2026

flexibility.

You might also be wondering why we're decoding the JSON request into the `input` struct and then copying the data into the `Movie` struct, rather than just decoding into the `Movie` struct directly.

The problem with decoding directly into a `Movie` struct is that a client could provide the keys `id` and `version` in their JSON request, and the corresponding values would be decoded into the `ID` and `Version` fields of the `Movie` struct *without any error* — even though we don't want the client to ever send `id` or `version` fields in the request.

> **Note:** There are a couple of open proposals which might make it possible to tag specific struct fields as 'read-only' when decoding JSON in the future — in which case we could tag the `ID` and `Version` fields as 'read-only' and decode directly into the `Movie` struct. But for now, decoding into an intermediary `input` struct (like we are in our handler) is a clean, simple, and robust approach — albeit a little bit repetitive.

OK, with those explanations out of the way, you should be able to start the application again and things should work the same as before from the client's perspective. If you make an invalid request, you should get a response containing the error messages similar to this:

```
$ BODY='{"title":"","year":1000,"runtime":"-123m","genres":["sci-fi","sci-fi"]}'
$ curl -i -d "$BODY" localhost:4000/v1/movies
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json
Date: Thu, 10 Sep 2026 10:55:02 GMT
Content-Length: 179

{
    "error": {
        "genres": "must not contain duplicate values",
        "runtime": "must be a positive number",
        "title": "must be provided",
        "year": "must be greater than 1888"
    }
}
```

Feel free to play around with this, and try sending different values in the JSON until you're happy that all the validation checks are working as expected.

dego_vila@yahoo.com 30 Oct 2026

Chapter 5.

Database setup and configuration

In this next section of the book, we're going to move forward with our project build and set up a SQL database to persistently store our movie data.

Unlike the first *Let's Go* book (where we used MySQL as our database), this time we're going to use PostgreSQL. It's open-source, very reliable and has some helpful modern features — including support for array and JSON data types, full-text search, and geospatial queries. We'll use a couple of these modern PostgreSQL features as we progress through our build.

In this section you'll learn:

- How to install and set up PostgreSQL on your local machine.
- How to use the `psql` interactive tool to create databases, PostgreSQL extensions and user accounts.
- How to initialize a database connection pool in Go and configure its settings to improve performance and stability.

dego_vila@yahoo.com 30 Oct 2026

Chapter 5.1.

Setting up PostgreSQL

Installing PostgreSQL

If you're following along, you'll need to install PostgreSQL on your computer at this point. The official PostgreSQL documentation contains comprehensive download and installation instructions for all types of operating systems, but if you're using macOS you should be able to install the latest version with:

```
$ brew install postgresql@18
```

Or if you're using a Linux distribution you should be able to install PostgreSQL via your package manager. For example, if your OS supports the `apt` package manager (like Debian and Ubuntu do) you can install it with:

```
$ sudo apt install postgresql
```

On Windows machines you can install PostgreSQL using the Chocolatey package manager with the command:

```
> choco install postgresql
```

Connecting to the PostgreSQL interactive terminal

When PostgreSQL was installed, a `psql` binary should also have been created on your computer. This contains a terminal-based front-end for working with PostgreSQL.

You can check that this is available by running the `psql --version` command from your terminal like so:

```
$ psql --version
psql (PostgreSQL) 18.4 (Ubuntu 18.4-1.pgdg22.04+1)
```

If you're not already familiar with PostgreSQL, the process for connecting to it for the first time using `psql` can be a bit unintuitive. So let's take a moment to walk through it.

dego_vila@yahoo.com 30 Oct 2026

When PostgreSQL is freshly installed it only has one user account: a superuser called `postgres`. In the first instance, we need to connect to PostgreSQL as this superuser to do anything — and from there we can perform any setup steps that we need to, like creating a database and creating other users.

During installation, an *operating system* user named `postgres` should also have been created on your machine. On Unix-based systems you can check your `/etc/passwd` file to confirm this, like so:

```
$ cat /etc/passwd | grep 'postgres'
postgres:x:127:134:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

This is important because, by default, PostgreSQL uses an authentication scheme called *peer authentication* for any connections from the local machine. Peer authentication means that if the current operating system user's username matches a valid PostgreSQL user username, they can log in to PostgreSQL as that user with no further authentication. There are no passwords involved.

So it follows that if we switch to the *operating system* user called `postgres`, we should be able to connect to PostgreSQL using `psql` without needing any further authentication. In fact, you can do both these things in one step with the following command:

```
$ sudo -u postgres psql
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
Type "help" for help.

postgres=#
```

So, just to confirm, what we've done here is use the `sudo` command (*superuser do*) to run the `psql` command as the *operating system user* called `postgres`. That opens a session in the interactive terminal-based front-end, where we're authenticated as the PostgreSQL superuser `postgres`.

If you want, you can confirm this by running a `"SELECT current_user"` query to see which PostgreSQL user you currently are:

```
postgres=# SELECT current_user;
 current_user
--------------
 postgres
(1 row)
```

dego_vila@yahoo.com 30 Oct 2026

Creating databases, users, and extensions

While we're connected as the `postgres` superuser, let's create a new database for our project called `greenlight`, and then connect to it using the `\c` command like so:

```
postgres=# CREATE DATABASE greenlight;
CREATE DATABASE
postgres=# \c greenlight
You are now connected to database "greenlight" as user "postgres".
greenlight=#
```

> **Hint:** In PostgreSQL the `\` character indicates a *meta command*. Some other useful meta commands are `\l` to list all databases, `\dt` to list tables, and `\du` to list users. You can also run `\?` to see the full list of available meta commands.

Now that our `greenlight` database exists and we're connected to it, there are a couple of tasks we need to complete.

The first task is to create a new `greenlight` user, without superuser permissions, with *password-based* authentication, instead of peer authentication. We'll use this user to execute SQL migrations and connect to the database from our Go application. In order to run the migrations successfully, we'll also need to set this user to be the owner of the `greenlight` database.

PostgreSQL also has the concept of *extensions*, which add additional features on top of the standard functionality. A list of the extensions that ship with PostgreSQL can be found here, and there are also some others that you can download separately.

In this project we're going to use the `citext` extension. This adds a case-insensitive character string type to PostgreSQL, which we will use later in the book to store user email addresses.

Go ahead and run the following commands to create a new `greenlight` user with a specific password, set them as the owner of the `greenlight` database, and also add the `citext` extension to our database:

```
greenlight=# CREATE ROLE greenlight WITH LOGIN PASSWORD 'pa55word';
CREATE ROLE
greenlight=# ALTER DATABASE greenlight OWNER TO greenlight;
ALTER DATABASE
greenlight=# CREATE EXTENSION IF NOT EXISTS citext;
CREATE EXTENSION
```

dego_vila@yahoo.com 30 Oct 2026

> **Important:** If you're following along, make sure to keep a mental note of the password you set for the `greenlight` user. You'll need it in the upcoming steps.

Once that's successfully done, you can type `exit` or `\q` to close the terminal-based front-end and revert to being your normal operating system user.

```
greenlight=# exit
```

Connecting as the new user

Before we go any further, let's prove to ourselves that everything is set up correctly and try connecting to the `greenlight` database as the `greenlight` user. When prompted, enter the password that you set in the step above.

```
$ psql --host=localhost --dbname=greenlight --username=greenlight
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT current_user;
 current_user
--------------
 greenlight
(1 row)

greenlight=> exit
```

Great! That confirms that our database and the new `greenlight` user with password credentials are working correctly, and that we're able to execute SQL statements as that user without any issues.

Additional information

Optimizing PostgreSQL settings

The default settings that PostgreSQL ships with are quite conservative, and you can often improve the performance of your database by tweaking the values in your `postgresql.conf` file.

dego_vila@yahoo.com 30 Oct 2026

You can check where your `postgresql.conf` file lives with the following SQL query:

```
$ sudo -u postgres psql -c 'SHOW config_file;'
               config_file
----------------------------------------
 /etc/postgresql/15/main/postgresql.conf
(1 row)
```

This article provides a good introduction to some of the most important PostgreSQL settings, and guidance on what values are reasonable to use as a starting point. If you're interested in optimizing PostgreSQL, I recommend giving this a read.

Alternatively, you can use this web-based tool to generate suggested values based on your available system hardware. A nice feature of this tool is that it also outputs `ALTER SYSTEM` SQL statements, which you can run against your database to change the settings instead of altering your `postgresql.conf` file manually.

dego_vila@yahoo.com 30 Oct 2026

Connecting to PostgreSQL

OK, now that our new `greenlight` database is set up, let's look at how to connect to it from our Go application.

As you probably remember from *Let's Go*, to work with a SQL database we need to use a *database driver* to act as a 'middleman' between Go and the database itself. You can find a list of available drivers for PostgreSQL in the Go wiki, but for our project we'll opt for the popular, reliable, and well-established `pq` package.

If you're coding along with this book, then go ahead and use `go get` to download the latest `v1.N.N` release of `pq` like so:

```
$ go get github.com/lib/pq@v1
go: downloading github.com/lib/pq v1.12.3
go get: added github.com/lib/pq v1.12.3
```

To connect to the database we'll also need a *data source name* (DSN), which is basically a string that contains the necessary connection parameters. The exact format of the DSN will depend on which database driver you're using (and should be described in the driver documentation), but when using `pq` you should be able to connect to your local `greenlight` database as the `greenlight` user with the following DSN:

```
postgres://greenlight:pa55word@localhost/greenlight
```

Establishing a connection pool

The code that we'll use for connecting to the `greenlight` database from our Go application is almost exactly the same as in the first *Let's Go* book. So we won't dwell on the details, and hopefully this will all feel very familiar.

At a high level:

- We want the DSN to be configurable at runtime, so we will pass it to the application using a command-line flag rather than hard-coding it. For simplicity during development, we'll use the DSN above as the default value for the flag.
- In our `cmd/api/main.go` file we'll create a new `openDB()` helper function. In this helper

dego_vila@yahoo.com 30 Oct 2026

> we'll use the `sql.Open()` function to establish a new `sql.DB` connection pool, then — because connections to the database are established lazily as and when needed for the first time — we also need to use the `db.PingContext()` method to actually create a connection and verify that everything is set up correctly.

Let's head back to our `cmd/api/main.go` file and update it like so:

File: cmd/api/main.go

```
package main

import (
    "context"      // New import
    "database/sql" // New import
    "flag"
    "fmt"
    "log/slog"
    "net/http"
    "os"
    "time"

    // Import the pq driver so that it can register itself with the database/sql
    // package. Note that we alias this import to the blank identifier, to stop the Go
    // compiler complaining that the package isn't being used.
    _ "github.com/lib/pq"
)

const version = "1.0.0"

// Add a db struct field to hold the configuration settings for our database connection
// pool. For now this only holds the DSN, which we will read in from a command-line flag.
type config struct {
    port int
    env  string
    db   struct {
        dsn string
    }
}

type application struct {
    config config
    logger *slog.Logger
}

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    // Read the DSN value from the db-dsn command-line flag into the config struct. We
    // default to using our development DSN if no flag is provided.
    flag.StringVar(&cfg.db.dsn, "db-dsn", "postgres://greenlight:pa55word@localhost/greenlight", "PostgreSQL DSN")

    flag.Parse()

    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

    // Call the openDB() helper function (see below) to create the connection pool,
    // passing in the config struct as an argument. If this returns an error, we log
    // it and exit the application immediately.
```

dego_vila@yahoo.com 30 Oct 2026

```
    db, err := openDB(cfg)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }

    // Defer a call to db.Close() so that the connection pool is closed before the
    // main() function exits.
    defer db.Close()

    // Also log a message to say that the connection pool has been successfully
    // established.
    logger.Info("database connection pool established")

    app := &application{
        config: cfg,
        logger: logger,
    }

    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", cfg.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(logger.Handler(), slog.LevelError),
    }

    logger.Info("starting server", "addr", srv.Addr, "env", cfg.env)

    // Because the err variable is now already declared in the code above, we need
    // to use the = operator here, instead of the := operator.
    err = srv.ListenAndServe()
    logger.Error(err.Error())
    os.Exit(1)
}

// The openDB() function returns a sql.DB connection pool.
func openDB(cfg config) (*sql.DB, error) {
    // Use sql.Open() to create an empty connection pool, using the DSN from the config
    // struct.
    db, err := sql.Open("postgres", cfg.db.dsn)
    if err != nil {
        return nil, err
    }

    // Create a context with a 5-second timeout deadline.
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    // Use PingContext() to establish a new connection to the database, passing in the
    // context we created above as a parameter. If the connection couldn't be
    // established successfully within the 5-second deadline, then this will return an
    // error. If we get this error, or any other, we close the connection pool and
    // return the error.
    err = db.PingContext(ctx)
    if err != nil {
        db.Close()
        return nil, err
    }

    // Return the sql.DB connection pool.
    return db, nil
}
```

dego_vila@yahoo.com 30 Oct 2026

> **Important:** We'll talk about using `context` to manage timeouts properly later in the book, so don't worry about this too much at the moment. For now, it's sufficient to know that if the `PingContext()` call could not complete successfully in 5 seconds, then it will return an error.

Once the `cmd/api/main.go` file is updated, go ahead and run the application again. You should now see a log message on startup confirming that the connection pool has been successfully established. Similar to this:

```
$ go run ./cmd/api
time=2026-09-11T15:02:59.768+02:00 level=INFO msg="database connection pool established"
time=2026-09-11T15:02:59.768+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

> **Note:** If you receive the error message `pq: SSL is not enabled on the server` you should set your DSN to: `postgres://greenlight:pa55word@localhost/greenlight?sslmode=disable`.

Decoupling the DSN

At the moment the default command-line flag value for our DSN is explicitly included as a string in the `cmd/api/main.go` file.

Even though the username and password in the DSN are just for the development database on your local machine, it would be preferable to not have this information hard-coded into our project files (which could be shared or distributed in the future).

So let's take some steps to decouple the DSN from our project code and instead store it as an environment variable on your local machine.

If you're following along, create a new `GREENLIGHT_DB_DSN` environment variable by adding the following line to either your `$HOME/.profile` or `$HOME/.bashrc` files:

```
File: $HOME/.profile

...

export GREENLIGHT_DB_DSN='postgres://greenlight:pa55word@localhost/greenlight'
```

Once that's done you'll need to reboot your computer, or — if that's not convenient right

dego_vila@yahoo.com 30 Oct 2026

now — run the `source` command on the file that you've just edited to effect the change. For example:

```
$ source $HOME/.profile
```

> **Note:** Running `source` will affect *the current terminal window only*. So, if you switch to a different terminal window, you'll need to run it again for it to take effect.

Either way, once you've rebooted or run `source` you should be able to see the value for the `GREENLIGHT_DB_DSN` environment variable in your terminal by running the `echo` command. Like so:

```
$ echo $GREENLIGHT_DB_DSN
postgres://greenlight:pa55word@localhost/greenlight
```

Now let's update our `cmd/api/main.go` file to access the environment variable using the `os.Getenv()` function, and set this as the default value for our DSN command-line flag.

It's fairly straightforward in practice:

File: cmd/api/main.go

```
package main

...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    // Use the value of the GREENLIGHT_DB_DSN environment variable as the default value
    // for our db-dsn command-line flag.
    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")

    flag.Parse()

    ...
}
```

If you restart the application again now, you should find that it compiles correctly and works just like before.

You can also try specifying the `-help` flag when running the application. This should output the descriptive text and default values for our three command-line flags, including the DSN

dego_vila@yahoo.com 30 Oct 2026

value pulled through from the environment variable. Similar to this:

```
$ go run ./cmd/api -help
Usage of /home/alex/.cache/go-build/86/86254c40f9b4d6163e9508e779a7d48be2cae84eb7f4e887f59d7235102e0b35-d/api:
  -db-dsn string
        PostgreSQL DSN (default "postgres://greenlight:pa55word@localhost/greenlight")
  -env string
        Environment (development|staging|production) (default "development")
  -port int
        API server port (default 4000)
```

Additional information

Using the DSN with psql

A nice side effect of storing the DSN in an environment variable is that you can use it to easily connect to the `greenlight` database as the `greenlight` user, rather than specifying all the connection options manually when running `psql`. Like so:

```
$ psql $GREENLIGHT_DB_DSN
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=>
```

dego_vila@yahoo.com 30 Oct 2026

Configuring the database connection pool

In the first *Let's Go* book we talked through the `sql.DB` connection pool at a high level and demonstrated the core principles of how to use it. But in this chapter we're going to go in-depth — explaining how the connection pool works behind the scenes, and exploring the settings we can use to change and optimize its behavior.

> **Note:** This chapter contains quite a lot of theory, and although it's interesting, it's not *critical* to the application build. If you find it too heavy-going, feel free to skim over it for now and then circle back to it later.

So how *does* the `sql.DB` connection pool work?

The most important thing to understand is that a `sql.DB` pool contains two types of connections — 'in-use' connections and 'idle' connections. A connection is marked as in-use when you are using it to perform a database task, such as executing a SQL statement or querying rows, and when the task is complete the connection is then marked as idle.

When you instruct Go to perform a database task, it will first check if any idle connections are available in the pool. If one is available, then Go will reuse this existing connection and mark it as in-use for the duration of the task. If there are no idle connections in the pool when you need one, then Go will create a new additional connection.

When Go reuses an idle connection from the pool, any problems with the connection are handled gracefully. Bad connections will automatically be retried twice before giving up, at which point Go will remove the bad connection from the pool and create a new one to carry out the task.

Configuring the pool

The connection pool has four methods that we can use to configure its behavior. Let's talk through them one-by-one.

The SetMaxOpenConns method

dego_vila@yahoo.com 30 Oct 2026

The `SetMaxOpenConns()` method allows you to set an upper `MaxOpenConns` limit on the number of 'open' connections (in-use + idle connections) in the pool. By default, the number of open connections is unlimited.

> **Important:** Note that 'open' connections are equal to 'in-use' plus 'idle' connections — *it is not* just 'in-use' connections.

Broadly speaking, the higher that you set the `MaxOpenConns` limit, the more database queries can be performed concurrently and the lower the risk is that the *connection pool itself* will be a bottleneck in your application.

But leaving it unlimited isn't necessarily the best thing to do. By default PostgreSQL has a hard limit of 100 open connections and, if this hard limit is hit under heavy load, it will cause our `pq` driver to return a `"sorry, too many clients already"` error.

> **Note:** The hard limit on open connections can be changed in your `postgresql.conf` file using the `max_connections` setting.

To avoid this error, it makes sense limit the number of open connections in our pool to comfortably below 100 — leaving enough headroom for any other applications or sessions that also need to use PostgreSQL.

The other benefit of setting a `MaxOpenConns` limit is that it acts as a very rudimentary throttle, and prevents the database from being swamped by a huge number of tasks all at once.

But setting a limit comes with an important caveat. If the `MaxOpenConns` limit is reached, and all connections are in-use, then any further database tasks will be forced to wait until a connection becomes free and marked as idle. In the context of our API, the user's HTTP request could 'hang' indefinitely while waiting for a free connection. So to mitigate this, it's important to always set a timeout on database tasks using the `context.Context` type. We'll explain how to do that later in the book.

The SetMaxIdleConns method

The `SetMaxIdleConns()` method sets an upper `MaxIdleConns` limit on the number of idle connections in the pool. By default, the maximum number of idle connections is 2.

In theory, allowing a higher number of idle connections in the pool will improve performance because it makes it less likely that a new connection needs to be established

dego_vila@yahoo.com 30 Oct 2026

from scratch — therefore helping to save resources.

But it's also important to realize that keeping an idle connection alive comes at a cost. It takes up memory which can otherwise be used for your application and database, and it's also possible that if a connection is idle for too long then it may become unusable. For example, by default MySQL will automatically close any connections which haven't been used for 8 hours.

So, potentially, setting `MaxIdleConns` too high may result in more connections becoming unusable and more resources being used than if you had a smaller idle connection pool — with fewer connections that are used more frequently. As a guideline: *you only want to keep a connection idle if you're likely to be using it again soon*.

Another thing to point out is that the `MaxIdleConns` limit should always be less than or equal to `MaxOpenConns`. Go enforces this and will automatically reduce the `MaxIdleConns` limit if necessary.

The SetConnMaxLifetime method

The `SetConnMaxLifetime()` method sets the `ConnMaxLifetime` limit — the maximum length of time that a connection can be reused for. By default, there's no maximum lifetime and connections will be reused forever.

If we set `ConnMaxLifetime` to one hour, for example, it means that all connections will be marked as 'expired' one hour after they were first created, and cannot be reused after they've expired. But note:

- This doesn't guarantee that a connection will exist in the pool for a whole hour; it's possible that a connection will become unusable for some reason and be automatically closed before then.
- A connection can still be in use more than one hour after being created — it just cannot *start* to be reused after that time.
- This isn't an idle timeout. The connection will expire one hour after it was first created — not one hour after it last became idle.
- Once every second Go runs a background cleanup operation to remove expired connections from the pool.

In theory, leaving `ConnMaxLifetime` unlimited (or setting a long lifetime) will help performance because it makes it less likely that new connections will need to be created from scratch. But in certain situations, it can be useful to enforce a shorter lifetime. For example:

dego_vila@yahoo.com 30 Oct 2026

- If your SQL database enforces a maximum lifetime on connections, it makes sense to set `ConnMaxLifetime` to a slightly shorter value.
- To help facilitate swapping databases gracefully behind a load balancer.

If you do decide to set a `ConnMaxLifetime` on your pool, it's important to bear in mind the frequency at which connections will expire (and subsequently be recreated). For example, if you have 100 open connections in the pool and a `ConnMaxLifetime` of 1 minute, then your application can potentially kill and recreate up to 1.67 connections (on average) every second. You don't want the frequency to be so great that it ultimately hinders performance.

The SetConnMaxIdleTime method

The `SetConnMaxIdleTime()` method sets the `ConnMaxIdleTime` limit. This works in a very similar way to `ConnMaxLifetime`, except it sets the maximum length of time that a connection can be *idle* for before it is marked as expired. By default there's no limit.

If we set `ConnMaxIdleTime` to 1 hour, for example, any connections that have sat idle in the pool for 1 hour since last being used will be marked as expired and removed by the background cleanup operation.

This setting is really useful because it means that we can set a relatively high limit on the number of idle connections in the pool, but periodically free up resources by removing any idle connections that we know aren't really being used anymore.

Putting it into practice

So that's a lot of information to take in… and what does it mean in practice? Let's summarize all the above into some actionable points.

1. As a rule of thumb, you should explicitly set a `MaxOpenConns` value. This should be comfortably below any hard limits on the number of connections imposed by your database and infrastructure, and you may also want to consider keeping it fairly low to act as a rudimentary throttle.

   For this project we'll set a `MaxOpenConns` limit of 25 connections. I've found this to be a reasonable starting point for small-to-medium web applications and APIs, but ideally you should tweak this value for your hardware depending on the results of benchmarking and load-testing.

2. In general, higher `MaxOpenConns` and `MaxIdleConns` values will lead to better performance. But the returns are diminishing, and you should be aware that having a

dego_vila@yahoo.com 30 Oct 2026

too-large idle connection pool (with connections that are not frequently reused) can actually lead to reduced performance and unnecessary resource consumption.

Because `MaxIdleConns` should always be less than or equal to `MaxOpenConns`, we'll also limit `MaxIdleConns` to 25 connections for this project.

3. To mitigate the risk from point 2 above, you should generally set a `ConnMaxIdleTime` value to remove idle connections that haven't been used for a long time. In this project we'll set a `ConnMaxIdleTime` duration of 15 minutes.

4. It's probably OK to leave `ConnMaxLifetime` as unlimited, unless your database imposes a hard limit on connection lifetime, or you need it specifically to facilitate something like gracefully swapping databases. Neither of those things apply in this project, so we'll leave this as the default unlimited setting.

Configuring the connection pool

Rather than hard-coding these settings, let's update the `cmd/api/main.go` file to accept them as command-line flags.

The command-line flag for the `ConnMaxIdleTime` value is particularly interesting, because we want it to convey a *duration of time*, like `5s` (5 seconds) or `10m` (10 minutes). To assist with this we can use the `flag.DurationVar()` function to read in the command-line flag value, which will automatically convert it to a `time.Duration` type for us.

Go ahead and update the `cmd/api/main.go` file as follows:

File: cmd/api/main.go

```
package main

...

// Add maxOpenConns, maxIdleConns and maxIdleTime fields to hold the configuration
// settings for the connection pool.
type config struct {
    port int
    env  string
    db   struct {
        dsn          string
        maxOpenConns int
        maxIdleConns int
        maxIdleTime  time.Duration
    }
}

...

func main() {
```

dego_vila@yahoo.com 30 Oct 2026

```
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")

    // Read the connection pool settings from command-line flags into the config struct.
    // Note that the default values we're using are the ones we discussed above.
    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    flag.Parse()

    ...
}

func openDB(cfg config) (*sql.DB, error) {
    db, err := sql.Open("postgres", cfg.db.dsn)
    if err != nil {
        return nil, err
    }

    // Set the maximum number of open (in-use + idle) connections in the pool. Note that
    // passing a value less than or equal to 0 will mean there is no limit.
    db.SetMaxOpenConns(cfg.db.maxOpenConns)

    // Set the maximum number of idle connections in the pool. Again, passing a value
    // less than or equal to 0 will mean there is no limit.
    db.SetMaxIdleConns(cfg.db.maxIdleConns)

    // Set the maximum idle timeout for connections in the pool. Passing a duration less
    // than or equal to 0 will mean that connections are not closed due to their idle time.
    db.SetConnMaxIdleTime(cfg.db.maxIdleTime)

    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    err = db.PingContext(ctx)
    if err != nil {
        db.Close()
        return nil, err
    }

    return db, nil
}
```

If you go ahead and run the application again now, everything should still work correctly. For the `db-max-idle-time` flag, you can pass in any value acceptable to the `time.ParseDuration()` function, such as `300ms` (300 milliseconds), `5s` (5 seconds) or `2h45m` (2 hours and 45 minutes). The valid time units are `ns`, `us` (or `µs`), `ms`, `s`, `m` and `h`.

For example:

```
$ go run ./cmd/api -db-max-open-conns=50 -db-max-idle-conns=50 -db-max-idle-time=2h30m
```

At this point you won't really notice any obvious changes to the application, and there's not

dego_vila@yahoo.com 30 Oct 2026

much we can do to demonstrate the impact of these settings. But later in the book we'll do a bit of load testing, and explain how to monitor the state of the connection pool in real-time using the `db.Stats()` method. At that point, you'll be able to see some of the behavior that we've talked about in this chapter in action.

dego_vila@yahoo.com 30 Oct 2026

Chapter 6.

SQL migrations

Let's get back to something a bit more concrete and tangible in this next section of the book, and take steps to create a `movies` table in our `greenlight` database.

To do this, we could simply use the `psql` tool again and run the necessary `CREATE TABLE` statement against our database. But instead, we're going to explore how to use *SQL migrations* to create the table (and more generally, to manage database schema changes throughout the project).

You'll learn about:

- The high-level principles behind SQL migrations and why they are useful.
- How to use the command-line `migrate` tool to programmatically manage changes to your database schema.

dego_vila@yahoo.com 30 Oct 2026

An overview of SQL migrations

In case you're not familiar with the idea of SQL migrations, at a very high level the concept works like this:

1. For every change that you want to make to your database schema (like creating a table, adding a column, or removing an unused index) you create a pair of *migration files*. One file is the 'up' migration which contains the SQL statements necessary to implement the change, and the other is a 'down' migration which contains the SQL statements to reverse (or *roll back*) the change.

2. Each pair of migration files is numbered sequentially, usually `0001, 0002, 0003...` or with a Unix timestamp, to indicate the order in which migrations should be applied to a database.

3. You use some kind of tool or script to execute or roll back the SQL statements in the sequential migration files against your database. The tool keeps track of which migrations have already been applied, so that only the necessary SQL statements are actually executed.

Using migrations to manage your database schema, rather than manually executing the SQL statements yourself, has a few benefits:

- The database schema (along with its evolution and changes) is completely described by the 'up' and 'down' SQL migration files. And because these are just regular files containing some SQL statements, they can be included and tracked alongside the rest of your code in a version control system.

- It's possible to replicate the current database schema precisely on another machine by running the necessary 'up' migrations. This is a big help when you need to manage and synchronize database schemas in different environments (development, testing, production, etc.).

- It's possible to roll back database schema changes if necessary by applying the appropriate 'down' migrations.

Installing the migrate tool

dego_vila@yahoo.com 30 Oct 2026

To manage SQL migrations in this project we're going to use the `migrate` command-line tool.

Detailed installation instructions for different operating systems can be found here, but on macOS you should be able to install it with the command:

```
$ brew install golang-migrate
```

And on Linux and Windows, the easiest method is to download a pre-built binary and move it to a location on your system path. For example, on Linux:

```
$ cd /tmp
$ curl -L https://github.com/golang-migrate/migrate/releases/download/v4.20.1/migrate.linux-amd64.tar.gz | tar xvz
$ mv migrate ~/go/bin/
```

Before you continue, please check that it's available and working on your machine by trying to execute the `migrate` binary with the `-version` flag. It should output the current version number similar to this:

```
$ migrate -version
4.20.1
```

Additional information

Adding migrate as a module tool dependency

The `migrate` tool itself is written in Go, which means that — in theory — you could use the `go get -tool` command to add `migrate` as a tool dependency to your `go.mod` file, instead of installing it as a binary like we have done.

But we haven't done this for two reasons:

1. At the time of writing, it's not possible to actually run `migrate` as a tool dependency due to the fact that the `go tool` command doesn't currently support build tags. There's an open issue about this here.

2. The `migrate` codebase has a *ton* of child dependencies. If you add it to your `go.mod` as a tool dependency, and you also want to vendor dependencies (which we will do later in

dego_vila@yahoo.com 30 Oct 2026

the book), it will bloat your repository by around 280MB.

dego_vila@yahoo.com 30 Oct 2026

Working with SQL migrations

Now that the `migrate` tool is installed, let's illustrate how to use it by creating a new `movies` table in our database.

The first thing we need to do is generate a pair of *migration files* using the `migrate create` command. If you're following along, go ahead and run the following command in your terminal:

```
$ migrate create -seq -ext=.sql -dir=./migrations create_movies_table
/home/alex/Projects/greenlight/migrations/000001_create_movies_table.up.sql
/home/alex/Projects/greenlight/migrations/000001_create_movies_table.down.sql
```

In this command:

- The `-seq` flag indicates that we want to use sequential numbering like `0001, 0002, ...` for the migration files (instead of a Unix timestamp, which is the default).
- The `-ext` flag indicates that we want to give the migration files the extension `.sql`.
- The `-dir` flag indicates that we want to store the migration files in the `./migrations` directory (which will be created automatically if it doesn't already exist).
- The name `create_movies_table` is a descriptive label that we give the migration files to signify their contents.

If you look in your `migrations` directory, you should now see a pair of new 'up' and 'down' migration files like so:

```
./migrations/
├── 000001_create_movies_table.down.sql
└── 000001_create_movies_table.up.sql
```

At the moment these two new files are completely empty. Let's edit the 'up' migration file to contain the necessary `CREATE TABLE` statement for our `movies` table, like so:

dego_vila@yahoo.com 30 Oct 2026

File: migrations/000001_create_movies_table.up.sql

```
CREATE TABLE IF NOT EXISTS movies (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    created_at timestamp(0) with time zone NOT NULL DEFAULT NOW(),
    title text NOT NULL,
    year integer NOT NULL,
    runtime bigint NOT NULL,
    genres text[] NOT NULL,
    version integer NOT NULL DEFAULT 1
);
```

Notice here how the fields and types in this table are analogous to the fields and types in the `Movie` struct that we created earlier? This is important because it means we'll be able to easily map the data in each row of our `movies` table to a single `Movie` struct in our Go code.

If you're not familiar with the different PostgreSQL data types in the SQL statement above, the official documentation provides a comprehensive overview. But the most important things to point out are:

- The `id` column is configured to be the primary key for the table. The `GENERATED ALWAYS AS IDENTITY` clause instructs PostgreSQL to assign a new, automatically-incrementing, integer value to it for every inserted row, starting at 1 by default.
- It's widely considered good practice to use the `bigint` type (which has a maximum value of `9223372036854775807`) for primary key columns, instead of the regular `integer` type (which has a maximum value of `2147483647`).
- We've also used a `bigint` type for the `runtime` column. This is because our Go code stores the movie runtime as a `Runtime` type with underlying type `time.Duration`, which in turn is an integer value in *nanoseconds*. A movie runtime represented in nanoseconds will generally exceed the maximum value of an `integer` column, so we use `bigint` instead to ensure there is enough space to store it.
- The `genres` column has the type `text[]` which is an array of zero-or-more `text` values. It's important to note that arrays in PostgreSQL are themselves queryable and indexable, which is something that we'll demonstrate later in the book.
- You might remember from *Let's Go* that working with `NULL` values in Go can be awkward, and where possible it's easiest to just set `NOT NULL` constraints on every table column along with appropriate `DEFAULT` values — like we have above.
- For storing strings we're using the `text` type, instead of the alternative `varchar` or `varchar(n)` types. If you're interested, this excellent blog post explains why `text` is generally the best character type to use in PostgreSQL.

dego_vila@yahoo.com 30 Oct 2026

Alright, let's move on to the 'down' migration and add the SQL statements needed to reverse the 'up' migration that we just wrote.

File: migrations/000001_create_movies_table.down.sql

```
DROP TABLE IF EXISTS movies;
```

The `DROP TABLE` command in PostgreSQL always removes any indexes and constraints that exist for the target table, so this single statement is sufficient to reverse the 'up'.

Great, that's our first pair of migration files ready to go!

While we are at it, let's also create a second pair of migration files containing `CHECK` constraints to enforce some of our business rules at the database-level. Specifically, we want to make sure that the `runtime` value is always greater than zero, the `year` value is between 1888 and the current year, and the `genres` array always contains between 1 and 5 items.

Again, if you're following along, run the command below to create a second pair of migration files:

```
$ migrate create -seq -ext=.sql -dir=./migrations add_movies_check_constraints
/home/alex/Projects/greenlight/migrations/000002_add_movies_check_constraints.up.sql
/home/alex/Projects/greenlight/migrations/000002_add_movies_check_constraints.down.sql
```

And then add the following SQL statements to add and drop the `CHECK` constraints respectively:

File: migrations/000002_add_movies_check_constraints.up.sql

```
ALTER TABLE movies ADD CONSTRAINT movies_runtime_check CHECK (runtime >= 0);

ALTER TABLE movies ADD CONSTRAINT movies_year_check CHECK (year BETWEEN 1888 AND date_part('year', now()));

ALTER TABLE movies ADD CONSTRAINT genres_length_check CHECK (array_length(genres, 1) BETWEEN 1 AND 5);
```

File: migrations/000002_add_movies_check_constraints.down.sql

```
ALTER TABLE movies DROP CONSTRAINT IF EXISTS movies_runtime_check;

ALTER TABLE movies DROP CONSTRAINT IF EXISTS movies_year_check;

ALTER TABLE movies DROP CONSTRAINT IF EXISTS genres_length_check;
```

dego_vila@yahoo.com 30 Oct 2026

When we insert or update data in our `movies` table, if any of these checks fail our database driver will return an error similar to this:

```
pq: new row for relation "movies" violates check constraint "movies_year_check"
```

> **Note:** It's perfectly acceptable for a single migration file to contain multiple SQL statements, as we see in the two files above. In fact, we could have just included the `CHECK` constraints along with the `CREATE TABLE` statement in the first pair of migration files, but for the purpose of this book having them in a separate second migration helps us to illustrate how the `migrate` tool works.

Executing the migrations

Alright, now we're ready to run the two 'up' migrations against our `greenlight` database.

If you're following along, go ahead and use the following command to execute the migrations, passing in the database DSN from your environment variable. If everything is set up correctly, you should see some output confirming that the migrations have been executed successfully. Similar to this:

```
$ migrate -path=./migrations -database=$GREENLIGHT_DB_DSN up
1/u create_movies_table (38.19761ms)
2/u add_movies_check_constraints (63.284269ms)
```

At this point, it's worth opening a connection to your database and listing the tables with the `\dt` meta command:

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> \dt
               List of relations
 Schema |       Name        | Type  |   Owner
--------+-------------------+-------+------------
 public | movies            | table | greenlight
 public | schema_migrations | table | greenlight
(2 rows)
```

You should see that the `movies` table has been created, along with a `schema_migrations`

dego_vila@yahoo.com 30 Oct 2026

table, both of which are owned by the `greenlight` user.

The `schema_migrations` table is automatically generated by the `migrate` tool and used to track which migrations have been applied. Let's take a quick look inside it:

```
greenlight=> SELECT * FROM schema_migrations;
 version | dirty
---------+-------
       2 | f
(1 row)
```

The `version` column here indicates that our migration files up to (and including) number `2` in the sequence have been executed against the database. The value of the `dirty` column is `false`, which indicates that the migration files were cleanly executed *without any errors* and the SQL statements they contain were successfully applied *in full*.

If you like, you can also run the `\d` meta command on the `movies` table to see the structure of the table and confirm that the `CHECK` constraints were created correctly. Like so:

```
greenlight-> \d movies
                                  Table "public.movies"
   Column   |            Type             | Collation | Nullable |           Default
------------+-----------------------------+-----------+----------+-----------------------------
 id         | bigint                      |           | not null | generated always as identity
 created_at | timestamp(0) with time zone |           | not null | now()
 title      | text                        |           | not null |
 year       | integer                     |           | not null |
 runtime    | bigint                      |           | not null |
 genres     | text[]                      |           | not null |
 version    | integer                     |           | not null | 1
Indexes:
    "movies_pkey" PRIMARY KEY, btree (id)
Check constraints:
    "genres_length_check" CHECK (array_length(genres, 1) >= 1 AND array_length(genres, 1) <= 5)
    "movies_runtime_check" CHECK (runtime >= 0)
    "movies_year_check" CHECK (year >= 1888 AND year::double precision <= date_part('year'::text, now()))
```

Additional information

Migrating to a specific version

As an alternative to looking at the `schema_migrations` table, if you want to see which migration version your database is currently on you can run the `migrate` tool's `version` command, like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ migrate -path=./migrations -database=$EXAMPLE_DSN version
2
```

You can also migrate up or down to a specific version by using the `goto` command:

```
$ migrate -path=./migrations -database=$EXAMPLE_DSN goto 1
```

Executing down migrations

You can use the `down` command to roll back by a specific number of migrations. For example, to roll back the *most recent migration* you would run:

```
$ migrate -path=./migrations -database =$EXAMPLE_DSN down 1
```

Personally, I generally prefer to use the `goto` command to perform roll backs (as it's more explicit about the target version) and reserve use of the `down` command for rolling-back *all migrations*, like so:

```
$ migrate -path=./migrations -database=$EXAMPLE_DSN down
Are you sure you want to apply all down migrations? [y/N]
y
Applying all down migrations
2/d create_bar_table (39.988791ms)
1/d create_foo_table (59.460276ms)
```

Another variant of this is the `drop` command, which will remove all tables from the database including the `schema_migrations` table — but the database itself will remain, along with anything else that has been created like sequences and enums. Because of this, using `drop` can leave your database in a messy and unknown state, and it's generally better to stick with the `down` command if you want to roll back everything.

Fixing errors in SQL migrations

It's important to talk about what happens when you make a syntax error in your SQL migration files, because the behavior of the `migrate` tool can be a bit confusing to start with.

When you run a migration that contains an error, all SQL statements up to the erroneous one will be applied and then the `migrate` tool will exit with a message describing the error. Similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ migrate -path=./migrations -database=$EXAMPLE_DSN up
1/u create_foo_table (36.6328ms)
2/u create_bar_table (71.835442ms)
error: migration failed: syntax error at end of input in line 0: CREATE TABLE (details: pq: syntax error at end of input)
```

If the migration file which failed contained multiple SQL statements, then it's possible that the migration file was *partially* applied before the error was encountered. In turn, this means that the database is in an unknown state as far as the `migrate` tool is concerned.

Accordingly, the `version` field in the `schema_migrations` field will contain the number for the *failed migration* and the `dirty` field will be set to `true`. At this point, if you run another migration (even a 'down' migration) you will get an error message similar to this:

```
Dirty database version {X}. Fix and force version.
```

What you need to do is investigate the original error and figure out if the migration file which failed was partially applied. If it was, then you need to manually roll back the partially applied migration.

Once that's done, then you must also 'force' the `version` number in the `schema_migrations` table to the correct value. For example, to force the database version number to `1` you should use the `force` command like so:

```
$ migrate -path=./migrations -database=$EXAMPLE_DSN force 1
```

Once you force the version, the database is considered 'clean' and you should be able to run migrations again without any problem.

Remote migration files

The `migrate` tool also supports reading migration files from *remote sources* including Amazon S3 and GitHub repositories. For example:

```
$ migrate -source="s3://<bucket>/<path>" -database=$EXAMPLE_DSN up
$ migrate -source="github://owner/repo/path#ref" -database=$EXAMPLE_DSN up
$ migrate -source="github://user:personal-access-token@owner/repo/path#ref" -database=$EXAMPLE_DSN up
```

More information about this functionality and a full list of the supported remote resources can be found here.

Running migrations on application startup

dego_vila@yahoo.com 30 Oct 2026

If you want, it is also possible to use the `golang-migrate/migrate` Go package (not the command-line tool) to automatically execute your database migrations on application startup.

We *won't* be using this approach in the book, so if you're following along please don't change your code. But roughly, the pattern looks like this:

```
package main

import (
    "context"
    "database/sql"
    "flag"
    "fmt"
    "log"
    "net/http"
    "os"
    "time"

    "github.com/golang-migrate/migrate/v4"                  // New import
    "github.com/golang-migrate/migrate/v4/database/postgres" // New import
    _ "github.com/golang-migrate/migrate/v4/source/file"     // New import
    _ "github.com/lib/pq"
)

func main() {
    ...

    db, err := openDB(cfg)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
    defer db.Close()

    logger.Info("database connection pool established")

    migrationDriver, err := postgres.WithInstance(db, &postgres.Config{})
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }

    migrator, err := migrate.NewWithDatabaseInstance("file:///path/to/your/migrations", "postgres", migrationDriver)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }

    err = migrator.Up()
    if err != nil && err != migrate.ErrNoChange {
        logger.Error(err.Error())
        os.Exit(1)
    }

    logger.Info("database migrations applied")

    ...
}
```

dego_vila@yahoo.com 30 Oct 2026

Although this works — and it might initially seem appealing — tightly coupling the execution of migrations with your application source code can potentially be limiting and problematic in the longer term.

The article decoupling database migrations from server startup provides a good discussion on this, and I recommend reading it if this is a topic that you're interested in. It's Python-focused, but don't let that put you off — the same principles apply in Go applications too.

dego_vila@yahoo.com 30 Oct 2026

Chapter 7.

CRUD operations

In this next section of the book we're going to focus on building up the functionality for *creating, reading, updating and deleting* movies in our system.

We'll make quite rapid progress over the next few chapters, and by the end of this section we'll have the following API endpoints finished and working in full:

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PUT	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie
DELETE	/v1/movies/:id	deleteMovieHandler	Delete a specific movie

In this section you'll learn:

- How to create a *database model* that isolates all the logic for executing SQL queries against your database.
- How to implement the four basic CRUD (create, read, update and delete) operations on a specific resource in the context of an API.

dego_vila@yahoo.com 30 Oct 2026

Chapter 7.1.

Setting up the movie model

In this chapter we're going to set up the skeleton code for our movie *database model*, which will handle reading and writing movie records to and from our PostgreSQL database.

Let's head back to the `internal/models/movies.go` file and create a `MovieModel` struct type and some placeholder methods for performing basic CRUD (create, read, update and delete) actions against our `movies` database table.

File: internal/models/movies.go

```
package models

import (
    "database/sql" // New import
    "time"

    "greenlight.alexedwards.net/internal/validator"
)

...

// Define a MovieModel struct type which wraps a sql.DB connection pool.
type MovieModel struct {
    DB *sql.DB
}

// Add a placeholder method for inserting a new record in the movies table.
func (m MovieModel) Insert(movie Movie) (Movie, error) {
    return Movie{}, nil
}

// Add a placeholder method for fetching a specific record from the movies table.
func (m MovieModel) Get(id int) (Movie, error) {
    return Movie{}, nil
}

// Add a placeholder method for updating a specific record in the movies table.
func (m MovieModel) Update(movie Movie) (Movie, error) {
    return Movie{}, nil
}

// Add a placeholder method for deleting a specific record from the movies table.
func (m MovieModel) Delete(id int) error {
    return nil
}
```

As an additional step, we're going to wrap our `MovieModel` in a parent `Models` struct. Doing this is totally optional, but it has the benefit of giving you a convenient single 'container' which can hold and represent *all* your database models as your application grows.

dego_vila@yahoo.com 30 Oct 2026

If you're following along, create a new `internal/models/models.go` file and add the following code:

```
$ touch internal/models/models.go
```

File: internal/models/models.go

```
package models

import (
    "database/sql"
    "errors"
)

// Define a custom ErrRecordNotFound error. We'll return this from our Get() method when
// looking up a movie that doesn't exist in our database.
var (
    ErrRecordNotFound = errors.New("record not found")
)

// Create a Models struct which wraps the MovieModel. We'll add other models to this,
// like a UserModel and PermissionModel, as our build progresses.
type Models struct {
    Movies MovieModel
}

// For ease of use, we also add a NewModels() method which returns a Models struct containing
// the initialized MovieModel.
func NewModels(db *sql.DB) Models {
    return Models{
        Movies: MovieModel{DB: db},
    }
}
```

And now, let's edit our `cmd/api/main.go` file so that the `Models` struct is initialized in our `main()` function, and then passed to our handlers as a dependency. Like so:

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "flag"
    "fmt"
    "log/slog"
    "net/http"
    "os"
    "time"

    "greenlight.alexedwards.net/internal/models" // New import

    _ "github.com/lib/pq"
)

...
```

dego_vila@yahoo.com 30 Oct 2026

```
// Add a models field to hold our new Models struct.
type application struct {
    config config
    logger *slog.Logger
    models models.Models
}

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")

    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    flag.Parse()

    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

    db, err := openDB(cfg)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }

    defer db.Close()

    logger.Info("database connection pool established")

    // Use the models.NewModels() function to initialize a Models struct, passing in the
    // connection pool as a parameter.
    app := &application{
        config: cfg,
        logger: logger,
        models: models.NewModels(db),
    }

    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", cfg.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(logger.Handler(), slog.LevelError),
    }

    logger.Info("starting server", "addr", srv.Addr, "env", cfg.env)

    err = srv.ListenAndServe()
    logger.Error(err.Error())
    os.Exit(1)
}

...
```

If you want, you can try restarting the application at this point. You should find that the code compiles and runs successfully.

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/api
time=2026-09-11T15:49:28.016+02:00 level=INFO msg="database connection pool established"
time=2026-09-11T15:49:28.016+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

One of the nice things about this pattern is that the code to execute actions on our `movies` table will be very clear and readable from the perspective of our API handlers. For example, we'll be able to execute the `Insert()` method by simply writing:

```
app.models.Movies.Insert(...)
```

The general structure is also easy to extend. When we create more database models in the future, all we have to do is include them in the `Models` struct and they will automatically be available to our API handlers.

dego_vila@yahoo.com 30 Oct 2026

Creating a new movie

Let's begin with the `Insert()` method of our database model and update this to create a new record in our `movies` table. Specifically, we want it to execute the following SQL query:

```
INSERT INTO movies (title, year, runtime, genres)
VALUES ($1, $2, $3, $4)
RETURNING id, created_at, version
```

There are a few things about this query which warrant a bit of explanation.

- It uses `$N` notation to represent *placeholder parameters* for the data that we want to insert in the `movies` table. As we explained in *Let's Go*, every time that you pass untrusted input data from a client to a SQL database it's important to use placeholder parameters to help prevent SQL injection attacks, unless you have a very specific reason for not using them.
- We're only inserting values for `title`, `year`, `runtime` and `genres`. The remaining columns in the `movies` table will be filled with *system-generated* values when the record is inserted — the `id` will be an auto-incrementing integer, and the `created_at` and `version` values will default to the current time and `1` respectively.
- At the end of the query we have a `RETURNING` clause. This is a PostgreSQL-specific clause (it's not part of the SQL standard) that you can use to return values from any record that is being manipulated by an `INSERT`, `UPDATE` or `DELETE` statement. In this query we're using it to return the system-generated `id`, `created_at` and `version` values.

Executing the SQL query

Throughout this project we'll stick with using Go's `database/sql` package to execute our database queries, rather than using a third-party ORM or other tool. We talked about how to use `database/sql` and its various features, behaviors and gotchas in *Let's Go*, so hopefully this will feel familiar to you. Consider it a short refresher!

Normally, you would use Go's `Exec()` method to execute an `INSERT` statement against a database table. But because our SQL query is returning a single row of data (thanks to the `RETURNING` clause), we'll need to use the `QueryRow()` method here instead.

dego_vila@yahoo.com 30 Oct 2026

Head back to your `internal/models/movies.go` file and update it like so:

File: internal/models/movies.go

```
package models

import (
    "database/sql"
    "time"

    "greenlight.alexedwards.net/internal/validator"

    "github.com/lib/pq" // New import
)

...

// The Insert() method accepts a movie struct which should contain the data for the
// new record.
func (m MovieModel) Insert(movie Movie) (Movie, error) {
    // Define a SQL query which inserts a new record in the movies table, and returns
    // the system-generated data.
    query := `
        INSERT INTO movies (title, year, runtime, genres)
        VALUES ($1, $2, $3, $4)
        RETURNING id, created_at, version`

    // Create an args slice containing the values for the placeholder parameters.
    // Declaring this slice immediately next to our SQL query helps to make it nice
    // and clear *what values are being used where* in the query.
    args := []any{movie.Title, movie.Year, movie.Runtime, pq.Array(movie.Genres)}

    // Use the QueryRow() method to execute the SQL query on our connection pool,
    // passing in the elements of the args slice as variadic arguments and scanning
    // the system-generated id, created_at and version values into the movie struct.
    err := m.DB.QueryRow(query, args...).Scan(&movie.ID, &movie.CreatedAt, &movie.Version)

    // Return the updated movie struct containing the system-generated data.
    return movie, err
}

...
```

That code is nice and succinct, but there are a few important things to mention.

The first thing is the the `Insert()` method returns a copy of the `Movie` struct containing the `id`, `created_at` and `version` values that were automatically generated by PostgreSQL. We'll use this updated struct in our `createMovieHandler` when sending the JSON response.

The next thing is the placeholder parameter inputs, which we declare in an `args` slice like this:

```
args := []any{movie.Title, movie.Year, movie.Runtime, pq.Array(movie.Genres)}
```

Storing the inputs in a slice isn't strictly necessary, but as mentioned in the code comments

dego_vila@yahoo.com 30 Oct 2026

above it's a nice pattern that can help the clarity of your code. Personally, I usually do this for SQL queries with more than three placeholder parameters.

Also, notice the final value in the slice? In order to store our `movie.Genres` value (which is a `[]string` slice) in the database, we need to pass it through the `pq.Array()` adapter function before executing the SQL query.

Behind the scenes, the `pq.Array()` adapter takes our `[]string` slice and converts it to a `pq.StringArray` type. In turn, the `pq.StringArray` type implements the `driver.Valuer` and `sql.Scanner` interfaces necessary to translate our native `[]string` slice to and from a value that our PostgreSQL database can understand and store in a `text[]` array column.

> **Hint:** You can also use the `pq.Array()` adapter function in the same way with `[]bool`, `[]byte`, `[]int`, `[]float32` and `[]float64` slices in your Go code.

Hooking it up to our API handler

Now for the exciting part. Let's hook up the `Insert()` method to our `createMovieHandler` so that our `POST /v1/movies` endpoint works in full. Like so:

File: cmd/api/movies.go

```
package main

...

func (app *application) createMovieHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title   string         `json:"title"`
        Year    int            `json:"year"`
        Runtime models.Runtime `json:"runtime"`
        Genres  []string       `json:"genres"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    movie := models.Movie{
        Title:   input.Title,
        Year:    input.Year,
        Runtime: input.Runtime,
```

dego_vila@yahoo.com 30 Oct 2026

```
        Genres:  input.Genres,
    }

    v := validator.New()

    if models.ValidateMovie(v, movie); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Call the Insert() method on our movies model, passing in the validated
    // movie struct. This will create a record in the database and return an
    // updated copy of the movie struct containing the system-generated information.
    movie, err = app.models.Movies.Insert(movie)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // When sending an HTTP response, we want to include a Location header to let the
    // client know which URL they can find the newly-created resource at. We make an
    // empty http.Header map and then use the Set() method to add a new Location header,
    // interpolating the system-generated ID for our new movie in the URL.
    headers := make(http.Header)
    headers.Set("Location", fmt.Sprintf("/v1/movies/%d", movie.ID))

    // Write a JSON response with a 201 Created status code, the movie data in the
    // response body, and the Location header.
    err = app.writeJSON(w, http.StatusCreated, envelope{"movie": movie}, headers)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}

...
```

OK, let's try this out!

Restart the API, then open up a second terminal window and make the following request to the `POST /v1/movies` endpoint:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"title":"Moana","year":2016,"runtime":"107m", "genres":["animation","adventure"]}'
$ curl -i -d "$BODY" localhost:4000/v1/movies
HTTP/1.1 201 Created
Content-Type: application/json
Location: /v1/movies/1
Date: Fri, 11 Sep 2026 13:53:16 GMT
Content-Length: 152

{
    "movie": {
        "id": 1,
        "title": "Moana",
        "year": 2016,
        "runtime": "107m",
        "genres": [
            "animation",
            "adventure"
        ],
        "version": 1
    }
}
```

That's looking perfect. We can see that the JSON response contains all the information for the new movie, including the system-generated ID and version numbers. And the response also includes the `Location: /v1/movies/1` header, pointing to the URL that will later represent the movie in our system.

Creating additional records

While we're at it, let's create a few more records in the system to help us demonstrate different functionality as our build progresses.

If you're coding along, please run the following commands to create three more movie records in the database:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"title":"Black Panther","year":2018,"runtime":"134m","genres":["action","adventure"]}'
$ curl -d "$BODY" localhost:4000/v1/movies
{
    "movie": {
        "id": 2,
        "title": "Black Panther",
        "year": 2018,
        "runtime": "134m",
        "genres": [
            "action",
            "adventure"
        ],
        "version": 1
    }
}

$ BODY='{"title":"The Odyssey","year":2026, "runtime":"172m","genres":["action","adventure","fantasy"]}'
$ curl -d "$BODY" localhost:4000/v1/movies
{
    "movie": {
        "id": 3,
        "title": "The Odyssey",
        "year": 2026,
        "runtime": "172m",
        "genres": [
            "action",
            "adventure",
            "fantasy"
        ],
        "version": 1
    }
}

$ BODY='{"title":"The Breakfast Club","year":1986, "runtime":"96m","genres":["drama"]}'
$ curl -d "$BODY" localhost:4000/v1/movies
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1986,
        "runtime": "96m",
        "genres": [
            "drama"
        ],
        "version": 1
    }
}
```

At this point you might also want to take a look at PostgreSQL to confirm that the records have been created properly. You should see that the contents of the `movies` table now looks similar to this (including the appropriate movie `genres` in an array).

dego_vila@yahoo.com 30 Oct 2026

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT * FROM movies;
 id |       created_at       |       title        | year |    runtime     |           genres           | version
----+------------------------+--------------------+------+----------------+----------------------------+---------
  1 | 2026-09-11 15:53:16+02 | Moana              | 2016 |  6420000000000 | {animation,adventure}      |       1
  2 | 2026-09-11 15:54:22+02 | Black Panther      | 2018 |  8040000000000 | {action,adventure}         |       1
  3 | 2026-09-11 15:54:34+02 | The Odyssey        | 2026 | 10320000000000 | {action,adventure,fantasy} |       1
  4 | 2026-09-11 15:54:44+02 | The Breakfast Club | 1986 |  5760000000000 | {drama}                    |       1
(4 rows)
```

Additional information

$N notation

A nice feature of the PostgreSQL placeholder parameter `$N` notation is that you can use the same parameter value in multiple places in your SQL statement. For example, it's perfectly acceptable to write code like this:

```
// This SQL statement uses the $1 parameter twice, and the value `123` will be used in
// both locations where $1 appears.
stmt := "UPDATE foo SET bar = $1 + $2 WHERE bar = $1"
err := db.Exec(stmt, 123, 456)
if err != nil {
    ...
}
```

Executing multiple statements

Occasionally you might find yourself in the position where you want to execute more than one SQL statement in the same database call, like this:

```
stmt := `
    UPDATE foo SET bar = true;
    UPDATE foo SET baz = false;`

err := db.Exec(stmt)
if err != nil {
    ...
}
```

Having multiple statements in the same call is supported by the `pq` driver, *so long as the*

dego_vila@yahoo.com 30 Oct 2026

statements do not contain any placeholder parameters. If they do contain placeholder parameters, then you'll receive the following error message at runtime:

```
pq: cannot insert multiple commands into a prepared statement
```

To work around this, you will need to either split out the statements into separate database calls, or if that's not possible, you can create a custom function in PostgreSQL which acts as a wrapper around the multiple SQL statements that you want to run.

dego_vila@yahoo.com 30 Oct 2026

Chapter 7.3.

Fetching a movie

Now let's move on to the code for *fetching and displaying the data for a specific movie* .

Again, we'll begin in our database model here, and start by updating the `Get()` method to execute the following SQL query:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE id = $1
```

Since our `movies` table uses the `id` column as its primary key, this query will only ever return exactly one database row (or none at all). So, it's appropriate for us to execute this query using Go's `QueryRow()` method again.

If you're following along, open up your `internal/models/movies.go` file and update it like so:

File: internal/models/movies.go

```
package models

import (
    "database/sql"
    "errors" // New import
    "time"

    "greenlight.alexedwards.net/internal/validator"

    "github.com/lib/pq"
)

...

func (m MovieModel) Get(id int) (Movie, error) {
    // The PostgreSQL id value that we're using for the movie ID starts
    // auto-incrementing at 1 by default, so we know that no movies will have ID values
    // less than that. To avoid making an unnecessary database call, we take a shortcut
    // and return an ErrRecordNotFound error straight away.
    if id < 1 {
        return Movie{}, ErrRecordNotFound
    }

    // Define the SQL query for retrieving the movie data.
    query := `
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE id = $1`

    // Declare a Movie struct to hold the data returned by the query.
    var movie Movie
```

dego_vila@yahoo.com 30 Oct 2026

```
    var movie Movie

    // Execute the query using the QueryRow() method, passing in the provided id value
    // as a placeholder parameter, and scan the response data into the fields of the
    // Movie struct. Importantly, note that we need to convert the scan target for the
    // genres column using the pq.Array() adapter function again.
    err := m.DB.QueryRow(query, id).Scan(
        &movie.ID,
        &movie.CreatedAt,
        &movie.Title,
        &movie.Year,
        &movie.Runtime,
        pq.Array(&movie.Genres),
        &movie.Version,
    )

    // Handle any errors. If there was no matching movie found, Scan() will return
    // a sql.ErrNoRows error. We check for this and return our custom ErrRecordNotFound
    // error instead.
    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return Movie{}, ErrRecordNotFound
        default:
            return Movie{}, err
        }
    }

    // Otherwise, return the Movie struct.
    return movie, nil
}

...
```

Hopefully the code above should feel clear and familiar — it's a straight lift of the pattern that we discussed in detail in *Let's Go*.

The only real thing of note is the fact that we need to use the `pq.Array()` adapter again when scanning in the genres data from the PostgreSQL `text[]` array. If we didn't use this adapter, we would get the following error at runtime:

```
sql: Scan error on column index 5, name "genres": unsupported Scan, storing driver.Value type []uint8 into type *[]string
```

Updating the API handler

OK, the next thing we need to do is update our `showMovieHandler` so that it calls the `Get()` method we just made. The handler should check whether `Get()` returns an `ErrRecordNotFound` error — and if it does, the client should be sent a `404 Not Found` response. Otherwise, we can go ahead and render the returned `Movie` struct in a JSON response.

Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

import (
    "errors" // New import
    "fmt"
    "net/http"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"
)

...

func (app *application) showMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        app.notFoundResponse(w, r)
        return
    }

    // Call the Get() method to fetch the data for a specific movie. We also need to
    // use the errors.Is() function to check if it returns a models.ErrRecordNotFound
    // error, in which case we send a 404 Not Found response to the client.
    movie, err := app.models.Movies.Get(id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    err = app.writeJSON(w, http.StatusOK, envelope{"movie": movie}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Great! That's all nice and succinct, thanks to the structure and helpers that we've already put in place.

Feel free to give this a try by restarting the API and looking up a movie that you've already created in the database. For example:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i localhost:4000/v1/movies/2
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 11 Sep 2026 14:12:22 GMT
Content-Length: 157

{
    "movie": {
        "id": 2,
        "title": "Black Panther",
        "year": 2018,
        "runtime": "134m",
        "genres": [
            "action",
            "adventure"
        ],
        "version": 1
    }
}
```

And likewise, you can also try making a request with a movie ID that doesn't exist in the database yet (but is otherwise valid). In that scenario you should receive a `404 Not Found` response like so:

```
$ curl -i localhost:4000/v1/movies/42
HTTP/1.1 404 Not Found
Content-Type: application/json
Date: Fri, 11 Sep 2026 14:12:50 GMT
Content-Length: 58

{
    "error": "the requested resource could not be found"
}
```

dego_vila@yahoo.com 30 Oct 2026

Updating a movie

In this chapter, we'll continue building up our application and add a brand-new endpoint that allows clients to *update the data for a specific movie*.

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PUT	**/v1/movies/:id**	**updateMovieHandler**	**Update the details of a specific movie**

More precisely, we'll set up the endpoint so that a client can edit the `title`, `year`, `runtime` and `genres` values for a movie. In our project the `id` and `created_at` values should never change once they've been created, and the `version` value isn't something that the client should control, so we won't allow those fields to be edited.

For now, we'll configure this endpoint so that it performs a *full replacement* of the values for a movie. This means that the client will need to provide values for *all* editable fields in their JSON request body… even if they only want to change one of them.

For example, if a client wants to add the genre `sci-fi` to the movie *Black Panther* in our database, they would need to send a JSON request body which looks like this:

```
{
    "title": "Black Panther",
    "year": 2018,
    "runtime": "134m",
    "genres": [
            "action",
            "adventure",
            "sci-fi"
    ]
}
```

Executing the SQL query

Let's start in our database model again, and edit the `Update()` method to execute the

dego_vila@yahoo.com 30 Oct 2026

following SQL query:

```
UPDATE movies
SET title = $1, year = $2, runtime = $3, genres = $4, version = version + 1
WHERE id = $5
RETURNING version
```

Notice here that we're incrementing the `version` value as part of the query. And then at the end we're using the `RETURNING` clause to return this new, incremented, `version` value.

Like before, this query returns a single row of data so we'll also need to use Go's `QueryRow()` method to execute it. If you're following along, head back to your `internal/models/movies.go` file and fill in the `Update()` method like so:

File: internal/models/movies.go

```
package models

...

func (m MovieModel) Update(movie Movie) (Movie, error) {
    // Declare the SQL query for updating the record and returning the new version
    // number.
    query := `
        UPDATE movies
        SET title = $1, year = $2, runtime = $3, genres = $4, version = version + 1
        WHERE id = $5
        RETURNING version`

    // Create an args slice containing the values for the placeholder parameters.
    args := []any{
        movie.Title,
        movie.Year,
        movie.Runtime,
        pq.Array(movie.Genres),
        movie.ID,
    }

    // Use the QueryRow() method to execute the query, passing in the args slice as a
    // variadic parameter and scanning the new version value into the movie struct.
    err := m.DB.QueryRow(query, args...).Scan(&movie.Version)
    return movie, err
}

...
```

It's important to emphasize that — just like our `Insert()` method — the `Update()` method returns a copy of the `Movie` struct updated with the new system-generated version number.

Creating the API handler

dego_vila@yahoo.com 30 Oct 2026

Now let's head back to our `cmd/api/movies.go` file and update it to include the brand-new `updateMovieHandler` method.

Method	URL Pattern	Handler	Action
PUT	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie

The nice thing about this handler is that the groundwork is already in place — our job here really just involves linking up the code and helper functions that we've already written to handle the request.

Specifically, we'll need to:

1. Extract the movie ID from the URL using the `app.readIDParam()` helper.
2. Fetch the corresponding movie record from the database using the `Get()` method that we made in the previous chapter.
3. Read the JSON request body containing the updated movie data into an `input` struct.
4. Copy the data across from the `input` struct to the movie record.
5. Check that the updated movie record is valid using the `models.ValidateMovie()` function.
6. Call the `Update()` method to store the updated movie record in our database.
7. Write the updated movie data in a JSON response using the `app.writeJSON()` helper.

So let's go ahead and do exactly that:

File: cmd/api/movies.go

```
package main

...

func (app *application) updateMovieHandler(w http.ResponseWriter, r *http.Request) {
    // Extract the movie ID from the URL.
    id, err := app.readIDParam(r)
    if err != nil {
        app.notFoundResponse(w, r)
        return
    }

    // Fetch the existing movie record from the database, sending a 404 Not Found
    // response to the client if we couldn't find a matching record.
    movie, err := app.models.Movies.Get(id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
```

dego_vila@yahoo.com 30 Oct 2026

```
        return
    }

    // Declare an input struct to hold the expected data from the client.
    var input struct {
        Title   string         `json:"title"`
        Year    int            `json:"year"`
        Runtime models.Runtime `json:"runtime"`
        Genres  []string       `json:"genres"`
    }

    // Read the JSON request body data into the input struct.
    err = app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Copy the values from the request body to the appropriate fields of the movie
    // record.
    movie.Title = input.Title
    movie.Year = input.Year
    movie.Runtime = input.Runtime
    movie.Genres = input.Genres

    // Validate the updated movie record, sending the client a 422 Unprocessable Entity
    // response if any checks fail.
    v := validator.New()

    if models.ValidateMovie(v, movie); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Pass the updated movie record to our new Update() method.
    movie, err = app.models.Movies.Update(movie)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Write the updated movie record in a JSON response.
    err = app.writeJSON(w, http.StatusOK, envelope{"movie": movie}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Lastly, to finish this off, we also need to update our application routes to include the new endpoint. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    // Add the route for the PUT /v1/movies/:id endpoint.
    router.HandlerFunc(http.MethodPut, "/v1/movies/:id", app.updateMovieHandler)

    return app.recoverPanic(router)
}
```

Using the new endpoint

And with that, we're now ready to try this out!

To demonstrate, let's continue with the example we gave at the start of this chapter and update our record for *Black Panther* to include the genre `sci-fi`. As a reminder, the record currently looks like this:

```
$ curl localhost:4000/v1/movies/2
{
    "movie": {
        "id": 2,
        "title": "Black Panther",
        "year": 2018,
        "runtime": "134m",
        "genres": [
            "action",
            "adventure"
        ],
        "version": 1
    }
}
```

To make the update to the genres field we can execute the following API call:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"title":"Black Panther","year":2018,"runtime":"134m","genres":["sci-fi","action","adventure"]}'
$ curl -X PUT -d "$BODY" localhost:4000/v1/movies/2
{
    "movie": {
        "id": 2,
        "title": "Black Panther",
        "year": 2018,
        "runtime": "134m",
        "genres": [
            "sci-fi",
            "action",
            "adventure"
        ],
        "version": 2
    }
}
```

That's looking great — we can see from the response that the movie genres have been updated to include `"sci-fi"`, and the version number has been incremented to `2` like we would expect.

You can also verify that the change has been persisted by making a `GET /v1/movies/2` request again, like so:

```
$ curl localhost:4000/v1/movies/2
{
    "movie": {
        "id": 2,
        "title": "Black Panther",
        "year": 2018,
        "runtime": "134m",
        "genres": [
            "sci-fi",
            "action",
            "adventure"
        ],
        "version": 2
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 7.5.

Deleting a movie

In this chapter, we'll add our final CRUD endpoint so that a client can *delete a specific movie* from our system.

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PUT	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie
DELETE	**/v1/movies/:id**	**deleteMovieHandler**	**Delete a specific movie**

Compared to the other endpoints in our API, the behavior that we want to implement here is quite straightforward.

- If a movie with the `id` provided in the URL exists in the database, we want to delete the corresponding record and return a success message to the client.
- If the movie `id` doesn't exist, we want to return a `404 Not Found` response to the client.

The SQL query to delete the record in our database is also simple:

```
DELETE FROM movies
WHERE id = $1
```

In this case the SQL query returns no rows, so it's appropriate for us to use Go's `Exec()` method to execute it.

One of the nice things about `Exec()` is that it returns a `sql.Result` value, which contains information about the *number of rows that the query affected*. In our scenario, this is really useful information.

- If the number of rows affected is `1`, then we know that the movie existed in the table and has now been deleted… so we can send the client a success message.
- Conversely, if the number of rows affected is `0` we know that no movie with that `id` existed at the point we tried to delete it, and we can send the client a `404 Not Found`

dego_vila@yahoo.com 30 Oct 2026

response.

Adding the new endpoint

Let's go ahead and update the `Delete()` method in our database model. Essentially, we want this to execute the SQL query above and return an `ErrRecordNotFound` error if the number of rows affected is `0`. Like so:

File: internal/models/movies.go

```
package models

...

func (m MovieModel) Delete(id int) error {
    // Return an ErrRecordNotFound error if the movie ID is less than 1.
    if id < 1 {
        return ErrRecordNotFound
    }

    // Construct the SQL query to delete the record.
    query := `
        DELETE FROM movies
        WHERE id = $1`

    // Execute the SQL query using the Exec() method, passing in the id variable as
    // the value for the placeholder parameter. The Exec() method returns a sql.Result
    // value.
    result, err := m.DB.Exec(query, id)
    if err != nil {
        return err
    }

    // Call the RowsAffected() method on the sql.Result value to get the number of rows
    // affected by the query.
    rowsAffected, err := result.RowsAffected()
    if err != nil {
        return err
    }

    // If no rows were affected, we know that the movies table didn't contain a record
    // with the provided ID at the moment we tried to delete it. In that case we
    // return an ErrRecordNotFound error.
    if rowsAffected == 0 {
        return ErrRecordNotFound
    }

    return nil
}
```

Once that's done, let's head to our `cmd/api/movies.go` file and add a new `deleteMovieHandler` method. In this we need to read the movie ID from the request URL, call the `Delete()` method that we just made, and — based on the return value from `Delete()` — send the appropriate response to the client.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) deleteMovieHandler(w http.ResponseWriter, r *http.Request) {
    // Extract the movie ID from the URL.
    id, err := app.readIDParam(r)
    if err != nil {
        app.notFoundResponse(w, r)
        return
    }

    // Delete the movie from the database, sending a 404 Not Found response to the
    // client if there isn't a matching record.
    err = app.models.Movies.Delete(id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Return a 200 OK status code along with a success message.
    err = app.writeJSON(w, http.StatusOK, envelope{"message": "movie successfully deleted"}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

> **Note:** You may prefer to send an empty response body and a `204 No Content` status code here, rather than a `"movie successfully deleted"` message. It really depends on who your clients are — if they are humans, then sending a message similar to the above is a nice UX touch; if they are machines, then a `204 No Content` response is probably sufficient.

Finally, we need to hook up the new route in our `cmd/api/routes.go` file:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPut, "/v1/movies/:id", app.updateMovieHandler)
    // Add the route for the DELETE /v1/movies/:id endpoint.
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    return app.recoverPanic(router)
}
```

OK, let's restart the API and try this out by deleting *The Odyssey* from our movie database (this should have an ID of `3` if you've been following along). The delete operation should work without any problems, and you should receive the confirmation message like so:

```
$ curl -X DELETE localhost:4000/v1/movies/3
{
    "message": "movie successfully deleted"
}
```

If you repeat the same request to delete the already-deleted movie, you should now get a `404 Not Found` response and error message. Similar to this:

```
$ curl -X DELETE localhost:4000/v1/movies/3
{
    "error": "the requested resource could not be found"
}
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 8.

Advanced CRUD operations

In this section of the book we're going to look at a few more 'advanced' patterns that you might want to use in your API's CRUD endpoints.

You'll learn:

- How to support *partial* updates to a resource (so that the client only needs to send the data that they want to change).
- How to use *optimistic concurrency control* to avoid race conditions when two clients try to update the same resource at the same time.
- How to use *context timeouts* to terminate long-running database queries and prevent unnecessary resource use.

dego_vila@yahoo.com 30 Oct 2026

Handling partial updates

In this chapter we're going to change the behavior of the `updateMovieHandler` so that it supports *partial updates* of the movie records. Conceptually this is a little more complicated than making a *complete replacement*, which is why we laid the groundwork with that approach first.

As an example, let's say that we notice that the release year for *The Breakfast Club* is incorrect in our database (it should actually be 1985, not 1986). It would be nice if we could send a JSON request containing only the change that needs to be applied, instead of the full movie data, like so:

```
{"year": 1985}
```

Let's quickly look at what happens if we try to send this request right now:

```
$ curl -X PUT -d '{"year": 1985}' localhost:4000/v1/movies/4
{
    "error": {
        "genres": "must be provided",
        "runtime": "must be provided",
        "title": "must be provided"
    }
}
```

As we mentioned earlier in the book, when decoding the request body, any fields in our `input` struct which *don't* have a corresponding key-value pair in the JSON will retain their zero value. We happen to check for these zero values during validation and return the error messages that you see above.

In the context of a partial update, this causes a problem. How do we tell the difference between:

- A client *providing a key-value pair which has a zero-value value* — like `{"title": ""}` — in which case we want to return a validation error.
- A client *not providing a key-value pair* in their JSON at all — in which case we want to 'skip' updating the field but not send a validation error.

To help answer this, let's quickly remind ourselves of what the zero values are for different Go types.

dego_vila@yahoo.com 30 Oct 2026

Go type	Zero value
`int*`, `uint*`, `float*`, `complex`	`0`
`string`	`""`
`bool`	`false`
`func`, `array`, `slice`, `map`, `chan` and **pointers**	`nil`

The key thing to notice here is that *pointers have the zero value* `nil`.

So — in theory — we could change the fields in our `input` struct to be pointers. Then to see if a client has provided a particular key-value pair in the JSON, we can simply check whether the corresponding field in the `input` struct equals `nil` or not.

```
// Use pointers for the Title, Year and Runtime fields.
var input struct {
    Title   *string         `json:"title"`   // This will be nil if there is no corresponding key in the JSON.
    Year    *int            `json:"year"`    // Likewise...
    Runtime *models.Runtime `json:"runtime"` // Likewise...
    Genres  []string        `json:"genres"`  // We don't need to change this because slices already have the zero value nil.
}
```

Performing the partial update

Let's put this into practice, and edit our `updateMovieHandler` method so it supports partial updates as follows:

File: cmd/api/movies.go

```
package main

...

func (app *application) updateMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        app.notFoundResponse(w, r)
        return
    }

    // Retrieve the movie record as normal.
    movie, err := app.models.Movies.Get(id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
```

dego_vila@yahoo.com 30 Oct 2026

```
        return
    }

    // Use pointers for the Title, Year and Runtime fields.
    var input struct {
        Title   *string         `json:"title"`
        Year    *int            `json:"year"`
        Runtime *models.Runtime `json:"runtime"`
        Genres  []string        `json:"genres"`
    }

    // Decode the JSON as normal.
    err = app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // If the input.Title value is nil, then we know that no corresponding "title"
    // key-value pair was provided in the JSON request body. So we move on and leave the
    // movie record unchanged. Otherwise, we update the movie record with the new title
    // value. Importantly, because input.Title is now a pointer to a string, we need
    // to dereference the pointer using the * operator to get the underlying value
    // before assigning it to our movie record.
    if input.Title != nil {
        movie.Title = *input.Title
    }

    // We also do the same for the other fields in the input struct.
    if input.Year != nil {
        movie.Year = *input.Year
    }
    if input.Runtime != nil {
        movie.Runtime = *input.Runtime
    }
    if input.Genres != nil {
        movie.Genres = input.Genres // Note that we don't need to dereference a slice.
    }

    v := validator.New()

    if models.ValidateMovie(v, movie); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    movie, err = app.models.Movies.Update(movie)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    err = app.writeJSON(w, http.StatusOK, envelope{"movie": movie}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

To summarize this: we've changed our `input` struct so that all the fields now have the zero value `nil`. After parsing the JSON request, we then go through the `input` struct fields and only update the movie record if the new value is *not* `nil`.

In addition to this, for API endpoints which perform *partial updates* on a resource, it's appropriate to use the HTTP method `PATCH` rather than `PUT` (which is intended for replacing a resource in full).

So, before we try out our new code, let's quickly update our `cmd/api/routes.go` file so that our `updateMovieHandler` is only used for `PATCH` requests.

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    // Require a PATCH request, rather than PUT.
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    return app.recoverPanic(router)
}
```

Demonstration

With that all set up, let's check that this partial update functionality works by correcting the release year for *The Breakfast Club* to 1985. Like so:

```
$ curl -X PATCH -d '{"year": 1985}' localhost:4000/v1/movies/4
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1985,
        "runtime": "96m",
        "genres": [
            "drama"
        ],
        "version": 2
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

Again, that's looking good. We can see that the `year` value has been correctly updated, and the `version` number has been incremented, but none of the other data fields have been changed.

Let's also quickly try the same request but including an empty `title` value. In this case the update will be blocked and you should receive a validation error, like so:

```
$ curl -X PATCH -d '{"year": 1985, "title": ""}' localhost:4000/v1/movies/4
{
    "error": {
        "title": "must be provided"
    }
}
```

Additional information

Null values in JSON

One special case to be aware of is when the client explicitly supplies a field in the JSON request with the value `null` — like `{"title": null}`.

When decoding a JSON `null` value, the decode destination will *always be set to the zero-value for the Go type* — which for pointers happens to be `nil`.

So, with the way that we've constructed the code in our handler, our application will effectively ignore any `null` values and act like that field wasn't included in the JSON request at all. For example, the following request would result in no changes to the movie record (apart from the `version` number being incremented):

```
$ curl -X PATCH -d '{"title": null, "year": null}' localhost:4000/v1/movies/4
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1985,
        "runtime": "96m",
        "genres": [
            "drama"
        ],
        "version": 3
    }
}
```

In an ideal world this type of request would return some kind of validation error. But there's

dego_vila@yahoo.com 30 Oct 2026

no straightforward way to determine the difference between the client not supplying a key-value pair in the JSON, or supplying it with the value `null`.

In most cases, it will probably suffice to explain this special case behavior in client documentation for the endpoint, and say something like *"JSON fields with `null` values will be ignored and will remain unchanged"*.

dego_vila@yahoo.com 30 Oct 2026

Optimistic concurrency control

The eagle-eyed among you might have noticed a small problem in our `updateMovieHandler` — there is a race condition if two clients try to update the same movie record at exactly the same time.

To illustrate this, let's pretend that we have two clients using our API: Alice and Bob. Alice wants to correct the runtime value for *The Breakfast Club* to 97m, and Bob wants to add the genre 'comedy' to the same movie.

Now imagine that Alice and Bob send these two update requests at *exactly* the same time. As we explained in *Let's Go*, Go's `http.Server` handles each HTTP request in a separate goroutine, so when this happens the code in our `updateMovieHandler` will be running concurrently in two different goroutines.

Let's step through what could potentially happen in this scenario:

1. Alice's goroutine calls `app.models.Movies.Get()` to retrieve a copy of the movie record (which has version number `N`).
2. Bob's goroutine calls `app.models.Movies.Get()` to retrieve a copy of the movie record (which also has version `N`).
3. Alice's goroutine changes the runtime to 97 minutes in its copy of the movie record.
4. Bob's goroutine updates the genres to include 'comedy' in its copy of the movie record.
5. Alice's goroutine calls `app.models.Movies.Update()` with its copy of the movie record. The movie record is written to the database and the version number is incremented to `N+1`.
6. Bob's goroutine calls `app.models.Movies.Update()` with its copy of the movie record. The movie record is written to the database and the version number is incremented to `N+2`.

Despite making two separate updates, only Bob's update will be reflected in the database at the end because the two goroutines were *racing* each other to make the change. Alice's update to the movie runtime will be lost when Bob's update overwrites it with the old runtime value retrieved during step 2 in the list above. And this happens silently — there's nothing to inform either Alice or Bob of the problem.

dego_vila@yahoo.com 30 Oct 2026

> **Note:** This specific type of race condition is known as a *data race*. Data races can occur when two or more goroutines try to use a piece of shared data (in this example the movie record) at the same time, but the result of their operations is dependent on the exact order that the scheduler executes their instructions.

Preventing the data race

Now we understand that the data race exists and why it's happening, *how can we prevent it?*

There are a couple of options, but the simplest and cleanest approach in this case is to use a form of optimistic locking based on the `version` number in our movie record.

The fix works like this:

1. Alice and Bob's goroutines both call `app.models.Movies.Get()` to retrieve a copy of the movie record. Both of these copies have the version number `N`.
2. Alice and Bob's goroutines make their respective changes to the movie record.
3. Alice and Bob's goroutines call `app.models.Movies.Update()` with their copies of the movie record. But the update is only executed *if the version number in the database is still* `N`. If it has changed, then we don't execute the update and send the client an error message instead.

This means that the first update request that reaches our database will succeed, and whoever is making the second update will receive an error message instead of having their change applied.

To make this work, we'll need to change the SQL statement for updating a movie so that it looks like this:

```
UPDATE movies
SET title = $1, year = $2, runtime = $3, genres = $4, version = version + 1
WHERE id = $5 AND version = $6
RETURNING version
```

Notice that in the `WHERE` clause we're now looking for a record with a specific ID *and a specific version number*?

If no matching record can be found, this query will result in a `sql.ErrNoRows` error and we know that the version number has been changed (or the record has been deleted completely). Either way, it's a form of *edit conflict* and we can use this as a trigger to send

dego_vila@yahoo.com 30 Oct 2026

the client an appropriate error response.

Implementing optimistic locking

OK, that's enough theory… let's put this into practice!

We'll start by creating a custom `ErrEditConflict` error that we can return from our database models in the event of a conflict. We'll use this later in the book when working with user records too, so it makes sense to define it in the `internal/models/models.go` file like so:

File: internal/models/models.go

```
package models

import (
    "database/sql"
    "errors"
)

var (
    ErrRecordNotFound = errors.New("record not found")
    ErrEditConflict   = errors.New("edit conflict")
)

...
```

Next, let's update our database model's `Update()` method to execute the new SQL query and manage the situation where a matching record couldn't be found.

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

...

func (m MovieModel) Update(movie Movie) (Movie, error) {
    // Add the 'AND version = $6' clause to the SQL query.
    query := `
        UPDATE movies
        SET title = $1, year = $2, runtime = $3, genres = $4, version = version + 1
        WHERE id = $5 AND version = $6
        RETURNING version`

    args := []any{
        movie.Title,
        movie.Year,
        movie.Runtime,
        pq.Array(movie.Genres),
        movie.ID,
        movie.Version, // Add the expected movie version.
    }

    // Execute the SQL query. If no matching row could be found, we know the movie
    // version has changed (or the record has been deleted) and we return our custom
    // ErrEditConflict error.
    err := m.DB.QueryRow(query, args...).Scan(&movie.Version)
    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return Movie{}, ErrEditConflict
        default:
            return Movie{}, err
        }
    }

    return movie, nil
}

...
```

Next let's head to our `cmd/api/errors.go` file and create a new `editConflictResponse()` helper. We want this to send a `409 Conflict` response, along with a plain-English error message that explains the problem to the client.

File: cmd/api/errors.go

```
package main

...

func (app *application) editConflictResponse(w http.ResponseWriter, r *http.Request) {
    message := "unable to update the record due to an edit conflict, please try again"
    app.errorResponse(w, r, http.StatusConflict, message)
}
```

And then as the final step, we need to change our `updateMovieHandler` so that it checks for

dego_vila@yahoo.com 30 Oct 2026

an `ErrEditConflict` error and calls the `editConflictResponse()` helper if necessary. Like so:

File: cmd/api/movies.go

```
package main

...

func (app *application) updateMovieHandler(w http.ResponseWriter, r *http.Request) {

    ...

    // Intercept any ErrEditConflict error and call the new editConflictResponse()
    // helper.
    movie, err = app.models.Movies.Update(movie)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrEditConflict):
            app.editConflictResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    err = app.writeJSON(w, http.StatusOK, envelope{"movie": movie}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

At this point, our `updateMovieHandler` should now be safe from the race condition that we've been talking about. If two goroutines are executing the code at the same time, the first update will succeed, and the second will fail because the `version` number in the database no longer matches the expected value.

Let's try this out by using the `xargs` command to send a bunch of concurrent requests to our endpoint. Assuming that your computer executes the requests closely enough together, you should find that some requests succeed but the others now fail with a `409 Conflict` status code. Like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ xargs -I % -P8 curl -X PATCH -d '{"runtime": "97m"}' "localhost:4000/v1/movies/4" < <(printf '%s\n' {1..8})
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1985,
        "runtime": "97m",
        "genres": [
            "drama"
        ],
        "version": 4
    }
}
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1985,
        "runtime": "97m",
        "genres": [
            "drama"
        ],
        "version": 5
    }
}
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1985,
        "runtime": "97m",
        "genres": [
            "drama"
        ],
        "version": 6
    }
}
{
    "error": "unable to update the record due to an edit conflict, please try again"
}
{
    "error": "unable to update the record due to an edit conflict, please try again"
}
{
    "error": "unable to update the record due to an edit conflict, please try again"
}
{
    "error": "unable to update the record due to an edit conflict, please try again"
}
{
    "movie": {
        "id": 4,
        "title": "The Breakfast Club",
        "year": 1985,
        "runtime": "97m",
        "genres": [
            "drama"
        ],
        "version": 7
    }
}
```

Bringing this to a close, the race condition that we've been demonstrating in this chapter is

dego_vila@yahoo.com 30 Oct 2026

fairly innocuous. But in other applications this exact class of race condition can have much more serious consequences — such as when updating the stock level for a product in an online store, or updating the balance of an account.

As I mentioned briefly in *Let's Go*, it's good to get into the habit of thinking about race conditions whenever you write code, and structure your applications to either manage them or avoid them completely — no matter how innocuous they might seem.

Additional information

Round-trip locking

One of the nice things about the optimistic locking pattern that we've used here is that you can extend it so the client passes the version number that *they expect* in an `If-Match` or `X-Expected-Version` header.

In certain applications, this can be useful to help the client ensure they are not sending *their update request* based on outdated information.

Very roughly, you could implement this by adding a check to your `updateMovieHandler` like so:

dego_vila@yahoo.com 30 Oct 2026

```
func (app *application) updateMovieHandler(w http.ResponseWriter, r *http.Request) {
    id, err := app.readIDParam(r)
    if err != nil {
        app.notFoundResponse(w, r)
        return
    }

    movie, err := app.models.Movies.Get(id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // If the request contains a X-Expected-Version header, verify that the movie
    // version in the database matches the expected version specified in the header.
    if r.Header.Get("X-Expected-Version") != "" {
        if strconv.Itoa(int(movie.Version)) != r.Header.Get("X-Expected-Version") {
            app.editConflictResponse(w, r)
            return
        }
    }

    ...
}
```

Locking on other fields or types

Using an incrementing integer `version` number as the basis for an optimistic lock is safe and computationally cheap. I'd recommend using this approach unless you have a specific reason not to.

As an alternative, you could use a `last_updated` timestamp as the basis for the lock. But this is less safe — there's the theoretical possibility that two clients could update a record at exactly the same time, and using a timestamp also introduces the risk of further problems if your server's clock is wrong or becomes wrong over time.

If it's important to you that the version identifier isn't guessable, then a good option is to use a high-entropy random string such as a UUID in the `version` field. PostgreSQL has a UUID type and the `uuid-ossp` extension which you could use for this purpose like so:

```
UPDATE movies
SET title = $1, year = $2, runtime = $3, genres = $4, version = uuid_generate_v4()
WHERE id = $5 AND version = $6
RETURNING version
```

dego_vila@yahoo.com 30 Oct 2026

Managing SQL query timeouts

So far in our code we've been using Go's `Exec()` and `QueryRow()` methods to run our SQL queries. But Go also provides *context-aware* variants of these two methods: `ExecContext()` and `QueryRowContext()`. These variants accept a `context.Context` instance as the first parameter which you can leverage to *terminate running database queries*.

This feature can be useful when *you have a SQL query that is taking longer to run than expected*. When this happens, it suggests a problem — either with that particular query or your database or application more generally — and you probably want to cancel the query (in order to free up resources), log an error for further investigation, and return a `500 Internal Server Error` response to the client.

In this chapter, we'll update our application to do exactly that.

Mimicking a long-running query

To help demonstrate how this all works, let's start by adapting our database model's `Get()` method so that it mimics a long-running query. Specifically, we'll update our SQL query to return a `pg_sleep(8)` value, which will make PostgreSQL sleep for 8 seconds before returning its result.

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

...

func (m MovieModel) Get(id int) (Movie, error) {
    if id < 1 {
        return Movie{}, ErrRecordNotFound
    }

    // Update the query to return pg_sleep(8) as the first value.
    query := `
        SELECT pg_sleep(8), id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE id = $1`

    var movie Movie

    // Importantly, update the Scan() arguments so that the pg_sleep(8) return value
    // is scanned into a []byte slice.
    err := m.DB.QueryRow(query, id).Scan(
        &[]byte{}, // Add this line.
        &movie.ID,
        &movie.CreatedAt,
        &movie.Title,
        &movie.Year,
        &movie.Runtime,
        pq.Array(&movie.Genres),
        &movie.Version,
    )

    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return Movie{}, ErrRecordNotFound
        default:
            return Movie{}, err
        }
    }

    return movie, nil
}

...
```

If you restart the application and make a request to the `GET /v1/movies/:id` endpoint, you should find that the request hangs for 8 seconds before you finally get a successful response displaying the movie information. Similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -w '\nTime: %{time_total}s \n' localhost:4000/v1/movies/1
{
    "movie": {
        "id": 1,
        "title": "Moana",
        "year": 2016,
        "runtime": "107m",
        "genres": [
            "animation",
            "adventure"
        ],
        "version": 1
    }
}

Time: 8.012983s
```

> **Note:** In the `curl` command above we're using the `-w` flag to annotate the HTTP response with the total time taken for the command to complete. For more details about the timing information available in curl please see this excellent blog post.

Adding a query timeout

Now that we've got some code that mimics a long-running query, let's enforce a timeout so that the SQL query is automatically canceled if it doesn't complete within 3 seconds.

To do this we need to:

1. Use the `context.WithTimeout()` function to create a `context.Context` instance with a 3-second timeout deadline.
2. Execute the SQL query using the `QueryRowContext()` method, passing the `context.Context` instance as a parameter.

I'll demonstrate:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

import (
    "context" // New import
    "database/sql"
    "errors"
    "time"

    "greenlight.alexedwards.net/internal/validator"

    "github.com/lib/pq"
)

...

func (m MovieModel) Get(id int) (Movie, error) {
    if id < 1 {
        return Movie{}, ErrRecordNotFound
    }

    query := `
        SELECT pg_sleep(8), id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE id = $1`

    var movie Movie

    // Use the context.WithTimeout() function to create a context.Context which carries a
    // 3-second timeout deadline. Note that we're using the empty context.Background()
    // as the 'parent' context.
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)

    // Importantly, use defer to make sure that we cancel the context before the Get()
    // method returns.
    defer cancel()

    // Use the QueryRowContext() method to execute the query, passing in the context
    // (including the deadline) as the first argument.
    err := m.DB.QueryRowContext(ctx, query, id).Scan(
        &[]byte{},
        &movie.ID,
        &movie.CreatedAt,
        &movie.Title,
        &movie.Year,
        &movie.Runtime,
        pq.Array(&movie.Genres),
        &movie.Version,
    )

    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return Movie{}, ErrRecordNotFound
        default:
            return Movie{}, err
        }
    }

    return movie, nil
}

...
```

dego_vila@yahoo.com 30 Oct 2026

There are a couple of things in the code above that I'd like to emphasize and explain:

- The `defer cancel()` line is necessary because it ensures that the resources associated with our context will always be released before the `Get()` method returns, thereby preventing a memory leak. Without it, the resources won't be released until either the 3-second timeout is hit or the parent context (which in this specific example is `context.Background()`) is canceled.
- The timeout countdown begins from the moment that the context is created using `context.WithTimeout()`. Any time spent executing code between creating the context and calling `QueryRowContext()` will count towards the timeout.

OK, let's try this out.

If you restart the application and make another request to the `GET /v1/movies/:id` endpoint, you should now get an error response similar to this after a 3-second delay:

```
$ curl -w '\nTime: %{time_total}s \n' localhost:4000/v1/movies/1
{
    "error": "the server encountered a problem and could not process your request"
}

Time: 3.025179s
```

If you go back to the terminal window which is running the application, you should also see a log line with the error message `"pq: canceling statement due to user request (57014)"`. Like so:

```
$ go run ./cmd/api
time=2026-09-11T19:05:01.381+02:00 level=INFO msg="database connection pool established"
time=2026-09-11T19:05:01.381+02:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-09-11T19:05:07.518+02:00 level=ERROR msg="pq: canceling statement due to user request (57014)" method=GET uri=/v1/mo
```

At first the wording of this error message might seem odd… until you learn that the message "canceling statement due to user request" is coming from PostgreSQL. In that light it makes sense: *our application* is the user of PostgreSQL, and we're purposefully canceling the query after 3 seconds.

So, this is actually really good, and things are working as we would hope.

After 3 seconds, the context timeout is reached and our `pq` database driver sends a cancellation signal to PostgreSQL[†]. PostgreSQL then terminates the running query, the corresponding resources are freed-up, and it returns the error message that we see above. The client is then sent a `500 Internal Server Error` response, and the error is logged so

dego_vila@yahoo.com 30 Oct 2026

that we know something has gone wrong.

> † More precisely, our context (the one with the 3-second timeout) has a `Done` channel, and when the timeout is reached the `Done` channel will be closed. While the SQL query is running, our database driver `pq` is also running a background goroutine which listens on this `Done` channel. If the channel gets closed, then `pq` sends a cancellation signal to PostgreSQL. PostgreSQL terminates the query, and then sends the error message that we see above as a response to the original `pq` goroutine. That error message is then returned to our database model's `Get()` method.

Timeouts outside of PostgreSQL

There's another important thing to point out here: *it's possible that the timeout deadline will be hit before the PostgreSQL query even starts*.

You might remember that earlier in the book we configured our `sql.DB` connection pool to allow a maximum of 25 open connections. If all those connections are in-use, then any additional queries will be 'queued' by `sql.DB` until a connection becomes available. In this scenario — or any other which causes a delay — it's possible that the timeout deadline will be hit before a free database connection even becomes available. If this happens then `QueryRowContext()` will return a `context.DeadlineExceeded` error.

In fact, we can demonstrate this in our application by setting the maximum open connections to 1 and making two concurrent requests to our endpoint. Go ahead and restart the API using a `-db-max-open-conns=1` flag, like so:

```
$ go run ./cmd/api -db-max-open-conns=1
time=2026-09-11T19:10:01.143+02:00 level=INFO msg="database connection pool established"
time=2026-09-11T19:10:01.143+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Then in another terminal window make two requests to the `GET /v1/movies/:id` endpoint at the same time. At the moment that the 3-second timeout is reached, we should have one running SQL query and the other still 'queued' in the `sql.DB` connection pool. You should get two error responses which look like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ xargs -I % -P2 curl localhost:4000/v1/movies/1 < <(printf '%s\n' {1..2})
{
    "error": "the server encountered a problem and could not process your request"
}
{
    "error": "the server encountered a problem and could not process your request"
}
```

When you now head back to your original terminal, you should see two different error messages:

```
$ go run ./cmd/api -db-max-open-conns=1
time=2026-09-11T19:17:11.308+02:00 level=INFO msg="database connection pool established"
time=2026-09-11T19:17:11.308+02:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-09-11T19:17:21.123+02:00 level=ERROR msg="context deadline exceeded" method=GET uri=/v1/movies/1
time=2026-09-11T19:17:21.132+02:00 level=ERROR msg="pq: canceling statement due to user request (57014)" method=GET uri=/v1/mo
```

Here the `"pq: canceling statement due to user request (57014)"` error message relates to the running SQL query being terminated, whereas the `"context deadline exceeded"` message relates to the queued SQL query being canceled *before a free database connection even became available*.

In a similar vein, it's also possible that the timeout deadline will be hit later on when the data returned from the query is being processed with `Scan()`. If this happens, then `Scan()` will also return a `context.DeadlineExceeded` error.

Updating our database model

Let's quickly update our database model to use a 3-second timeout deadline for all our operations. While we're at it, we'll remove the `pg_sleep(8)` clause from our `Get()` method too.

File: internal/models/movies.go

```
package models

...

func (m MovieModel) Insert(movie Movie) (Movie, error) {
    query := `
        INSERT INTO movies (title, year, runtime, genres)
        VALUES ($1, $2, $3, $4)
        RETURNING id, created_at, version`

    args := []any{movie.Title, movie.Year, movie.Runtime, pq.Array(movie.Genres)}

    // Create a context with a 3-second timeout.
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Use QueryRowContext() and pass the context as the first argument.
    err := m.DB.QueryRowContext(ctx, query, args...).Scan(&movie.ID, &movie.CreatedAt, &movie.Version)
    return movie, err
}

func (m MovieModel) Get(id int) (Movie, error) {
    if id < 1 {
        return Movie{}, ErrRecordNotFound
    }

    // Remove the pg_sleep(8) clause.
    query := `
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE id = $1`

    var movie Movie

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Remove &[]byte{} from the first Scan() destination.
    err := m.DB.QueryRowContext(ctx, query, id).Scan(
        &movie.ID,
        &movie.CreatedAt,
        &movie.Title,
        &movie.Year,
        &movie.Runtime,
        pq.Array(&movie.Genres),
        &movie.Version,
    )

    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return Movie{}, ErrRecordNotFound
        default:
            return Movie{}, err
        }
    }

    return movie, nil
}

func (m MovieModel) Update(movie Movie) (Movie, error) {
    query := `
        UPDATE movies
        SET title = $1, year = $2, runtime = $3, genres = $4, version = version + 1
        WHERE id = $5 AND version = $6
        RETURNING version`

    args := []any{
        movie.Title,
        movie.Year,
        movie.Runtime,
        pq.Array(movie.Genres),
        movie.ID,
        movie.Version,
    }

    // Create a context with a 3-second timeout.
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Use QueryRowContext() and pass the context as the first argument.
    err := m.DB.QueryRowContext(ctx, query, args...).Scan(&movie.Version)
```

dego_vila@yahoo.com 30 Oct 2026

```
    err := m.DB.QueryRowContext(ctx, query, args...).Scan(&movie.Version)
    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return Movie{}, ErrEditConflict
        default:
            return Movie{}, err
        }
    }

    return movie, nil
}

func (m MovieModel) Delete(id int) error {
    if id < 1 {
        return ErrRecordNotFound
    }

    query := `
        DELETE FROM movies
        WHERE id = $1`

    // Create a context with a 3-second timeout.
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Use ExecContext() and pass the context as the first argument.
    result, err := m.DB.ExecContext(ctx, query, id)
    if err != nil {
        return err
    }

    rowsAffected, err := result.RowsAffected()
    if err != nil {
        return err
    }

    if rowsAffected == 0 {
        return ErrRecordNotFound
    }

    return nil
}
```

Additional information

Using the request context

As an alternative to the pattern that we've used in the code above, we could create a context with a timeout *in our handlers* using the *request context* as the parent — and then pass that on to our database model.

But — and it's a big but — doing this introduces a lot of behavioral complexity, and for *most* applications the benefits aren't large enough to make the trade-off worthwhile.

dego_vila@yahoo.com 30 Oct 2026

The details behind this are very interesting, but also quite intricate and heavy-going. For that reason I've discussed it further in this appendix.

dego_vila@yahoo.com 30 Oct 2026

Chapter 9.

Filtering, sorting, and pagination

In this section of the book we're going to focus on building up the functionality for a new `GET /v1/movies` endpoint, which will return the details of multiple movies in a JSON array.

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
GET	**/v1/movies**	**listMoviesHandler**	**Show the details of all movies**
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PATCH	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie
DELETE	/v1/movies/:id	deleteMovieHandler	Delete a specific movie

We'll develop the functionality for this endpoint incrementally, starting out by returning data for *all* movies and then gradually making it more useful and usable by adding filtering, sorting, and pagination functionality.

In this section you'll learn how to:

- Return the details of multiple resources in a single JSON response.
- Accept and apply optional filter parameters to narrow down the returned data set.
- Implement full-text search on your database fields using PostgreSQL's inbuilt functionality.
- Accept and safely apply sort parameters to change the order of results in the data set.
- Develop a pragmatic reusable pattern to support pagination on large data sets, and return pagination metadata in your JSON responses.

dego_vila@yahoo.com 30 Oct 2026

Parsing query string parameters

Over the next few chapters, we're going to configure the `GET /v1/movies` endpoint so that a client can control which movie records are returned via *query string parameters*. For example:

```
/v1/movies?title=godfather&genres=crime,drama&page=1&page_size=5&sort=-year
```

If a client sends a query string like this, it is essentially saying to our API: *"please return the first 5 records where the movie title includes `godfather` and the genres include `crime` and `drama`, sorted by descending release year"*.

> **Note**: In the `sort` parameter we will use the `-` character to denote descending sort order. So, for example, the parameter `sort=title` implies an ascending alphabetical sort on movie title, whereas `sort=-title` implies a descending sort.

So the first thing we're going to look at is *how to parse these query string parameters in our Go code*.

As you can hopefully remember from *Let's Go*, we can retrieve the query string data from a request by calling the `r.URL.Query()` method. This returns a `url.Values` type, which is basically a map holding the query string data.

We can then extract values from this map using the `Get()` method, which will return the value for a specific key as a `string` type, or the empty string `""` if no matching key exists in the query string.

In our case, we'll need to carry out extra post-processing on some of these query string values too. Specifically:

- The `genres` parameter will potentially contain multiple *comma-separated values* — like `genres=crime,drama`. We will want to split these values apart and store them in a `[]string` slice.
- The `page` and `page_size` parameters will contain numbers, and we will want to convert these query string values into Go `int` types.

In addition to that:

dego_vila@yahoo.com 30 Oct 2026

- There are some validation checks that we'll want to apply to the query string values, like making sure that `page` and `page_size` are not negative numbers.
- We want our application to set some sensible *default values* in case parameters like `page`, `page_size` and `sort` aren't provided by the client.

Creating helper functions

To assist with this, we're going to create three new helper functions: `readString()`, `readInt()` and `readCSV()`. We'll use these helpers to extract and parse values from the query string, or return a default 'fallback' value if necessary.

Head to your `cmd/api/helpers.go` file and add the following code:

File: cmd/api/helpers.go

```
package main

import (
    "encoding/json/jsontext"
    "encoding/json/v2"
    "errors"
    "fmt"
    "io"
    "net/http"
    "net/url" // New import"
    "slices"
    "strconv"
    "strings"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator" // New import

    "github.com/julienschmidt/httprouter"
)

...

// The readString() helper returns a string value from the query string, or the provided
// default value if no matching key could be found.
func (app *application) readString(qs url.Values, key string, defaultValue string) string {
    // Extract the value for a given key from the query string. If no key exists this
    // will return the empty string "".
    s := qs.Get(key)

    // If no key exists (or the value is empty) then return the default value.
    if s == "" {
        return defaultValue
    }

    // Otherwise return the string.
    return s
}

// The readCSV() helper reads a string value from the query string and then splits it
// into a slice on the comma character. If no matching key could be found, it returns
// the provided default value.
```

dego_vila@yahoo.com 30 Oct 2026

```
func (app *application) readCSV(qs url.Values, key string, defaultValue []string) []string {
    // Extract the value from the query string.
    csv := qs.Get(key)

    // If no key exists (or the value is empty) then return the default value.
    if csv == "" {
        return defaultValue
    }

    // Otherwise parse the value into a []string slice and return it.
    return strings.Split(csv, ",")
}


// The readInt() helper reads a string value from the query string and converts it to an
// integer before returning. If no matching key could be found it returns the provided
// default value. If the value couldn't be converted to an integer, then we record an
// error message in the provided Validator instance.
func (app *application) readInt(qs url.Values, key string, defaultValue int, v *validator.Validator) int {
    // Extract the value from the query string.
    s := qs.Get(key)

    // If no key exists (or the value is empty) then return the default value.
    if s == "" {
        return defaultValue
    }

    // Try to convert the value to an int. If this fails, add an error message to the
    // validator instance and return the default value.
    i, err := strconv.Atoi(s)
    if err != nil {
        v.AddError(key, "must be an integer value")
        return defaultValue
    }

    // Otherwise, return the converted integer value.
    return i
}
```

Adding the API handler and route

Next up, let's create a new `listMoviesHandler` for our `GET /v1/movies` endpoint. For now, this handler will simply parse the request query string using the helpers we just made, and then dump the contents out in an HTTP response.

If you're following along, go ahead and create the `listMoviesHandler` like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) listMoviesHandler(w http.ResponseWriter, r *http.Request) {
    // To keep things consistent with our other handlers, we'll define an input struct
    // to hold the expected values from the request query string.
    var input struct {
        Title    string
        Genres   []string
        Page     int
        PageSize int
        Sort     string
    }

    // Initialize a new Validator instance.
    v := validator.New()

    // Call r.URL.Query() to get the url.Values map containing the query string data.
    qs := r.URL.Query()

    // Use our helpers to extract the title and genres query string values, falling back
    // to defaults of an empty string and an empty slice respectively if they are not
    // provided by the client.
    input.Title = app.readString(qs, "title", "")
    input.Genres = app.readCSV(qs, "genres", []string{})

    // Get the page and page_size query string values as integers. Notice that we set
    // the default page value to 1 and default page_size to 20, and that we pass the
    // validator instance as the final argument here.
    input.Page = app.readInt(qs, "page", 1, v)
    input.PageSize = app.readInt(qs, "page_size", 20, v)

    // Extract the sort query string value, falling back to "id" if it is not provided
    // by the client (which will imply an ascending sort on movie ID).
    input.Sort = app.readString(qs, "sort", "id")

    // Check the Validator instance for any errors and use the failedValidationResponse()
    // helper to send the client a response if necessary.
    if !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Dump the contents of the input struct in an HTTP response.
    fmt.Fprintf(w, "%+v\n", input)
}
```

Then we need to create the `GET /v1/movies` route in our `cmd/api/routes.go` file, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    // Add the route for the GET /v1/movies endpoint.
    router.HandlerFunc(http.MethodGet, "/v1/movies", app.listMoviesHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    return app.recoverPanic(router)
}
```

And with that, we're ready to see this in action!

Go ahead and try sending a request to the `GET /v1/movies` endpoint containing the expected query string parameters, like below.

> **Important:** When using curl to send a request containing more than one query string parameter, you **must** wrap the URL in quotes for it to work correctly.

```
$ curl "localhost:4000/v1/movies?title=godfather&genres=crime,drama&page=1&page_size=5&sort=year"
{Title:godfather Genres:[crime drama] Page:1 PageSize:5 Sort:year}
```

That's looking good — we can see that the values provided in our query string have all been parsed correctly and are included in the `input` struct.

If you want, try making a request with *no* query string parameters. In this case, you should see that the values in the `input` struct take on the defaults we specified in our `listMoviesHandler` code. Like so:

```
$ curl localhost:4000/v1/movies
{Title: Genres:[] Page:1 PageSize:20 Sort:id}
```

Creating a Filters struct

dego_vila@yahoo.com 30 Oct 2026

The `page`, `page_size` and `sort` query string parameters are things that you'll potentially want to use on other endpoints in your API too. So, to help make this easier, let's quickly split them out into a reusable `Filters` struct.

If you're following along, go ahead and create a new `internal/models/filters.go` file:

```
$ touch internal/models/filters.go
```

And then add the following code:

File: internal/models/filters.go

```
package models

type Filters struct {
    Page     int
    PageSize int
    Sort     string
}
```

Once that's done, head back to your `listMoviesHandler` and update it to use the new `Filters` struct like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) listMoviesHandler(w http.ResponseWriter, r *http.Request) {
    // Embed the new Filters struct.
    var input struct {
        Title  string
        Genres []string
        models.Filters
    }

    v := validator.New()

    qs := r.URL.Query()

    input.Title = app.readString(qs, "title", "")
    input.Genres = app.readCSV(qs, "genres", []string{})

    // Read the page and page_size query string values into the embedded struct.
    input.Filters.Page = app.readInt(qs, "page", 1, v)
    input.Filters.PageSize = app.readInt(qs, "page_size", 20, v)

    // Read the sort query string value into the embedded struct.
    input.Filters.Sort = app.readString(qs, "sort", "id")

    if !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}
```

At this point, you should be able to run the API again and everything should continue to work like before.

dego_vila@yahoo.com 30 Oct 2026

Validating query string parameters

Thanks to the `readInt()` helper that we made in the previous chapter, our API should already be returning validation errors if the `page` and `page_size` query string parameters don't contain integer values. Go ahead and try it out if you like:

```
$ curl "localhost:4000/v1/movies?page=abc&page_size=abc"
{
    "error": {
        "page": "must be an integer value",
        "page_size": "must be an integer value"
    }
}
```

But we still need to perform some additional sanity checks on the query string values provided by the client. In particular, we want to check that:

- The `page` value is between 1 and 10,000,000.
- The `page_size` value is between 1 and 100.
- The `sort` parameter contains a known and supported value for our movies table. Specifically, we'll allow `"id"`, `"title"`, `"year"`, `"runtime"`, `"-id"`, `"-title"`, `"-year"` or `"-runtime"`.

To fix this, let's open up the `internal/models/filters.go` file and create a new `ValidateFilters()` function which conducts these checks on the values.

We'll follow the same pattern that we used for the `ValidateMovie()` function earlier on to do this, like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/filters.go

```
package models

import (
    "greenlight.alexedwards.net/internal/validator" // New import
)

// Add a SortSafelist field to hold the supported sort values.
type Filters struct {
    Page         int
    PageSize     int
    Sort         string
    SortSafelist []string
}

func ValidateFilters(v *validator.Validator, f Filters) {
    // Check that the page and page_size parameters contain sensible values.
    v.Check(f.Page > 0, "page", "must be greater than zero")
    v.Check(f.Page <= 10_000_000, "page", "must be a maximum of 10 million")
    v.Check(f.PageSize > 0, "page_size", "must be greater than zero")
    v.Check(f.PageSize <= 100, "page_size", "must be a maximum of 100")

    // Check that the sort parameter matches a value in the safelist.
    v.Check(validator.PermittedValue(f.Sort, f.SortSafelist...), "sort", "invalid sort value")
}
```

Then we need to update our `listMoviesHandler` to set the supported values in the `SortSafelist` field, and subsequently call this new `ValidateFilters()` function.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) listMoviesHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title  string
        Genres []string
        models.Filters
    }

    v := validator.New()

    qs := r.URL.Query()

    input.Title = app.readString(qs, "title", "")
    input.Genres = app.readCSV(qs, "genres", []string{})

    input.Filters.Page = app.readInt(qs, "page", 1, v)
    input.Filters.PageSize = app.readInt(qs, "page_size", 20, v)

    input.Filters.Sort = app.readString(qs, "sort", "id")
    // Add the supported sort values for this endpoint to the sort safelist.
    input.Filters.SortSafelist = []string{"id", "title", "year", "runtime", "-id", "-title", "-year", "-runtime"}

    // Execute the validation checks on the Filters struct and send a response
    // containing the errors if necessary.
    if models.ValidateFilters(v, input.Filters); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    fmt.Fprintf(w, "%+v\n", input)
}
```

If you restart the API and try making a request with some invalid `page`, `page_size` and `sort` parameters, you should now receive an error response containing the relevant validation failure messages. Similar to this:

```
$ curl "localhost:4000/v1/movies?page=-1&page_size=-1&sort=foo"
{
    "error": {
        "page": "must be greater than zero",
        "page_size": "must be greater than zero",
        "sort": "invalid sort value"
    }
}
```

Feel free to test this out more if you like, and try sending different query string values until you’re confident that the validation checks are all working correctly.

dego_vila@yahoo.com 30 Oct 2026

Listing data

OK, let's move on and get our `GET /v1/movies` endpoint returning some real data.

For now, we'll ignore any query string values provided by the client and return *all* movie records, sorted by movie ID. This will give us a solid base from which we can develop the more specialized functionality around filtering, sorting, and pagination.

Our aim in this chapter will be to get the endpoint to return a JSON response containing an array of all movies, similar to this:

```
{
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        },
        ... etc.
    ]
}
```

Updating the application

To retrieve this data from our PostgreSQL database, let's create a new `GetAll()` method on our database model which executes the following SQL query:

dego_vila@yahoo.com 30 Oct 2026

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
ORDER BY id
```

Because we're expecting this SQL query to return multiple records, we'll need to run it using Go's `QueryContext()` method. We already explained how this works in detail in *Let's Go*, so let's jump into the code:

File: internal/models/movies.go

```
package models

...

// Create a new GetAll() method which returns a slice of movies. Although we're not
// using them right now, we've set this up to accept the various filter parameters as
// arguments.
func (m MovieModel) GetAll(title string, genres []string, filters Filters) ([]Movie, error) {
    // Construct the SQL query to retrieve all movie records.
    query := `
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        ORDER BY id`

    // Create a context with a 3-second timeout.
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Use QueryContext() to execute the query. This returns a sql.Rows resultset
    // containing the result.
    rows, err := m.DB.QueryContext(ctx, query)
    if err != nil {
        return nil, err
    }

    // Importantly, defer a call to rows.Close() to ensure that the resultset is closed
    // before GetAll() returns.
    defer rows.Close()

    // Initialize an empty slice to hold the movie data.
    movies := []Movie{}

    // Use rows.Next to iterate through the rows in the resultset.
    for rows.Next() {
        // Initialize an empty Movie struct to hold the data for an individual movie.
        var movie Movie

        // Scan the values from the row into the Movie struct. Again, note that we're
        // using the pq.Array() adapter on the genres field here.
        err := rows.Scan(
            &movie.ID,
            &movie.CreatedAt,
            &movie.Title,
            &movie.Year,
            &movie.Runtime,
            pq.Array(&movie.Genres),
            &movie.Version,
        )
        if err != nil {
            return nil, err
        }
```

dego_vila@yahoo.com 30 Oct 2026

```
        }

        // Add the Movie struct to the slice.
        movies = append(movies, movie)
    }

    // After the rows.Next() loop has finished, call rows.Err() to retrieve any error
    // that was encountered during the iteration.
    if err = rows.Err(); err != nil {
        return nil, err
    }

    // If everything went OK, then return the slice of movies.
    return movies, nil
}
```

Next up, we need to adapt the `listMoviesHandler` so that it calls the new `GetAll()` method to retrieve the movie data, and then writes this data as a JSON response.

If you're following along, go ahead and update the handler like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) listMoviesHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title  string
        Genres []string
        models.Filters
    }

    v := validator.New()

    qs := r.URL.Query()

    input.Title = app.readString(qs, "title", "")
    input.Genres = app.readCSV(qs, "genres", []string{})

    input.Filters.Page = app.readInt(qs, "page", 1, v)
    input.Filters.PageSize = app.readInt(qs, "page_size", 20, v)

    input.Filters.Sort = app.readString(qs, "sort", "id")
    input.Filters.SortSafelist = []string{"id", "title", "year", "runtime", "-id", "-title", "-year", "-runtime"}

    if models.ValidateFilters(v, input.Filters); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Call the GetAll() method to retrieve the movies, passing in the various filter
    // parameters.
    movies, err := app.models.Movies.GetAll(input.Title, input.Genres, input.Filters)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Send a JSON response containing the movie data.
    err = app.writeJSON(w, http.StatusOK, envelope{"movies": movies}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

And now we should be ready to try this out.

Go ahead and restart the API, then when you make a `GET /v1/movies` request you should see the slice of movies returned by `GetAll()` rendered as a JSON array. Similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl localhost:4000/v1/movies
{
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        },
        {
            "id": 4,
            "title": "The Breakfast Club",
            "year": 1985,
            "runtime": "97m",
            "genres": [
                "drama"
            ],
            "version": 7
        }
    ]
}
```

dego_vila@yahoo.com 30 Oct 2026

Filtering lists

In this chapter we're going to start putting our query string parameters to use, so that clients can search for movies with a specific title or genres.

Specifically, we'll build a reductive filter which allows clients to search based on a case-insensitive exact match for movie title and/or one or more movie genres. For example:

```
// List all movies.
/v1/movies

// List movies where the title is a case-insensitive exact match for 'black panther'.
/v1/movies?title=black+panther

// List movies where the genres include 'adventure'.
/v1/movies?genres=adventure

// List movies where the title is a case-insensitive exact match for 'moana' AND the
// genres include both 'animation' AND 'adventure'.
/v1/movies?title=moana&genres=animation,adventure
```

> **Note:** The `+` symbol in the query strings above is a URL-encoded space character. Alternatively you could use `%20` instead… either will work in the context of a query string.

Dynamic filtering in the SQL query

The hardest part of building a dynamic filtering feature like this is the SQL query to retrieve the data — we need it to work with no filters, filters on both `title` and `genres`, or a filter on only one of them.

To deal with this, one option is to build up the SQL query dynamically at runtime… with the necessary SQL for each filter concatenated or interpolated into the `WHERE` clause. But this approach can make your code messy and difficult to understand, especially for large queries which need to support lots of filter options.

In this book we'll opt for a different technique and use a fixed SQL query which looks like this:

dego_vila@yahoo.com 30 Oct 2026

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (LOWER(title) = LOWER($1) OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY id
```

This SQL query is designed so that each of the filters behaves like it is 'optional'. For example, the condition `(LOWER(title) = LOWER($1) OR $1 = '')` will evaluate as `true` if the placeholder parameter `$1` is a case-insensitive match for the movie title *or* the placeholder parameter equals `''`. So this filter condition will essentially be 'skipped' when the movie title being searched for is the empty string `""`.

The `(genres @> $2 OR $2 = '{}')` condition works in the same way. The `@>` symbol is the 'contains' operator for PostgreSQL arrays, and this condition will return `true` if each value in the placeholder parameter `$2` appears in the database `genres` field *or* the placeholder parameter contains an empty array.

You'll remember that earlier in the book we set up our `listMoviesHandler` so that the empty string `""` and an empty slice are used as the default values for the `title` and `genres` filter parameters:

```
input.Title = app.readString(qs, "title", "")
input.Genres = app.readCSV(qs, "genres", []string{})
```

So, putting this all together, it means that if a client doesn't provide a `title` parameter in their query string, then the value for the `$1` placeholder will be the empty string `""`, and the filter condition in the SQL query will evaluate to `true` and act like it has been 'skipped'. Likewise with the `genres` parameter.

> **Note:** PostgreSQL also provides a range of other useful array operators and functions, including the `&&` 'overlap' operator, the `<@` 'contained by' operator, and the `array_length()` function. A complete list can be found here.

Alright, let's head back to our `internal/models/movies.go` file, and update the `GetAll()` method to use this new query. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/movies.go

```
package models

...

func (m MovieModel) GetAll(title string, genres []string, filters Filters) ([]Movie, error) {
    // Update the SQL query to include the filter conditions.
    query := `
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE (LOWER(title) = LOWER($1) OR $1 = '')
        AND (genres @> $2 OR $2 = '{}')
        ORDER BY id`

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Pass the title and genres as the placeholder parameter values.
    rows, err := m.DB.QueryContext(ctx, query, title, pq.Array(genres))
    if err != nil {
        return nil, err
    }

    defer rows.Close()

    movies := []Movie{}

    for rows.Next() {
        var movie Movie

        err := rows.Scan(
            &movie.ID,
            &movie.CreatedAt,
            &movie.Title,
            &movie.Year,
            &movie.Runtime,
            pq.Array(&movie.Genres),
            &movie.Version,
        )
        if err != nil {
            return nil, err
        }

        movies = append(movies, movie)
    }

    if err = rows.Err(); err != nil {
        return nil, err
    }

    return movies, nil
}
```

Now let’s restart the application and try this out, using the examples that we gave at the start of the chapter. If you’ve been following along, the responses should look similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?title=black+panther"
{
    "movies": [
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}
```

```
$ curl "localhost:4000/v1/movies?genres=adventure"
{
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}
```

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?title=moana&genres=animation,adventure"
{
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        }
    ]
}
```

You can also try making a request with a filter that doesn't match any records. In this case, you should get an empty JSON array in the response like so:

```
$ curl "localhost:4000/v1/movies?genres=western"
{
    "movies": []
}
```

This is shaping up nicely. Our API endpoint is now returning the appropriately filtered movie records, and we have a pattern that we can easily extend to include other filtering rules in the future (such as a filter on the movie year or runtime) if we want to.

dego_vila@yahoo.com 30 Oct 2026

Full-text search

In this chapter we're going to make our movie title filter easier to use by adapting it to support *partial matches*, rather than requiring a match on the full title. So, for example, if a client wants to find *The Breakfast Club* they will be able to find it with just the query string `title=breakfast`.

There are a few different ways we could implement this feature in our codebase, but an effective and intuitive method (from a client point of view) is to leverage PostgreSQL's *full-text search* functionality, which allows you to perform natural language searches on text fields in your database.

PostgreSQL full-text search is a powerful and highly configurable tool, and explaining how it works and the available options in full could easily take up a whole book in itself. So we'll keep the explanations in this chapter high level, and focus on the practical implementation.

To implement a basic full-text search on our `title` field, we're going to update our SQL query to look like this:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY id
```

That looks pretty complicated at first glance, so let's break it down and explain what's going on.

The `to_tsvector('simple', title)` function takes a movie title and splits it into *lexemes*. We specify the `simple` configuration, which means that the lexemes are just lowercase versions of the words in the title[†]. For example, the movie title `"The Breakfast Club"` would be split into the lexemes `'breakfast' 'club' 'the'`.

> † Other 'non-simple' configurations may apply additional rules to the lexemes, such as the removal of common words or applying language-specific stemming.

The `plainto_tsquery('simple', $1)` function takes a search value and turns it into a formatted *query term* that PostgreSQL full-text search can understand. It normalizes the search value (again using the `simple` configuration), strips any special characters, and

dego_vila@yahoo.com 30 Oct 2026

inserts the *and operator* `&` between the words. As an example, the search value `"The Club"` would result in the query term `'the' & 'club'`.

The `@@` operator is the matches operator. In our statement we are using it to check whether the generated *query term matches the lexemes in the title*. To continue the example, the query term `'the' & 'club'` will match rows which contain *both* lexemes `'the'` and `'club'`.

There are a lot of specialist words in the paragraphs above, but if we illustrate it with a couple of examples it's actually very intuitive:

```
// Return all movies where the title includes the case-insensitive word 'panther'.
/v1/movies?title=panther

// Return all movies where the title includes the case-insensitive words 'the' and
// 'club'.
/v1/movies?title=the+club
```

Let's go ahead and put this into action. Open up your `internal/models/movies.go` file and update the `GetAll()` method to use the new SQL query like so:

File: internal/models/movies.go

```
package models

...

func (m MovieModel) GetAll(title string, genres []string, filters Filters) ([]Movie, error) {
    // Use full-text search for the title filter.
    query := `
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
        AND (genres @> $2 OR $2 = '{}')
        ORDER BY id`

    // Nothing else below needs to change.
    ...
}
```

If you're following along, restart the application and try making some requests with different values for the movie title. You should find that partial searches now work as we described above.

For example:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?title=panther"
{
    "movies": [
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}

$ curl "localhost:4000/v1/movies?title=the+club"
{
    "movies": [
        {
            "id": 4,
            "title": "The Breakfast Club",
            "year": 1985,
            "runtime": "97m",
            "genres": [
                "drama"
            ],
            "version": 7
        }
    ]
}
```

Adding indexes

To keep our SQL query performing quickly as the dataset grows, it's sensible to use indexes to help avoid full table scans and avoid generating the lexemes for the `title` field every time the query is run.

> **Note:** If you are not familiar with the concept of indexes in SQL databases, the official PostgreSQL documentation provides a good introduction and a summary of different index types. I recommend reading these quickly to give you an overview before continuing.

In our case it makes sense to create GIN indexes on both the `genres` field and the lexemes generated by `to_tsvector()`, both of which are used in the `WHERE` clause of our SQL query.

If you're following along, go ahead and create a new pair of migration files:

dego_vila@yahoo.com 30 Oct 2026

```
$ migrate create -seq -ext .sql -dir ./migrations add_movies_indexes
```

Then add the following statements to the ‘up’ and ‘down’ migration files to create and drop the necessary indexes:

File: migrations/000003_add_movies_indexes.up.sql

```
CREATE INDEX IF NOT EXISTS movies_title_idx ON movies USING GIN (to_tsvector('simple', title));
CREATE INDEX IF NOT EXISTS movies_genres_idx ON movies USING GIN (genres);
```

File: migrations/000003_add_movies_indexes.down.sql

```
DROP INDEX IF EXISTS movies_title_idx;
DROP INDEX IF EXISTS movies_genres_idx;
```

Once that’s done, you should be able to execute the ‘up’ migration to add the indexes to your database:

```
$ migrate -path ./migrations -database $GREENLIGHT_DB_DSN up
3/u add_movies_indexes (38.638777ms)
```

Additional information

Non-simple configuration and more information

As mentioned above, you can also use a language-specific configuration for full-text searches instead of the `simple` configuration that we’re currently using. When you create lexemes or query terms using a language-specific configuration, it will strip out common words for the language and perform word stemming.

So, for example, if you use the `english` configuration, then the lexemes generated for `"One Flew Over the Cuckoo's Nest"` would be `'cuckoo' 'flew' 'nest' 'one'`. Or with the `spanish` configuration, the lexemes for `"Los lunes al sol"` would be `'lun' 'sol'`.

You can retrieve a list of all available configurations by running the `\dF` meta-command in PostgreSQL:

dego_vila@yahoo.com 30 Oct 2026

```
postgres=# \dF
             List of text search configurations
   Schema   |    Name    |              Description
------------+------------+-------------------------------------
 pg_catalog | arabic     | configuration for arabic language
 pg_catalog | danish     | configuration for danish language
 pg_catalog | dutch      | configuration for dutch language
 pg_catalog | english    | configuration for english language
 ...
```

And if you wanted to use the `english` configuration to search our movies, you could update the SQL query like so:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (to_tsvector('english', title) @@ plainto_tsquery('english', $1) OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY id
```

If you'd like to learn more about PostgreSQL full-text search, the official documentation is an excellent source of information and examples.

Using STRPOS and ILIKE

If you don't want to use full-text search for the partial movie title lookup, some alternatives are the PostgreSQL `STRPOS()` function and `ILIKE` operator.

The PostgreSQL `STRPOS()` function allows you to check for the existence of a substring in a particular database field. We could use it in our SQL query like this:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (STRPOS(LOWER(title), LOWER($1)) > 0 OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY id
```

From a client perspective, the downside of this is that it may return some unintuitive results. For example, searching for `title=the` would return both *The Breakfast Club* and *Black Panther* in our dataset.

From a server perspective it's also not ideal for large datasets. Because there's no effective way to index the `title` field to see if the `STRPOS()` condition is met, it means the query could potentially require a full-table scan each time it is run.

Another option is the `ILIKE` operator, which allows you to find rows which match a specific (case-insensitive) pattern. We could use it in our SQL query like so:

dego_vila@yahoo.com 30 Oct 2026

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (title ILIKE $1 OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY id
```

This approach would be better from a server point of view because it's possible to create an index on the `title` field using the `pg_trgm` extension and a GIN index (for details please see this post).

From the client side it's arguably better that the `STRPOS()` approach too, as they can control the matching behavior by prefixing/suffixing the search term with a `%` wildcard character (which will need to be escaped to `%25` in the URL query string). For example, to search for movies with a title that starts with "the", a client could send the query string parameter `title=the%25`.

dego_vila@yahoo.com 30 Oct 2026

Chapter 9.6.

Sorting Lists

Now let's update our `GET /v1/movies` endpoint so that the client can specify how the movies are sorted in the JSON response.

As we briefly explained earlier, we want to let the client control the sort order via a query string parameter in the format `sort={-}{field_name}`, where an optional `-` prefix is used to indicate a descending sort order. For example:

```
// Sort the movies on the title field in ascending alphabetical order.
/v1/movies?sort=title

// Sort the movies on the year field in descending numerical order.
/v1/movies?sort=-year
```

Behind the scenes, we will want to translate this into an `ORDER BY` clause in our SQL query, so that a query string parameter like `sort=-year` would result in a SQL query like this:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (STRPOS(LOWER(title), LOWER($1)) > 0 OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY year DESC --<-- Order the result by descending year
```

The difficulty here is that the values for the `ORDER BY` clause need to be generated at runtime based on the query string values from the client. Ideally we'd use placeholder parameters to insert these dynamic values into our query, but unfortunately it's *not possible to use placeholder parameters for column names or SQL keywords* (including `ASC` and `DESC`).

So instead, we'll need to interpolate these dynamic values into our query using `fmt.Sprintf()` — making sure that the values are checked against a strict safelist first to prevent a SQL injection attack.

When working with PostgreSQL, it's also important to be aware that the order of returned rows *is only guaranteed by the rules that your `ORDER BY` clause imposes*. From the official documentation:

> *If sorting is not chosen, the rows will be returned in an unspecified order. The actual order in that case will depend on the scan and join plan types and the order on disk, but it must not be relied on. A particular output ordering can only be guaranteed if the sort step is explicitly*

dego_vila@yahoo.com 30 Oct 2026

> *chosen.*

That means if we don't include an `ORDER BY` clause, then PostgreSQL may return the movies in any order — *and the order may or may not change each time the query is run*.

Likewise, in our database, multiple movies will have the same `year` value. If we order based on the `year` column, then the movies are guaranteed to be ordered by year, but the movies *for* a particular year could appear in any order at any time.

This point is particularly important in the context of an endpoint which provides pagination. We need to make sure that the order of movies is perfectly consistent between requests to prevent items in the list 'jumping' between the pages.

Fortunately, guaranteeing the order is simple — we just need to ensure that the `ORDER BY` clause always includes a primary key column (or another column with a unique constraint on it). So, in our case, we can apply a secondary sort on the `id` column to ensure an always-consistent order. Like so:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (STRPOS(LOWER(title), LOWER($1)) > 0 OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY year DESC, id ASC
```

Implementing sorting

To get the dynamic sorting working, let's begin by updating our `Filters` struct to include some `sortColumn()` and `sortDirection()` helpers that transform a query string value (like `-year`) into values we can use in our SQL query.

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/filters.go

```
package models

import (
    "slices"  // New import
    "strings" // New import

    "greenlight.alexedwards.net/internal/validator"
)

type Filters struct {
    Page         int
    PageSize     int
    Sort         string
    SortSafelist []string
}

// The sortColumn() function checks that the client-provided Sort field matches one of
// the entries in our safelist and if it does, extract the column name from the Sort
// field by stripping the leading hyphen character (if one exists).
func (f Filters) sortColumn() string {
    if !slices.Contains(f.SortSafelist, f.Sort) {
        panic("unsafe sort parameter: " + f.Sort)
    }

    return strings.TrimPrefix(f.Sort, "-")
}

// Return the sort direction ("ASC" or "DESC") depending on the prefix character of the
// Sort field.
func (f Filters) sortDirection() string {
    if strings.HasPrefix(f.Sort, "-") {
        return "DESC"
    }

    return "ASC"
}

...
```

Notice that the `sortColumn()` function is constructed in such a way that it will panic if the client-provided `Sort` value doesn't match one of the entries in our safelist. In theory this shouldn't happen — the `Sort` value should have already been checked well in advance by calling the `ValidateFilters()` function — but this is a sensible last-ditch check to include to stop a SQL injection attack from occurring.

> **Note:** If you want, you could change the `sortColumn()` function to return an error, rather than panicking, but on balance I think it's reasonable to panic here. The panic is used as a fail-safe 'guard' to prevent a potentially dangerous operation from happening, rather than being used to handle an error that we expect to see in normal operation. If you're unsure about when it's appropriate to use `panic()`, you might like to read this blog post for more discussion on the subject.

dego_vila@yahoo.com 30 Oct 2026

Now let's update our `internal/models/movies.go` file to call those methods, and interpolate the return values into our SQL query's `ORDER BY` clause. Like so:

File: internal/models/movies.go

```
package models

import (
    "context"
    "database/sql"
    "errors"
    "fmt" // New import
    "time"

    "greenlight.alexedwards.net/internal/validator"

    "github.com/lib/pq"
)

...

func (m MovieModel) GetAll(title string, genres []string, filters Filters) ([]Movie, error) {
    // Add an ORDER BY clause and interpolate the sort column and direction. Importantly
    // notice that we also include a secondary sort on the movie ID to ensure a
    // consistent ordering.
    query := fmt.Sprintf(`
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
        AND (genres @> $2 OR $2 = '{}')
        ORDER BY %s %s, id ASC`, filters.sortColumn(), filters.sortDirection())

    // Nothing else below needs to change.
    ...
}
```

And once that's done, we should be ready to try this out.

Restart the application then, as an example, go ahead and try making a request for the movies sorted by descending `title`. You should get a response which looks like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?sort=-title"
{
    "movies": [
        {
            "id": 4,
            "title": "The Breakfast Club",
            "year": 1985,
            "runtime": "97m",
            "genres": [
                "drama"
            ],
            "version": 7
        },
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}
```

In contrast, using a sort parameter of descending `runtime` should yield a response in a totally different order. Similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?sort=-runtime"
{
    "movies": [
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        },
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 4,
            "title": "The Breakfast Club",
            "year": 1985,
            "runtime": "97m",
            "genres": [
                "drama"
            ],
            "version": 7
        }
    ]
}
```

dego_vila@yahoo.com 30 Oct 2026

Paginating lists

If you have an endpoint which returns a list with hundreds or thousands of records, then for performance or usability reasons you might want to implement some form of *pagination* on the endpoint — so that it only returns a subset of the records in a single HTTP response.

To help demonstrate how to do this, in this chapter we're going to update the `GET /v1/movies` endpoint so that it supports the concept of 'pages' and a client can request a specific 'page' of the movies list by using our `page` and `page_size` query string parameters. For example:

```
// Return the 5 records on page 1 (records 1-5 in the dataset)
/v1/movies?page=1&page_size=5

// Return the next 5 records on page 2 (records 6-10 in the dataset)
/v1/movies?page=2&page_size=5

// Return the next 5 records on page 3 (records 11-15 in the dataset)
/v1/movies?page=3&page_size=5
```

Basically, changing the `page_size` parameter will alter the number of movies shown on each 'page', and increasing the `page` parameter by one will show you the next 'page' of movies in the list.

The LIMIT and OFFSET clauses

Behind the scenes, the simplest way to support this style of pagination is by adding `LIMIT` and `OFFSET` clauses to our SQL query.

The `LIMIT` clause allows you to set the maximum number of records that a SQL query should return, and `OFFSET` allows you to 'skip' a specific number of rows before starting to return records from the query.

Within our application, we'll just need to translate the `page` and `page_size` values provided by the client to the appropriate `LIMIT` and `OFFSET` values for our SQL query. The math is pretty straightforward:

```
LIMIT = page_size
OFFSET = (page - 1) * page_size
```

dego_vila@yahoo.com 30 Oct 2026

Or to give a concrete example, if a client makes the following request:

```
/v1/movies?page_size=5&page=3
```

We would need to 'translate' this into the following SQL query:

```
SELECT id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY %s %s, id ASC
LIMIT 5 OFFSET 10
```

Let's start by adding some helper methods to our `Filters` struct for calculating the appropriate `LIMIT` and `OFFSET` values.

If you're following along, go ahead and update the `internal/models/filters.go` file like so:

File: internal/models/filters.go

```
package models

...

type Filters struct {
    Page         int
    PageSize     int
    Sort         string
    SortSafelist []string
}

...

func (f Filters) limit() int {
    return f.PageSize
}

func (f Filters) offset() int {
    return (f.Page - 1) * f.PageSize
}

...
```

> **Note:** The `offset()` method involves multiplying two `int` values, which can theoretically cause an integer overflow. However, this risk is mitigated by the validation rules we created in `ValidateFilters()`, where we cap `page_size` at `100` and `page` at `10,000,000`. As a result, the maximum possible value returned by `offset()` is `990,000,000`, which remains well below `math.MaxInt`, the threshold for overflow.

dego_vila@yahoo.com 30 Oct 2026

Updating the database model

As the final stage in this process, we need to update our database model's `GetAll()` method to add the appropriate `LIMIT` and `OFFSET` clauses to the SQL query.

File: internal/models/movies.go

```
package models

...

func (m MovieModel) GetAll(title string, genres []string, filters Filters) ([]Movie, error) {
    // Update the SQL query to include the LIMIT and OFFSET clauses with placeholder
    // parameter values.
    query := fmt.Sprintf(`
        SELECT id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
        AND (genres @> $2 OR $2 = '{}')
        ORDER BY %s %s, id ASC
        LIMIT $3 OFFSET $4`, filters.sortColumn(), filters.sortDirection())

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // As our SQL query now has quite a few placeholder parameters, let's collect the
    // values for the placeholders in a slice. Notice here how we call the limit() and
    // offset() methods on the Filters struct to get the appropriate values for the
    // LIMIT and OFFSET clauses.
    args := []any{title, pq.Array(genres), filters.limit(), filters.offset()}

    // And then pass the args slice to QueryContext() as a variadic parameter.
    rows, err := m.DB.QueryContext(ctx, query, args...)
    if err != nil {
        return nil, err
    }

    // The remaining code doesn't need to change.
    ...
}
```

Once that's done we should be ready to try this out.

Restart the server, then go ahead and make the following request with a `page_size=2` parameter:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?page_size=2"
{
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}
```

That’s looking good. Our endpoint is now returning the first two movie records only from our database (using the default sort order of ascending movie ID).

Next, try requesting the second page of results. If you’ve been following along, this page should include the one remaining record in our system, like so:

```
$ curl "localhost:4000/v1/movies?page_size=2&page=2"
{
    "movies": [
        {
            "id": 4,
            "title": "The Breakfast Club",
            "year": 1985,
            "runtime": "97m",
            "genres": [
                "drama"
            ],
            "version": 7
        }
    ]
}
```

If you try to request the third page, you should get an empty JSON array in the response like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?page_size=2&page=3"
{
    "movies": []
}
```

dego_vila@yahoo.com 30 Oct 2026

Returning pagination metadata

At this point, the pagination on our `GET /v1/movies` endpoint is working nicely, but it would be even better if we could include some additional metadata along with the response. Information like the *current* and *last* page numbers, and the *total number of available records* would help give the client context about the response and make navigating through the pages easier.

In this chapter we'll improve the response so that it includes additional pagination metadata, similar to this:

```
{
    "metadata": {
        "current_page": 1,
        "page_size": 20,
        "first_page": 1,
        "last_page": 42,
        "total_records": 832
    },
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        ...
    ]
}
```

Calculating the total records

The challenging part of doing this is generating the `total_records` figure. We want this to reflect the total number of available records *given the* `title` *and* `genres` *filters that are applied* — not the absolute total of records in the `movies` table.

A neat way to do this is to adapt our existing SQL query to include a window function which counts the total number of filtered rows, like so:

dego_vila@yahoo.com 30 Oct 2026

```
SELECT count(*) OVER(), id, created_at, title, year, runtime, genres, version
FROM movies
WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
AND (genres @> $2 OR $2 = '{}')
ORDER BY %s %s, id ASC
LIMIT $3 OFFSET $4
```

The inclusion of the `count(*) OVER()` expression at the start of the query will result in the filtered record count being included as the first value in each row. A bit like this:

```
 count | id |       created_at       |       title        | year |    runtime    |          genres           | version
-------+----+------------------------+--------------------+------+---------------+---------------------------+---------
     3 |  1 | 2020-11-27 17:17:25+01 | Moana              | 2016 | 6420000000000 | {animation,adventure}     |       1
     3 |  2 | 2020-11-27 18:01:45+01 | Black Panther      | 2018 | 8040000000000 | {sci-fi,action,adventure} |       2
     3 |  4 | 2020-11-27 18:02:20+01 | The Breakfast Club | 1985 | 5760000000000 | {drama}            |       6
```

When PostgreSQL executes this SQL query, the (very simplified) sequence of events runs broadly like this:

1. The `WHERE` clause is used to filter the data in the `movies` table and get the *qualifying rows*.
2. The window function `count(*) OVER()` is applied, which counts all the qualifying rows.
3. The `ORDER BY` rules are applied and the qualifying rows are sorted.
4. The `LIMIT` and `OFFSET` rules are applied and the appropriate sub-set of sorted qualifying rows is returned.

Updating the code

With that brief explanation out of the way, let's get this up and running. We'll begin by updating the `internal/models/filters.go` file to define a new `Metadata` struct to hold the pagination metadata, along with a helper to calculate the values. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/filters.go

```
package models

import (
    "slices"
    "strings"

    "greenlight.alexedwards.net/internal/validator"
)

...

// Define a new Metadata struct for holding the pagination metadata.
type Metadata struct {
    CurrentPage  int `json:"current_page,omitzero"`
    PageSize     int `json:"page_size,omitzero"`
    FirstPage    int `json:"first_page,omitzero"`
    LastPage     int `json:"last_page,omitzero"`
    TotalRecords int `json:"total_records,omitzero"`
}

// The calculateMetadata() function calculates the appropriate pagination metadata
// values given the total number of records, current page, and page size values. Note
// that when the last page value is calculated we are dividing two int values, and
// when dividing integer types in Go the result will also be an integer type, with
// the modulus (or remainder) dropped. So, for example, if there were 12 records in total
// and a page size of 5, the last page value would be (12+5-1)/5 = 3.2, which is then
// truncated to 3 by Go.
func calculateMetadata(totalRecords, page, pageSize int) Metadata {
    if totalRecords == 0 {
        // Note that we return an empty Metadata struct if there are no records.
        return Metadata{}
    }

    return Metadata{
        CurrentPage:  page,
        PageSize:     pageSize,
        FirstPage:    1,
        LastPage:     (totalRecords + pageSize - 1) / pageSize,
        TotalRecords: totalRecords,
    }
}
```

Then we need to head back to our `GetAll()` method and update it to use our new SQL query (with the window function) to get the total records count. Then, if everything works successfully, we'll use the `calculateMetadata()` function to generate the pagination metadata and return it along with the movie data.

Go ahead and update the `GetAll()` function like so:

File: internal/models/movies.go

```
package models

...

// Update the function signature to return a Metadata struct.
func (m MovieModel) GetAll(title string, genres []string, filters Filters) ([]Movie, Metadata, error) {
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Update the SQL query to include the window function which counts the total
    // (filtered) records.
    query := fmt.Sprintf(`
        SELECT count(*) OVER(), id, created_at, title, year, runtime, genres, version
        FROM movies
        WHERE (to_tsvector('simple', title) @@ plainto_tsquery('simple', $1) OR $1 = '')
        AND (genres @> $2 OR $2 = '{}')
        ORDER BY %s %s, id ASC
        LIMIT $3 OFFSET $4`, filters.sortColumn(), filters.sortDirection())

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    args := []any{title, pq.Array(genres), filters.limit(), filters.offset()}

    rows, err := m.DB.QueryContext(ctx, query, args...)
    if err != nil {
        return nil, Metadata{}, err // Update this to return an empty Metadata struct.
    }

    defer rows.Close()

    // Declare a totalRecords variable.
    totalRecords := 0
    movies := []Movie{}

    for rows.Next() {
        var movie Movie

        err := rows.Scan(
            &totalRecords, // Scan the count from the window function into totalRecords.
            &movie.ID,
            &movie.CreatedAt,
            &movie.Title,
            &movie.Year,
            &movie.Runtime,
            pq.Array(&movie.Genres),
            &movie.Version,
        )
        if err != nil {
            return nil, Metadata{}, err // Update this to return an empty Metadata struct.
        }

        movies = append(movies, movie)
    }

    if err = rows.Err(); err != nil {
        return nil, Metadata{}, err // Update this to return an empty Metadata struct.
    }

    // Generate a Metadata struct, passing in the total record count and pagination
    // parameters from the client.
    metadata := calculateMetadata(totalRecords, filters.Page, filters.PageSize)

    // Include the metadata struct when returning.
    return movies, metadata, nil
}
```

Finally, we need to update our `listMoviesHandler` handler to receive the `Metadata` struct returned by `GetAll()` and include the information in the JSON response. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/movies.go

```
package main

...

func (app *application) listMoviesHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Title  string
        Genres []string
        models.Filters
    }

    v := validator.New()

    qs := r.URL.Query()

    input.Title = app.readString(qs, "title", "")
    input.Genres = app.readCSV(qs, "genres", []string{})

    input.Filters.Page = app.readInt(qs, "page", 1, v)
    input.Filters.PageSize = app.readInt(qs, "page_size", 20, v)

    input.Filters.Sort = app.readString(qs, "sort", "id")
    input.Filters.SortSafelist = []string{"id", "title", "year", "runtime", "-id", "-title", "-year", "-runtime"}

    if models.ValidateFilters(v, input.Filters); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Accept the metadata struct as a return value.
    movies, metadata, err := app.models.Movies.GetAll(input.Title, input.Genres, input.Filters)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Include the metadata in the response envelope.
    err = app.writeJSON(w, http.StatusOK, envelope{"movies": movies, "metadata": metadata}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Feel free to restart the API and try out this new functionality by making some different requests to the `GET /v1/movies` endpoint. You should find that the correct pagination metadata is now included in the response. For example:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?page=1&page_size=2"
{
    "metadata": {
        "current_page": 1,
        "page_size": 2,
        "first_page": 1,
        "last_page": 2,
        "total_records": 3
    },
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}
```

And if you try making a request with a filter applied, you should see that the `last_page` value and `total_records` count changes to reflect the applied filters. For example, by requesting only movies with the genre “adventure” we can see that the `total_records` count drops down to `2`:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl "localhost:4000/v1/movies?genres=adventure"
{
    "metadata": {
        "current_page": 1,
        "page_size": 20,
        "first_page": 1,
        "last_page": 1,
        "total_records": 2
    },
    "movies": [
        {
            "id": 1,
            "title": "Moana",
            "year": 2016,
            "runtime": "107m",
            "genres": [
                "animation",
                "adventure"
            ],
            "version": 1
        },
        {
            "id": 2,
            "title": "Black Panther",
            "year": 2018,
            "runtime": "134m",
            "genres": [
                "sci-fi",
                "action",
                "adventure"
            ],
            "version": 2
        }
    ]
}
```

Lastly, if you make a request with a too-high page value, you should get a response with an empty metadata object and an empty movies array, like this:

```
$ curl "localhost:4000/v1/movies?page=100"
{
    "metadata": {},
    "movies": []
}
```

Over the last few chapters, we've had to put in a lot of work on the `GET /v1/movies` endpoint. But the end result is really powerful. The client now has a lot of control over what their response contains, with filtering, pagination and sorting all supported.

With the `Filters` struct that we've created, we've also got something that we can easily drop into any other endpoints that need pagination and sorting functionality in the future. And if you take a step back and look at the final code we've written in the `listMoviesHandler` and our database model's `GetAll()` method, there isn't *so much* more code than there was in the earlier iterations of the endpoint.

dego_vila@yahoo.com 30 Oct 2026

Chapter 10.

Rate limiting

If you're building an API for public use, then it's quite likely that you'll want to implement some form of *rate limiting* to prevent clients from making *too many requests too quickly*, and putting excessive strain on your server.

In this section of the book we're going to create some middleware to help with that.

Essentially, we want this middleware to check how many requests have been received in the last 'N' seconds and — if there have been too many — then it should send the client a `429 Too Many Requests` response. We'll position this middleware before our main application handlers, so that it carries out this check *before* we do any processing like decoding a JSON request body or querying our database.

You'll learn:

- About the principles behind *token-bucket* rate limiter algorithms and how we can apply them in the context of an API or web application.
- How to create middleware to rate-limit requests to your API endpoints, first by making a single global rate limiter, then extending it to support per-client limiting based on IP address.
- How to make rate limiter behavior configurable at runtime, including disabling the rate limiter altogether for testing purposes.

dego_vila@yahoo.com 30 Oct 2026

Chapter 10.1.

Global rate limiting

Let's build things up slowly and start by creating a single *global rate limiter* for our application. This will consider *all the requests* that our API receives (rather than having separate rate limiters for every individual client).

Instead of writing our own rate-limiting logic from scratch, which would be quite complex and time-consuming, we can leverage the `x/time/rate` package to help us here. This provides a tried-and-tested implementation of a *token bucket* rate limiter.

If you're following along, please go ahead and download the latest version of this package like so:

```
$ go get golang.org/x/time/rate@latest
go: downloading golang.org/x/time v0.16.0
go: added golang.org/x/time v0.16.0
```

Before we start writing any code, let's take a moment to explain how token-bucket rate limiters work. The official `x/time/rate` documentation says:

> *A Limiter controls how frequently events are allowed to happen. It implements a "token bucket" of size `b`, initially full and refilled at rate `r` tokens per second.*

Putting that into the context of our API application…

- We will have a bucket that starts with `b` tokens in it.
- Each time we receive an HTTP request, we will remove one token from the bucket.
- Every `1/r` seconds, a token is added back to the bucket — up to a maximum of `b` total tokens.
- If we receive an HTTP request and the bucket is empty, then we should return a `429 Too Many Requests` response.

In practice this means that our application would allow a maximum 'burst' of `b` HTTP requests in quick succession, but over time it would allow an average of `r` requests per second.

In order to create a token bucket rate limiter from `x/time/rate`, we will need to use the `NewLimiter()` function. This has a signature which looks like this:

dego_vila@yahoo.com 30 Oct 2026

```
// Note that the Limit type is an 'alias' for float64.
func NewLimiter(r Limit, b int) *Limiter
```

So if we want to create a rate limiter which allows an average of 2 requests per second, with a maximum of 4 requests in a single 'burst', we could do so with the following code:

```
// Allow 2 requests per second, with a maximum of 4 requests in a burst.
limiter := rate.NewLimiter(2, 4)
```

Enforcing a global rate limit

OK, with that high-level explanation out of the way, let's jump into some code and see how this works in practice.

One of the nice things about the middleware pattern that we are using is that it is straightforward to include 'initialization' code which only runs once when we *wrap* something with the middleware, rather than running on every request that the middleware handles.

```
func (app *application) exampleMiddleware(next http.Handler) http.Handler {

    // Any code here will run only once, when we wrap something with the middleware.

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {

        // Any code here will run for every request that the middleware handles.

        next.ServeHTTP(w, r)
    })
}
```

In our case, we'll make a new `rateLimit()` middleware method which creates a new rate limiter as part of the 'initialization' code, and then uses this rate limiter for every request that it subsequently handles.

If you're following along, open up the `cmd/api/middleware.go` file and create the middleware like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/middleware.go

```
package main

import (
    "fmt"
    "net/http"

    "golang.org/x/time/rate" // New import
)

...

func (app *application) rateLimit(next http.Handler) http.Handler {
    // Initialize a new rate limiter which allows an average of 2 requests per second,
    // with a maximum of 4 requests in a single 'burst'.
    limiter := rate.NewLimiter(2, 4)

    // The function we are returning is a closure, which 'closes over' the limiter
    // variable.
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Call limiter.Allow() to see if the request is permitted, and if it's not,
        // then we call the rateLimitExceededResponse() helper to return a 429 Too Many
        // Requests response (we will create this helper in a minute).
        if !limiter.Allow() {
            app.rateLimitExceededResponse(w, r)
            return
        }

        next.ServeHTTP(w, r)
    })
}
```

In this code, whenever we call the `Allow()` method on the rate limiter exactly one token will be consumed from the bucket. If there are no tokens left in the bucket, then `Allow()` will return `false` and that acts as the trigger for us to send the client a `429 Too Many Requests` response.

It's also important to note that the code behind the `Allow()` method is protected by a mutex and is safe for concurrent use.

Let's now go to our `cmd/api/errors.go` file and create the `rateLimitExceededResponse()` helper. Like so:

File: cmd/api/errors.go

```
package main

...

func (app *application) rateLimitExceededResponse(w http.ResponseWriter, r *http.Request) {
    message := "rate limit exceeded"
    app.errorResponse(w, r, http.StatusTooManyRequests, message)
}
```

dego_vila@yahoo.com 30 Oct 2026

Then, lastly, in the `cmd/api/routes.go` file we want to add the `rateLimit()` middleware to our middleware chain. This should come after our panic recovery middleware (so that any panics in `rateLimit()` are recovered), but otherwise we want it to be used as early as possible to prevent unnecessary work for our server.

Go ahead and update the file accordingly:

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.listMoviesHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    // Wrap the router with the rateLimit() middleware.
    return app.recoverPanic(app.rateLimit(router))
}
```

Now we should be ready to try this out!

Restart the API, then in another terminal window execute the following command to issue a batch of 6 requests to our `GET /v1/healthcheck` endpoint in quick succession. You should get responses which look like this:

dego_vila@yahoo.com 30 Oct 2026

```
$ for i in {1..6}; do curl http://localhost:4000/v1/healthcheck; done
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "error": "rate limit exceeded"
}
{
    "error": "rate limit exceeded"
}
```

We can see from this that the first 4 requests succeed, due to our limiter being set up to permit a ‘burst’ of 4 requests in quick succession. But once those 4 requests were used up, the tokens in the bucket ran out and our API began to return the `"rate limit exceeded"` error response instead.

If you wait a second and rerun this command, you should find that some requests in the second batch succeed again, due to the token bucket being refilled at the rate of two tokens every second.

dego_vila@yahoo.com 30 Oct 2026

IP-based rate limiting

Using a global rate limiter can be useful when you want to enforce a strict limit on the total rate of requests to your API, and you don't care about where the requests are coming from. But it's generally more common to want an individual rate limiter for each client, so that one bad client making too many requests doesn't affect all the others.

A conceptually straightforward way to implement this is to create an in-memory *map of rate limiters*, using the IP address for each client as the map key.

Each time a new client makes a request to our API, we will initialize a new rate limiter and add it to the map. For any subsequent requests, we will retrieve the client's rate limiter from the map and check whether the request is permitted by calling its `Allow()` method, just like we did before.

But there's one thing to be aware of: *by default, maps are not safe for concurrent use*. This is a problem for us because our `rateLimit()` middleware may be running in multiple goroutines at the same time (remember, Go's `http.Server` handles each HTTP request in its own goroutine).

From the Go blog:

> *Maps are not safe for concurrent use: it's not defined what happens when you read and write to them simultaneously. If you need to read from and write to a map from concurrently executing goroutines, the accesses must be mediated by some kind of synchronization mechanism.*

So to get around this, we'll need to synchronize access to the map of rate limiters using a `sync.Mutex` (a mutual exclusion lock), so that only one goroutine can read or write to the map at any moment in time.

> **Important:** How mutexes work, and how to use them, can be quite confusing if you haven't encountered them before and it's impossible to fully explain in a few short sentences. I've written a much more detailed article — Understanding Mutexes — which provides a proper explanation. If you're not already confident with mutexes, I highly recommend reading this before you continue.

Now let's talk about IP addresses.

dego_vila@yahoo.com 30 Oct 2026

The request's `r.RemoteAddr` field should contain the IP address of the client making the request. But… in the real world it's possible that there will be proxy servers positioned between your application and the client, meaning that the IP address stored in `r.RemoteAddr` may not actually be the true IP address of the original client — instead it might be the IP address of a proxy.

Well-behaved proxies will typically add a `X-Forwarded-For` or `X-Real-IP` header to the request, containing the IP of the original client. So we can increase the chance of getting the real client's IP by checking for these headers and — if they exist — using the IP address from them.

Although we could write the logic to do this ourselves, I recommend using the `realip` package to help with this. It's very small, and simply retrieves the client IP address from any `X-Forwarded-For` or `X-Real-IP` headers, falling back on `r.RemoteAddr` if neither of them are present.

If you're following along, go ahead and install the latest version of `realip` using the `go get` command:

```
$ go get github.com/tomasen/realip@latest
go: downloading github.com/tomasen/realip v0.0.0-20180522021738-f0c99a92ddce
go get: added github.com/tomasen/realip v0.0.0-20180522021738-f0c99a92ddce
```

OK, with that setup out of the way, let's jump into the code and update our `rateLimit()` middleware to implement the changes.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/middleware.go

```
package main

import (
    "fmt"
    "net/http"
    "sync" // New import

    "github.com/tomasen/realip" // New import
    "golang.org/x/time/rate"
)

...

func (app *application) rateLimit(next http.Handler) http.Handler {
    // Declare a mutex and a map to hold the clients' IP addresses and rate limiters.
    var (
        mu      sync.Mutex
        clients = make(map[string]*rate.Limiter)
    )

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Use the realip.FromRequest() function to get the client's IP address.
        ip := realip.FromRequest(r)

        // Lock the mutex to prevent this code from being executed concurrently.
        mu.Lock()

        // Check to see if the IP address already exists in the map. If it doesn't, then
        // initialize a new rate limiter and add the IP address and limiter to the map.
        if _, found := clients[ip]; !found {
            clients[ip] = rate.NewLimiter(2, 4)
        }

        // Call the Allow() method on the rate limiter for the current IP address. If
        // the request isn't allowed, unlock the mutex and send a 429 Too Many Requests
        // response, just like before.
        if !clients[ip].Allow() {
            mu.Unlock()
            app.rateLimitExceededResponse(w, r)
            return
        }

        // Very importantly, unlock the mutex before calling the next handler in the
        // chain. Notice that we DON'T use defer to unlock the mutex, as that would mean
        // that the mutex isn't unlocked until all the handlers downstream of this
        // middleware have also returned.
        mu.Unlock()

        next.ServeHTTP(w, r)
    })
}
```

Deleting old limiters

The code above will work, but there's a slight problem — the `clients` map will grow indefinitely, taking up more and more resources with every new IP address and rate limiter that we add.

dego_vila@yahoo.com 30 Oct 2026

To prevent this, let's update our code so that we also record the *last seen* time for each client. We can then run a background goroutine in which we periodically delete any clients that we haven't seen recently from the `clients` map.

To make this work, we'll need to create a custom `client` struct which holds both the rate limiter and last seen time for each client, and launch the background cleanup goroutine when initializing the middleware.

Like so:

File: cmd/api/middleware.go

```
package main

import (
    "fmt"
    "net/http"
    "sync"
    "time" // New import

    "github.com/tomasen/realip"
    "golang.org/x/time/rate"
)

...

func (app *application) rateLimit(next http.Handler) http.Handler {
    // Define a client struct to hold the rate limiter and last seen time for each
    // client.
    type client struct {
        limiter  *rate.Limiter
        lastSeen time.Time
    }

    var (
        mu sync.Mutex
        // Update the map so the values are pointers to a client struct.
        clients = make(map[string]*client)
    )

    // Launch a background goroutine which removes old entries from the clients map once
    // every minute.
    go func() {
        for {
            time.Sleep(time.Minute)

            // Lock the mutex to prevent any rate limiter checks from happening while
            // the cleanup is taking place.
            mu.Lock()

            // Loop through all clients. If they haven't been seen within the last three
            // minutes, delete the corresponding entry from the map.
            for ip, client := range clients {
                if time.Since(client.lastSeen) > 3*time.Minute {
                    delete(clients, ip)
                }
            }

            // Importantly, unlock the mutex when the cleanup is complete.
            mu.Unlock()
```

dego_vila@yahoo.com 30 Oct 2026

```
        }
    }()

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        ip := realip.FromRequest(r)

        mu.Lock()

        if _, found := clients[ip]; !found {
            // Create and add a new client struct to the map if it doesn't already exist.
            clients[ip] = &client{limiter: rate.NewLimiter(2, 4)}
        }

        // Update the last seen time for the client.
        clients[ip].lastSeen = time.Now()

        if !clients[ip].limiter.Allow() {
            mu.Unlock()
            app.rateLimitExceededResponse(w, r)
            return
        }

        mu.Unlock()

        next.ServeHTTP(w, r)
    })
}
```

At this point, if you restart the API and try making a batch of requests in quick succession again, you should find that the rate limiter continues to work correctly from the perspective of an individual client — just like it did before.

dego_vila@yahoo.com 30 Oct 2026

```
$ for i in {1..6}; do curl  http://localhost:4000/v1/healthcheck; done
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "error": "rate limit exceeded"
}
{
    "error": "rate limit exceeded"
}
```

Additional information

Distributed applications

Using this pattern for rate-limiting will only work if your API application is running on a single-machine. If your infrastructure is distributed, with your application running on multiple servers behind a load balancer, then you'll need to use an alternative approach.

If you're using HAProxy or Nginx as a load balancer or reverse proxy, both of these have built-in functionality for rate limiting that you can use. Alternatively, you could use a fast database like Redis to maintain a request count for clients, running on a server which all your application servers can communicate with.

dego_vila@yahoo.com 30 Oct 2026

Configuring the rate limiters

At the moment our requests-per-second and burst values are hard-coded into the `rateLimit()` middleware. This is OK, but it would be more flexible if they were configurable at runtime instead. Likewise, it would be useful to have an easy way to turn off rate limiting altogether (for example, you might want to disable it when running benchmarks or carrying out load testing, and all requests are coming a small number of IP addresses).

To make these things configurable, let's head back to our `cmd/api/main.go` file and update the `config` struct and command-line flags like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/main.go

```
package main

...

type config struct {
    port int
    env  string
    db   struct {
        dsn          string
        maxOpenConns int
        maxIdleConns int
        maxIdleTime  time.Duration
    }
    // Add a new limiter struct containing fields for the requests-per-second and burst
    // values, and a boolean field which we can use to enable/disable rate limiting
    // altogether.
    limiter struct {
        rps     float64
        burst   int
        enabled bool
    }
}
...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")

    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    // Create command-line flags to read the settings values into the config struct.
    // Notice that we use true as the default for the 'enabled' setting.
    flag.Float64Var(&cfg.limiter.rps, "limiter-rps", 2, "Rate limiter maximum requests per second")
    flag.IntVar(&cfg.limiter.burst, "limiter-burst", 4, "Rate limiter maximum burst")
    flag.BoolVar(&cfg.limiter.enabled, "limiter-enabled", true, "Enable rate limiter")

    flag.Parse()

    ...
}

...
```

And then let's update our `rateLimit()` middleware to use these settings, like so:

File: cmd/api/middleware.go

```
package main

...

func (app *application) rateLimit(next http.Handler) http.Handler {
    // If rate limiting is not enabled, return the next handler in the chain with
    // with no further action
```

dego_vila@yahoo.com 30 Oct 2026

```
    // with no further action.
    if !app.config.limiter.enabled {
        return next
    }

    type client struct {
        limiter  *rate.Limiter
        lastSeen time.Time
    }

    var (
        mu      sync.Mutex
        clients = make(map[string]*client)
    )

    go func() {
        for {
            time.Sleep(time.Minute)

            mu.Lock()

            for ip, client := range clients {
                if time.Since(client.lastSeen) > 3*time.Minute {
                    delete(clients, ip)
                }
            }

            mu.Unlock()
        }
    }()

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        ip := realip.FromRequest(r)

        mu.Lock()

        if _, found := clients[ip]; !found {
            clients[ip] = &client{
                // Use the requests-per-second and burst values from the config
                // struct.
                limiter: rate.NewLimiter(rate.Limit(app.config.limiter.rps), app.config.limiter.burst),
            }
        }

        clients[ip].lastSeen = time.Now()

        if !clients[ip].limiter.Allow() {
            mu.Unlock()
            app.rateLimitExceededResponse(w, r)
            return
        }

        mu.Unlock()

        next.ServeHTTP(w, r)
    })
}
```

Once that's done, let's try this out by running the API with the `-limiter-burst` flag and the burst value reduced to 2:

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/api/ -limiter-burst=2
time=2026-09-12T13:28:53.179+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:28:53.180+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

If you issue a batch of six requests in quick succession again, you should now find that only the first two succeed:

```
$ for i in {1..6}; do curl  http://localhost:4000/v1/healthcheck; done
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
{
    "error": "rate limit exceeded"
}
{
    "error": "rate limit exceeded"
}
{
    "error": "rate limit exceeded"
}
{
    "error": "rate limit exceeded"
}
```

Similarly, you can try disabling the rate limiter altogether with the `-limiter-enabled=false` flag like so:

```
$ go run ./cmd/api/ -limiter-enabled=false
time=2026-09-12T13:29:39.348+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:29:39.348+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

And you should find that all requests now complete successfully, no matter how many you make.

dego_vila@yahoo.com 30 Oct 2026

```
$ for i in {1..6}; do curl http://localhost:4000/v1/healthcheck; done
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
...
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 11.

Graceful shutdown

In this next section of the book we're going to talk about an important but often overlooked topic: *how to safely stop your running application*.

At the moment, when we stop our API application (usually by pressing `Ctrl+C`) it is terminated *immediately* with no opportunity for in-flight HTTP requests to complete. This isn't ideal for two reasons:

- It means that clients won't receive responses to their in-flight requests — all they will experience is a hard closure of the HTTP connection.
- Any work being carried out by our handlers may be left in an incomplete state.

We're going to mitigate these problems by adding *graceful shutdown* functionality to our application, so that in-flight HTTP requests have the opportunity to complete processing *before* the application is terminated.

You'll learn about:

- Shutdown signals — what they are, how to send them, and how to listen for them in your API application.
- How to use these signals to trigger a graceful shutdown of the HTTP server using Go's `Shutdown()` method.

dego_vila@yahoo.com 30 Oct 2026

Sending shutdown signals

When our application is running, we can terminate it at any time by sending it a specific signal. A common way to do this, which you've probably been using, is by pressing `Ctrl+C` on your keyboard to send an *interrupt* signal — also known as a `SIGINT`.

But this isn't the only type of signal that can stop our application. Some of the other common ones are:

Signal	Description	Keyboard shortcut	Catchable
`SIGINT`	Interrupt from keyboard	`Ctrl+C`	Yes
`SIGQUIT`	Quit from keyboard	`Ctrl+\`	Yes
`SIGKILL`	Kill process (terminate immediately)	-	No
`SIGTERM`	Terminate process in an orderly manner	-	Yes

It's important to explain upfront that some signals are *catchable* and others are not. Catchable signals can be intercepted by our application and either ignored, or used to trigger a certain action (such as a graceful shutdown). Other signals, like `SIGKILL`, are not catchable and cannot be intercepted.

Let's take a quick look at these signals in action. If you're following along, go ahead and start the API application with the same `go run ./cmd/api` command as normal:

```
$ go run ./cmd/api
time=2026-09-12T13:30:48.270+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:30:48.271+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Doing this should start a process with the name `api` on your machine. You can use the `pgrep` command to verify that this process exists, like so:

```
$ pgrep -l api
4414 api
```

In my case, I can see that the `api` process is running and has the *process ID* `4414` (your process ID will most likely be different).

dego_vila@yahoo.com 30 Oct 2026

Once that's confirmed, try sending a `SIGKILL` signal to the `api` process using the `pkill` command like so:

```
$ pkill -SIGKILL api
```

If you go back to the terminal window that is running the API application, you should see that it has been terminated and the final line in the output stream is `signal: killed`. Similar to this:

```
$ go run ./cmd/api
time=2026-09-12T13:30:48.270+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:30:48.271+02:00 level=INFO msg="starting server" addr=:4000 env=development
signal: killed
```

Feel free to repeat the same process, but sending a `SIGTERM` signal instead:

```
$ pkill -SIGTERM api
```

This time you should see the line `signal: terminated` at the end of the output, like so:

```
$ go run ./cmd/api
time=2026-09-12T13:32:10.474+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:32:10.474+02:00 level=INFO msg="starting server" addr=:4000 env=development
signal: terminated
```

You can also try sending a `SIGQUIT` signal — either by pressing `Ctrl+\` on your keyboard or running `pkill -SIGQUIT api`. This will cause the application to exit with a stack dump, similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/api
time=2026-09-12T13:33:51.605+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:33:51.605+02:00 level=INFO msg="starting server" addr=:4000 env=development
SIGQUIT: quit
PC=0x411ace m=0 sigcode=0

goroutine 0 gp=0xb97ea0 m=0 mp=0xb98e40 [idle]:
internal/runtime/syscall/linux.Syscall6()
        /usr/local/go/src/internal/runtime/syscall/linux/asm_linux_amd64.s:36 +0xe fp=0x7ffc3b6c14f8 sp=0x7ffc3b6c14f0 pc=0x41
internal/runtime/syscall/linux.EpollWait(0x73404ad0?, {0x7ffc3b6c1584?, 0x7aee73403450?, 0x7aee737e26e0?}, 0x737ef7e9?, 0x7aee
        /usr/local/go/src/internal/runtime/syscall/linux/syscall_linux.go:32 +0x45 fp=0x7ffc3b6c1548 sp=0x7ffc3b6c14f8 pc=0x41
runtime.netpoll(0x289a8350e000?)
        /usr/local/go/src/runtime/netpoll_epoll.go:119 +0xd3 fp=0x7ffc3b6c1bd8 sp=0x7ffc3b6c1548 pc=0x44add3
runtime.findRunnable()
        /usr/local/go/src/runtime/proc.go:3769 +0x97c fp=0x7ffc3b6c1da8 sp=0x7ffc3b6c1bd8 pc=0x4574bc
runtime.schedule()
        /usr/local/go/src/runtime/proc.go:4179 +0xb1 fp=0x7ffc3b6c1de8 sp=0x7ffc3b6c1da8 pc=0x458b11
runtime.goexit0(0x289a835aa780?)
        /usr/local/go/src/runtime/proc.go:4515 +0x18 fp=0x7ffc3b6c1e00 sp=0x7ffc3b6c1de8 pc=0x4598d8
runtime.mcall()
        /usr/local/go/src/runtime/asm_amd64.s:463 +0x53 fp=0x7ffc3b6c1e18 sp=0x7ffc3b6c1e00 pc=0x48cdd3
...
exit status 2
```

We can see that these signals are effective in terminating our application — but the problem we have is that they all cause our application to exit *immediately*.

Fortunately, Go provides tools in the `os/signals` package that we can use to intercept catchable signals and trigger a graceful shutdown of our application. We'll look at how to do that in the next chapter.

dego_vila@yahoo.com 30 Oct 2026

Intercepting shutdown signals

Before we get into the nuts and bolts of how to intercept signals, let's move the code related to our `http.Server` out of the `main()` function and into a separate file. This will give us a clean and clear starting point from which we can build up the graceful shutdown functionality.

If you're following along, create a new `cmd/api/server.go` file:

```
$ touch cmd/api/server.go
```

And then add a new `app.serve()` method which initializes and starts our `http.Server`, like so:

File: cmd/api/server.go

```
package main

import (
    "fmt"
    "log/slog"
    "net/http"
    "time"
)

func (app *application) serve() error {
    // Declare an HTTP server using the same settings as in our main() function.
    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", app.config.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(app.logger.Handler(), slog.LevelError),
    }

    // Likewise log a "starting server" message.
    app.logger.Info("starting server", "addr", srv.Addr, "env", app.config.env)

    // Start the server as normal, returning any error.
    return srv.ListenAndServe()
}
```

With that in place, we can simplify our `main()` function to use this new `app.serve()` method like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "flag"
    "log/slog"
    "os"
    "time"

    "greenlight.alexedwards.net/internal/models"

    _ "github.com/lib/pq"
)

...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")

    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    flag.Float64Var(&cfg.limiter.rps, "limiter-rps", 2, "Rate limiter maximum requests per second")
    flag.IntVar(&cfg.limiter.burst, "limiter-burst", 4, "Rate limiter maximum burst")
    flag.BoolVar(&cfg.limiter.enabled, "limiter-enabled", true, "Enable rate limiter")

    flag.Parse()

    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

    db, err := openDB(cfg)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
    defer db.Close()

    logger.Info("database connection pool established")

    app := &application{
        config: cfg,
        logger: logger,
        models: models.NewModels(db),
    }

    // Call app.serve() to start the server.
    err = app.serve()
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
}

...
```

dego_vila@yahoo.com 30 Oct 2026

Catching SIGINT and SIGTERM signals

Next, we want to update our application so that it 'catches' any `SIGINT` and `SIGTERM` signals. As we mentioned above, `SIGKILL` signals are not catchable (and will always cause the application to terminate immediately), and we'll leave `SIGQUIT` with its default behavior (as it's handy if you want to execute a non-graceful shutdown via a keyboard shortcut).

To catch the signals, we'll need to spin up a background goroutine which runs for the lifetime of our application. In this background goroutine, we can use the `signal.Notify()` function to listen for specific signals and relay them to a channel for further processing.

I'll demonstrate.

Open up the `cmd/api/server.go` file and update it like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/server.go

```
package main

import (
    "fmt"
    "log/slog"
    "net/http"
    "os"        // New import
    "os/signal" // New import
    "syscall"   // New import
    "time"
)

func (app *application) serve() error {
    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", app.config.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(app.logger.Handler(), slog.LevelError),
    }

    // Start a background goroutine.
    go func() {
        // Create a quit channel which carries os.Signal values.
        quit := make(chan os.Signal, 1)

        // Use signal.Notify() to listen for incoming SIGINT and SIGTERM signals and
        // relay them to the quit channel. Any other signals will not be caught by
        // signal.Notify() and will retain their default behavior.
        signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)

        // Read the signal from the quit channel. This code will block until a signal is
        // received.
        s := <-quit

        // Log a message to say that the signal has been caught. Notice that we also
        // call the String() method on the signal to get the signal name and include it
        // in the log entry attributes.
        app.logger.Info("caught signal", "signal", s.String())

        // Exit the application with a 0 (success) status code.
        os.Exit(0)
    }()

    // Start the server as normal.
    app.logger.Info("starting server", "addr", srv.Addr, "env", app.config.env)

    return srv.ListenAndServe()
}
```

At the moment this new code isn't doing much — after intercepting the signal, all we do is log a message and then exit our application. But the important thing is that it demonstrates the pattern of how to catch specific signals and handle them in your code.

One thing I'd like to quickly emphasize about this: our `quit` channel is a buffered channel with size 1.

dego_vila@yahoo.com 30 Oct 2026

We need to use a buffered channel here because `signal.Notify()` does not wait for a receiver to be available when sending a signal to the `quit` channel. If we had used a regular (non-buffered) channel here instead, a signal could be 'missed' if our `quit` channel is not ready to receive at the exact moment that the signal is sent. By using a buffered channel, we avoid this problem and ensure that we never miss a signal.

OK, let's try this out.

First, run the application and then press `Ctrl+C` on your keyboard to send a `SIGINT` signal. You should see a `"caught signal"` log entry with `"signal":"interrupt"` in the attributes, similar to this:

```
$ go run ./cmd/api
time=2026-09-12T13:37:09.747+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:37:09.747+02:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-09-12T13:37:21.613+02:00 level=INFO msg="caught signal" signal=interrupt
```

You can also restart the application and try sending a `SIGTERM` signal. This time, the log entry attributes should contain `signal=terminated`, like so:

```
$ pkill -SIGTERM api
```

```
$ go run ./cmd/api
time=2026-09-12T13:37:53.804+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:37:53.804+02:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-09-12T13:38:00.540+02:00 level=INFO msg="caught signal" signal=terminated
```

In contrast, sending a `SIGKILL` or `SIGQUIT` signal will continue to cause the application to exit immediately *without the signal being caught*, so you shouldn't see a `"caught signal"` message in the logs. For example, if you restart the application and issue a `SIGKILL`...

```
$ pkill -SIGKILL api
```

The application should be terminated immediately, and the logs will look like this:

```
$ go run ./cmd/api
time=2026-09-12T13:38:28.530+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:38:28.530+02:00 level=INFO msg="starting server" addr=:4000 env=development
signal: killed
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 11.3.

Executing the shutdown

Intercepting the signals is all well and good, but it's not very useful until we do something with them! In this chapter, we're going to update our application so that the `SIGINT` and `SIGTERM` signals we intercept trigger a graceful shutdown of our application.

Specifically, after receiving one of these signals we will call the `Shutdown()` method on our HTTP server. The official documentation describes this as follows:

> *Shutdown gracefully shuts down the server without interrupting any active connections. Shutdown works by first closing all open listeners, then closing all idle connections, and then waiting indefinitely for connections to return to idle and then shut down.*

The pattern to implement this in practice is difficult to describe with words, so let's jump into the code and talk through the details as we go along.

File: cmd/api/server.go

```
package main

import (
    "context" // New import
    "errors"  // New import
    "fmt"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func (app *application) serve() error {
    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", app.config.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(app.logger.Handler(), slog.LevelError),
    }

    // Create a shutdownError channel. We will use this to receive any errors returned
    // by the graceful Shutdown() function.
    shutdownError := make(chan error)

    go func() {
        // Intercept the signals, as before.
        quit := make(chan os.Signal, 1)
        signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
        s := <-quit
```

dego_vila@yahoo.com 30 Oct 2026

```
        // Update the log entry to say "stopping server" instead of "caught signal",
        // including the server address and signal for reference in the log line
        // attributes.
        app.logger.Info("stopping server", "addr", srv.Addr, "signal", s.String())

        // Call Shutdown() on our server, passing in an empty context with no values or
        // deadline. Shutdown() will return nil if the graceful shutdown was successful,
        // or an error (which may happen because of a problem closing the listeners). We
        // relay this return value to the shutdownError channel.
        shutdownError <- srv.Shutdown(context.Background())
    }()

    app.logger.Info("starting server", "addr", srv.Addr, "env", app.config.env)

    // Calling Shutdown() on our server will cause ListenAndServe() to immediately
    // return a http.ErrServerClosed error. So if we see this error, it is actually a
    // good thing and an indication that the graceful shutdown has started. So we check
    // specifically for this, only returning the error if it is NOT http.ErrServerClosed.
    err := srv.ListenAndServe()
    if !errors.Is(err, http.ErrServerClosed) {
        return err
    }

    // Otherwise, we wait to receive the return value from Shutdown() on the
    // shutdownError channel. If the return value is an error, we know that there was a
    // problem with the graceful shutdown and we return the error.
    err = <-shutdownError
    if err != nil {
        return err
    }

    // At this point we know that the graceful shutdown completed successfully and we
    // log a message to indicate that.
    app.logger.Info("shutdown complete")
    return nil
}
```

At first glance this code might seem a bit complex, but at a high level what it's doing can be summarized very simply: *when we receive a `SIGINT` or `SIGTERM` signal, we instruct our server to stop accepting any new HTTP requests, and wait for any in-flight requests to complete before the application is terminated*.

It's important to be aware that the `Shutdown()` method does not wait for any background tasks to complete, nor does it close hijacked long-lived connections like WebSockets. Instead, you will need to implement your own logic to coordinate a graceful shutdown of these things. We'll look at some techniques for doing this later in the book.

But that aside, this should now be working nicely in our application.

To help demonstrate the graceful shutdown functionality, you can add a 4 second sleep delay to the `healthcheckHandler` method, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/healthcheck.go

```
package main

import (
    "net/http"
    "time" // New import
)

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    env := envelope{
        "status": "available",
        "system_info": map[string]string{
            "environment": app.config.env,
            "version":     version,
        },
    }

    // Add a 4 second delay.
    time.Sleep(4 * time.Second)

    err := app.writeJSON(w, http.StatusOK, env, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Then start the API, and in another terminal window issue a request to the healthcheck endpoint followed by a `SIGTERM` signal.

```
$ curl localhost:4000/v1/healthcheck & pkill -SIGTERM api
```

In the logs for the server, you should immediately see a `"stopping server"` message following the `SIGTERM` signal, similar to this:

```
$ go run ./cmd/api
time=2026-09-12T13:40:36.615+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:40:36.615+02:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-09-12T13:40:46.322+02:00 level=INFO msg="stopping server" addr=:4000 signal=terminated
```

Then after a 4-second delay for the in-flight request to complete, our `healthcheckHandler` should return the JSON response as normal and you should see that our API has logged a final `"shutdown complete"` message before exiting cleanly:

```
$ go run ./cmd/api
time=2026-09-12T13:40:36.615+02:00 level=INFO msg="database connection pool established"
time=2026-09-12T13:40:36.615+02:00 level=INFO msg="starting server" addr=:4000 env=development
time=2026-09-12T13:40:46.322+02:00 level=INFO msg="stopping server" addr=:4000 signal=terminated
time=2026-09-12T13:40:50.494+02:00 level=INFO msg="shutdown complete"
```

dego_vila@yahoo.com 30 Oct 2026

Notice the 4 second delay between the `"stopping server"` and `"shutdown complete"` messages in the timestamps above?

So this is working really well now. Any time that we want to gracefully shut down our application, we can do so by sending a `SIGINT` (`Ctrl+C`) or `SIGTERM` signal. Our handlers will have the opportunity to complete their work, and our clients will receive a proper HTTP response. And if we ever want to exit immediately, without a graceful shutdown, we can still do so by sending a `SIGQUIT` (`Ctrl+\`) or `SIGKILL` signal instead.

Lastly, if you're following along, please revert `healthcheckHandler` to remove the 4 second sleep. Like so:

File: cmd/api/healthcheck.go

```
package main

import (
    "net/http"
)

func (app *application) healthcheckHandler(w http.ResponseWriter, r *http.Request) {
    env := envelope{
        "status": "available",
        "system_info": map[string]string{
            "environment": app.config.env,
            "version":     version,
        },
    }

    err := app.writeJSON(w, http.StatusOK, env, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Additional information

Shutdown timeouts

When we introduced the `srv.Shutdown()` functionality above, we said that it first closes all listeners, then waits indefinitely for all connections to return to idle before returning.

But what happens if one of the connections is 'stuck'? For example, perhaps your handler code is waiting on a resource that is currently unavailable, or maybe there's a bug that means it's in some form of deadlock. In this case, `srv.Shutdown()` will be waiting indefinitely, and your application will be left in a state where it's still running but not

dego_vila@yahoo.com 30 Oct 2026

accepting new HTTP requests.

To deal with this, it's possible to pass in a `context.Context` with a deadline when calling `srv.Shutdown()`. When you do this, it effectively sets a maximum time limit on how long the shutdown should take, giving any in-flight requests a fixed 'grace period' to complete before `srv.Shutdown()` returns and the application is terminated.

For example, to set a time limit of 30 seconds, you would do this:

```
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

// If the context deadline is exceeded, srv.Shutdown() will return a
// context.DeadlineExceeded error.
shutdownError <- srv.Shutdown(ctx)
```

But if you consider the intent of the `SIGTERM` and `SIGKILL` signals respectively, then in most cases I think it's better to not set a time limit. The SUSE Linux documentation describes the intent of these two signals nicely:

> *SIGTERM asks the process to terminate gracefully. SIGTERM is the preferred way to end a process because it allows the program to shut down gracefully, saving data and releasing resources properly.*
>
> *SIGKILL is used as a last resort when a process is unresponsive to SIGTERM or when you need to stop a process immediately without any delay.*

If we set an arbitrary time limit when shutting down in response to a `SIGTERM` signal, it's possible that the time limit will be hit and some in-progress work will be abandoned in an incomplete state (a dirty shutdown) — in much the same way that it would if we sent a `SIGKILL` signal to the application. That goes against the expected behavior of `SIGTERM` and the whole point of having a graceful shutdown.

And if the application is unresponsive and 'stuck' in a state where it's unable to cleanly shut down due to a deadlock or other issue, then that's why we have the `SIGKILL` signal available to us to terminate the application.

If we hard-code a shutdown time limit in our application code, it both deviates from the expected behavior of the `SIGTERM` signal, and it takes away the flexibility and choice available to us — or our orchestration code — to decide if and when to send a `SIGKILL` if the application is unresponsive.

But setting a shutdown timeout can be useful in scenarios where you need a predictable and time-bounded shutdown in response to a `SIGTERM` signal, and are happy to trade the

dego_vila@yahoo.com 30 Oct 2026

potential risk of a dirty shutdown for that.

dego_vila@yahoo.com 30 Oct 2026

Chapter 12.

User model setup and registration

In the upcoming sections of this book, we're going to shift our focus towards *users*: registering them, activating them, authenticating them, and restricting access to our API endpoints depending on the permissions that they have.

But before we can do these things, we need to lay some groundwork. Specifically, we need to:

- Create a new `users` table in PostgreSQL for storing our user data.
- Create a `UserModel` which contains the code for interacting with our `users` table, validating user data, and hashing user passwords.
- Develop a `POST /v1/users` endpoint which can be used to register new users in our application.

dego_vila@yahoo.com 30 Oct 2026

Setting up the users database table

Let's begin by creating a new `users` table in our database. If you're following along, use the `migrate` tool to generate a new pair of SQL migration files:

```
$ migrate create -seq -ext=.sql -dir=./migrations create_users_table
/home/alex/Projects/greenlight/migrations/000004_create_users_table.up.sql
/home/alex/Projects/greenlight/migrations/000004_create_users_table.down.sql
```

And then add the following SQL statements to the 'up' and 'down' files respectively:

File: migrations/000004_create_users_table.up.sql

```
CREATE TABLE IF NOT EXISTS users (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    created_at timestamp(0) with time zone NOT NULL DEFAULT NOW(),
    name text NOT NULL,
    email citext UNIQUE NOT NULL,
    password_hash bytea NOT NULL,
    activated bool NOT NULL,
    version integer NOT NULL DEFAULT 1
);
```

File: migrations/000004_create_users_table.down.sql

```
DROP TABLE IF EXISTS users;
```

There are a few interesting things about this `CREATE TABLE` statement that I'd like to quickly explain:

1. The `email` column has the type `citext` (case-insensitive text). This type stores text data exactly as it is entered — without changing the case in any way — but *comparisons* against the data are always case-insensitive… including lookups on associated indexes.

2. We've also got a `UNIQUE` constraint on the `email` column. Combined with the `citext` type, this means that no two rows in the database can have the same email value — even if they have different cases. This essentially enforces a database-level business rule that *no two users should exist with the same email address*.

3. The `password_hash` column has the type `bytea` (binary string). In this column we'll store a *one-way hash of the user's password* generated using bcrypt — not the password itself in plaintext form.

dego_vila@yahoo.com 30 Oct 2026

4. The `activated` column stores a boolean value to denote whether a user account is 'active' or not. We will set this to `false` by default when creating a new user, and require the user to confirm their email address before we set it to `true`.

5. We've also included a `version` number column, which we will increment each time a user record is updated. This will allow us to use optimistic locking to prevent race conditions when updating user records, in the same way we did with movies earlier in the book.

OK, let's execute the 'up' migration:

```
$ migrate -path=./migrations -database=$GREENLIGHT_DB_DSN up
4/u create_users_table (62.43511ms)
```

And then you should be able to connect to your database and verify that the new `users` table has been created, as expected:

```
$ psql $GREENLIGHT_DB_DSN
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> \d users
                                      Table "public.users"
    Column     |            Type             | Collation | Nullable |              Default
---------------+-----------------------------+-----------+----------+-----------------------------------
 id            | bigint                      |           | not null | generated always as identity
 created_at    | timestamp(0) with time zone |           | not null | now()
 name          | text                        |           | not null |
 email         | citext                      |           | not null |
 password_hash | bytea                       |           | not null |
 activated     | boolean                     |           | not null |
 version       | integer                     |           | not null | 1
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "users_email_key" UNIQUE CONSTRAINT, btree (email)
```

One important thing to point out here: the `UNIQUE` constraint on our `email` column has automatically been assigned the name `users_email_key`. This will become relevant in the next chapter, when we need to handle any errors caused by a user registering twice with the same email address.

dego_vila@yahoo.com 30 Oct 2026

Setting up the users model

Now that our database table is set up, we're going to update our `internal/models` package to contain a new `User` struct (to represent the data for an individual user), and create a `UserModel` type (which we will use to perform various SQL queries against our `users` table).

If you're following along, go ahead and create an `internal/models/users.go` file to hold this new code:

```
$ touch internal/models/users.go
```

Let's start by defining the `User` struct, along with some helper methods for setting and verifying the password for a user.

As we mentioned earlier, in this project we will use bcrypt to hash a user's password before storing it in the database. So the first thing we need to do is install the `golang.org/x/crypto/bcrypt` package, which provides an easy-to-use Go implementation of the bcrypt algorithm.

```
$ go get golang.org/x/crypto/bcrypt@latest
go: downloading golang.org/x/crypto v0.57.0
go: added golang.org/x/crypto v0.57.0
```

Then in the `internal/models/users.go` file, go ahead and create the `User` struct and helper methods like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/users.go

```go
package models

import (
    "errors"
    "time"

    "golang.org/x/crypto/bcrypt"
)

// Define a User struct to represent an individual user. Importantly, notice how we are
// using the json:"-" struct tag to prevent the Password and Version fields from appearing
// in any output when we encode it to JSON. Also notice that the Password field uses the
// custom password type defined below.
type User struct {
    ID        int       `json:"id"`
    CreatedAt time.Time `json:"created_at"`
    Name      string    `json:"name"`
    Email     string    `json:"email"`
    Password  password  `json:"-"`
    Activated bool      `json:"activated"`
    Version   int       `json:"-"`
}

// Create a custom password type which is a struct containing the plaintext and hashed
// versions of the password for a user. The plaintext field is a *pointer* to a string,
// so that we're able to distinguish between a plaintext password not being present in
// the struct at all, versus a plaintext password which is the empty string "".
type password struct {
    plaintext *string
    hash      []byte
}

// The Set() method calculates the bcrypt hash of a plaintext password, and stores both
// the hash and the plaintext versions in the struct.
func (p *password) Set(plaintextPassword string) error {
    hash, err := bcrypt.GenerateFromPassword([]byte(plaintextPassword), 12)
    if err != nil {
        return err
    }

    p.plaintext = &plaintextPassword
    p.hash = hash

    return nil
}

// The Matches() method checks whether the provided plaintext password matches the
// hashed password stored in the struct, returning true if it matches and false
// otherwise.
func (p *password) Matches(plaintextPassword string) (bool, error) {
    err := bcrypt.CompareHashAndPassword(p.hash, []byte(plaintextPassword))
    if err != nil {
        switch {
        case errors.Is(err, bcrypt.ErrMismatchedHashAndPassword):
            return false, nil
        default:
            return false, err
        }
    }

    return true, nil
}
```

dego_vila@yahoo.com 30 Oct 2026

We explained how the `golang.org/x/crypto/bcrypt` package works previously in *Let's Go*, but let's quickly recap the key points:

- The `bcrypt.GenerateFromPassword()` function generates a bcrypt hash of a password using a specific cost parameter (in the code above, we use a cost of `12`). The higher the cost, the slower and more computationally expensive it is to generate the hash. There is a balance to be struck here — we want the cost to be prohibitively expensive for attackers, but also not *so slow* that it harms the user experience of our API. This function returns a *hash string* in the format:

```
$2a$[cost]$[22-character salt][31-character hash]
```

- The `bcrypt.CompareHashAndPassword()` function works by re-hashing the provided password using the *same salt and cost parameter* that is in the hash string that we're comparing against. The re-hashed value is then checked against the original hash string using the `subtle.ConstantTimeCompare()` function, which performs a comparison in constant time (to mitigate the risk of a timing attack). If they don't match, then it will return a `bcrypt.ErrMismatchedHashAndPassword` error.

Adding validation checks

Let's move on and create some validation checks for our `User` struct. Specifically, we want to:

- Check that the `Name` field is not the empty string, and the value is less than 500 bytes long.
- Check that the `Email` field is not the empty string, and that it matches the regular expression for email addresses that we added in our `validator` package earlier in the book.
- If the `Password.plaintext` field is not `nil`, then check that the value is not the empty string and is between 8 and 72 bytes long.
- Check that the `Password.hash` field is never `nil`.

dego_vila@yahoo.com 30 Oct 2026

> **Note:** When creating a bcrypt hash the input is truncated to a maximum of 72 bytes. So, if someone uses a very long password, it means that any bytes after that would effectively be ignored when creating the hash. To avoid any confusion for users, we'll simply enforce a hard maximum length of 72 bytes on the password in our validation checks. If you don't want to enforce a maximum length, you could pre-hash the password instead.

Additionally, we're going to want to use the email and plaintext password validation checks again independently later in the book, so we'll define those checks in some standalone functions.

Go ahead and update the `internal/models/users.go` file like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/users.go

```
package models

import (
    "errors"
    "time"

    "greenlight.alexedwards.net/internal/validator" // New import

    "golang.org/x/crypto/bcrypt"
)

...

func ValidateEmail(v *validator.Validator, email string) {
    v.Check(email != "", "email", "must be provided")
    v.Check(validator.Matches(email, validator.EmailRX), "email", "must be a valid email address")
}

func ValidatePasswordPlaintext(v *validator.Validator, password string) {
    v.Check(password != "", "password", "must be provided")
    v.Check(len(password) >= 8, "password", "must be at least 8 bytes long")
    v.Check(len(password) <= 72, "password", "must not be more than 72 bytes long")
}

func ValidateUser(v *validator.Validator, user User) {
    v.Check(user.Name != "", "name", "must be provided")
    v.Check(len(user.Name) <= 500, "name", "must not be more than 500 bytes long")

    // Call the standalone ValidateEmail() helper.
    ValidateEmail(v, user.Email)

    // If the plaintext password is not nil, call the standalone
    // ValidatePasswordPlaintext() helper.
    if user.Password.plaintext != nil {
        ValidatePasswordPlaintext(v, *user.Password.plaintext)
    }

    // If the password hash is ever nil, this will be due to a logic error in our
    // codebase (probably because we forgot to set a password for the user). It's a
    // useful sanity check to include here, but it's not a problem with the data
    // provided by the client. So rather than adding an error to the validation map we
    // panic instead.
    if user.Password.hash == nil {
        panic("missing password hash for user")
    }
}
```

Creating the UserModel

The next step in this process is setting up a `UserModel` type which isolates the database interactions with our PostgreSQL `users` table.

We'll follow the same pattern here that we used for our `MovieModel`, and implement the following three methods:

- `Insert()` to create a new user record in the database.

dego_vila@yahoo.com 30 Oct 2026

- `GetByEmail()` to retrieve the data for a user with a specific email address.
- `Update()` to change the data for a specific user.

Open up the `internal/models/users.go` file again, and add the following code:

File: internal/models/users.go

```
package models

import (
    "context"      // New import
    "database/sql" // New import
    "errors"
    "time"

    "greenlight.alexedwards.net/internal/validator"

    "github.com/lib/pq" // New import
    "golang.org/x/crypto/bcrypt"
)

// Define a custom ErrDuplicateEmail error.
var (
    ErrDuplicateEmail = errors.New("duplicate email")
)

...

// Create a UserModel struct which wraps the connection pool.
type UserModel struct {
    DB *sql.DB
}

// Insert a new record in the database for the user. Note that the id, created_at and
// version fields are all automatically generated by our database, so we use the
// RETURNING clause to read them into the User struct after the insert, and return the
// updated struct, in the same way that we did when inserting a movie.
func (m UserModel) Insert(user User) (User, error) {
    query := `
        INSERT INTO users (name, email, password_hash, activated)
        VALUES ($1, $2, $3, $4)
        RETURNING id, created_at, version`

    args := []any{user.Name, user.Email, user.Password.hash, user.Activated}

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // If the table already contains a record with this email address, then when we try
    // to perform the insert there will be a violation of the UNIQUE "users_email_key"
    // constraint that we set up in the previous chapter. We check for this error
    // specifically, and return a custom ErrDuplicateEmail error instead.
    err := m.DB.QueryRowContext(ctx, query, args...).Scan(&user.ID, &user.CreatedAt, &user.Version)
    if err != nil {
        var pqErr *pq.Error
        switch {
        case errors.As(err, &pqErr) && pqErr.Code == "23505" && pqErr.Constraint == "users_email_key":
            return User{}, ErrDuplicateEmail
        default:
            return User{}, err
        }
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
    return user, nil
}

// Retrieve the User details from the database based on the user's email address.
// Because we have a UNIQUE constraint on the email column, this SQL query will only
// return one record (or none at all, in which case we return an ErrRecordNotFound error).
func (m UserModel) GetByEmail(email string) (User, error) {
    query := `
        SELECT id, created_at, name, email, password_hash, activated, version
        FROM users
        WHERE email = $1`

    var user User

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    err := m.DB.QueryRowContext(ctx, query, email).Scan(
        &user.ID,
        &user.CreatedAt,
        &user.Name,
        &user.Email,
        &user.Password.hash,
        &user.Activated,
        &user.Version,
    )

    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return User{}, ErrRecordNotFound
        default:
            return User{}, err
        }
    }

    return user, nil
}

// Update the details for a specific user. Notice that we check against the version
// field to help prevent any race conditions during the request cycle, just like we did
// when updating a movie. And we also check for a violation of the "users_email_key"
// constraint when performing the update, just like we did when inserting the user
// record originally.
func (m UserModel) Update(user User) (User, error) {
    query := `
        UPDATE users
        SET name = $1, email = $2, password_hash = $3, activated = $4, version = version + 1
        WHERE id = $5 AND version = $6
        RETURNING version`

    args := []any{
        user.Name,
        user.Email,
        user.Password.hash,
        user.Activated,
        user.ID,
        user.Version,
    }

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    err := m.DB.QueryRowContext(ctx, query, args...).Scan(&user.Version)
    if err != nil {
        var pqErr *pq.Error
```

dego_vila@yahoo.com 30 Oct 2026

```
        switch {
        case errors.As(err, &pqErr) && pqErr.Code == "23505" && pqErr.Constraint == "users_email_key":
            return User{}, ErrDuplicateEmail
        case errors.Is(err, sql.ErrNoRows):
            return User{}, ErrEditConflict
        default:
            return User{}, err
        }
    }

    return user, nil
}
```

Hopefully that feels nice and straightforward — we're using the same code patterns that we did for the CRUD operations on our `movies` table earlier in the book.

The only difference is that in some of the methods we're specifically checking for errors caused by a violation of our unique `users_email_key` constraint. To do this check, we use the `errors.As()` function to check if the error has the type `*pq.Error`. If it does, then we check that the PostgreSQL error code matches `23505` (which is the code for a unique constraint violation) and that the affected constraint is `users_email_key`. If all those things are true, then we return a `ErrDuplicateEmail` error.

As we'll see in the next chapter, by treating this as a special case and returning `ErrDuplicateEmail`, we'll be able to respond to clients with a message to say that *"this email address is already in use"*, rather than sending them a `500 Internal Server Error` response like we normally would.

To finish all this off, the final thing we need to do is update our `internal/models/models.go` file to include the new `UserModel` in our parent `Models` struct. Like so:

File: internal/models/models.go

```
package models

...

type Models struct {
    Movies MovieModel
    Users  UserModel // Add a new Users field.
}

func NewModels(db *sql.DB) Models {
    return Models{
        Movies: MovieModel{DB: db},
        Users:  UserModel{DB: db}, // Initialize a new UserModel instance.
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

Registering a user

Now that we've laid the groundwork, let's start putting it to use by creating a new API endpoint to manage the process of registering (or *signing up*) a new user.

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
GET	/v1/movies	listMoviesHandler	Show the details of all movies
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PATCH	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie
DELETE	/v1/movies/:id	deleteMovieHandler	Delete a specific movie
POST	**/v1/users**	**registerUserHandler**	**Register a new user**

When a client calls this new `POST /v1/users` endpoint, we will expect them to provide the following details for the new user in a JSON request body. Similar to this:

```
{
    "name": "Alice Smith",
    "email": "alice@example.com",
    "password": "pa55word"
}
```

When we receive this, the `registerUserHandler` should create a new `User` struct containing these details, validate it with the `ValidateUser()` helper, and then pass it to our `UserModel.Insert()` method to create a new database record.

In fact, we've already written most of the code we need for the `registerUserHandler` — it's now just a matter of piecing it all together in the correct order.

If you're following along, go ahead and create a new `cmd/api/users.go` file:

```
$ touch cmd/api/users.go
```

dego_vila@yahoo.com 30 Oct 2026

And then add the new `registerUserHandler` method containing the following code:

File: cmd/api/users.go

```
package main

import (
    "errors"
    "net/http"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"
)

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {
    // Create an anonymous struct to hold the expected data from the request body.
    var input struct {
        Name     string `json:"name"`
        Email    string `json:"email"`
        Password string `json:"password"`
    }

    // Parse the request body into the anonymous struct.
    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Copy the data from the request body into a new User struct. Notice also that we
    // set the Activated field to false, which isn't strictly necessary because the
    // Activated field will have the zero value of false by default. But setting this
    // explicitly helps to make our intentions clear to anyone reading the code.
    user := models.User{
        Name:      input.Name,
        Email:     input.Email,
        Activated: false,
    }

    // Use the Password.Set() method to generate and store the hashed and plaintext
    // passwords.
    err = user.Password.Set(input.Password)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    v := validator.New()

    // Validate the user struct and return the error messages to the client if any of
    // the checks fail.
    if models.ValidateUser(v, user); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Insert the user data into the database.
    user, err = app.models.Users.Insert(user)
```

dego_vila@yahoo.com 30 Oct 2026

```
    if err != nil {
        switch {
        // If we get a ErrDuplicateEmail error, use the v.AddError() method to manually
        // add a message to the validator instance, and then call our
        // failedValidationResponse() helper.
        case errors.Is(err, models.ErrDuplicateEmail):
            v.AddError("email", "a user with this email address already exists")
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Write a JSON response containing the user data along with a 201 Created status
    // code.
    err = app.writeJSON(w, http.StatusCreated, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Before we try this out, we also need to add the new `POST /v1/users` endpoint to our `cmd/api/routes.go` file. Like so:

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.listMoviesHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    // Add the route for the POST /v1/users endpoint.
    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)

    return app.recoverPanic(app.rateLimit(router))
}
```

Once that's done, make sure all the files are saved and fire up the API.

Then go ahead and make a request to the `POST /v1/users` endpoint to register a new user with the email address `alice@example.com`. You should get a `201 Created` response displaying the details for the user, similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"name": "Alice Smith", "email": "alice@example.com", "password": "pa55word"}'
$ curl -i -d "$BODY" localhost:4000/v1/users
HTTP/1.1 201 Created
Content-Type: application/json
Date: Sat, 12 Sep 2026 11:55:46 GMT
Content-Length: 152

{
    "user": {
        "id": 1,
        "created_at": "2026-09-12T13:55:47+02:00",
        "name": "Alice Smith",
        "email": "alice@example.com",
        "activated": false
    }
}
```

> **Hint:** If you're following along, remember the password you used in the request above — you'll need it later!

That's looking good. We can see from the status code that the user record has been successfully created, and in the JSON response we can see the system-generated information for the new user — including the user's ID and activation status.

If you take a look at your PostgreSQL database, you should also see the new record in the `users` table. Similar to this:

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT * FROM users;
 id |       created_at       |    name     |       email       |           password_hash             | activated | version
----+------------------------+-------------+-------------------+------------------------------------+-----------+---------
  1 | 2026-09-12 13:55:47+02 | Alice Smith | alice@example.com | \x243261243132243766676748414e5a... | f         |       1
(1 row)
```

> **Note:** The `psql` tool always displays `bytea` values as a hex-encoded string. So the `password_hash` field in the output above displays a *hex encoding* of the bcrypt hash. If you want, you can run the following query to append the regular string version to the table too: `SELECT *, encode(password_hash, 'escape') FROM users`.

OK, let's try making another request to our API but with some invalid user details. This time our validation checks will kick in and the client should receive the relevant error messages. For example:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"name": "", "email": "bob@invalid.", "password": "pass"}'
$ curl -d "$BODY" localhost:4000/v1/users
{
    "error": {
        "email": "must be a valid email address",
        "name": "must be provided",
        "password": "must be at least 8 bytes long"
    }
}
```

Lastly, try registering a second account for `alice@example.com`. This time you should get a validation error containing an *"a user with this email address already exists"* message, like so:

```
$ BODY='{"name": "Alice Jones", "email": "alice@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/users
{
    "error": {
        "email": "a user with this email address already exists"
    }
}
```

If you want, you can also try sending some requests using alternative casings of `alice@example.com` — such as `ALICE@example.com` or `Alice@Example.com`. Because the `email` column in our database has the type `citext`, these alternative versions will be successfully identified as duplicates too.

Additional information

Email case-sensitivity

Let's talk quickly about email addresses case-sensitivity in a bit more detail.

- Thanks to the specifications in RFC 2821, the domain part of an email address (`username@domain`) is case-insensitive. This means we can be confident that the real-life user behind `alice@example.com` is the same person as `alice@EXAMPLE.COM`.
- The username part of an email address *may or may not* be case-sensitive — it depends on the email provider. Almost every major email provider treats the username as case-insensitive, but it is not absolutely guaranteed. All we can say here is that the real-life user behind the address `alice@example.com` is *very probably* (but not definitely) the same as `ALICE@example.com`.

dego_vila@yahoo.com 30 Oct 2026

So, what does this mean for our application?

From a security point of view, we should always store the email address using the exact casing provided by the user during registration, and **we should send them emails using that exact casing only**. If we don't, there is a risk that emails could be delivered to the wrong real-life user. It's particularly important to be aware of this in any workflows that use email for authentication purposes, such as a password-reset workflow.

However, because `alice@example.com` and `ALICE@example.com` are *very probably* the same user, we should generally treat email addresses as case-insensitive for comparison purposes.

In our registration workflow, using a case-insensitive comparison prevents users from accidentally (or intentionally) registering multiple accounts by just using different casing. And from a user-experience point-of-view, in workflows like login, activation or password resets, it's more forgiving for users if we don't require them to submit their request with exactly the same email casing that they used when registering.

User enumeration

It's important to be aware that our registration endpoint is vulnerable to *user enumeration*. For example, if an attacker wants to know whether `alice@example.com` has an account with us, all they need to do is send a request like this:

```
$ BODY='{"name": "Alice Jones", "email": "alice@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/users
{
    "error": {
        "email": "a user with this email address already exists"
    }
}
```

And they have the answer right there. We're explicitly telling the attacker that `alice@example.com` is already a user.

So, what are the risks of leaking this information?

The first and most obvious risk relates to user privacy. For services that are sensitive or confidential you probably don't want to make it obvious who has an account. The second risk is that it makes it easier for an attacker to compromise a user's account. Once they know a user's email address, they can potentially:

- Target the user via social engineering or another type of tailored attack to get their password.

dego_vila@yahoo.com 30 Oct 2026

- Search for the email address in leaked password tables, and try those same passwords on our service.

Preventing enumeration attacks typically requires two things:

1. Making sure that the response sent to the client is always exactly the same, irrespective of whether a user exists or not. Generally, this means changing your response wording to be ambiguous, and notifying the user of any problems in a side-channel (such as sending them an email to inform them that they already have an account).
2. Making sure that the *time taken* to send the response is always the same, irrespective of whether a user exists or not. In Go, this generally means offloading work to a background goroutine.

These mitigations, unfortunately, tend to increase the complexity of your application and add friction and obscurity to your workflows. For all your regular users who are not attackers, they're a negative from a UX point of view. You have to ask: *is it worth the trade-off?*

There are a few things to think about when answering this question. How important is user privacy in your application? How attractive (high-value) is a compromised account to an attacker? How important is it to reduce friction in your user workflows? The answers to those questions will vary from project-to-project, and will help form the basis for your decision.

It's worth noting that many big-name services, including GitHub, Amazon and Twitter, don't prevent user enumeration (at least not on their registration pages). I'm not suggesting that this makes it OK — just that those companies have decided that the additional friction for the user is worse than the privacy and security risks *in their specific case*.

dego_vila@yahoo.com 30 Oct 2026

Chapter 13.

Sending emails

In this section of the book we're going to inject some interactivity into our API, and adapt our `registerUserHandler` so that it sends the user a welcome email after they successfully register.

In the process of doing this we'll cover a few interesting topics. You'll learn:

- How to use the Mailtrap SMTP service to send and monitor test emails during development.
- How to use the `html/template` package and Go's *embedded files* functionality to create dynamic and easy-to-manage templates for your email content.
- How to create a reusable `internal/mailer` package for sending emails from your application.
- How to implement a pattern for sending emails in background goroutines, and how to wait for these to complete during a graceful shutdown.

dego_vila@yahoo.com 30 Oct 2026

SMTP server setup

In order to develop our email sending functionality, we'll need access to an SMTP (Simple Mail Transfer Protocol) server that we can safely use for testing purposes.

There are a huge number of SMTP service providers (such as Postmark, Sendgrid or Amazon SES) that we *could* use to send our emails — or you can even install and run your own SMTP server. But in this book we're going to use Mailtrap.

The reason for using Mailtrap is because it's a specialist service for *sending emails during development and testing*. Essentially, it delivers all emails to an inbox that you can access, instead of sending them to the actual recipient.

I've got no affiliation with the company — I just find that the service works well and is simple to use. They also offer a 'free forever' plan, which should be sufficient for anyone who is coding along with this book.

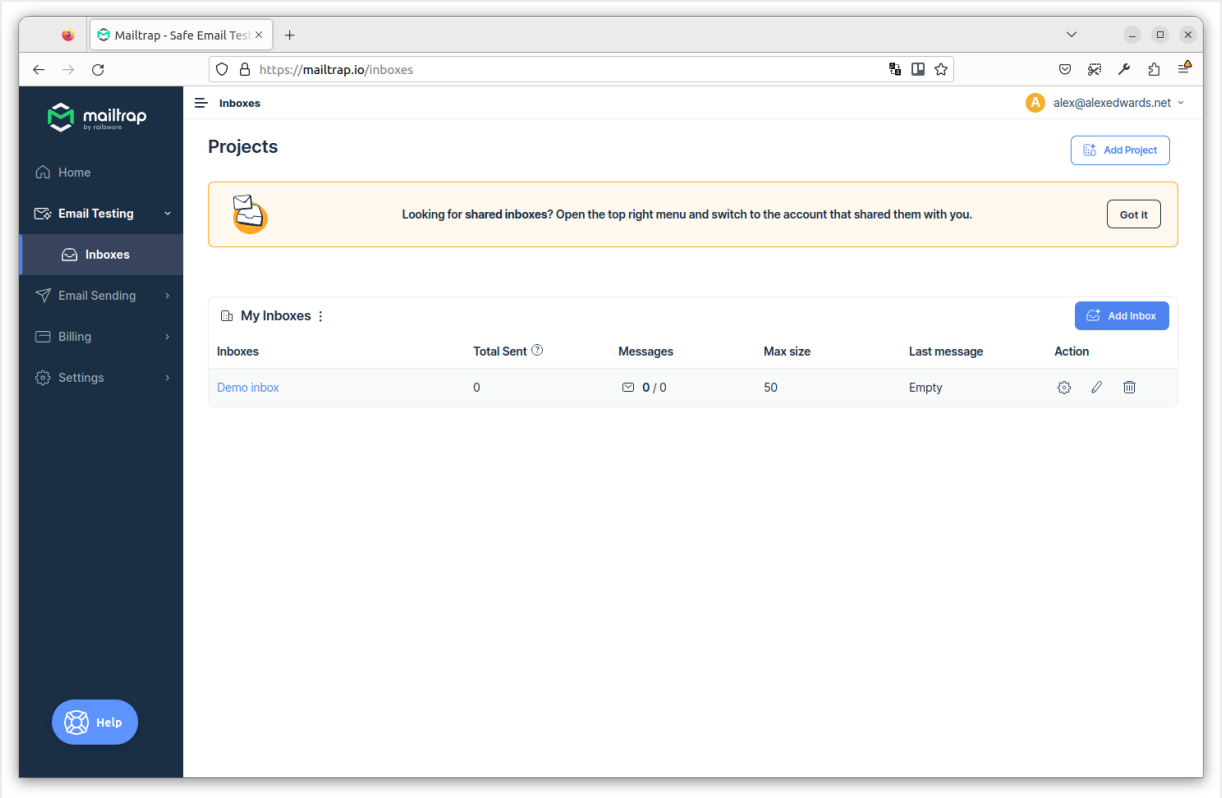
> **Note:** If you already have your own SMTP server, or there is an alternative SMTP provider that you want to use instead, that's absolutely fine. Feel free to skip ahead to the next chapter.

Setting up Mailtrap

To set up a Mailtrap account, head to the signup page where you can register using either your email address or your Google or GitHub accounts (if you have them).

Once you're registered and logged in, use the menu to navigate to **Testing › Inboxes**. You should see a page listing your available *inboxes*, similar to the screenshot below.

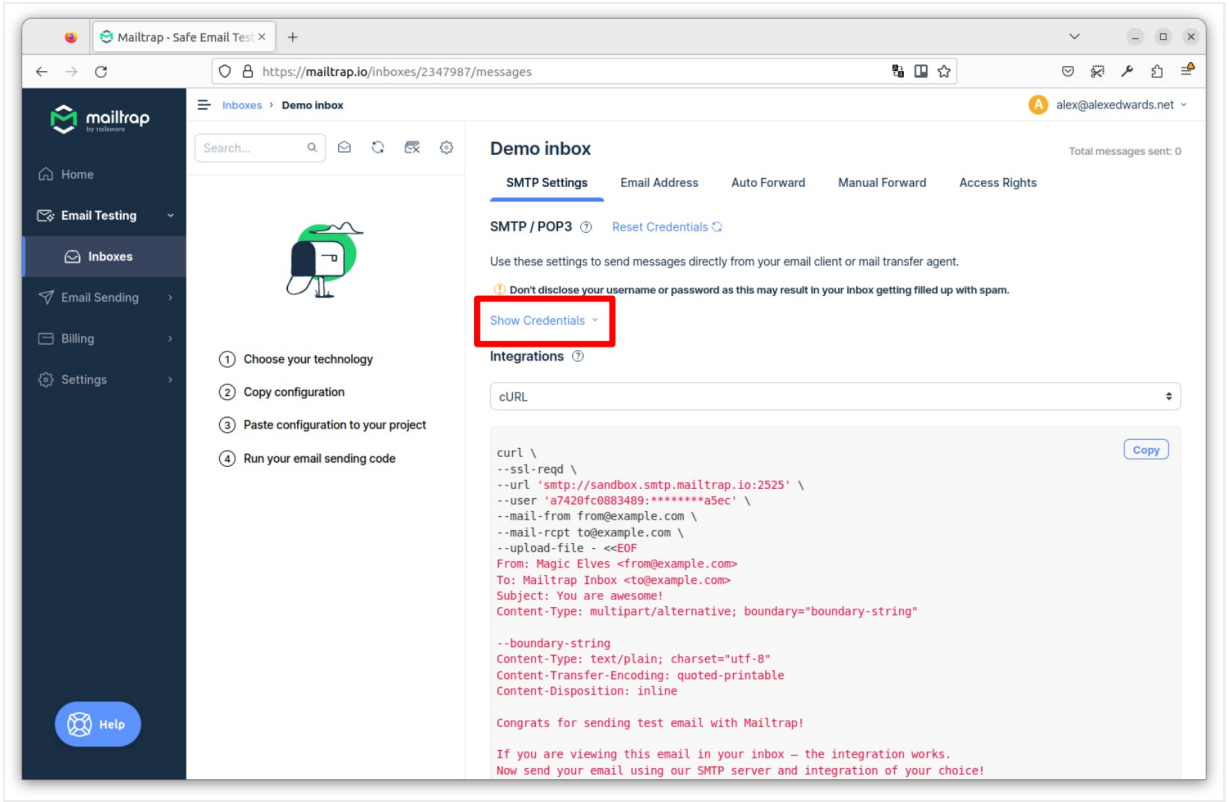
dego_vila@yahoo.com 30 Oct 2026



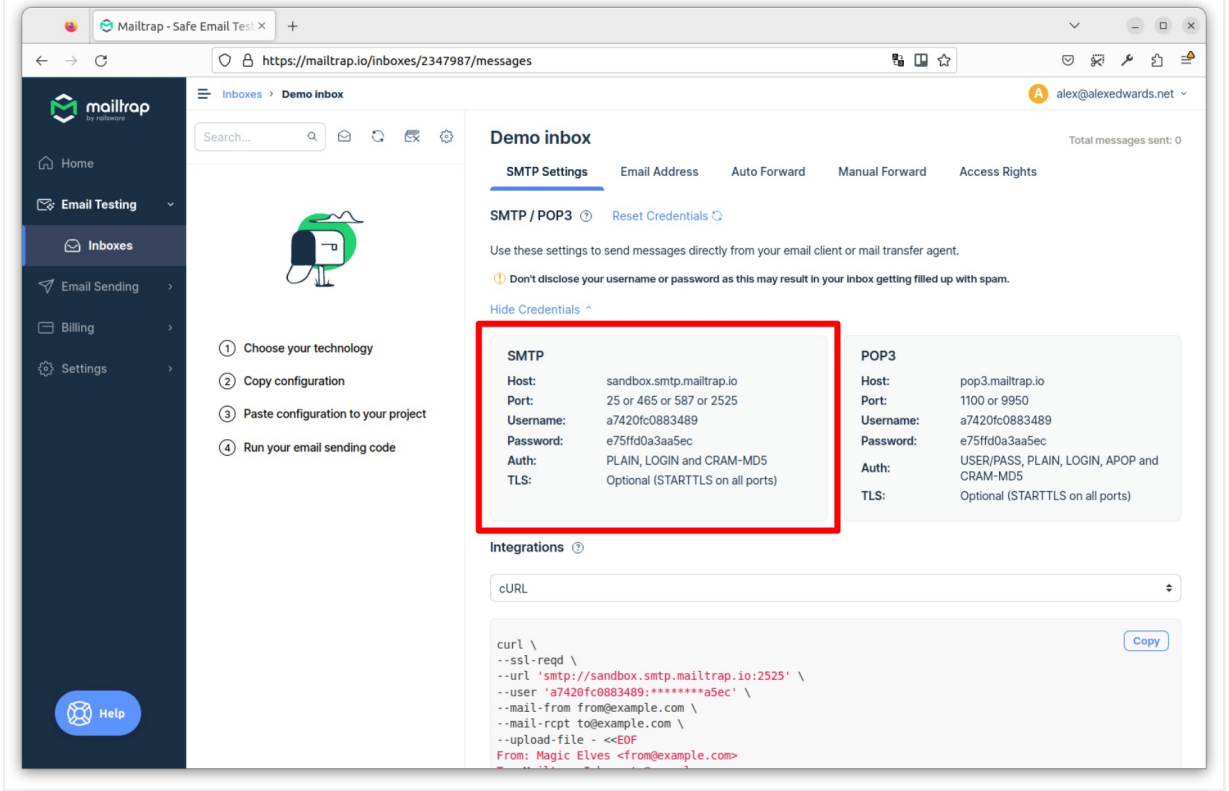
Every Mailtrap account comes with one free inbox, which by default is called **Demo inbox**. You can change the name if you want by clicking the pencil icon under **Actions**.

If you go ahead and click through to that inbox, you should see that it's currently empty and contains no emails, similar to this:

dego_vila@yahoo.com 30 Oct 2026



Each inbox has its own set of SMTP credentials, which you can display by clicking the **Show Credentials** link (highlighted by the red box in the screenshot above). This will expand to display the inbox SMTP credentials, similar to the screenshot below.



dego_vila@yahoo.com 30 Oct 2026

Basically, any emails that you send using these SMTP credentials will end up in this inbox, instead of being sent to the actual recipient.

If you're following along, make a note of the credentials displayed on your screen (or just keep the browser tab open) — you'll need them in the next chapter.

> **Note:** The credentials in the screenshot above have been reset and are no longer valid, so please don't try to use them!

dego_vila@yahoo.com 30 Oct 2026

Creating email templates

To start with, we'll keep the content of the welcome email really simple, with a short message to let the user know that their registration was successful and to confirm their ID number. Similar to this:

```
Hi,

Thanks for signing up for a Greenlight account. We're excited to have you on board!

For future reference, your user ID number is 123.

Thanks,

The Greenlight Team
```

> **Note:** Including the user ID probably isn't something you'd normally do in a welcome email, but it's a simple way for us to demonstrate how to include *dynamic data* in emails — rather than just static content.

There are several different approaches we could take to define and manage the content for this email, but a convenient and flexible way is to use Go's templating functionality from the `html/template` package.

If you're following along, begin by creating a new `internal/mailer/templates` folder in your project directory and then add a `user_welcome.tmpl` file. Like so:

```
$ mkdir -p internal/mailer/templates
$ touch internal/mailer/templates/user_welcome.tmpl
```

Inside this file, we're going to define three *named templates* to use as part of our welcome email:

- A `"subject"` template containing the subject line for the email.
- A `"plainBody"` template containing the plain-text variant of the email message body.
- A `"htmlBody"` template containing the HTML variant of the email message body.

Go ahead and update the `internal/mailer/templates/user_welcome.tmpl` file to include the content for these templates:

dego_vila@yahoo.com 30 Oct 2026

File: internal/mailer/templates/user_welcome.tmpl

```
{{define "subject"}}Welcome to Greenlight!{{end}}

{{define "plainBody"}}
Hi,

Thanks for signing up for a Greenlight account. We're excited to have you on board!

For future reference, your user ID number is {{.ID}}.

Thanks,

The Greenlight Team
{{end}}

{{define "htmlBody"}}
<!doctype html>
<html>

<head>
    <meta name="viewport" content="width=device-width" />
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>

<body>
    <p>Hi,</p>
    <p>Thanks for signing up for a Greenlight account. We're excited to have you on board!</p>
    <p>For future reference, your user ID number is {{.ID}}.</p>
    <p>Thanks,</p>
    <p>The Greenlight Team</p>
</body>

</html>
{{end}}
```

If you've read *Let's Go*, this template syntax and structure should look very familiar to you, and we won't dwell on the details again here. But basically:

- We've defined the three *named templates* using the `{{define "..."}}...{{end}}` tags.
- You can render dynamic data in these templates via the `.` character (referred to as *dot*). In the next chapter we'll pass a `User` struct to the templates as dynamic data, which means that we can then render the user's ID using the tag `{{.ID}}` in the templates.

> **Note:** If you need to frequently change the text of emails or require them to be user-editable, then it might be appropriate to store these templates as strings in your database instead. But I've found that storing them in a file, like we are here, is a less complicated approach and a good starting point for most projects.

dego_vila@yahoo.com 30 Oct 2026

Sending a welcome email

Now that we have the content for the welcome email written, let's create the code to send it.

To send emails we *could* use Go's `net/smtp` package from the standard library. But unfortunately it's been frozen for a few years, and doesn't support some of the features that you might need in more advanced use cases, such as adding attachments.

So instead, I recommend using the third-party `wneessen/go-mail` package to help send emails. It's well tested, has good documentation, and a very clear and usable API.

If you're coding along, please use `go get` to download the latest release of this package:

```
$ go get github.com/wneessen/go-mail@latest
go: downloading github.com/wneessen/go-mail v0.8.1
go: downloading golang.org/x/text v0.42.0
go: added github.com/wneessen/go-mail v0.8.1
```

Creating an email helper

Rather than writing all the code for sending the welcome email in our `registerUserHandler`, in this chapter we're going to create a new `internal/mailer` package which wraps up the logic for parsing our email templates and sending emails.

In addition to that, we're also going to use Go's embedded files functionality, so that the email template files will be *built into our binary* when we create it later. This is really nice because it means we won't have to deploy these template files separately to our production server.

It's probably easiest to demonstrate how this works by getting straight into the code and talking through the details as we go.

Let's begin by creating a new `internal/mailer/mailer.go` file:

```
$ touch internal/mailer/mailer.go
```

And then go ahead and add the following code:

dego_vila@yahoo.com 30 Oct 2026

File: internal/mailer/mailer.go

```
package mailer

import (
    "bytes"
    "embed"
    "time"

    "github.com/wneessen/go-mail"

    // Import the html/template and text/template packages. Because these share the same
    // package name ("template") we need to disambiguate them and alias them to ht and tt
    // respectively.
    ht "html/template"
    tt "text/template"
)

// Below we declare a new variable with the type embed.FS (embedded file system) to hold
// our email templates. This has a comment directive in the format `//go:embed <path>`
// IMMEDIATELY ABOVE it, which indicates to Go that we want to store the contents of the
// ./templates directory in the templateFS embedded file system variable.
// ↓↓↓

//go:embed "templates"
var templateFS embed.FS

// Define a Mailer struct which contains a mail.Client instance (used to connect to a
// SMTP server) and the sender information for your emails (the name and address you
// want the email to be from, such as "Alice Smith <alice@example.com>").
type Mailer struct {
    client *mail.Client
    sender string
}

func New(host string, port int, username, password, sender string) (*Mailer, error) {
    // Initialize a new mail.Dialer instance with the given SMTP server settings. We
    // also configure this to use a 5-second timeout whenever we send an email. I've
    // split the NewClient arguments over multiple lines for readability, but you can
    // make this a single line if you prefer.
    client, err := mail.NewClient(
        host,
        mail.WithSMTPAuth(mail.SMTPAuthLogin),
        mail.WithPort(port),
        mail.WithUsername(username),
        mail.WithPassword(password),
        mail.WithTimeout(5*time.Second),
    )
    if err != nil {
        return nil, err
    }

    // Return a Mailer instance containing the client and sender information.
    mailer := &Mailer{
        client: client,
        sender: sender,
    }

    return mailer, nil
}

// Define a Send() method on the Mailer type. This takes the recipient email address
// as the first parameter, the name of the file containing the templates, and any
// dynamic data for the templates as an any parameter.
func (m *Mailer) Send(recipient string, templateFile string, data any) error {
    // Use the ParseFS() method from text/template to parse the required template file
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Use the ParseFS() method from text/template to parse the required template file
    // from the embedded file system.
    textTmpl, err := tt.New("").ParseFS(templateFS, "templates/"+templateFile)
    if err != nil {
        return err
    }

    // Execute the named template "subject", passing in the dynamic data and storing the
    // result in a bytes.Buffer variable.
    subject := new(bytes.Buffer)
    err = textTmpl.ExecuteTemplate(subject, "subject", data)
    if err != nil {
        return err
    }

    // Follow the same pattern to execute the "plainBody" template and store the result
    // in the plainBody variable.
    plainBody := new(bytes.Buffer)
    err = textTmpl.ExecuteTemplate(plainBody, "plainBody", data)
    if err != nil {
        return err
    }

    // Use the ParseFS() method from html/template this time to parse the required template
    // file from the embedded file system.
    htmlTmpl, err := ht.New("").ParseFS(templateFS, "templates/"+templateFile)
    if err != nil {
        return err
    }

    // And execute the "htmlBody" template and store the result in the htmlBody variable.
    htmlBody := new(bytes.Buffer)
    err = htmlTmpl.ExecuteTemplate(htmlBody, "htmlBody", data)
    if err != nil {
        return err
    }

    // Use the mail.NewMsg() function to initialize a new mail.Msg instance.
    // Then we use the To(), From() and Subject() methods to set the email recipient,
    // sender and subject headers, the SetBodyString() method to set the plain-text body,
    // and the AddAlternativeString() method to set the HTML body.
    msg := mail.NewMsg()

    err = msg.To(recipient)
    if err != nil {
        return err
    }

    err = msg.From(m.sender)
    if err != nil {
        return err
    }

    msg.Subject(subject.String())
    msg.SetBodyString(mail.TypeTextPlain, plainBody.String())
    msg.AddAlternativeString(mail.TypeTextHTML, htmlBody.String())

    // Call the DialAndSend() method on the dialer, passing in the message to send. This
    // opens a connection to the SMTP server, sends the message, then closes the
    // connection.
    return m.client.DialAndSend(msg)
}
```

dego_vila@yahoo.com 30 Oct 2026

Using embedded file systems

Before we continue, let's take a quick moment to discuss embedded file systems in more detail, because there are a couple of things that can be confusing when you first encounter them.

- You can only use the `//go:embed` directive on global variables at package level, not within functions or methods. If you try to use it in a function or method, you'll get the error `"go:embed cannot apply to var inside func"` at compile time.
- When you use the directive `//go:embed "<path>"` to create an embedded file system, the path should be *relative to the source code file containing the directive*. So in our case, `//go:embed "templates"` embeds the contents of the directory at `internal/mailer/templates`.
- The embedded file system is rooted in the directory which contains the `//go:embed` directive. So, in our case, to get the `user_welcome.tmpl` file we need to retrieve it from `templates/user_welcome.tmpl` in the embedded file system.
- Paths cannot contain `.` or `..` elements, nor may they begin or end with a `/`. This essentially restricts you to only embedding files that are contained within the same directory (or a subdirectory) as the source code which has the `//go:embed` directive.
- If the path is for a directory, then all files in the directory are recursively embedded, except for files with names that begin with `.` or `_`. If you want to include these files you should use the `*` wildcard character in the path, like `//go:embed "templates/*"`
- You can specify multiple directories and files in one directive. For example: `//go:embed "images" "styles/css" "favicon.ico"`.
- The path separator should always be a forward slash, even on Windows machines.

Using our mail helper

Now that our email helper package is in place, we need to hook it up to the rest of our code in the `cmd/api/main.go` file. Specifically, we need to do two things:

- Adapt our code to accept the configuration settings for the SMTP server as command-line flags.
- Initialize a new `Mailer` instance and make it available to our handlers via the `application` struct.

dego_vila@yahoo.com 30 Oct 2026

If you're following along, please make sure to use your own Mailtrap SMTP server settings from the previous chapter as the default values for the command-line flags here — not the exact values I'm using in the code below.

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "flag"
    "log/slog"
    "os"
    "time"

    "greenlight.alexedwards.net/internal/mailer" // New import
    "greenlight.alexedwards.net/internal/models"

    _ "github.com/lib/pq"
)

const version = "1.0.0"

// Update the config struct to hold the SMTP server settings.
type config struct {
    port int
    env  string
    db   struct {
        dsn          string
        maxOpenConns int
        maxIdleConns int
        maxIdleTime  time.Duration
    }
    limiter struct {
        enabled bool
        rps     float64
        burst   int
    }
    smtp struct {
        host     string
        port     int
        username string
        password string
        sender   string
    }
}

// Update the application struct to hold a pointer to a new Mailer instance.
type application struct {
    config config
    logger *slog.Logger
    models models.Models
    mailer *mailer.Mailer
}

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")
```

dego_vila@yahoo.com 30 Oct 2026

```
    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    flag.BoolVar(&cfg.limiter.enabled, "limiter-enabled", true, "Enable rate limiter")
    flag.Float64Var(&cfg.limiter.rps, "limiter-rps", 2, "Rate limiter maximum requests per second")
    flag.IntVar(&cfg.limiter.burst, "limiter-burst", 4, "Rate limiter maximum burst")

    // Read the SMTP server configuration settings into the config struct, using the
    // Mailtrap settings as the default values. IMPORTANT: If you're following along,
    // make sure to replace the default values for smtp-username and smtp-password
    // with your own Mailtrap credentials.
    flag.StringVar(&cfg.smtp.host, "smtp-host", "sandbox.smtp.mailtrap.io", "SMTP host")
    flag.IntVar(&cfg.smtp.port, "smtp-port", 25, "SMTP port")
    flag.StringVar(&cfg.smtp.username, "smtp-username", "a7420fc0883489", "SMTP username")
    flag.StringVar(&cfg.smtp.password, "smtp-password", "e75ffd0a3aa5ec", "SMTP password")
    flag.StringVar(&cfg.smtp.sender, "smtp-sender", "Greenlight <no-reply@greenlight.alexedwards.net>", "SMTP sender")

    flag.Parse()

    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))

    db, err := openDB(cfg)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
    defer db.Close()

    logger.Info("database connection pool established")

    // Initialize a new Mailer instance using the settings from the command line
    // flags.
    mailer, err := mailer.New(cfg.smtp.host, cfg.smtp.port, cfg.smtp.username, cfg.smtp.password, cfg.smtp.sender)
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }

    // And add it to the application struct.
    app := &application{
        config: cfg,
        logger: logger,
        models: models.NewModels(db),
        mailer: mailer,
    }

    err = app.serve()
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
}

...
```

And then the final thing we need to do is update our `registerUserHandler` to actually send the email, which we can do like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/users.go

```
package main

...

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {

    ... // Nothing above here needs to change.

    // Call the Send() method on our Mailer, passing in the user's email address,
    // name of the template file, and the User struct containing the new user's data.
    err = app.mailer.Send(user.Email, "user_welcome.tmpl", user)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    err = app.writeJSON(w, http.StatusCreated, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Alright, let's try this out!

Run the application, then use `curl` in another terminal window to register a brand-new user with the email address `bob@example.com`:

```
$ BODY='{"name": "Bob Jones", "email": "bob@example.com", "password": "pa55word"}'
$ curl -w '\nTime: %{time_total}\n' -d "$BODY" localhost:4000/v1/users
{
    "user": {
        "id": 3,
        "created_at": "2026-09-13T11:03:12+02:00",
        "name": "Bob Jones",
        "email": "bob@example.com",
        "activated": false
    }
}

Time: 2.331957
```

If everything is set up correctly you should get a `201 Created` response containing the new user details, similar to the response above.

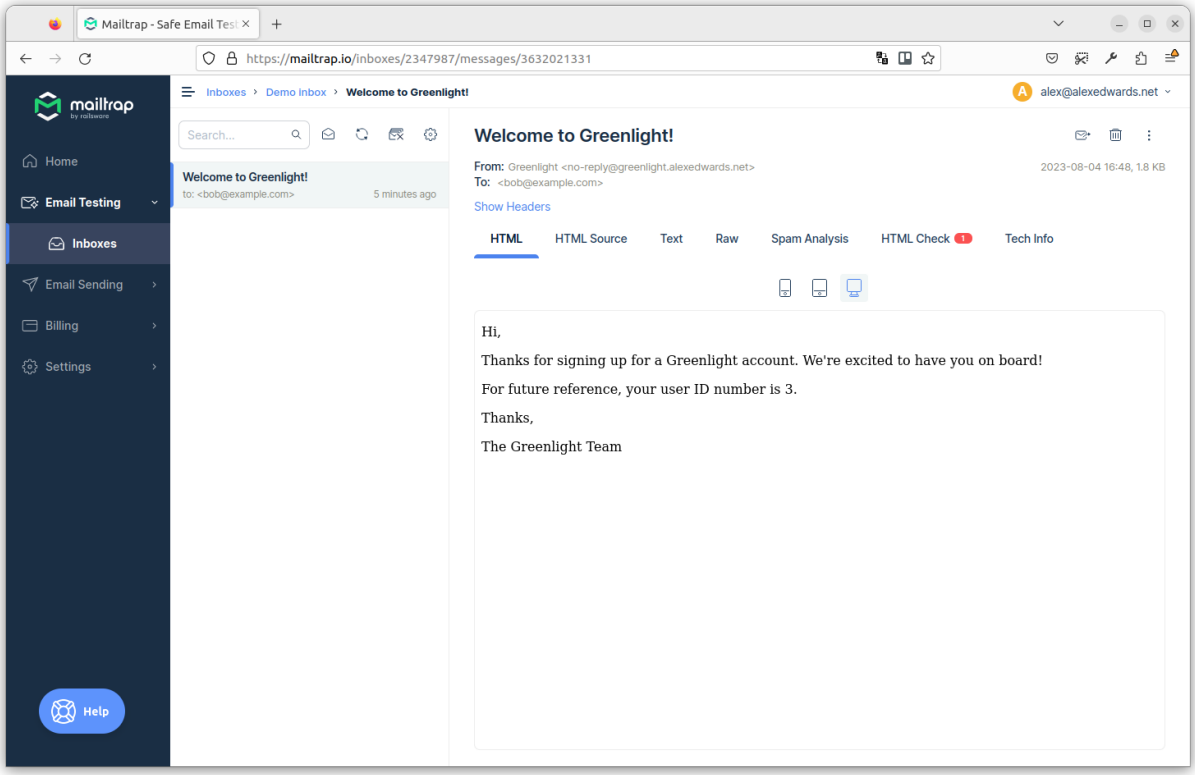
> **Important:** If you receive a `"dial tcp: connect: connection refused"` error this means that your application could not connect to the Mailtrap SMTP server. Please double check that your Mailtrap credentials are correct, or try using port `2525` instead of port `25`.

dego_vila@yahoo.com 30 Oct 2026

> **Note:** I've used the `-w` flag again to display the total time taken for the request to complete, which in my case was ≈ 2.3 seconds. That's quite a long time for an HTTP request to complete, and we'll speed it up in the next chapter by sending the welcome email in a background goroutine.

Checking the email in Mailtrap

If you've been following along and are using the Mailtrap SMTP server credentials, when you go back to your account you should now see the welcome email for `bob@example.com` in your **Demo inbox**, like so:



If you want, you can also click on the **Text** tab to see the plain-text version of the email, and the **Raw** tab to see the complete email including headers.

Wrapping this up, we covered quite a lot of ground in this chapter. But the nice thing about the pattern we've built is that it's easy to extend. If we want to send other emails from our application in the future, we can simply make an additional file in our `internal/mailer/templates` folder with the email content, and then send it from our handlers in the same way that we have done here.

dego_vila@yahoo.com 30 Oct 2026

Additional information

Retrying email send attempts

If you want, you can make the email sending process a bit more robust by adding some basic ‘retry’ functionality to the `Mailer.Send()` method. For example:

```
func (m Mailer) Send(recipient, templateFile string, data any) error {
    ...

    // Try sending the email up to three times before aborting and returning the final
    // error. We sleep for 500 milliseconds between each attempt.
    for i := 1; i <= 3; i++ {
        err = m.client.DialAndSend(msg)
        if err == nil {
            return nil
        }

        // If it didn't work, sleep for a short time and retry.
        if i != 3 {
            time.Sleep(500 * time.Millisecond)
        }
    }

    return err
}
```

This retry functionality is a relatively simple addition to our code, but it helps to increase the probability that emails are successfully sent in the event of transient network issues. If you’re sending emails in a background process (like we will do in the next chapter), you might want to make the sleep duration even longer here, as it won’t materially impact the client and gives more time for transient issues to resolve.

dego_vila@yahoo.com 30 Oct 2026

Sending background emails

As we mentioned briefly in the last chapter, sending the welcome email from the `registerUserHandler` method adds quite a lot of latency to the total request/response round-trip for the client.

One way we could reduce this latency is by sending the email in a *background goroutine*. This would effectively 'decouple' the task of sending an email from the rest of the code in our `registerUserHandler`, and means that we could return an HTTP response to the client without waiting for the email sending to complete.

At its very simplest, we could adapt our handler to execute the email send in a background goroutine like this:

File: cmd/api/users.go

```
package main
...

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {

    ...

    // Launch a goroutine which runs an anonymous function that sends the welcome email.
    go func() {
        err := app.mailer.Send(user.Email, "user_welcome.tmpl", user)
        if err != nil {
            // Importantly, if there is an error sending the email then we use the
            // app.logger.Error() helper to manage it, instead of the
            // app.serverErrorResponse() helper like before.
            app.logger.Error(err.Error())
        }
    }()

    // Note that we also change this to send the client a 202 Accepted status code.
    // This status code indicates that the request has been accepted for processing, but
    // the processing has not yet been completed.
    err = app.writeJSON(w, http.StatusAccepted, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

When this code is executed now, a new 'background' goroutine will be launched for sending the welcome email. The code in this background goroutine will be executed *concurrently* with the subsequent code in our `registerUserHandler`, which means we are no longer waiting for the email to be sent before we return a JSON response to the client. Most likely, the background goroutine will still be executing its code long after the

dego_vila@yahoo.com 30 Oct 2026

`registerUserHandler` has returned.

There are a couple of things I'd like to emphasize here:

- We use the `app.logger.Error()` method to manage any errors in our background goroutine. This is because by the time we encounter the errors, the client will probably have already been sent a `202 Accepted` response by our `writeJSON()` helper.

  Note that we don't want to use the `app.serverErrorResponse()` helper to handle any errors in our background goroutine, as that would result in us trying to write a *second HTTP response* and we would get a `"http: superfluous response.WriteHeader call"` error from our `http.Server` at runtime.

- The code running in the background goroutine forms a *closure* over the `user` and `app` variables. It's important to be aware that these 'closed over' variables are *not scoped to the background goroutine*, which means that any changes you make to them will be reflected in the rest of your codebase. For a simple example of this, see the following playground code.

  In our case we aren't changing the value of these variables in any way, so this behavior won't cause us any issues. But it is important to keep in mind.

OK, let's try this out!

Restart the API, then go ahead and register another new user with the email address `carol@example.com`. Like so:

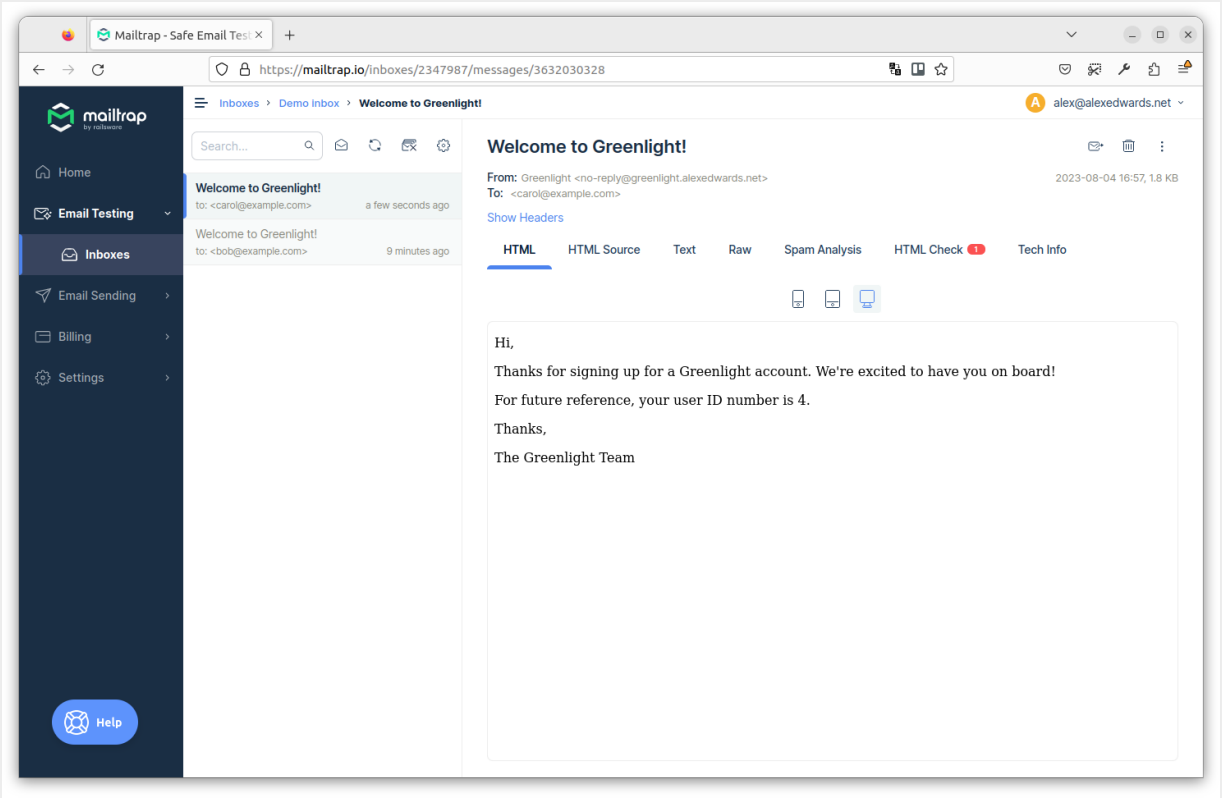
```
$ BODY='{"name": "Carol Smith", "email": "carol@example.com", "password": "pa55word"}'
$ curl -w '\nTime: %{time_total}\n' -d "$BODY" localhost:4000/v1/users
{
    "user": {
        "id": 4,
        "created_at": "2026-09-13T11:09:16+02:00",
        "name": "Carol Smith",
        "email": "carol@example.com",
        "activated": false
    }
}

Time: 0.268639
```

This time, you should see that the time taken to return the response is much faster — in my case 0.27 seconds compared to the previous 2.33 seconds.

And if you take a look at your Mailtrap inbox, you should see that the email for `carol@example.com` has been delivered correctly. Like so:

dego_vila@yahoo.com 30 Oct 2026



Recovering panics

It's important to bear in mind that any panic which happens in this background goroutine *will not* be automatically recovered by our `recoverPanic()` middleware or Go's `http.Server`, and will cause our whole application to terminate.

In very simple background goroutines this is less of a worry. But the code involved in sending an email is quite complex (including calls to a third-party package) and the risk of a runtime panic is non-negligible. So we need to make sure that any panic in this background goroutine is manually recovered, using a similar pattern to the one in our `recoverPanic()` middleware.

I'll demonstrate.

Reopen your `cmd/api/users.go` file and update the `registerUserHandler` like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/users.go

```
package main

import (
    "errors"
    "fmt" // New import
    "net/http"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"
)

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {

    ...

    // Launch a background goroutine to send the welcome email.
    go func() {
        // Run a deferred function which uses recover() to catch any panic, and log an
        // error message instead of terminating the application.
        defer func() {
            pv := recover()
            if pv != nil {
                app.logger.Error(fmt.Sprintf("%v", pv))
            }
        }()

        // Send the welcome email.
        err := app.mailer.Send(user.Email, "user_welcome.tmpl", user)
        if err != nil {
            app.logger.Error(err.Error())
        }
    }()

    err = app.writeJSON(w, http.StatusAccepted, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Using a helper function

If you need to execute a lot of background tasks in your application, it can get tedious to keep repeating the same panic recovery code — and there's a risk that you might forget to include it altogether.

To help take care of this, it's possible to create a simple helper function which wraps the panic recovery logic. If you're following along, open your `cmd/api/helpers.go` file and create a new `background()` helper method as follows:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

// The background() helper accepts an arbitrary function as a parameter.
func (app *application) background(fn func()) {
    // Launch a background goroutine.
    go func() {
        // Recover any panic.
        defer func() {
            pv := recover()
            if pv != nil {
                app.logger.Error(fmt.Sprintf("%v", pv))
            }
        }()

        // Execute the arbitrary function that we passed as the parameter.
        fn()
    }()
}
```

This `background()` helper leverages the fact that Go has first-class functions, which means that functions can be *assigned to variables* and *passed as parameters* to other functions.

In this case, we've set up the `background()` helper so that it accepts any function with the signature `func()` as a parameter and stores it in the variable `fn`. It then spins up a background goroutine, uses a deferred function to recover any panics and log the error, and then executes the function itself by calling `fn()`.

Now that this is in place, let's update our `registerUserHandler` to use it like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/users.go

```
package main

import (
    "errors"
    "net/http"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"
)

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {

    ...

    // Use the background helper to execute an anonymous function that sends the welcome
    // email.
    app.background(func() {
        err := app.mailer.Send(user.Email, "user_welcome.tmpl", user)
        if err != nil {
            app.logger.Error(err.Error())
        }
    })

    err = app.writeJSON(w, http.StatusAccepted, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

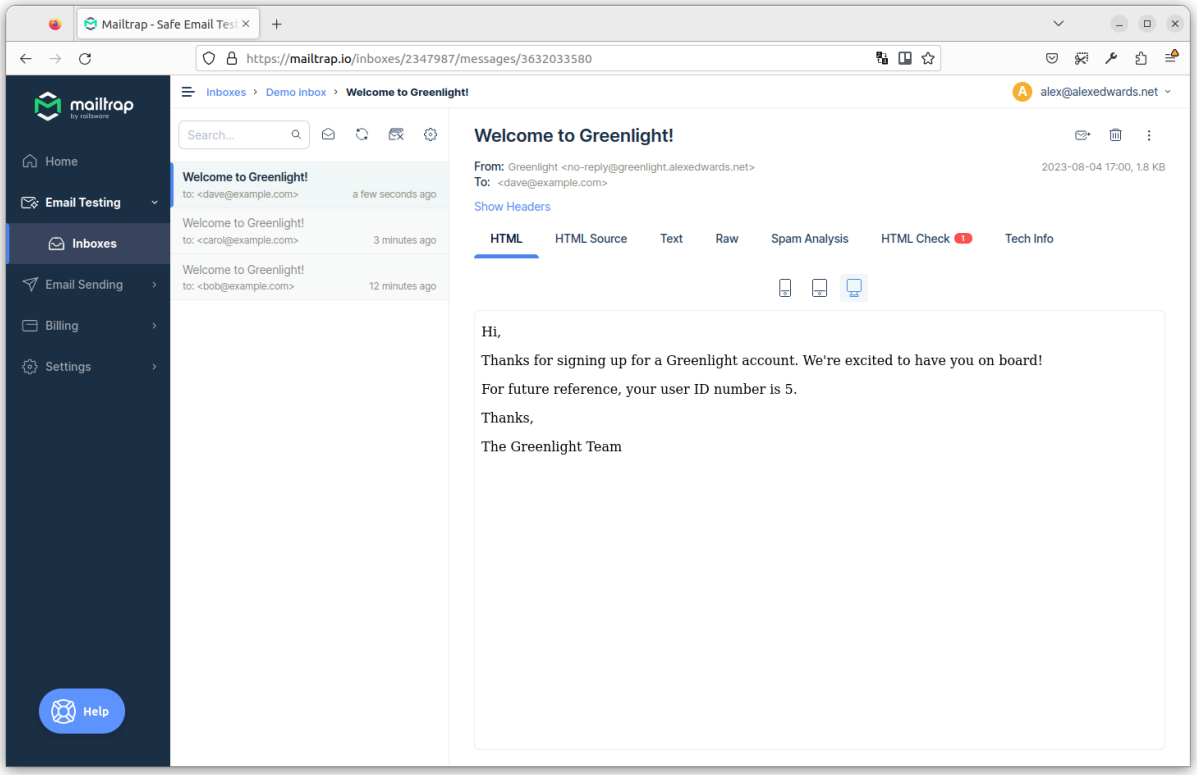
Let’s double-check that this is still working. Restart the API, then create another new user with the email address `dave@example.com`:

```
$ BODY='{"name": "Dave Smith", "email": "dave@example.com", "password": "pa55word"}'
$ curl -w '\nTime: %{time_total}\n' -d "$BODY" localhost:4000/v1/users
{
    "user": {
        "id": 5,
        "created_at": "2026-09-13T11:13:27+02:00",
        "name": "Dave Smith",
        "email": "dave@example.com",
        "activated": false
    }
}

Time: 0.267692
```

If everything is set up correctly, you’ll now see the corresponding email appear in your Mailtrap inbox again.

dego_vila@yahoo.com 30 Oct 2026



dego_vila@yahoo.com 30 Oct 2026

Graceful shutdown of background tasks

Sending our welcome email in the background is working well, but there's still an issue we need to address.

When we initiate a graceful shutdown of our application, it *won't wait for any background goroutines that we've launched to complete*. So — if we happen to shut down our server at an unlucky moment — it's possible that a new client will be created on our system but they will never be sent their welcome email.

Fortunately, we can prevent this by using Go's `sync.WaitGroup` functionality to coordinate the graceful shutdown and our background goroutines.

An introduction to sync.WaitGroup

When you want to wait for a collection of goroutines to finish their work, the principal tool to help with this is the `sync.WaitGroup` type.

The way that `sync.WaitGroup` works is conceptually a bit like a 'counter'. Each time you launch a background goroutine using the `WaitGroup.Go()` method it increments a counter by 1, and when each goroutine finishes, it decrements the counter by 1. You can then use the `WaitGroup.Wait()` method to block execution of your code and wait until the counter equals zero — at which point you know that all your background goroutines have finished.

Let's take a quick look at a standalone example of how it works in practice.

In the code below, we'll launch five goroutines that print out `"hello from a goroutine"`, and use `sync.WaitGroup` to wait for them all to complete before the program exits.

dego_vila@yahoo.com 30 Oct 2026

```
package main

import (
    "log"
    "sync"
    "time"
)

func main() {
    // Declare a new WaitGroup.
    var wg sync.WaitGroup

    // Execute a loop 5 times.
    for range 5 {
        // Use the Go() method to launch a new goroutine, which sleeps for 1
        // second and then prints a log message.
        wg.Go(func() {
            time.Sleep(time.Second)
            log.Println("hello from a goroutine ")
        })
    }

    log.Println("all goroutines launched")

    // Wait() blocks until the WaitGroup counter is zero --- essentially blocking until all
    // goroutines have completed.
    wg.Wait()

    log.Println("all goroutines finished")
}
```

If you run the above code, you'll see that the output looks like this:

```
2009/11/10 23:00:00 all goroutines launched
2009/11/10 23:00:01 hello from a goroutine
2009/11/10 23:00:01 hello from a goroutine
2009/11/10 23:00:01 hello from a goroutine
2009/11/10 23:00:01 hello from a goroutine
2009/11/10 23:00:01 hello from a goroutine
2009/11/10 23:00:01 all goroutines finished
```

We can see from the log messages and timestamps that all the goroutines are launched, but the call to `wg.Wait()` blocks any further execution of our code until all the goroutines finish around 1 second later. Once they have all completed, the block is lifted and the final log message gets printed.

Fixing our application

Let's update our application to incorporate a `sync.WaitGroup` that coordinates our graceful shutdown and background goroutines.

We'll begin in our `cmd/api/main.go` file, and edit the `application` struct to contain a new

dego_vila@yahoo.com 30 Oct 2026

`sync.WaitGroup`. Like so:

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "flag"
    "log/slog"
    "os"
    "sync" // New import
    "time"

    "greenlight.alexedwards.net/internal/mailer"
    "greenlight.alexedwards.net/internal/models"

    _ "github.com/lib/pq"
)

...

// Include a sync.WaitGroup in the application struct. The zero value for a
// sync.WaitGroup type is a valid, usable, sync.WaitGroup with a 'counter' value of 0,
// so we don't need to do anything else to initialize it before we can use it.
type application struct {
    config config
    logger *slog.Logger
    models models.Models
    mailer *mailer.Mailer
    wg     sync.WaitGroup
}

...
```

Next let's head to the `cmd/api/helpers.go` file and update the `app.background()` helper to launch goroutines using this `sync.WaitGroup`, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/helpers.go

```
package main

...

func (app *application) background(fn func()) {
    // Use the Go() method to launch the background goroutine. The code inside
    // this is exactly the same as in the previous chapter --- we're just now
    // launching the goroutine using the WaitGroup's Go() method instead of the
    // regular go() keyword.
    app.wg.Go(func() {
        defer func() {
            pv := recover()
            if pv != nil {
                app.logger.Error(fmt.Sprintf("%v", pv))
            }
        }()

        fn()
    })
}
```

Then the final thing we need to do is update our graceful shutdown functionality so that it waits for any of these background goroutines to complete before terminating the application. We can do that by adapting our `app.serve()` method like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/server.go

```
package main

...

func (app *application) serve() error {
    srv := &http.Server{
        Addr:         fmt.Sprintf(":%d", app.config.port),
        Handler:      app.routes(),
        IdleTimeout:  time.Minute,
        ReadTimeout:  5 * time.Second,
        WriteTimeout: 10 * time.Second,
        ErrorLog:     slog.NewLogLogger(app.logger.Handler(), slog.LevelError),
    }

    shutdownError := make(chan error)

    go func() {
        quit := make(chan os.Signal, 1)
        signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
        s := <-quit

        app.logger.Info("stopping server", "addr", srv.Addr, "signal", s.String())
        shutdownError <- srv.Shutdown(context.Background())
    }()

    app.logger.Info("starting server", "addr", srv.Addr, "env", app.config.env)

    err := srv.ListenAndServe()
    if !errors.Is(err, http.ErrServerClosed) {
        return err
    }

    err = <-shutdownError
    if err != nil {
        return err
    }

    // Log a message to say that we're waiting for any background goroutines to
    // complete their tasks.
    app.logger.Info("waiting for background tasks")

    // Call Wait() to block until our application WaitGroup counter is zero --- essentially
    // blocking until the background goroutines have finished.
    app.wg.Wait()

    app.logger.Info("shutdown complete")
    return nil
}
```

To try this out, go ahead and restart the API and then send a request to the `POST /v1/users` endpoint immediately followed by a `SIGTERM` signal. For example:

```
$ BODY='{"name": "Edith Smith", "email": "edith@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/users & pkill -SIGTERM api &
```

When you do this, your server logs should look similar to the output below:

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/api
time=2026-09-13T11:17:48.051+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T11:17:48.051+02:00 level=INFO msg="starting server" addr=:4000 env=development
...
time=2026-09-13T11:18:04.678+02:00 level=INFO msg="stopping server" addr=:4000 signal=terminated
time=2026-09-13T11:18:04.945+02:00 level=INFO msg="waiting for background tasks"
time=2026-09-13T11:18:07.592+02:00 level=INFO msg="shutdown complete
```

Notice that after `"waiting for background tasks"` is logged there is a pause of a couple of seconds, while the background email sending completes, finally followed by the `"shutdown complete"` message?

This nicely illustrates how the graceful shutdown process waited for the welcome email to be sent (which took about 2.6 seconds in my case) before finally terminating the application.

dego_vila@yahoo.com 30 Oct 2026

Chapter 14.

User activation

At the moment, a user can register for an account with our *Greenlight* API, but we don't know for sure that the email address they provided during registration *actually belongs to them*.

So, in this section of the book, we're going to confirm that a user used their own, real, email address by including 'account activation' instructions in their welcome email.

There are several reasons for having an account activation step, but the main benefits are that it adds an additional hoop for bots to jump through, and helps prevent abuse by people who register with a fake email address or one that doesn't belong to them.

To give you an overview upfront, the account activation process will work like this:

1. As part of the registration process for a new user we will create a cryptographically-secure random *activation token* that is impossible to guess.
2. We will then store a hash of this activation token in a new `tokens` table, alongside the new user's ID and an expiry time for the token.
3. We will send the original (unhashed) activation token to the user in their welcome email.
4. The user subsequently submits their token to a new `PUT /v1/users/activated` endpoint.
5. If the hash of the token exists in the `tokens` table and hasn't expired, then we'll set the `activated` status for the relevant user to `true`.
6. Lastly, we'll delete the activation token from our `tokens` table so that it cannot be used again.

In this section of the book, you'll learn how to:

- Implement a secure 'account activation' workflow which verifies a new user's email address.
- Generate cryptographically-secure random tokens using Go's `crypto/rand` package.
- Generate fast hashes of data using the `crypto/sha256` package.
- Implement patterns for working with cross-table relationships in your database, including setting up foreign keys and retrieving related data via SQL `JOIN` queries.

dego_vila@yahoo.com 30 Oct 2026

Chapter 14.1.

Setting up the tokens database table

Let's begin by creating a new `tokens` table in our database to store the activation tokens for our users. If you're following along, run the following command to create a new pair of migration files:

```
$ migrate create -seq -ext .sql -dir ./migrations create_tokens_table
/home/alex/Projects/greenlight/migrations/000005_create_tokens_table.up.sql
/home/alex/Projects/greenlight/migrations/000005_create_tokens_table.down.sql
```

And then add the following SQL statements to the 'up' and 'down' migration files respectively:

```
File: migrations/000005_create_tokens_table.up.sql
```

```
CREATE TABLE IF NOT EXISTS tokens (
    hash bytea PRIMARY KEY,
    user_id bigint NOT NULL REFERENCES users ON DELETE CASCADE,
    expiry timestamp(0) with time zone NOT NULL,
    scope text NOT NULL
);
```

```
File: migrations/000005_create_tokens_table.down.sql
```

```
DROP TABLE IF EXISTS tokens;
```

Let's quickly step through the columns in this new `tokens` table and explain their purpose.

- The `hash` column will contain a SHA-256 hash of the activation token. It's important to emphasize that we will only store a hash of the activation token in our database — not the activation token itself.

  We want to hash the token before storing it for the same reason that we bcrypt a user's password — it provides an extra layer of protection if the database is ever compromised or leaked. Because our activation token is going to be a high-entropy random string (128 bits) — rather than something low-entropy like a typical user password — it is sufficient to use a fast algorithm like SHA-256 to create the hash, instead of a slow algorithm like bcrypt.

- The `user_id` column will contain the ID of the user associated with the token. We use the

dego_vila@yahoo.com 30 Oct 2026

`REFERENCES users` syntax to create a foreign key constraint against the primary key of our `users` table, which ensures that any value in the `user_id` column has a corresponding `id` entry in our `users` table.

We also use the `ON DELETE CASCADE` syntax to instruct PostgreSQL to *automatically delete all records for a user in our `tokens` table when the parent record in the `users` table is deleted*.

> **Note:** A common alternative to `ON DELETE CASCADE` is `ON DELETE RESTRICT`, which in our case would *prevent* a parent record in the `users` table from being deleted if the user has any tokens in our `tokens` table. If you use `ON DELETE RESTRICT`, you would need to manually delete any tokens for the user *before* you delete the user record itself.

- The `expiry` column will contain the time when a token is considered 'expired' and no longer valid. Setting a short expiry time is good from a security point of view because it helps reduce the window of possibility for a successful brute-force attack against the token. And it also helps in the scenario where the user is *sent a token but doesn't use it*, and their email account is compromised at a later time. By setting a short time limit, it reduces the time window that the compromised token could be used.

  Of course, the security risks here need to be weighed up against usability, and we want the expiry time to be long enough for a user to be able to activate the account at their leisure. In our case, we'll set the expiry time for our activation tokens to 3 days from the moment the token was created.

- The `scope` column will denote what *purpose* the token can be used for. Later in the book we'll also need to create and store *authentication tokens*, and most of the code and storage requirements for these are exactly the same as for our activation tokens. So instead of creating separate tables (and the code to interact with them), we'll store them in one table with a value in the `scope` column to restrict the purpose that the token can be used for.

OK, with those explanations out of the way, you should be able to execute the 'up' migration with the following command:

```
$ migrate -path=./migrations -database=$GREENLIGHT_DB_DSN up
5/u create_tokens_table (21.568194ms)
```

dego_vila@yahoo.com 30 Oct 2026

Creating secure activation tokens

The integrity of our activation process hinges on one key thing: the 'unguessability' of the token that we send to the user's email address. If the token is easy to guess or can be brute-forced, then it would be possible for an attacker to activate a user's account even if they don't have access to the user's email inbox.

Because of this, we want the token to be generated by a *cryptographically secure random number generator* (CSPRNG) and have enough entropy (or *randomness*) that it is impossible to guess. An easy way to create a token that matches these criteria is by using the `rand.Text()` function from the `crypto/rand` package.

This will generate tokens that contain 128 bits (16 bytes) of entropy, encoded using the standard base32 alphabet. In practice, this means that `rand.Text()` returns strings that are 26 characters long and look like this:

```
CN5MWVETIILGP32FBV3EOGBNRV
LYDISI72PTLGTIVEDSV5IATEAR
EADOYJU5WJC3CCR3KZPSJW5BJA
```

If you're following along, go ahead and create a new `internal/models/tokens.go` file. This will act as the home for all our logic related to creating and managing tokens over the next couple of chapters.

```
$ touch internal/models/tokens.go
```

Then in this file let's define a `Token` struct (to represent the data for an individual token), and a `generateToken()` function that we can use to create a new token.

This is another time where it's probably easiest to jump straight into the code, and describe what's happening as we go along.

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/tokens.go

```
package models

import (
    "crypto/rand"
    "crypto/sha256"
    "time"
)

// Define constants for the token scope. For now we just define the scope "activation"
// but we'll add additional scopes later in the book.
const (
    ScopeActivation = "activation"
)

// Define a Token struct to hold the data for an individual token. This includes the
// plaintext and hashed versions of the token, associated user ID, expiry time and
// scope.
type Token struct {
    Plaintext string
    Hash      []byte
    UserID    int
    Expiry    time.Time
    Scope     string
}

func generateToken(userID int, ttl time.Duration, scope string) Token {
    // Create a Token instance. In this, we set the Plaintext field to be a random
    // token generated by rand.Text(), and also set values for the user ID, expiry, and
    // scope of the token. Notice that we add the provided ttl (time-to-live) duration
    // argument to the current time to get the expiry time.
    token := Token{
        Plaintext: rand.Text(),
        UserID:    userID,
        Expiry:    time.Now().Add(ttl),
        Scope:     scope,
    }

    // Generate a SHA-256 hash of the plaintext token string. This will be the value
    // that we store in the `hash` column of our database table. Note that the
    // sha256.Sum256() function returns an *array* of length 32, so to make it easier to
    // work with we convert it to a slice using the [:] operator before storing it.
    hash := sha256.Sum256([]byte(token.Plaintext))
    token.Hash = hash[:]

    return token
}
```

Creating the TokenModel and validation checks

OK, let's move on and set up a `TokenModel` type which encapsulates the database interactions with our PostgreSQL `tokens` table. We'll follow a very similar pattern to the `MovieModel` and `UsersModel` again, and we'll implement the following three methods on it:

- `Insert()` to insert a new token record in the database.
- `New()` will be a shortcut method which creates a new token using the `generateToken()`

dego_vila@yahoo.com 30 Oct 2026

function and then calls `Insert()` to store the data.

- `DeleteAllForUser()` to delete all tokens with a specific scope for a specific user.

We'll also create a new `ValidateTokenPlaintext()` function, which will check that a plaintext token provided by a client in the future is exactly 26 bytes long.

Open up the `internal/models/tokens.go` file again, and add the following code:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/tokens.go

```
package models

import (
    "context" // New import
    "crypto/rand"
    "crypto/sha256"
    "database/sql" // New import
    "time"

    "greenlight.alexedwards.net/internal/validator" // New import
)

...

// Check that the plaintext token has been provided and is exactly 26 bytes long.
func ValidateTokenPlaintext(v *validator.Validator, tokenPlaintext string) {
    v.Check(tokenPlaintext != "", "token", "must be provided")
    v.Check(len(tokenPlaintext) == 26, "token", "must be 26 bytes long")
}

// Define the TokenModel type.
type TokenModel struct {
    DB *sql.DB
}

// The New() method is a shortcut which creates a new Token struct and then inserts the
// data in the tokens table.
func (m TokenModel) New(userID int, ttl time.Duration, scope string) (Token, error) {
    token := generateToken(userID, ttl, scope)

    err := m.insert(token)
    return token, err
}

// insert() adds the data for a specific token to the tokens table.
func (m TokenModel) insert(token Token) error {
    query := `
        INSERT INTO tokens (hash, user_id, expiry, scope)
        VALUES ($1, $2, $3, $4)`

    args := []any{token.Hash, token.UserID, token.Expiry, token.Scope}

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    _, err := m.DB.ExecContext(ctx, query, args...)
    return err
}

// DeleteAllForUser() deletes all tokens for a specific user and scope.
func (m TokenModel) DeleteAllForUser(scope string, userID int) error {
    query := `
        DELETE FROM tokens
        WHERE scope = $1 AND user_id = $2`

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    _, err := m.DB.ExecContext(ctx, query, scope, userID)
    return err
}
```

dego_vila@yahoo.com 30 Oct 2026

And finally, we need to update the `internal/models/models.go` file so that the new `TokenModel` is included in our parent `Models` struct. Like so:

File: internal/models/models.go

```
package models

...

type Models struct {
    Movies MovieModel
    Tokens TokenModel // Add a new Tokens field.
    Users  UserModel
}

func NewModels(db *sql.DB) Models {
    return Models{
        Movies: MovieModel{DB: db},
        Tokens: TokenModel{DB: db}, // Initialize a new TokenModel instance.
        Users:  UserModel{DB: db},
    }
}
```

At this point you should be able to restart the application, and everything should work without a hitch.

```
$ go run ./cmd/api/
time=2026-09-13T11:24:10.207+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T11:24:10.207+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

dego_vila@yahoo.com 30 Oct 2026

Sending activation tokens

The next step is to hook this up to our `registerUserHandler`, so that we generate an activation token when a user signs up and include it in their welcome email — similar to this:

```
Hi,

Thanks for signing up for a Greenlight account. We're excited to have you on board!

For future reference, your user ID number is 123.

Please send a request to the `PUT /v1/users/activated` endpoint with the following JSON
body to activate your account:

{"token": "RMMCV3MZCEBYQADXBODCLTAF6L"}

Please note that this is a one-time use token and it will expire in 3 days.

Thanks,

The Greenlight Team
```

The most important thing about this email is that we're instructing the user to activate by issuing a `PUT` request to our API. We're not asking them to activate by *clicking a link* that contains the token as part of the URL path or query string.

Having a user click a link to activate via a `GET` request (which is used by default when clicking a link) would certainly be more convenient, but in the case of our API it has some big drawbacks. In particular:

- It would violate the HTTP principle that the `GET` method should only be used for 'safe' requests which retrieve resources — not for requests that modify something (like a user's activation status).
- It's possible that the user's web browser or antivirus will pre-fetch the link URL in the background, inadvertently activating the account. This Stack Overflow comment explains the risk of this nicely:

> *That could result in a scenario where a malicious actor (Eve) wants to make an account using someone else's email (Alice). Eve signs up, and Alice received an email. Alice opens the email because she is curious about an account she didn't request. Her browser (or antivirus) requests the URL in the background, inadvertently activating the account.*

dego_vila@yahoo.com 30 Oct 2026

All in all, you should make sure that any actions which change the state of your application (including activating a user) are only ever executed via `POST`, `PUT`, `PATCH` or `DELETE` requests — not by `GET` requests.

> **Note:** If your API is the back-end for a website, then you could adapt this email so that it asks the user to click a link that takes them to a page on your website. They can *then* click a button on the page to 'confirm their activation', which performs the `PUT` request to your API that actually activates the user. We'll look at this pattern in more detail in the next chapter.

But for now, if you're following along, go ahead and update your welcome email templates to include the activation token as follows:

dego_vila@yahoo.com 30 Oct 2026

File: internal/mailer/templates/user_welcome.tmpl

```
{{define "subject"}}Welcome to Greenlight!{{end}}

{{define "plainBody"}}
Hi,

Thanks for signing up for a Greenlight account. We're excited to have you on board!

For future reference, your user ID number is {{.userID}}.

Please send a request to the `PUT /v1/users/activated` endpoint with the following JSON
body to activate your account:

{"token": "{{.activationToken}}"}

Please note that this is a one-time use token and it will expire in 3 days.

Thanks,

The Greenlight Team
{{end}}

{{define "htmlBody"}}
<!doctype html>
<html>

<head>
    <meta name="viewport" content="width=device-width" />
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
</head>

<body>
    <p>Hi,</p>
    <p>Thanks for signing up for a Greenlight account. We're excited to have you on board!</p>
    <p>For future reference, your user ID number is {{.userID}}.</p>
     <p>Please send a request to the <code>PUT /v1/users/activated</code> endpoint with the
    following JSON body to activate your account:</p>
    <pre><code>
    {"token": "{{.activationToken}}"}
    </code></pre>
    <p>Please note that this is a one-time use token and it will expire in 3 days.</p>
    <p>Thanks,</p>
    <p>The Greenlight Team</p>
</body>

</html>
{{end}}
```

Next we'll need to update the `registerUserHandler` to generate a new activation token, and pass it to the welcome email template as dynamic data, along with the user ID.

Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/users.go

```
package main

import (
    "errors"
    "net/http"
    "time" // New import

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"
)

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {

    ...

    user, err = app.models.Users.Insert(user)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrDuplicateEmail):
            v.AddError("email", "a user with this email address already exists")
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // After the user record has been created in the database, generate a new activation
    // token for the user.
    token, err := app.models.Tokens.New(user.ID, 3*24*time.Hour, models.ScopeActivation)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    app.background(func() {
        // As there are now multiple pieces of data that we want to pass to our email
        // templates, we create a map to act as a 'holding structure' for the data. This
        // contains the plaintext version of the activation token for the user, along
        // with their ID.
        data := map[string]any{
            "activationToken": token.Plaintext,
            "userID":          user.ID,
        }

        // Send the welcome email, passing in the map above as dynamic data.
        err := app.mailer.Send(user.Email, "user_welcome.tmpl", data)
        if err != nil {
            app.logger.Error(err.Error())
        }
    })

    err = app.writeJSON(w, http.StatusAccepted, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

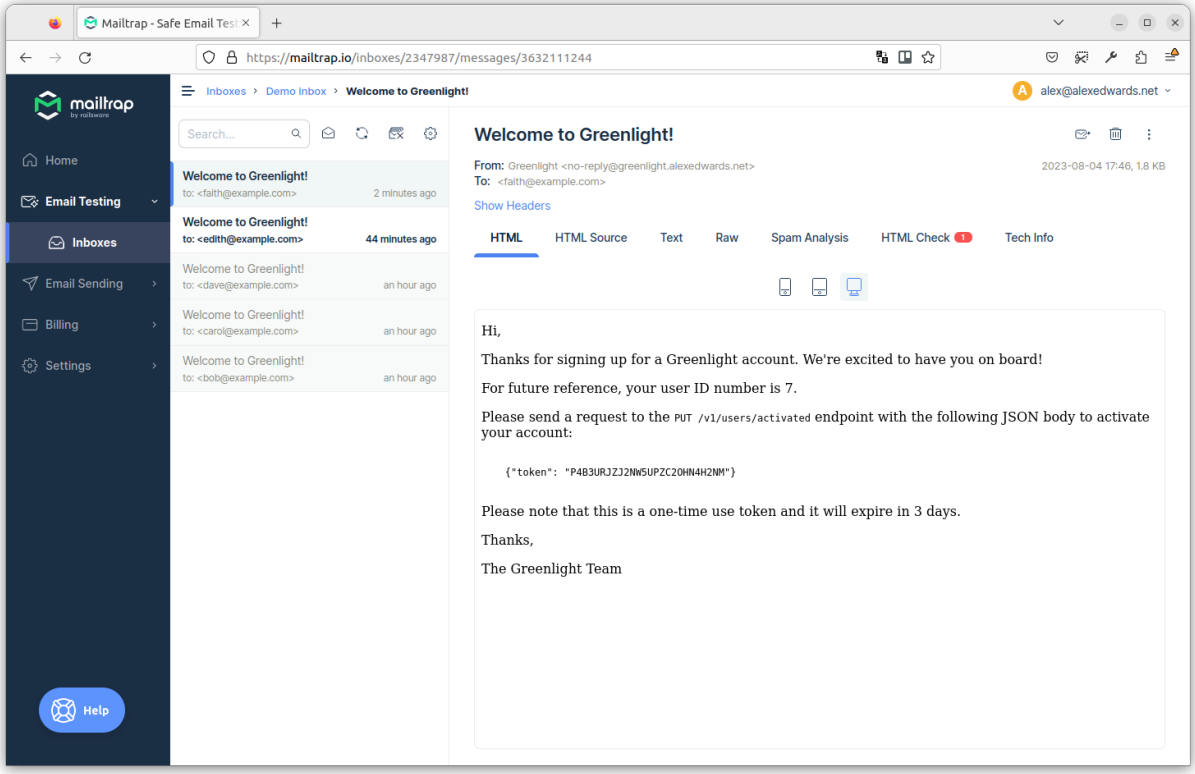
OK, let’s see how this works…

dego_vila@yahoo.com 30 Oct 2026

Restart the application, and then register a new user account with the email address `faith@example.com`. Similar to this:

```
$ BODY='{"name": "Faith Smith", "email": "faith@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/users
{
    "user": {
        "id": 7,
        "created_at": "2026-09-13T11:27:07+02:00",
        "name": "Faith Smith",
        "email": "faith@example.com",
        "activated": false
    }
}
```

And if you open your Mailtrap inbox again, you should now see the new welcome email containing the activation token for `faith@example.com`, like so:



So, in my case, we can see that `faith@example.com` has been sent the activation token `P4B3URJZJ2NW5UPZC2OHN4H2NM`. If you're following along, your token should be a different 26-character string.

Out of interest, let's quickly look inside the `tokens` table in our PostgreSQL database. Again, the exact values in your database will be different, but it should look similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT * FROM tokens;
                               hash                               | user_id |         expiry         |   scope
------------------------------------------------------------------+---------+------------------------+------------
 \x09bcb40206b25fe511bfef4d56cbe8c4a141869fc29612fa984b371ef086f5f5 |       7 | 2026-09-16 11:27:07+02 | activation
```

We can see here that the activation token hash is displayed as the value:

```
09bcb40206b25fe511bfef4d56cbe8c4a141869fc29612fa984b371ef086f5f5
```

As we mentioned earlier, `psql` always displays values in `bytea` columns as a hex-encoded string. So what we're seeing here is a *hexadecimal encoding of the SHA-256 hash of the plaintext token* `P4B3URJZJ2NW5UPZC2OHN4H2NM` that we sent in the welcome email.

> **Note:** If you want, you can verify that this is correct by entering a plaintext activation token from the welcome email into this online SHA-256 hash generator. You should see that the result matches the hex-encoded value inside your PostgreSQL database.

Notice too that the `user_id` value of `7` is correct for our `faith@example.com` user, the expiry time has been correctly set to three days from now, and the token also has the scope value `activation`.

Additional information

A standalone endpoint for generating tokens

You may also want to provide a standalone endpoint for generating and sending activation tokens to your users. This can be useful if you need to resend an activation token, such as when a user doesn't activate their account within the 3-day time limit, or they never receive their welcome email.

The code to implement this endpoint is a mix of patterns that we've talked about already, so rather than repeating them in the main flow of the book, the instructions are included in this appendix.

dego_vila@yahoo.com 30 Oct 2026

Activating a user

In this chapter we're going to move on to the part of the activation workflow where we actually activate a user. But before we write any code, I'd like to quickly talk about the relationship between users and tokens in our system.

What we have is known in relational database terms as a *one-to-many relationship* — where one user may have many tokens, but a token can only belong to one user.

When you have a one-to-many relationship like this, you'll potentially want to execute queries against the relationship from two different sides. In our case, for example, we might want to either:

- Retrieve the user associated with a token.
- Retrieve all tokens associated with a user.

To implement these queries in your code, a clean and clear approach is to update your database models to include some additional methods, like this:

```
UserModel.GetForToken(token)   → Retrieve the user associated with a token
TokenModel.GetAllForUser(user) → Retrieve all tokens associated with a user
```

The nice thing about this approach is that the returned entities align with each model's main responsibility: the `UserModel` method is returning a user, and the `TokenModel` method is returning tokens.

Creating the activateUserHandler

Now that we've got a very high-level idea of how we're going to query the user ↔ token relationship in our database models, let's start to build up the code for activating a user.

In order to do this, we'll need to add a new `PUT /v1/users/activated` endpoint to our API:

dego_vila@yahoo.com 30 Oct 2026

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
GET	/v1/movies	listMoviesHandler	Show the details of all movies
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PATCH	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie
DELETE	/v1/movies/:id	deleteMovieHandler	Delete a specific movie
POST	/v1/users	registerUserHandler	Register a new user
PUT	**/v1/users/activated**	**activateUserHandler**	**Activate a specific user**

And the workflow will look like this:

1. The user submits the plaintext activation token (which they just received in their email) to the `PUT /v1/users/activated` endpoint.
2. We validate the plaintext token to check that it matches the expected format, sending the client an error message if necessary.
3. We then call the `UserModel.GetForToken()` method to retrieve the details of the user associated with the provided token. If there is no matching token found, or it has expired, we send the client an error message.
4. We activate the associated user by setting `Activated = true` for the user and update the user record in our database.
5. We delete all activation tokens for the user from the `tokens` table. We can do this using the `TokenModel.DeleteAllForUser()` method that we made earlier.
6. We send the updated user details in a JSON response.

Let's begin in our `cmd/api/users.go` file and create the new `activateUserHandler` to work through these steps:

File: cmd/api/users.go

```
package main

...

func (app *application) activateUserHandler(w http.ResponseWriter, r *http.Request) {
    // Parse the plaintext activation token from the request body.
    var input struct {
        TokenPlaintext string `json:"token"`
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Validate the plaintext token provided by the client.
    v := validator.New()

    if models.ValidateTokenPlaintext(v, input.TokenPlaintext); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Retrieve the details of the user associated with the token using the
    // GetForToken() method (which we will create in a minute). If no matching record
    // is found, then we let the client know that the token they provided is not valid.
    user, err := app.models.Users.GetForToken(models.ScopeActivation, input.TokenPlaintext)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            v.AddError("token", "invalid or expired activation token")
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Update the user's activation status.
    user.Activated = true

    // Save the updated user record in our database, checking for any edit conflicts in
    // the same way that we did for our movie records.
    user, err = app.models.Users.Update(user)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrEditConflict):
            app.editConflictResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // If everything went successfully, then we delete all activation tokens for the
    // user.
    err = app.models.Tokens.DeleteAllForUser(models.ScopeActivation, user.ID)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Send the updated user details to the client in a JSON response.
    err = app.writeJSON(w, http.StatusOK, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

If you try to compile the application at this point, you'll get an error because the `UserModel.GetForToken()` method doesn't yet exist. Let's go ahead and create that now.

The UserModel.GetForToken method

As we mentioned above, we want the `UserModel.GetForToken()` method to retrieve the details of the user associated with a particular activation token. If there is no matching token found, or it has expired, we want this to return a `ErrRecordNotFound` error instead.

In order to do that, we'll need to execute the following SQL query on our database:

```
SELECT users.id, users.created_at, users.name, users.email, users.password_hash, users.activated, users.version
FROM users
INNER JOIN tokens
ON users.id = tokens.user_id
WHERE tokens.hash = $1
AND tokens.scope = $2
AND tokens.expiry > $3
```

This is more complicated than most of the SQL queries we've used so far, so let's take a moment to explain what it is doing.

In this query we are using `INNER JOIN` to join together information from the `users` and `tokens` tables. Specifically, we're using the `ON users.id = tokens.user_id` clause to specify that we want to join records *where the user* `id` *value equals the token* `user_id`.

Behind the scenes, you can think of `INNER JOIN` as creating an 'interim' table containing the joined data from *both* tables. Then, in our SQL query, we use the `WHERE` clause to filter this interim table to leave only rows where the token hash and token scope match specific placeholder parameter values, and the token expiry is after a specific time. Because the token hash is also a primary key, we will always be left with exactly one record which contains the details of the user associated with the token hash (or no records at all, if there wasn't a matching token).

> **Hint:** If you're not familiar with performing joins in SQL, then this article provides a good overview of the different types of joins, how they work, and some examples that should help assist your understanding.

If you're following along, open up your `internal/models/users.go` file and add a `GetForToken()` method which executes this SQL query like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/users.go

```
package models

import (
    "context"
    "crypto/sha256" // New import
    "database/sql"
    "errors"
    "time"

    "greenlight.alexedwards.net/internal/validator"

    "github.com/lib/pq"
    "golang.org/x/crypto/bcrypt"
)

...

func (m UserModel) GetForToken(tokenScope, tokenPlaintext string) (User, error) {
    // Calculate the SHA-256 hash of the plaintext token provided by the client.
    // Remember that this returns a byte *array* with length 32, not a slice.
    tokenHash := sha256.Sum256([]byte(tokenPlaintext))

    // Set up the SQL query.
    query := `
        SELECT users.id, users.created_at, users.name, users.email, users.password_hash, users.activated, users.version
        FROM users
        INNER JOIN tokens
        ON users.id = tokens.user_id
        WHERE tokens.hash = $1
        AND tokens.scope = $2
        AND tokens.expiry > $3`

    // Create a slice containing the query arguments. Notice how we use the [:] operator
    // to get a slice containing the token hash, rather than passing in the array (which
    // is not supported by the pq driver), and that we pass the current time as the
    // value to check against the token expiry.
    args := []any{tokenHash[:], tokenScope, time.Now()}

    var user User

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    // Execute the query, scanning the return values into a User struct. If no matching
    // record is found we return an ErrRecordNotFound error.
    err := m.DB.QueryRowContext(ctx, query, args...).Scan(
        &user.ID,
        &user.CreatedAt,
        &user.Name,
        &user.Email,
        &user.Password.hash,
        &user.Activated,
        &user.Version,
    )
    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return User{}, ErrRecordNotFound
        default:
            return User{}, err
        }
    }

    // Return the matching user
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Return the matching user.
    return user, nil
}
```

Now that this is in place, the final thing we need to do is add the `PUT /v1/users/activated` endpoint to the `cmd/api/routes.go` file. Like so:

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.listMoviesHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    // Add the route for the PUT /v1/users/activated endpoint.
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    return app.recoverPanic(app.rateLimit(router))
}
```

As an aside, I should quickly explain that the reason we're using `PUT` rather than `POST` for this endpoint is because it's *idempotent*.

If a client sends the same `PUT /v1/users/activated` request multiple times, the first will succeed (assuming the token is valid) and then any subsequent requests will result in an error being sent to the client (because the token has been used and deleted from the database). But the important thing is that *nothing in our application state (i.e. database) changes after that first request*.

Basically, there are no application state side-effects from the client sending the same request multiple times, which means that the endpoint is idempotent and using `PUT` is more appropriate than `POST`.

Alright, let's restart the API and then try this out.

First, try making some requests to the `PUT /v1/users/activated` endpoint containing some invalid tokens. You should get the appropriate error messages in response, like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -X PUT -d '{"token": "invalid"}' localhost:4000/v1/users/activated
{
    "error": {
        "token": "must be 26 bytes long"
    }
}

$ curl -X PUT -d '{"token": "ABCDEFGHIJKLMNOPQRSTUVWXYZ"}' localhost:4000/v1/users/activated
{
    "error": {
        "token": "invalid or expired activation token"
    }
}
```

Then try making a request using a valid activation token from one of your emails (which will be in your Mailtrap inbox if you're following along). In my case, I'll use the token `P4B3URJZJ2NW5UPZC2OHN4H2NM` to activate the user `faith@example.com` (who we created in the previous chapter).

You should get a JSON response back with an `activated` field that confirms that the user has been activated, similar to this:

```
$ curl -X PUT -d '{"token": "P4B3URJZJ2NW5UPZC2OHN4H2NM"}' localhost:4000/v1/users/activated
{
    "user": {
        "id": 7,
        "created_at": "2026-09-13T11:27:07+02:00",
        "name": "Faith Smith",
        "email": "faith@example.com",
        "activated": true
    }
}
```

And if you try repeating the request again with the same token, you should now get an `"invalid or expired activation token"` error due to the fact we have deleted all activation tokens for `faith@example.com`.

```
$ curl -X PUT -d '{"token": "P4B3URJZJ2NW5UPZC2OHN4H2NM"}' localhost:4000/v1/users/activated
{
    "error": {
        token": "invalid or expired activation token"
    }
}
```

> **Important:** In production, with activation tokens for real accounts, you must make sure that the tokens are only ever accepted over an encrypted HTTPS connection — not via regular HTTP like we are using here.

dego_vila@yahoo.com 30 Oct 2026

Lastly, let's take a quick look in our database to see the state of our `users` table.

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT email, activated, version FROM users;
       email       | activated | version
-------------------+-----------+---------
 alice@example.com | f         |       1
 bob@example.com   | f         |       1
 carol@example.com | f         |       1
 dave@example.com  | f         |       1
 edith@example.com | f         |       1
 faith@example.com | t         |       2
```

In contrast to all the other users, we can see that `faith@example.com` has now got the value `activated = true` and the version number for their user record has been bumped up to `2`.

Additional information

Web application workflow

If your API is the backend to a website, rather than a completely standalone service, you can tweak the activation workflow to make it simpler and more intuitive for users while still being secure.

There are two main options here. The first, and most robust, option is to ask the user to copy-and-paste the token into a form on your website which then performs the `PUT /v1/users/activate` request for them using some JavaScript. The welcome email to support that workflow could look something like this:

dego_vila@yahoo.com 30 Oct 2026

```
Hi,

Thanks for signing up for a Greenlight account. We're excited to have you on board!

For future reference, your user ID number is 123.

To activate your Greenlight account please visit https://example.com/users/activate and
enter the following code:

--------------------------
RMMCV3MZCEBYQADXBODCLTAF6L
--------------------------

Please note that this code will expire in 3 days and can only be used once.

Thanks,

The Greenlight Team
```

This approach is fundamentally simple and secure — effectively your website just provides a form that performs the `PUT` request for the user, rather than them needing to do it manually using `curl` or another tool.

> **Note:** When creating the link in this email, don't rely on the `Host` header from `r.Host` to construct the URL, as that would be vulnerable to a host header injection attack. The URL domain should be either be hard-coded or passed in as a command-line flag when starting the application.

Alternatively, if you don't want the user to copy-and-paste a token, you could ask them to click a link containing the token which takes them to a page on your website. Similar to this:

```
Hi,

Thanks for signing up for a Greenlight account. We're excited to have you on board!

For future reference, your user ID number is 123.

To activate your Greenlight account please click the following link:

https://example.com/users/activate?token=RMMCV3MZCEBYQADXBODCLTAF6L

Please note that this link will expire in 3 days and can only be used once.

Thanks,

The Greenlight Team
```

This page should then display a button that says something like 'Confirm your account activation', and some JavaScript on the webpage can extract the token from the URL and submit it to your `PUT /v1/users/activate` API endpoint when the user clicks the button.

dego_vila@yahoo.com 30 Oct 2026

If you go with this second option, you also need to take steps to avoid the token being leaked in a referrer header if the user navigates to a different site. You can use the `Referrer-Policy: Origin` header or `<meta name="referrer" content="origin">` HTML tag to mitigate this, although you should be aware that it's not supported by absolutely all web browsers (support is currently at 97%).

In all cases though, whatever the email and workflow looks like in terms of the front-end and user-experience, the back-end API endpoint that we've implemented is the same and doesn't need to change.

SQL query timing attack

It's worth pointing out that the SQL query we're using in `UserModel.GetForToken()` is theoretically vulnerable to a timing attack, because PostgreSQL's evaluation of the `tokens.hash = $1` condition is not performed in constant-time.

```
SELECT users.id, users.created_at, users.name, users.email, users.password_hash, users.activated, users.version
FROM users
INNER JOIN tokens
ON users.id = tokens.user_id
WHERE tokens.hash = $1    --<-- This is vulnerable to a timing attack
AND tokens.scope = $2
AND tokens.expiry > $3
```

Although it would be very tricky to pull off, in theory an attacker could issue thousands of requests to our `PUT /v1/users/activated` endpoint and analyze tiny discrepancies in the average response time to build up a picture of a hashed activation token value in the database.

But, in our case, even if a timing attack was successful it would only leak the *hashed* token value from the database — not the plaintext token value that the user actually needs to submit to activate their account.

So the attacker would still need to use brute-force to find a 26-character string which happens to have the same SHA-256 hash that they discovered from the timing attack. This is incredibly difficult to do, and not viable with current technology.

dego_vila@yahoo.com 30 Oct 2026

Chapter 15.

Authentication

In this section of the book we're going to look at how to *authenticate* requests to our API, so that we know exactly which user a particular request is coming from.

> **Remember**: *Authentication* is about confirming *who a user is*, whereas *authorization* is about checking whether that user is permitted to do something.

We will:

- Lay out the possible approaches to API authentication and talk through their relative pros and cons.
- Implement a stateful token-based authentication pattern, which allows clients to exchange their user credentials for a time-limited authentication token identifying who they are.

dego_vila@yahoo.com 30 Oct 2026

Authentication options

Before we get started on writing any code, let's talk about *how* we're going to authenticate requests to our API and find out which user a request is coming from.

Choosing a high-level approach to API authentication can be tricky — there are many different options, and it's not always immediately clear which one is the right fit for your project. So in this chapter we'll discuss some of the most common approaches at a high level, talk through their relative pros and cons, and finish with some general guidelines about when they're appropriate to use.

Specifically, the five approaches that we'll compare are:

- Basic authentication
- Stateful token authentication
- Stateless token authentication
- API key authentication
- OAuth 2.0 / OpenID Connect

> **Important:** For all authentication methods that we're describing in this chapter, it's assumed that — in production environments — your API only communicates with clients over HTTPS.

HTTP basic authentication

Perhaps the simplest way to determine *who is making a request* to your API is to use HTTP basic authentication.

With this method, the client includes an `Authorization` header with every request containing their credentials. The credentials need to be in the format `username:password` and base64 encoded. So, for example, to authenticate as `alice@example.com:pa55word` the client would send the following header:

```
Authorization: Basic YWxpY2VAZXhhbXBsZS5jb206cGE1NXdvcmQ=
```

dego_vila@yahoo.com 30 Oct 2026

In your API, you can then extract the credentials from this header using Go's `Request.BasicAuth()` method, and verify that they're correct before continuing to process the request.

A big plus of HTTP basic authentication is how simple it is for clients. They can just send the same header with every request — and HTTP basic authentication is supported out-of-the-box by most programming languages, web browsers, and tools such as `curl` and `wget`.

It's often useful in the scenario where your API doesn't have 'real' user accounts, but you want a quick and easy way to restrict access to it or protect it from prying eyes.

For APIs with 'real' user accounts and — in particular — hashed passwords, it's not such a great fit. Comparing the password provided by a client against a (slow) hashed password is a deliberately costly operation, and when using HTTP basic authentication you need to do that check for every request. That will create a lot of extra work for your API server and add significant latency to responses.

But even then, basic authentication can still be a good choice if traffic to your API is very low and response speed is not important to you.

Token authentication

The high-level idea behind *token authentication* (also sometimes known as *bearer token authentication*) works like this:

1. The client sends a request to your API containing their credentials (typically username or email address, and password).

2. The API verifies that the credentials are correct, generates a *bearer token* which represents the user, and sends it back to the user. The token expires after a set period of time, after which the user will need to resubmit their credentials again to get a new token.

3. For subsequent requests to the API, the client includes the token in an `Authorization` header like this:

    ```
    Authorization: Bearer <token>
    ```

4. When your API receives this request, it checks that the token hasn't expired and examines the token value to determine who the user is.

dego_vila@yahoo.com 30 Oct 2026

For APIs where user passwords are hashed (like ours), this approach is better than basic authentication because it means that the slow password check only has to be done periodically — either when creating a token for the first time or after a token has expired.

The downside is that managing tokens can be complicated for clients — they will need to implement the necessary logic for caching tokens, monitoring and managing token expiry, and periodically generating new tokens.

We can break down token authentication further into two sub-types: *stateful* and *stateless* token authentication. They are quite different in terms of their pros and cons, so let's discuss them both separately.

Stateful token authentication

In a stateful token approach, the value of the token is a high-entropy cryptographically-secure random string. This token — or a fast hash of it — is stored server-side in a database, alongside the user ID and an expiry time for the token.

When the client sends back the token in subsequent requests, your application can look up the token in the database, check that it hasn't expired, and retrieve the corresponding user ID to find out who the request is coming from.

The big advantage of this is that your API maintains control over the tokens — it's straightforward to revoke tokens on a per-token or per-user basis by deleting them from the database or marking them as expired.

Conceptually it's also simple and robust — the security is provided by the token being 'unguessable', which is why it's important to use a high-entropy cryptographically-secure random value for the token.

So, what are the downsides?

Apart from the awkwardness for clients in having to manage tokens, it's difficult to find much to criticize about this approach. Perhaps the fact that it requires a database lookup is a negative — but in most cases you will need to make a database lookup to check the user's activation status or retrieve additional information about them *anyway*.

Stateless token authentication

In contrast, stateless tokens encode the user ID and expiry time *in the token itself*. The token is cryptographically signed to prevent tampering and (in some cases) encrypted to prevent the contents being read.

dego_vila@yahoo.com 30 Oct 2026

There are a few different technologies that you can use to create stateless tokens. Encoding the information in a JWT (JSON Web Token) is probably the most well-known approach, but PASETO and nacl/secretbox are viable alternatives too. Although the implementation details of these technologies are different, the overarching pros and cons in terms of authentication are similar.

The main selling point of using stateless tokens for authentication is that the work to encode and decode the token can be done in memory, and all the information required to identify the user is contained within the token itself. There's no need to perform a database lookup to find out who a request is coming from.

The primary downside of stateless tokens is that they can't easily be revoked once they are issued.

In an emergency, you could effectively revoke *all* tokens by changing the secret used for signing your tokens (forcing all users to reauthenticate), or another workaround is to maintain a blocklist of revoked tokens in a database (although that defeats the 'stateless' aspect of having stateless tokens).

> **Note:** You should generally avoid storing additional information in a stateless token, such as a user's activation status or permissions, and using that as the basis for *authorization* checks. During the lifetime of the token, the information encoded into it will potentially become stale and out-of-sync with the real data in your system — and relying on stale data for authorization checks can easily lead to unexpected behavior for users and various security issues.

Finally, with JWTs in particular, the fact that they're highly configurable means that there are *lots of things you can get wrong*. The Critical vulnerabilities in JSON Web Token libraries and JWT Security Best Practices articles provide a good introduction to the type of things you need to be careful of here.

Because of these downsides, stateless tokens — and JWTs in particular — are generally not the best choice for managing authentication in most API applications.

But they *can* be very useful in a scenario where you need *delegated authentication* — where the application *creating* the authentication token is different from the application *consuming* it, and those applications don't share any state (which means that using stateful tokens isn't an option). For instance, if you're building a service which has a microservice-style architecture, then a stateless token created by an 'authentication' service can subsequently be passed to other services to identify the user.

dego_vila@yahoo.com 30 Oct 2026

API-key authentication

The idea behind API-key authentication is that a user has a non-expiring secret 'key' associated with their account. This key should be a high-entropy cryptographically-secure random string, and a fast hash of the key (SHA256 or SHA512) should be stored alongside the corresponding user ID in your database.

The user then passes their key with each request to your API in a header like this:

```
Authorization: Key <key>
```

On receiving it, your API can regenerate the fast hash of the key and use it to look up the corresponding user ID from your database.

Conceptually, this isn't a million miles away from the stateful token approach — the main difference is that the keys are permanent keys, rather than temporary tokens.

On one hand, this is nice for the client as they can use the same key for every request and they don't need to write code to manage tokens or expiry. On the other hand, the user now has two long-lived secrets to manage which can potentially compromise their account: their password, and their API key.

Supporting API keys also adds additional complexity to your API application — you'll need a way for users to regenerate their API key if they lose it or the key is compromised, and you may also wish to support multiple API keys for the same user, so they can use different keys for different purposes.

It's also important to note that API keys themselves should only ever be communicated to users over a secure channel, and you should treat them with the same level of care that you would a user's password.

OAuth 2.0 / OpenID Connect

Another option is to leverage OAuth 2.0 for authentication. With this approach, information about your users (and their passwords) is stored by a third-party *identity provider* like Google or Facebook rather than yourself.

The first thing to mention here is that *OAuth 2.0 is not an authentication protocol*, and you shouldn't really use it for authenticating users. The `oauth.net` website has a great article explaining this, and I highly recommend reading it.

dego_vila@yahoo.com 30 Oct 2026

If you want to implement authentication checks against a third-party identity provider, you should use OpenID Connect (which is built directly on top of OAuth 2.0).

There's a comprehensive overview of OpenID Connect here, but at a very, very, high level it works like this:

- When you want to authenticate a request, you redirect the user to an 'authentication and consent' form hosted by the identity provider.
- If the user consents, then the identity provider sends your API an *authorization code*.
- Your API then sends the authorization code to another endpoint provided by the identity provider. They verify the authorization code, and if it's valid they will send you a JSON response containing an *ID token*.
- This ID token is itself a JWT. You need to validate and decode this JWT to get the actual user information, which includes things like their email address, name, birth date, timezone etc.
- Now that you know who the user is, you can then implement a stateful or stateless authentication token pattern so that you don't have to go through the whole process for every subsequent request.

Like all the other options we've looked at, there are pros and cons to using OpenID Connect. The big plus is that you don't need to persistently store user information or passwords yourself. The big downside is that it's quite complex — although there are some helper packages like `coreos/go-oidc` which do a good job of masking that complexity and providing a simple interface for the OpenID Connect workflow that you can hook in to.

It's also important to point out that using OpenID Connect requires all your users to have an account with the identity provider, and the 'authentication and consent' step requires human interaction via a web browser — which is probably fine if your API is the back-end for a website, but not ideal if it is a 'standalone' API with other computer programs as clients.

What authentication approach should I use?

It's difficult to give blanket guidance on what authentication approach is best to use for your API. As with most things in programming, different tools are appropriate for different jobs.

But as simple, rough, rules-of-thumb:

- If your API doesn't have 'real' user accounts with slow password hashes, then HTTP basic authentication can be a good — and often overlooked — fit.

dego_vila@yahoo.com 30 Oct 2026

- If you don't want to store user passwords yourself, all your users have accounts with a third-party identity provider that supports OpenID Connect, and your API is the back-end for a website… then use OpenID Connect.
- If you require delegated authentication, such as when your API has a microservice architecture with different services for performing authentication and performing other tasks, then use stateless authentication tokens.
- Otherwise use API keys or stateful authentication tokens. In general:
    - Stateful authentication tokens are a nice fit for APIs that act as the back-end for a website or single-page application, as there is a natural moment when the user logs in where they can be exchanged for user credentials.
    - In contrast, API keys can be better for more 'general purpose' APIs because they're permanent and simpler for developers to use in their applications and scripts.

In the rest of this book, we're going to implement authentication using the *stateful authentication token* pattern. In our case we've already built a lot of the necessary logic for this as part of our *activation tokens* work.

> **Note:** Although I *really* don't recommend using JWTs unless you need some form of *delegated authentication*, I'm aware that they hold a lot of mindshare in the developer community and are often used more widely than that.
>
> So, because of this, I've also included an appendix at the end of the book which explains how the *Greenlight* API can be adapted to use JWTs for stateless authentication.

dego_vila@yahoo.com 30 Oct 2026

Generating authentication tokens

In this chapter we're going to focus on building up the code for a new `POST/v1/tokens/authentication` endpoint, which will allow a client to exchange their credentials (email address and password) for a stateful authentication token.

> **Note:** For conciseness, rather than repeating the words 'stateful authentication token' throughout the rest of this book, from now on we'll simply refer to this as the user's *authentication token*.

At a high level, the process for exchanging a user's credentials for an authentication token will work like this:

1. The client sends a JSON request to a new `POST/v1/tokens/authentication` endpoint containing their credentials (email and password).
2. We will look up the user record based on the email, and check if the password provided is the correct one for the user. If it's not, then we send an error response.
3. If the password is correct, we'll use our `app.models.Tokens.New()` method to generate a token with an expiry time of 24 hours and the scope `"authentication"`.
4. We send this authentication token back to the client in a JSON response body.

Let's begin in our `internal/models/tokens.go` file.

We need to update this file to define a new `"authentication"` scope, and add some struct tags to customize how the `Token` struct appears when it is encoded to JSON. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/tokens.go

```
package models

...

const (
    ScopeActivation     = "activation"
    ScopeAuthentication = "authentication" // Include a new authentication scope.
)

// Add struct tags to control how the struct appears when encoded to JSON.
type Token struct {
    Plaintext string    `json:"token"`
    Hash      []byte    `json:"-"`
    UserID    int       `json:"-"`
    Expiry    time.Time `json:"expiry"`
    Scope     string    `json:"-"`
}

...
```

These new struct tags mean that only the `Plaintext` and `Expiry` fields will be included when encoding a `Token` struct — all the other fields will be omitted. We also rename the `Plaintext` field to `"token"`, just because it's a more meaningful name for clients than 'plaintext' is.

Altogether, this means that when we encode a `Token` struct to JSON the result will look similar to this:

```
{
    "token": "X3ASTT2CDAN66BACKSCI4SU7SI",
    "expiry": "2021-01-18T13:00:25.648511827+01:00"
}
```

Building the endpoint

Now let's get into the meat of this chapter and set up all the code for the new `POST/v1/tokens/authentication` endpoint. By the time we're finished, our API routes will look like this:

dego_vila@yahoo.com 30 Oct 2026

Method	URL Pattern	Handler	Action
GET	/v1/healthcheck	healthcheckHandler	Show application information
GET	/v1/movies	listMoviesHandler	Show the details of all movies
POST	/v1/movies	createMovieHandler	Create a new movie
GET	/v1/movies/:id	showMovieHandler	Show the details of a specific movie
PATCH	/v1/movies/:id	updateMovieHandler	Update the details of a specific movie
DELETE	/v1/movies/:id	deleteMovieHandler	Delete a specific movie
POST	/v1/users	registerUserHandler	Register a new user
PUT	/v1/users/activated	activateUserHandler	Activate a specific user
POST	**/v1/tokens/authentication**	**createAuthenticationTokenHandler**	**Generate a new authentication token**

If you're following along, go ahead and create a new `cmd/api/tokens.go` file:

```
$ touch cmd/api/tokens.go
```

And in this new file we'll add the code for the `createAuthenticationTokenHandler`.

Essentially, we want this handler to *exchange the user's email address and password for an authentication token*, like so:

File: cmd/api/tokens.go

```
package main

import (
    "errors"
    "net/http"
    "time"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"
)

func (app *application) createAuthenticationTokenHandler(w http.ResponseWriter, r *http.Request) {
    // Parse the email and password from the request body.
    var input struct {
```

dego_vila@yahoo.com 30 Oct 2026

```
    Email    string `json:"email"`
    Password string `json:"password"`
}

err := app.readJSON(w, r, &input)
if err != nil {
    var badRequestErr *badRequestError
    switch {
    case errors.As(err, &badRequestErr):
        app.badRequestResponse(w, r, err)
    default:
        app.serverErrorResponse(w, r, err)
    }
    return
}

// Validate the email and password provided by the client.
v := validator.New()

models.ValidateEmail(v, input.Email)
models.ValidatePasswordPlaintext(v, input.Password)

if !v.Valid() {
    app.failedValidationResponse(w, r, v.Errors)
    return
}

// Lookup the user record based on the email address. If no matching user was
// found, then we call the app.invalidCredentialsResponse() helper to send a 401
// Unauthorized response to the client (we will create this helper in a moment).
user, err := app.models.Users.GetByEmail(input.Email)
if err != nil {
    switch {
    case errors.Is(err, models.ErrRecordNotFound):
        app.invalidCredentialsResponse(w, r)
    default:
        app.serverErrorResponse(w, r, err)
    }
    return
}

// Check if the provided password matches the actual password for the user.
match, err := user.Password.Matches(input.Password)
if err != nil {
    app.serverErrorResponse(w, r, err)
    return
}

// If the passwords don't match, then we call the app.invalidCredentialsResponse()
// helper again and return.
if !match {
    app.invalidCredentialsResponse(w, r)
    return
}

// Otherwise, if the password is correct, we generate a new token with a 24-hour
// expiry time and the scope 'authentication'.
token, err := app.models.Tokens.New(user.ID, 24*time.Hour, models.ScopeAuthentication)
if err != nil {
    app.serverErrorResponse(w, r, err)
    return
}

// Encode the token to JSON and send it in the response along with a 201 Created
// status code.
err = app.writeJSON(w, http.StatusCreated, envelope{"authentication_token": token}, nil)
```

dego_vila@yahoo.com 30 Oct 2026

```
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

Let's quickly create the `invalidCredentialsResponse()` helper in our `cmd/api/errors.go` file too:

File: cmd/api/errors.go

```
package main

...

func (app *application) invalidCredentialsResponse(w http.ResponseWriter, r *http.Request) {
    message := "invalid authentication credentials"
    app.errorResponse(w, r, http.StatusUnauthorized, message)
}
```

Then lastly, we need to include the `POST /v1/tokens/authentication` endpoint in our application routes.

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.listMoviesHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    // Add the route for the POST /v1/tokens/authentication endpoint.
    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    return app.recoverPanic(app.rateLimit(router))
}
```

With all that complete, we should now be able to generate an authentication token.

Go ahead and make a request to the new `POST /v1/tokens/authentication` endpoint with a

dego_vila@yahoo.com 30 Oct 2026

valid email address and password for one of the users that you've previously created. You should get a `201 Created` response and a JSON body containing an authentication token, similar to this:

```
$ BODY='{"email": "alice@example.com", "password": "pa55word"}'
$ curl -i -d "$BODY" localhost:4000/v1/tokens/authentication
HTTP/1.1 201 Created
Content-Type: application/json
Date: Sun, 13 Sep 2026 10:25:45 GMT
Content-Length: 125

{
    "authentication_token": {
        "token": "IEYZQUBEMPPAKPOAWTPV6YJ6RM",
        "expiry": "2026-09-14T12:25:45.987655189+02:00"
    }
}
```

In contrast, if you try making a request with a well-formed but unknown email address, or an incorrect password, you should get an error response. For example:

```
$ BODY='{"email": "alice@example.com", "password": "wrong pa55word"}'
$ curl -i -d "$BODY" localhost:4000/v1/tokens/authentication
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Date: Sun, 13 Sep 2026 10:27:09 GMT
Content-Length: 51

{
    "error": "invalid authentication credentials"
}
```

Before we continue, let's quickly have a look at the `tokens` table in our PostgreSQL database to check that the authentication token has been created.

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT * FROM tokens WHERE scope = 'authentication';
                              hash                              | user_id |         expiry         |     scope
----------------------------------------------------------------+---------+------------------------+----------------
\x4390d2ff4af7346dd4238ffccb8a5b18e8c3af9aa8cf57852895ad0f8ee2c50d |       1 | 2026-09-14 12:25:46+02 | authentication
```

That's looking good. We can see that the token is associated with the user with ID `1` (which if you've been following along will be the user `alice@example.com`) and has the correct scope and expiry time.

dego_vila@yahoo.com 30 Oct 2026

Additional information

The Authorization header

Occasionally you might come across other APIs or tutorials where authentication tokens are sent back to the client in an `Authorization` header, rather than in the response body like we are in this chapter.

You *can* do that, and in most cases it will probably work fine. But it's important to be conscious that you're making a willful violation of the HTTP specifications: `Authorization` is a *request* header, not a response header.

dego_vila@yahoo.com 30 Oct 2026

Authenticating requests

Now that our clients have a way to exchange their credentials for an authentication token, let's look at how we can use that token to *authenticate them*, so we know exactly which user a request is coming from.

Essentially, once a client has an authentication token we will expect them to include it with all subsequent requests in an `Authorization` header, like so:

```
Authorization: Bearer IEYZQUBEMPPAKPOAWTPV6YJ6RM
```

When we receive these requests, we'll use a new `authenticate()` middleware method to execute the following logic:

- If no `Authorization` header was provided at all, then we will call the next handler in the chain as normal with no further action.
- If the `Authorization` header is provided but the token is not valid, then we will send the client a `401 Unauthorized` response and an error message to let them know that their token is malformed or invalid.
- If the `Authorization` header is provided and the token is valid, then we know that the user *has authenticated*. We look up the user's details from the database, store these details in the *request context*, and then call the next handler in the chain.

Reading and writing to the request context

The other setup step, before we get into creating the `authenticate()` middleware itself, relates to *storing the authenticated user's details in the request context*.

We discussed what request context is and how to use it in detail in *Let's Go*, so if any of this feels unfamiliar then I recommend rereading that section of the book before continuing. But as a quick reminder:

- Every `http.Request` that our application processes has a `context.Context` embedded in it, which we can use to store key-value pairs containing arbitrary data during the lifetime of the request. In this case we want to store a `User` struct containing the authenticated user's information.

dego_vila@yahoo.com 30 Oct 2026

- Any values stored in the request context have the type `any`. This means that after retrieving a value from the request context you need to assert it back to its original type before using it.
- It's good practice to use your own custom type for the request context keys. This helps prevent naming collisions between your code and any third-party packages which are also using the request context to store information.

To help with this, let's create a new `cmd/api/context.go` file containing some helper methods for reading/writing the `User` struct to and from the request context.

If you're following along, go ahead and create the new file:

```
$ touch cmd/api/context.go
```

And then add the following code:

File: cmd/api/context.go

```
package main

import (
    "context"
    "net/http"

    "greenlight.alexedwards.net/internal/models"
)

// Define a custom contextKey type, with the underlying type string.
type contextKey string

// Convert the string "authenticatedUser" to a contextKey type and assign it to the
// authenticatedUserContextKey constant. We'll use this constant as the key for getting
// and setting authenticated user information in the request context.
const authenticatedUserContextKey = contextKey("authenticatedUser")

// The contextSetAuthenticatedUser() method returns a new copy of the request with the
// provided User struct added to the context. Note that we use our
// authenticatedUserContextKey constant as the key.
func (app *application) contextSetAuthenticatedUser(r *http.Request, user models.User) *http.Request {
    ctx := context.WithValue(r.Context(), authenticatedUserContextKey, user)
    return r.WithContext(ctx)
}

// The contextGetAuthenticatedUser() method retrieves the User struct from the request
// context. If the context does not contain a valid User struct under the
// authenticatedUserContextKey key, then the type assertion will fail and the value of
// the `ok` variable will be false.
func (app *application) contextGetAuthenticatedUser(r *http.Request) (models.User, bool) {
    user, ok := r.Context().Value(authenticatedUserContextKey).(models.User)
    return user, ok
}
```

dego_vila@yahoo.com 30 Oct 2026

Creating the authentication middleware

Now that we've got those things in place, we're ready to start work on our `authenticate()` middleware itself.

Open up your `cmd/api/middleware.go` file, and add the following code:

File: cmd/api/middleware.go

```
package main

import (
    "errors" // New import
    "fmt"
    "net/http"
    "strings" // New import
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/models"    // New import
    "greenlight.alexedwards.net/internal/validator" // New import

    "github.com/tomasen/realip"
    "golang.org/x/time/rate"
)

...

func (app *application) authenticate(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Add the "Vary: Authorization" header to the response. This indicates to any
        // caches that the response may vary based on the value of the Authorization
        // header in the request.
        w.Header().Add("Vary", "Authorization")

        // Retrieve the value of the Authorization header from the request. This will
        // return the empty string "" if there is no such header found.
        authorizationHeader := r.Header.Get("Authorization")

        // If there is no Authorization header found, call the next handler in the chain
        // as normal and then return without executing any of the code below.
        if authorizationHeader == "" {
            next.ServeHTTP(w, r)
            return
        }

        // Otherwise, we expect the value of the Authorization header to be in the format
        // "Bearer <token>". We try to split this into its constituent parts, and if the
        // header isn't in the expected format we return a 401 Unauthorized response
        // using the invalidAuthenticationTokenResponse() helper (which we will create
        // in a moment).
        headerParts := strings.Split(authorizationHeader, " ")
        if len(headerParts) != 2 || headerParts[0] != "Bearer" {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        // Extract the actual authentication token from the header parts.
        token := headerParts[1]

        // Validate the token to make sure it is in a sensible format.
```

dego_vila@yahoo.com 30 Oct 2026

```
        v := validator.New()

        // If the token isn't valid, use the invalidAuthenticationTokenResponse()
        // helper to send a response, rather than the failedValidationResponse() helper
        // that we'd normally use.
        if models.ValidateTokenPlaintext(v, token); !v.Valid() {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        // Retrieve the details of the user associated with the authentication token,
        // again calling the invalidAuthenticationTokenResponse() helper if no
        // matching record was found. IMPORTANT: Notice that we are using
        // ScopeAuthentication as the first parameter here.
        user, err := app.models.Users.GetForToken(models.ScopeAuthentication, token)
        if err != nil {
            switch {
            case errors.Is(err, models.ErrRecordNotFound):
                app.invalidAuthenticationTokenResponse(w, r)
            default:
                app.serverErrorResponse(w, r, err)
            }
            return
        }

        // At this point, we know that the current user has authenticated using
        // a valid token, and we add their user information to the request context
        // with the contextSetAuthenticatedUser() helper that we just made.
        r = app.contextSetAuthenticatedUser(r, user)

        // Call the next handler in the chain.
        next.ServeHTTP(w, r)
    })
}
```

There's quite a lot of code there, but the key thing take away is this: After this middleware is called, if the request is coming from an authenticated user with a valid token, *then the request context will contain the user's details*.

We can then access those authenticated user's details at a later point by simply calling the `contextGetAuthenticatedUser()` helper function.

OK, let's start finishing this up by heading to our `cmd/api/errors.go` file and creating the `invalidAuthenticationTokenResponse()` helper.

File: cmd/api/errors.go

```
package main

...

func (app *application) invalidAuthenticationTokenResponse(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("WWW-Authenticate", "Bearer")

    message := "invalid or missing authentication token"
    app.errorResponse(w, r, http.StatusUnauthorized, message)
}
```

dego_vila@yahoo.com 30 Oct 2026

> **Note:** We're including a `WWW-Authenticate: Bearer` header here to help inform or remind the client that we expect them to authenticate using a bearer token.

Finally, we need to add the `authenticate()` middleware to our handler chain. We want to use this middleware on *all* requests — after our panic recovery and rate limiter middleware, but before our router.

Go ahead and update the `cmd/api/routes.go` file accordingly:

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.listMoviesHandler)
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.createMovieHandler)
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.showMovieHandler)
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.updateMovieHandler)
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.deleteMovieHandler)

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    // Use the authenticate() middleware on all requests.
    return app.recoverPanic(app.rateLimit(app.authenticate(router)))
}
```

Demonstration

Let's test this out by first making a request with *no* `Authorization` header. You should find that the request still completes successfully. Like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl localhost:4000/v1/healthcheck
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
```

Then let's try the same thing, but with a valid authentication token in the `Authorization` header. This time, the relevant authenticated user details should be added to the request context, and you should again get a successful response. For example:

```
$ curl -d '{"email": "alice@example.com", "password": "pa55word"}' localhost:4000/v1/tokens/authentication
{
    "authentication_token": {
        "token": "FXCZM44TVLC6ML2NXTOW5OHFUE",
        "expiry": "2026-09-14T12:37:52.482832815+02:00"
    }
}

$ curl -H "Authorization: Bearer FXCZM44TVLC6ML2NXTOW5OHFUE" localhost:4000/v1/healthcheck
{
    "status": "available",
    "system_info": {
        "environment": "development",
        "version": "1.0.0"
    }
}
```

> **Hint:** If you get an error response here, make sure that you're using the correct authentication token from the first response in the second request.

In contrast, you can also try sending some requests with an invalid authentication token or a malformed `Authorization` header. In these cases you should get a `401 Unauthorized` response, like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -i -H "Authorization: Bearer XXXXXXXXXXXXXXXXXXXXXXXXXX" localhost:4000/v1/healthcheck
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Vary: Authorization
Www-Authenticate: Bearer
Date: Sun, 13 Sep 2026 10:39:45 GMT
Content-Length: 56

{
    "error": "invalid or missing authentication token"
}


$ curl -i -H "Authorization: INVALID" localhost:4000/v1/healthcheck
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Vary: Authorization
Www-Authenticate: Bearer
Date: Sun, 13 Sep 2026 10:40:10 GMT
Content-Length: 56

{
    "error": "invalid or missing authentication token"
}
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 16.

Permission-based authorization

By the time a request leaves our `authenticate()` middleware, there are now two possible states for the request context. Either:

- The request context contains a `User` struct under the `authenticatedUserContextKey` key — containing the details of the authenticated user who sent the request.
- Or the request context does not contain anything under the `authenticatedUserContextKey` key, in which case we know the request is *not* coming from an authenticated user.

In this section of the book, we're going to take this to the next natural stage and look at how to perform different *authorization* checks to restrict access to our API endpoints. Specifically, you'll learn how to:

- Add checks so that only *authenticated and activated users* are able to access the various `/v1/movies**` endpoints.
- Implement a *permission-based authorization* pattern, which provides fine-grained control over exactly *which users can access which endpoints*.

dego_vila@yahoo.com 30 Oct 2026

Requiring user activation

As we mentioned a moment ago, the first thing we're going to do in terms of authorization is restrict access to our `/v1/movies**` endpoints — so that they can only be accessed by users who are successfully authenticated, and who have activated their account.

Carrying out these kinds of checks is an ideal task for some middleware, so let's jump in and make a new `requireActivatedUser()` middleware method to handle this. In this middleware, we want to:

- Lookup the authenticated user's details from the request context by calling the `contextGetAuthenticatedUser()` helper. If this helper does not return any user details, we know the request is *not* coming from an authenticated user and we should send a `401 Unauthorized` response and an error message saying "`you must be authenticated to access this resource`".
- Otherwise, if the request is coming from an authenticated user, we want to check the user details to see if their account has been activated. If they are *not activated* we should send a `403 Forbidden` response and an error message saying "`your user account must be activated to access this resource`".

> **Hint:** A `401 Unauthorized` response should be used when you have missing or bad authentication, and a `403 Forbidden` response should be used afterwards, when the user is authenticated but isn't allowed to perform the requested operation.

So first, let's head to our `cmd/api/errors.go` file and add a couple of new helpers for sending those error messages. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/errors.go

```
package main

...

func (app *application) authenticationRequiredResponse(w http.ResponseWriter, r *http.Request) {
    message := "you must be authenticated to access this resource"
    app.errorResponse(w, r, http.StatusUnauthorized, message)
}

func (app *application) inactiveAccountResponse(w http.ResponseWriter, r *http.Request) {
    message := "your user account must be activated to access this resource"
    app.errorResponse(w, r, http.StatusForbidden, message)
}
```

And then let's create the new `requireActivatedUser()` middleware for carrying out the checks. The code we need is nice and succinct:

File: cmd/api/middleware.go

```
package main

...

func (app *application) requireActivatedUser(next http.HandlerFunc) http.HandlerFunc {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Use the contextGetAuthenticatedUser() helper that we made earlier to retrieve
        // the authenticated user information from the request context. If there
        // is no such information in the request context, the found variable will
        // be false and we call the authenticationRequiredResponse() method to inform
        // the client that they should authenticate before trying again.
        authenticatedUser, found := app.contextGetAuthenticatedUser(r)
        if !found {
            app.authenticationRequiredResponse(w, r)
            return
        }

        // If the user is not activated, use the inactiveAccountResponse() helper to
        // inform them that they need to activate their account.
        if !authenticatedUser.Activated {
            app.inactiveAccountResponse(w, r)
            return
        }

        // Call the next handler in the chain.
        next.ServeHTTP(w, r)
    })
}
```

Notice here that our `requireActivatedUser()` middleware has a slightly different signature to the other middleware we've built in this book. Instead of accepting and returning an `http.Handler`, it accepts and returns an `http.HandlerFunc`.

This is a small change, but it makes it possible to wrap our `/v1/movie**` handler functions directly with this middleware, without needing to make any further conversions.

dego_vila@yahoo.com 30 Oct 2026

Go ahead and update the `cmd/api/routes.go` file to do exactly that, as follows:

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    // Use the requireActivatedUser() middleware on our five /v1/movies** endpoints.
    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requireActivatedUser(app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requireActivatedUser(app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requireActivatedUser(app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requireActivatedUser(app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requireActivatedUser(app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    return app.recoverPanic(app.rateLimit(app.authenticate(router)))
}
```

Demonstration

Alright, let's try out these changes!

We'll begin by calling the `GET /v1/movies/:id` endpoint without an authentication token. When doing this you should now receive a `401 Unauthorized` response, like so:

```
$ curl -i localhost:4000/v1/movies/1
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Vary: Authorization
Date: Sun, 13 Sep 2026 10:54:16 GMT
Content-Length: 66

{
    "error": "you must be authenticated to access this resource"
}
```

Next, let's try making a request as a user who has an account, but has not yet activated. If you've been following along, you should be able to use the `alice@example.com` user to do this, like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"email": "alice@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/tokens/authentication
{
    "authentication_token": {
        "token": "2O4YHHWDHVVWWDNKN2UZR722BU",
        "expiry": "2026-09-14T12:55:09.19901084+02:00"
    }
}

$ curl -i -H "Authorization: Bearer 2O4YHHWDHVVWWDNKN2UZR722BU" localhost:4000/v1/movies/1
HTTP/1.1 403 Forbidden
Content-Type: application/json
Vary: Authorization
Date: Sun, 13 Sep 2026 10:55:49 GMT
Content-Length: 76

{
    "error": "your user account must be activated to access this resource"
}
```

Great, we can see that this now results in a `403 Forbidden` response from our new `inactiveAccountResponse()` helper.

Finally, let's try making a request as an activated user.

If you're coding along, you might like to quickly connect to your PostgreSQL database and double-check which users are already activated.

```
$ psql $GREENLIGHT_DB_DSN
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT email FROM users WHERE activated = true;
       email
-------------------
 faith@example.com
(1 row)
```

In my case the only activated user is `faith@example.com`, so let's try making a request as them. When making a request as an activated user, all the checks in our `requireActivatedUser()` middleware will pass and you should get a successful response. Similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"email": "faith@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/tokens/authentication
{
    "authentication_token": {
        "token": "ZFIKQ344EYM5KEP6JL2RHLRPJQ",
        "expiry": "2026-09-14T12:57:03.631156329+02:00"
    }
}

$ curl -H "Authorization: Bearer ZFIKQ344EYM5KEP6JL2RHLRPJQ" localhost:4000/v1/movies/1
{
    "movie": {
        "id": 1,
        "title": "Moana",
        "year": 2016,
        "runtime": "107m",
        "genres": [
            "animation",
            "adventure"
        ],
        "version": 1
    }
}
```

Additional information

In-handler checks

If you only have a couple of endpoints where you want to perform authorization checks, then rather than using middleware it can often be easier to do the checks inside the relevant handlers instead. For example:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    authenticatedUser, found := app.contextGetAuthenticatedUser(r)
    if !found {
        app.authenticationRequiredResponse(w, r)
        return
    }

    if !authenticatedUser.Activated {
        app.inactiveAccountResponse(w, r)
        return
    }

    // The rest of the handler logic goes here...
}
```

dego_vila@yahoo.com 30 Oct 2026

Setting up the permissions database table

Restricting our API so that movie data can only be accessed and edited by activated users is useful, but sometimes you might need a more granular level of control. For example, in our case we might be happy for 'regular' users of our API to *read* the movie data (so long as they are activated), but we want to restrict *write* access to a smaller subset of trusted users.

In this chapter we're going to introduce the concept of *permissions* to our application, so that only users with a specific permission can perform specific operations. In our case, we'll create two permissions: a `movies:read` permission which will allow a user to fetch and filter movies, and a `movies:write` permission which will allow users to create, edit and delete movies.

The required permissions will align with our API endpoints like so:

Method	URL Pattern	Required permission
GET	/v1/healthcheck	—
GET	/v1/movies	movies:read
POST	/v1/movies	movies:write
GET	/v1/movies/:id	movies:read
PATCH	/v1/movies/:id	movies:write
DELETE	/v1/movies/:id	movies:write
POST	/v1/users	—
PUT	/v1/users/activated	—
POST	/v1/tokens/authentication	—

Relationship between permissions and users

The relationship between permissions and users is a great example of a *many-to-many* relationship. One user may have *many permissions*, and the same permission may belong to

dego_vila@yahoo.com 30 Oct 2026

many users.

The classic way to manage a many-to-many relationship in a relational database like PostgreSQL is to create a *joining table* between the two entities. I'll quickly explain how this works.

Let's say that we are storing our user data in a `users` table which looks like this:

id	email	…
1	alice@example.com	…
2	bob@example.com	…

And our permissions data is stored in a `permissions` table like this:

id	code
1	movies:read
2	movies:write

Then we can create a joining table called `users_permissions` to store the information about *which users have which permissions*, similar to this:

user_id	permission_id
1	1
2	1
2	2

In the example above, the user `alice@example.com` (user ID `1`) has the `movies:read` (permission ID `1`) permission only, whereas `bob@example.com` (user ID `2`) has both the `movies:read` and `movies:write` permissions.

Just like the one-to-many relationship that we looked at earlier in the book, you may want to query this relationship from both sides in your database models. For example, in your database models you might want to create the following methods:

dego_vila@yahoo.com 30 Oct 2026

```
PermissionModel.GetAllForUser(user)       → Retrieve all permissions for a user
UserModel.GetAllForPermission(permission) → Retrieve all users with a specific permission
```

Creating the SQL migrations

Let's put this into practice and make a SQL migration which creates new `permissions` and `users_permissions` tables in our database, following the pattern that we've just described above.

Go ahead and run the following command to create the migration files:

```
$ migrate create -seq -ext .sql -dir ./migrations add_permissions
/home/alex/Projects/greenlight/migrations/000006_add_permissions.up.sql
/home/alex/Projects/greenlight/migrations/000006_add_permissions.down.sql
```

And then add the following SQL statements to the 'up' migration file:

File: migrations/000006_add_permissions.up.sql

```
CREATE TABLE IF NOT EXISTS permissions (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    code text NOT NULL
);

CREATE TABLE IF NOT EXISTS users_permissions (
    user_id bigint NOT NULL REFERENCES users ON DELETE CASCADE,
    permission_id bigint NOT NULL REFERENCES permissions ON DELETE CASCADE,
    PRIMARY KEY (user_id, permission_id)
);

-- Add the two permissions to the table.
INSERT INTO permissions (code)
VALUES
    ('movies:read'),
    ('movies:write');
```

There are a couple of important things to point out here:

- The `PRIMARY KEY (user_id, permission_id)` line sets a *composite primary key* on our `users_permissions` table, where the primary key is made up of both the `user_id` and `permission_id` columns. Setting this as the primary key essentially means that the same user/permission combination can only appear once in the table and cannot be duplicated.

- When creating the `users_permissions` table we use the `REFERENCES users` syntax to create a foreign key constraint against the primary key of our `users` table, which ensures

dego_vila@yahoo.com 30 Oct 2026

that any value in the `user_id` column has a corresponding entry in our `users` table. And likewise, we use the `REFERENCES permissions` syntax to ensure that the `permission_id` column has a corresponding entry in the `permissions` table.

Let's also add the necessary `DROP TABLE` statements to the 'down' migration file, like so:

File: migrations/000006_add_permissions.down.sql

```
DROP TABLE IF EXISTS users_permissions;
DROP TABLE IF EXISTS permissions;
```

Now that's done, please go ahead and run the migration:

```
$ migrate -path ./migrations -database $GREENLIGHT_DB_DSN up
6/u add_permissions (22.74009ms)
```

dego_vila@yahoo.com 30 Oct 2026

Setting up the permissions model

Next, let's head to our `internal/models` package and add a `PermissionModel` to manage the interactions with our new tables. For now, the only thing we want to include in this model is a `GetAllForUser()` method to *return all permission codes for a specific user*. The idea is that we'll be able to use this in our handlers and middleware like so:

```
// Return a slice of the permission codes for the user with ID = 1. This would return
// something like []string{"movies:read", "movies:write"}.
app.models.Permissions.GetAllForUser(1)
```

Behind the scenes, the SQL statement that we need to fetch the permission codes for a specific user looks like this:

```
SELECT permissions.code
FROM permissions
INNER JOIN users_permissions ON users_permissions.permission_id = permissions.id
INNER JOIN users ON users_permissions.user_id = users.id
WHERE users.id = $1
```

In this query we are using the `INNER JOIN` clause to join our `permissions` table to our `users_permissions` table, and then using it again to join *that* to the `users` table. Then we use the `WHERE` clause to filter the result, leaving only rows which relate to a specific user ID.

Let's go ahead and set up the new `PermissionModel`. First create a new `internal/models/permissions.go` file:

```
$ touch internal/models/permissions.go
```

And then add the following code:

File: internal/models/permissions.go

```
package models

import (
    "context"
    "database/sql"
    "slices"
    "time"
)

// Define a Permissions slice, which we will use to hold the permission codes (such as
```

dego_vila@yahoo.com 30 Oct 2026

```
// Define a Permissions slice, which we will use to hold the permission codes (such as
// "movies:read" and "movies:write") for a single user.
type Permissions []string

// Add a helper method to check whether the Permissions slice contains a specific
// permission code.
func (p Permissions) Include(code string) bool {
    return slices.Contains(p, code)
}

// Define the PermissionModel type.
type PermissionModel struct {
    DB *sql.DB
}

// The GetAllForUser() method returns all permission codes for a specific user in a
// Permissions slice. The code in this method should feel very familiar --- it uses the
// standard pattern that we've already seen before for retrieving multiple data rows in
// an SQL query.
func (m PermissionModel) GetAllForUser(userID int) (Permissions, error) {
    query := `
        SELECT permissions.code
        FROM permissions
        INNER JOIN users_permissions ON users_permissions.permission_id = permissions.id
        INNER JOIN users ON users_permissions.user_id = users.id
        WHERE users.id = $1`

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    rows, err := m.DB.QueryContext(ctx, query, userID)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var permissions Permissions

    for rows.Next() {
        var permission string

        err := rows.Scan(&permission)
        if err != nil {
            return nil, err
        }

        permissions = append(permissions, permission)
    }
    if err = rows.Err(); err != nil {
        return nil, err
    }

    return permissions, nil
}
```

Then the last thing we need to do is add the `PermissionModel` to our parent `Model` struct, so it's available to our handlers and middleware. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/models.go

```
package models

...

type Models struct {
    Movies      MovieModel
    Permissions PermissionModel // Add a new Permissions field.
    Tokens      TokenModel
    Users       UserModel
}

func NewModels(db *sql.DB) Models {
    return Models{
        Movies:      MovieModel{DB: db},
        Permissions: PermissionModel{DB: db}, // Initialize a new PermissionModel instance.
        Tokens:      TokenModel{DB: db},
        Users:       UserModel{DB: db},
    }
}
```

dego_vila@yahoo.com 30 Oct 2026

Checking permissions

Now that our `PermissionModel` is set up, let's use it to restrict access to our API endpoints.

Specifically, we'll create a new `requirePermission()` middleware, which follows the same pattern as our existing `requireActivatedUser()` middleware but additionally checks that the a user has a specific permission.

- The `requirePermission()` middleware will accept a specific permission code like `"movies:read"` as an argument.
- In the middleware, we'll call the `app.models.Permissions.GetAllForUser()` method (which we just made) to get a slice of their permissions.
- Then we can check to see if the slice contains the specific permission code needed. If it doesn't, we should send the client a `403 Forbidden` response.

To put this into practice, let's first make a `missingPermissionResponse()` helper function for sending the `403 Forbidden` response with an appropriate error message.

File: cmd/api/errors.go

```
package main

...

func (app *application) missingPermissionResponse(w http.ResponseWriter, r *http.Request) {
    message := "your user account doesn't have the necessary permissions to access this resource"
    app.errorResponse(w, r, http.StatusForbidden, message)
}
```

Then let's head to our `cmd/api/middleware.go` file and create the new `requirePermission()` middleware method, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/middleware.go

```
package main

...

// Notice that the first argument in the requirePermission() function represents the
// permission code that we want the user to have.
func (app *application) requirePermission(code string, next http.HandlerFunc) http.HandlerFunc {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Check that the user is authenticated and activated, just like before.
        authenticatedUser, found := app.contextGetAuthenticatedUser(r)
        if !found {
            app.authenticationRequiredResponse(w, r)
            return
        }

        if !authenticatedUser.Activated {
            app.inactiveAccountResponse(w, r)
            return
        }

        // Get the slice of permissions for the user.
        permissions, err := app.models.Permissions.GetAllForUser(authenticatedUser.ID)
        if err != nil {
            app.serverErrorResponse(w, r, err)
            return
        }

        // Check if the slice includes the required permission. If it doesn't, then
        // return a 403 Forbidden response.
        if !permissions.Include(code) {
            app.missingPermissionResponse(w, r)
            return
        }

        // Otherwise they have the required permission so we call the next handler in
        // the chain.
        next.ServeHTTP(w, r)
    })
}
```

> **Note:** Our `requirePermission()` middleware actually does three things. It checks that the user sending the request is authenticated, then checks that they are activated, and then finally checks that they have the appropriate permission.
>
> Given this, perhaps the name `requireActivatedUserWithPermission()` would be more appropriate than simply `requirePermission()`, but I've opted for the shorter version in this book purely for conciseness. If you're following along and would prefer to use the longer name, feel free.

OK, now that's done, the last step is to update our `cmd/api/routes.go` file to utilize the new middleware on the necessary endpoints.

dego_vila@yahoo.com 30 Oct 2026

Go ahead and update the routes so that our API requires the `"movies:read"` permission for the endpoints that fetch movie data, and the `"movies:write"` permission for the endpoints that create, edit or delete a movie.

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    // Update each of the /v1/movies** endpoints to use the requirePermission() middleware,
    // passing in the required permission code as the first parameter.
    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requirePermission("movies:read", app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requirePermission("movies:write", app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requirePermission("movies:read", app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requirePermission("movies:write", app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requirePermission("movies:write", app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    return app.recoverPanic(app.rateLimit(app.authenticate(router)))
}
```

Demonstration

Showing this in action is a bit awkward because, if you've been following along, none of the users in our database currently have any permissions set for them.

To help demonstrate this new functionality, let's open `psql` and add some permissions. Specifically, we will:

- Activate the user `alice@example.com`.
- Give *all* users the `"movies:read"` permission.
- Give the user `faith@example.com` the `"movies:write"` permission.

If you're following along, open your `psql` prompt:

dego_vila@yahoo.com 30 Oct 2026

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=>
```

And execute the following statements:

```
-- Set the activated field for alice@example.com to true.
UPDATE users SET activated = true WHERE email = 'alice@example.com';

-- Give all users the 'movies:read' permission
INSERT INTO users_permissions
SELECT id, (SELECT id FROM permissions WHERE code = 'movies:read') FROM users;

-- Give faith@example.com the 'movies:write' permission
INSERT INTO users_permissions
VALUES (
    (SELECT id FROM users WHERE email = 'faith@example.com'),
    (SELECT id FROM permissions WHERE code = 'movies:write')
);

-- List all activated users and their permissions.
SELECT email, array_agg(permissions.code) as permissions
FROM permissions
INNER JOIN users_permissions ON users_permissions.permission_id = permissions.id
INNER JOIN users ON users_permissions.user_id = users.id
WHERE users.activated = true
GROUP BY email;
```

Once that completes, you should see a list of the currently activated users and their permissions, similar to this:

```
       email       |        permissions
-------------------+---------------------------
 alice@example.com | {movies:read}
 faith@example.com | {movies:read,movies:write}
(2 rows)
```

> **Note:** In that final SQL query, we're using the aggregation function `array_agg()` and a `GROUP BY` clause to output the permissions associated with each email address as an array.

Now that our users have some permissions assigned to them, we're ready to try this out.

To begin with, let's try making some requests as `alice@example.com` to our `GET /v1/movies/1` and `DELETE /v1/movies/1` endpoints. The first request should work correctly,

dego_vila@yahoo.com 30 Oct 2026

but the second should fail because the user doesn't have the necessary `movies:write` permission.

```
$ BODY='{"email": "alice@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/tokens/authentication
{
    "authentication_token": {
        "token": "OPFXEPOYZWMGNWXWKMYIMEGATU",
        "expiry": "2026-09-14T13:05:20.601721729+02:00"
    }
}

$ curl -H "Authorization: Bearer OPFXEPOYZWMGNWXWKMYIMEGATU" localhost:4000/v1/movies/1
{
    "movie": {
        "id": 1,
        "title": "Moana",
        "year": 2016,
        "runtime": "107m",
        "genres": [
            "animation",
            "adventure"
        ],
        "version": 1
    }
}

$ curl -X DELETE -H "Authorization: Bearer OPFXEPOYZWMGNWXWKMYIMEGATU" localhost:4000/v1/movies/1
{
    "error": "your user account doesn't have the necessary permissions to access this resource"
}
```

Great, that's working just as we expected — the `DELETE` operation is blocked because `alice@example.com` doesn't have the necessary `movies:write` permission.

In contrast, let's try the same operation but with `faith@example.com` as the user. This time the `DELETE` operation should work correctly, like so:

```
$ BODY='{"email": "faith@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/tokens/authentication
{
    "authentication_token": {
        "token": "E42XD5OBBBO4MPUPYGLLY2GURE",
        "expiry": "2026-09-14T13:10:06.790353626+02:00"
    }
}

$ curl -X DELETE -H "Authorization: Bearer E42XD5OBBBO4MPUPYGLLY2GURE" localhost:4000/v1/movies/1
{
    "message": "movie successfully deleted"
}
```

dego_vila@yahoo.com 30 Oct 2026

Granting permissions

Our permissions model and authorization middleware are now functioning well. But — at the moment — when a new user registers an account they *don't have any permissions*. In this chapter we're going to change that so that new users are automatically *granted* the `"movies:read"` permission by default.

Updating the permissions model

In order to grant permissions to a user, we'll need to update our `PermissionModel` to include an `AddForUser()` method, which adds *one or more permission codes for a specific user* to our database. The idea is that we will be able to use it in our handlers like this:

```
// Add the "movies:read" and "movies:write" permissions for the user with ID = 2.
app.models.Permissions.AddForUser(2, "movies:read", "movies:write")
```

Behind the scenes, the SQL statement that we need to insert this data looks like this:

```
INSERT INTO users_permissions
SELECT $1, permissions.id FROM permissions WHERE permissions.code = ANY($2)
```

In this query, the `$1` parameter will be the user's ID, and the `$2` parameter will be *a PostgreSQL array* of the permission codes that we want to add for the user, like `{'movies:read', 'movies:write'}`.

So what's happening here is that the `SELECT ...` statement on the second line creates an 'interim' table with rows made up of the user ID *and the corresponding IDs for the permission codes in the array*. Then we insert the contents of this interim table into our `user_permissions` table.

Let's go ahead and create the `AddForUser()` method in the `internal/models/permissions.go` file:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/permissions.go

```
package models

import (
    "context"
    "database/sql"
    "slices"
    "time"

    "github.com/lib/pq" // New import
)

...

// Add the provided permission codes for a specific user. Notice that we're using a
// variadic parameter for the codes so that we can assign multiple permissions in a
// single call.
func (m PermissionModel) AddForUser(userID int, codes ...string) error {
    query := `
        INSERT INTO users_permissions
        SELECT $1, permissions.id FROM permissions WHERE permissions.code = ANY($2)`

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    _, err := m.DB.ExecContext(ctx, query, userID, pq.Array(codes))
    return err
}
```

Updating the registration handler

Now that's in place, let's update our `registerUserHandler` so that new users are automatically granted the `movies:read` permission when they register. Like so:

File: cmd/api/users.go

```
package main

...

func (app *application) registerUserHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Name     string `json:"name"`
        Email    string `json:"email"`
        Password string `json:"password"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
    }

    user := models.User{
        Name:      input.Name,
        Email:     input.Email,
        Activated: false,
    }

    err = user.Password.Set(input.Password)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    v := validator.New()

    if models.ValidateUser(v, user); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    user, err = app.models.Users.Insert(user)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrDuplicateEmail):
            v.AddError("email", "a user with this email address already exists")
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Add the "movies:read" permission for the new user.
    err = app.models.Permissions.AddForUser(user.ID, "movies:read")
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    token, err := app.models.Tokens.New(user.ID, 3*24*time.Hour, models.ScopeActivation)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    app.background(func() {
        data := map[string]any{
            "activationToken": token.Plaintext,
            "userID":          user.ID,
        }

        err := app.mailer.Send(user.Email, "user_welcome.tmpl", data)
        if err != nil {
            app.logger.Error(err.Error())
        }
    })

    err = app.writeJSON(w, http.StatusAccepted, envelope{"user": user}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}

...
```

dego_vila@yahoo.com 30 Oct 2026

Let's check that this is working correctly by registering a brand-new user with the email address `grace@example.com`:

```
$ BODY='{"name": "Grace Smith", "email": "grace@example.com", "password": "pa55word"}'
$ curl -d "$BODY" localhost:4000/v1/users
{
    "user": {
        "id": 8,
        "created_at": "2026-09-13T13:14:06+02:00",
        "name": "Grace Smith",
        "email": "grace@example.com",
        "activated": false
    }
}
```

If you open `psql`, you should be able to see that they have the `movies:read` permission by running the following SQL query:

```
SELECT email, code FROM users
INNER JOIN users_permissions ON users.id = users_permissions.user_id
INNER JOIN permissions ON users_permissions.permission_id = permissions.id
WHERE users.email = 'grace@example.com';
```

Like so:

```
$ psql $GREENLIGHT_DB_DSN
Password for user greenlight:
psql (18.4 (Ubuntu 18.4-1.pgdg22.04+1)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> SELECT email, code FROM users
greenlight-> INNER JOIN users_permissions ON users.id = users_permissions.user_id
greenlight-> INNER JOIN permissions ON users_permissions.permission_id = permissions.id
greenlight-> WHERE users.email = 'grace@example.com';
       email       |    code
-------------------+------------
 grace@example.com | movies:read
(1 row)
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 17.

Cross-origin requests

In this section of the book, we're going to switch to a completely new topic and update our application so that it supports cross-origin requests (CORS) from JavaScript.

You'll learn:

- What cross-origin requests are, and why web browsers prevent them by default.
- The difference between *simple* and *preflight* cross-origin requests.
- How to use `Access-Control` headers to allow or disallow specific cross-origin requests.
- About the security considerations you need to be aware of when configuring CORS settings in your application.

dego_vila@yahoo.com 30 Oct 2026

An overview of CORS

Before we dive into any code, or start talking about cross-origin requests specifically, let's take a moment to define what we mean by the term *origin*.

Basically, if two URLs have the same *scheme*, *host* and *port* (if specified) they are said to share the same origin. To help illustrate this, let's compare the following URLs:

URL A	URL B	Same origin?	Reason
`https://foo.com/a`	`http://foo.com/a`	No	Different scheme (`http` vs `https`)
`http://foo.com/a`	`http://www.foo.com/a`	No	Different host (`foo.com` vs `www.foo.com`)
`http://foo.com/a`	`http://foo.com:443/a`	No	Different port (no port vs `443`)
`http://foo.com/a`	`http://foo.com/b`	Yes	Only the path is different
`http://foo.com/a`	`http://foo.com/a?b=c`	Yes	Only the query string is different
`http://foo.com/a#b`	`http://foo.com/a#c`	Yes	Only the fragment is different

Understanding what origins are is important because all web browsers implement a security mechanism known as the *same-origin policy*. There are some very small differences in how browsers implement this policy, but broadly speaking:

- A webpage on one origin can *embed* certain types of resources from another origin in their HTML — including images, CSS, and JavaScript files. For example, doing this in your webpage is OK:

```
<img src="http://anotherorigin.com/example.png" alt="example image">
```

- A webpage on one origin can *send* data to a different origin. For example, it's OK for an HTML form in a webpage to submit data to a different origin.
- But a webpage on one origin is *not* allowed to *read* data from a different origin.

The key thing here is the final bullet-point: the same-origin policy prevents a (potentially malicious) website on another origin from *reading* (possibly confidential) information from

dego_vila@yahoo.com 30 Oct 2026

your website.

It's important to emphasize that cross-origin *sending of data* is not prevented by the same-origin policy, despite also being dangerous. In fact, this is why CSRF attacks are possible and why we need to take additional steps to prevent them — such as using `SameSite` cookies and CSRF tokens.

As a developer, the time that you're most likely to run up against the same-origin policy is when making cross-origin requests from JavaScript running in a browser.

For example, let's say that you have a webpage at `https://foo.com` containing some front-end JavaScript code. If this JavaScript tries to make an HTTP request to `https://bar.com/models.json` (a different origin), then the request will be sent and processed by the `bar.com` server, but the user's web browser will *block the response* so that the JavaScript code from `https://foo.com` cannot see it.

Generally speaking, the same-origin policy is an extremely useful security safeguard. But while it's good in the general case, in certain circumstances you might want to relax the restriction.

For example, if you have an API at `api.example.com` and a trusted JavaScript front-end application running on `www.example.com`, then you'll probably want to allow cross-origin requests from the trusted `www.example.com` domain to your API.

Or perhaps you have a completely open public API, and you want to allow cross-origin requests *from anywhere* so it's easy for other developers to integrate with their own websites.

Fortunately, most modern web browsers allow you to allow or disallow specific cross-origin requests to your API by setting `Access-Control` headers on your API responses. We'll explain exactly how to do that and how these headers work over the next few chapters of this book.

dego_vila@yahoo.com 30 Oct 2026

Demonstrating the same-origin policy

To demonstrate how the same-origin policy works and how to relax it for requests to our API, we need to simulate *a request to our API from a different origin* .

As we're already using Go in this book, let's quickly make a second, very simple, Go application to make this cross-origin request. Essentially, we want this second application to serve a webpage containing some JavaScript, which in turn makes a request to our `GET /v1/healthcheck` endpoint.

If you're following along, create a new `cmd/examples/cors/simple/main.go` file to hold the code for this second application:

```
$ mkdir -p cmd/examples/cors/simple
$ touch cmd/examples/cors/simple/main.go
```

And add the following content:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/examples/cors/simple/main.go

```
package main

import (
    "flag"
    "log"
    "net/http"
)

// Define a string constant containing the HTML for the webpage. This consists of a <h1>
// header tag, and some JavaScript which fetches the JSON from our GET /v1/healthcheck
// endpoint and writes it inside the <div id="output"></div> element.
const html = `
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
</head>
<body>
    <h1>Simple CORS</h1>
    <div id="output"></div>
    <script>
        document.addEventListener('DOMContentLoaded', function() {
            fetch("http://localhost:4000/v1/healthcheck").then(
                function (response) {
                    response.text().then(function (text) {
                        document.getElementById("output").innerHTML = text;
                    });
                },
                function(err) {
                    document.getElementById("output").innerHTML = err;
                }
            );
        });
    </script>
</body>
</html>`

func main() {
    // Make the server address configurable at runtime via a command-line flag.
    addr := flag.String("addr", ":9000", "Server address")
    flag.Parse()

    log.Printf("starting server on %s", *addr)

    // Start an HTTP server listening on the given address, which responds to all
    // requests with the webpage HTML above.
    err := http.ListenAndServe(*addr, http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte(html))
    }))
    log.Fatal(err)
}
```

The Go code here should be nice and familiar to you already, but let's take a closer look at the JavaScript code in the `<script>` tag and explain what it does.

dego_vila@yahoo.com 30 Oct 2026

```
<script>
    document.addEventListener('DOMContentLoaded', function() {
        fetch("http://localhost:4000/v1/healthcheck").then(
            function (response) {
                response.text().then(function (text) {
                    document.getElementById("output").innerHTML = text;
                });
            },
            function(err) {
                document.getElementById("output").innerHTML = err;
            }
        );
    });
</script>
```

In this code:

- We use the `fetch()` function to make a request to our API healthcheck endpoint. By default this sends a `GET` request, but it's also possible to configure this to use different HTTP methods and add custom headers. We'll explain how to do that later.
- The `fetch()` method works *asynchronously* and returns a promise. We use the `then()` method on the promise to set up two callback functions: the first callback is executed if `fetch()` is successful, and the second is executed if there is a failure.
- In our 'successful' callback we read the response body with `response.text()`, and use `document.getElementById("output").innerHTML` to replace the contents of the `<div id="output"></div>` element with this response body.
- In the 'failure' callback we replace the contents of the `<div id="output"></div>` element with the error message.
- This logic is all wrapped up in `document.addEventListener('DOMContentLoaded', function(){…})`, which essentially means that `fetch()` won't be called until the user's web browser has completely loaded the HTML document.

> **Note:** I've explained the above because it's interesting, but this book isn't about JavaScript and you don't need to worry about the details here. All you really need to know is that *the JavaScript makes an HTTP request to our API's healthcheck endpoint, and then dumps the response into the* `<div id="output"></div>` *element*.

Demonstration

OK, let's try this out. Go ahead and start up the new application:

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/examples/cors/simple
2026/09/13 13:26:26 starting server on :9000
```

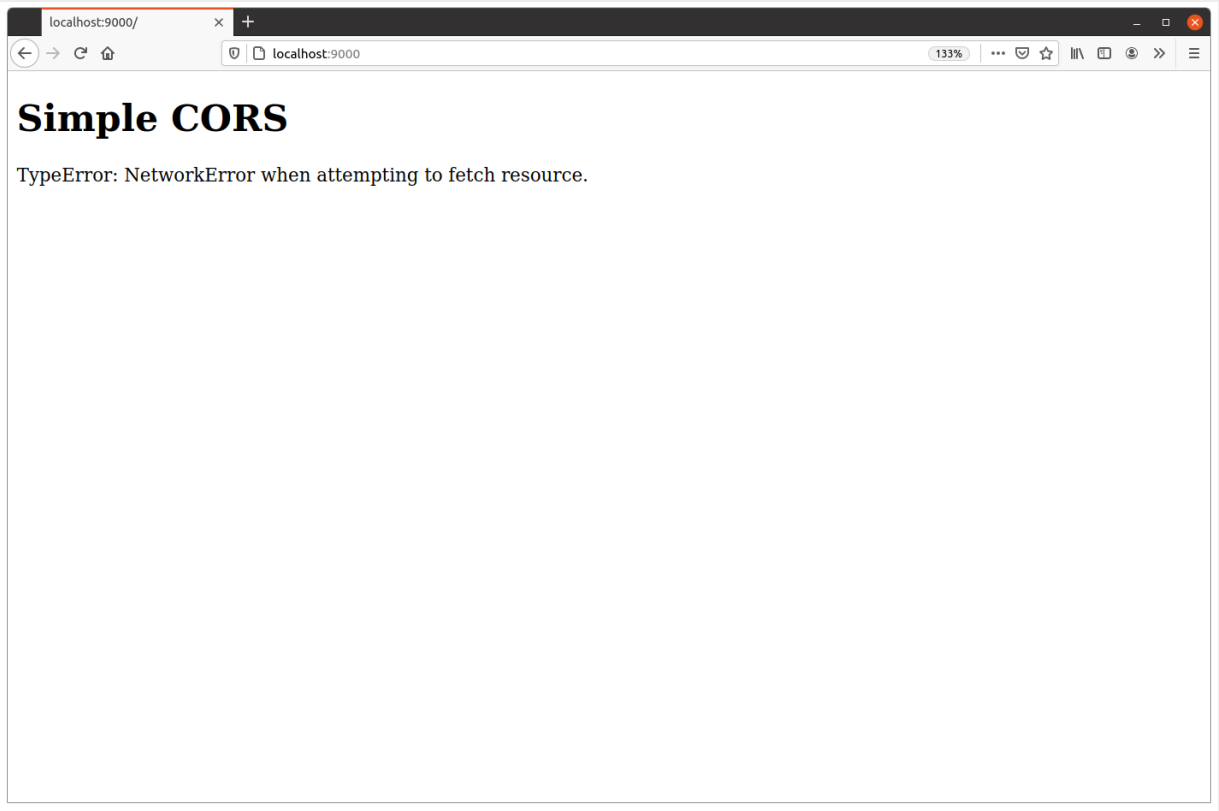
And then open a second terminal window and start our regular API application at the same time:

```
$ go run ./cmd/api
time=2026-09-13T13:26:37.171+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T13:26:37.171+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

At this point, you should now have the API running on the origin `http://localhost:4000` and the webpage with the JavaScript running on the origin `http://localhost:9000`. Because the ports are different, these are two different origins.

So, when you visit `http://localhost:9000` in your web browser, the `fetch()` action to `http://localhost:4000/v1/healthcheck` should be forbidden by the same-origin policy. Specifically, our API should receive and process the request, but your web browser should block the response from being read by the JavaScript code.

Let's take a look. If you open your web browser and visit `http://localhost:9000`, you should see the *Simple CORS* header followed by an error message similar to this:

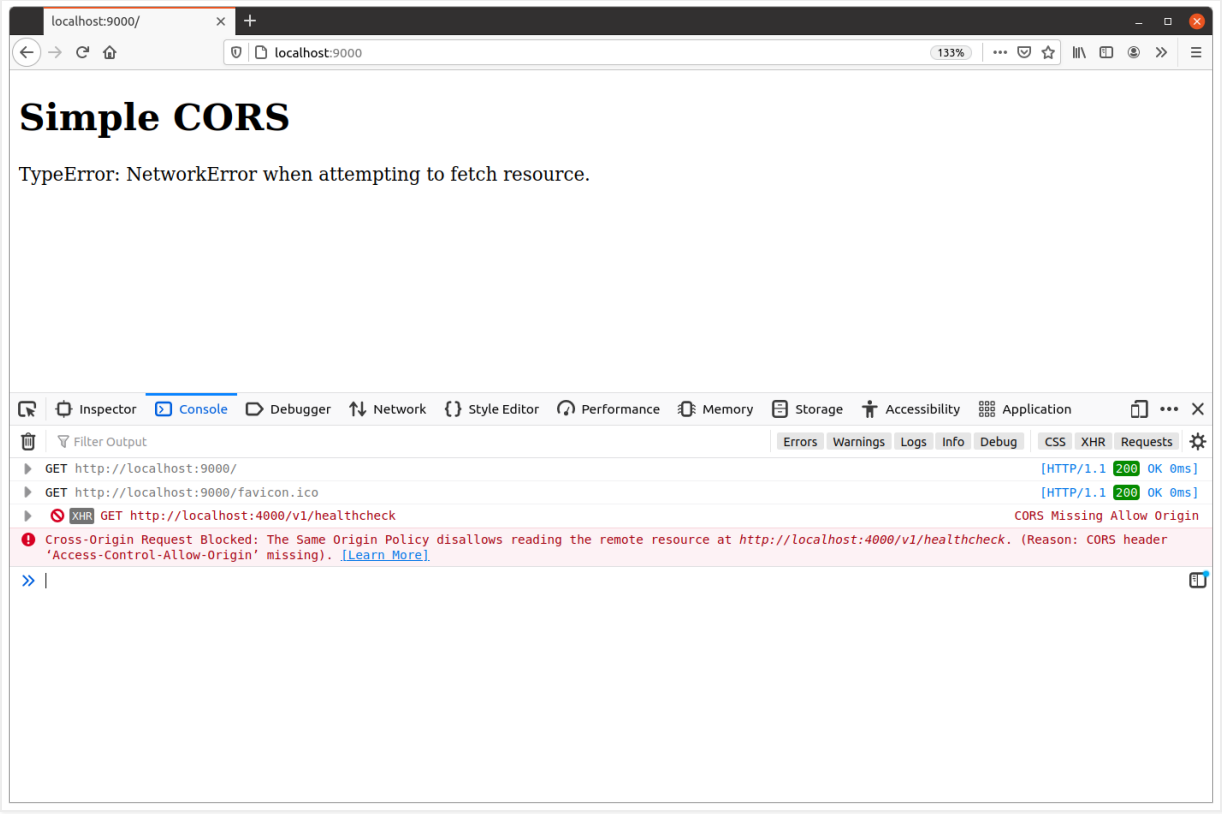


dego_vila@yahoo.com 30 Oct 2026

> **Note:** The error message you see here is defined by your web browser, so yours might be slightly different if you're not using Firefox.

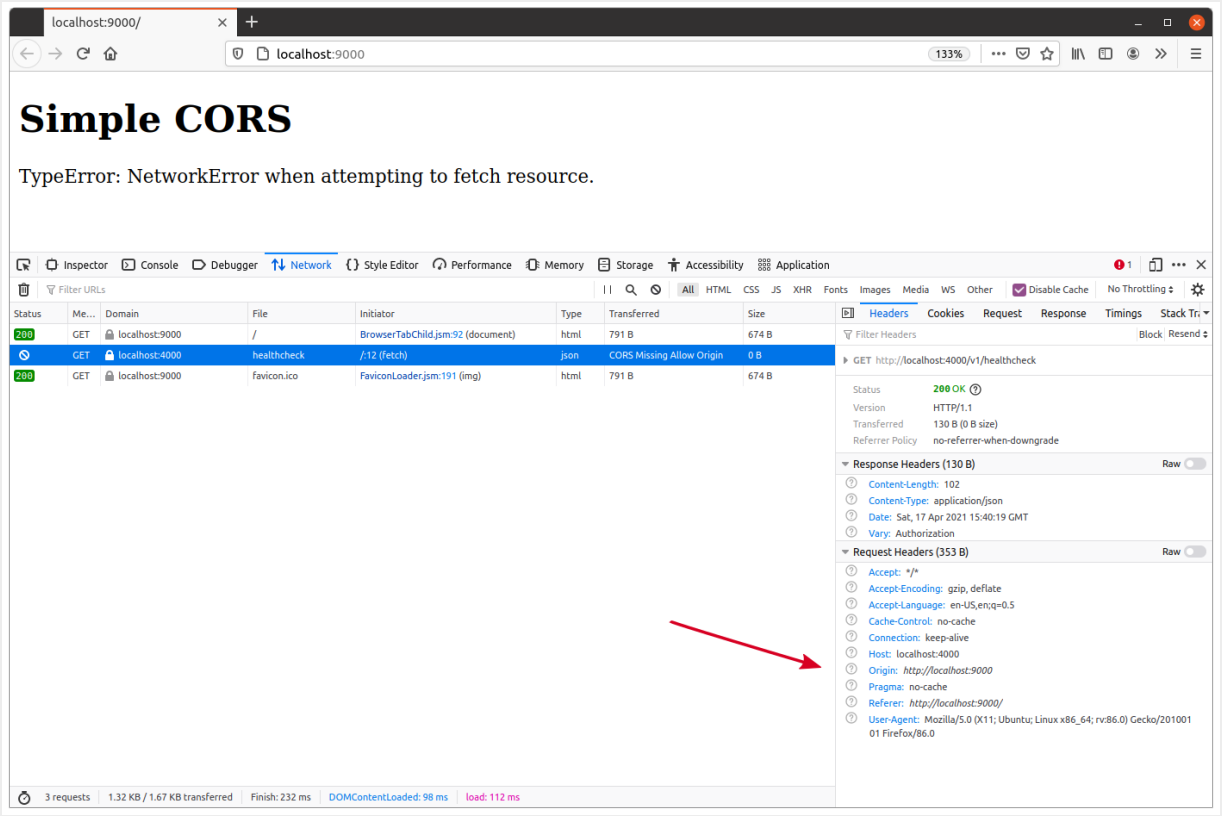
It's also helpful here to open your browser's developer tools, refresh the page, and look at your console log. You should see a message stating that the response from our `GET /v1/healthcheck` endpoint was blocked from being read due to the same-origin policy, similar to this:

> *Cross-Origin Request Blocked: The Same Origin Policy disallows reading the remote resource at http://localhost:4000/v1/healthcheck.*



You may also want to open the network activity tab in your developer tools and examine the HTTP headers associated with the blocked request.

dego_vila@yahoo.com 30 Oct 2026



There are a couple of important things to point out here.

The first thing is that the headers demonstrate that the request was sent to our API, which processed the request and returned a successful `200 OK` response to the web browser containing all our standard response headers. To reiterate: *the request itself was not prevented by the same-origin policy — it's just that the browser won't let JavaScript see the response*.

The second thing is that the web browser automatically set an `Origin` header on the request to show where the request originates from (highlighted by the red arrow above). You'll see that the header looks like this:

```
Origin: http://localhost:9000
```

We'll use this header in the next chapter to help us selectively relax the same-origin policy, depending on whether we trust the origin that a request is coming from.

Finally, it's important to emphasize that the same-origin policy is *a web browser thing only*.

Outside of a web browser, anyone can make a request to our API from anywhere, using `curl`, `wget` or any other means and read the response. That's completely unaffected and unchanged by the same-origin policy.

dego_vila@yahoo.com 30 Oct 2026

Simple CORS requests

Let's now make some changes to our API to *relax the same-origin policy*, so that JavaScript can read the responses from our API endpoints.

To start with, the simplest way to achieve this is by setting the following header on all our API responses:

```
Access-Control-Allow-Origin: *
```

The `Access-Control-Allow-Origin` response header is used to indicate to a browser that it's OK to share a response with a different origin. In this case, the header value is the wildcard `*` character, which means that it's OK to share the response with *any other origin*.

Let's go ahead and create a small `enableCORS()` middleware function in our API application which sets this header:

File: cmd/api/middleware.go

```
package main

...

func (app *application) enableCORS(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "*")

        next.ServeHTTP(w, r)
    })
}
```

And then update your `cmd/api/routes.go` file so that this middleware is used on all the application routes. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requirePermission("movies:read", app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requirePermission("movies:write", app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requirePermission("movies:read", app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requirePermission("movies:write", app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requirePermission("movies:write", app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    // Add the enableCORS() middleware.
    return app.recoverPanic(app.enableCORS(app.rateLimit(app.authenticate(router))))
}
```

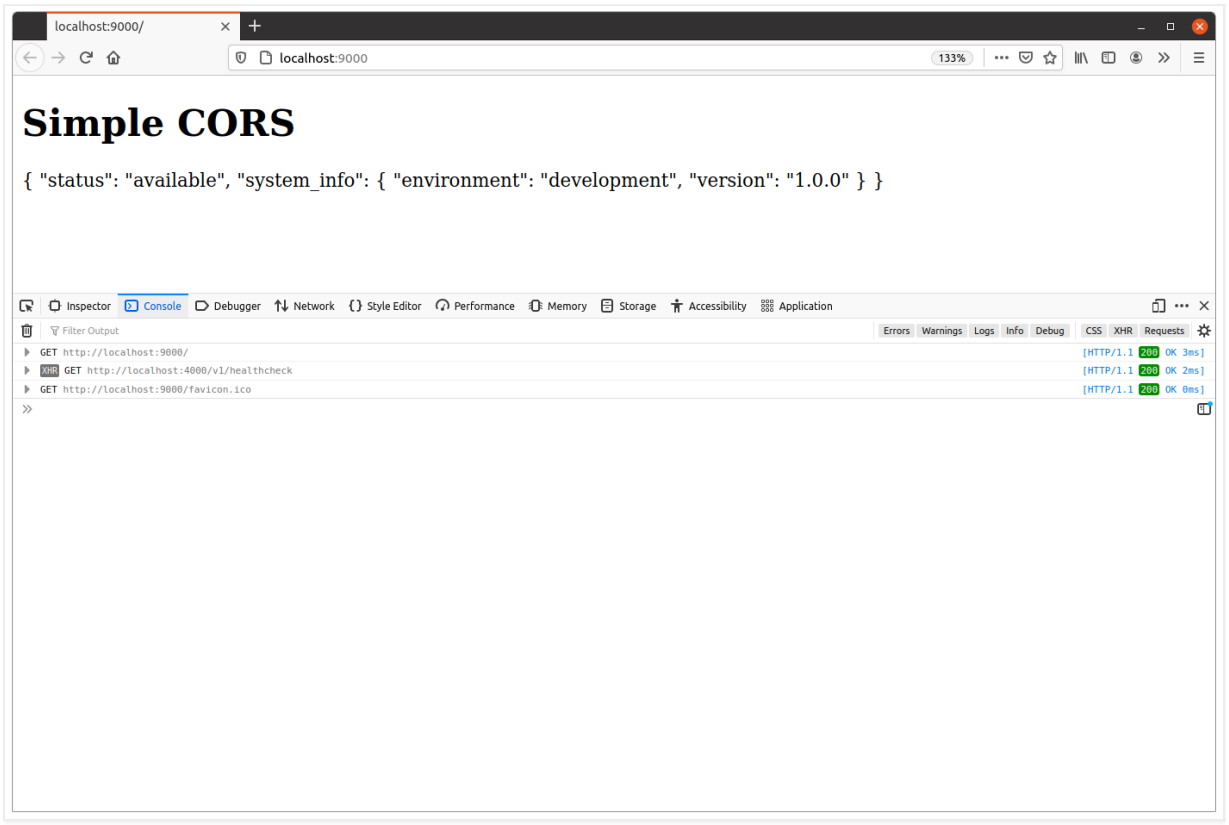
It's important to point out here that the `enableCORS()` middleware is deliberately positioned early in the middleware chain.

If we positioned it after our rate limiter, for example, any cross-origin requests that exceed the rate limit would *not have the* `Access-Control-Allow-Origin` *header set*. This means that they would be blocked by the client's web browser due to the same-origin policy, rather than the client receiving a `429 Too Many Requests` response like they should.

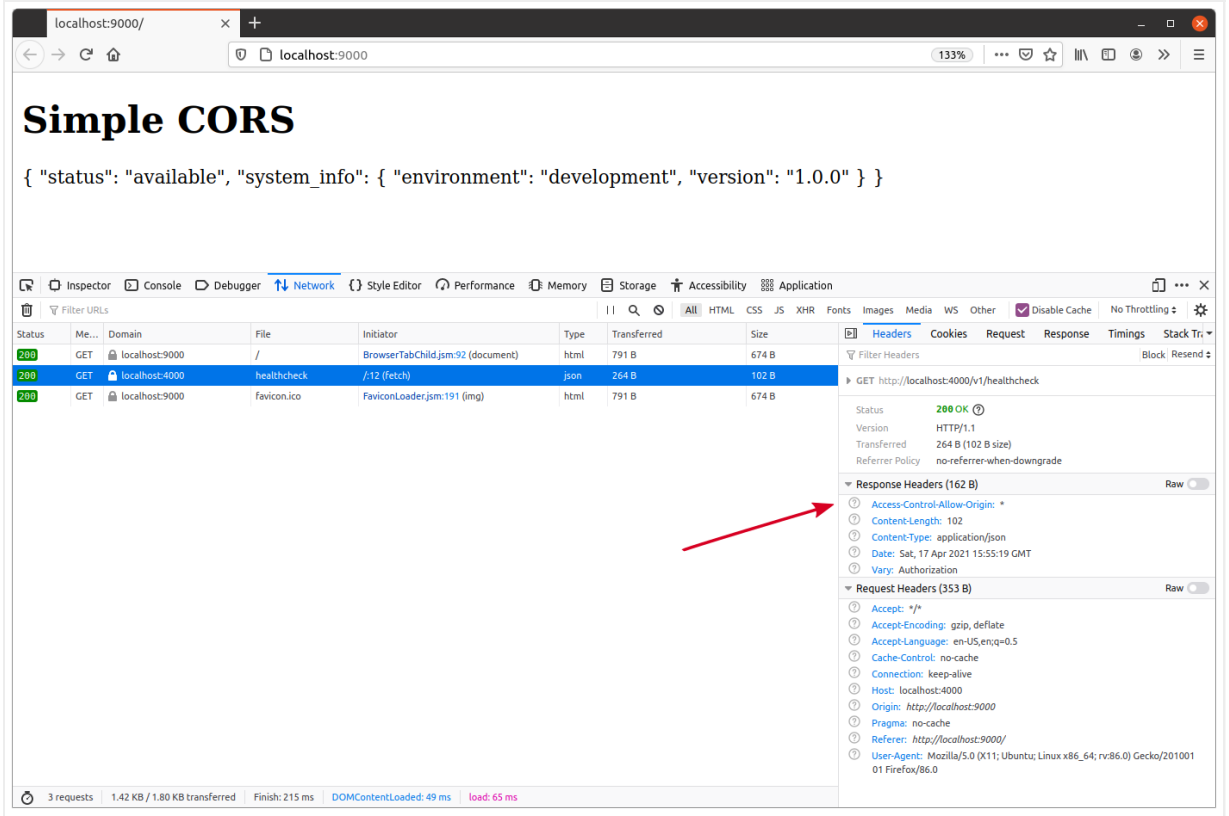
OK, let's try this out.

Restart the API application and then try visiting `http://localhost:9000` in your browser again. This time the cross-origin request should complete successfully, and you should see the JSON from our healthcheck handler presented on the webpage. Like so:

dego_vila@yahoo.com 30 Oct 2026



I also recommend taking a quick look at the request and response headers for the JavaScript `fetch()` request again. You should see that the `Access-Control-Allow-Origin: *` header has been set on the response, similar to this:



dego_vila@yahoo.com 30 Oct 2026

Restricting origins

Using a wildcard to allow cross-origin requests, like we are in the code above, can be useful in certain circumstances (like when you have a completely public API with no access control checks). But more often you'll probably want to restrict CORS to a much smaller set of *trusted origins*.

To do this, you need to explicitly include the trusted origins in the `Access-Control-Allow-Origin` header instead of using a wildcard. For example, if you only want to allow CORS from the origin `https://www.example.com` you could send the following header in your responses:

```
Access-Control-Allow-Origin: https://www.example.com
```

If you only have one, fixed, origin that you want to allow requests from, then doing this is quite simple — you can just update your `enableCORS()` middleware to hard-code the necessary origin value.

But if you need to support multiple trusted origins, or you want the value to be configurable at runtime, then things get a bit more complex.

One of the problems is that — in practice — you can only specify *exactly one origin* in the `Access-Control-Allow-Origin` header. You can't include a list of multiple origin values, separated by spaces or commas like you might expect.

To work around this limitation, you'll need to update your `enableCORS()` middleware to check if the value of the `Origin` header matches one of your trusted origins. If it does, then you can reflect (or *echo*) that value back in the `Access-Control-Allow-Origin` response header.

> **Note:** The web origin specification does permit multiple *space-separated* values in the `Access-Control-Allow-Origin` header but, unfortunately, no web browsers actually support this.

Supporting multiple dynamic origins

Let's update our API so that cross-origin requests are restricted to a list of trusted origins, configurable at runtime.

dego_vila@yahoo.com 30 Oct 2026

The first thing we'll do is add a new `-cors-trusted-origins` command-line flag to our API application, which we can use to specify the list of trusted origins at runtime. We'll set this up so that the origins must be separated by a space character — like so:

```
$ go run ./cmd/api -cors-trusted-origins="https://www.example.com https://staging.example.com"
```

In order to process this command-line flag, we can combine the `flags.Func()` and `strings.Fields()` functions to split the origin values into a `[]string` slice ready for use.

If you're following along, open your `cmd/api/main.go` file and add the following code:

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "flag"
    "log/slog"
    "os"
    "strings" // New import
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/mailer"
    "greenlight.alexedwards.net/internal/models"

    _ "github.com/lib/pq"
)

const version = "1.0.0"

type config struct {
    port int
    env  string
    db   struct {
        dsn          string
        maxOpenConns int
        maxIdleConns int
        maxIdleTime  time.Duration
    }
    limiter struct {
        enabled bool
        rps     float64
        burst   int
    }
    smtp struct {
        host     string
        port     int
        username string
        password string
        sender   string
    }
    // Add a cors struct and trustedOrigins field with the type []string.
    cors struct {
        trustedOrigins []string
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
}

...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", os.Getenv("GREENLIGHT_DB_DSN"), "PostgreSQL DSN")

    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    flag.BoolVar(&cfg.limiter.enabled, "limiter-enabled", true, "Enable rate limiter")
    flag.Float64Var(&cfg.limiter.rps, "limiter-rps", 2, "Rate limiter maximum requests per second")
    flag.IntVar(&cfg.limiter.burst, "limiter-burst", 4, "Rate limiter maximum burst")

    flag.StringVar(&cfg.smtp.host, "smtp-host", "sandbox.smtp.mailtrap.io", "SMTP host")
    flag.IntVar(&cfg.smtp.port, "smtp-port", 25, "SMTP port")
    flag.StringVar(&cfg.smtp.username, "smtp-username", "a7420fc0883489", "SMTP username")
    flag.StringVar(&cfg.smtp.password, "smtp-password", "e75ffd0a3aa5ec", "SMTP password")
    flag.StringVar(&cfg.smtp.sender, "smtp-sender", "Greenlight <no-reply@greenlight.alexedwards.net>", "SMTP sender")

    // Use the flag.Func() function to process the -cors-trusted-origins command line
    // flag. In this we use the strings.Fields() function to split the flag value into a
    // slice based on whitespace characters and assign it to our config struct.
    // Importantly, if the -cors-trusted-origins flag is not present, contains the empty
    // string, or contains only whitespace, then strings.Fields() will return an empty
    // []string slice.
    flag.Func("cors-trusted-origins", "Trusted CORS origins (space separated)", func(val string) error {
        cfg.cors.trustedOrigins = strings.Fields(val)
        return nil
    })

    flag.Parse()

    ...
}

...
```

> **Note:** If you'd like to find out more about the `flag.Func()` functionality and some of the ways in which you can use it, I've written a much more detailed blog post about it here.

Once that's done, the next step is to update our `enableCORS()` middleware. Specifically, we want the middleware to check if the value of the request `Origin` header is an exact, case-sensitive, match for one of our trusted origins. If there is a match, then we should set an `Access-Control-Allow-Origin` response header which reflects (or echoes) back the value of the request's `Origin` header.

Otherwise, we should allow the request to proceed as normal *without* setting an `Access-`

dego_vila@yahoo.com 30 Oct 2026

`Control-Allow-Origin` response header. In turn, that means that any cross-origin responses will be blocked by a web browser, just like they were originally.

A side effect of this is that the *response will be different depending on the origin that the request is coming from*. Specifically, the value of the `Access-Control-Allow-Origin` header may be different in the response, or it may not even be included at all.

So because of this we should make sure to always set a `Vary: Origin` response header to inform any caches that the response may be different. This is actually really important, and it can be the cause of subtle bugs like this one if you forget to do it. As a rule of thumb:

If your code makes a decision about what to return based on the content of a request header, you should include that header name in your `Vary` response header — even if the request didn't include that header.

OK, let's update our `enableCORS()` middleware in line with the logic above, like so:

File: cmd/api/middleware.go

```
package main

...

func (app *application) enableCORS(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Add the "Vary: Origin" header.
        w.Header().Add("Vary", "Origin")

        // Get the value of the request's Origin header.
        origin := r.Header.Get("Origin")

        // Only run this if there's an Origin request header present.
        if origin != "" {
            // Loop through the list of trusted origins, checking to see if the request
            // origin exactly matches one of them. If there are no trusted origins, then
            // the loop won't be iterated.
            for i := range app.config.cors.trustedOrigins {
                if origin == app.config.cors.trustedOrigins[i] {
                    // If there is a match, then set a "Access-Control-Allow-Origin"
                    // response header with the request origin as the value and break
                    // out of the loop.
                    w.Header().Set("Access-Control-Allow-Origin", origin)
                    break
                }
            }
        }

        // Call the next handler in the chain.
        next.ServeHTTP(w, r)
    })
}
```

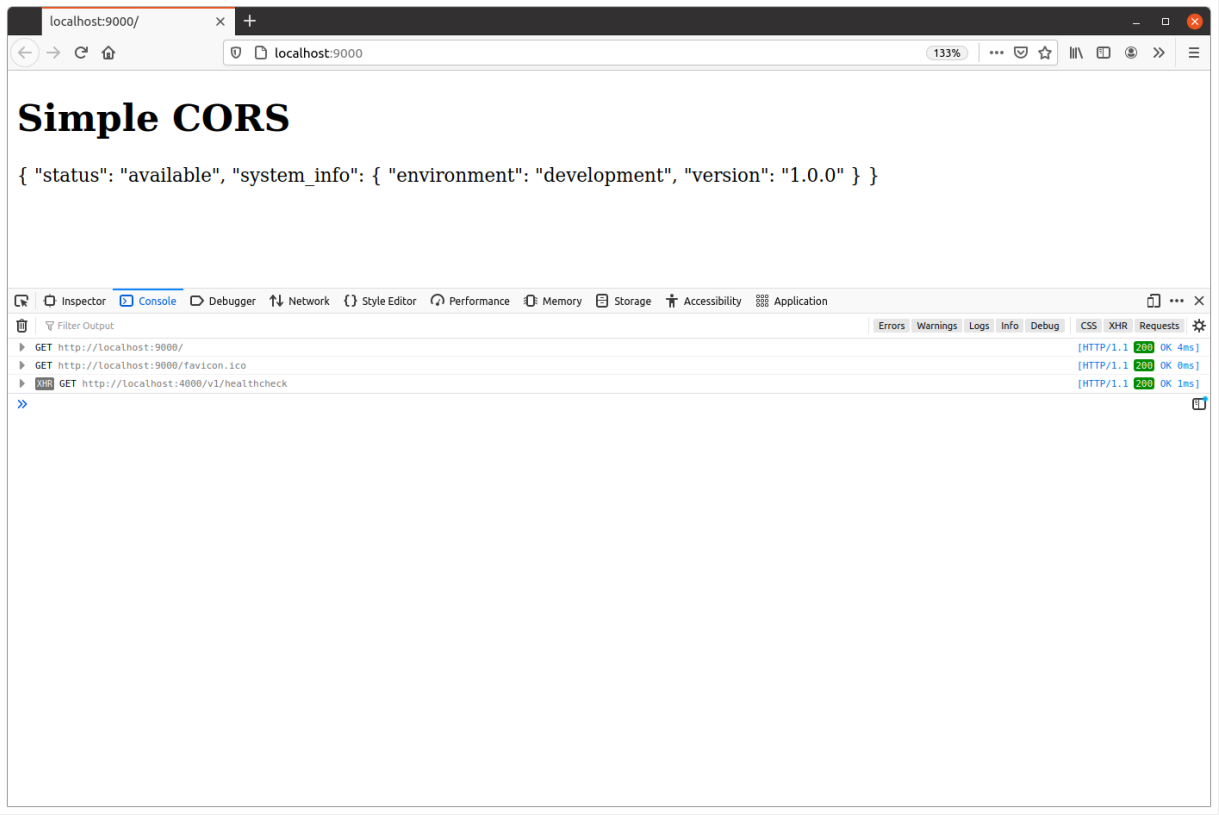
And with those changes complete, we're now ready to try this out again.

dego_vila@yahoo.com 30 Oct 2026

Restart your API, passing in `http://localhost:9000` and `http://localhost:9001` as trusted origins like so:

```
$ go run ./cmd/api -cors-trusted-origins="http://localhost:9000 http://localhost:9001"
time=2026-09-13T13:35:17.790+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T13:35:17.790+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

And when you refresh `http://localhost:9000` in your browser, you should find that the cross-origin request still works successfully.



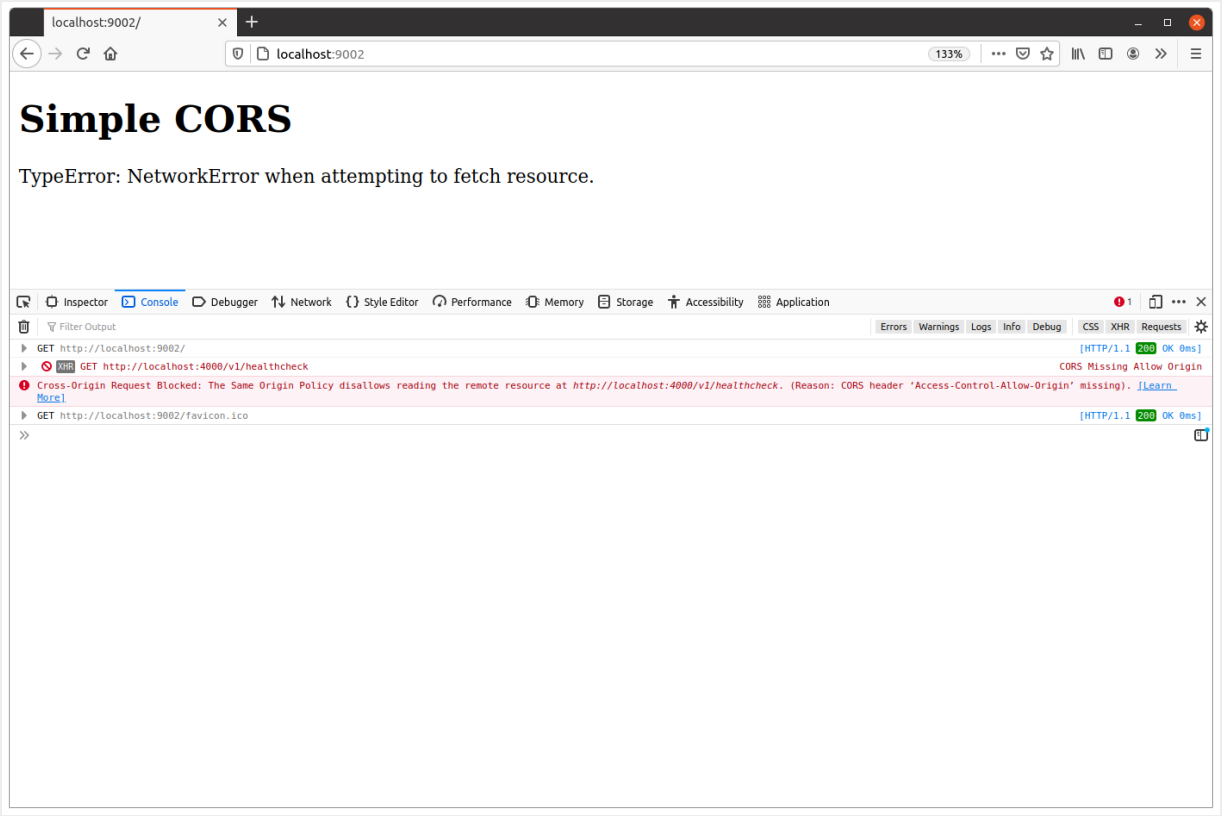
If you want, you can also try running the `cmd/examples/cors/simple` application with `:9001` as the server address, and you should find that the cross-origin request works from that too.

In contrast, try running the `cmd/examples/cors/simple` application with the address `:9002`.

```
$ go run ./cmd/examples/cors/simple --addr=":9002"
2026/09/13 13:36:29 starting server on :9002
```

This will give the webpage an origin of `http://localhost:9002` — which isn't one of our trusted origins — so when you visit `http://localhost:9002` in your browser you should find that the cross-origin request is blocked. Like so:

dego_vila@yahoo.com 30 Oct 2026



Additional information

Partial origin matches

If you have a lot of trusted origins that you want to support, then you might be tempted to check for a partial match on the origin to see if it 'starts with' or 'ends with' a specific value, or matches a regular expression. If you do this, you must be careful to avoid any unintentional matches.

As a simple example, if `http://example.com` and `http://www.example.com` are your trusted origins, your first thought might check that the request `Origin` header ends with `example.com`. This would be a bad idea, as an attacker could register the domain name `attackerexample.com` and any requests from that origin would pass your check.

This is just one simple example — and the following blog posts discuss some of the other vulnerabilities that can arise when using partial match or regular expression checks:

- Security Risks of CORS
- Exploiting CORS misconfigurations for Bitcoins and bounties

dego_vila@yahoo.com 30 Oct 2026

Generally, it's best to check the `Origin` request header against an explicit safelist of full-length trusted origins, like we have done in this chapter.

The null origin

It's important to never include the value `"null"` as a trusted origin in your safelist. This is because the request header `Origin: null` can be forged by an attacker by sending a request from a sandboxed iframe.

Authentication and CORS

If your API endpoint requires *credentials* (cookies or HTTP basic authentication) you should also set an `Access-Control-Allow-Credentials: true` header in your responses. If you don't set this header, then the web browser will prevent any cross-origin responses with credentials from being read by JavaScript.

Importantly, you must never use the wildcard `Access-Control-Allow-Origin: *` header in conjunction with `Access-Control-Allow-Credentials: true`, as this would allow any website to make a credentialed cross-origin request to your API.

Also, importantly, if you want credentials to be sent with a cross-origin request then you'll need to explicitly specify this in your JavaScript. For example, with `fetch()` you should set the `credentials` value of the request to `'include'`. Like so:

```
fetch("https://api.example.com", {credentials: 'include'}).then( ... );
```

Or if using `XMLHTTPRequest` you should set the `withCredentials` property to `true`. For example:

```
var xhr = new XMLHttpRequest();
xhr.open('GET', 'https://api.example.com');
xhr.withCredentials = true;
xhr.send(null);
```

dego_vila@yahoo.com 30 Oct 2026

Preflight CORS requests

The cross-origin request that we made from JavaScript in the previous chapter is known as a *simple* cross-origin request. Broadly speaking, cross-origin requests are classified as 'simple' when *all* the following conditions are met:

- The request HTTP method is one of the three CORS-safe methods: `HEAD`, `GET` or `POST`.
- The request headers are all either forbidden headers or one of the four CORS-safe headers:
  - `Accept`
  - `Accept-Language`
  - `Content-Language`
  - `Content-Type`
- The value for the `Content-Type` header (if set) is one of:
  - `application/x-www-form-urlencoded`
  - `multipart/form-data`
  - `text/plain`

When a cross-origin request doesn't meet these conditions, the web browser will trigger an initial 'preflight' request *before sending the real request*. The purpose of this preflight request is to determine whether the *real* cross-origin request will be permitted or not.

Demonstrating a preflight request

To help demonstrate how preflight requests work and what we need to do to deal with them, let's create another example webpage under the `cmd/examples/cors/` directory.

We'll set up this webpage so it makes a request to our `POST /v1/tokens/authentication` endpoint. When calling this endpoint we'll include an email address and password in a JSON request body, along with a `Content-Type: application/json` header. And because the header `Content-Type: application/json` isn't allowed in a 'simple' cross-origin request, this should trigger a preflight request to our API.

Go ahead and create a new file at `cmd/examples/cors/preflight/main.go`:

dego_vila@yahoo.com 30 Oct 2026

```
$ mkdir -p cmd/examples/cors/preflight
$ touch cmd/examples/cors/preflight/main.go
```

And add the code below, which follows a very similar pattern to the one we used a couple of chapters ago.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/examples/cors/preflight/main.go

```
package main

import (
    "flag"
    "log"
    "net/http"
)

// Define a string constant containing the HTML for the webpage. This consists of a <h1>
// header tag, and some JavaScript which calls our POST /v1/tokens/authentication
// endpoint and writes the response body inside the <div id="output"></div> tag.
const html = `
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
</head>
<body>
    <h1>Preflight CORS</h1>
    <div id="output"></div>
    <script>
        document.addEventListener('DOMContentLoaded', function() {
            fetch("http://localhost:4000/v1/tokens/authentication", {
                method: "POST",
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify({
                    email: 'alice@example.com',
                    password: 'pa55word'
                })
            }).then(
                function (response) {
                    response.text().then(function (text) {
                        document.getElementById("output").innerHTML = text;
                    });
                },
                function(err) {
                    document.getElementById("output").innerHTML = err;
                }
            );
        });
    </script>
</body>
</html>`

func main() {
    addr := flag.String("addr", ":9000", "Server address")
    flag.Parse()

    log.Printf("starting server on %s", *addr)

    err := http.ListenAndServe(*addr, http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte(html))
    }))
    log.Fatal(err)
}
```

If you’re following along, go ahead and run this application:

dego_vila@yahoo.com 30 Oct 2026

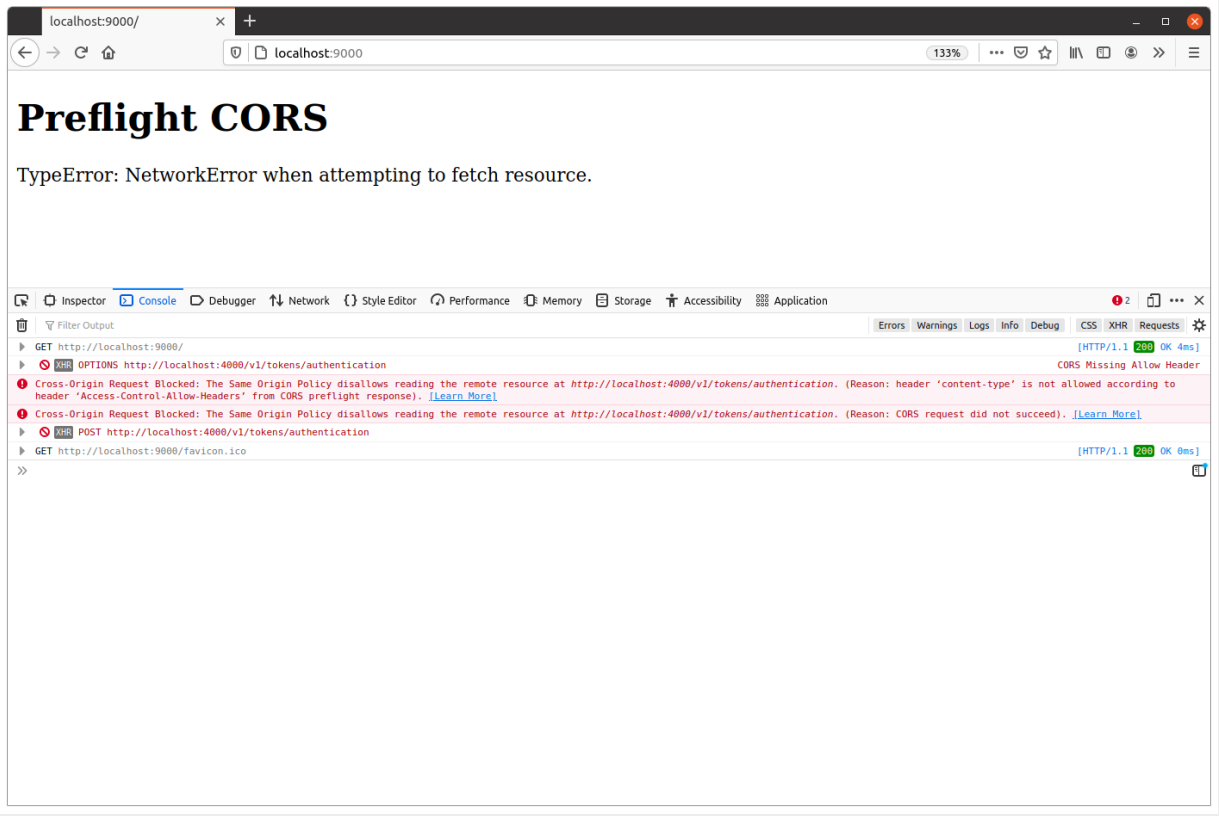
```
$ go run ./cmd/examples/cors/preflight
2026/09/13 13:54:30 starting server on :9000
```

Then open a second terminal window and start our regular API application at the same time with `http://localhost:9000` as a trusted origin:

```
$ go run ./cmd/api -cors-trusted-origins="http://localhost:9000"
time=2026-09-13T13:54:56.951+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T13:54:56.952+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Once both are running, open your web browser and navigate to `http://localhost:9000`. If you look at the console log in your developer tools, you should see a message similar to this:

> *Cross-Origin Request Blocked: The Same Origin Policy disallows reading the remote resource at http://localhost:4000/v1/tokens/authentication. (Reason: header 'content-type' is not allowed according to header 'Access-Control-Allow-Headers' from CORS preflight response).*



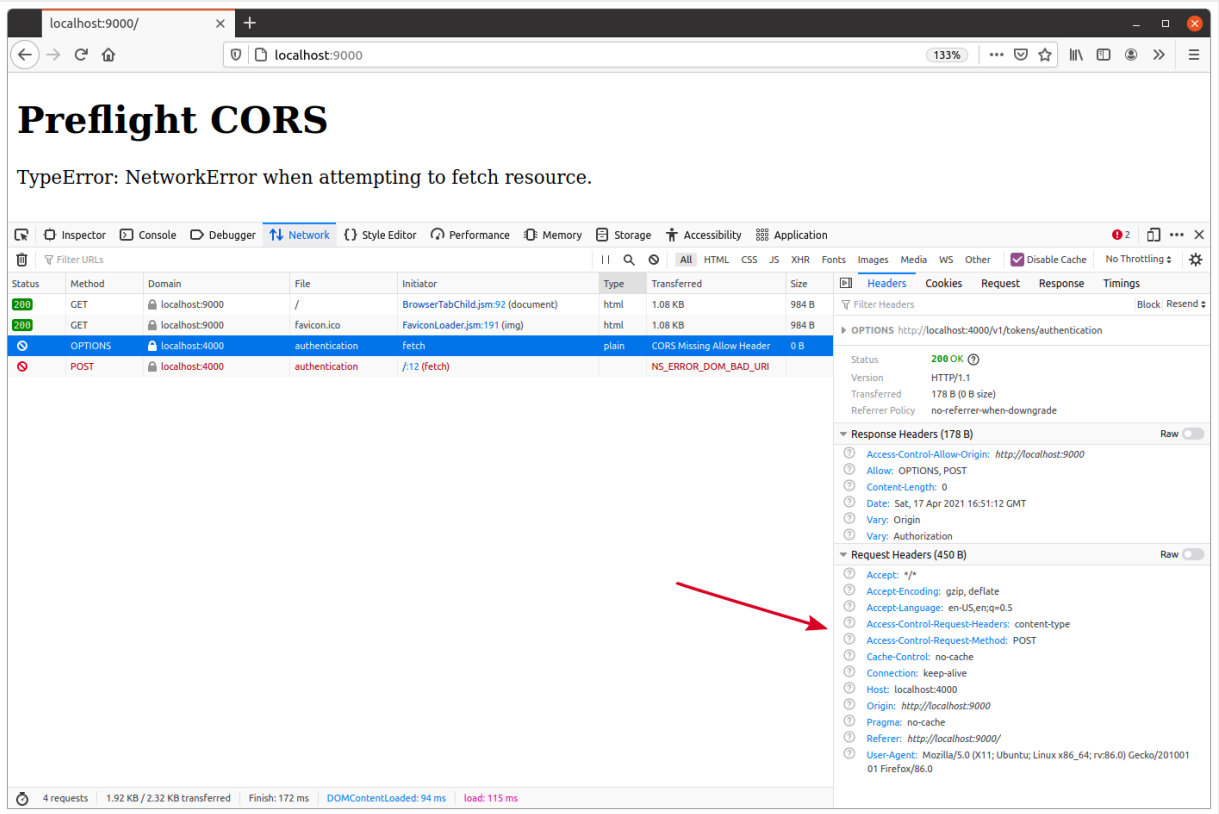
We can see that there are two requests here marked as 'blocked' by the browser:

- An `OPTIONS /v1/tokens/authentication` request (this is the preflight request).

dego_vila@yahoo.com 30 Oct 2026

- A `POST /v1/tokens/authentication` request (this is the 'real' request).

Let's take a closer look at the preflight request in the network tab of the developer tools:



The interesting thing here is the preflight request headers. They might look slightly different for you depending on the browser you're using, but broadly they should look something like this:

```
Accept: */*
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.5
Access-Control-Request-Headers: content-type
Access-Control-Request-Method: POST
Cache-Control: no-cache
Connection: keep-alive
Host: localhost:4000
Origin: http://localhost:9000
Pragma: no-cache
Referer: http://localhost:9000/
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:86.0) Gecko/20100101 Firefox/86.0
```

There are three headers here which are relevant to CORS:

- `Origin` — As we saw previously, this lets our API know what origin the preflight request is coming from.
- `Access-Control-Request-Method` — This lets our API know what HTTP method will be

dego_vila@yahoo.com 30 Oct 2026

used for the real request (in this case, we can see that the real request will be a `POST`).

- `Access-Control-Request-Headers` — This lets our API know what HTTP headers will be sent with the real request (in this case we can see that the real request will include a `content-type` header).

It's important to note that `Access-Control-Request-Headers` won't list *all* the headers that the real request will use. Only headers that are *not* CORS-safe or forbidden will be listed. If there are no such headers, then `Access-Control-Request-Headers` may be omitted from the preflight request entirely.

Responding to preflight requests

In order to respond to a preflight request, the first thing we need to do is identify that it *is* a preflight request — rather than just a regular (possibly even cross-origin) `OPTIONS` request.

To do that, we can leverage the fact that preflight requests always have three components: the HTTP method `OPTIONS`, an `Origin` header, and an `Access-Control-Request-Method` header. If any one of these pieces is missing, we know that it is not a preflight request.

Once we identify that it is a preflight request, we need to send a `200 OK` response with some special headers to let the browser know whether or not it's OK for the real request to proceed. These are:

- An `Access-Control-Allow-Origin` response header, which reflects the value of the preflight request's `Origin` header (just like in the previous chapter).
- An `Access-Control-Allow-Methods` header listing the HTTP methods that can be used in real cross-origin requests to the URL.
- An `Access-Control-Allow-Headers` header listing the request headers that can be included in real cross-origin requests to the URL.

In our case, we could set the following response headers to allow cross-origin requests for *all of our endpoints*:

```
Access-Control-Allow-Origin: <reflected trusted origin>
Access-Control-Allow-Methods: OPTIONS, PUT, PATCH, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
```

dego_vila@yahoo.com 30 Oct 2026

> **Important:** When responding to a preflight request it's not necessary to include the CORS-safe methods `HEAD`, `GET` or `POST` in the `Access-Control-Allow-Methods` header. Likewise, it's not necessary to include forbidden or CORS-safe headers in `Access-Control-Allow-Headers`.

When the web browser receives these headers, it compares the values to the method and (case-insensitive) headers that it wants to use in the real request. If the method or any of the headers are not allowed, then the browser will block the real request.

Updating our middleware

Let's put this into action and update our `enableCORS()` middleware so it intercepts and responds to any preflight requests. Specifically, we want to:

1. Set a `Vary: Access-Control-Request-Method` header on all responses, as the response will be different depending on whether or not this header exists in the request.
2. Check whether the request is a preflight cross-origin request or not. If it's not, then we should allow the request to proceed as normal.
3. Otherwise, if it is a preflight cross-origin request, then we should add the `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers` headers as described above and send a `200 OK` response.

Go ahead and update the `cmd/api/middleware.go` file like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/middleware.go

```
package main

...

func (app *application) enableCORS(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Add("Vary", "Origin")

        // Add the "Vary: Access-Control-Request-Method" header.
        w.Header().Add("Vary", "Access-Control-Request-Method")

        origin := r.Header.Get("Origin")

        if origin != "" {
            for i := range app.config.cors.trustedOrigins {
                if origin == app.config.cors.trustedOrigins[i] {
                    w.Header().Set("Access-Control-Allow-Origin", origin)

                    // Check if the request has the HTTP method OPTIONS and contains the
                    // "Access-Control-Request-Method" header. If it does, then we treat
                    // it as a preflight request.
                    if r.Method == http.MethodOptions && r.Header.Get("Access-Control-Request-Method") != "" {
                        // Set the necessary preflight response headers, as discussed
                        // previously.
                        w.Header().Set("Access-Control-Allow-Methods", "OPTIONS, PUT, PATCH, DELETE")
                        w.Header().Set("Access-Control-Allow-Headers", "Authorization, Content-Type")

                        // Write the headers along with a 200 OK status and return from
                        // the middleware with no further action.
                        w.WriteHeader(http.StatusOK)
                        return
                    }

                    break
                }
            }
        }

        next.ServeHTTP(w, r)
    })
}
```

There are a couple of additional things to point out here:

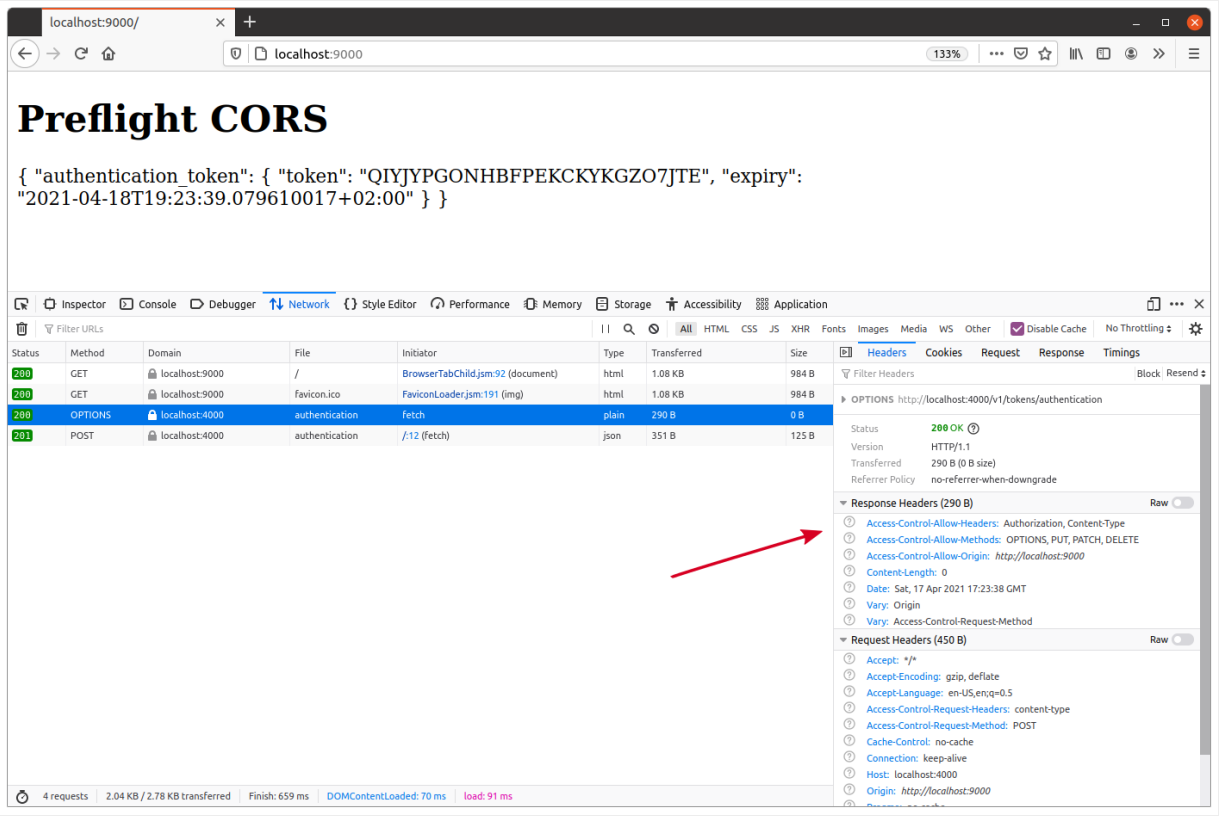
- When we respond to a preflight request we deliberately send the HTTP status `200 OK` rather than `204 No Content` — even though there is no response body. This is because certain browser versions may not support `204 No Content` responses and subsequently block the real request.
- If you allow the `Authorization` header in cross-origin requests, like we are in the code above, it's important to not set the wildcard `Access-Control-Allow-Origin: *` header or reflect the `Origin` header without checking against a list of trusted origins. Otherwise, this would leave your service vulnerable to a distributed brute-force attack against any authentication credentials that are passed in that header.

dego_vila@yahoo.com 30 Oct 2026

OK, let's try this out. Restart your API, again setting `http://localhost:9000` as a trusted origin like so:

```
$ go run ./cmd/api -cors-trusted-origins="http://localhost:9000"
time=2026-09-13T14:00:42.866+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T14:00:42.866+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Then open http://localhost:9000 in your browser again. This time you should see that the cross-origin `fetch()` to `POST /v1/tokens/authentication` succeeds, and you now get an authentication token in the response. Similar to this:



> **Note:** If you look at the details for the preflight request, you should see that our new CORS headers have been set on the preflight response, shown by the red arrow in the screenshot above.

Additional information

Caching preflight responses

dego_vila@yahoo.com 30 Oct 2026

If you want, you can also add an `Access-Control-Max-Age` header to your preflight responses. This indicates the number of seconds that the information provided by the `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers` headers can be cached by the browser.

For example, to allow the values to be cached for 60 seconds you can set the following header on your preflight response:

```
Access-Control-Max-Age: 60
```

If you don't set an `Access-Control-Max-Age` header, current versions of Chrome/Chromium and Firefox will default to caching these preflight response values for 5 seconds. Older versions or other browsers may have different defaults, or not cache the values at all.

Setting a long `Access-Control-Max-Age` duration might seem like an appealing way to reduce requests to your API — and it is! But you also need to be careful. Not all browsers provide a way to clear the preflight cache, so if you send back the wrong headers the user will be stuck with them until the cache expires.

If you want to disable caching altogether, you can set the value to `-1`:

```
Access-Control-Max-Age: -1
```

It's also important to be aware that browsers may impose a hard maximum on how long the headers can be cached for. The MDN documentation says:

> - *Firefox caps this at 24 hours (86400 seconds).*
> - *Chromium (prior to v76) caps at 10 minutes (600 seconds).*
> - *Chromium (starting in v76) caps at 2 hours (7200 seconds).*

Preflight wildcards

If you have a complex or rapidly changing API then it might be awkward to maintain a hard-coded safelist of methods and headers for the preflight response. You might think: *I just want to allow all HTTP methods and headers for cross-origin requests*.

In this case, both the `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers` headers allow you to use a wildcard `*` character like so:

dego_vila@yahoo.com 30 Oct 2026

```
Access-Control-Allow-Methods: *
Access-Control-Allow-Headers: *
```

But using these comes with some important caveats:

- Wildcards in these headers are currently only supported by 95% of browsers. Any browsers which don't support them will block the preflight request.
- The `Authorization` header cannot be wildcarded. Instead, you will need to include this explicitly in the header like `Access-Control-Allow-Headers: Authorization, *`.
- Wildcards are not supported for credentialed requests (those with cookies or HTTP basic authentication). For these, the character `*` will be treated as the literal string `"*"`, rather than as a wildcard.

dego_vila@yahoo.com 30 Oct 2026

Chapter 18.

Metrics

When your application is running in production with real-world traffic — or if you're carrying out targeted load testing — you might want some up-close insight into how it's performing and what resources it's using.

For example, you might want to answer questions like:

- How much memory is my application using? How is this changing over time?
- How many goroutines are currently in use? How is this changing over time?
- How many database connections are in use and how many are idle? Do I need to change the connection pool settings?
- What is the ratio of successful HTTP responses to both client and server errors? Are error rates elevated above normal?

Having insight into these things can help inform your hardware and configuration choices, and act as an early warning sign of potential problems (such as memory leaks).

To assist with this, Go's standard library includes the `expvar` package which makes it easy to collate and view different application metrics at runtime.

In this section you'll learn:

- How to use the `expvar` package to view application metrics in JSON format via an HTTP handler.
- What default application metrics are available, and how to create your own custom application metrics for monitoring the number of active goroutines and the database connection pool.
- How to use middleware to monitor request-level application metrics, including the counts of different HTTP response status codes.

dego_vila@yahoo.com 30 Oct 2026

Exposing metrics with expvar

Viewing metrics for our application is made easy by the fact that the `expvar` package provides an `expvar.Handler()` function which returns an *HTTP handler exposing your application metrics*.

By default this handler displays information about memory usage, along with a reminder of what command-line flags you used when starting the application, all outputted in JSON format.

So the first thing that we're going to do is mount this handler at a new `GET /debug/vars` endpoint, like so:

Method	URL Pattern	Handler	Action
…	…	…	…
GET	/debug/vars	expvar.Handler()	Display application metrics

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

import (
    "expvar" // New import
    "net/http"

    "github.com/julienschmidt/httprouter"
)

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requirePermission("movies:read", app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requirePermission("movies:write", app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requirePermission("movies:read", app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requirePermission("movies:write", app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requirePermission("movies:write", app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    // Register a new GET /debug/vars endpoint pointing to the expvar handler.
    router.Handler(http.MethodGet, "/debug/vars", expvar.Handler())

    return app.recoverPanic(app.enableCORS(app.rateLimit(app.authenticate(router))))
}
```

> **Note:** Using the endpoint `GET /debug/vars` for the `expvar` handler is *conventional* but certainly not necessary. If you prefer, it's perfectly fine to register it at an alternative endpoint like `GET /v1/metrics` instead.

OK, let's try this out.

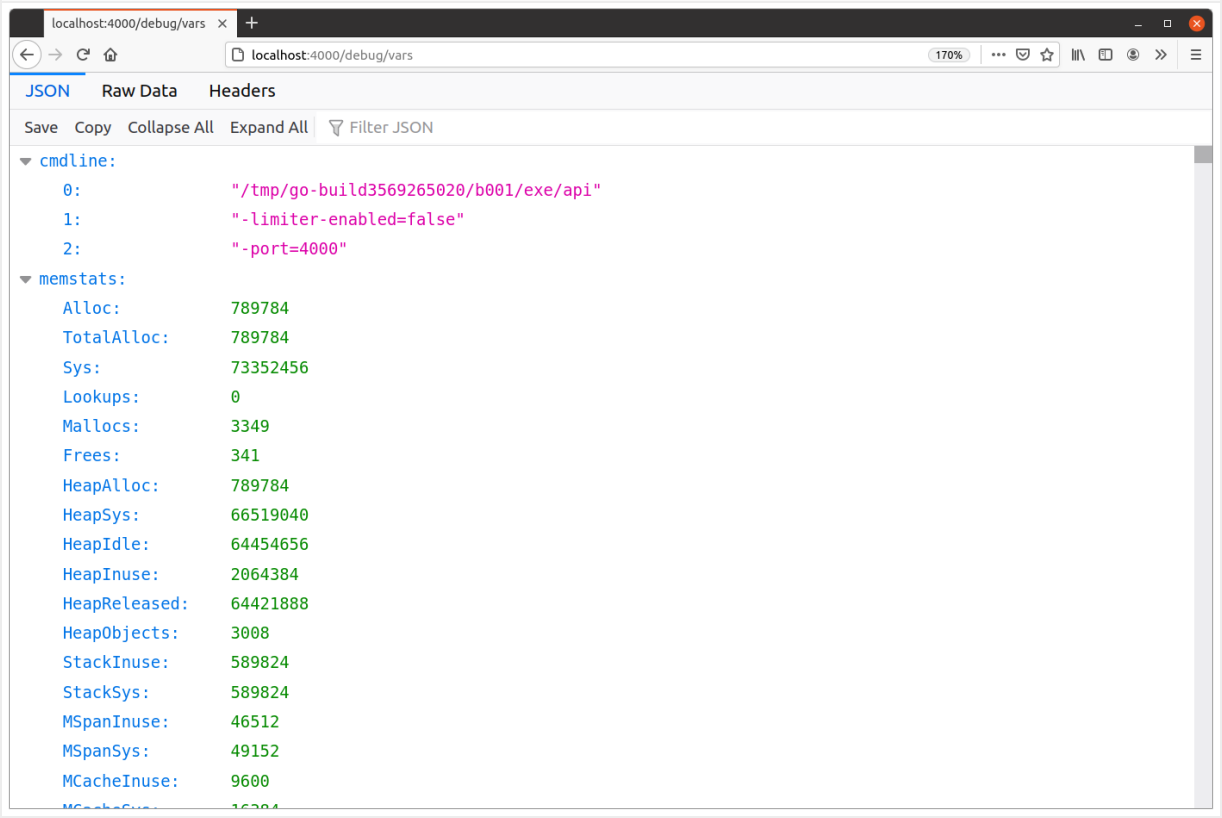
Go ahead and restart the API, passing in a couple of command-line flags for demonstration purposes. Like so:

```
$ go run ./cmd/api -limiter-enabled=false -port=4000
time=2026-09-13T14:05:51.303+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T14:05:51.303+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

And if you visit `http://localhost:4000/debug/vars` in your web browser, you should see a JSON response containing information about your running application.

dego_vila@yahoo.com 30 Oct 2026

In my case, the response looks like this:



We can see that the JSON here currently contains two top-level items: `"cmdline"` and `"memstats"`. Let's quickly talk through what these represent.

The `"cmdline"` item contains an array of the command-line arguments used to run the application, beginning with the program name. This is essentially a JSON representation of the `os.Args` variable, and it's useful if you want to see exactly what non-default settings were used when starting the application.

The `"memstats"` item contains a 'moment-in-time' snapshot of memory usage, as returned by the `runtime.MemStats()` function. Documentation and descriptions for all of the values can be found here, but the most important ones are:

- `TotalAlloc` — Cumulative bytes allocated on the heap (will not decrease).
- `HeapAlloc` — Current number of bytes on the heap.
- `HeapObjects` — Current number of objects on the heap.
- `Sys` — Total bytes of memory obtained from the OS (i.e. total memory reserved by the Go runtime for the heap, stacks, and other internal data structures).
- `NumGC` — Number of completed garbage collector cycles.
- `NextGC` — The target heap size of the next garbage collector cycle (Go aims to keep `HeapAlloc` ≤ `NextGC`).

dego_vila@yahoo.com 30 Oct 2026

> **Hint:** If any of the terms above are unfamiliar to you, then I strongly recommend reading the Understanding Allocations in Go blog post which provides a great introduction to how Go allocates memory and the concepts of the heap and the stack.

dego_vila@yahoo.com 30 Oct 2026

Creating custom metrics

The default information exposed by the `expvar` handler is a good start, but we can make it even more useful by exposing some additional custom metrics in the JSON response.

To illustrate this, we'll start really simple and first expose our application version number in the JSON. If you don't remember, the version number is currently defined as the string constant `"1.0.0"` in our `main.go` file.

The code to do this breaks down into two basic steps: first we need to register a custom variable with the `expvar` package, and then we need to set the value for the variable itself. In one line, the code looks roughly like this:

```
expvar.NewString("version").Set(version)
```

The first part of this — `expvar.NewString("version")` — creates a new `expvar.String` type, then *publishes it* so it appears in the `expvar` handler's JSON response with the name `"version"`, and then returns a pointer to it. Then we use the `Set()` method on it to assign an actual value to the pointer.

Two other things to note:

- The `expvar.String` type is safe for concurrent use. So — if you want to — it's OK to manipulate this value at runtime from your application handlers.
- If you try to register two `expvar` variables with the same name, you'll get a runtime panic when the duplicate variable is registered.

Let's go ahead and integrate this code into our `main()` function, like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "expvar" // New import
    "flag"
    "log/slog"
    "os"
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/mailer"
    "greenlight.alexedwards.net/internal/models"

    _ "github.com/lib/pq"
)

// Remember, our version number is just a constant string (for now).
const version = "1.0.0"

...

func main() {
    ...

    // Publish a new "version" variable in the expvar handler containing our application
    // version number (currently the constant "1.0.0").
    expvar.NewString("version").Set(version)

    app := &application{
        config: cfg,
        logger: logger,
        models: models.NewModels(db),
        mailer: mailer,
    }

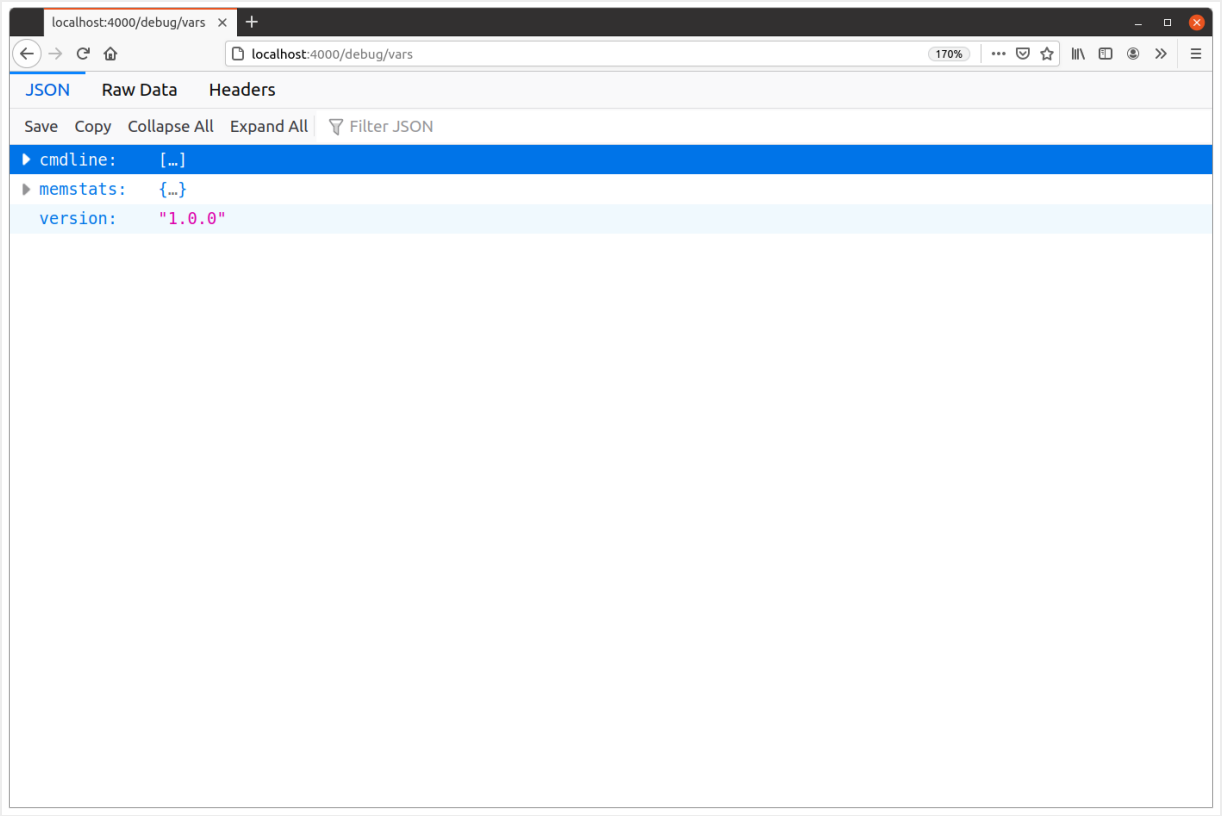
    err = app.serve()
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
}

...
```

If you restart the API and visit `http://localhost:4000/debug/vars` in your web browser again, you should now see a `"version": "1.0.0"` item in the JSON.

Similar to this:

dego_vila@yahoo.com 30 Oct 2026



> **Note:** In the code above we used the `expvar.NewString()` function to register and publish a string in the `expvar` handler. But Go also provides functions for a few other common data types: `NewFloat()`, `NewInt()` and `NewMap()`. All these work in a very similar way, and we'll put them to use in the next chapter.

Dynamic metrics

Occasionally you might want to publish metrics which require you to call other code — or do some kind of pre-processing — to generate the necessary information. To help with this there is the `expvar.Publish()` function, which allows you to publish the *result of a function* in the JSON output.

For example, if you want to publish the number of currently active goroutines from Go's `runtime.NumGoroutine()` function, you could write the following code:

```
expvar.Publish("goroutines", expvar.Func(func() any {
    return runtime.NumGoroutine()
}))
```

It's important to point out here that the `any` value returned from this function *must* encode

dego_vila@yahoo.com 30 Oct 2026

to JSON without any errors. If it can't be encoded to JSON, then it will be omitted from the `expvar` output and the response from the `GET /debug/vars` endpoint will be malformed. Any error will be silently discarded.

In the case of the code snippet above, `runtime.NumGoroutine()` returns a regular `int` type — which will encode to a JSON number. So there's no problem with that here.

OK, let's add this code to our `main()` function, along with two other functions which:

- Publish information about the state of our database connection pool (such as the number of idle and in-use connections) via the `db.Stats()` method.
- Publish the current Unix timestamp with second precision.

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "expvar"
    "flag"
    "log/slog"
    "os"
    "runtime" // New import
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/mailer"
    "greenlight.alexedwards.net/internal/models"

    _ "github.com/lib/pq"
)

...

func main() {
    ...

    expvar.NewString("version").Set(version)

    // Publish the number of active goroutines.
    expvar.Publish("goroutines", expvar.Func(func() any {
        return runtime.NumGoroutine()
    }))

    // Publish the database connection pool statistics.
    expvar.Publish("database", expvar.Func(func() any {
        return db.Stats()
    }))

    // Publish the current Unix timestamp.
    expvar.Publish("timestamp", expvar.Func(func() any {
        return time.Now().Unix()
    }))

    app := &application{
        config: cfg,
        logger: logger,
        models: models.NewModels(db),
        mailer: mailer,
    }

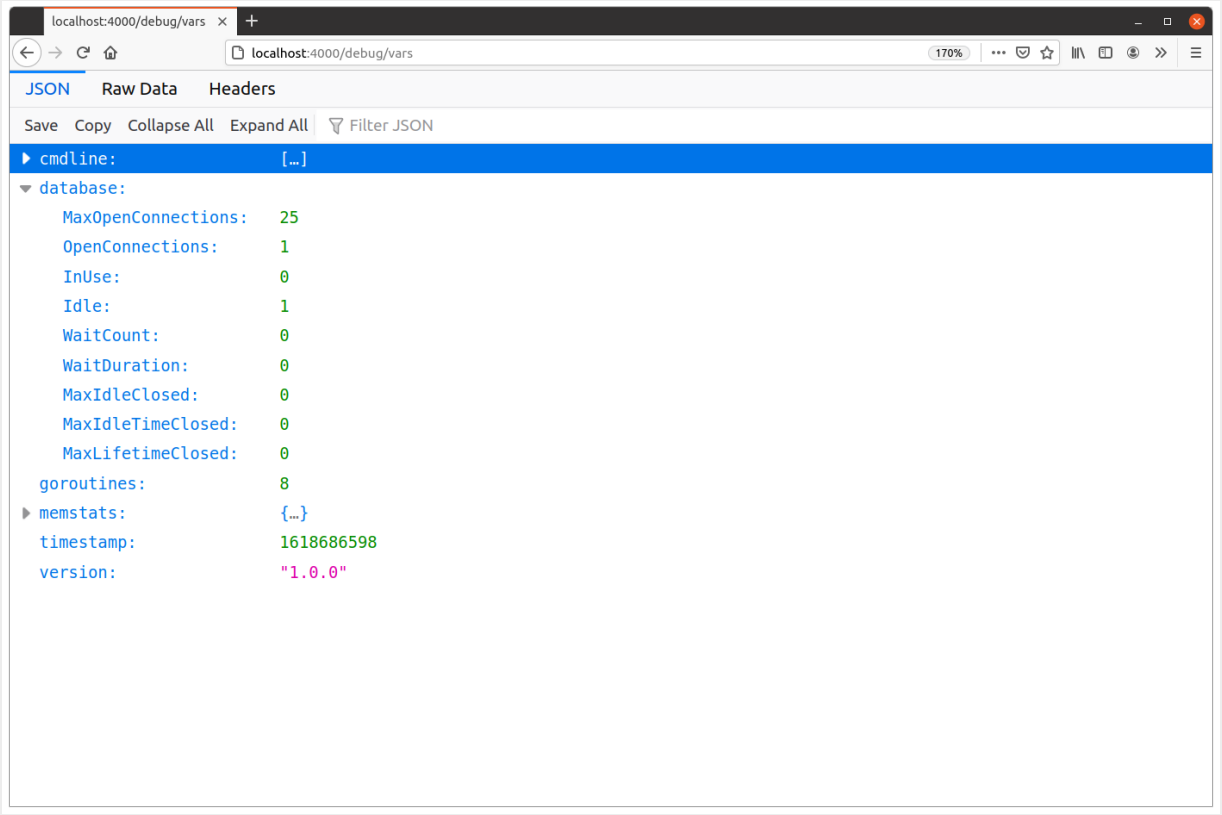
    err = app.serve()
    if err != nil {
        logger.Error(err.Error())
        os.Exit(1)
    }
}

...
```

If you restart the API and open the `GET /debug/vars` endpoint in your browser again, you should now see the additional `"database"`, `"goroutines"` and `"timestamp"` items in the

dego_vila@yahoo.com 30 Oct 2026

JSON. Like so:



In my case, I can see that the application currently has 8 active goroutines, and the database connection pool is in its 'initial' state with just one currently idle connection (which was created when our code called `db.PingContext()` on startup).

If you like, you can use a tool like `hey` to generate some requests to your application and see how these figures change under load.

Go ahead and run the application *with rate limiting disabled*, like so:

```
$ go run ./cmd/api/ -limiter-enabled=false
time=2026-09-13T20:25:28.115+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:25:28.115+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Then use `hey` to send a batch of requests to the `POST /v1/tokens/authentication` endpoint (which is slow and costly because it checks a bcrypt-hashed password), like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ BODY='{"email": "alice@example.com", "password": "pa55word"}'
$ hey -d "$BODY" -m "POST" http://localhost:4000/v1/tokens/authentication

Summary:
  Total:        8.0979 secs
  Slowest:      2.4612 secs
  Fastest:      1.6169 secs
  Average:      1.9936 secs
  Requests/sec: 24.6977

  Total data:   24975 bytes
  Size/request: 124 bytes

Response time histogram:
  1.617 [1]  |■
  1.701 [6]  |■■■■■
  1.786 [10] |■■■■■■■■■
  1.870 [26] |■■■■■■■■■■■■■■■■■■■■■■■■
  1.955 [36] |■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  2.039 [46] |■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  2.123 [36] |■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  2.208 [21] |■■■■■■■■■■■■■■■■■■■
  2.292 [12] |■■■■■■■■■■■
  2.377 [4]  |■■■
  2.461 [2]  |■■

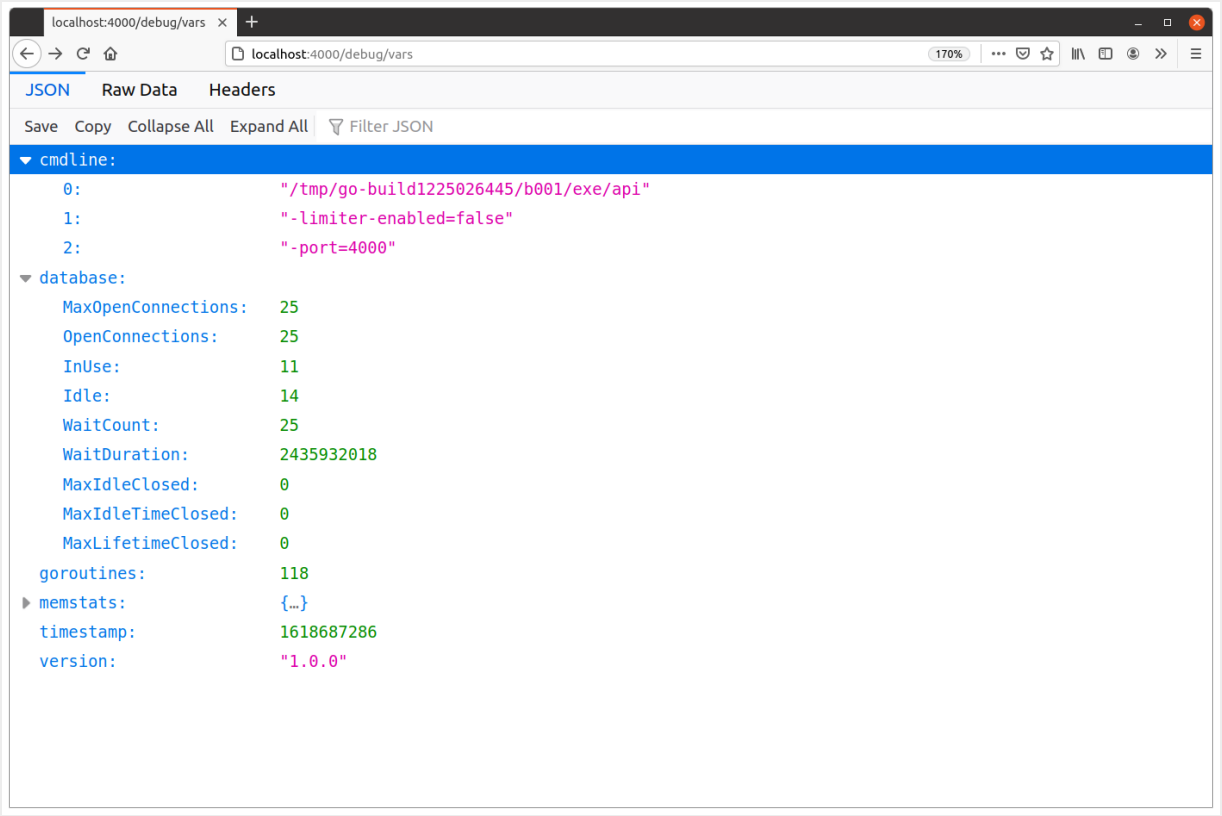

Latency distribution:
  10% in 1.8143 secs
  25% in 1.8871 secs
  50% in 1.9867 secs
  75% in 2.1000 secs
  90% in 2.2017 secs
  95% in 2.2642 secs
  99% in 2.3799 secs

Details (average, fastest, slowest):
  DNS+dialup: 0.0009 secs, 1.6169 secs, 2.4612 secs
  DNS-lookup: 0.0005 secs, 0.0000 secs, 0.0030 secs
  req write: 0.0002 secs, 0.0000 secs, 0.0051 secs
  resp wait: 1.9924 secs, 1.6168 secs, 2.4583 secs
  resp read: 0.0000 secs, 0.0000 secs, 0.0001 secs

Status code distribution:
  [201] 200 responses
```

If you visit the `GET /debug/vars` endpoint while the `hey` tool is running, you should see that your application metrics now look quite different:

dego_vila@yahoo.com 30 Oct 2026



At the moment I took this screenshot, we can see that my API application had 118 active goroutines, with 11 database connections in use and 14 connections sitting idle.

There are a couple of other interesting things to point out too.

The database `WaitCount` figure of 25 is the total number of times that our application had to wait for a database connection to become available in our `sql.DB` pool (because all connections were in-use). Likewise, `WaitCountDuration` is the cumulative amount of time (in nanoseconds) spent waiting for a connection. From these, it's possible to calculate that *when our application did have to wait for a database connection, the average wait time was approximately 98 milliseconds*. Ideally, you want to be seeing zeroes or very low numbers for these two things under normal load in production.

Also, the `MaxIdleTimeClosed` figure is the total count of the number of connections that have been closed because they reached their `ConnMaxIdleTime` limit (which in our case is set to 15 minutes by default). If you leave the application running but don't use it, and come back in 15 minutes time, you should see that the number of open connections has dropped to zero and the `MaxIdleTimeClosed` count has increased accordingly.

You might like to play around with this and try changing some of the configuration parameters for the connection pool to see how it affects the behavior of these figures under load. For example:

dego_vila@yahoo.com 30 Oct 2026

```
$ go run ./cmd/api -limiter-enabled=false -db-max-open-conns=50 -db-max-idle-conns=50 -db-max-idle-time=20s -port=4000
```

Additional information

Protecting the metrics endpoint

It's important to be aware that these metrics provide very useful information to anyone who wants to perform a denial-of-service attack against your application, and that the `"cmdline"` values may also expose potentially sensitive information (like a database DSN).

So you should make sure to restrict access to the `GET /debug/vars` endpoint when running in a production environment.

There are a few different approaches you could take to do this.

One option is to leverage our existing authentication process and create a `metrics:view` permission so that only certain trusted users can access the endpoint. Another option would be to use HTTP Basic Authentication to restrict access to the endpoint.

In our case, when we deploy our application in production later we will run it behind Caddy as a reverse proxy. As part of our Caddy set up, we'll restrict access to the `GET /debug/vars` endpoint so that it can only be accessed via connections from the local machine, rather than being exposed on the internet.

Removing default metrics

It's currently not possible to remove the default `"cmdline"` and `"memstats"` items from the `expvar` handler, even if you want to. There's an open issue regarding this, and hopefully it will become possible to omit these in a future version of Go.

dego_vila@yahoo.com 30 Oct 2026

Request-level metrics

In this chapter we're going to create some new middleware to record custom *request-level* metrics for our application. We'll start by recording the following three things:

- The total number of requests received.
- The total number of responses sent.
- The total (cumulative) time taken to process all requests in microseconds.

All these values will be integers, so we'll be able to register these metrics with the `expvar` package using the `expvar.NewInt()` function.

Let's jump straight into the code and create a new `metrics()` middleware method, which initializes the necessary `expvar` variables and then updates them each time we process a request. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/middleware.go

```
package main

import (
    "errors"
    "expvar" // New import
    "fmt"
    "net/http"
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"

    "github.com/tomasen/realip"
    "golang.org/x/time/rate"
)

...

func (app *application) metrics(next http.Handler) http.Handler {
    // Initialize the new expvar variables when the middleware chain is first built.
    var (
        totalRequestsReceived           = expvar.NewInt("total_requests_received")
        totalResponsesSent              = expvar.NewInt("total_responses_sent")
        totalProcessingTimeMicroseconds = expvar.NewInt("total_processing_time_µs")
    )

    // The following code will be run for every request...
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Record the time that we started to process the request.
        start := time.Now()

        // Use the Add() method to increment the number of requests received by 1.
        totalRequestsReceived.Add(1)

        // Call the next handler in the chain.
        next.ServeHTTP(w, r)

        // On the way back up the middleware chain, increment the number of responses
        // sent by 1.
        totalResponsesSent.Add(1)

        // Calculate the number of microseconds since we began to process the request,
        // then increment the total processing time by this amount.
        duration := time.Since(start).Microseconds()
        totalProcessingTimeMicroseconds.Add(duration)
    })
}
```

Once that's done, we need to update the `cmd/api/routes.go` file to include the new `metrics()` middleware, right at the beginning of the chain. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requirePermission("movies:read", app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requirePermission("movies:write", app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requirePermission("movies:read", app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requirePermission("movies:write", app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requirePermission("movies:write", app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)

    router.Handler(http.MethodGet, "/debug/vars", expvar.Handler())

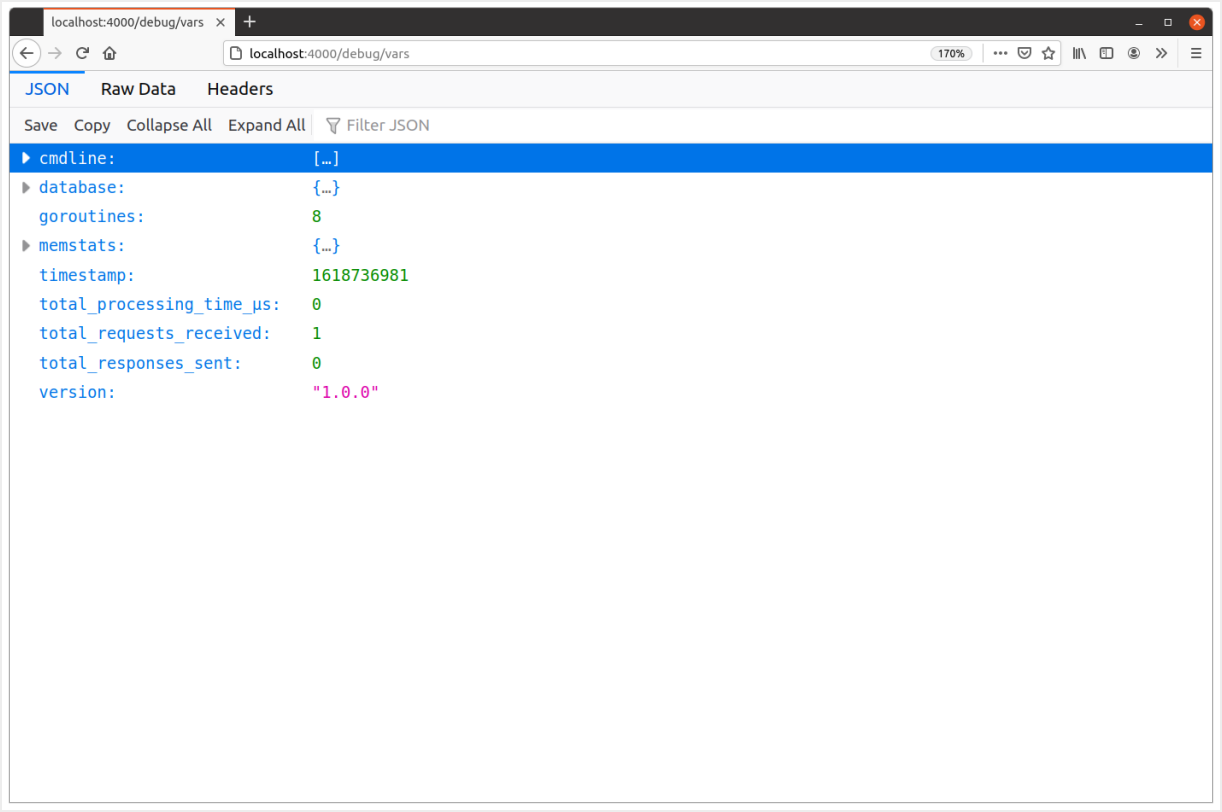
    // Use the new metrics() middleware at the start of the chain.
    return app.metrics(app.recoverPanic(app.enableCORS(app.rateLimit(app.authenticate(router)))))
}
```

OK, let's try this out. Go ahead and run the API with rate limiting disabled again:

```
$ go run ./cmd/api -limiter-enabled=false
time=2026-09-13T20:34:29.946+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:34:29.946+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

And when you visit `localhost:4000/debug/vars` in your browser, you should now see that the JSON response includes items for `total_requests_received`, `total_responses_sent` and `total_processing_time_µs`. Like so:

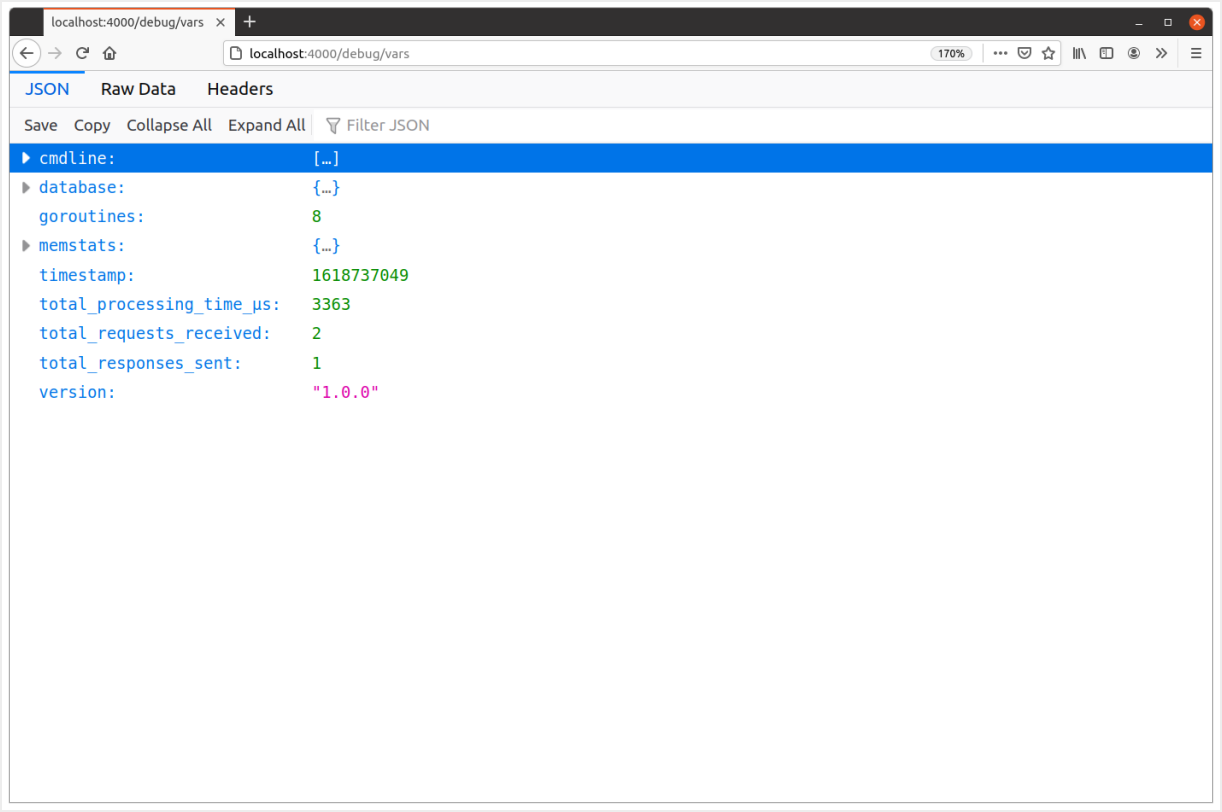
dego_vila@yahoo.com 30 Oct 2026



At this point, we can see that our API has received one request and sent zero responses. That makes sense if you think about it — at the moment that *this JSON response* was generated, it hadn't actually been sent.

If you refresh the page, you should see these numbers increment — including an increase in the `total_processing_time_µs` value:

dego_vila@yahoo.com 30 Oct 2026



Let's try this out under load and use `hey` to make some requests to the `POST /v1/tokens/authentication` endpoint again.

First restart the API:

```
$ go run ./cmd/api -limiter-enabled=false
time=2026-09-13T20:35:46.512+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:35:46.512+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

And then use `hey` to generate the load like so:

dego_vila@yahoo.com 30 Oct 2026

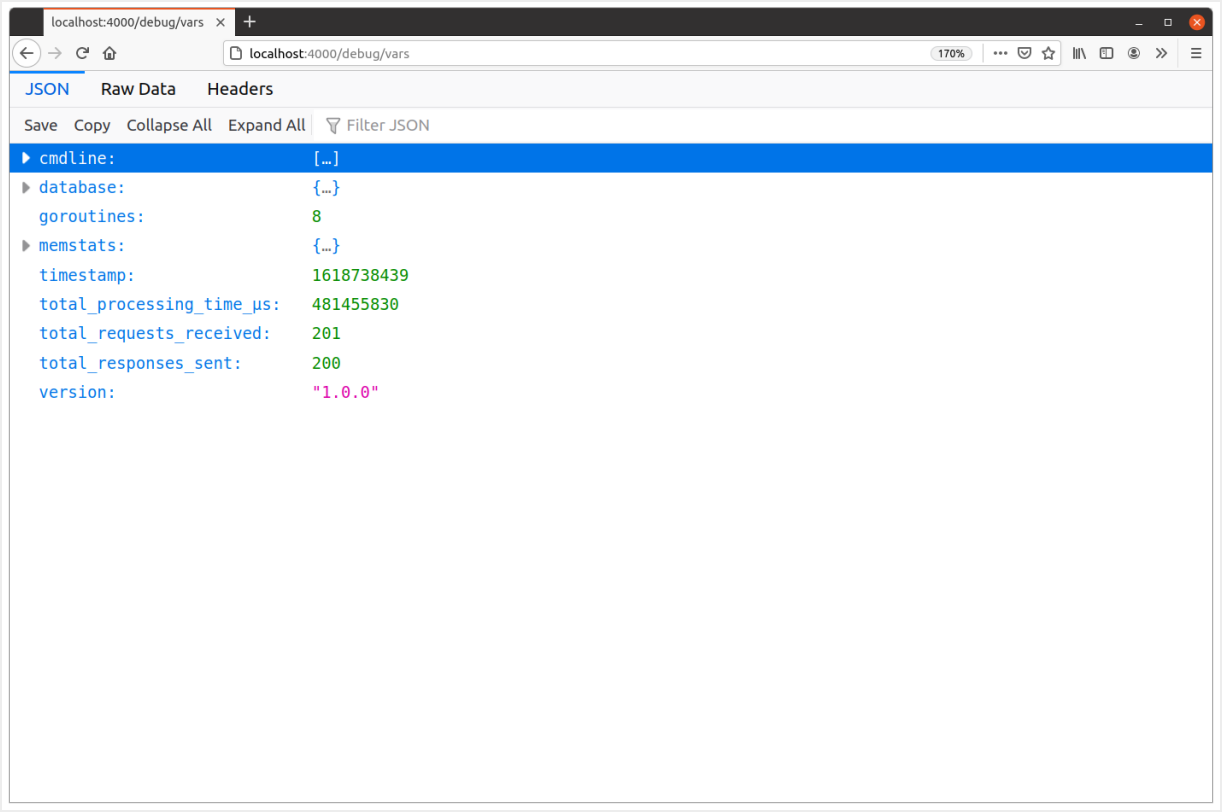
```
$ BODY='{"email": "alice@example.com", "password": "pa55word"}'
$ hey -d "$BODY" -m "POST" http://localhost:4000/v1/tokens/authentication

Summary:
  Total:        9.9141 secs
  Slowest:      3.0333 secs
  Fastest:      1.6788 secs
  Average:      2.4302 secs
  Requests/sec: 20.1732

  Total data:   24987 bytes
  Size/request: 124 bytes

Response time histogram:
  1.679 [1]  |■
  1.814 [1]  |■
  1.950 [3]  |■■■
  2.085 [8]  |■■■■■■■■
  2.221 [28] |■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  2.356 [39] |■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  2.491 [46] |■■■■■■■■■■■■■■■■■■■
```

dego_vila@yahoo.com 30 Oct 2026



Based on the `total_processing_time_µs` and `total_responses_sent` values, we can calculate that the average processing time per request was approximately 2.4 seconds (remember, this endpoint is deliberately slow and computationally expensive since we need to verify the bcrypted user password).

```
481455830 µs / 201 reqs =  2395303 µs/req = 2.4 s/req
```

This aligns with the data that we see in the results summary from `hey`, which gives the average response time as 2.4302 secs. The `hey` figure is the time for the complete round-trip, so it will always be a little higher as it includes network latency and the time taken for `http.Server` to manage the request.

Additional information

Calculating additional metrics

Based on the information in the `GET /debug/vars` response, you can also derive some additional interesting metrics. Such as…

dego_vila@yahoo.com 30 Oct 2026

- The number of ‘active’ in-flight requests:

```
total_requests_received - total_responses_sent
```

- The average number of requests received per second (between calls A and B to the `GET /debug/vars` endpoint):

```
(total_requests_received_B - total_requests_received_A) / (timestamp_B - timestamp_A)
```

- The average processing time per request (between calls A and B to the `GET /debug/vars` endpoint):

```
(total_processing_time_µs_B - total_processing_time_µs_A) / (total_requests_received_B - total_requests_received_A)
```

dego_vila@yahoo.com 30 Oct 2026

Recording HTTP status codes

As well as recording the total count of responses sent, we can take this further and extend our `metrics()` middleware to start tallying exactly which HTTP status codes our responses had.

The tricky part of doing this is finding out *what HTTP status code a response has* in our `metrics()` middleware. Unfortunately Go doesn't support this out-of-the-box — there is no built-in way to examine an `http.ResponseWriter` to see what status code is going to be sent to a client.

To capture the response status code, we'll instead need to create our own custom `http.ResponseWriter` that records a copy of the HTTP status code for future access.

Before Go 1.20 was released, doing this was brittle and awkward to get right. But now Go has the `http.ResponseController` type with support for `http.ResponseWriter` *unwrapping*, it's much more straightforward to implement yourself.

> **Hint:** If you need to work with an older version of Go, I highly recommend using the third-party `httpsnoop` package rather than implementing your own custom `http.ResponseWriter`.

Effectively, we want to create a struct that wraps an existing `http.ResponseWriter` and has custom `Write()` and `WriteHeader()` methods implemented on it which record the response status code. Importantly, we should also implement an `Unwrap()` method on it, which returns the original, wrapped, `http.ResponseWriter` value.

The code we want to write looks like this:

dego_vila@yahoo.com 30 Oct 2026

```
// The metricsResponseWriter type wraps an existing http.ResponseWriter and also
// contains a field for recording the response status code, and a boolean flag to
// indicate whether the response headers have already been written.
type metricsResponseWriter struct {
    wrapped       http.ResponseWriter
    statusCode    int
    headerWritten bool
}

// This function returns a new metricsResponseWriter instance which wraps a given
// http.ResponseWriter and has a status code of 200 (which is the status
// code that Go will send in an HTTP response by default).
func newMetricsResponseWriter(w http.ResponseWriter) *metricsResponseWriter {
    return &metricsResponseWriter{
        wrapped:    w,
        statusCode: http.StatusOK,
    }
}

// The Header() method is a simple 'pass through' to the Header() method of the
// wrapped http.ResponseWriter.
func (mw *metricsResponseWriter) Header() http.Header {
    return mw.wrapped.Header()
}

// Again, the WriteHeader() method does a 'pass through' to the WriteHeader()
// method of the wrapped http.ResponseWriter. But after this returns,
// we also record the response status code (if it hasn't already been recorded)
// and set the headerWritten field to true to indicate that the HTTP response
// headers have now been written.
func (mw *metricsResponseWriter) WriteHeader(statusCode int) {
    mw.wrapped.WriteHeader(statusCode)

    if !mw.headerWritten {
        mw.statusCode = statusCode
        mw.headerWritten = true
    }
}

// Likewise the Write() method does a 'pass through' to the Write() method of the
// wrapped http.ResponseWriter. Calling this will automatically write any
// response headers, so we set the headerWritten field to true.
func (mw *metricsResponseWriter) Write(b []byte) (int, error) {
    mw.headerWritten = true
    return mw.wrapped.Write(b)
}

// We also need an Unwrap() method which returns the existing wrapped
// http.ResponseWriter.
func (mw *metricsResponseWriter) Unwrap() http.ResponseWriter {
    return mw.wrapped
}
```

The important thing to note is that our `metricsResponseWriter` type *satisfies* the `http.ResponseWriter` interface. It has `Header()`, `WriteHeader()` and `Write()` methods with the appropriate signature, so we can use it in our handlers just like normal.

Also, notice that we don't record the status code until *after* the 'pass through' call in the `WriteHeader()` method. This is because a panic in that operation (potentially due to an invalid status code) may mean that a different status code is ultimately sent to the client.

dego_vila@yahoo.com 30 Oct 2026

Lastly, we also set a default status code of `200 OK` in the `newMetricsResponseWriter()` function. It's important that we set this default here, in case a handler doesn't ever call `Write()` or `WriteHeader()`.

But ultimately, our `metricsResponseWriter` is really just a light layer over the top of an existing `http.ResponseWriter` value.

Let's move on, and adapt our `metrics()` middleware to use this. We'll also need to publish a new `total_responses_sent_by_status` variable using the `expvar.NewMap()` function. This will give us a map in which we can store the different HTTP status codes, along with a running count of responses for each status.

Go ahead and update the `cmd/api/middleware.go` file as follows:

File: cmd/api/middleware.go

```
package main

import (
    "errors"
    "expvar"
    "fmt"
    "net/http"
    "strconv" // New import
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"

    "github.com/tomasen/realip"
    "golang.org/x/time/rate"
)

...

type metricsResponseWriter struct {
    wrapped       http.ResponseWriter
    statusCode    int
    headerWritten bool
}

func newMetricsResponseWriter(w http.ResponseWriter) *metricsResponseWriter {
    return &metricsResponseWriter{
        wrapped:    w,
        statusCode: http.StatusOK,
    }
}

func (mw *metricsResponseWriter) Header() http.Header {
    return mw.wrapped.Header()
}

func (mw *metricsResponseWriter) WriteHeader(statusCode int) {
    mw.wrapped.WriteHeader(statusCode)

    if !mw.headerWritten {
```

dego_vila@yahoo.com 30 Oct 2026

```
        mw.statusCode = statusCode
        mw.headerWritten = true
    }
}

func (mw *metricsResponseWriter) Write(b []byte) (int, error) {
    mw.headerWritten = true
    return mw.wrapped.Write(b)
}

func (mw *metricsResponseWriter) Unwrap() http.ResponseWriter {
    return mw.wrapped
}

func (app *application) metrics(next http.Handler) http.Handler {
    var (
        totalRequestsReceived           = expvar.NewInt("total_requests_received")
        totalResponsesSent              = expvar.NewInt("total_responses_sent")
        totalProcessingTimeMicroseconds = expvar.NewInt("total_processing_time_µs")

        // Declare a new expvar map to hold the count of responses for each HTTP status
        // code.
        totalResponsesSentByStatus = expvar.NewMap("total_responses_sent_by_status")
    )

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()

        totalRequestsReceived.Add(1)

        // Create a new metricsResponseWriter, which wraps the original
        // http.ResponseWriter value that the metrics middleware received.
        mw := newMetricsResponseWriter(w)

        // Call the next handler in the chain using the new metricsResponseWriter
        // as the http.ResponseWriter value.
        next.ServeHTTP(mw, r)

        totalResponsesSent.Add(1)

        // At this point, the response status code should be stored in the
        // mw.statusCode field. Note that the expvar map is string-keyed, so we
        // need to use the strconv.Itoa() function to convert the status code
        // (which is an integer) to a string. Then we use the Add() method on
        // our new totalResponsesSentByStatus map to increment the count for the
        // given status code by 1.
        totalResponsesSentByStatus.Add(strconv.Itoa(mw.statusCode), 1)

        duration := time.Since(start).Microseconds()
        totalProcessingTimeMicroseconds.Add(duration)
    })
}
```

Alright, let's try this out. Run the API again, but this time *leave the rate limiter enabled*. Like so:

```
$ go run ./cmd/api
time=2026-09-13T20:46:04.220+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:46:04.220+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

dego_vila@yahoo.com 30 Oct 2026

Then use `hey` again to generate some load against the `POST /v1/tokens/authentication` endpoint. This should result in a small number of successful `201 Created` responses, but many more `429 Too Many Requests` responses due to the rate limit being hit.

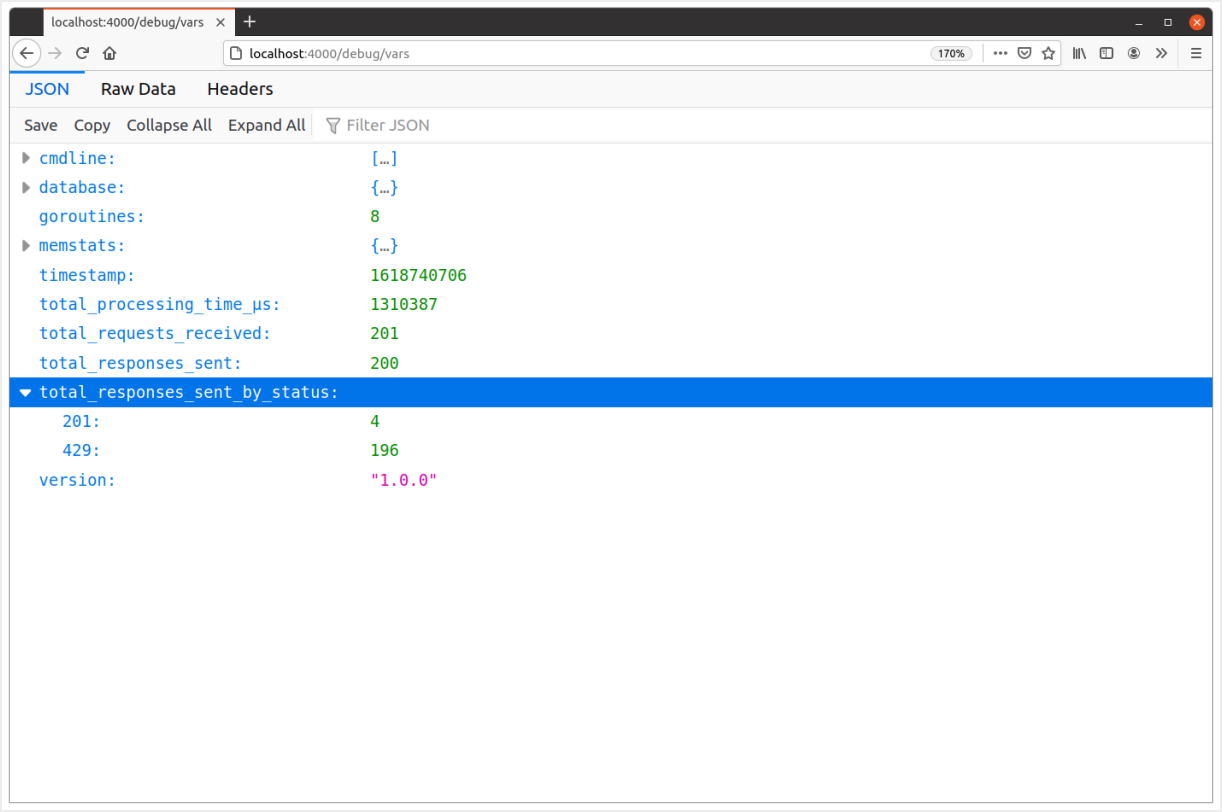
```
$ BODY='{"email": "alice@example.com", "password": "pa55word"}'
$ hey -d "$BODY" -m "POST" http://localhost:4000/v1/tokens/authentication

Summary:
  Total:        0.3351 secs
  Slowest:      0.3334 secs
  Fastest:      0.0001 secs
  Average:      0.0089 secs
  Requests/sec: 596.9162

  Total data: 7556 bytes
  Size/request: 37 bytes

Response time histogram:
  0.000 [1]   |
  0.033 [195] |■■■
```

dego_vila@yahoo.com 30 Oct 2026



Additional information

Visualizing and analyzing metrics

Now that we have some good application-level metrics being recorded, there is the whole question of *what should you do with them?*

The answer to this will be different from project-to-project.

For some projects, it might be sufficient to manually spot check the metrics every so often — or only when you suspect a problem — and make sure that nothing looks unusual or out of place.

In other projects, you might want to write a script to periodically fetch the JSON data from the `GET /debug/vars` endpoint and carry out further analysis. This might include functionality to alert you if something appears to be abnormal.

At the other end of the spectrum, you might want to use a tool like Prometheus to fetch and visualize the data from the endpoint, and display graphs of the metrics in real-time.

There are a lot of different options, and the right thing to do really depends on the needs of

dego_vila@yahoo.com 30 Oct 2026

your project and business. But in all cases, using the `expvar` package to collect and publish the metrics gives you a great platform from which you can integrate any external monitoring, alerting or visualization tools.

Embedded http.ResponseWriter

If you want, you could change the `metricsResponseWriter` struct so that it *embeds* an `http.ResponseWriter` instead of wrapping it. Like this:

```
type metricsResponseWriter struct {
    http.ResponseWriter
    statusCode    int
    headerWritten bool
}

func newMetricsResponseWriter(w http.ResponseWriter) *metricsResponseWriter {
    return &metricsResponseWriter{
        ResponseWriter: w,
        statusCode:     http.StatusOK,
    }
}

func (mw *metricsResponseWriter) WriteHeader(statusCode int) {
    mw.ResponseWriter.WriteHeader(statusCode)

    if !mw.headerWritten {
        mw.statusCode = statusCode
        mw.headerWritten = true
    }
}

func (mw *metricsResponseWriter) Write(b []byte) (int, error) {
    mw.headerWritten = true
    return mw.ResponseWriter.Write(b)
}

func (mw *metricsResponseWriter) Unwrap() http.ResponseWriter {
    return mw.ResponseWriter
}
```

This will give you the same end result as the original approach. However, the gain is that you don't need to write a `Header()` method for the `metricsResponseWriter` struct (it's automatically promoted from the embedded `http.ResponseWriter`). Whereas the loss — at least in my eyes — is that it's a bit less clear and explicit than using a `wrapped` field. Either approach is fine, it's really just a matter of taste which one you prefer.

dego_vila@yahoo.com 30 Oct 2026

Chapter 19.

Building, versioning and quality control

In this section of the book we're going to shift our focus from writing code to *managing and maintaining* our project, and take steps to help automate common tasks and prepare our API for deployment.

Specifically, you'll learn how to:

- Use a makefile to automate common tasks in your project, such as creating and executing migrations.
- Carry out quality control checks of your code using the `go vet` and `staticcheck` tools.
- Vendor third-party packages, in case they ever become unavailable in the future.
- Build and run executable binaries for your applications, reduce their size, and cross-compile binaries for different platforms.
- Leverage Git to generate automated version numbers for your application.

dego_vila@yahoo.com 30 Oct 2026

Creating and using makefiles

In this first chapter we're going to look at how to use the GNU `make` utility and makefiles to help automate common tasks in your project.

The `make` tool should be pre-installed in most Linux distributions, but if it's not already on your machine you should be able to install it via your package manager. For example, if your OS supports the `apt` package manager (like Debian and Ubuntu do) you can install it with:

```
$ sudo apt install make
```

It's also probably already on your machine if you use macOS, but if not you can use `brew` to install it:

```
$ brew install make
```

On Windows machines you can install `make` using the Chocolatey package manager with the command:

```
> choco install make
```

A simple makefile

Now that the `make` utility is installed on your system, let's create our first iteration of a *makefile*. We'll start simple, and then build things up step-by-step.

A makefile is essentially a text file which contains one or more *rules* that the `make` utility can run. Each rule has a *target* and contains a sequence of *commands* which are executed when the rule is run. Generally speaking, makefile rules have the following structure:

```
# comment (optional)
target:
 command
 command
 ...
```

dego_vila@yahoo.com 30 Oct 2026

> **Important:** Please note that each command in a makefile rule must start with a tab character, **not spaces**. If you happen to be reading this in PDF format, the tab will appear as a single space character in the snippet above, but please make sure it is a tab character in your own code.

If you've been following along, you should already have an empty `Makefile` in the root of your project directory. Let's jump in and create a rule which executes the `go run ./cmd/api` command to run our API application. Like so:

```
File: Makefile
```

```
run:
 go run ./cmd/api
```

Make sure that the `Makefile` is saved, and then you can execute a specific rule by running `$ make <target>` from your terminal.

Let's go ahead and call `make run` to start the API:

```
$ make run
go run ./cmd/api
time=2026-09-13T20:50:49.271+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:50:49.271+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

Great, that's worked well. When we type `make run`, the make utility looks for a file called `Makefile` or `makefile` in the current directory and then executes the commands associated with the `run` target.

One thing to point out — by default `make` prints the commands themselves in the terminal output. We can see that in the code above where the first line of the output is `go run ./cmd/api`. If you want, it's possible to suppress commands from being printed by prefixing them with the `@` character.

Environment variables

When we execute a `make` rule, every environment variable that is available to `make` when it starts is transformed into a *make variable* with the same name and value. We can then access these variables using the syntax `${VARIABLE_NAME}` in our makefile.

To illustrate this, let's create two additional rules — a `psql` rule for connecting to our

dego_vila@yahoo.com 30 Oct 2026

database and an `up` rule to execute our database migrations. If you've been following along, both of these rules will need access to the database DSN value from your `GREENLIGHT_DB_DSN` environment variable.

Go ahead and update your `Makefile` to include these two new rules like so:

File: Makefile

```
run:
 go run ./cmd/api

psql:
 psql ${GREENLIGHT_DB_DSN}

up:
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up
```

OK, let's try this out by running `make up` to execute our database migrations:

```
$ make up
migrate -path ./migrations -database postgres://greenlight:pa55word@localhost/greenlight up
no change
```

You should see from the output that the value of your `GREENLIGHT_DB_DSN` environment variable is successfully pulled through and used in the `make` rule. If you've been following along, there shouldn't be any outstanding migrations to apply, so this should then exit successfully with no further action.

We're also starting to see the benefits of using a makefile here — being able to type `make up` is a big improvement on having to remember and use the full command for executing our 'up' migrations.

Likewise, if you want, you can also try running `make psql` to connect to the `greenlight` database with `psql`.

Passing arguments

The `make` utility also allows you to pass *named arguments* when executing a particular rule. To illustrate this, let's add a `migration` rule to our makefile to generate a new pair of migration files. The idea is that when we execute this rule we'll pass the name of the migration files as an argument, similar to this:

```
$ make migration name=create_example_table
```

dego_vila@yahoo.com 30 Oct 2026

The syntax to access the value of named arguments is exactly the same as for accessing environment variables. So, in the example above, we could access the migration file name via `${name}` in our makefile.

Go ahead and update the makefile to include this new `migration` rule, like so:

File: Makefile

```
run:
 go run ./cmd/api

psql:
 psql ${GREENLIGHT_DB_DSN}

migration:
 migrate create -seq -ext=.sql -dir=./migrations ${name}

up:
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up
```

And if you execute this new rule with the `name=create_example_table` argument you should see the following output:

```
$ make migration name=create_example_table
migrate create -seq -ext=.sql -dir=./migrations create_example_table
/home/alex/Projects/greenlight/migrations/000007_create_example_table.up.sql
/home/alex/Projects/greenlight/migrations/000007_create_example_table.down.sql
```

You'll now also have two new empty migration files with the name `create_example_table` in your migrations folder. Like so:

```
$ ls ./migrations/
000001_create_movies_table.down.sql           000004_create_users_table.up.sql
000001_create_movies_table.up.sql             000005_create_tokens_table.down.sql
000002_add_movies_check_constraints.down.sql  000005_create_tokens_table.up.sql
000002_add_movies_check_constraints.up.sql    000006_add_permissions.down.sql
000003_add_movies_indexes.down.sql            000006_add_permissions.up.sql
000003_add_movies_indexes.up.sql              000007_create_example_table.down.sql
000004_create_users_table.down.sql            000007_create_example_table.up.sql
```

If you're following along, we won't actually be using these two new migration files, so feel free to delete them:

```
$ rm migrations/000007*
```

dego_vila@yahoo.com 30 Oct 2026

> **Note:** Variable names in makefiles are case-sensitive, so `foo`, `FOO`, and `Foo` all refer to different variables. The `make` documentation suggests using lower case letters for variable names that only serve an internal purpose in the makefile, and using upper case variable names otherwise.

Namespacing targets

As your makefile continues to grow, you might want to start *namespacing* your target names to provide some differentiation between rules and help organize the file. For example, in a large makefile rather than having the target name `up` it would be clearer to give it the name `db/migrations/up` instead.

I recommend using the `/` character as a namespace separator, rather than a period, hyphen or the `:` character. In fact, the `:` character should be strictly avoided in target names as it can cause problems when using *target prerequisites* (something that we'll cover in a moment).

Let's update our target names to use some sensible namespaces, like so:

File: Makefile

```
run/api:
 go run ./cmd/api

db/psql:
 psql ${GREENLIGHT_DB_DSN}

db/migrations/new:
 migrate create -seq -ext=.sql -dir=./migrations ${name}

db/migrations/up:
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up
```

And you should be able to execute the rules by typing the full target name when running `make`. For example:

```
$ make run/api
go run ./cmd/api
time=2026-09-13T20:54:08.083+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:54:08.083+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

A nice feature of using the `/` character as the namespace separator is that you get tab completion in the terminal when typing target names. For example, if you type `make`

dego_vila@yahoo.com 30 Oct 2026

`db/migrations/` and then hit tab on your keyboard the remaining targets under the namespace will be listed. Like so:

```
$ make db/migrations/
new  up
```

Prerequisite targets and asking for confirmation

The general syntax for a makefile rule that I gave at the start of this chapter was a slight simplification, because it's also possible to specify *prerequisite* targets.

```
target: prerequisite-target-1 prerequisite-target-2 ...
 command
 command
 ...
```

When you specify a prerequisite target for a rule, the corresponding commands for the prerequisite targets will be run *before* executing the actual target commands.

Let's leverage this functionality to ask the user for *confirmation to continue* before executing our `db/migrations/up` rule.

To do this, we'll create a new `confirm` target which asks the user `Are you sure? [y/N]` and exits with an error if they do not enter `y`. Then we'll use this new `confirm` target as a prerequisite for `db/migrations/up`.

Go ahead and update your `Makefile` as follows:

File: Makefile

```
# Create the new confirm target.
confirm:
 @echo -n 'Are you sure? [y/N] ' && read ans && [ $${ans:-N} = y ]

run/api:
 go run ./cmd/api

db/psql:
 psql ${GREENLIGHT_DB_DSN}

db/migrations/new:
 migrate create -seq -ext=.sql -dir=./migrations ${name}

# Include it as prerequisite.
db/migrations/up: confirm
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up
```

dego_vila@yahoo.com 30 Oct 2026

The code in the `confirm` target is taken from this StackOverflow post. Essentially, what happens here is that we ask the user `Are you sure? [y/N]` and then read the response. We then use the code `[ ${ans:-N} = y ]` to evaluate the response — this will return `true` if the user enters `y` and `false` if they enter anything else. If a command in a makefile returns `false`, then `make` will stop running the rule and exit with an error message — essentially stopping the rule in its tracks.

We've used the `@` character at the start of the command, to prevent the command itself from being printed out.

And finally, notice that we have set `confirm` as a prerequisite for the `db/migrations/up` target.

Let's try this out and see what happens when we enter `y`:

```
$ make db/migrations/up
Are you sure? [y/N] y
migrate -path ./migrations -database postgres://greenlight:pa55word@localhost/greenlight up
no change
```

That looks good — the commands in our `db/migrations/up` rule have been executed as we would expect.

In contrast, let's try the same thing again but enter any other letter when asked for confirmation. This time, `make` should exit *without* executing anything in the `db/migrations/up` rule. Like so:

```
$ make db/migrations/up
Are you sure? [y/N] n
make: *** [Makefile:3: confirm] Error 1
```

Using a `confirm` rule as a prerequisite target like this is a really nice reusable pattern. Any time you have a makefile rule which does something destructive or dangerous, you can now just include `confirm` as a prerequisite target to ask the user for confirmation to continue.

Displaying help information

Another small thing that we can do to make our makefile more user-friendly is to include some comments and help functionality. Specifically, we'll prefix each rule in our makefile with a comment in the following format:

dego_vila@yahoo.com 30 Oct 2026

```
## <example target call>: <help text>
```

Then we'll create a new `help` rule which parses the makefile itself, extracts the help text from the comments using `sed`, formats them into a table and then displays them to the user.

If you're following along, go ahead and update your `Makefile` so that it looks like this:

File: Makefile

```
## help: print this help message
help:
 @echo 'Usage:'
 @sed -n 's/^##//p' ${MAKEFILE_LIST} | column -t -s ':' |  sed -e 's/^/ /'

confirm:
 @echo -n 'Are you sure? [y/N] ' && read ans && [ $${ans:-N} = y ]

## run/api: run the cmd/api application
run/api:
 go run ./cmd/api

## db/psql: connect to the database using psql
db/psql:
 psql ${GREENLIGHT_DB_DSN}

## db/migrations/new name=$1: create a new database migration
db/migrations/new:
 migrate create -seq -ext=.sql -dir=./migrations ${name}

## db/migrations/up: apply all up database migrations
db/migrations/up: confirm
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up
```

> **Note:** `MAKEFILE_LIST` is a special variable which contains the name of the makefile being parsed by `make`.

And if you now execute the `help` target, you should get a response which lists all the available targets and the corresponding help text. Similar to this:

```
$ make help
Usage:
  help                      print this help message
  run/api                   run the cmd/api application
  db/psql                   connect to the database using psql
  db/migrations/new name=$1   create a new database migration
  db/migrations/up            apply all up database migrations
```

I should also point out that positioning the `help` rule as the first thing in the `Makefile` is a deliberate move. If you run `make` *without specifying a target* then it will default to executing

dego_vila@yahoo.com 30 Oct 2026

the first rule in the file.

So this means that if you try to run `make` without a target you'll now be presented with the help information, like so:

```
$ make
Usage:
  help                       print this help message
  run/api                    run the cmd/api application
  db/psql                    connect to the database using psql
  db/migrations/new name=$1  create a new database migration
  db/migrations/up           apply all up database migrations
```

Phony targets

In this chapter we've been using `make` to execute *actions*, but another (and arguably, the primary) purpose of `make` is to help create files on disk where *the name of a target is the name of a file* being created by the rule.

If you're using `make` primarily to execute actions, like we are, then this can cause a problem if there is a file in your project directory *with the same path* as a target name.

If you want, you can demonstrate this problem by creating a file called `./run/api` in the root of your project directory, like so:

```
$ mkdir run && touch run/api
```

And then if you execute `make run/api`, instead of our API application starting up you'll get the following message:

```
$ make run/api
make: 'run/api' is up to date.
```

Because we already have a file on disk at `./run/api`, the `make` tool considers this rule to have already been executed and so returns the message that we see above without taking any further action.

To work around this, we can declare our makefile targets to be phony targets:

> *A phony target is one that is not really the name of a file; rather it is just a name for a rule to be executed.*

dego_vila@yahoo.com 30 Oct 2026

To declare a target as phony, you can make it prerequisite of the special `.PHONY` target. The syntax looks like this:

```
.PHONY: target
target: prerequisite-target-1 prerequisite-target-2 ...
 command
 command
 ...
```

Let's go ahead and update out `Makefile` so that all our rules have phony targets, like so:

File: Makefile

```
## help: print this help message
.PHONY: help
help:
 @echo 'Usage:'
 @sed -n 's/^##//p' ${MAKEFILE_LIST} | column -t -s ':' |  sed -e 's/^/ /'

.PHONY: confirm
confirm:
 @echo -n 'Are you sure? [y/N] ' && read ans && [ $${ans:-N} = y ]

## run/api: run the cmd/api application
.PHONY: run/api
run/api:
 go run ./cmd/api

## db/psql: connect to the database using psql
.PHONY: db/psql
db/psql:
 psql ${GREENLIGHT_DB_DSN}

## db/migrations/new name=$1: create a new database migration
.PHONY: db/migrations/new
db/migrations/new:
 migrate create -seq -ext=.sql -dir=./migrations ${name}

## db/migrations/up: apply all up database migrations
.PHONY: db/migrations/up
db/migrations/up: confirm
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up
```

If you run `make run/api` again now, it should now correctly recognize this as a phony target and execute the rule for us:

```
$ make run/api
go run ./cmd/api -db-dsn=postgres://greenlight:pa55word@localhost/greenlight
time=2026-09-13T20:58:22.941+02:00 level=INFO msg="database connection pool established"
time=2026-09-13T20:58:22.941+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

You might think that it's only necessary to declare targets as phony if you have a conflicting file name, but in practice *not declaring a target as phony when it actually is* can lead to bugs

dego_vila@yahoo.com 30 Oct 2026

or confusing behavior. For example, imagine if in the future someone unknowingly creates a file called `confirm` in the root of the project directory. This would mean that our `confirm` rule is never executed, which in turn would lead to dangerous or destructive rules being executed without confirmation.

To avoid this kind of bug, if you have a makefile rule which carries out an action (rather than creating a file) then it's best to get into the habit of declaring it as phony.

If you're following along, you can go ahead and remove the contents of the `run` directory that we just made. Like so:

```
$ rm -rf run/
```

All in all, this is shaping up nicely. Our makefile is starting to contain some helpful functionality, and we'll continue to add more to it over the next few chapters of this book.

Although this is one of the last things we're doing in this build, creating a makefile in the root of a project directory is normally one of the very *first* things I do when starting a project. I find that using a makefile for common tasks helps save both typing and mental overhead during development, and — in the longer term — it acts as a useful entry point and a reminder of how things work when you come back to a project after a long break.

dego_vila@yahoo.com 30 Oct 2026

Managing environment variables

Using the `make run/api` command to run our API application opens up an opportunity to tweak our command-line flags, and remove the default value for our database DSN from the `main.go` file. Like so:

File: cmd/api/main.go

```
package main

...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    // Use the empty string "" as the default value for the db-dsn command-line flag,
    // rather than os.Getenv("GREENLIGHT_DB_DSN") like we were previously.
    flag.StringVar(&cfg.db.dsn, "db-dsn", "", "PostgreSQL DSN")

    ...
}
```

Instead, we can update our makefile so that the DSN value from the `GREENLIGHT_DB_DSN` environment variable is passed in as part of the rule. If you're following along, please go ahead and update the `run/api` rule as follows:

File: Makefile

```
...

## run/api: run the cmd/api application
.PHONY: run/api
run/api:
 go run ./cmd/api -db-dsn=${GREENLIGHT_DB_DSN}

...
```

This is a small change but a really nice one, because it means that the *default configuration values for our application no longer change depending on the operating environment*. The command-line flag values passed at runtime are the *sole* mechanism for configuring our application settings, and there are still no secrets hard-coded in our project files.

During development running our application remains nice and easy — all we need to do is

dego_vila@yahoo.com 30 Oct 2026

type `make run/api`, like so:

```
$ make run/api
go run ./cmd/api -db-dsn=postgres://greenlight:pa55word@localhost/greenlight
time=2026-09-14T10:42:31.758+02:00 level=INFO msg="database connection pool established"
time=2026-09-14T10:42:31.758+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

> **Hint:** If you're not comfortable with the DSN value (which contains a password) being displayed on your screen when you type `make run/api`, remember that you can use the `@` character at the start of the command to prevent it from being printed.

Using a .envrc file

If you like, you could also *remove* the `GREENLIGHT_DB_DSN` environment variable from your `$HOME/.profile` or `$HOME/.bashrc` files, and store it in a `.envrc` file in the root of your project directory instead.

If you're following along, go ahead and create a new `.envrc` file like so:

```
$ touch .envrc
```

File: .envrc

```
export GREENLIGHT_DB_DSN=postgres://greenlight:pa55word@localhost/greenlight
```

You can then use a tool like direnv to automatically load the variables from the `.envrc` file into your current shell, or alternatively, you can add an `include` command at the top of your `Makefile` to load them instead. Like so:

File: Makefile

```
# Include variables from the .envrc file
include .envrc

## help: print this help message
.PHONY: help
help:
 @echo 'Usage:'
 @sed -n 's/^##//p' ${MAKEFILE_LIST} | column -t -s ':' |  sed -e 's/^/ /'

...
```

dego_vila@yahoo.com 30 Oct 2026

This approach is particularly convenient in projects where you need to make frequent changes to your environment variables, because it means that you can just edit the `.envrc` file without needing to reboot your computer or run `source` after each change.

Another nice benefit of this approach is that it provides a degree of separation between variables if you're working on multiple projects on the same machine.

> **Important:** If you use this approach and your `.envrc` file contains any secrets, you must be careful to not commit the file into a version control system (like Git or Mercurial).

In a few chapters' time we'll start version controlling our codebase with Git, so let's preemptively add an ignore rule so that the `.envrc` file is never committed.

```
$ echo '.envrc' >> .gitignore
```

dego_vila@yahoo.com 30 Oct 2026

Quality-controlling code

In this chapter we're going to add new `audit` and `tidy` rules to our `Makefile` to check, test and tidy up our codebase automatically. It's useful to have rules like these that you can routinely run before you commit changes into your version control system or build any binaries.

The `audit` rule *won't* make any changes to our codebase, but will simply report any problems. In particular it will:

- Use the `go mod tidy -diff` command to check if the `go.mod` and `go.sum` files are out of date and need to be fixed (which you can do by running `go mod tidy`).
- Use the `go mod verify` command to check that the dependencies on your computer (located in your module cache located at `$GOPATH/pkg/mod`) haven't been changed since they were downloaded and that they match the cryptographic hashes in your `go.sum` file. Running this helps ensure that the dependencies being used are the exact ones that you expect.
- Use the `go vet ./...` command to check all `.go` files in the project directory. The `go vet` tool runs a variety of *analyzers* which carry out static analysis of your code and warn you about things which might be wrong but won't be picked up by the compiler — such as unreachable code, unnecessary assignments, and badly-formed build tags.
- Use the third-party `staticcheck` tool to carry out some additional static analysis checks.
- Use the `go test -race -vet=off ./...` command to run all tests in the project directory. By default, `go test` automatically executes a small subset of the `go vet` checks before running any tests, so to avoid duplication we'll use the `-vet=off` flag to turn this off. The `-race` flag enables Go's race detector, which can help pick up certain classes of race conditions while tests are running.

In contrast, the `tidy` rule *will* actually make changes to the codebase. It will:

- Use the `go mod tidy` command to prune any unused dependencies from the `go.mod` and `go.sum` files, and add any missing dependencies.
- Use the `go fmt ./...` command to format all `.go` files in the project directory, according to the Go standard. This will reformat files 'in place' and output the names of any changed files.

dego_vila@yahoo.com 30 Oct 2026

- Use the `go fix ./...` command to automatically update your code to use modern Go features. This will silently rewrite files 'in place' without any terminal output, and make changes like rewriting loops to use the Go 1.22 range-over-int syntax where it makes sense to, and replacing `interface{}` with `any`. There's a good introduction to the `go fix` tool on the official blog.

Using staticcheck

If you're following along, the first thing we want to do is use the `go get -tool` command to add the latest version of `staticcheck` as a *tool dependency* to the `go.mod` file. Like so:

```
$ go get -tool honnef.co/go/tools/cmd/staticcheck@latest
go: downloading golang.org/x/sync v0.23.0
go: downloading golang.org/x/mod v0.41.0
go: added github.com/BurntSushi/toml v1.4.1-0.20240526193622-a339e1f7089c
go: added golang.org/x/exp/typeparams v0.0.0-20231108232855-2478ac86f678
go: added honnef.co/go/tools v0.8.1
```

Once that's finished, you should now be able to execute `staticcheck` using the `go tool` command, like so:

```
$ go tool staticcheck --version
staticcheck 2026.2.1 (0.8.1)

$ go tool staticcheck ./...
cmd/api/middleware.go:133:25: func (*application).requireActivatedUser is unused (U1000)
```

> **Note:** If you're not familiar with the `go get -tool` and `go tool` commands (which were introduced in Go 1.24 to help manage developer tooling in your projects), please see this blog post for more information and examples.

If you've been following along with the application build, you should see output from `staticcheck` similar to the above. It reports one failure with the code `U1000`, alerting us to the fact that the `requireActivatedUser()` middleware function is currently unused.

We *could* remove the function, or comment it out, but for the purpose of this book let's demonstrate how to ignore this specific failure by adding a linter directive for the failure code `U1000` to the `requireActivatedUser()` middleware.

Go ahead and add the directive like so:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/middleware.go

```
package main

...

//lint:ignore U1000 middleware currently unused, but might be in the future
func (app *application) requireActivatedUser(next http.HandlerFunc) http.HandlerFunc {
 ...
}
```

And if you run `staticcheck` again, it should complete with no failures reported.

Creating the tidy and audit rules

Now let's go ahead and create the new `tidy` and `audit` rules in our makefile. While we're at it, let's also add some comment blocks to help organize and clarify the purpose of our different makefile rules, like so:

dego_vila@yahoo.com 30 Oct 2026

File: Makefile

```
include .envrc

# ==================================================================================== #
# HELPERS
# ==================================================================================== #

## help: print this help message
.PHONY: help
help:
 @echo 'Usage:'
 @sed -n 's/^##//p' ${MAKEFILE_LIST} | column -t -s ':' |  sed -e 's/^/ /'

.PHONY: confirm
confirm:
 @echo -n 'Are you sure? [y/N] ' && read ans && [ $${ans:-N} = y ]

# ==================================================================================== #
# DEVELOPMENT
# ==================================================================================== #

## run/api: run the cmd/api application
.PHONY: run/api
run/api:
 go run ./cmd/api -db-dsn=${GREENLIGHT_DB_DSN}

## db/psql: connect to the database using psql
.PHONY: db/psql
db/psql:
 psql ${GREENLIGHT_DB_DSN}

## db/migrations/new name=$1: create a new database migration
.PHONY: db/migrations/new
db/migrations/new:
 migrate create -seq -ext=.sql -dir=./migrations ${name}

## db/migrations/up: apply all up database migrations
.PHONY: db/migrations/up
db/migrations/up: confirm
 migrate -path ./migrations -database ${GREENLIGHT_DB_DSN} up

# ==================================================================================== #
# QUALITY CONTROL
# ==================================================================================== #

## tidy: tidy module dependencies, and format and modernize all .go files
.PHONY: tidy
tidy:
 go mod tidy
 go fix ./...
 go fmt ./...

## audit: run quality control checks
.PHONY: audit
audit:
 go mod tidy -diff
 go mod verify
 go vet ./...
 go tool staticcheck ./...
 go test -race -vet=off ./...
```

Now that’s done, all you need to do is type `make tidy` and `make audit` to execute these

dego_vila@yahoo.com 30 Oct 2026

checks. Let's give it a try.

If you've been following along closely, the output should look very similar to this:

```
$ make tidy
go mod tidy
go: downloading github.com/google/go-cmp v0.6.0
go fix ./...
go fmt ./...

$ make audit
go mod tidy -diff
go mod verify
all modules verified
go vet ./...
go tool staticcheck ./...
go test -race -vet=off ./...
?       greenlight.alexedwards.net/cmd/api      [no test files]
?       greenlight.alexedwards.net/cmd/examples/cors/preflight  [no test files]
?       greenlight.alexedwards.net/cmd/examples/cors/simple     [no test files]
?       greenlight.alexedwards.net/internal/models        [no test files]
?       greenlight.alexedwards.net/internal/jsonlog     [no test files]
?       greenlight.alexedwards.net/internal/mailer      [no test files]
?       greenlight.alexedwards.net/internal/validator   [no test files]
```

That's looking good. The `make tidy` command resulted in an additional package needing to be downloaded on my machine, but apart from that, all the checks completed successfully without any problems.

Additional information

Testing

We covered the topic of testing in a lot of detail in the first *Let's Go* book, and the same principles apply again here. If you like, feel free to revisit the testing section of *Let's Go* and reimplement some of those same patterns in your API. For example, you might like to try:

- Creating an end-to-end test for the `GET /v1/healthcheck` endpoint to verify that the headers and response body are what you expect.
- Creating a unit test for the `rateLimit()` middleware to confirm that it sends a `429 Too Many Requests` response after a certain number of requests.
- Creating an end-to-end integration test, using a test database instance, which confirms that the `authenticate()` and `requirePermission()` middleware work together correctly to allow or disallow access to specific endpoints.

dego_vila@yahoo.com 30 Oct 2026

Module mirrors and vendoring

One of the risks of using third-party packages in your Go code is that the version control repository where it is hosted (e.g GitHub, GitLab or BitBucket) will be temporarily unavailable when you need to fetch a copy of the code. Or worse, the owner might delete the repository altogether without adequate warning. For example, the `httprouter` package plays a central part in our application, and if the author ever decided to delete it from GitHub it would be a headache to scramble and replace it with an alternative.

(I'm not suggesting this is likely to happen with `httprouter` — just using it as an example!)

Fortunately, Go provides two ways in which we can mitigate this risk: *module mirrors* and *vendoring*.

Module mirrors

Go supports using *module mirrors* (sometimes also known as *module proxies*), which are services that mirror source code from the original, authoritative, repositories.

The most well-known (and well-used) module mirror is `proxy.golang.org`, which is maintained by the Go team at Google and contains copies of the source code from tens of thousands of open-source Go repositories.

By default, whenever you fetch a module using `go get` or one of the `go mod *` commands, the `go` tool will first attempt to retrieve the appropriate version of the source code from the `proxy.golang.org` mirror. If it can't be found in the mirror, then it will fall back to fetching the code from the authoritative repository.

If you run the `go env` command on your machine, you should see that there is a `GOPROXY` environment variable setting similar to this:

```
$ go env
...
GOPRIVATE=''
GOPROXY='https://proxy.golang.org,direct'
GOROOT='/usr/local/go'
...
```

This setting contains the list of sources that the `go` tool will fetch source code from,

dego_vila@yahoo.com 30 Oct 2026

separated by commas (`,`) or pipes (`|`).

Unless you've already changed it from the default value, you should see that the `GOPROXY` setting equals `'https://proxy.golang.org,direct'`. This means that the `go` tool will try to fetch source code from `https://proxy.golang.org` first, then fall back to fetching a copy directly from the authoritative host (thanks to the `direct` directive).

You can change this to use alternative module mirrors — such as `goproxy.io` — if you prefer. For example, if you want use `https://goproxy.io` as your primary source, then use `https://proxy.golang.org` as a secondary source, then fall back to a direct fetch, you could update your `GOPROXY` setting using the `go env -w` command like so:

```
$ go env -w GOPROXY="https://goproxy.io,https://proxy.golang.org,direct"
```

Or if you want to disable module mirrors altogether, you can simply set the value to `direct` like so:

```
$ go env -w GOPROXY="direct"
```

It's important to explain that if you use the pipe character `|` as a separator, rather than a comma, it subtly changes the behavior.

When a comma is used as the separator, the `go` tool will fall back to fetching from the next source only after receiving a `404` (not found) or `410` (gone) HTTP response from the mirror. If there is any other error, including a network timeout, the error is considered terminal and the fetch will fail.

Whereas if you use `|` as the separator, it will fall back to trying the next source after *any* error, which is useful if the mirror is temporarily unavailable or experiencing problems.

```
$ go env -w GOPROXY="https://proxy.golang.org|direct"
```

But in general, I recommend leaving `GOPROXY` with its default setting unless you have a good reason to change it — like you want to use an alternative mirror, or `proxy.golang.org` is unavailable for a long period of time.

Vendoring

Go's module mirror functionality is great, but it isn't a silver bullet for all developers and all

dego_vila@yahoo.com 30 Oct 2026

projects.

For example, perhaps you don't trust or wish to rely on module mirrors provided by Google or other third-parties. Or maybe you need to routinely work in an environment without network access. In those scenarios you probably still want to mitigate the risk of a disappearing dependency, but using a module mirror isn't possible or appealing.

You should also be aware that the default `proxy.golang.org` module mirror doesn't absolutely guarantee that it will store source code forever. From the FAQs:

> *proxy.golang.org does not save all modules forever. There are a number of reasons for this, but one reason is if proxy.golang.org is not able to detect a suitable license. In this case, only a temporarily cached copy of the module will be made available, and may become unavailable if it is removed from the original source and becomes outdated.*

Additionally, if you need to come back to a 'cold' codebase in 5 or 10 years' time, will the `proxy.golang.org` module mirror still be available? Hopefully it will — but it's hard to say for sure.

So, for these reasons, it can still be sensible to *vendor* your project dependencies using the `go mod vendor` command. Vendoring dependencies in this way basically stores a complete copy of the source code for third-party packages in a `vendor` folder in your project.

Let's demonstrate how to do this. We'll start by adapting our `make tidy` rule to also call the `go mod verify` and `go mod vendor` commands, like so:

File: Makefile

```
...

# ================================================================================ #
# QUALITY CONTROL
# ================================================================================ #

## tidy: tidy and vendor module dependencies, and format and modernize all .go files
.PHONY: tidy
tidy:
 go mod tidy
 go mod verify
 go mod vendor
 go fix ./...
 go fmt ./...

...
```

Just to be clear about what's going on behind-the-scenes here, let's quickly step through what will happen when we run `make tidy`:

dego_vila@yahoo.com 30 Oct 2026

- The `go mod tidy` command will make sure the `go.mod` and `go.sum` files list all the necessary dependencies for our project (and no unnecessary ones).
- The `go mod verify` command will verify that the dependencies stored in your module cache (located on your machine at `$GOPATH/pkg/mod`) match the checksums in the `go.sum` file.
- The `go mod vendor` command will then copy the necessary source code from your module cache into a new `vendor` directory in your project root.

Let's try this out and run the new `tidy` rule like so:

```
$ make tidy
go mod tidy
go mod verify
all modules verified
go mod vendor
go fix ./...
go fmt ./...
```

Once that's completed, you should see that a new `vendor` directory has been created containing copies of all the source code along with a `modules.txt` file. The directory structure in your `vendor` folder should look similar to this:

```
$ tree -L 3 ./vendor/
./vendor/
├── github.com
│   ├── BurntSushi
│   │   └── toml
│   ├── julienschmidt
│   │   └── httprouter
│   ├── lib
│   │   └── pq
│   ├── tomasen
│   │   └── realip
│   └── wneessen
│       └── go-mail
├── golang.org
│   └── x
│       ├── crypto
│       ├── exp
│       ├── mod
│       ├── sync
│       ├── text
│       ├── time
│       └── tools
├── honnef.co
│   └── go
│       └── tools
└── modules.txt
```

Now, when you run a command such as `go run`, `go test` or `go build`, the `go` tool will recognize the presence of a `vendor` folder and *the dependency code in the vendor folder* will

dego_vila@yahoo.com 30 Oct 2026

be used — rather than the code in the module cache on your local machine.

If you like, go ahead and try running the API application. You should find that everything compiles and continues to work just like before.

```
$ make run/api
go run ./cmd/api -db-dsn=postgres://greenlight:pa55word@localhost/greenlight
time=2026-09-14T11:07:11.379+02:00 level=INFO msg="database connection pool established"
time=2026-09-14T11:07:11.379+02:00 level=INFO msg="starting server" addr=:4000 env=development
```

> **Note:** If you want to confirm that it's really the vendored dependencies being used, you can run `go clean -modcache` to remove *everything* from your local module cache. When you run the API again, you should find that it still starts up correctly *without* needing to refetch the dependencies from the Go module mirror.

Because all the dependency source code is now stored in your project repository itself, it's easy to check it into Git (or an alternative version control system) alongside the rest of your code. This is reassuring because it gives you ownership of all the code used to build and run your applications, kept under version control.

The downside of this, of course, is that it adds size and bloat to your project repository. This is of particular concern in projects that have a lot of dependencies and the repository will be cloned *a lot*, such as projects where a CI/CD system clones the repository with each new commit.

Let's also take a quick look in the `vendor/modules.txt` file that was created. If you've been following along it should look similar to this:

dego_vila@yahoo.com 30 Oct 2026

File: vendor/modules.txt

```
# github.com/BurntSushi/toml v1.4.1-0.20240526193622-a339e1f7089c
## explicit; go 1.18
github.com/BurntSushi/toml
github.com/BurntSushi/toml/internal
# github.com/julienschmidt/httprouter v1.3.0
## explicit; go 1.7
github.com/julienschmidt/httprouter
# github.com/lib/pq v1.12.3
## explicit; go 1.21
github.com/lib/pq
github.com/lib/pq/internal/pgpass
github.com/lib/pq/internal/pgservice
github.com/lib/pq/internal/pqsql
github.com/lib/pq/internal/pqtime
github.com/lib/pq/internal/pqutil
github.com/lib/pq/internal/proto
github.com/lib/pq/oid
github.com/lib/pq/pqerror
github.com/lib/pq/scram
# github.com/tomasen/realip v0.0.0-20180522021738-f0c99a92ddce
## explicit
github.com/tomasen/realip
# github.com/wneessen/go-mail v0.8.1
## explicit; go 1.25.0
github.com/wneessen/go-mail
... etc.
```

This `vendor/modules.txt` file is essentially a *manifest* of the vendored packages and their version numbers. When vendoring is being used, the `go` tool will check that the module version numbers in `modules.txt` are consistent with the version numbers in the `go.mod` file. If there's any inconsistency, then the `go` tool will report an error.

> **Note:** It's important to point out that there's no easy way to verify that the *checksums of the vendored dependencies* match the checksums in the `go.sum` file. Or, in other words, there's no equivalent to `go mod verify` which works *directly* on the contents of the `vendor` folder.
>
> To mitigate that, it's a good idea to run *both* `go mod verify` and `go mod vendor` regularly — which is one of the reasons for including them both as part of the `make tidy` rule. Using `go mod verify` will verify that the dependencies in your module cache match the `go.sum` file, and `go mod vendor` will copy those same dependencies from the module cache into your `vendor` folder.

Lastly, you should avoid making any changes to the code in the `vendor` directory. Doing so can potentially cause confusion (because the code would no longer be consistent with the original version of the source code) and — besides — running `go mod vendor` will overwrite any changes you make each time you run it. If you need to change the code for a

dego_vila@yahoo.com 30 Oct 2026

dependency, it's much better to fork it and import the forked version instead.

Additional information

Inconsistent vendoring error

Every time you add, remove or upgrade a dependency, make sure to run `go mod vendor` or the `make tidy` command immediately afterwards to synchronize the contents of the `vendor` folder with your `go.mod` file. If you don't, you'll run into an "inconsistent vendoring" error similar to this:

```
$ make run/api
go: inconsistent vendoring in /home/alex/Projects/greenlight:
        github.com/foo/bar@v0.1.0: is explicitly
            required in go.mod, but not marked as explicit in vendor/modules.txt

        To ignore the vendor directory, use -mod=readonly or -mod=mod.
        To sync the vendor directory, run:
                go mod vendor
```

The ./... pattern

Most of the `go` tools support the `./...` wildcard pattern, like `go fmt ./...`, `go vet ./...` and `go test ./...`. This pattern matches the current directory and all sub-directories, *excluding the* `vendor` *directory*.

Generally speaking, this is useful because it means that we're not formatting, vetting or testing the code in our `vendor` directory unnecessarily — and our `make audit` rule won't fail due to any problems that might exist *within* those vendored packages.

dego_vila@yahoo.com 30 Oct 2026

Building binaries

So far we've been running our API using the `go run` command (or more recently, `make run/api`). But in this chapter we're going to focus on explaining how to *build an executable binary* that you can distribute and run on other machines without needing the Go toolchain installed.

To build a binary we need to use the `go build` command. As a simple example, usage looks like this:

```
$ go build -o=./bin/api ./cmd/api
```

When we run this command, `go build` will *compile* the `cmd/api` package (and any dependent packages) into files containing machine code, and then *link* these together to form an executable binary. In the command above, the executable binary will be output to `./bin/api`.

For convenience, let's add a new `build/api` rule to our makefile which runs this command, like so:

File: Makefile

```
...

# ================================================================================ #
# BUILD
# ================================================================================ #

## build/api: build the cmd/api application
.PHONY: build/api
build/api:
	go build -o=./bin/api ./cmd/api
```

Once that's done, go ahead and execute the `make build/api` rule. You should see that an executable binary file gets created at `./bin/api`.

```
$ make build/api
go build -o=./bin/api ./cmd/api
$ ls -l ./bin/
total 16376
-rwxrwxr-x 1 alex alex 16764876 Sep 14 11:23 api
```

dego_vila@yahoo.com 30 Oct 2026

And you should be able to run this executable to start your API application, passing in any command-line flag values as necessary. For example:

```
$ ./bin/api -port=4040 -db-dsn=postgres://greenlight:pa55word@localhost/greenlight
time=2026-09-14T11:24:44.427+02:00 level=INFO msg="database connection pool established"
time=2026-09-14T11:24:44.427+02:00 level=INFO msg="starting server" addr=:4040 env=development
```

Reducing binary size

If you take a closer look at the executable binary you'll see that it weighs in at 16764876 bytes (about 16.7MB).

```
$ ls -l ./bin/api
-rwxrwxr-x 1 alex alex 16764876 Sep 14 11:23 ./bin/api
```

> **Note:** If you're following along, your binary size may be slightly different. It depends on your operating system, and the exact version of Go and the third-party dependencies that you're using.

It's possible to reduce the binary size — normally by around 25% — by instructing the Go linker to strip symbol tables and DWARF debugging information from the binary. We can do this as part of the `go build` command by using the *linker flag* `-ldflags="-s"` as follows:

File: Makefile

```
...

# ==================================================================================== #
# BUILD
# ==================================================================================== #

## build/api: build the cmd/api application
.PHONY: build/api
build/api:
 go build -ldflags='-s' -o=./bin/api ./cmd/api
```

If you run `make build/api` again, you should now find that the binary is noticeably smaller (in my case about 11.5MB).

```
$ make build/api
go build -ldflags='-s' -o=./bin/api ./cmd/api
$ ls -l ./bin/api
-rwxrwxr-x 1 alex alex 11542791 Sep 14 11:28 ./bin/api
```

dego_vila@yahoo.com 30 Oct 2026

It's important to be aware that stripping out this information will make it harder to debug an executable using a tool like Delve or gdb. But, generally, it's not often that you'll need to do this — and there's even an open proposal from Rob Pike to make omitting DWARF information the default behavior of the linker in the future.

> **Note:** The linker flag `-ldflags='-s'` strips out *both* symbol tables and DWARF debugging information. If you want to only omit the DWARF debugging information, you can use the linker flag `-ldflags='-w'` instead. If you want to only omit the symbol table, you can use the flags `-ldflags='-s -w=0'`

Cross-compilation

By default, the `go build` command will output a binary suitable for use on your *local machine's operating system and architecture*. But it also supports cross-compilation, so you can generate a binary suitable for use on a different machine. This is particularly useful if you're developing on one operating system and deploying on another.

To see a list of all the operating system/architecture combinations that Go supports, you can run the `go tool dist list` command like so:

```
$ go tool dist list
aix/ppc64
android/386
android/amd64
android/arm
android/arm64
darwin/amd64
...
```

And you can specify the operating system and architecture that you want to create the binary for by setting `GOOS` and `GOARCH` environment variables when running `go build`. For example:

```
$ GOOS=linux GOARCH=amd64 go build {args}
```

In the next section of the book, we're going to walk through how to deploy an executable binary on an Ubuntu Linux server hosted by DigitalOcean. For this, we'll need a binary which is designed to run on a machine with a `linux/amd64` OS and architecture combination.

So let's update our `make build/api` rule so that it creates two binaries — one for use on

dego_vila@yahoo.com 30 Oct 2026

your local machine, and another for deploying to the Ubuntu Linux server.

File: Makefile

```
...

# ================================================================================ #
# BUILD
# ================================================================================ #

## build/api: build the cmd/api application
.PHONY: build/api
build/api:
 go build -ldflags='-s' -o=./bin/api ./cmd/api
 GOOS=linux GOARCH=amd64 go build -ldflags='-s' -o=./bin/linux_amd64/api ./cmd/api
```

If you're following along, go ahead and run `make build/api` again.

You should see that two binaries are now created — with the cross-compiled binary located under the `./bin/linux_amd64` directory, like so:

```
$ make build/api
go build -ldflags='-s' -o=./bin/api ./cmd/api
GOOS=linux GOARCH=amd64 go build -ldflags='-s' -o=./bin/linux_amd64/api ./cmd/api
$ tree ./bin
./bin
├── api
└── linux_amd64
    └── api
```

As a general rule, you probably don't want to commit your Go binaries into version control alongside your source code as they will significantly inflate the size of your repository.

So, if you're following along, let's quickly add an additional rule to the `.gitignore` file which instructs Git to ignore the contents of the `bin` directory.

```
$ echo 'bin/' >> .gitignore
$ cat .gitignore
.envrc
bin/
```

Additional information

Build caching

dego_vila@yahoo.com 30 Oct 2026

It's important to note that the `go build` command caches build output in the Go *build cache*. This cached output will be reused again in future builds where appropriate, which can significantly speed up the overall build time for your application.

If you're not sure where your build cache is, you can check by running the `go env GOCACHE` command:

```
$ go env GOCACHE
/home/alex/.cache/go-build
```

You should also be aware that the build cache does not automatically detect any changes to C libraries that your code imports with `cgo`. So, if you've changed a C library since the last build, you'll need to use the `-a` flag to force all packages to be rebuilt when running `go build`. Alternatively, you could use `go clean` to purge the cache:

```
$ go build -a -o=/bin/foo ./cmd/foo       # Force all packages to be rebuilt
$ go clean -cache                         # Remove everything from the build cache
```

> **Note:** If you ever run `go build` on a non-`main` package, the build output will be stored in the build cache so it can be reused, but no executable will be produced.

dego_vila@yahoo.com 30 Oct 2026

Managing and automating version numbers

Right at the start of this book, we hard-coded the version number for our application as the constant `"1.0.0"` in the `cmd/api/main.go` file.

In this chapter, we're going to take steps to make it easier to view and manage this version number, and also explain how you can generate version numbers automatically based on Git commits and integrate them into your application.

Displaying the version number

Let's start by updating our application so that we can easily check the version number by running the binary with a `-version` command-line flag, similar to this:

```
$ ./bin/api -version
Version:        1.0.0
```

Conceptually, this is fairly straightforward to implement. We need to define a boolean `version` command-line flag, check for this flag on startup, and then print out the version number and exit the application if necessary.

If you're following along, go ahead and update your `cmd/api/main.go` file like so:

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "expvar"
    "flag"
    "fmt" // New import
    "log/slog"
    "os"
    "runtime"
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/mailer"
    "greenlight.alexedwards.net/internal/models"
```

dego_vila@yahoo.com 30 Oct 2026

```
    _ "github.com/lib/pq"
)

const version = "1.0.0"

...

func main() {
    var cfg config

    flag.IntVar(&cfg.port, "port", 4000, "API server port")
    flag.StringVar(&cfg.env, "env", "development", "Environment (development|staging|production)")

    flag.StringVar(&cfg.db.dsn, "db-dsn", "", "PostgreSQL DSN")

    flag.IntVar(&cfg.db.maxOpenConns, "db-max-open-conns", 25, "PostgreSQL max open connections")
    flag.IntVar(&cfg.db.maxIdleConns, "db-max-idle-conns", 25, "PostgreSQL max idle connections")
    flag.DurationVar(&cfg.db.maxIdleTime, "db-max-idle-time", 15*time.Minute, "PostgreSQL max connection idle time")

    flag.BoolVar(&cfg.limiter.enabled, "limiter-enabled", true, "Enable rate limiter")
    flag.Float64Var(&cfg.limiter.rps, "limiter-rps", 2, "Rate limiter maximum requests per second")
    flag.IntVar(&cfg.limiter.burst, "limiter-burst", 4, "Rate limiter maximum burst")

    flag.StringVar(&cfg.smtp.host, "smtp-host", "sandbox.smtp.mailtrap.io", "SMTP host")
    flag.IntVar(&cfg.smtp.port, "smtp-port", 25, "SMTP port")
    flag.StringVar(&cfg.smtp.username, "smtp-username", "a7420fc0883489", "SMTP username")
    flag.StringVar(&cfg.smtp.password, "smtp-password", "e75ffd0a3aa5ec", "SMTP password")
    flag.StringVar(&cfg.smtp.sender, "smtp-sender", "Greenlight <no-reply@greenlight.alexedwards.net>", "SMTP sender")

    flag.Func("cors-trusted-origins", "Trusted CORS origins (space separated)", func(val string) error {
        cfg.cors.trustedOrigins = strings.Fields(val)
        return nil
    })

    // Create a new version boolean flag with the default value of false.
    displayVersion := flag.Bool("version", false, "Display version and exit")

    flag.Parse()

    // If the version flag value is true, then print out the version number and
    // immediately exit.
    if *displayVersion {
        fmt.Printf("Version:\t%s\n", version)
        os.Exit(0)
    }

    ...
}

...
```

OK, let's try this out. Go ahead and rebuild the executable binaries using `make build/api`, then run the `./bin/api` binary with the `-version` flag.

You should find that it prints out the version number and then exits, similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ make build/api
go build -ldflags="-s" -o="./bin/api" ./cmd/api
GOOS=linux GOARCH=amd64 go build -ldflags="-s" -o="./bin/linux_amd64/api" ./cmd/api

$ ./bin/api -version
Version:        1.0.0
```

> **Remember:** Boolean command-line flags *without a value* are interpreted as having the value `true`. So running our application with `-version` is the same as running it with `-version=true`.

Automated version numbering with Git

> **Important:** This part of the book is only really relevant if you use Git or a similar version control system. If you don't, then that's absolutely fine and you can skip ahead to the next chapter with no issues.

Go embeds *version control* information in your executable binaries when you run `go build` on a `main` package that is tracked with Git, Mercurial, or Fossil.

There are two ways to access this version control information — either by using the `go version -m` command on your binary, or from within your application code itself by calling `debug.ReadBuildInfo()`.

Let's take a look at both approaches.

If you're following along (and haven't done it already), please go ahead and initialize a new Git repository in the root of your project directory:

```
$ git init
Initialized empty Git repository in /home/alex/Projects/greenlight/.git/
```

Then make a new commit containing all the files in your project directory, like so:

```
$ git add .
$ git commit -m "Initial commit"
```

If you look at your commit history using the `git log` command, you'll see the *hash* for this commit.

dego_vila@yahoo.com 30 Oct 2026

```
$ git log
commit 58ca896f7a67141a2c30c820e6b993997b4cebbc (HEAD -> master)
Author: Alex Edwards <alex@alexedwards.net>
Date:   Mon Sep 14 11:41:15 2026 +0200

    Initial commit
```

In my case, the commit hash is `58ca896f7a67141a2c30c820e6b993997b4cebbc` — but yours will very likely be a different value.

Next, run `make build/api` again to generate a new binary and then use the `go version -m` command on it. Like so:

```
$ make build/api
go build -ldflags="-s" -o=./bin/api ./cmd/api
GOOS=linux GOARCH=amd64 go build -ldflags="-s" -o=./bin/linux_amd64/api ./cmd/api

$ go version -m ./bin/api
./bin/api: go1.27.0
        path    greenlight.alexedwards.net/cmd/api
        mod     greenlight.alexedwards.net      v0.0.0-20260914094115-58ca896f7a67
        dep     github.com/julienschmidt/httprouter     v1.3.0
        dep     github.com/lib/pq       v1.12.3
        dep     github.com/tomasen/realip       v0.0.0-20180522021738-f0c99a92ddce
        dep     github.com/wneessen/go-mail     v0.8.1
        dep     golang.org/x/crypto     v0.57.0
        dep     golang.org/x/text       v0.42.0
        dep     golang.org/x/time       v0.16.0
        build   -buildmode=exe
        build   -compiler=gc
        build   -ldflags=-s
        build   CGO_ENABLED=1
        build   CGO_CFLAGS=
        build   CGO_CPPFLAGS=
        build   CGO_CXXFLAGS=
        build   CGO_LDFLAGS=
        build   GOARCH=amd64
        build   GOOS=linux
        build   GOAMD64=v1
        build   vcs=git
        build   vcs.revision=58ca896f7a67141a2c30c820e6b993997b4cebbc
        build   vcs.time=2026-09-14T09:41:15Z
        build   vcs.modified=false
```

The output from `go version -m` shows us some interesting information about the binary. We can see the version of Go that it was built with (`go1.27.0` in my case), the module dependencies, and information about the build settings — including the linker flags used and the OS and architecture it was built for.

However, the things that we're most interested in right now are the `mod` line and the `vcs` build settings at the bottom.

- `vcs=git` tells us that the version control system being used is Git.

dego_vila@yahoo.com 30 Oct 2026

- `vcs.revision` is the hash for the latest Git commit.
- `vcs.time` is the time that this commit was made.
- `vcs.modified` tells us whether the code tracked by the Git repository has been modified since the commit was made. A value of `false` indicates that the code has not been modified, meaning that the binary was built using the *exact code from the* `vcs.revision` *commit*. A value of `true` indicates that the version control repository was 'dirty' when the binary was built — and the code used to build the binary may not be the exact code from the `vcs.revision` commit.

The line `mod greenlight.alexedwards.net v0.0.0-20260914094115-58ca896f7a67` tells us that the main module in the compiled binary is `greenlight.alexedwards.net` and that the module has the version `v0.0.0-20260914094115-58ca896f7a67`.

> **Important**: `v0.0.0-20260914094115-58ca896f7a67` is a pseudo-version generated automatically by Go. Because we haven't (yet) used `git tag` to tag a commit with a specific semantic version number, it starts with the *base version* `v0.0.0`, followed by the timestamp that the latest commit was made, and then the first 12 characters of the latest commit hash.

As I mentioned briefly above, all the information that you see in the `go version -m` output is also available to you at runtime. More specifically, you can access it by calling the `debug.ReadBuildInfo()` function, which will return a `debug.BuildInfo` struct that contains essentially the same information that we saw when running the `go version -m` command

Let's leverage this and adapt our `main.go` file so that the `version` value is set to the pseudo-version that we've just discussed, rather than the hardcoded constant `"1.0.0"`.

To assist with this, we'll create a small `internal/vcs` package containing a `Version()` function, like so:

```
$ mkdir internal/vcs
$ touch internal/vcs/vcs.go
```

dego_vila@yahoo.com 30 Oct 2026

File: internal/vcs/vcs.go

```
package vcs

import (
    "runtime/debug"
)

func Version() string {
    // Use debug.ReadBuildInfo() to retrieve a debug.BuildInfo struct. If this is available,
    // the ok value will be true, and we return the pseudo-version contained in the
    // Main.Version field.
    bi, ok := debug.ReadBuildInfo()
    if ok {
        return bi.Main.Version
    }

    return ""
}
```

Now that's in place, let's head back to our `main.go` file and update it to set the version number using this new `vcs.Version()` function:

File: cmd/api/main.go

```
package main

import (
    "context"
    "database/sql"
    "expvar"
    "flag"
    "fmt"
    "log/slog"
    "os"
    "runtime"
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/mailer"
    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/vcs" // New import

    _ "github.com/lib/pq"
)

// Make version a variable (rather than a constant) and set its value to vcs.Version().
var (
    version = vcs.Version()
)

...
```

Alright, let's try this out. Go ahead and rebuild the binary again…

dego_vila@yahoo.com 30 Oct 2026

```
$ make build/api
go build -ldflags="-s" -o=./bin/api ./cmd/api
GOOS=linux GOARCH=amd64 go build -ldflags="-s" -o=./bin/linux_amd64/api ./cmd/api
```

And then run it with the `-version` flag:

```
$ ./bin/api -version
Version:        v0.0.0-20260914094115-58ca896f7a67+dirty
```

Great! That's now reporting the pseudo-version number contained in the binary. But because we've changed the codebase since the previous commit, the pseudo-version now has the `+dirty` suffix to indicate that the codebase has uncommitted changes.

Let's fix that by committing our recent changes…

```
$ git add .
$ git commit -m "Generate version number automatically"
```

And when you rebuild the binary and check the version number again you should see a new version number without the `+dirty` suffix, similar to this:

```
$ make build/api
go build -ldflags="-s" -o=./bin/api ./cmd/api
GOOS=linux GOARCH=amd64 go build -ldflags="-s" -o=./bin/linux_amd64/api ./cmd/api

$ ./bin/api -version
Version:        v0.0.0-20260914100226-137493bb8ad9
```

> **Important:** Version control information is only embedded in a binary by default when you run `go build`. It is never embedded when you use `go run`, and is only embedded when using `go test` if you use the `-buildvcs=true` flag. You can see this behavior in action if you use `go run` in conjunction with our `-version` flag; no version control information is embedded and the pseudo-version reported has the value `(devel)`.
>
> ```
> $ go run ./cmd/api/ -version
> Version:        (devel)
> ```

Tagging releases

dego_vila@yahoo.com 30 Oct 2026

In some projects you may want to annotate certain Git commits with a semantic version number tag, often to denote a formal release.

To illustrate this, let's add the `v1.0.0` tag to our latest commit like so:

```
$ git tag v1.0.0
$ git log
commit 137493bb8ad92284478ca3afa5edd18ad7099b02 (HEAD -> master, tag: v1.0.0)
Author: Alex Edwards <alex@alexedwards.net>
Date:   Mon Sep 14 12:02:26 2026 +0200

    Generate version number automatically

...
```

If you rebuild the application and check the version again, you should see that it now reports `v1.0.0` as the version, instead of the automatically generated pseudo-version.

```
$ make build/api
$ ./bin/api -version
Version:        v1.0.0
```

Now, just to illustrate what happens next, let's make another (empty) commit to our codebase.

```
$ git commit --allow-empty -m "Empty commit"
[master cb0197d] Empty commit
```

At this point, our *base version* is still `v1.0.0`, but we are *ahead of the commit* that we tagged as `v1.0.0`. So now Go will no longer report `v1.0.0` as the version, and instead will go back reporting a pseudo-version again.

When you rebuild the application and check it, you should see that the version looks a bit like this:

```
$ make build/api
$ ./bin/api -version
Version:        v1.0.1-0.20260914100442-cb0197d78fc5
```

The timestamp and commit hash are those of the latest commit, just like before. But we can also see that the semantic version number part is `v1.0.1-0`, which is a little bit weird at first glance.

What's happening here is that Go is applying the following rule: *if you have the base version*

dego_vila@yahoo.com 30 Oct 2026

`vX.Y.Z`, *but your latest commits are ahead of it, then the start of the pseudo-version will be in the format* `vX.Y.(Z+1)-0`. For more information about the rationale behind this, please see the Go Modules Reference.

So all in all, this is really good. We've got a version number that is automatically generated based on our Git history and embedded in our binary, and it's easy to identify exactly what code a particular binary contains or a running application is using — all we need to do is run the binary with the `-version` flag, or call the healthcheck endpoint, and then cross-reference the version number against the Git repository history.

Additional information

Reporting just the commit timestamp and hash

If you aren't tagging commits, and want to use just the latest commit timestamp and hash as your version number, you can loop through the `debug.BuildInfo.Settings` field to extract the `vcs.time`, `vcs.revision` and `vcs.modified` values and create your own version string. Like so:

```
func Version() string {
    var (
        time     string
        revision string
        modified bool
    )

    bi, ok := debug.ReadBuildInfo()
    if ok {
        for _, s := range bi.Settings {
            switch s.Key {
            case "vcs.time":
                time = s.Value
            case "vcs.revision":
                revision = s.Value
            case "vcs.modified":
                if s.Value == "true" {
                    modified = true
                }
            }
        }
    }

    if modified {
        return fmt.Sprintf("%s-%s+dirty", time, revision)
    }

    return fmt.Sprintf("%s-%s", time, revision)
}
```

dego_vila@yahoo.com 30 Oct 2026

Using this would result in version numbers that look similar to this:

```
2026-09-14T10:04:42Z-cb0197d78fc5a2643fbba757e64dfee692360918+dirty
```

Using linker flags

Prior to Go 1.18 the idiomatic way to manage version numbers automatically was to 'burn-in' the version number when building the binary using the `-X` linker flag. Using `debug.ReadBuildInfo()` is now the preferred method, but the old approach can still be useful if you need to set the version number to something that isn't available via `debug.ReadBuildInfo()`.

For example, if you wanted to set the version number to the value of a `VERSION` environment variable *on the machine building the binary*, you could use the `-X` linker flag to 'burn-in' this value to the `main.version` variable. Like so:

File: Makefile

```
...

## build/api: build the cmd/api application
.PHONY: build/api
build/api:
    go build -ldflags='-s -X main.version=${VERSION}' -o=./bin/api ./cmd/api
    GOOS=linux GOARCH=amd64 go build -ldflags='-s -X main.version=${VERSION}' -o=./bin/linux_amd64/api ./cmd/api
```

dego_vila@yahoo.com 30 Oct 2026

Chapter 20.

Deployment and hosting

In this final section of the book we'll look at how to deploy our API application to a production server and expose it on the internet.

Every project and project team will have different technical and business needs in terms of hosting and deployment, so it's impossible to lay out a one-size-fits-all approach here.

To make the content in this section as widely applicable and portable as possible, we'll focus on hosting the application on a self-managed Linux server (something provided by a myriad of hosting companies worldwide) and using standard Linux tooling to manage server configuration and deployment.

We'll also be automating the server configuration and deployment process as much as possible, so that it's easy to make *continuous deployments* and possible to *replicate the server* in the future if you need to.

If you're planning to follow along, we'll be using DigitalOcean as the hosting provider in this book. DigitalOcean isn't free, but it's good value and the cost of running a server starts at $4 USD per month. If you don't want to use DigitalOcean, you should be able to follow basically the same approach we outline here with any other Linux hosting provider.

In terms of infrastructure and architecture, we'll run everything on a single Ubuntu Linux server. Our stack will consist of a PostgreSQL database and the executable binary for our Greenlight API, operating in much the same way that we've seen so far in this book. But in addition to this, we'll also run Caddy as a *reverse proxy* in front of the Greenlight API.

Using Caddy has a couple of benefits. It will automatically handle and terminate HTTPS connections for us — including automatically generating and managing TLS certificates via Let's Encrypt and ZeroSSL — and we can also use Caddy to easily restrict internet access to our metrics endpoint.

In this section you'll learn how to:

- Commission an Ubuntu Linux server running on DigitalOcean to host your application.
- Automate the configuration of the server — including creating user accounts, configuring the firewall and installing necessary software.
- Automate the process of updating your application and deploying changes to the server.

dego_vila@yahoo.com 30 Oct 2026

- How to run your application as a background service using systemd, as a non-root user.
- Use Caddy as a reverse proxy in front of your application to automatically manage TLS certificates and handle HTTPS connections.

dego_vila@yahoo.com 30 Oct 2026

Chapter 20.1.

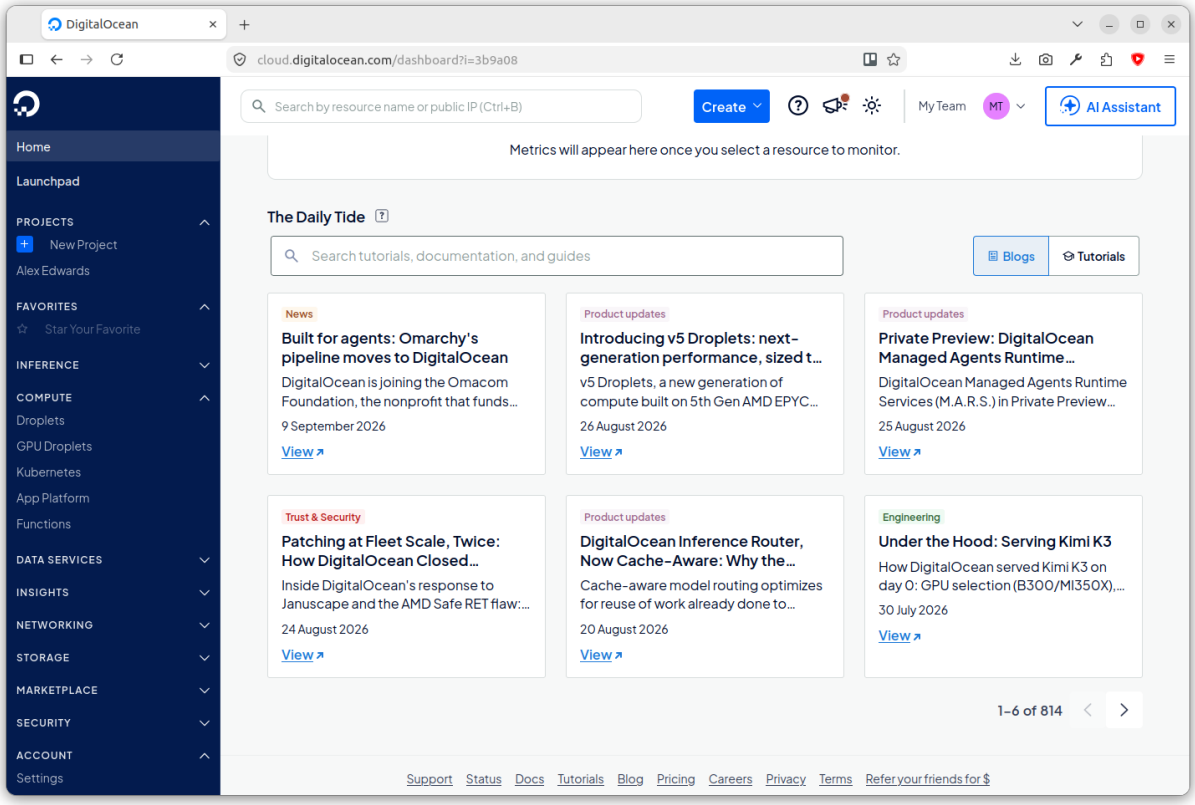
Creating a DigitalOcean droplet

The first thing that we need to do is commission a server on DigitalOcean to host our application.

Strictly speaking, what we're going to commission is actually a *virtual machine* known in DigitalOcean terminology as a *droplet*.

If you'd like to follow along with this stage of the book, you'll need to register for a DigitalOcean account if you don't already have one. As part of the registration process, you'll be prompted to confirm your email address and then add a minimum of $5 USD pre-paid credit to your account using either a credit/debit card or PayPal.

Once you've completed the registration and added the credit, you should find yourself presented with your account homepage, which should look similar to this:



Creating an SSH key

In order to log in to droplets in your DigitalOcean account you'll need an *SSH keypair*.

dego_vila@yahoo.com 30 Oct 2026

> **Suggestion:** If you're unfamiliar with SSH, SSH keys, or public-key cryptography generally, then I recommend reading through the first half of this guide to get an overview before continuing.

If you already have an SSH keypair that you're happy to use for this purpose, then that's great, and you can skip ahead to the next section.

But if you don't, you'll need to create a keypair using the `ssh-keygen` command on your local machine. Similar to this:

```
$ ssh-keygen -t rsa -b 4096 -C "greenlight@greenlight.alexedwards.net" -f $HOME/.ssh/id_rsa_greenlight
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alex/.ssh/id_rsa_greenlight
Your public key has been saved in /home/alex/.ssh/id_rsa_greenlight.pub
The key fingerprint is:
SHA256:MXASjIyE1p2BAGZ70zkUV058rA65hm3sxdIcnWLGkwg greenlight@greenlight.alexedwards.net
The key's randomart image is:
+---[RSA 4096]----+
|o*+oo*Bo+o.      |
|+.oo=E++o. o     |
|.. o +. *.= .    |
|  . . .+ % o     |
|      + S +      |
|     . B *       |
|      + o        |
|       .         |
|                 |
+----[SHA256]-----+
```

This will generate two new files in your `$HOME/.ssh` folder:

- The `$HOME/.ssh/id_rsa_greenlight` file contains your *private key*. Make sure to keep this secure, because anyone who has access to it will be able to impersonate you.
- The `$HOME/.ssh/id_rsa_greenlight.pub` file contains your *public key*. We'll upload a copy of this public key to DigitalOcean.

If you open your public key file in a text editor you should see that the content looks similar to this (line breaks added for readability):

dego_vila@yahoo.com 30 Oct 2026

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQDBheUdwyzUt056EsvUidpaGqL3zodDAffHbVVPPN7AJal5/oL6
hzmpPoCGIZueU3Fra2BPzrVtBTsNLOm0UwyQ3G8D474ETsWqlgtU3M3DBvdeI0sAaQdGxH8SkgUGswRUPNNzVG3V
xvu5aOludfZ0J1kKEkS1PzWXRll2YoKlzSO42Ne0Gzo++ZdbQWl0Y/C0sLb2sBBviIxXHU8dXmp3823yUErqkWrF
ZGGBhAco9t18gUe6MLei1+AyK+VHnRbCYvStrId7qExEs2dPzCmaEec01wCnLJ6LaRYZQnFpBRuzIZ9dTwwsJH+T
cXGo87x8MRnGY6nKNVoz8lupbSyjxkHw3PBTfkelJh+tNKiFzxs8J8WiHDfJKzrKDDQVUGbE3TYZXddGSxvi+rv0
Sfrf85zgvPjRVa2E6tjTl6nD8CnC+3wlU/01gRjVxVtPx7B9n51f+k8n2vMm8UozAv6+YruE1zZoHRHw9IvPCEy3
5B4l6GJxWzAqzjTns7kJR3Qk+xzcu2jOAehc+Do8MMx+xegzOzlgRY3mbPx8jbB3L1WmjNF6vV2BrJR/NxKoRgTf
nAQA44JaevaG4+KpbVZvkvSNoaI8uP6z8b5AptUHz/QO9Gc5M+n2EAPFjv/lNMN+0g5ZmMH8n6NpBXzP9Qmgujgn
hsGP+GmI4ZgvuUgjTQ== greenlight@greenlight.alexedwards.net
```

And if you run the `ssh-add -l` command, you should see your new SSH key listed in the output, similar to this:

```
$ ssh-add -l
4096 SHA256:MXASjIyE1p2BAGZ70zkUV058rA65hm3sxdIcnWLGkwg greenlight@greenlight.alexedwards.net (RSA)
```

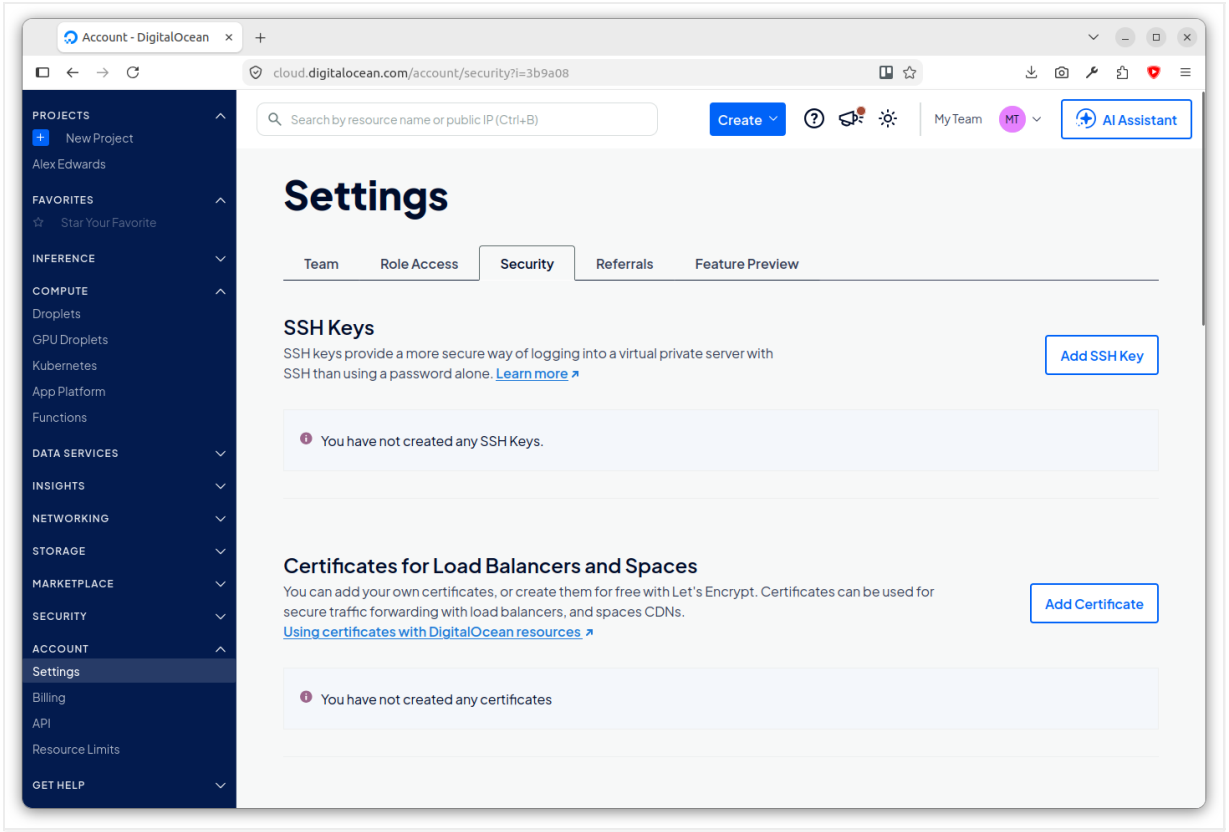
If you don't see your key listed, then please add it to your *SSH agent* like so:

```
$ ssh-add $HOME/.ssh/id_rsa_greenlight
Enter passphrase for /home/alex/.ssh/id_rsa_greenlight:
Identity added: /home/alex/.ssh/id_rsa_greenlight (greenlight@greenlight.alexedwards.net)
```

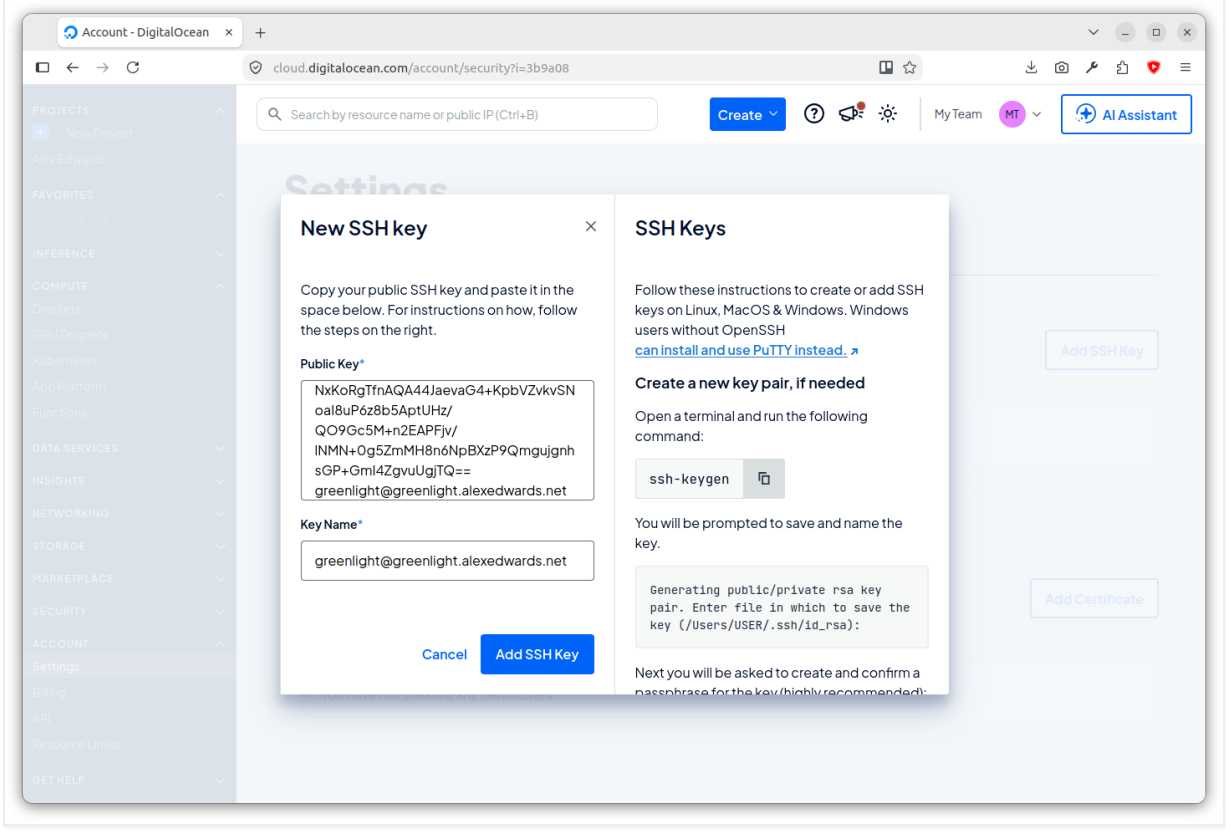
Adding the SSH key to DigitalOcean

Now you've got an SSH keypair you can use, head back to your DigitalOcean control panel and navigate to **Account › Settings** and then the **Security** tab.

dego_vila@yahoo.com 30 Oct 2026

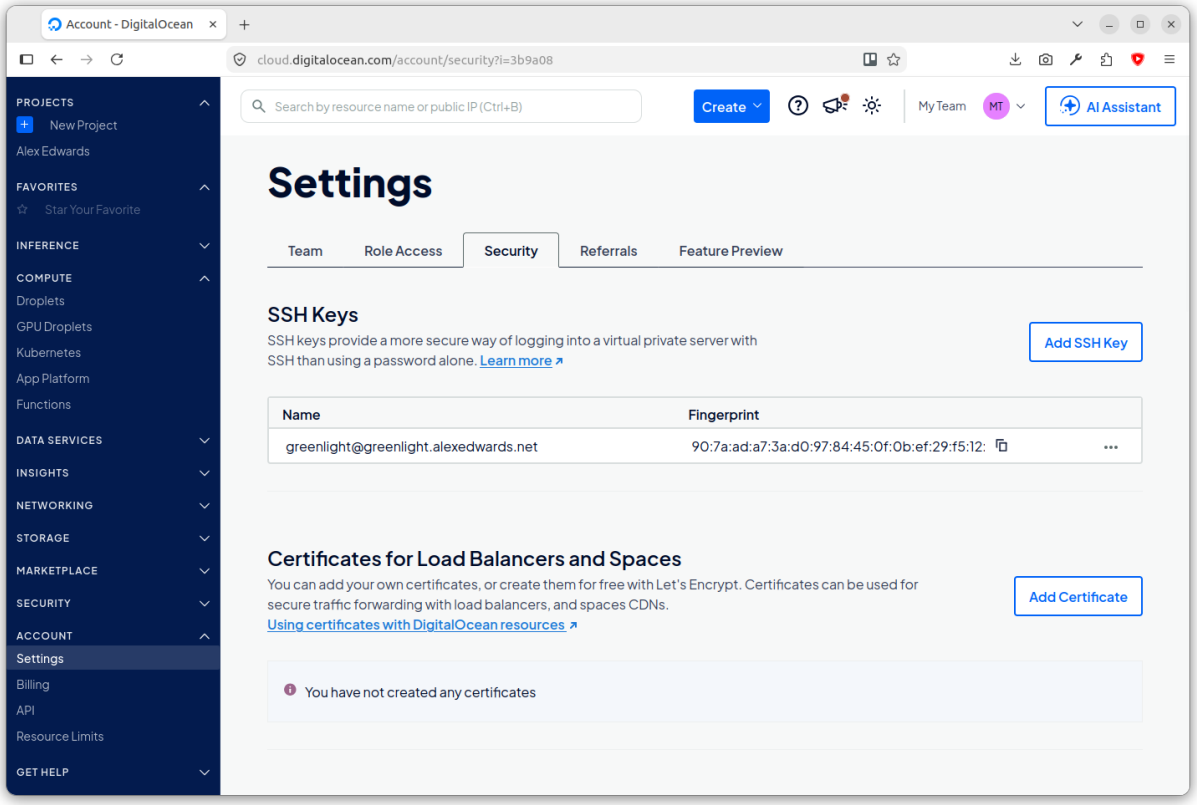


Click the **Add SSH Key** button, then in the popup window that appears paste in the text contents from your `$HOME/.ssh/id_rsa_greenlight.pub` public key file, give it a memorable name, and submit the form, similar to the screenshot below.



dego_vila@yahoo.com 30 Oct 2026

The screen should then update to confirm that your SSH key has been successfully added, like so:



Creating a droplet

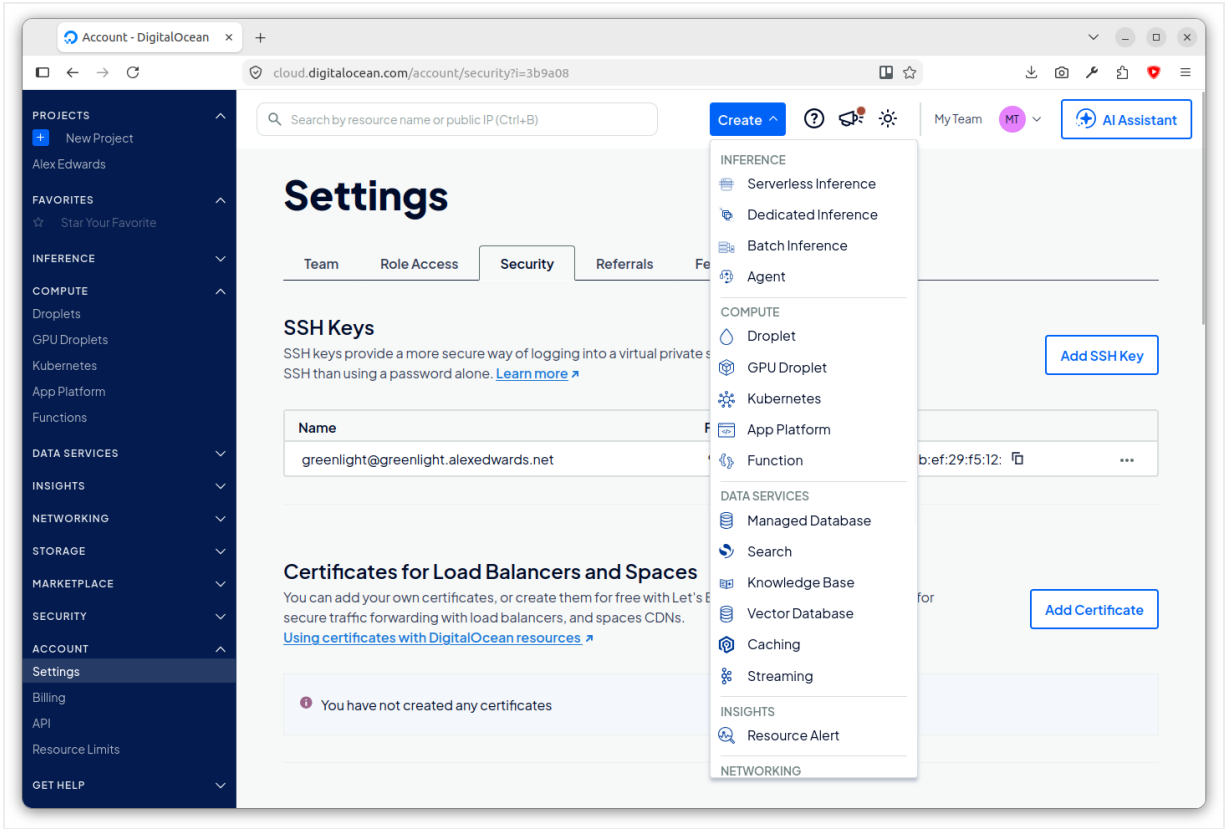
Now that you have a valid SSH key added to your account, it's time to actually create a droplet.

There are a couple of ways that you can do this. It's possible to do so programmatically via the DigitalOcean API or using the official command-line tool, and if you need to create or manage a lot of servers then I recommend using these.

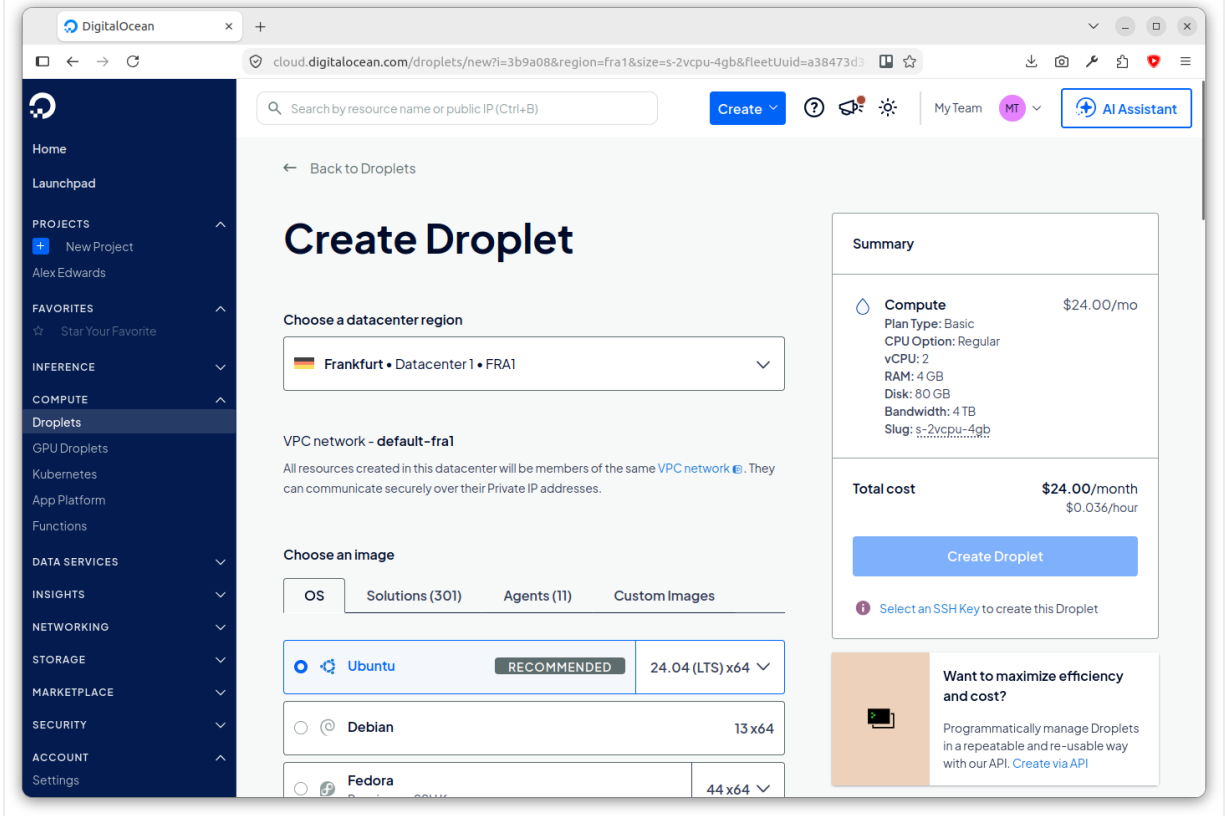
Or alternatively, it's possible to create a droplet manually via your control panel on the DigitalOcean website. This is the approach we'll take in this book, partly because it's simple enough to do as a one-off, and partly because it helps give an overview of the available droplet settings if you haven't used DigitalOcean before.

Go ahead and click the blue **Create** button at the top of the screen and select **Droplet** from the dropdown menu:

dego_vila@yahoo.com 30 Oct 2026

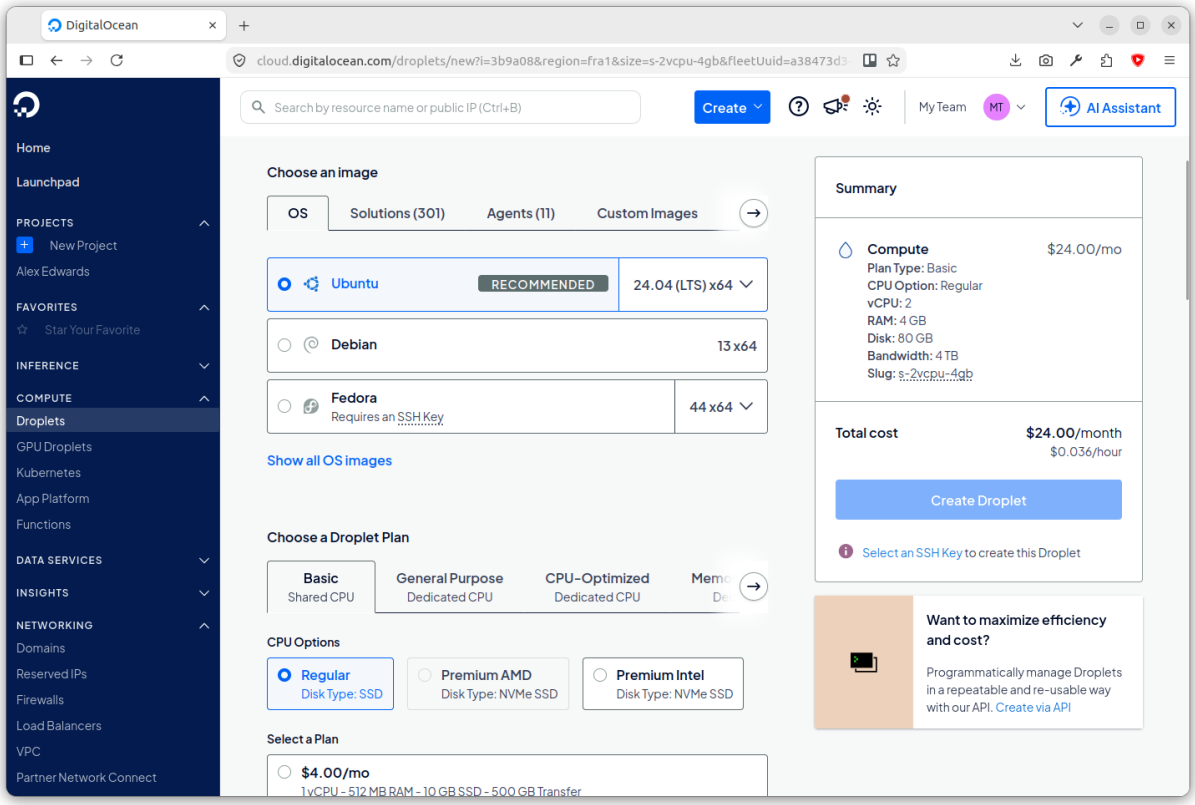


This will then take you to the options page for creating a new droplet. The first thing to choose is the data center where your droplet will be physically hosted. I'm going to choose **Frankfurt** but feel free to pick an alternative location if you like.



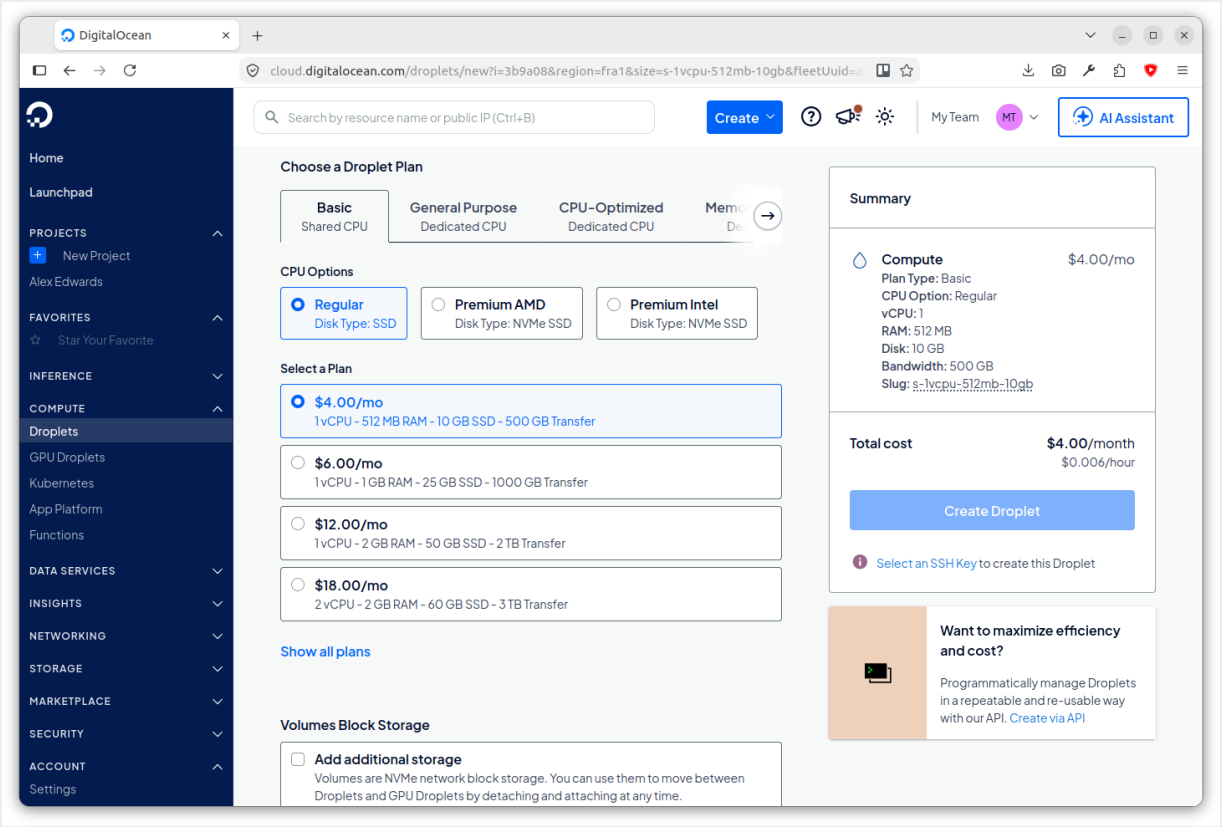
dego_vila@yahoo.com 30 Oct 2026

The next thing to choose is the operating system for your droplet. If you're following along, please select **Ubuntu 24.04 (LTS) x64**.



Then you need to pick a plan that matches the technical specifications you need for the droplet. In this case we'll select the **Basic** plan with a **Regular CPU** at **$4/month**, which will give us a virtual machine with 512MB RAM, 10GB of disk space, and 500GB of outbound data transfer each month (inbound data transfer is unrestricted).

dego_vila@yahoo.com 30 Oct 2026

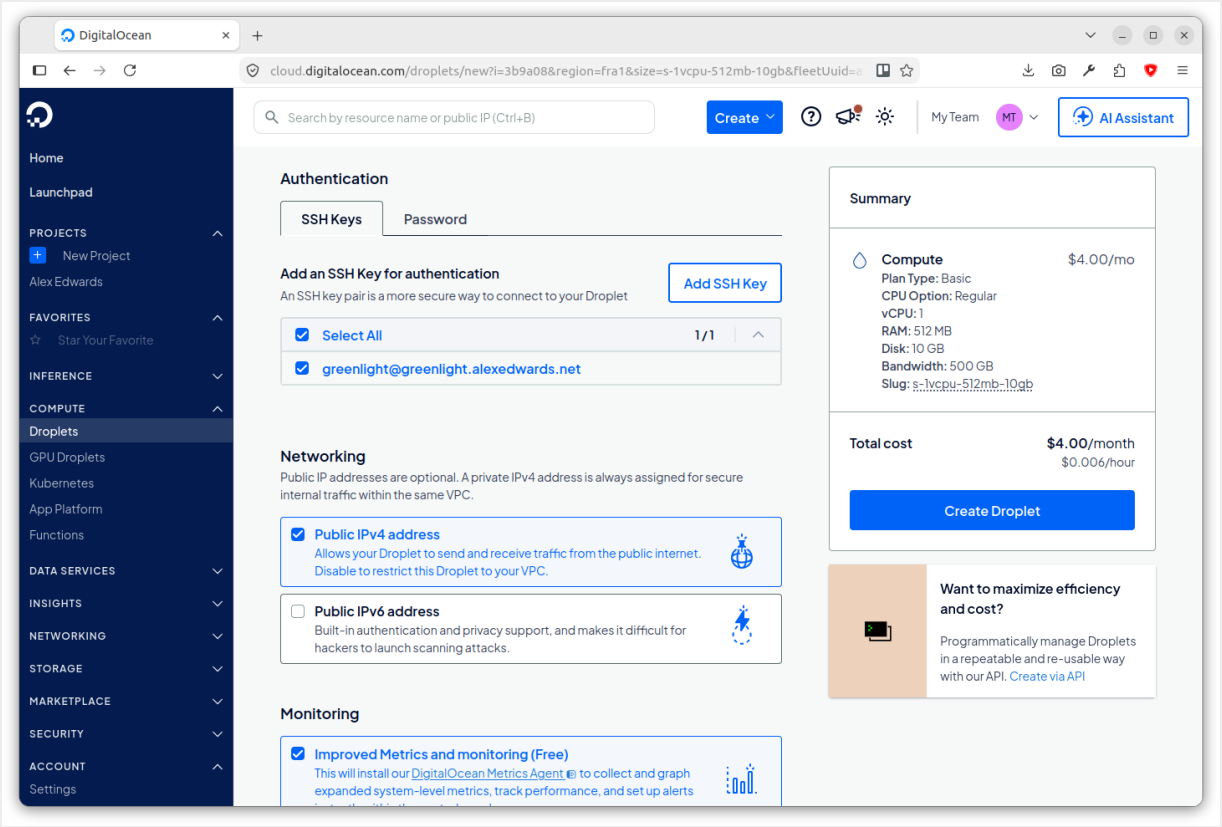


The next option allows us to add block storage to the droplet. This is essentially a droplet-independent storage volume which acts like a local hard disk and can be moved between different droplets. It's not something we need right now, so you can leave this unchecked.

You can also choose to pay an charge to enable automated daily or weekly droplet backups. If you select this option, then DigitalOcean will periodically take a 'snapshot' of your droplet and store it temporarily. You can then restore a droplet to its snapshotted state via the control panel if you ever need to. It's entirely up to you whether to enable backups or not — but it's a simple and hassle-free safety net.

Under the **Authentication** section, make sure that the SSH key that you just uploaded is checked.

dego_vila@yahoo.com 30 Oct 2026

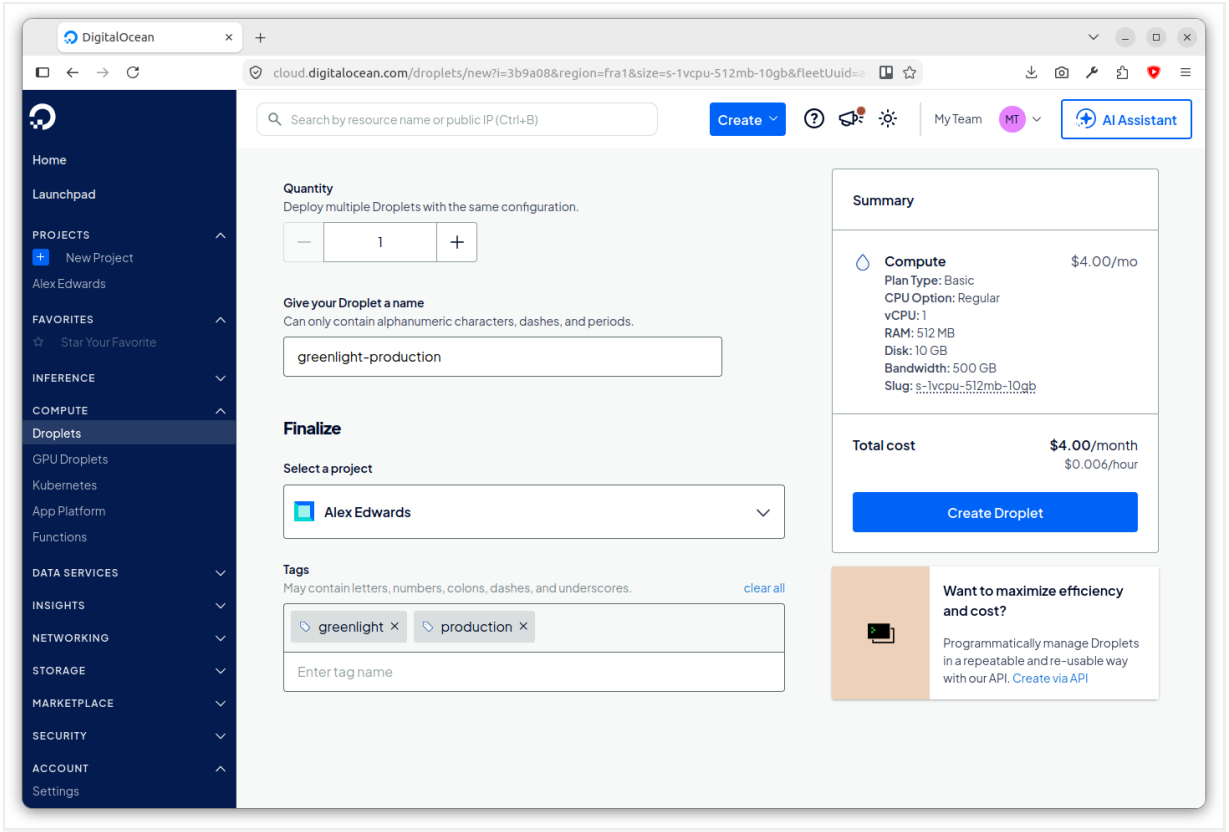


After that leave the remaining options with their default settings and scroll down to the **Give your Droplet a name** box.

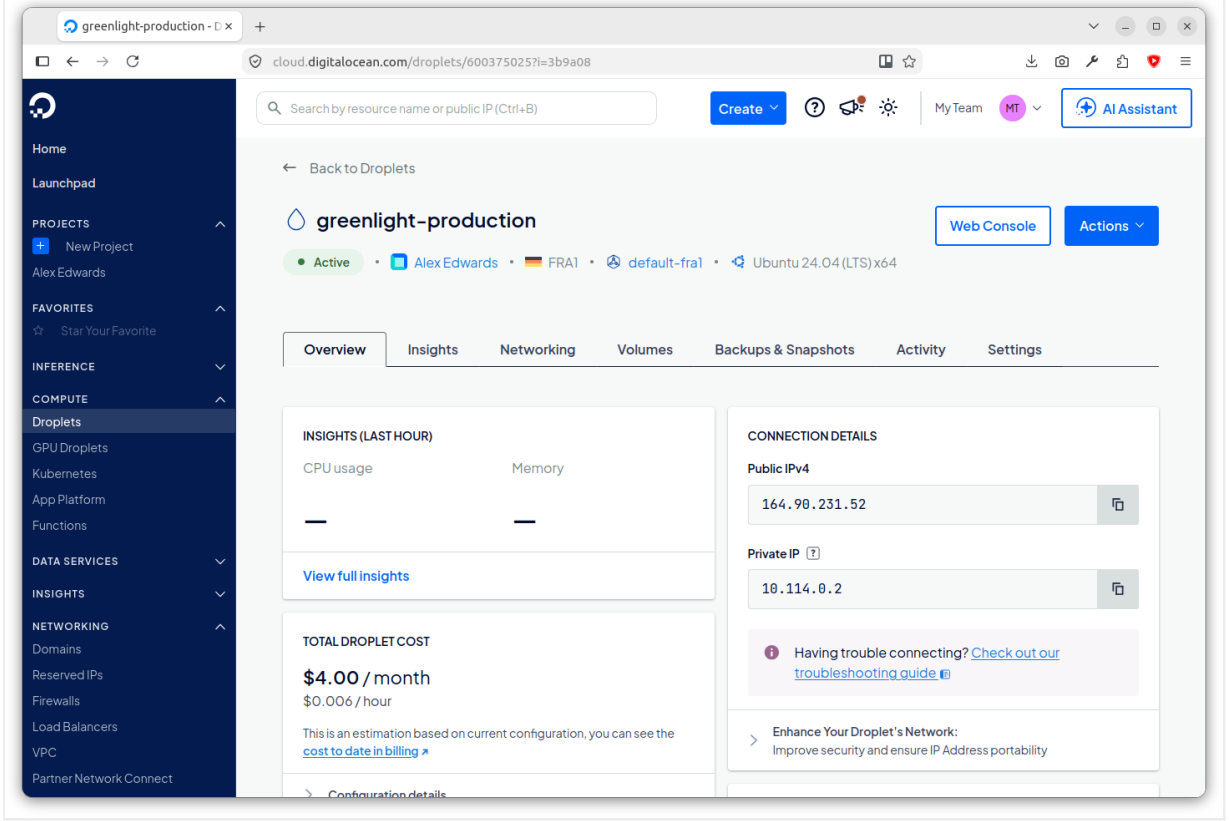
This name will be the **hostname** for the droplet. Amongst other things, the hostname is used as the main identifier for the droplet in the DigitalOcean control panel, and it's also what you'll see when you SSH into the droplet later to administer it. So you should pick a name that is sensible and easy-to-recognize. I'm going to use the hostname `greenlight-production`, but feel free to use something different if you like.

Adding tags to your droplet is completely optional, but if you do a lot of work with DigitalOcean they can be a useful way to help filter and manage droplets. I'll use the tags `greenlight` and `production` here.

dego_vila@yahoo.com 30 Oct 2026



Once that's all set, go ahead and click the **Create Droplet** button. You should see a progress bar while the droplet is being set up for you, and after a minute or two the droplet should be live and ready to use.



dego_vila@yahoo.com 30 Oct 2026

The most important thing at this point is noting the public IP address for the droplet, which in my case is `164.90.231.52`.

OK, now that the droplet is set up, it's time for the moment of truth!

Open a new terminal window and try connecting to the droplet via SSH as the `root` user, using the droplet IP address. Like so…

```
$ ssh root@164.90.231.52
The authenticity of host '164.90.231.52 (164.90.231.52)' can't be established.
ED25519 key fingerprint is SHA256:Sgc0TCVGVMs1usanEOI68SAtcvpegEaOYzO/q08eTIw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '164.90.231.52' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Sep 14 13:58:45 UTC 2026

  System load:  0.92              Processes:             103
  Usage of /:   21.6% of 8.65GB   Users logged in:       0
  Memory usage: 41%               IPv4 address for eth0: 164.90.231.52
  Swap usage:   0%                IPv4 address for eth0: 10.19.0.5

Expanded Security Maintenance for Applications is not enabled.

139 updates can be applied immediately.
114 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@greenlight-production:~#
```

> **Note:** As this is the first time that you're using SSH to connect to this droplet you'll get a message saying that "the authenticity of host can't be established". This is fine in this instance and just type `yes` to continue. If your SSH key is protected with a passphrase, you may also be prompted to enter that too.

Great, that looks like it's working well. Our Ubuntu Linux droplet is up and running, and we've been able to successfully connect to it as the `root` user over SSH.

dego_vila@yahoo.com 30 Oct 2026

You can go ahead and type `exit` to terminate the SSH connection and return to the terminal on your local machine, like so:

```
root@greenlight-production:~# exit
logout
Connection to 164.90.231.52 closed.
```

dego_vila@yahoo.com 30 Oct 2026

Server configuration and installing software

Now that our Ubuntu Linux droplet has been successfully commissioned, we need to do some housekeeping to secure the server and get it ready to use. Rather than do this manually, in this chapter we're going to create a reusable script to automate these setup tasks.

> **Note:** In this chapter we'll use a regular bash script to perform the setup tasks, but if you're happy using a configuration management tool such as Ansible, then please feel free to implement the same setup process using that instead.

At a high level, we want our setup script to do the following things:

- Update all packages on the server, including applying security updates.
- Set the server timezone (in my case I'll set it to `Europe/Berlin`) and install support for all locales.
- Create a `greenlight` user on the server, which we can use for day-to-day maintenance and for running our API application (rather than using the all-powerful `root` user account). We should also add the `greenlight` user to the `sudo` group, so that it can perform actions as `root` if necessary.
- Copy the `root` user's `$HOME/.ssh` directory into the `greenlight` user's home directory. This will enable us to authenticate as the `greenlight` user using the same SSH key pair that we used to authenticate as the `root` user. We should also force the `greenlight` user to set a new password the first time they log in.
- Configure firewall settings to only permit traffic on ports `22` (SSH), `80` (HTTP) and `443` (HTTPS). We'll also install fail2ban to automatically temporarily ban an IP address if it makes too many failed SSH login attempts.
- Install PostgreSQL. We'll also create the `greenlight` database and user, and create a system-wide `GREENLIGHT_DB_DSN` environment variable for connecting to the database.
- Install the `migrate` tool, using the pre-built binaries from GitHub, in exactly the same

dego_vila@yahoo.com 30 Oct 2026

way that we did earlier in the book.

- Install Caddy by following the official installation instructions for Ubuntu.
- Reboot the droplet.

If you're following along, create a new `remote/setup` folder in your project directory and add a script file called `01.sh`, like so:

```
$ mkdir -p remote/setup
$ touch remote/setup/01.sh
```

Then add the following code:

File: remote/setup/01.sh

```
#!/bin/bash
set -eu

# ============================================================================== #
# VARIABLES
# ============================================================================== #

# Set the timezone for the server. A full list of available timezones can be found by
# running timedatectl list-timezones.
TIMEZONE=Europe/Berlin

# Set the name of the new user to create.
USERNAME=greenlight

# Prompt to enter a password for the PostgreSQL greenlight user (rather than hard-coding
# a password in this script).
read -p "Enter password for greenlight DB user: " DB_PASSWORD

# Force all output to be presented in en_US for the duration of this script. This avoids
# any "setting locale failed" errors while this script is running, before we have
# installed support for all locales. Do not change this setting!
export LC_ALL=en_US.UTF-8

# ============================================================================== #
# SCRIPT LOGIC
# ============================================================================== #

# Enable the "universe" repository.
add-apt-repository --yes universe

# Update all software packages.
apt update

# Set the system timezone and install all locales.
timedatectl set-timezone ${TIMEZONE}
apt --yes install locales-all

# Add the new user (and give them sudo privileges).
useradd --create-home --shell "/bin/bash" --groups sudo "${USERNAME}"

# Force a password to be set for the new user the first time they log in.
passwd --delete "${USERNAME}"
```

dego_vila@yahoo.com 30 Oct 2026

```
chage --lastday 0 "${USERNAME}"

# Copy the SSH keys from the root user to the new user.
rsync --archive --chown=${USERNAME}:${USERNAME} /root/.ssh /home/${USERNAME}

# Configure the firewall to allow SSH, HTTP and HTTPS traffic.
ufw allow 22
ufw allow 80/tcp
ufw allow 443/tcp
ufw --force enable

# Install fail2ban.
apt --yes install fail2ban

# Install the migrate CLI tool.
curl -L https://github.com/golang-migrate/migrate/releases/download/v4.20.1/migrate.linux-amd64.tar.gz | tar xvz
mv migrate /usr/local/bin/migrate

# Install PostgreSQL.
apt --yes install postgresql

# Set up the greenlight DB and create a user account with the password entered earlier.
sudo -i -u postgres psql -c "CREATE DATABASE greenlight"
sudo -i -u postgres psql -d greenlight -c "CREATE EXTENSION IF NOT EXISTS citext"
sudo -i -u postgres psql -d greenlight -c "CREATE ROLE greenlight WITH LOGIN PASSWORD '${DB_PASSWORD}'"
sudo -i -u postgres psql -d greenlight -c "ALTER DATABASE greenlight OWNER TO greenlight"

# Add a DSN for connecting to the greenlight database to the system-wide environment
# variables in the /etc/environment file.
echo "GREENLIGHT_DB_DSN='postgres://greenlight:${DB_PASSWORD}@localhost/greenlight'" >> /etc/environment

# Install Caddy (see https://caddyserver.com/docs/install#debian-ubuntu-raspbian).
apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-ar
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo chmod o+r /usr/share/keyrings/caddy-stable-archive-keyring.gpg
sudo chmod o+r /etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install caddy

# Upgrade all packages.
apt --yes upgrade

echo "Script complete! Rebooting..."
reboot
```

OK, let's now run this script on our new DigitalOcean droplet. This will be a two-step process:

1. First we need to copy the script to the droplet (which we will do using `rsync`).
2. Then we need to connect to the droplet via SSH and execute the script.

Go ahead and run the following command to `rsync` the contents of the `/remote/setup` folder to the `root` user's home directory on the droplet. Remember to replace the IP address with your own!

dego_vila@yahoo.com 30 Oct 2026

```
$ rsync -rP --delete ./remote/setup root@164.90.231.52:/root
sending incremental file list
setup/
setup/01.sh
       3,513 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=0/2)
```

> **Note:** In this `rsync` command the `-r` flag indicates that we want to copy the contents of `./remote/setup` recursively, the `-P` flag indicates that we want to display progress of the transfer, and the `--delete` flag indicates that we want to delete any extraneous files from the destination directory on the droplet.

Now that a copy of our setup script is on the droplet, let's use the `ssh` command to execute the script on the remote machine as the `root` user. We'll use the `-t` flag to force *pseudo-terminal allocation*, which is useful when executing screen-based programs on a remote machine.

Go ahead and run the script, entering a password for the `greenlight` *PostgreSQL user*, like so:

```
$ ssh -t root@164.90.231.52 "bash /root/setup/01.sh"
Enter password for greenlight DB user: pa55word1234
Adding component(s) 'universe' to all repositories.
Get:1 http://mirrors.digitalocean.com/ubuntu noble InRelease [256 kB]
Hit:2 http://mirrors.digitalocean.com/ubuntu noble-updates InRelease
Hit:3 http://mirrors.digitalocean.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
...
Script complete! Rebooting...
Connection to 164.90.231.52 closed.
```

dego_vila@yahoo.com 30 Oct 2026

> **Important:** While this script is running you may get a message saying: `"A new version of configuration file /etc/ssh/sshd_config is available, but the version installed currently has been locally modified"`.
>
> This is because DigitalOcean automatically makes some adaptations to the `sshd_config` file, including disabling password-based access when you are using SSH keys for authentication. Please select `keep the local version currently installed` and then `<Ok>` to continue.
>
> You may also see a message saying: `Newer kernel available… Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.` Please `<Ok>` it to continue, and the script will automatically reboot the server.

After a few minutes the script should complete successfully and the droplet will be rebooted — which will also kick you off the SSH connection.

Connecting as the greenlight user

After waiting a minute or two for the reboot to complete, try connecting to the droplet as the `greenlight` user over SSH. This should work correctly (and the SSH key pair you created in the previous chapter should be used to authenticate the connection) *but you will be prompted to set a password*.

dego_vila@yahoo.com 30 Oct 2026

```
$ ssh greenlight@164.90.231.52
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Sep 14 16:10:06 CEST 2026

  System load:  0.49              Processes:             112
  Usage of /:   30.9% of 8.65GB   Users logged in:       1
  Memory usage: 45%               IPv4 address for eth0: 164.90.231.52
  Swap usage:   0%                IPv4 address for eth0: 10.19.0.5

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

WARNING: Your password has expired.
You must change your password now and login again!
New password:
Retype new password:
passwd: password updated successfully
Connection to 164.90.231.52 closed.
```

The password you enter here will be the regular password for the `greenlight` user on your droplet (i.e. the password you will need to type whenever you are logged in and want to execute a `sudo` command). Just to be clear, it is *not* an SSH password for connecting as the `greenlight` user.

Go ahead and enter whatever password you would like, confirm it, and check that you see a message saying that the password change has been successful. The SSH connection will then be automatically terminated.

If you reconnect as the `greenlight` user, everything should now work normally and you should be able to execute commands on the droplet. Like so:

```
$ ssh greenlight@164.90.231.52
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

...

greenlight@greenlight-production:~$ whoami
greenlight
```

dego_vila@yahoo.com 30 Oct 2026

While we're connected to the droplet, let's take a quick look around and check that some key things are working as we would expect.

First, let's check that the `migrate` binary has been downloaded and is on the system path by running it with the `-version` flag. All being well, this should print the `migrate` version number like so:

```
greenlight@greenlight-production:~$ migrate -version
4.20.1
```

Next, let's verify that PostgreSQL is running, and that we can connect to it using the DSN in the `GREENLIGHT_DB_DSN` environment variable:

```
greenlight@greenlight-production:~$ psql $GREENLIGHT_DB_DSN
psql (16.15 (Ubuntu 16.15-0ubuntu0.24.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

greenlight=> exit
```

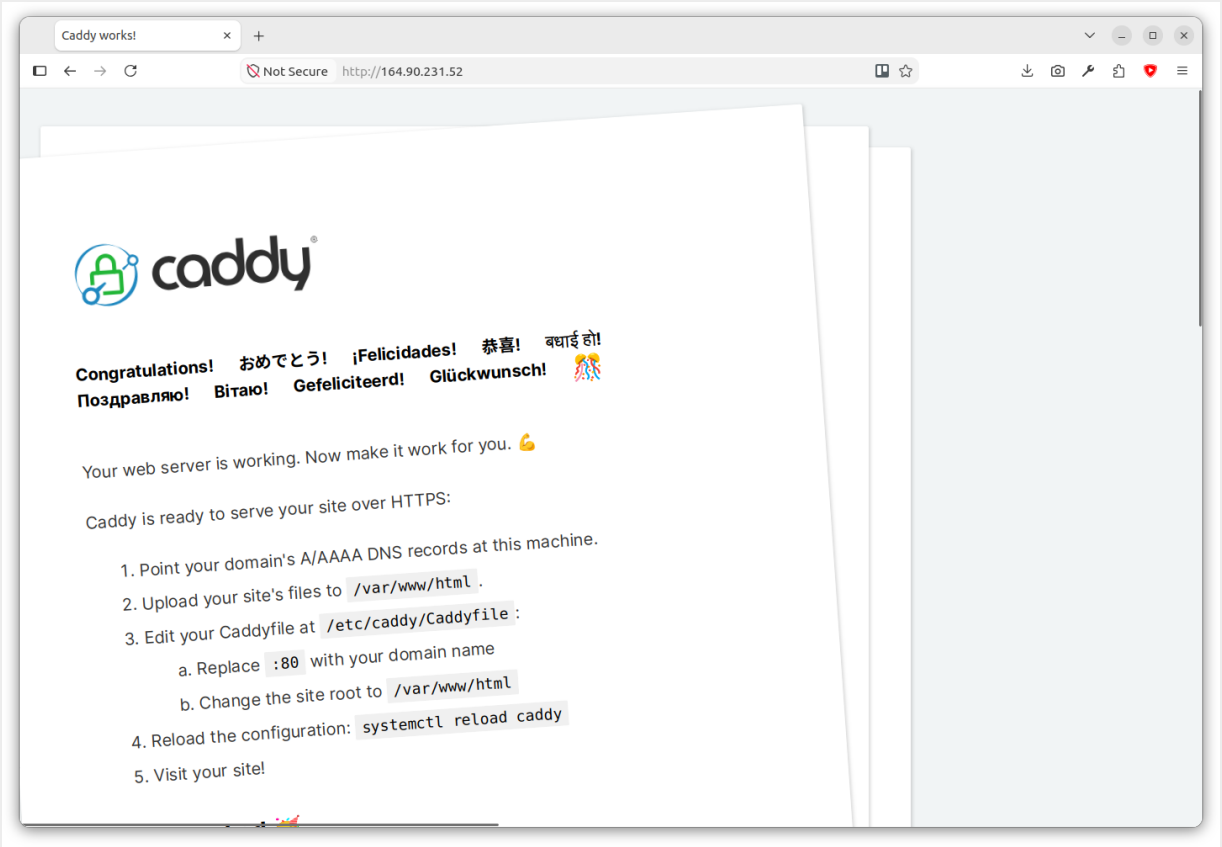
Finally, let's check that Caddy is running by calling `systemctl status caddy` to see the status of the Caddy background service. You'll need to use `sudo` here to run this command as `root`.

```
greenlight@greenlight-production:~$ sudo systemctl status caddy
● caddy.service - Caddy
     Loaded: loaded (/usr/lib/systemd/system/caddy.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-09-14 16:08:31 CEST; 4min 27s ago
       Docs: https://caddyserver.com/docs/
   Main PID: 802 (caddy)
      Tasks: 7 (limit: 503)
     Memory: 32.5M (peak: 49.3M)
        CPU: 163ms
     CGroup: /system.slice/caddy.service
             └─802 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile
...
```

You should see that the status of the Caddy service is `active (running)` — confirming that Caddy is working successfully.

This means that you should be able to visit your droplet directly in a web browser via `http://<your_droplet_ip>`, and you should see the following Caddy welcome page.

dego_vila@yahoo.com 30 Oct 2026



Connecting to the droplet

To make connecting to the droplet a bit easier, and so we don't have to remember the IP address, let's quickly add a makefile rule for initializing an SSH connection to the droplet as the `greenlight` user. Like so:

File: Makefile

```
...

# ==================================================================================== #
# PRODUCTION
# ==================================================================================== #

production_host_ip = '164.90.231.52'

## production/connect: connect to the production server
.PHONY: production/connect
production/connect:
 ssh greenlight@${production_host_ip}
```

Once that's in place, you can then connect to your droplet whenever you need to by simply typing `make production/connect`:

dego_vila@yahoo.com 30 Oct 2026

```
$ make production/connect
ssh greenlight@'164.90.231.52'
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

...

greenlight@greenlight-production:~$
```

Additional information

Future changes to the droplet configuration

If you need to make further changes to your droplet configuration or settings, you can create an additional `remote/setup/02.sh` script and then execute it in the following way:

```
$ rsync -rP --delete ./remote/setup greenlight@164.90.231.52:~
$ ssh -t greenlight@164.90.231.52 "sudo bash /home/greenlight/setup/02.sh"
```

dego_vila@yahoo.com 30 Oct 2026

Deployment and executing migrations

At this point our droplet is set up with all the software and user accounts that we need, so let's move on to deploying and running our API application.

At a very high level, our deployment process will consist of three actions:

1. Copying the application binary and SQL migration files to the droplet.
2. Executing the migrations against the PostgreSQL database on the droplet.
3. Starting the application binary as a *background service*.

For now, we'll just focus on steps 1 and 2, and tackle running our application as a *background service* in the next chapter.

Let's begin by creating a new `make production/deploy/api` rule in our makefile, which we will use to execute these first two steps automatically. Like so:

File: Makefile

```
...

# ==================================================================================== #
# PRODUCTION
# ==================================================================================== #

production_host_ip = '164.90.231.52'

## production/connect: connect to the production server
.PHONY: production/connect
production/connect:
 ssh greenlight@${production_host_ip}

## production/deploy/api: deploy the api to production
.PHONY: production/deploy/api
production/deploy/api:
 rsync -P ./bin/linux_amd64/api greenlight@${production_host_ip}:~
 rsync -rP --delete ./migrations greenlight@${production_host_ip}:~
 ssh -t greenlight@${production_host_ip} 'migrate -path ~/migrations -database $$GREENLIGHT_DB_DSN up'
```

Let's quickly break down what this new rule is doing.

First, it runs the `rsync` command to copy the `./bin/linux_amd64/api` executable binary (the one specifically built for Linux) and the `./migrations` folder into the home directory for the `greenlight` user on the droplet.

Then it uses the `ssh` command with the `-t` flag to run these database migrations on our

dego_vila@yahoo.com 30 Oct 2026

droplet as the `greenlight` user with the following command:

```
'migrate -path ~/migrations -database $$GREENLIGHT_DB_DSN up'
```

Because the `$` character has a special meaning in makefiles, we are *escaping it* in the command above by prefixing it with an additional dollar character like `$$`. This means that the command that actually runs on our droplet will be `'migrate -path ~/migrations -database $GREENLIGHT_DB_DSN up'`, and in turn, the migration will be run *using the droplet environment variable* `GREENLIGHT_DB_DSN`.

It's also important to note that we're surrounding this command with single quotes. If we used double quotes, it would be an *interpreted string* and we would need to use an additional escape character `\` like so:

```
"migrate -path ~/migrations -database \$$GREENLIGHT_DB_DSN up"
```

Alright, let's try this out!

Go ahead and execute the `make production/deploy/api` rule that we just made. You should see that all the files copy across successfully (this may take a minute or two to complete depending on your connection speed), and the migrations are then applied to your database. Like so:

dego_vila@yahoo.com 30 Oct 2026

```
$ make production/deploy/api
rsync -P ./bin/linux_amd64/api greenlight@'164.90.231.52':~
api
     11,559,175 100%  844.05kB/s    0:00:13 (xfr#1, to-chk=0/1)
rsync -rP --delete ./migrations greenlight@'164.90.231.52':~
sending incremental file list
migrations/
migrations/000001_create_movies_table.down.sql
             28 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=11/13)
migrations/000001_create_movies_table.up.sql
            310 100%  302.73kB/s    0:00:00 (xfr#2, to-chk=10/13)
migrations/000002_add_movies_check_constraints.down.sql
            198 100%  193.36kB/s    0:00:00 (xfr#3, to-chk=9/13)
migrations/000002_add_movies_check_constraints.up.sql
            289 100%  282.23kB/s    0:00:00 (xfr#4, to-chk=8/13)
migrations/000003_add_movies_indexes.down.sql
             78 100%   76.17kB/s    0:00:00 (xfr#5, to-chk=7/13)
migrations/000003_add_movies_indexes.up.sql
            170 100%  166.02kB/s    0:00:00 (xfr#6, to-chk=6/13)
migrations/000004_create_users_table.down.sql
             27 100%   26.37kB/s    0:00:00 (xfr#7, to-chk=5/13)
migrations/000004_create_users_table.up.sql
            320 100%  312.50kB/s    0:00:00 (xfr#8, to-chk=4/13)
migrations/000005_create_tokens_table.down.sql
             28 100%   27.34kB/s    0:00:00 (xfr#9, to-chk=3/13)
migrations/000005_create_tokens_table.up.sql
            203 100%  198.24kB/s    0:00:00 (xfr#10, to-chk=2/13)
migrations/000006_add_permissions.down.sql
             73 100%   71.29kB/s    0:00:00 (xfr#11, to-chk=1/13)
migrations/000006_add_permissions.up.sql
            478 100%  466.80kB/s    0:00:00 (xfr#12, to-chk=0/13)
ssh -t greenlight@'164.90.231.52' 'migrate -path ~/migrations -database $GREENLIGHT_DB_DSN up'
1/u create_movies_table (16.707687ms)
2/u add_movies_check_constraints (27.199029ms)
3/u add_movies_indexes (40.874205ms)
4/u create_users_table (61.139392ms)
5/u create_tokens_table (77.450502ms)
6/u add_permissions (96.037463ms)
Connection to 164.90.231.52 closed.
```

Let’s quickly connect to our droplet and verify that everything has worked.

```
$ make production/connect
ssh greenlight@'164.90.231.52'
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

...
greenlight@greenlight-production:~$ ls -R
.:
api  migrations

./migrations:
000001_create_movies_table.down.sql           000004_create_users_table.down.sql
000001_create_movies_table.up.sql             000004_create_users_table.up.sql
000002_add_movies_check_constraints.down.sql  000005_create_tokens_table.down.sql
000002_add_movies_check_constraints.up.sql    000005_create_tokens_table.up.sql
000003_add_movies_indexes.down.sql            000006_add_permissions.down.sql
000003_add_movies_indexes.up.sql              000006_add_permissions.up.sql
```

So far so good. We can see that the home directory of our `greenlight` user contains the `api`

dego_vila@yahoo.com 30 Oct 2026

executable binary and a folder containing the migration files.

Let's also connect to the database using `psql` and verify that the tables have been created by using the `\dt` meta command. Your output should look similar to this:

```
greenlight@greenlight-production:~$ psql $GREENLIGHT_DB_DSN
psql (12.6 (Ubuntu 12.6-0ubuntu0.20.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

greenlight=> \dt
               List of relations
 Schema |       Name        | Type  |   Owner
--------+-------------------+-------+------------
 public | movies            | table | greenlight
 public | permissions       | table | greenlight
 public | schema_migrations | table | greenlight
 public | tokens            | table | greenlight
 public | users             | table | greenlight
 public | users_permissions | table | greenlight
(6 rows)
```

Running the API

While we're connected to the droplet, let's try running the `api` executable binary. We can't listen for incoming connections on port `80` because Caddy is already using this, so we'll listen on the unrestricted port `4000` instead.

If you've been following along, port `4000` on your droplet should currently be blocked by the firewall rules, so we'll need to relax this temporarily to allow incoming requests. Go ahead and do that like so:

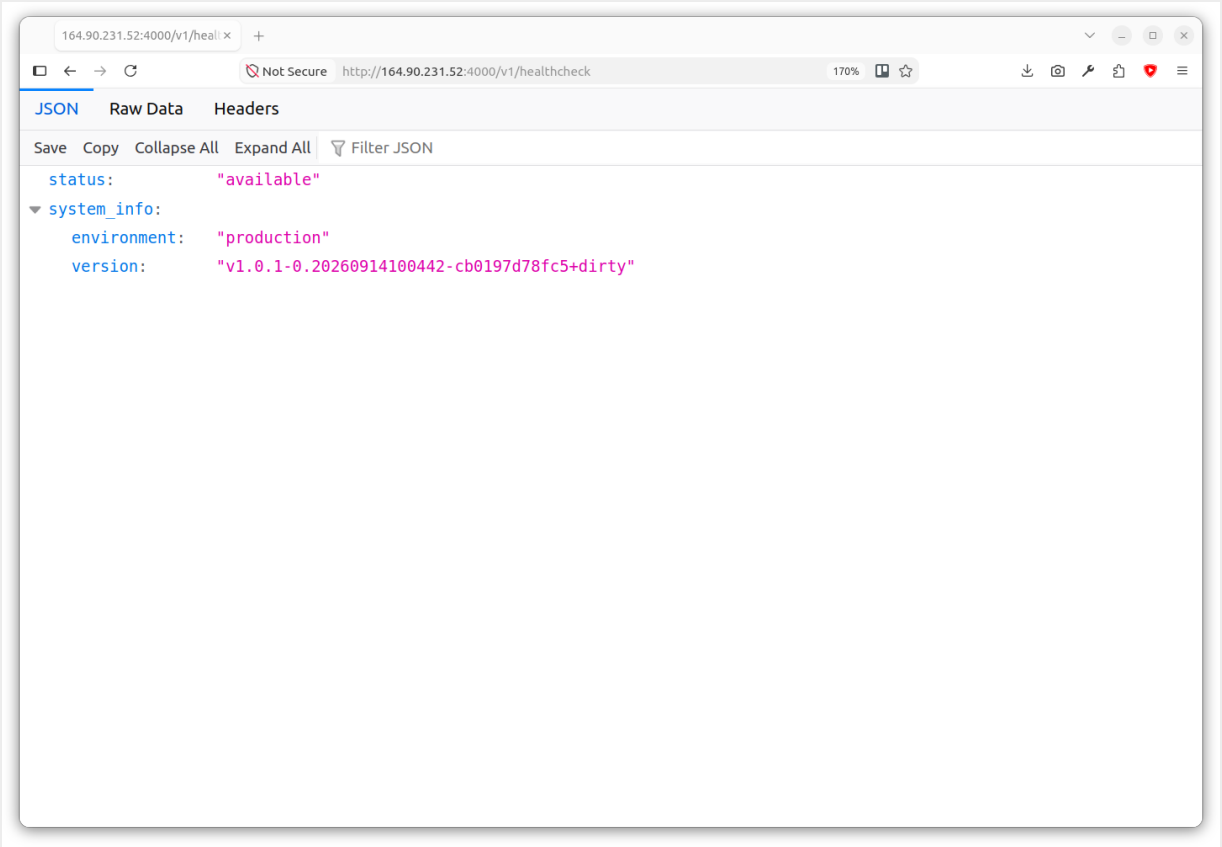
```
greenlight@greenlight-production:~$ sudo ufw allow 4000/tcp
Rule added
Rule added (v6)
```

And then start the API with the following command:

```
greenlight@greenlight-production:~$ ./api -port=4000 -db-dsn=$GREENLIGHT_DB_DSN -env=production
time=2026-09-14T17:22:19.256+02:00 level=INFO msg="database connection pool established"
time=2026-09-14T17:22:19.257+02:00 level=INFO msg="starting server" addr=:4000 env=production
```

At this point you should be able to visit `http://<your_droplet_ip>:4000/v1/healthcheck` in your web browser and get a successful response from the healthcheck endpoint, similar to this:

dego_vila@yahoo.com 30 Oct 2026



Lastly, head back to your SSH terminal window and press `Ctrl+C` to stop the API from running on your droplet. You should see it gracefully shut down like so:

```
greenlight@greenlight-production:~$ /home/greenlight/api -port=4000 -db-dsn=$GREENLIGHT_DB_DSN -env=production
time=2026-09-14T17:24:20.795+02:00 level=INFO msg="database connection pool established"
time=2026-09-14T17:24:20.795+02:00 level=INFO msg="starting server" addr=:4000 env=production
^Ctime=2026-09-14T17:29:50.680+02:00 level=INFO msg="stopping server" addr=:4000 signal=interrupt
time=2026-09-14T17:29:50.680+02:00 level=INFO msg="waiting for background tasks"
time=2026-09-14T17:29:50.681+02:00 level=INFO msg="shutdown complete"
```

dego_vila@yahoo.com 30 Oct 2026

Running the API as a background service

Now that we know our API executable works fine on our production droplet, the next step is to configure it to run as a *background service*, including starting up automatically when the droplet is rebooted.

There are a few different tools we could use to do this, but in this book we will use systemd — a collection of tools for managing services that ships with Ubuntu (and many other Linux distributions).

In order to run our API application as a background service, the first thing we need to do is make a unit file, which informs systemd how and when to run the service.

If you're following along, head back to a terminal window on your local machine and create a new `remote/production/api.service` file in your project directory:

```
$ mkdir remote/production
$ touch remote/production/api.service
```

And then add the following markup:

dego_vila@yahoo.com 30 Oct 2026

File: remote/production/api.service

```
[Unit]
# Description is a human-readable name for the service.
Description=Greenlight API service

# Wait until PostgreSQL is running and the network is "up" before starting the service.
After=postgresql.service
After=network-online.target
Wants=network-online.target

# Configure service start rate limiting. If the service is (re)started more than 5 times
# in 600 seconds then don't permit it to start anymore.
StartLimitIntervalSec=600
StartLimitBurst=5

[Service]
# Execute the API binary as the greenlight user, loading the environment variables from
# /etc/environment and using the working directory /home/greenlight.
Type=exec
User=greenlight
Group=greenlight
EnvironmentFile=/etc/environment
WorkingDirectory=/home/greenlight
ExecStart=/home/greenlight/api -port=4000 -db-dsn=${GREENLIGHT_DB_DSN} -env=production

# Automatically restart the service after a 5-second wait if it exits with a non-zero
# exit code. If it restarts more than 5 times in 600 seconds, then the rate limit we
# configured above will be hit and it won't be restarted anymore.
Restart=on-failure
RestartSec=5

[Install]
# Start the service automatically at boot time (the 'multi-user.target' describes a boot
# state when the system will accept logins).
WantedBy=multi-user.target
```

Now that we've got a unit file set up, the next step is to install this unit file on our droplet and start up the service. Essentially we need to do three things:

1. To install the unit file, we need to copy it into the `/etc/systemd/system/` folder on our droplet. Because this folder is owned by the `root` user on our droplet, we need to break this down into two steps: first we need to copy the unit file into the `greenlight` user's home directory, and secondly use the `sudo mv` command to move it to its final location.
2. Then we need to run the `systemctl enable api` command on our droplet to make `systemd` aware of the new unit file and automatically enable the service when the droplet is rebooted.
3. Finally, we need to run `systemctl restart api` to start (or restart) the service.

All three of these tasks will need to be run with `sudo`.

Let's update the `production/deploy/api` makefile rule to automate these tasks for us. Like so:

dego_vila@yahoo.com 30 Oct 2026

File: Makefile

```
...

# ================================================================================ #
# PRODUCTION
# ================================================================================ #

production_host_ip = '164.90.231.52'

...

## production/deploy/api: deploy the api to production
.PHONY: production/deploy/api
production/deploy/api:
 rsync -P ./bin/linux_amd64/api greenlight@${production_host_ip}:~
 rsync -rP --delete ./migrations greenlight@${production_host_ip}:~
 rsync -P ./remote/production/api.service greenlight@${production_host_ip}:~
 ssh -t greenlight@${production_host_ip} '\
  migrate -path ~/migrations -database $$GREENLIGHT_DB_DSN up \
  && sudo mv ~/api.service /etc/systemd/system/ \
  && sudo systemctl enable api \
  && sudo systemctl restart api \
 '
```

Save the makefile, then go ahead and run the `production/deploy/api` rule again. You should be prompted to enter the password for the `greenlight` user when it's executing, and the rule should complete without any errors. The output you see should look similar to this:

dego_vila@yahoo.com 30 Oct 2026

```
$ make production/deploy/api
rsync -P ./bin/linux_amd64/api greenlight@'164.90.231.52':~
api
     11,559,175 100%  787.18MB/s    0:00:00 (xfr#1, to-chk=0/1)
rsync -rP --delete ./migrations greenlight@'164.90.231.52':~
sending incremental file list
migrations/000001_create_movies_table.down.sql
             28 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=11/13)
migrations/000001_create_movies_table.up.sql
            310 100%  302.73kB/s    0:00:00 (xfr#2, to-chk=10/13)
migrations/000002_add_movies_check_constraints.down.sql
            198 100%   64.45kB/s    0:00:00 (xfr#3, to-chk=9/13)
migrations/000002_add_movies_check_constraints.up.sql
            289 100%   70.56kB/s    0:00:00 (xfr#4, to-chk=8/13)
migrations/000003_add_movies_indexes.down.sql
             78 100%   19.04kB/s    0:00:00 (xfr#5, to-chk=7/13)
migrations/000003_add_movies_indexes.up.sql
            170 100%   33.20kB/s    0:00:00 (xfr#6, to-chk=6/13)
migrations/000004_create_users_table.down.sql
             27 100%    5.27kB/s    0:00:00 (xfr#7, to-chk=5/13)
migrations/000004_create_users_table.up.sql
            320 100%   62.50kB/s    0:00:00 (xfr#8, to-chk=4/13)
migrations/000005_create_tokens_table.down.sql
             28 100%    4.56kB/s    0:00:00 (xfr#9, to-chk=3/13)
migrations/000005_create_tokens_table.up.sql
            203 100%   33.04kB/s    0:00:00 (xfr#10, to-chk=2/13)
migrations/000006_add_permissions.down.sql
             73 100%   11.88kB/s    0:00:00 (xfr#11, to-chk=1/13)
migrations/000006_add_permissions.up.sql
            478 100%   77.80kB/s    0:00:00 (xfr#12, to-chk=0/13)
rsync -P ./remote/production/api.service greenlight@'164.90.231.52':~
api.service
          1,266 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=0/1)
ssh -t greenlight@'164.90.231.52' '\
        migrate -path ~/migrations -database $GREENLIGHT_DB_DSN up \
        && sudo mv ~/api.service /etc/systemd/system/ \
        && sudo systemctl enable api \
        && sudo systemctl restart api \
'
no change
[sudo] password for greenlight:
Created symlink /etc/systemd/system/multi-user.target.wants/api.service → /etc/systemd/system/api.service.
Connection to 164.90.231.52 closed.
```

Next let's connect to the droplet and check the status of our new `api` service using the `sudo systemctl status api` command:

dego_vila@yahoo.com 30 Oct 2026

```
$ make production/connect
greenlight@greenlight-production:~$ sudo systemctl status api
● api.service - Greenlight API service
     Loaded: loaded (/etc/systemd/system/api.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-09-17 10:37:18 CEST; 1min 6s ago
   Main PID: 37503 (api)
      Tasks: 5 (limit: 503)
     Memory: 12.1M (peak: 12.3M)
        CPU: 36ms
     CGroup: /system.slice/api.service
             └─37503 /home/greenlight/api -port=4000 -db-dsn=postgres://greenlight:pa55word1234@localhost/greenlight -env=proc

Sep 17 10:37:18 greenlight-production systemd[1]: Starting api.service - Greenlight API service...
Sep 17 10:37:18 greenlight-production systemd[1]: Started api.service - Greenlight API service.
Sep 17 10:37:18 greenlight-production api[37503]: time=2026-09-17T10:37:18.168+02:00 level=INFO msg="database connection pool
Sep 17 10:37:18 greenlight-production api[37503]: time=2026-09-17T10:37:18.170+02:00 level=INFO msg="starting server" addr=:40
```

Great! This confirms that our `api` service is running successfully in the background and, in my case, that it has the PID (process ID) `37503`.

Out of interest, let's also quickly list the running processes for our `greenlight` user:

```
greenlight@greenlight-production:~$ ps -U greenlight
  PID TTY          TIME CMD
  37503 ?        00:00:00 api
  37525 ?        00:00:00 systemd
  37527 ?        00:00:00 (sd-pam)
  37588 ?        00:00:00 sshd
  37589 pts/0    00:00:00 bash
  37606 pts/0    00:00:00 ps
```

That tallies — we can see the `api` process listed here with the PID `37503`, confirming that it is indeed our `greenlight` user who is running the `api` binary.

Lastly, let's check from an external perspective and try making a request to our healthcheck endpoint again, either in a browser or with `curl`. You should get a successful response like so:

```
$ curl 164.90.231.52:4000/v1/healthcheck
{
 "status": "available",
 "system_info": {
  "environment": "production",
  "version": "v1.0.1-0.20260914100442-cb0197d78fc5+dirty"
 }
}
```

Restarting after reboot

By running the `systemctl enable api` command in our makefile after we copied across the

dego_vila@yahoo.com 30 Oct 2026

systemd unit file, our API service should be started up automatically after the droplet is rebooted.

If you like, you can verify this yourself. While you're connected to the droplet via SSH, go ahead and reboot it (make sure you are connected to the droplet and don't accidentally reboot your local machine!).

```
greenlight@greenlight-production:~$ sudo reboot
greenlight@greenlight-production:~$ Connection to 164.90.231.52 closed by remote host.
Connection to 164.90.231.52 closed.
make: *** [Makefile:91: production/connect] Error 255
```

Wait a minute for the reboot to complete and for the droplet to come back online. Then you should be able to reconnect and use `systemctl status` to check that the service is running again. Like so:

```
$ make production/connect
greenlight@greenlight-production:~$ sudo systemctl status api.service
[sudo] password for greenlight:
● api.service - Greenlight API service
     Loaded: loaded (/etc/systemd/system/api.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-09-17 10:45:44 CEST; 25s ago
   Main PID: 1138 (api)
      Tasks: 5 (limit: 503)
     Memory: 12.3M (peak: 12.5M)
        CPU: 36ms
     CGroup: /system.slice/api.service
             └─1138 /home/greenlight/api -port=4000 -db-dsn=postgres://greenlight:pa55word1234@localhost/greenlight -env=produ

Sep 17 10:45:44 greenlight-production systemd[1]: Starting api.service - Greenlight API service...
Sep 17 10:45:44 greenlight-production systemd[1]: Started api.service - Greenlight API service.
Sep 17 10:45:44 greenlight-production api[1138]: time=2026-09-17T10:45:44.572+02:00 level=INFO msg="database connection pool e
Sep 17 10:45:44 greenlight-production api[1138]: time=2026-09-17T10:45:44.573+02:00 level=INFO msg="starting server" addr=:400
```

Disable port 4000

If you've been following along with the steps in this section of the book, let's revert the temporary firewall change that we made earlier and disallow traffic on port 4000 again.

dego_vila@yahoo.com 30 Oct 2026

```
greenlight@greenlight-production:~$ sudo ufw delete allow 4000/tcp
Rule deleted
Rule deleted (v6)
greenlight@greenlight-production:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
22                         ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
443/tcp                    ALLOW       Anywhere
22 (v6)                    ALLOW       Anywhere (v6)
80/tcp (v6)                ALLOW       Anywhere (v6)
443/tcp (v6)               ALLOW       Anywhere (v6)
```

Additional information

Listening on a restricted port

If you're *not* planning to run your application behind a reverse proxy, and want to listen for requests directly on port 80 or 443, you'll need to set up your unit file so that the service has the `CAP_NET_BIND_SERVICE` capability (which will allow it to bind to a restricted port). For example:

```
[Unit]
Description=Greenlight API service

After=postgresql.service
After=network-online.target
Wants=network-online.target

StartLimitIntervalSec=600
StartLimitBurst=5

[Service]
Type=exec
User=greenlight
Group=greenlight
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_BIND_SERVICE
EnvironmentFile=/etc/environment
WorkingDirectory=/home/greenlight
ExecStart=/home/greenlight/api -port=80 -db-dsn=${GREENLIGHT_DB_DSN} -env=production

Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Viewing logs

dego_vila@yahoo.com 30 Oct 2026

It's possible to view the logs for your background service using the `journalctl` command, like so:

```
$ sudo journalctl -u api
Sep 17 10:37:18 greenlight-production systemd[1]: Starting api.service - Greenlight API service...
Sep 17 10:37:18 greenlight-production systemd[1]: Started api.service - Greenlight API service.
Sep 17 10:37:18 greenlight-production api[37503]: time=2026-09-17T10:37:18.168+02:00 level=INFO msg="database connection pool
Sep 17 10:37:18 greenlight-production api[37503]: time=2026-09-17T10:37:18.170+02:00 level=INFO msg="starting server" addr=:40
...
```

The `journalctl` command is really powerful and offers a wide variety of parameters that you can use to filter your log messages and customize the formatting. This article provides a great introduction to the details of `journalctl` and is well worth a read.

Configuring the SMTP provider

Our unit file is currently set up to start our API with the following command:

```
ExecStart=/home/greenlight/api -port=4000 -db-dsn=${GREENLIGHT_DB_DSN} -env=production
```

It's important to remember that apart from the `port`, `db-dsn` and `env` flags that we're specifying here, our application will still be using the default values for the other settings which are hardcoded into the `cmd/api/main.go` file — including the SMTP credentials for your Mailtrap inbox. Under normal circumstances, you would want to set your production SMTP credentials as part of this command in the unit file too.

Additional unit file options

Systemd unit files offer a huge range of configuration options, and we've only just scratched the surface in this chapter. For more information, the Understanding Systemd Units and Unit Files article is a really good overview, and you can find comprehensive (albeit dense) information in the man pages.

There is also this gist which talks through the various security options that you can use to help harden your service.

dego_vila@yahoo.com 30 Oct 2026

Using Caddy as a reverse proxy

We're now in the position where we have our Greenlight API application running as a background service on our droplet, and listening for HTTP requests on port `4000`. And we also have Caddy running as a background service and listening for HTTP requests on port `80`.

So the next step in setting up our production environment is to configure Caddy to act as a *reverse proxy* and forward any HTTP requests that it receives onward to our API.

The simplest way to configure Caddy is to create a *Caddyfile* — which contains a series of rules describing what we want Caddy to do. If you're following along, please go ahead and create a new `remote/production/Caddyfile` file in your project directory:

```
$ touch remote/production/Caddyfile
```

And then add the following content, making sure to replace the IP address with the address of your own droplet:

```
File: remote/production/Caddyfile
```

```
http://164.90.231.52 {
  reverse_proxy localhost:4000
}
```

As you can probably guess by looking at it, this rule tells Caddy that we want it to listen for HTTP requests to `164.90.231.52` and then act as a reverse proxy, forwarding the request to port `4000` on the local machine (where our API is listening).

> **Hint:** Caddyfiles have *a lot* of available settings and we'll demonstrate some of the more common ones as we progress through this chapter. But if you plan on using Caddy in your own production projects, then I recommend reading through the official Caddyfile documentation, which is excellent.

The next thing we need to do is upload this Caddyfile to our droplet and reload Caddy for the changes to take effect.

dego_vila@yahoo.com 30 Oct 2026

To manage this, let's update our `production/deploy/api` rule. This will follow the same basic pattern that we used in the previous chapter for uploading our `api.service` file, but instead we'll copy our Caddyfile to `/etc/caddy/Caddyfile` on the server.

Like so:

File: Makefile

```
# ==================================================================================== #
# PRODUCTION
# ==================================================================================== #

production_host_ip = '164.90.231.52'

...

## production/deploy/api: deploy the api to production
.PHONY: production/deploy/api
production/deploy/api:
 rsync -P ./bin/linux_amd64/api greenlight@${production_host_ip}:~
 rsync -rP --delete ./migrations greenlight@${production_host_ip}:~
 rsync -P ./remote/production/api.service greenlight@${production_host_ip}:~
 rsync -P ./remote/production/Caddyfile greenlight@${production_host_ip}:~
 ssh -t greenlight@${production_host_ip} '\
  migrate -path ~/migrations -database $$GREENLIGHT_DB_DSN up \
  && sudo mv ~/api.service /etc/systemd/system/ \
  && sudo systemctl enable api \
  && sudo systemctl restart api \
  && sudo mv ~/Caddyfile /etc/caddy/ \
  && sudo systemctl reload caddy \
 '
```

Once you've made that change, go ahead and run the rule to deploy the Caddyfile into production:

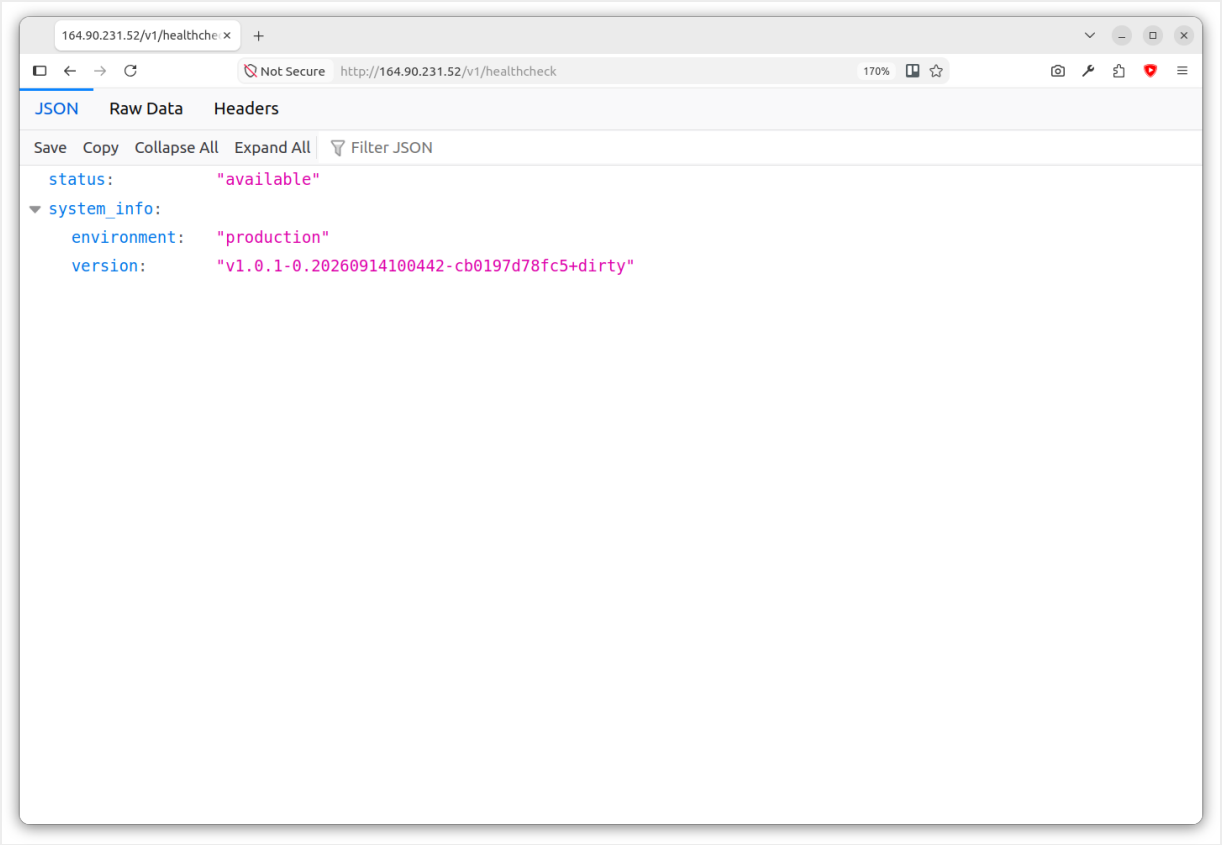
dego_vila@yahoo.com 30 Oct 2026

```
$ make production/deploy/api
rsync -P ./bin/linux_amd64/api greenlight@'164.90.231.52':~
api
     11,559,175 100%    1.20GB/s    0:00:00 (xfr#1, to-chk=0/1)
rsync -rP --delete ./migrations greenlight@'164.90.231.52':~
sending incremental file list
migrations/000001_create_movies_table.down.sql
             28 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=11/13)
migrations/000001_create_movies_table.up.sql
            310 100%  302.73kB/s    0:00:00 (xfr#2, to-chk=10/13)
migrations/000002_add_movies_check_constraints.down.sql
            198 100%  193.36kB/s    0:00:00 (xfr#3, to-chk=9/13)
migrations/000002_add_movies_check_constraints.up.sql
            289 100%  282.23kB/s    0:00:00 (xfr#4, to-chk=8/13)
migrations/000003_add_movies_indexes.down.sql
             78 100%   76.17kB/s    0:00:00 (xfr#5, to-chk=7/13)
migrations/000003_add_movies_indexes.up.sql
            170 100%  166.02kB/s    0:00:00 (xfr#6, to-chk=6/13)
migrations/000004_create_users_table.down.sql
             27 100%   26.37kB/s    0:00:00 (xfr#7, to-chk=5/13)
migrations/000004_create_users_table.up.sql
            320 100%  312.50kB/s    0:00:00 (xfr#8, to-chk=4/13)
migrations/000005_create_tokens_table.down.sql
             28 100%   27.34kB/s    0:00:00 (xfr#9, to-chk=3/13)
migrations/000005_create_tokens_table.up.sql
            203 100%  198.24kB/s    0:00:00 (xfr#10, to-chk=2/13)
migrations/000006_add_permissions.down.sql
             73 100%   71.29kB/s    0:00:00 (xfr#11, to-chk=1/13)
migrations/000006_add_permissions.up.sql
            478 100%  466.80kB/s    0:00:00 (xfr#12, to-chk=0/13)
rsync -P ./remote/production/api.service greenlight@'164.90.231.52':~
api.service
          1,266 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=0/1)
rsync -P ./remote/production/Caddyfile greenlight@'164.90.231.52':~
Caddyfile
             55 100%    0.00kB/s    0:00:00 (xfr#1, to-chk=0/1)
ssh -t greenlight@'164.90.231.52' '\
        migrate -path ~/migrations -database $GREENLIGHT_DB_DSN up \
        && sudo mv ~/api.service /etc/systemd/system/ \
        && sudo systemctl enable api \
        && sudo systemctl restart api \
        && sudo mv ~/Caddyfile /etc/caddy/ \
        && sudo systemctl reload caddy \
'
no change
[sudo] password for greenlight:
Connection to 164.90.231.52 closed.
```

You should see that the Caddyfile is copied across and the reload executes cleanly without any errors.

At this point you can visit `http://<your_droplet_ip>/v1/healthcheck` in a web browser, and you should find that the request is successfully forwarded on from Caddy to our API. Like so:

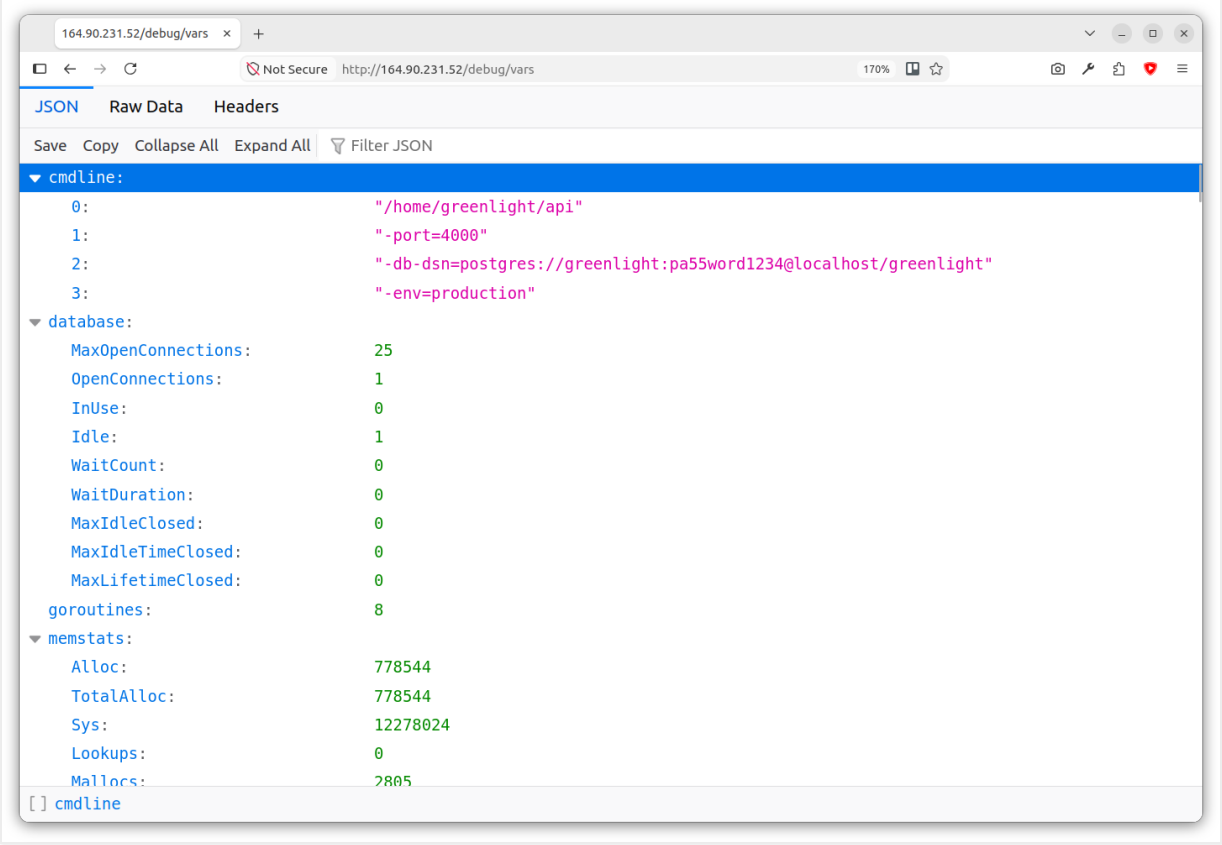
dego_vila@yahoo.com 30 Oct 2026



Blocking access to application metrics

While we're in the browser, let's navigate to the `GET /debug/vars` endpoint which displays our application metrics. You should see a response similar to this:

dego_vila@yahoo.com 30 Oct 2026



As we mentioned earlier, it's really not a good idea for this sensitive information to be publicly accessible.

Fortunately, it's very easy to block access to this by adding a new `respond` directive to our Caddyfile like so:

File: remote/production/Caddyfile

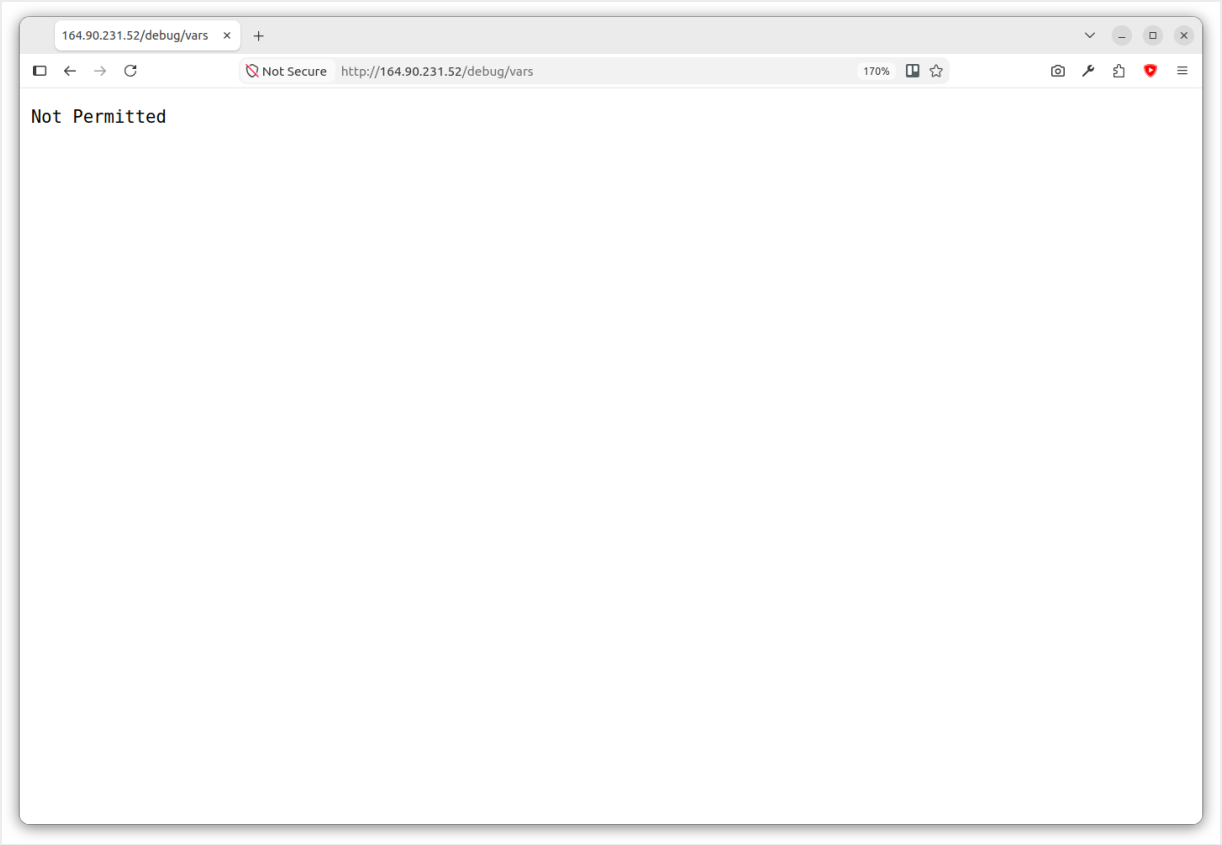
```
http://164.90.231.52 {
  respond /debug/* "Not Permitted" 403
  reverse_proxy localhost:4000
}
```

With this new directive we're instructing Caddy to send a `403 Forbidden` response for all requests which have a URL path beginning `/debug/`.

Go ahead and deploy this change to production again, and when you refresh the page in your web browser you should find that it is now blocked.

```
$ make production/deploy/api
```

dego_vila@yahoo.com 30 Oct 2026



Although the metrics are no longer publicly accessible, you can still access them by connecting to your droplet via SSH and making a request to `http://localhost:4000/debug/vars`.

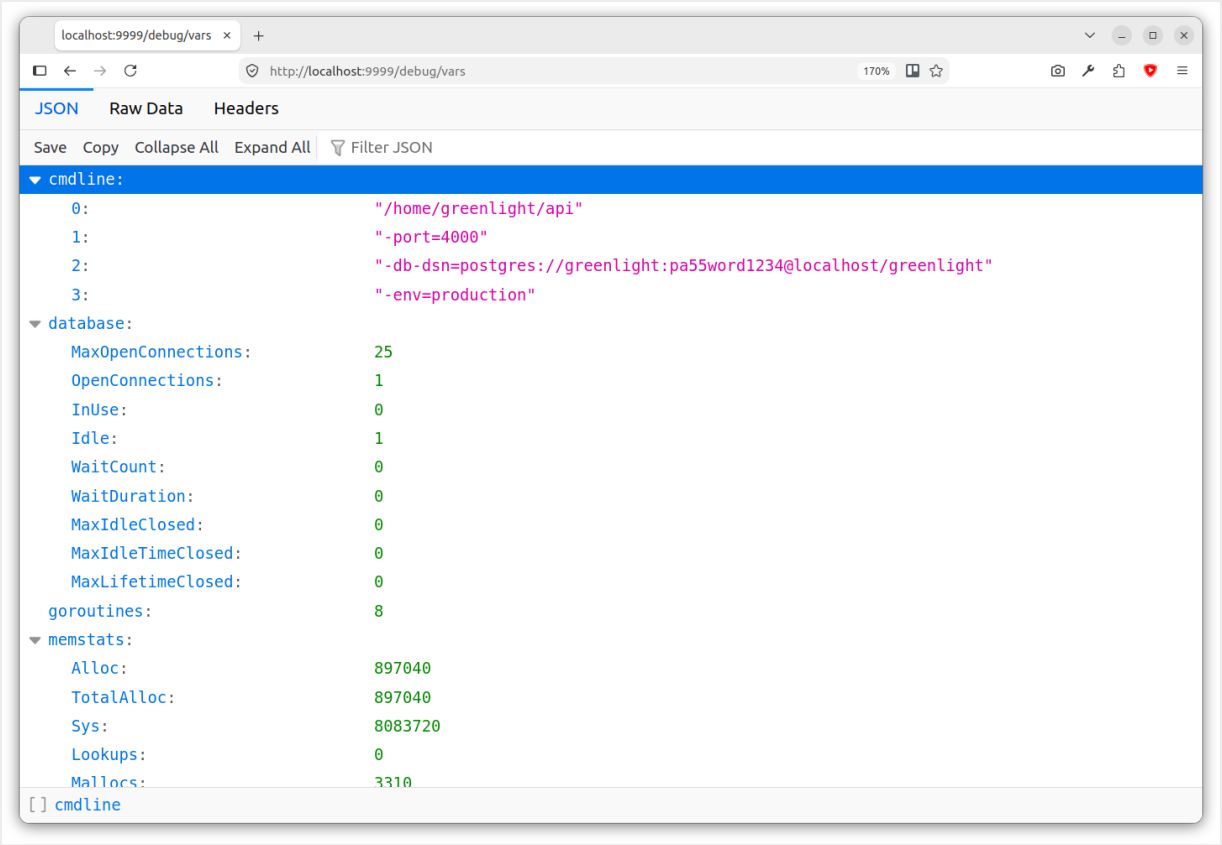
```
$ make production/connect
greenlight@greenlight-production:~$ curl http://localhost:4000/debug/vars
{
"cmdline": ...,
"database": ...,
"goroutines": 8,
"memstats": ...,
"timestamp": 1789635974,
"total_processing_time_µs": 4044,
"total_requests_received": 5,
"total_responses_sent": 4,
"total_responses_sent_by_status": {"200": 3, "404": 1},
"version": "v1.0.1-0.20260914100442-cb0197d78fc5+dirty"
}
```

Or alternatively, you can open an SSH tunnel to the droplet and view them using a web browser on your *local machine*. For example, you could open an SSH tunnel between port `4000` on the droplet and port `9999` on your local machine by running the following command (make sure to replace *both* IP addresses with your own droplet IP):

```
$ ssh -L :9999:164.90.231.52:4000 greenlight@164.90.231.52
```

dego_vila@yahoo.com 30 Oct 2026

While that tunnel is active, you should be able to visit `http://localhost:9999/debug/vars` in your web browser and see your application metrics, like so:



Using a domain name

For the next step of our deployment we're going to configure Caddy so that we can access our droplet via a domain name, instead of needing to use the IP address.

If you want to follow along with this step you'll need access to a domain name and the ability to update the DNS records for that domain name. If you don't have a domain name already available that you can use, then you can get one for free via the EU.org service.

I'm going to use the domain `greenlight.alexedwards.net` in the sample code here, but you should swap this out for your own domain if you're following along.

The first thing you'll need to do is configure the DNS records for your domain name so that they contain an `A` record pointing to the IP address for your droplet. So in my case the DNS record would look like this:

```
A     greenlight.alexedwards.net     164.90.231.52
```

dego_vila@yahoo.com 30 Oct 2026

> **Note:** If you're not sure how to alter your DNS records, your domain name registrar should provide guidance and documentation.

Once you've got the DNS record in place, the next task is to update the Caddyfile to use your domain name instead of your droplet's IP address. Go ahead and swap this out like so (remember to replace `greenlight.alexedwards.net` with your own domain name):

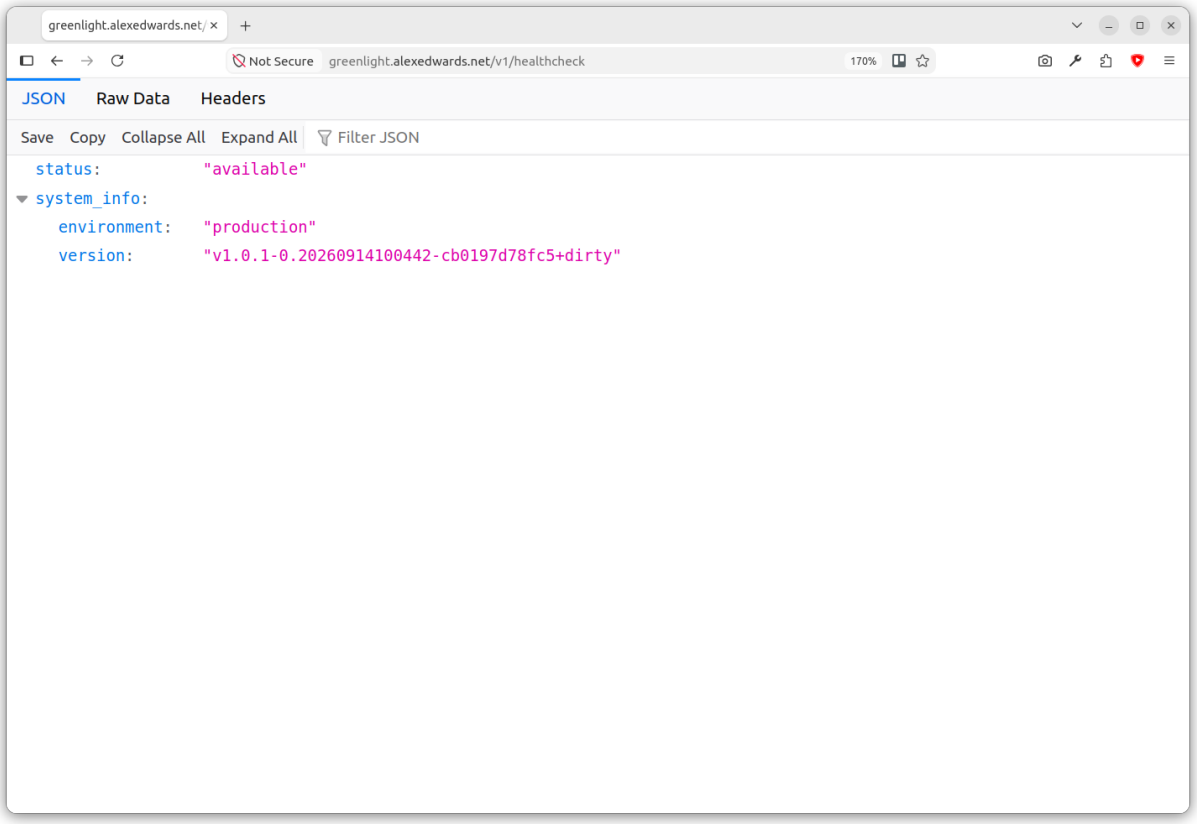
```
File: remote/production/Caddyfile
```

```
http://greenlight.alexedwards.net {
  respond /debug/* "Not Permitted" 403
  reverse_proxy localhost:4000
}
```

And then redeploy the Caddyfile to your droplet again:

```
$ make production/deploy/api
```

Once you've done that, you should now be able to access the API via your domain name by visiting `http://<your_domain_name>/v1/healthcheck` in your browser:



dego_vila@yahoo.com 30 Oct 2026

Enabling HTTPS

Now that we have a domain name set up we can utilize one of Caddy's headline features: *automatic HTTPS*.

Caddy will automatically handle provisioning and renewing TLS certificates for your domain via Let's Encrypt or ZeroSSL (depending on availability), as well as redirecting all HTTP requests to HTTPS. It's simple to set up, very robust, and saves you the overhead of needing to keep track of certificate renewals manually.

To enable this, we just need to update our `Caddyfile` so that it looks like this:

File: remote/production/Caddyfile

```
# Set the email address that should be used to contact you if there is a problem with
# your TLS certificates.
{
  email you@example.com
}

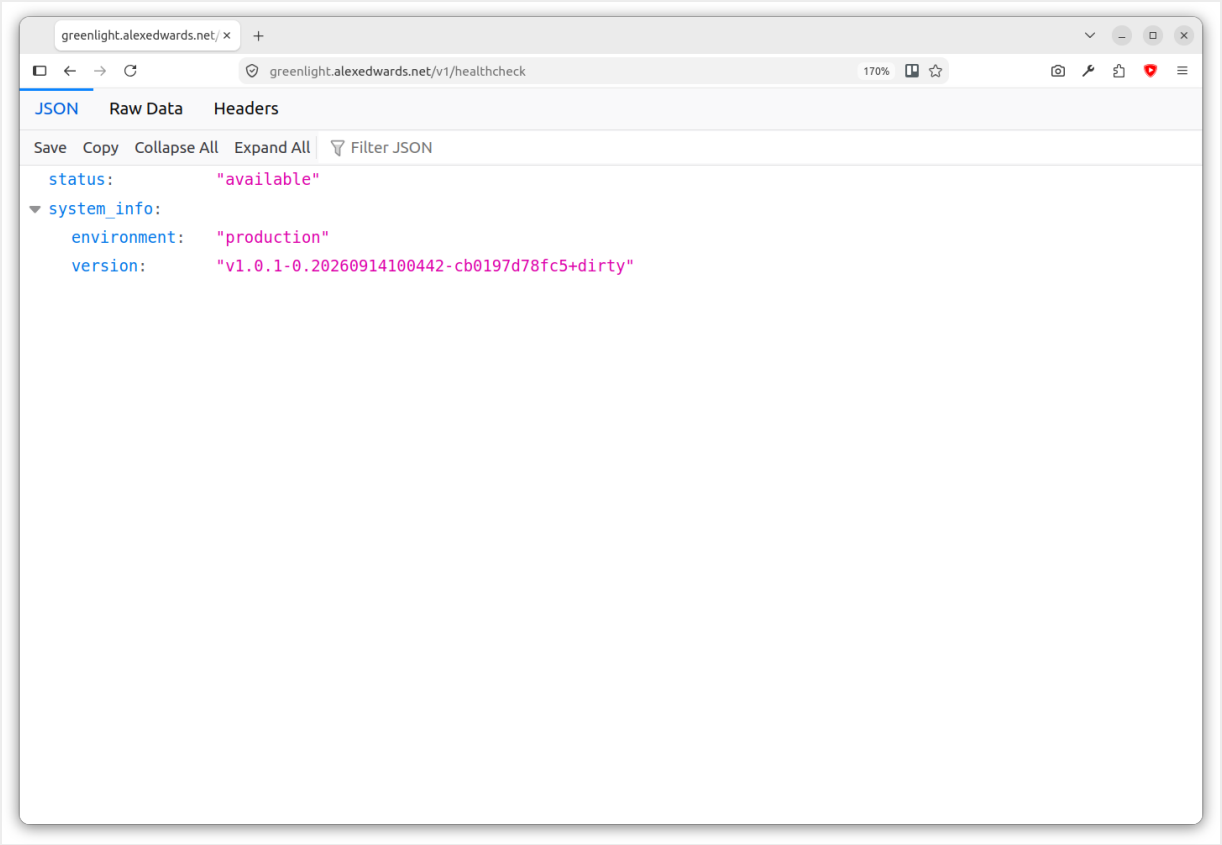
# Remove the http:// prefix from your site address.
greenlight.alexedwards.net {
  respond /debug/* "Not Permitted" 403
  reverse_proxy localhost:4000
}
```

For the final time, deploy this Caddyfile update to your droplet…

```
$ make production/deploy/api
```

And then when you refresh the page in your web browser, you should find that it is automatically redirected to an HTTPS version of the page.

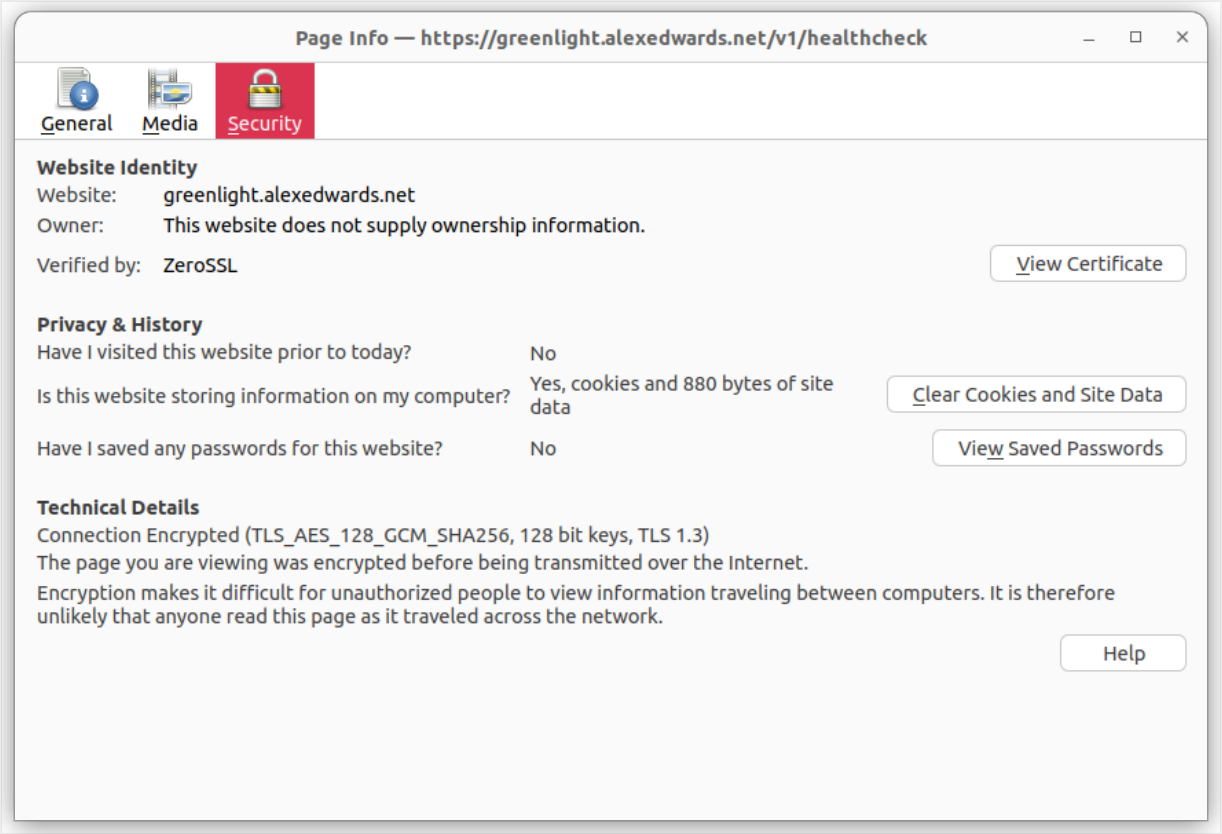
dego_vila@yahoo.com 30 Oct 2026



> **Important:** If you get an error when you refresh the page, wait 30 seconds and then try again. Sometimes it can take Caddy a moment to complete the certificate setup for the very first time.

If you're using Firefox, you can also look at the page information in your browser by pressing `Ctrl+i`. It should look similar to this:

dego_vila@yahoo.com 30 Oct 2026



We can see from this that the connection has been successfully encrypted using TLS 1.3 and the `TLS_AES_128_GCM_SHA256` cipher suite.

Lastly, if you want, you can use `curl` to try making an HTTP request to the application. You should see that this issues a `308 Permanent Redirect` to the HTTPS version of the application, like so:

```
$ curl -i http://greenlight.alexedwards.net
HTTP/1.1 308 Permanent Redirect
Connection: close
Location: https://greenlight.alexedwards.net/
Server: Caddy
Date: Thu, 17 Sep 2026 10:24:28 GMT
Content-Length: 0
```

Additional information

Scaling the infrastructure

Before launching a new service, it's often useful to do a thought experiment and ask yourself: *What happens as traffic to the service increases? How would I manage it?*

dego_vila@yahoo.com 30 Oct 2026

For this project, there's a clear path that we could follow to scale up and adapt the infrastructure to accommodate growth. Very roughly, the path looks something like this:

- Use single low-powered droplet running Caddy, PostgreSQL, and the Go application. This is what we currently have.
- ↓ Upgrade the droplet to have more CPU and/or memory as necessary.
- ↓ Move PostgreSQL to a separate droplet, or use a managed database.
- ↓ Upgrade droplets/managed databases to have more CPU and/or memory as necessary.
- If the droplet running the Go application is a bottleneck:
    - ↓ Profile and optimize your Go application code.
    - ↓ Run the Go application on multiple droplets, behind a load balancer.
- If the PostgreSQL database is a bottleneck:
    - ↓ Profile and optimize the database settings and database queries.
    - ↓ If appropriate, cache the results of expensive/frequent database queries.
    - ↓ If appropriate, move some operations to an in-memory database such as Redis.
    - ↓ Start using read-only database replicas for queries where possible.
    - ↓ Start sharding the database.

In other projects, these steps and the order of them will probably be different. And in some cases, it may be sensible to jump straight into a more complex architecture straight away, based on business requirements, traffic estimates and load testing results.

But taking a short moment to do a thought experiment like this is good. If you can see a clear path, then it provides some positive reassurance that growth can be accommodated without you optimizing the infrastructure or application prematurely. Whereas if you *can't* see a clear path to accommodate growth, then it highlights a potential problem and it may be sensible to revisit your infrastructure choices.

dego_vila@yahoo.com 30 Oct 2026

Chapter 21.

Appendices

- Managing password resets
- Creating additional activation tokens
- Authentication with JSON Web Tokens
- JSON encoding nuances
- JSON decoding nuances
- Request content timeouts

dego_vila@yahoo.com 30 Oct 2026

Managing password resets

If the API you're building is for a public audience, it's likely that you'll want to include functionality for a user to reset their password when they forget it.

To support this in your API, you could add the following two endpoints:

Method	URL Pattern	Handler	Action
POST	/v1/tokens/password-reset	createPasswordResetTokenHandler	Generate a new password reset token
PUT	/v1/users/password	updateUserPasswordHandler	Update the password for a specific user

And implement a workflow like this:

1. A client sends a request to the `POST /v1/tokens/password-reset` endpoint containing the email address of the user whose password they want to reset.
2. If a user with that email address exists in the `users` table, *and the user has already confirmed their email address by activating*, then generate a cryptographically-secure high-entropy random token.
3. Store a hash of this token in the `tokens` table, alongside the user ID and a short (30-60 minute) expiry time for the token.
4. Send the original (unhashed) token to the user in an email.
5. If the owner of the email address didn't request a password reset token, they can ignore the email.
6. Otherwise, they can submit the token to the `PUT /v1/users/password` endpoint along with their new password. If the hash of the token exists in the `tokens` table and hasn't expired, then generate a bcrypt hash of the new password and update the user's record.
7. Delete all existing password reset tokens for the user.

In our codebase, you could implement this workflow by creating a new `password-reset` token scope:

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/tokens.go

```
package models

...

const (
    ScopeActivation     = "activation"
    ScopeAuthentication = "authentication"
    ScopePasswordReset  = "password-reset"
)

...
```

And then adding the following two handlers:

File: cmd/api/tokens.go

```
package main

...

// Generate a password reset token and send it to the user's email address.
func (app *application) createPasswordResetTokenHandler(w http.ResponseWriter, r *http.Request) {
    // Parse and validate the user's email address.
    var input struct {
        Email string `json:"email"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    v := validator.New()

    if models.ValidateEmail(v, input.Email); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Try to retrieve the corresponding user record for the email address. If it can't
    // be found, return an error message to the client.
    user, err := app.models.Users.GetByEmail(input.Email)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            v.AddError("email", "no matching email address found")
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
    // Return an error message if the user is not activated.
    if !user.Activated {
        v.AddError("email", "user account must be activated")
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Otherwise, create a new password reset token with a 45-minute expiry time.
    token, err := app.models.Tokens.New(user.ID, 45*time.Minute, models.ScopePasswordReset)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Email the user with their password reset token.
    app.background(func() {
        data := map[string]any{
            "passwordResetToken": token.Plaintext,
        }

        // Since email addresses MAY be case sensitive, notice that we are sending this
        // email using the address stored in our database for the user --- not to the
        // input.Email address provided by the client in this request.
        err := app.mailer.Send(user.Email, "token_password_reset.tmpl", data)
        if err != nil {
            app.logger.Error(err.Error())
        }
    })

    // Send a 202 Accepted response and confirmation message to the client.
    env := envelope{"message": "an email will be sent to you containing password reset instructions"}

    err = app.writeJSON(w, http.StatusAccepted, env, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

File: cmd/api/users.go

```
package main

...

// Verify the password reset token and set a new password for the user.
func (app *application) updateUserPasswordHandler(w http.ResponseWriter, r *http.Request) {
    // Parse and validate the user's new password and password reset token.
    var input struct {
        Password       string `json:"password"`
        TokenPlaintext string `json:"token"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }
```

dego_vila@yahoo.com 30 Oct 2026

```
    v := validator.New()

    models.ValidatePasswordPlaintext(v, input.Password)
    models.ValidateTokenPlaintext(v, input.TokenPlaintext)

    if !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Retrieve the details of the user associated with the password reset token,
    // returning an error message if no matching record was found.
    user, err := app.models.Users.GetForToken(models.ScopePasswordReset, input.TokenPlaintext)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            v.AddError("token", "invalid or expired password reset token")
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Set the new password for the user.
    err = user.Password.Set(input.Password)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Save the updated user record in our database, checking for any edit conflicts as
    // normal.
    user, err = app.models.Users.Update(user)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrEditConflict):
            app.editConflictResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // If everything was successful, then delete all password reset tokens for the user.
    err = app.models.Tokens.DeleteAllForUser(models.ScopePasswordReset, user.ID)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Send the user a confirmation message.
    env := envelope{"message": "your password was successfully reset"}

    err = app.writeJSON(w, http.StatusOK, env, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

You should also create an `internal/mailer/templates/token_password_reset.tmpl` file containing the email templates for the password reset email, similar to this:

dego_vila@yahoo.com 30 Oct 2026

File: internal/mailer/templates/token_password_reset.tmpl

```
{{define "subject"}}Reset your Greenlight password{{end}}

{{define "plainBody"}}
Hi,

Please send a `PUT /v1/users/password` request with the following JSON body to set a new password:

{"password": "your new password", "token": "{{.passwordResetToken}}"}

Please note that this is a one-time use token and it will expire in 45 minutes. If you need
another token please make a `POST /v1/tokens/password-reset` request.

Thanks,

The Greenlight Team
{{end}}

{{define "htmlBody"}}
<!doctype html>
<html>
  <head>
    <meta name="viewport" content="width=device-width" />
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  </head>
  <body>
    <p>Hi,</p>
    <p>Please send a <code>PUT /v1/users/password</code> request with the following JSON body to set a new password:</p>
    <pre><code>
    {"password": "your new password", "token": "{{.passwordResetToken}}"}
    </code></pre>
    <p>Please note that this is a one-time use token and it will expire in 45 minutes.
    If you need another token please make a <code>POST /v1/tokens/password-reset</code> request.</p>
    <p>Thanks,</p>
    <p>The Greenlight Team</p>
  </body>
</html>
{{end}}
```

And add the necessary routes to the `cmd/api/routes.go` file:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requirePermission("movies:read", app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requirePermission("movies:write", app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requirePermission("movies:read", app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requirePermission("movies:write", app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requirePermission("movies:write", app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)
    // Add the PUT /v1/users/password endpoint.
    router.HandlerFunc(http.MethodPut, "/v1/users/password", app.updateUserPasswordHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)
    // Add the POST /v1/tokens/password-reset endpoint.
    router.HandlerFunc(http.MethodPost, "/v1/tokens/password-reset", app.createPasswordResetTokenHandler)

    router.Handler(http.MethodGet, "/debug/vars", expvar.Handler())

    return app.metrics(app.recoverPanic(app.enableCORS(app.rateLimit(app.authenticate(router)))))
}
```

Once those things are all in place, you should be able to get a new password reset token by making a request like this:

```
$ curl -X POST -d '{"email": "alice@example.com"}' localhost:4000/v1/tokens/password-reset
{
    "message": "an email will be sent to you containing password reset instructions"
}
```

And then you can initiate the actual password change by sending a request containing the token received in the email. For example:

```
$ BODY='{"password": "your new password", "token": "Y7QCRZ7FWOWYLXLAOC2VYOLIPY"}'
$ curl -X PUT -d "$BODY" localhost:4000/v1/users/password
{
    "message": "your password was successfully reset"
}
```

dego_vila@yahoo.com 30 Oct 2026

Additional information

Web application workflow

Just like the activation workflow we looked at earlier, if your API is the backend to a website (rather than a completely standalone service) you can tweak this password reset workflow to make it more intuitive for users.

For example, you could ask the user to enter the token into a form on your website along with their new password, and use some JavaScript to submit the form contents to your `PUT /v1/users/password` endpoint. The email to support that workflow could look something like this:

```
Hi,

To reset your password please visit https://example.com/users/password and enter the
following secret code along with your new password:

--------------------------
Y7QCRZ7FWOWYLXLAOC2VYOLIPY
--------------------------

Please note that this code will expire in 45 minutes. If you need another code please
visit https://example.com/tokens/password-reset.

Thanks,

The Greenlight Team
```

Alternatively, if you don't want the user to copy-and-paste a token, you could ask them to click a link containing the token which takes them to a page on your website. Similar to this:

```
Hi,

To reset your password please click the following link:

https://example.com/users/password?token=Y7QCRZ7FWOWYLXLAOC2VYOLIPY

Please note that this link will expire in 45 minutes. If you need another password reset
link please visit https://example.com/tokens/password-reset.

Thanks,

The Greenlight Team
```

This page can then display a form in which the user enters their new password, and some JavaScript on the webpage should then extract the token from the URL and submit it to the `PUT /v1/users/password` endpoint along with the new password. Again, if you go with this second option, you need to take steps to avoid the token being leaked in a referrer header.

dego_vila@yahoo.com 30 Oct 2026

Chapter 21.2.

Creating additional activation tokens

You may also want to add a standalone endpoint to your API for generating activation tokens. This can be useful in scenarios where a user doesn't activate their account in time, or they never receive their welcome email.

In this appendix, we'll quickly run through the code for doing that, and add the following endpoint:

Method	URL Pattern	Handler	Action
POST	/v1/tokens/activation	createActivationTokenHandler	Generate a new activation token

The code for the `createActivationTokenHandler` handler should look like this:

File: cmd/api/tokens.go

```
...

func (app *application) createActivationTokenHandler(w http.ResponseWriter, r *http.Request) {
    // Parse and validate the user's email address.
    var input struct {
        Email string `json:"email"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    v := validator.New()

    if models.ValidateEmail(v, input.Email); !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Try to retrieve the corresponding user record for the email address. If it can't
    // be found, return an error message to the client.
    user, err := app.models.Users.GetByEmail(input.Email)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            v.AddError("email", "no matching email address found")
            app.failedValidationResponse(w, r, v.Errors)
```

dego_vila@yahoo.com 30 Oct 2026

```
            app.failedValidationResponse(w, r, v.Errors)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    // Return an error if the user has already been activated.
    if user.Activated {
        v.AddError("email", "user has already been activated")
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    // Otherwise, create a new activation token.
    token, err := app.models.Tokens.New(user.ID, 3*24*time.Hour, models.ScopeActivation)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Email the user with their additional activation token.
    app.background(func() {
        data := map[string]any{
            "activationToken": token.Plaintext,
        }

        // Since email addresses MAY be case sensitive, notice that we are sending this
        // email using the address stored in our database for the user --- not to the
        // input.Email address provided by the client in this request.
        err := app.mailer.Send(user.Email, "token_activation.tmpl", data)
        if err != nil {
            app.logger.Error(err.Error())
        }
    })

    // Send a 202 Accepted response and confirmation message to the client.
    env := envelope{"message": "an email will be sent to you containing activation instructions"}

    err = app.writeJSON(w, http.StatusAccepted, env, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}
```

You'll also need to create an `internal/mailer/templates/token_activation.tmpl` file containing the necessary email templates, similar to this:

dego_vila@yahoo.com 30 Oct 2026

File: internal/mailer/templates/token_activation.tmpl

```
{{define "subject"}}Activate your Greenlight account{{end}}

{{define "plainBody"}}
Hi,

Please send a `PUT /v1/users/activated` request with the following JSON body to activate your account:

{"token": "{{.activationToken}}"}

Please note that this is a one-time use token and it will expire in 3 days.

Thanks,

The Greenlight Team
{{end}}

{{define "htmlBody"}}
<!doctype html>
<html>
  <head>
    <meta name="viewport" content="width=device-width" />
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  </head>
  <body>
    <p>Hi,</p>
    <p>Please send a <code>PUT /v1/users/activated</code> request with the following JSON body to activate your account:</p>
    <pre><code>
    {"token": "{{.activationToken}}"}
    </code></pre>
    <p>Please note that this is a one-time use token and it will expire in 3 days.</p>
    <p>Thanks,</p>
    <p>The Greenlight Team</p>
  </body>
</html>
{{end}}
```

And update the `cmd/api/routes.go` file to include the new endpoint:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/routes.go

```
package main

...

func (app *application) routes() http.Handler {
    router := httprouter.New()

    router.NotFound = http.HandlerFunc(app.notFoundResponse)
    router.MethodNotAllowed = http.HandlerFunc(app.methodNotAllowedResponse)

    router.HandlerFunc(http.MethodGet, "/v1/healthcheck", app.healthcheckHandler)

    router.HandlerFunc(http.MethodGet, "/v1/movies", app.requirePermission("movies:read", app.listMoviesHandler))
    router.HandlerFunc(http.MethodPost, "/v1/movies", app.requirePermission("movies:write", app.createMovieHandler))
    router.HandlerFunc(http.MethodGet, "/v1/movies/:id", app.requirePermission("movies:read", app.showMovieHandler))
    router.HandlerFunc(http.MethodPatch, "/v1/movies/:id", app.requirePermission("movies:write", app.updateMovieHandler))
    router.HandlerFunc(http.MethodDelete, "/v1/movies/:id", app.requirePermission("movies:write", app.deleteMovieHandler))

    router.HandlerFunc(http.MethodPost, "/v1/users", app.registerUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/activated", app.activateUserHandler)
    router.HandlerFunc(http.MethodPut, "/v1/users/password", app.updateUserPasswordHandler)

    router.HandlerFunc(http.MethodPost, "/v1/tokens/authentication", app.createAuthenticationTokenHandler)
    // Add the POST /v1/tokens/activation endpoint.
    router.HandlerFunc(http.MethodPost, "/v1/tokens/activation", app.createActivationTokenHandler)
    router.HandlerFunc(http.MethodPost, "/v1/tokens/password-reset", app.createPasswordResetTokenHandler)

    router.Handler(http.MethodGet, "/debug/vars", expvar.Handler())

    return app.metrics(app.recoverPanic(app.enableCORS(app.rateLimit(app.authenticate(router)))))
}
```

Now that those things are all in place, a user can request a new activation token by submitting their email address like this:

```
$ curl -X POST -d '{"email": "bob@example.com"}' localhost:4000/v1/tokens/activation
{
    "message": "an email will be sent to you containing activation instructions"
}
```

The token will be sent to them in an email, and they can then submit the token to the `PUT /v1/users/activated` endpoint to activate their account, in exactly the same way as if they received the token in their welcome email.

If you implement an endpoint like this, it's important to note that this would allow users to potentially have multiple valid activation tokens 'on the go' at any one time. That's fine — but you just need to make sure that you delete *all* the activation tokens for a user once they've successfully activated (not just the token that they used).

And again, if your API is the backend for a website, then you can tweak the emails and workflow to make it more intuitive by using the same kind of patterns that we've talked

dego_vila@yahoo.com 30 Oct 2026

about previously.

dego_vila@yahoo.com 30 Oct 2026

Authentication with JSON Web Tokens

In this appendix we're going to switch the authentication process for our API to use JSON Web Tokens (JWTs).

> **Important:** Using JWTs for this particular application doesn't have any benefits over our current 'stateful token' approach. The JWT approach is more complex, ultimately requires the same number of database lookups, and we lose the ability to revoke tokens. For those reasons, it doesn't make much sense to use them here. But I still want to explain the pattern for two reasons:
>
> - JWTs have a lot of mind-share, and as a developer it's likely that you will run into existing codebases that use them for authentication, or that outside influences mean you are forced to use them.
> - It's worth understanding how they work in case you need to implement APIs that require *delegated authentication*, which *is* a scenario that they're useful in.

> **Note:** If you're completely new to JWTs, I recommend reading the Introduction to JSON Web Tokens article as a starting point.

As we briefly explained earlier in the book, JWTs are a type of *stateless token*. They contain a set of *claims* which are signed (using either a symmetric or asymmetric signing algorithm) and then encoded using base64. In our case, we'll use JWTs to carry a *subject* claim containing the ID of an authenticated user.

There are a few different packages available which make working with JWTs relatively simple in Go. In most cases the `pascaldekloe/jwt` package is a good choice — it has a clear and simple API, and is designed to avoid a couple of the major JWT security vulnerabilities by default.

If you want to follow along, please install it like so:

```
$ go get github.com/pascaldekloe/jwt@v1
```

One of the first things that you need to consider when using JWTs is the choice of signing

dego_vila@yahoo.com 30 Oct 2026

algorithm.

If your JWT is going to be consumed by a *different* application to the one that created it, you should normally use an asymmetric-key algorithm like ECDSA or RSA. The 'creating' application uses its private key to sign the JWT, and the 'consuming' application uses the corresponding public key to verify the signature.

Whereas if your JWT is going to be *consumed by the same application that created it*, then the appropriate choice is a (simpler and faster) symmetric-key algorithm like HMAC-SHA256 with a random secret key. This is what we'll use for our API.

So, to get authentication with JWTs working, you'll first need to add a secret key for signing the JWTs to your `.envrc` file. For example:

```
File: .envrc
```

```
export GREENLIGHT_DB_DSN=postgres://greenlight:pa55word@localhost/greenlight
export JWT_SECRET=pei3einoh0Beem6uM6Ungohn2heiv5lah1ael4joopie5Jaigeikoozaoaew2Eh6
```

> **Note:** Your secret key should be a cryptographically secure random string with an underlying entropy of at least 32 bytes (256 bits).

And then you'll need to update your `Makefile` to pass in the secret key as a command-line flag when starting the application, like so:

```
File: Makefile
```

```
...

# ==================================================================================== #
# DEVELOPMENT
# ==================================================================================== #

## run/api: run the cmd/api application
.PHONY: run/api
run/api:
 go run ./cmd/api -db-dsn=${GREENLIGHT_DB_DSN} -jwt-secret=${JWT_SECRET}

...
```

Next you'll need to edit the `cmd/api/main.go` file so that it parses the JWT secret from the command-line flag into the `config` struct:

dego_vila@yahoo.com 30 Oct 2026

File: cmd/api/main.go

```
package main

...

type config struct {
    port int
    env  string
    db   struct {
        dsn          string
        maxOpenConns int
        maxIdleConns int
        maxIdleTime  time.Duration
    }
    limiter struct {
        enabled bool
        rps     float64
        burst   int
    }
    smtp struct {
        host     string
        port     int
        username string
        password string
        sender   string
    }
    cors struct {
        trustedOrigins []string
    }
    jwt struct {
        secret string // Add a new field to store the JWT signing secret.
    }
}

...

func main() {
    var cfg config

    ...

    // Parse the JWT signing secret from the command-line flag. Notice that we leave the
    // default value as the empty string if no flag is provided.
    flag.StringVar(&cfg.jwt.secret, "jwt-secret", "", "JWT secret")

    displayVersion := flag.Bool("version", false, "Display version and exit")

    flag.Parse()

    ...
}

...
```

And once that's done, you can change the `createAuthenticationTokenHandler()` so that it generates and sends a JWT instead of a stateful token. Like so:

File: cmd/api/tokens.go

```
package main
```

dego_vila@yahoo.com 30 Oct 2026

```
import (
    "errors"
    "net/http"
    "strconv" // New import
    "time"

    "greenlight.alexedwards.net/internal/models"
    "greenlight.alexedwards.net/internal/validator"

    "github.com/pascaldekloe/jwt" // New import
)

func (app *application) createAuthenticationTokenHandler(w http.ResponseWriter, r *http.Request) {
    var input struct {
        Email    string `json:"email"`
        Password string `json:"password"`
    }

    err := app.readJSON(w, r, &input)
    if err != nil {
        var badRequestErr *badRequestError
        switch {
        case errors.As(err, &badRequestErr):
            app.badRequestResponse(w, r, err)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    v := validator.New()

    models.ValidateEmail(v, input.Email)
    models.ValidatePasswordPlaintext(v, input.Password)

    if !v.Valid() {
        app.failedValidationResponse(w, r, v.Errors)
        return
    }

    user, err := app.models.Users.GetByEmail(input.Email)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrRecordNotFound):
            app.invalidCredentialsResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    match, err := user.Password.Matches(input.Password)
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    if !match {
        app.invalidCredentialsResponse(w, r)
        return
    }

    // Create a JWT claims struct containing the user ID as the subject, with an issued
    // time of now and validity window of the next 24 hours. We also set the issuer and
    // audience to a unique identifier for our application.
    var claims jwt.Claims
```

dego_vila@yahoo.com 30 Oct 2026

```
    var claims jwt.Claims
    claims.Subject = strconv.FormatInt(user.ID, 10)
    claims.Issued = jwt.NewNumericTime(time.Now())
    claims.NotBefore = jwt.NewNumericTime(time.Now())
    claims.Expires = jwt.NewNumericTime(time.Now().Add(24 * time.Hour))
    claims.Issuer = "greenlight.alexedwards.net"
    claims.Audiences = []string{"greenlight.alexedwards.net"}

    // Sign the JWT claims using the HMAC-SHA256 algorithm and the secret key from the
    // application config. This returns a []byte slice containing the JWT as a base64-
    // encoded string.
    jwtBytes, err := claims.HMACSign(jwt.HS256, []byte(app.config.jwt.secret))
    if err != nil {
        app.serverErrorResponse(w, r, err)
        return
    }

    // Convert the []byte slice to a string and return it in a JSON response.
    err = app.writeJSON(w, http.StatusCreated, envelope{"authentication_token": string(jwtBytes)}, nil)
    if err != nil {
        app.serverErrorResponse(w, r, err)
    }
}

...
```

Go ahead and vendor the new `github.com/pascaldekloe/jwt` dependency and run the API like so:

```
$ make vendor
$ make run/api
```

Then when you make a request to the `POST /v1/tokens/authentication` endpoint with a valid email address and password, you should now get a response containing a JWT like this (line breaks added for readability):

```
$ curl -X POST -d '{"email": "faith@example.com", "password": "pa55word"}' localhost:4000/v1/tokens/authentication
{
    "authentication_token": "eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJncmVlbmxpZ2h0LmFsZXhlZHdhcm
    RzLm5ldCIsInN1YiI6IjciLCJhdWQiOlsiZ3JlZW5saWdodC5hbGV4ZWR3YXJkcy5uZXQiXSwiZXhwIjoxNj
    E4OTM4MjY0LjgxOTIwNSwibmJmIjoxNjE4ODUxODY0LjgxOTIwNSwiaWF0IjoxNjE4ODUxODY0LjgxOTIwND
    h9.zNK1bJPl5rlr_YvjyOXuimJwaC3KgPqmW2M1u5RvgeA"
}
```

If you're curious, you can decode the base64-encoded JWT data. You should see that the content of the claims matches the information that you would expect, similar to this:

```
{"alg":"HS256"}{"iss":"greenlight.alexedwards.net","sub":"7","aud":["greenlight.alexedwards.net"],
"exp":1618938264.819205,"nbf":1618851864.819205,"iat":1618851864.8192048}...
```

Next you'll need to update the `authenticate()` middleware to accept JWTs in an

dego_vila@yahoo.com 30 Oct 2026

`Authorization: Bearer <jwt>` header, verify the JWT, and extract the user ID from the `subject` claim.

When we say “verify the JWT”, what we actually mean is the following four things:

- Check that the signature of the JWT matches the contents of the JWT, given our secret key. This will confirm that the token hasn’t been altered by the client.
- Check that the current time is between the “not before” and “expires” times for the JWT.
- Check that the JWT “issuer” is `"greenlight.alexedwards.net"`.
- Check that `"greenlight.alexedwards.net"` is in the JWT “audiences”.

Go ahead and update the `authenticate()` middleware as follows:

File: cmd/api/middleware.go

```
package main

import (
    "errors"
    "expvar"
    "fmt"
    "net/http"
    "strconv"
    "strings"
    "sync"
    "time"

    "greenlight.alexedwards.net/internal/models"

    "github.com/pascaldekloe/jwt" // New import
    "github.com/tomasen/realip"
    "golang.org/x/time/rate"
)

...

func (app *application) authenticate(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Add("Vary", "Authorization")

        authorizationHeader := r.Header.Get("Authorization")

        if authorizationHeader == "" {
            next.ServeHTTP(w, r)
            return
        }

        headerParts := strings.Split(authorizationHeader, " ")
        if len(headerParts) != 2 || headerParts[0] != "Bearer" {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        token := headerParts[1]

        // Parse the JWT and extract the claims. This will return an error if the JWT
        // content doesn't match the signature (i.e. the token has been tampered with)
        // or the algorithm isn't valid.
```

dego_vila@yahoo.com 30 Oct 2026

```
        // Or the algorithm isn't valid.
        claims, err := jwt.HMACCheck([]byte(token), []byte(app.config.jwt.secret))
        if err != nil {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        // Check if the JWT is still valid at this moment in time.
        if !claims.Valid(time.Now()) {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        // Check that the issuer is our application.
        if claims.Issuer != "greenlight.alexedwards.net" {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        // Check that our application is in the expected audiences for the JWT.
        if !claims.AcceptAudience("greenlight.alexedwards.net") {
            app.invalidAuthenticationTokenResponse(w, r)
            return
        }

        // At this point, we know that the JWT is all OK and we can trust the data in
        // it. We extract the user ID from the claims subject and convert it from a
        // string into an int.
        userID, err := strconv.Atoi(claims.Subject)
        if err != nil {
            app.serverErrorResponse(w, r, err)
            return
        }

        // Lookup the user record from the database.
        user, err := app.models.Users.Get(userID)
        if err != nil {
            switch {
            case errors.Is(err, models.ErrRecordNotFound):
                app.invalidAuthenticationTokenResponse(w, r)
            default:
                app.serverErrorResponse(w, r, err)
            }
            return
        }

        // Add the user record to the request context and continue as normal.
        r = app.contextSetAuthenticatedUser(r, user)

        next.ServeHTTP(w, r)
    })
}
```

Lastly, to get this working, you'll need to create a new `UserModel.Get()` method to retrieve the user details from the database based on their ID.

dego_vila@yahoo.com 30 Oct 2026

File: internal/models/users.go

```
package models

...

func (m UserModel) Get(id int) (User, error) {
    query := `
        SELECT id, created_at, name, email, password_hash, activated, version
        FROM users
        WHERE id = $1`

    var user User

    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel()

    err := m.DB.QueryRowContext(ctx, query, id).Scan(
        &user.ID,
        &user.CreatedAt,
        &user.Name,
        &user.Email,
        &user.Password.hash,
        &user.Activated,
        &user.Version,
    )

    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return User{}, ErrRecordNotFound
        default:
            return User{}, err
        }
    }

    return user, nil
}
```

You should now be able to make a request to one of the protected endpoints, and it will only succeed if your request contains a valid JWT in the `Authorization` header. For example:

dego_vila@yahoo.com 30 Oct 2026

```
$ curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJncmVlbmxpZ..." localhost:4000/v1/movies/2
{
    "movie": {
        "id": 2,
        "title": "Black Panther",
        "year": 2018,
        "runtime": "134m",
        "genres": [
            "sci-fi",
            "action",
            "adventure"
        ],
        "version": 2
    }
}

$ curl -H "Authorization: Bearer INVALID" localhost:4000/v1/movies/2
{
    "error": "invalid or missing authentication token"
}
```

> **Note:** Because JWTs are base64 encoded, you may be able to change the final character in the JWT and it will still be accepted as valid. This is OK, and a detailed explanation of why can be found in this StackOverflow post and this GitHub issue.

If you're planning to put a system into production which uses JWTs, I also recommend spending some time to read and fully understand the following two articles:

- JWT Security Best Practices
- Critical vulnerabilities in JSON Web Token libraries

dego_vila@yahoo.com 30 Oct 2026

JSON encoding nuances

Invalid UTF-8

When encoding a string to JSON, if the string contains any bytes that are not valid UTF-8, then the encoding will fail with a `"jsontext: invalid UTF-8"` error. You can use the `jsontext.AllowInvalidUTF8` option to change this behavior, so that any invalid byte sequences will be replaced with the Unicode replacement character in the JSON output instead of returning an error.

For example:

```
// A string containing an invalid UTF-8 byte (0xFF) between "ab" and "cd".
data := "ab\xffcd"

js, err := json.Marshal(data, jsontext.AllowInvalidUTF8(true))
fmt.Println(string(js), err)
```

Will output:

```
"ab�cd" <nil>
```

Working with pre-computed JSON

If you have a string or `[]byte` slice which contains 'pre-computed' or 'pre-encoded' JSON, by default Go will treat it just like any other string or `[]byte` slice during encoding. That means that a string will be escaped and encoded as a JSON string, and a byte slice will be encoded as a base64 JSON string.

For example, the following struct:

```
data := struct {
    Person string
}{
    Person: `{"name": "Alice", "age": 21}`,
}
```

Will encode to:

dego_vila@yahoo.com 30 Oct 2026

```
{"Person":"{\"name\": \"Alice\", \"age\": 21}"}
```

If you want to interpolate the pre-computed JSON without any changes, you'll need to convert the pre-computed JSON value to a `jsontext.Value` type. Go will then directly interpolate it into the rest of the JSON. For example:

```
data := struct {
    Person jsontext.Value
}{
    Person: jsontext.Value(`{"name": "Alice", "age": 21}`),
}
```

Will encode to the JSON:

```
{"Person":{"name":"Alice","age":21}}
```

> **Important**: Take care when using `jsontext.Value` to make sure that the pre-computed value contains valid JSON, otherwise you'll get an error when trying to encode the parent object. If you need to check this at runtime, you can do so by using the `jsontext.Value.IsValid()` method.

Embedding JSON in HTML

If you are encoding something (especially untrusted/user-provided data) to JSON with the ultimate aim of embedding this JSON inside some HTML, you should use the `jsontext.EscapeForHTML` option to escape any `<`, `>` and `&` characters in JSON strings.

This will prevent an HTML parser from misinterpreting any `<`, `>` and `&` characters in the JSON as real HTML markup — which could otherwise lead to a script injection attack or the HTML becoming mangled.

For example:

```
data := "<script>alert('hi!');</script>"

js, err := json.Marshal(data, jsontext.EscapeForHTML(true))
fmt.Println(string(js), err)
```

Will output a string appropriately escaped for use in HTML, like so:

dego_vila@yahoo.com 30 Oct 2026

```
"\u003cscript\u003ealert('hi!');\u003c/script\u003e" <nil>
```

Embedding JSON in JavaScript

Similarly, if you are encoding something to JSON with the ultimate aim of embedding it inside some JavaScript, you should use the `jsontext.EscapeForJS` option. This will ensure that all line or paragraph separator characters in the JSON are escaped, so that they aren't accidentally interpreted as line terminators in the JavaScript.

The embed directive

When you have nested structs in Go, by default they will be encoded as nested objects in the JSON. For example, the following `Car` struct:

```
type Car struct {
    Make   string `json:"make"`
    Model  string `json:"model"`
    Engine Engine `json:"engine"`
}

type Engine struct {
    Horsepower int `json:"hp"`
    Cylinders  int `json:"cylinders"`
}
```

Will encode to JSON with the following format:

```
{"make":"Ford","model":"Mustang","engine":{"hp":320,"cylinders":8}}
```

If you want, you can use the `embed` struct tag directive on the nested struct to 'promote' its fields into the parent JSON object when encoding it. For example, if you use the `embed` directive on the `Engine` field like this:

```
type Car struct {
    Make   string `json:"make"`
    Model  string `json:"model"`
    Engine Engine `json:",embed"`
}

type Engine struct {
    Horsepower int `json:"hp"`
    Cylinders  int `json:"cylinders"`
}
```

The `Car` struct will encode to JSON with the following format:

dego_vila@yahoo.com 30 Oct 2026

```
{"make":"Ford","model":"Mustang","hp":320,"cylinders":8}
```

It's also possible to use the `embed` directive on *maps* within a struct. When you do this, the key-value pairs in the map will be promoted into the parent JSON object, rather than appearing in a nested JSON object. Please see this Go Playground code for a demonstration.

The json.Marshaler and encoding.TextMarshaler interfaces

Earlier in this book we used the `json.MarshalToFunc()` function to create a custom marshaler which controls how a specific type is encoded to JSON, and then used the `json.WithMarshalers` option to tell Go that we wanted to use that custom marshaler.

However, it's also possible to control how a type is encoded to JSON by making the type satisfy the `json.Marshaler` or `json.MarshalerTo` interfaces. In practical terms, that means defining a `MarshalJSON()` or `MarshalJSONTo()` method on the type which specifies how the type should be encoded.

For example, let's say we have a `Color` struct representing an RGB color value, and we want to encode it as a JSON string containing the corresponding Hex color code — like `"#FF5722"` or `"#B0E0E6"`. To do that, we could give it a `MarshalJSON()` method like so:

```
type Color struct {
    R, G, B uint8
}

func (c Color) MarshalJSON() ([]byte, error) {
    hex := fmt.Sprintf("#%02X%02X%02X", c.R, c.G, c.B)
    return []byte(strconv.Quote(hex)), nil
}
```

Or a `MarshalJSONTo()` method like this:

```
func (c Color) MarshalJSONTo(enc *jsontext.Encoder) error {
    hex := fmt.Sprintf("#%02X%02X%02X", c.R, c.G, c.B)
    return enc.WriteToken(jsontext.String(hex))
}
```

When Go is encoding a particular type to JSON, it will first look to see if a custom marshaler for the type has been specified using the `json.WithMarshalers` option. If it cannot find one, it will then look for either a `MarshalJSONTo()` or `MarshalJSON()` method on the type to see how it should be encoded.

dego_vila@yahoo.com 30 Oct 2026

If there is no `MarshalJSONTo()` or `MarshalJSON()` method, it will then look to see if the type implements the `encoding.TextMarshaler` interface. In other words, it will check to see if the type has a `MarshalText()` method similar to this:

```
func (c Color) MarshalText() ([]byte, error) {
    hex := fmt.Sprintf("#%02X%02X%02X", c.R, c.G, c.B)
    return []byte(hex), nil
}
```

If it does, then Go will call this `MarshalText()` method to determine how to encode it, and output the result *as a JSON string*.

If no `MarshalText()` method exists on the type, it will fall back to using the default JSON representation for the type.

So, there are actually a few ways to control how a specific type is represented in JSON. The priority order goes:

1. A custom marshaler specified using the `json.WithMarshalers` option, falling back to…
2. A `MarshalJSONTo()` method, falling back to…
3. A `MarshalJSON()` method, falling back to…
4. A `MarshalText()` method.

Generally speaking, creating a custom marshaler and specifying it explicitly using the `json.WithMarshalers` option — like we did earlier in the book — is the best and most flexible approach because it gives the calling code control over how specific types should be encoded *for that specific operation*.

However, if you *always* want to represent a specific type in the same way in your JSON, or you want to set a 'fallback' JSON representation for that type if no custom marshaler is specified with the `json.WithMarshalers` option, then it can make sense to implement a `MarshalJSONTo()` or `MarshalJSON()` method on the type.

Integer, time.Time and net.IP values can be used as map keys

It's possible to encode a map which has integer values as the map keys. The integers will be automatically converted to strings in the resulting JSON (because the keys in a JSON object must always be strings). For example, the following map:

dego_vila@yahoo.com 30 Oct 2026

```
data := map[int]string{
    123: "foo",
    456_000: "bar",
}
```

Will encode to the JSON:

```
{"123":"foo","456000":"bar"}
```

In addition, map keys which have the `MarshalJSONTo()`, `MarshalJSON()`, or `MarshalText()` methods that we just talked about are also supported (so long as they return a JSON string representation).

This also means that you can also use `time.Time` and `net.IP` values as map keys out-of-the-box. For example, the following map:

```
t1 := time.Now()
t2 := t1.Add(24 * time.Hour)

data := map[time.Time]string{
    t1: "foo",
    t2: "bar",
}
```

Will encode to JSON which looks similar to this:

```
{"2009-11-10T23:00:00Z":"foo","2009-11-11T23:00:00Z":"bar"}
```

dego_vila@yahoo.com 30 Oct 2026

JSON decoding nuances

Struct tag directives

Using the struct tag `json:"-"` on a struct field will cause it to be *ignored* when decoding JSON, even if the JSON input contains a corresponding key-value pair. For example:

```
js := `{"name": "alice", "age": 21}`

var person struct {
    Name string `json:"name"`
    Age  int    `json:""`
}

err := json.UnmarshalRead(strings.NewReader(js), &person)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("%+v", person)
```

Will print out:

```
{Name:alice Age:0}
```

The `omitzero` and `omitempty` struct tag directives do not have any effect on JSON decoding behavior.

Existing values are retained when decoding into maps and structs

When decoding a JSON object into a struct or map, it will retain any existing values that the struct or map has and merge in the values from the JSON. For example:

dego_vila@yahoo.com 30 Oct 2026

```
// The player struct value already has the Name and Age fields set.
player := struct {
    Name  string `json:"name"`
    Age   int    `json:"age"`
    Score int    `json:"score"`
}{
    Name: "Johanna",
    Age:  37,
}

// And the age and score values from this JSON will be merged into the Age and
// Score fields, leaving Name untouched.
js := `{"age": 38, "score": 350}`

err := json.UnmarshalRead(strings.NewReader(js), &player)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("%+v", player)
```

Will print out:

```
{Name:Johanna Age:38 Score:350}
```

JSON null values

When decoding a JSON `null` value, it will always result in the decode destination being set to the zero-value for the Go type. So, if you are decoding a JSON `null` value into a `string` field, the value of the field after the decode will be `""`.

For example:

```
js := `{"code": null, "quantity": null, "color": null}`

product := struct {
    Code     string  `json:"code"`
    Quantity int     `json:"quantity"`
    Color    *string `json:"color"`
}{
    Code:     "PROD-001",
    Quantity: 7,
    Color:    new("blue"),
}

err := json.UnmarshalRead(strings.NewReader(js), &product)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("%+v", product)
```

Will print out:

dego_vila@yahoo.com 30 Oct 2026

```
{Code: Quantity:0 Color:<nil>}
```

Embedded fallbacks

When decoding into a struct, it's also possible to use the `embed` struct tag directive to create an *embedded fallback* field, which acts like a 'catch-all' for any key-value pairs in the JSON that don't have a matching struct field.

You can use this to turn any `map` field into a fallback destination, for example:

```
js := `{"code": "PROD-001", "quantity": 7, "color": "lilac", "weight_kg": 2.35}`

var product struct {
    Code     string         `json:"code"`
    Quantity int            `json:"quantity"`
    Attrs    map[string]any `json:",embed"`
}

err := json.UnmarshalRead(strings.NewReader(js), &product)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("%+v", product)
```

In this example, the `Attrs` field acts as the embedded fallback, and this code will print out:

```
{Code:PROD-001 Quantity:7 Attrs:map[color:lilac weight_kg:2.35]}
```

For the embedded fallback to work, the map must be of the type `map[~string]T` — that is, the map key must be either `string` or a type *with the underlying type* `string`. And the type `T` must be capable of storing all the values that you expect to receive in the JSON. In the example above, we've made the embedded fallback map a `map[string]any` type, so we can decode both JSON strings (the `color` field) and JSON numbers (the `weight_kg` field) into it successfully.

It's also possible to create an embedded fallback with `jsontext.Value` instead of a map. For example:

dego_vila@yahoo.com 30 Oct 2026

```
js := `{"code": "PROD-001", "quantity": 7, "color": "lilac", "weight_kg": 2.35}`

var product struct {
    Code     string         `json:"code"`
    Quantity int            `json:"quantity"`
    Attrs    jsontext.Value `json:",embed"`
}

err := json.UnmarshalRead(strings.NewReader(js), &product)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("%+v", product)
```

In this case, the `Attrs` field will contain a `jsontext.Value` containing a raw JSON object with all the unmappable key-value pairs, which you can then post-process if necessary. The output from the example above will look like this:

```
{Code:PROD-001 Quantity:7 Attrs:{"color":"lilac","weight_kg":2.35}}
```

Lastly, it's important to be aware that you cannot have multiple embedded fallback fields in the same struct — doing so will result in a `json.SemanticError` (with `ByteOffset = 0`) at runtime. And if you use the `json.RejectUnknownMembers` option in conjunction with an embedded fallback, it will result in the `json.RejectUnknownMembers` option effectively being ignored and the unmappable key-value pairs being decoded into the embedded fallback field.

Partial JSON decoding

If you have a lot of JSON input to process and only need a small part of it, it's often possible to leverage the `jsontext.Value` type to help deal with this. For example:

dego_vila@yahoo.com 30 Oct 2026

```
// Let's say that the only thing we're interested in is processing the "genres" array in
// the following JSON object
js := `{"title": "Top Gun", "genres": ["action", "romance"], "year": 1986}`

// Decode the JSON object to a map[string]jsontext.Value type. The jsontext.Value
// values in the map will retain their original, undecoded, JSON values.
var m map[string]jsontext.Value

err := json.UnmarshalRead(strings.NewReader(js), &m)
if err != nil {
    log.Fatal(err)
}

// We can then access the JSON "genres" value from the map and decode it as normal using
// the json.Unmarshal() function.
var genres []string

err = json.Unmarshal(m["genres"], &genres)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("genres: %v\n", genres)
```

This will print out:

```
genres: [action romance]
```

In this toy example, using `jsontext.Value` doesn't save us much work. But if you need to process a JSON object with *a lot* of key-value pairs and only need a few of them, then taking this approach can save you a lot of typing.

Decoding into the any type

It's possible to decode JSON values into an `any` type. When you do this, the underlying value that the `any` type holds will depend on the type of the JSON value being decoded.

JSON type	⇒	Underlying Go type of any
JSON boolean	⇒	`bool`
JSON string	⇒	`string`
JSON number	⇒	`float64`
JSON array	⇒	`[]any`
JSON object	⇒	`map[string]any`
JSON null	⇒	`nil`

dego_vila@yahoo.com 30 Oct 2026

Decoding into an `any` type can be useful in situations where:

- You don't know in advance exactly what you're decoding.
- You need to decode JSON arrays which contain items with different JSON types.
- The key-value pair in a JSON object doesn't always contain values with the same JSON type.

As an example, consider the following code:

```
// This JSON array contains both JSON string and JSON boolean types.
js := `["foo", true]`

// Decode the JSON into a []any slice.
var s []any

err := json.UnmarshalRead(strings.NewReader(js), &s)
if err != nil {
    log.Fatal(err)
}

// The first value in the slice will have the underlying Go type string, the second will
// have the underlying Go type bool. We can then type assert them and print them out
// the values along with their underlying type.
fmt.Printf("item: 0; type: %T; value: %v\n", s[0], s[0].(string))
fmt.Printf("item: 1; type: %T; value: %v\n", s[1], s[1].(bool))
```

This will print out:

```
item: 0; type: string; value: foo
item: 1; type: bool; value: true
```

Decoding a JSON number into an any type

As shown in the table above, when you decode a JSON number into an `any` type the value will have the underlying type `float64` — even if it is an integer in the original JSON. For example:

```
// This JSON array contains integer JSON numbers.
js := `[123, 9007199254740993]`

// Decode the JSON into a []any slice.
var s []any

err := json.UnmarshalRead(strings.NewReader(js), &s)
if err != nil {
    log.Fatal(err)
}

fmt.Printf("type: %T; value: %.0f\n", s[0], s[0])
fmt.Printf("type: %T; value: %.0f\n", s[1], s[1])
```

dego_vila@yahoo.com 30 Oct 2026

Will print:

```
type: float64; value: 123
type: float64; value: 9007199254740992
```

If you look closely at the code and output here, you'll notice a problem. The second element in the array was the JSON number `9007199254740993`, but after being decoded to a `float64` type in Go it has the incorrect value of `9007199254740992`. This is because `float64` types can only accurately represent certain integer values without a loss of precision. If you like, you can verify this yourself by running this Go playground example.

The docs for `encoding/json/v2` warn about about this potential loss of precision, which can happen with large number values.

> *v2 may still lose precision when unmarshaling into an any interface value, where unmarshal uses a float64 by default to represent a JSON number. To change the default, specify the WithUnmarshalers option with a custom unmarshaler that pre-populates the interface value with a concrete Go type that can preserve precision.*

Because of this, it's generally a good idea to structure your Go code and JSON inputs so that you skirt around the problem completely, and avoid decoding JSON number values into an `any` type.

But if you *can't* skirt around it, then like the docs suggest you can avoid the loss of precision by creating a custom unmarshaler for the `any` type, which decodes any JSON numbers into a `jsontext.Value` that contains the raw JSON value. You can then post-process this raw JSON value appropriately to avoid any precision loss. Building on the example from the official documentation, that could look something like this:

dego_vila@yahoo.com 30 Oct 2026

```
// Create a custom unmarshaler for the any type that decodes and JSON numbers
// as a jsontext.Value.
rawNumberUnmarshaler := json.UnmarshalFromFunc(
    func(dec *jsontext.Decoder, val *any) error {
        if dec.PeekKind() == jsontext.KindNumber {
            // Set val to an empty jsontext.Value.
            *val = jsontext.Value(nil)
        }
        // Return ErrUnsupported to indicate that we want to fall back to default
        // decode behavior, which will now decode into the empty jsontext.Value
        // we just set.
        return errors.ErrUnsupported
    })

js := `[123, 9007199254740993]`

var s []any

// Use the custom unmarshaler.
err := json.UnmarshalRead(strings.NewReader(js), &s, json.WithUnmarshalers(rawNumberUnmarshaler))
if err != nil {
    log.Fatal(err)
}
fmt.Printf("type: %T; value: %s\n", s[0], s[0])
fmt.Printf("type: %T; value: %s\n", s[1], s[1])

// Post process the raw jsontext.Value into an int, which will keep the full
// precision intact.
n, err := strconv.Atoi(s[1].(jsontext.Value).String())
if err != nil {
    log.Fatal(err)
}
fmt.Printf("type: %T; value: %d\n", n, n)
```

This will then output the following results, with the precison of the `9007199254740993` number intact, like so:

```
type: jsontext.Value; value: 123
type: jsontext.Value; value: 9007199254740993
type: int; value: 9007199254740993
```

dego_vila@yahoo.com 30 Oct 2026

Request context timeouts

As an alternative to the pattern that we implemented in chapter 8.3 for managing database timeouts, we could have created a context with a timeout *in our handlers* and then passed the context on to our database model.

Very roughly, a pattern like this:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    ...

    // Create a context.Context with a one-second timeout deadline and which has
    // context.Background() as the 'parent'.
    ctx, cancel := context.WithTimeout(context.Background(), time.Second)
    defer cancel()

    // Pass the context on to the Get() method.
    example, err := app.models.Example.Get(ctx, id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    ...
}
```

```
func (m ExampleModel) Get(ctx context.Context, id int) (*Example, error) {
    query := `SELECT ... FROM examples WHERE id = $1`

    var example Example

    err := m.DB.QueryRowContext(ctx, query, id).Scan(...)
    if err != nil {
        switch {
        case errors.Is(err, sql.ErrNoRows):
            return nil, ErrNotFound
        default:
            return nil, err
        }
    }

    return &example, nil
}
```

The key advantage of this pattern is the extra flexibility; whatever is calling the database model can easily control the timeout duration rather than it always being a fixed duration

dego_vila@yahoo.com 30 Oct 2026

set by the database model.

Using request context as the parent

As another option, you could use this pattern with the *request context* as the context parent (instead of `context.Background()`). Like so:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    ...

    // Use the request context as the parent.
    ctx, cancel := context.WithTimeout(r.Context(), time.Second)
    defer cancel()

    example, err := app.models.Example.Get(ctx, id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    ...
}
```

In many ways, doing this would be considered good practice — allowing context to 'flow' through your application is generally a good idea.

But using the request context as the parent introduces a lot of additional complexity.

The key thing to be aware of is that the *request context will be canceled* if the client closes their HTTP connection. From the `net/http` docs:

> *For incoming server requests, the [request] context is canceled when the client's connection closes, the request is canceled (with HTTP/2), or when the ServeHTTP method returns.*

If we use the request context as the parent, then this cancellation signal will bubble down to our database driver `pq` and the running SQL query will be terminated in exactly the same way that it is when a context timeout is reached. Importantly, it means we would receive the same `"pq: canceling statement due to user request (57014)"` error message that we do when a database query times out.

On one hand this is a positive thing — if there is no client left to return a response to, we may as well cancel the SQL query and free up resources.

dego_vila@yahoo.com 30 Oct 2026

On the other hand, we will receive the same error message in two very different scenarios:

- When a database query takes too long to complete, in which case we would want to log it as an error.
- When a client closes the connection — which can happen for many innocuous reasons, such as a user closing a browser tab or terminating a process. It's not *really* an error from our application's point of view, and we would probably want to either ignore it or perhaps log it as a warning.

Fortunately, it *is* possible to tell these two scenarios apart by calling the `ctx.Err()` method on the context. If the context was canceled (due to a client closing the connection), then `ctx.Err()` will return the error `context.Canceled`. If the timeout was reached, then it will return `context.DeadlineExceeded` instead. If both the deadline is reached and the context is canceled, then `ctx.Err()` will surface whichever happened first.

It's also important to be aware that you may receive a `context.Canceled` error if the client closes the connection while the query is queued by `sql.DB`, and likewise `Scan()` may return a `context.Canceled` error too.

Putting all that together, a sensible way to manage this is to check for the error `"pq: canceling statement due to user request (57014)"` in our database model and wrap it together with the error from `ctx.Err()` before returning. For example:

```
func (m ExampleModel) Get(ctx context.Context, id int) (*Example, error) {
 query := `SELECT ... FROM examples WHERE id = $1`

 var example Example

 err := m.DB.QueryRowContext(ctx, query, id).Scan(...)
 if err != nil {
  var pqErr *pq.Error
  switch {
  case errors.As(err, &pqErr) && pqErr.Code == "57014":
   // Wrap the error with ctx.Err().
   return nil, fmt.Errorf("%w: %w", err, ctx.Err())
  case errors.Is(err, sql.ErrNoRows):
   return nil, ErrNotFound
  default:
   return nil, err
  }
 }

 return &example, nil
}
```

Then in your handlers you can use `errors.Is()` to check if the error returned by the database model is equal to (or wraps) `context.Canceled`, and manage it accordingly.

dego_vila@yahoo.com 30 Oct 2026

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    ...

    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
    defer cancel()

    example, err := app.models.Example.Get(ctx, id)
    if err != nil {
        switch {
        // If the error is equal to or wraps context.Canceled, then it's not
        // really an error and we return without taking any further action.
        case errors.Is(err, context.Canceled):
            return
        case errors.Is(err, models.ErrNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    ...
}
```

As well as this additional error handling, you also need to be aware of the implications when using background goroutines. Bear in mind what I quoted earlier about request context cancellation:

> *For incoming server requests, the [request] context is canceled … when the ServeHTTP method returns.*

This means that if you derive your context from the request context, any SQL queries using the context in a long-running background goroutine will be canceled when the HTTP response is sent for the request! If you don't want that to be the case (and you probably don't), then you would need to create a brand-new context for the background goroutine using `context.Background()` anyway.

So, to summarize, using the request context as the parent context for database timeouts adds quite a lot of behavioral complexity and introduces nuances that you and anyone else working on the codebase needs to be aware of. You have to ask: *is it worth it?*

For most applications, on most endpoints, it's probably not. The exceptions are probably applications which frequently run close to saturation point of their resources, or for specific endpoints which execute slow running or very computationally expensive SQL queries. In those cases, canceling queries aggressively when a client disappears may have a meaningful positive impact and make it worth the trade-off.

The context.WithoutCancel function

dego_vila@yahoo.com 30 Oct 2026

Since Go 1.21, there is another option. You can use the request context as the parent context but wrap it with the `context.WithoutCancel()` function, like so:

```
func (app *application) exampleHandler(w http.ResponseWriter, r *http.Request) {
    ...

    ctx, cancel := context.WithTimeout(context.WithoutCancel(r.Context()), time.Second)
    defer cancel()

    example, err := app.models.Example.Get(ctx, id)
    if err != nil {
        switch {
        case errors.Is(err, models.ErrNotFound):
            app.notFoundResponse(w, r)
        default:
            app.serverErrorResponse(w, r, err)
        }
        return
    }

    ...
}
```

The `context.WithoutCancel()` function returns a copy of the parent context that is *not* canceled when parent context is canceled. But otherwise, it is identical to the parent — and importantly it *retains the same context values as the parent*.

So, when we use this like the example above, the context containing the database timeout *will not be cancelled if the client closes the HTTP connection, or when the ServeHTTP method returns*. That means we avoid all the complexity and extra error handling described above. A `"pq: canceling statement due to user request (57014)"` error will only be returned when there is a timeout.

And the advantage of using this over `context.Background()` as the parent is that the context will retain any values that the request context has.

So, if you want to control the timeout duration from your handlers (instead of having a fixed timeout in the database model), this pattern provides a nice balance of minimizing complexity while still allowing context values to 'flow' through your application.

dego_vila@yahoo.com 30 Oct 2026

Aligning integer types

Let's take a look again at the SQL we used for creating our `movies` table in PostgreSQL, way back earlier in the book.

```
CREATE TABLE IF NOT EXISTS movies (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    created_at timestamp(0) with time zone NOT NULL DEFAULT NOW(),
    title text NOT NULL,
    year integer NOT NULL,
    runtime bigint NOT NULL,
    genres text[] NOT NULL,
    version integer NOT NULL DEFAULT 1
);
```

In this table, the `id` column has the type `bigint`, which has a maximum value of `9223372036854775807`. And the `year` and `version` columns have the type `integer`, which has a maximum value of `2147483647`.

> **Note:** A table summarizing the available PostgreSQL number types can be found here.

In our `Movie` struct, all three of the corresponding fields have the type `int`.

```
type Movie struct {
    ID        int
    CreatedAt time.Time
    Title     string
    Year      int
    Runtime   Runtime
    Genres    []string
    Version   int
}
```

As you may already know, in Go the range of values that the `int` type supports is *architecture dependent*. If your Go code is running on a computer with a 32-bit architecture, then `int` can be used to represent numbers in the range `-2147483648` through `2147483647`. If it is running on a computer with a 64-bit architecture, then `int` can be used to represent a larger range of numbers, from `-9223372036854775808` through `9223372036854775807`.

dego_vila@yahoo.com 30 Oct 2026

> **Note:** Most modern architectures are 64-bit, so in *most* cases `int` will support the larger range. If you're ever unsure about the bit size of your computer, you can check on Unix-based OSes like Linux and macOS by running `$ getconf LONG_BIT` from your terminal.

So there is a potential mismatch here. If our Greenlight application happens to be running on a 32-bit architecture, the `ID` field in our `Movie` struct will not be able to hold the full range of PostgreSQL `bigint` values, and you will potentially get a runtime error that looks like this when trying to read the value from the database:

```
sql: Scan error on column index 1, name "id": converting driver.Value type int64 ("9000000000000000000") to a int:
value out of range
```

Conversely, if our Greenlight application is running on a 64-bit architecture, the `Year` and `Version` fields in our `Movie` struct could in theory hold a number that's too large to fit into the PostgreSQL `integer` type, and you will potentially get a runtime error that looks similar to this when attempting to insert it into the database:

```
pq: value "3000000000" is out of range for type integer
```

Avoiding mismatch errors

To avoid this class of runtime error, there are a couple of options.

The first option is to align the Go types with the PostgreSQL types and use the fixed, not-architecture-dependent, `int64` and `int32` Go types in the `Movie` struct instead. Like so:

```
type Movie struct {
    ID        int64
    CreatedAt time.Time
    Title     string
    Year      int32
    Runtime   Runtime
    Genres    []string
    Version   int32
}
```

So why aren't we doing this?

The main reason is convenience. The `int` type is the standard way to represent integers in Go, and the standard library and third-party packages typically use `int` in function

dego_vila@yahoo.com 30 Oct 2026

arguments and return values rather than `int32` or `int64`. So, by using `int` fields in our `Movie` struct, we make our code more interoperable and reduce the need for type conversions.

The second option is to configure your PostgreSQL table to match the range of values that the Go `int` type stores. So — assuming your Greenlight application is running on a 64-bit architecture — you could give the PostgreSQL integer fields the `bigint` type, like this:

```
CREATE TABLE IF NOT EXISTS movies (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    created_at timestamp(0) with time zone NOT NULL DEFAULT NOW(),
    title text NOT NULL,
    year bigint NOT NULL,
    runtime bigint NOT NULL,
    genres text[] NOT NULL,
    version bigint NOT NULL DEFAULT 1
);
```

This would align things nicely between your Go code and PostgreSQL tables, and prevent any errors arising from type mismatches. And by using `bigint` on all columns this also has the benefit of being more 'future-proof' — you won't have to scramble to make a schema migration if an `integer` column starts to run out of numbers.

So this definitely isn't a *bad* option, but realistically it's a bit wasteful. In practice, making the `year` column a `bigint` is simply unnecessary use of storage space — no way is a valid year value going to exceed `2147483647` during the lifetime of our application. And similarly, the version number is extremely unlikely to ever exceed `2147483647` too. Thinking practically and realistically, giving these columns the type `bigint` feels like overkill, and `integer` seems plenty sufficient.

But then again… storage is cheap, and for most applications the 4 bytes per row difference between `bigint` and `integer` is unlikely to have a meaningful impact.

So, there isn't a clear 'right thing to do' when it comes to aligning — or not aligning — number types. In this appendix, we've been talking about our movies struct and database table, but the points that we're making and the tradeoffs we're discussing apply to any database-backed Go application you're working on.

Generally, you should use `int` in your Go code, and `bigint` for primary key columns in your database. But beyond that, it's a grey area.

If you're worried about type mismatches and not worried about storage space for a specific project, then you might want to make all your PostgreSQL integer fields use `bigint` (or `integer` if your Go application is running on a 32-bit architecture).

dego_vila@yahoo.com 30 Oct 2026

Whereas if you are more worried about storage space and efficiency, then you should use the PostgreSQL number type that you think is most appropriate for the data. Just be aware that a mismatch *could* happen, and try to mitigate this risk by validating inputs at the application level before storing them in your database.

dego_vila@yahoo.com 30 Oct 2026

Chapter 22.

Feedback

Thanks for reading *Let's Go Further*. I hope that you've found it practical and informative, and that you'll enjoy using the knowledge and code patterns in your own projects 😊

I really appreciate any feedback (positive and negative), so that I can continue to improve the book in future updates. I've set up a short online survey at the URL below, and would be grateful if you could take a minute to fill it out:

› https://forms.gle/fTXaziT7GLb3LccW6

Cheers, and happy coding!

Alex